

D B A L I B R A R Y

The Autonomous Database Engineer

40 chapters · Free edition

© 2026 Rajesh Guntupalli

Chapter 1: Introduction to Autonomous Database Engineering

The database has always been the hardest system to automate. Unlike stateless compute, a database carries state — and every autonomous decision made against live data carries the weight of irreversibility. Yet the industry has crossed a threshold. Cloud-native platforms now demonstrate that databases can tune themselves, heal themselves, scale themselves, and in some cases, rewrite their own schemas in response to workload shifts — without a human in the loop. This chapter establishes the engineering philosophy, architectural vocabulary, and operational contracts that separate a self-managing database system from a well-monitored one. Understanding this boundary is the entire premise of the work ahead.

From Reactive DBA to Autonomous System: The Operational Paradigm Shift

The traditional DBA operates on an interrupt-driven model. An alert fires, a human investigates, a decision is made, a change is executed. At best, this loop runs in minutes. At worst, it runs in hours, and the damage is done. The Autonomous Database Engineer (ADE) inverts this model entirely: the system itself holds the logic for observation, diagnosis, and remediation. The human defines policy — not procedure.

This is not simply automation. Automation is running a script on a schedule. Autonomy is a system that evaluates its own state, infers the appropriate action from a rule set or model, and executes that action with awareness of downstream consequences. The distinction matters architecturally because autonomous systems must carry their own decision logic, not just their own execution paths.

Consider the difference between these two responses to a sudden spike in connection count on a SQL Server instance:

Reactive pattern: PagerDuty alerts on-call DBA → DBA SSHes into host → runs `SELECT login_name, COUNT(*) FROM sys.dm_exec_sessions GROUP BY login_name` → manually raises the connection ceiling in the application tier or tunes the server connection budget → writes post-mortem.

Autonomous pattern: Prometheus detects connection saturation against a threshold derived from the 90-day baseline → Alertmanager fires a webhook to the automation layer → PowerShell controller queries `sys.dm_exec_sessions`, classifies the connection source by login name, host, and wait type → if connections come from a known burst workload, the

application connection pool ceiling is raised by 20% and a TTL timer is set to roll it back; if connections come from an unknown source, the automation disables the offending login and pages the team with full context already attached.

The second pattern operates in seconds, arrives at nuance the human would have taken ten minutes to reach, and either resolves the incident or escalates it with maximum context. This is the operational baseline the ADE builds toward.

What makes this shift possible in 2024 is the convergence of three capabilities that previously existed independently: rich, low-latency telemetry from SQL Server DMVs, Query Store, and Extended Events; mature orchestration layers that can act on that telemetry programmatically; and large-scale inference engines — including LLM APIs — that can apply pattern-matching logic that would have required expert human judgment. None of these capabilities alone creates an autonomous system. Together, and with careful engineering, they do.

The Observe–Decide–Act Loop: The Core Architecture of Autonomous Database Systems

Every autonomous system, regardless of platform, can be decomposed into three functional stages. Understanding them as distinct engineering layers — not as a monolithic "automation platform" — is what separates systems that scale from systems that accumulate technical debt.

Observe is the continuous collection of signals from every layer of the database stack: query execution metrics, wait events, replication lag, storage I/O saturation, lock graphs, buffer pool hit ratios, Automatic Tuning action history, index fragmentation levels, and error log parse events. The observe layer must be complete — missing a signal class means the system has a blind spot that will eventually become an incident the system cannot self-diagnose.

Decide is the inference layer. Given the current observed state and historical baseline, what action, if any, should the system take? Decision logic ranges from simple threshold rules (`if replication lag > 30s and primary is alive, restart replication slot`) to ML-derived anomaly detection (`if the query pattern fingerprint has shifted more than  $2\sigma$  from the weekly baseline, flag for index reevaluation`) to LLM-based diagnostic reasoning (`given this wait event profile and this query plan, generate a remediation recommendation`). The decide layer must be auditable — every decision must be logged with the evidence that drove it.

Act is execution. This is where the system touches live infrastructure: executing SQL, reconfiguring connection pools, triggering failovers, scaling read replicas, rotating credentials, applying query hints. The act layer must be gated by impact classification. Not all actions carry equal risk, and the autonomous system must know the difference.

```
graph LR
    A[Observe  
metrics, wait events,  
replication lag, bloat] --> B[Decide  
threshold rules, ML anomaly  
detection, LLM reasoning]
    B --> C[Act  
SQL execution, failover,  
scaling, credential rotation]
    C -. feeds back into. -> A
```

The Observe-Decide-Act loop is the universal architecture underlying every autonomous system described in this book — each subsequent chapter is effectively a deep dive into one domain's version of this loop.

This three-layer model maps directly to the tooling stack for SQL Server environments:

```
-- Observe: pull current wait event distribution from sys.dm_exec_requests
SELECT
    r.wait_type,
    r.wait_time / 1000.0          AS wait_seconds,
    COUNT(*)                     AS session_count,
    AVG(r.wait_time / 1000.0)    AS avg_wait_seconds,
    r.blocking_session_id
FROM sys.dm_exec_requests r
WHERE r.wait_type IS NOT NULL
    AND r.wait_type NOT IN (
        'SLEEP_TASK', 'BROKER_TO_FLUSH', 'BROKER_EVENTHANDLER',
        'CHECKPOINT_QUEUE', 'DBMIRROR_EVENTS_QUEUE', 'SQLTRACE_BUFFER_FLUSH',
        'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE'
    )
GROUP BY r.wait_type, r.wait_time, r.blocking_session_id
ORDER BY session_count DESC;
```

Both queries feed into the same Python observation aggregator, which normalizes their output into a shared schema before passing it to the decision engine. This normalization step — building a unified telemetry contract across heterogeneous platforms — is one of the first real engineering challenges the ADE team faces, and getting it right determines how much of the decision logic can be platform-agnostic.

The act layer must expose a clean interface between "what to do" and "how to do it." The decision engine issues an intent — `SCALE_POOL_UP`, `RESTART_REPLICA`, `CANCEL_QUERY`, `REINDEX_TABLE` — and a platform-specific executor translates that intent into the appropriate SQL, API call, or shell command. This separation of concerns is what makes it possible to extend the system to new platforms without rebuilding the decision logic from scratch.

Taxonomy of Autonomous Actions: Impact Classification and the Autonomy Envelope

Not all autonomous actions are equal. A system that treats canceling a runaway query with the same decision weight as triggering a regional failover will eventually make a catastrophic mistake in a low-signal environment. The ADE defines an **autonomy envelope** — an explicit classification of which actions the system may take unilaterally, which require human confirmation, and which are permanently outside autonomous scope.

A practical three-tier classification for production database environments:

Tier 1 — Fully Autonomous (act immediately, log always):

- → Cancel queries exceeding defined CPU or duration thresholds
- → Adjust application connection pool ceilings within pre-approved bounds
- → Kill idle-in-transaction sessions beyond TTL
- → Rotate transaction log backup destinations on storage saturation
- → Scale read replicas horizontally within pre-approved count limits
- → Refresh materialized views on detected staleness

Tier 2 — Supervised Autonomous (act, but alert and allow human veto window):

- → Trigger automatic failover of an availability group replica
- → Add or drop indexes based on index usage statistics and missing index DMVs
- → Adjust index maintenance and statistics update thresholds database-wide
- → Expand connection pool beyond baseline ceiling
- → Apply query plan hints via `Query Store forced plans` or plan guides

Tier 3 — Human-in-the-Loop (propose only, never execute):

- → Schema migrations of any kind
- → Partitioning strategy changes on tables > 100 GB
- → Regional DR failovers with potential data loss
- → Credential rotation on service accounts with cross-system dependencies
- → Server configuration changes requiring a SQL Server service restart

This taxonomy must be codified in the automation layer as a runtime policy, not as human discipline. If a Tier 3 action is ever triggered programmatically, the system should raise a hard exception, log the evidence, and generate a structured proposal — a complete change request with rationale, predicted impact, and rollback plan — for human review.

```

# Action classification gate – enforced at the automation controller level
import logging
from enum import Enum
from dataclasses import dataclass, field
from typing import Callable, Optional
from datetime import datetime

class AutonomyTier(Enum):
    FULLY_AUTONOMOUS = 1
    SUPERVISED = 2
    HUMAN_IN_LOOP = 3

@dataclass
class AutonomousAction:
    action_type: str
    tier: AutonomyTier
    platform: str          # "sqlserver" (instance or AG name)
    target: str            # host:port or cluster name
    parameters: dict = field(default_factory=dict)
    evidence: dict = field(default_factory=dict)
    executor: Optional[Callable] = None

def dispatch_action(action: AutonomousAction, veto_window_seconds: int = 120):
    timestamp = datetime.utcnow().isoformat()
    logging.info(f"[{timestamp}] ACTION PROPOSED: {action.action_type} "
                f"tier={action.tier.name} target={action.target}")

    if action.tier == AutonomyTier.FULLY_AUTONOMOUS:
        action.executor(action.parameters)
        logging.info(f"[{timestamp}] ACTION EXECUTED: {action.action_type}")

    elif action.tier == AutonomyTier.SUPERVISED:
        notify_on_call(action, veto_window=veto_window_seconds)
        if not veto_received(action, timeout=veto_window_seconds):
            action.executor(action.parameters)
            logging.info(f"[{timestamp}] ACTION EXECUTED (no veto): {action.action_type}")
        else:
            logging.warning(f"[{timestamp}] ACTION VETOED: {action.action_type}")

    elif action.tier == AutonomyTier.HUMAN_IN_LOOP:
        proposal = generate_change_proposal(action)
        submit_to_ticketing_system(proposal)
        logging.info(f"[{timestamp}] PROPOSAL SUBMITTED: {action.action_type} "
                    f"ticket={proposal.ticket_id}")
        raise PermissionError(
            f"Tier 3 action '{action.action_type}' cannot be executed autonomously.
            "
            f"Proposal submitted: {proposal.ticket_id}"
        )

```

The veto window pattern in Tier 2 actions deserves attention. It gives the system operational velocity — it acts unless stopped — while preserving human authority. The on-call engineer receives a structured alert: the proposed action, the evidence that triggered it, the predicted outcome, and a single-click veto URL. This is a fundamentally different model than "alert the engineer and wait for them to act." The initiative has shifted to the system.

Self-Managing Capabilities in the Wild: Platform Precedents and What They Reveal

Before building autonomous systems, the ADE engineer should study the ones that already exist in production at scale, because they reveal the engineering decisions that actually hold up under operational load — not the ones that look elegant in architecture diagrams.

Oracle Autonomous Database is the most complete reference implementation. It uses machine learning to manage indexing, compression, partitioning, and resource allocation. Its UNDO advisor, space management, and automatic indexing are not rule-based systems — they are trained on workload patterns and make probabilistic decisions. The engineering lesson from Oracle ADB is that the most durable autonomous decisions are workload-derived, not threshold-derived. Thresholds require human maintenance; workload models update themselves.

Azure SQL Hyperscale demonstrates horizontal scale autonomy at the storage layer. The separation of compute from storage — with page servers handling storage independently of query compute — allows the system to scale storage capacity without touching the query layer. This architectural pattern (decoupled scaling domains) is directly applicable to SQL Server estates: containerized SQL Server deployments scale compute and storage independently, and Azure SQL Hyperscale applies the same decoupled pattern natively.

CockroachDB demonstrates autonomous rebalancing. As data volume shifts across nodes, CockroachDB's load balancer continuously redistributes range leases without operator intervention. The key insight is that the rebalancing logic runs inside the database engine itself, not as an external controller. For SQL Server estates, the lesson is that data placement logic that lives outside the database engine is inherently slower and more fragile than logic that lives inside it — prefer engine-native partitioning and distribution over external sharding controllers.

Azure SQL Database serverless demonstrates the same lifecycle autonomy in the SQL Server world — compute scales to zero on inactivity and resumes on connection. The autonomous capability here is lifecycle management: the system knows when a database is

idle and acts on that knowledge to release resources. This pattern also fits ephemeral environments (dev, test, feature branches) using containerized SQL Server instances with custom controller logic for provisioning and teardown.

These platforms also share a common failure mode: autonomous systems that act with insufficient observability generate cascading failures that are harder to diagnose than manually-caused ones, because there is no human decision log — only machine actions. Oracle ADB, Azure SQL, and CockroachDB all invest heavily in action audit trails for exactly this reason. The autonomous system must be more transparent about its decisions than a human would be, not less.

The Autonomous Database Stack: Infrastructure, Orchestration, and Intelligence Layers

Building a production-grade autonomous database system means assembling a stack with defined responsibilities at each layer. The mistake most teams make is treating this as a "tooling selection" problem when it is actually a contract-definition problem. Each layer must have a clear interface to the layers above and below it, and the boundaries must be enforced in code.

Infrastructure Layer — the physical or virtual substrate: cloud VMs, Kubernetes nodes, managed services (RDS, Azure SQL, Cloud SQL). This layer handles provisioning, networking, and physical failover. For SQL Server, Azure Elastic Pools and SQL Managed Instance expose ARM templates and Terraform resources, while on-premises estates provision through virtualization templates or Kubernetes operators for containerized SQL Server. The autonomous system does not operate at this layer directly — it communicates its intent to Terraform or Kubernetes, which translates intent into infrastructure state.

Orchestration Layer — the high-availability and lifecycle management layer: SQL Server Always On Availability Groups, Azure SQL auto-failover groups, and Kubernetes operators for containerized SQL Server. This layer owns the cluster topology. It knows which node is primary, which are replicas, what the replication lag is, and how to promote a standby. The autonomous system integrates with this layer via its API — it does not bypass it.

Observability Layer — Prometheus with SQL Server exporters (`sql_exporter`) or Azure Monitor with SQL Insights for cloud estates, Alertmanager for routing, Grafana for visualization, and structured log pipelines (Loki, Elasticsearch, or Azure Monitor). This layer is the sensory system of the autonomous stack. If this layer has gaps, the autonomous system is flying blind.

Intelligence Layer — the decision engine: rule-based policy execution, ML anomaly detection models, and optionally LLM-based reasoning for complex diagnostics. The intelligence layer consumes the observability layer's output and issues intents to the execution layer.

Execution Layer — the layer that translates intents into actions: PowerShell controllers calling SqlClient, Python controllers calling pyodbc, Kubernetes operator custom resources, REST API calls to cloud management planes, Terraform plan/apply pipelines.

```
-- Infrastructure health check: verify AG replica synchronization state
-- Feeds the orchestration layer's failover eligibility decision
SELECT
    ag.name                                AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    ars.synchronization_health_desc,
    ars.connected_state_desc,
    ars.operational_state_desc,
    adbrs.synchronization_state_desc,
    adbrs.log_send_queue_size              AS send_queue_kb,
    adbrs.log_send_rate                    AS send_rate_kb_per_sec,
    adbrs.redo_queue_size                   AS redo_queue_kb,
    adbrs.redo_rate                         AS redo_rate_kb_per_sec
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.dm_hadr_database_replica_states adbrs
    ON ar.replica_id = adbrs.replica_id
ORDER BY adbrs.log_send_queue_size DESC;
```

This query is not a monitoring query in the traditional sense — they are inputs to an orchestration decision. The autonomous system runs them on a defined polling interval (typically 10–30 seconds for HA decisions), evaluates the results against policy thresholds, and decides whether the current cluster topology is acceptable. If `log_send_queue_size` or `redo_queue_size` exceeds a threshold and the replica has been in that state for longer than a defined window, the system evaluates whether to promote an alternative replica, remove the lagging replica from the load balancer, or alert — based on the tier classification of those actions.

The intelligence layer does not need to be LLM-based to be effective. For most operational decisions, rule-based logic with strong observability is faster, more predictable, and easier to audit than probabilistic models. LLM integration becomes valuable at the boundary of the decision envelope — when the system encounters a state it has not seen before and needs

to generate a human-readable diagnosis with remediation options. That boundary exists in every production environment, and designing for it is part of building a complete autonomous system.

Designing for Autonomy from Day One: Principles That Prevent Rearchitecting Later

The most expensive mistake in autonomous systems engineering is bolting autonomy onto a system that was designed for manual operation. The observability hooks don't exist, the execution paths are locked inside manual runbooks, and the blast radius of any automated action is undefined. These systems accumulate automation debt faster than technical debt, and unwinding them is harder because the automation layer and the database layer have grown entangled.

The principles that prevent this:

Design for observability before designing for action. Every component that will eventually be acted on by an autonomous system must first be fully observable. This means Query Store enabled on all databases, Extended Events sessions for wait statistics and blocking, Automatic Tuning telemetry collection, and DMV collection pipelines running before the first production query — not as afterthoughts when the monitoring team asks for them.

Define the blast radius of every action before implementing it. What is the worst possible outcome if this action fires at the wrong time? What is the recovery path? If an autonomous system drops and recreates an index on a 500 GB table at peak load, what happens to replication lag, I/O saturation, and query latency? These questions must have answers before the action is placed in the Tier 1 catalog.

Build idempotent executors. Every action the autonomous system can take must be safe to execute multiple times without producing a worse outcome than executing it once. If the system retries a connection-pool reconfiguration because the first attempt's success signal was lost, the second execution should produce the same state, not stack on top of the first.

Version the decision logic. The rule set or model that drives autonomous decisions is code. It belongs in version control, it requires code review, and changes to it should follow the same deployment pipeline as application code. A decision logic change that incorrectly classifies a benign wait event as a critical failure is a production incident — and like all production incidents, it needs a traceable root cause.

Treat the autonomous system as a principal with its own identity. In SQL Server, the automation layer should connect as a dedicated login (`auto_dba` , `svc_autonomy_engine`) with precisely scoped permissions — no superuser, no `sa` . This makes its actions traceable in audit logs, makes permission failures diagnosable, and prevents the automation layer from acquiring capabilities beyond its defined scope.

```
-- Create a scoped login and user for the autonomous execution layer
CREATE LOGIN [svc_autonomy_engine] WITH PASSWORD = '...';
USE [master];
CREATE USER [svc_autonomy_engine] FOR LOGIN [svc_autonomy_engine];

-- Grant VIEW SERVER STATE for DMV access (observability)
GRANT VIEW SERVER STATE TO [svc_autonomy_engine];

-- Grant ALTER ANY CONNECTION to kill sessions (Tier 1)
GRANT ALTER ANY CONNECTION TO [svc_autonomy_engine];

-- Grant VIEW DATABASE STATE per database for Query Store access
USE [YourDatabase];
CREATE USER [svc_autonomy_engine] FOR LOGIN [svc_autonomy_engine];
GRANT VIEW DATABASE STATE TO [svc_autonomy_engine];

-- Explicitly deny schema-level DDL
DENY ALTER ON SCHEMA::dbo TO [svc_autonomy_engine];
DENY CREATE TABLE TO [svc_autonomy_engine];
DENY DROP TO [svc_autonomy_engine];
```

The principle of least privilege is not a security nicety at the ADE level — it is a containment strategy. When the autonomous system makes a wrong decision, its blast radius is bounded by its permissions. An automation layer with superuser access and a logic bug is a system-wide incident. An automation layer scoped to signal and monitor is a recoverable error.

Key Takeaways

- → The Autonomous Database Engineer builds systems that observe, decide, and act — not systems that execute scripts on schedules. The distinction is architectural, not cosmetic, and it determines whether the system degrades gracefully or catastrophically under unexpected conditions.
- → The Observe–Decide–Act loop is the universal architecture of autonomous database systems. Every tool, every integration, every custom controller maps to one of these three layers, and keeping those layers cleanly separated is what makes the system extensible and auditable.

- → The autonomy envelope — the explicit classification of actions into fully autonomous, supervised autonomous, and human-in-the-loop tiers — is the most important design artifact an ADE team produces. Without it, the system either under-acts (missing incidents it could resolve) or over-acts (making changes that require human judgment it doesn't have).
- → Production autonomous database platforms (Oracle ADB, Azure SQL Hyperscale, CockroachDB) share common architectural patterns: workload-derived decisions over threshold-derived decisions, decoupled scaling domains, and comprehensive action audit trails. These patterns are learnable and reproducible in self-managed SQL Server environments with the right orchestration and observability infrastructure.
- → The foundations of autonomous systems — full-stack observability, blast-radius-aware action classification, idempotent executors, versioned decision logic, and least-privilege service identities — must be designed in from the start. Retrofitting autonomy onto a manually-operated database system is significantly more expensive than building for it from the first deployment, and produces systems that are brittle at exactly the moments when autonomous operation matters most.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 2: The Evolution from AI-Driven DBA to Autonomous DBA

The gap between an AI-powered DBA and an autonomous database engineer is not a matter of better tooling — it is a fundamental shift in system philosophy. AI-powered systems surface insights and recommendations; autonomous systems act on them. This chapter traces that architectural evolution precisely: how observation loops become decision engines, how recommendation pipelines become self-executing control planes, and why the mental model of "the DBA as the decision maker" must be replaced by "the system as the decision maker, with the DBA as the system designer." Understanding this transition is the prerequisite for everything that follows.

From Reactive Alerting to Closed-Loop Autonomy

The traditional DBA workflow is a reactive cycle: a metric crosses a threshold, an alert fires, a human investigates, a human acts. This loop has a latency measured in minutes to hours. For systems handling millions of transactions per second, that latency is architecturally unacceptable.

The first step most teams take is adopting AI-assisted tooling — query advisors, anomaly detectors, tuning recommendations from tools like Azure SQL Advisor, AWS Performance Insights, or home-built Prometheus + Grafana stacks with ML-based anomaly detection. These systems reduce cognitive load, but they do not reduce latency. A recommendation that sits in a dashboard until a DBA reads it at 9 AM is not autonomous — it is a smarter ticket queue.

The closed-loop architecture replaces this with an observe–decide–act (ODA) cycle where all three phases are automated. The observation layer continuously ingests metrics. The decision layer evaluates those metrics against policies. The action layer executes changes without human initiation.

Consider a concrete example: runaway index fragmentation and stale statistics on SQL Server. A traditional AI-powered system detects that `avg_fragmentation_in_percent` on a high-write table's clustered index has crossed 40% and fires an alert. An autonomous system detects the same condition and immediately runs a targeted `ALTER INDEX ... REORGANIZE` or `REBUILD` chosen by page count and fragmentation band, then updates statistics with an appropriate sample rate if plan quality has not recovered within 60 seconds.

```
-- Autonomous index fragmentation pressure detection
-- Equivalent closed-loop signal in SQL Server
SELECT
    DB_NAME(ips.database_id)          AS database_name,
    OBJECT_NAME(ips.object_id)       AS table_name,
    i.name                           AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.index_type_desc,
    ips.alloc_unit_type_desc
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id
    AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 30
    AND ips.page_count > 1000
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

The critical design principle here is that the decision logic — "when fragmentation exceeds X% and hours since last index maintenance exceeds Y, execute Z" — is encoded in the control plane, not in a runbook that a human consults. The ODA loop is tight, consistent, and immune to the delays that plague human-in-the-loop systems.

The Architecture of an Autonomous Decision Engine

An autonomous database control plane is not a single service — it is a layered architecture with clearly separated concerns. Conflating observation, decision, and action into a monolithic script is the most common failure mode teams encounter when they first attempt autonomous systems. When something goes wrong (and it will), the inability to audit which layer made a decision, and why, turns autonomous systems into liability rather than leverage.

The reference architecture has four layers:

Layer 1: Signal Aggregation. Prometheus scrapes `sql_exporter` and `sql_exporter` metrics at 15-second intervals. Metrics are stored in a time-series backend (Prometheus TSDB or Thanos for long-term retention). This layer owns no logic — it is a pure data pipeline.

Layer 2: Policy Evaluation. A policy engine — whether custom Python, OPA (Open Policy Agent), or a purpose-built rule DSL — evaluates current metric state against declared policies. Policies are versioned, tested, and deployed through a CI/CD pipeline just like application code. This is the architectural separation that most teams skip, and it is the one that makes autonomous systems auditable.

Layer 3: Action Executor. The executor receives a directive from the policy engine and carries it out — executing a SQL statement, calling a Kubernetes operator API, modifying a Terraform state, or invoking a cloud provider API. The executor records every action with its full context: what metric triggered it, what policy matched, what the pre- and post-state was.

Layer 4: Feedback and Circuit Breaking. The feedback layer verifies that an action had its intended effect within an expected time window. If an index reorganization triggered by Layer 3 does not reduce fragmentation below the target within 5 minutes, the circuit breaker escalates — either attempting a more aggressive action or, if the escalation path is exhausted, paging a human with the full decision audit trail.

```
graph TD
    A["Layer 1: Signal Aggregation  
Prometheus scrapes sql_exporter / sql_exporter"] --> B["Layer 2: Policy Evaluation  
OPA or rule DSL vs. declared policies"]
    B --> C["Layer 3: Action Executor  
executes SQL, API, Terraform changes"]
    C --> D["Layer 4: Feedback & Circuit Breaking  
verifies intended effect within window"]
    D -->|effect confirmed| A
    D -->|effect not achieved| E["Escalate: more aggressive action"]
    E -->|escalation path exhausted| F["Page human with full decision audit trail"]
```

The four-layer autonomous control plane loop described above — the same separation of concerns Oracle Autonomous Database and CockroachDB implement internally.

This four-layer model maps cleanly onto how Oracle Autonomous Database and CockroachDB's internal control planes operate, though they do not expose these internals directly. When building on SQL Server, you are constructing equivalent machinery from engine-native capabilities — Automatic Tuning, Query Store, Extended Events — and composable automation components.


```

# Reference: simplified policy evaluation loop (SQL Server)
# Production implementations use OPA or a purpose-built rule engine

import pyodbc
import time
import logging
from dataclasses import dataclass
from typing import Callable, List

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

@dataclass
class Policy:
    name: str
    condition: Callable[[dict], bool]
    action: Callable[[dict, object], None]
    cooldown_seconds: int = 300

class AutonomousControlPlane:
    def __init__(self, conn_string: str, policies: List[Policy]):
        self.conn_string = conn_string
        self.policies = policies
        self.last_fired: dict = {}

    def collect_signals(self, conn) -> list:
        cur = conn.cursor()
        cur.execute("""
            SELECT DB_NAME(ips.database_id) AS dbname,
                   OBJECT_NAME(ips.object_id, ips.database_id) AS tablename,
                   ips.index_id,
                   ips.avg_fragmentation_in_percent,
                   ips.page_count
            FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL,
'LIMITED') ips
            WHERE ips.page_count > 1000
                  AND ips.index_id > 0
        """)
        cols = [d[0] for d in cur.description]
        return [dict(zip(cols, row)) for row in cur.fetchall()]

    def evaluate_and_act(self):
        conn = pyodbc.connect(self.conn_string, autocommit=True)
        try:
            signals = self.collect_signals(conn)
            for signal in signals:
                for policy in self.policies:
                    key = f"{policy.name}:{signal['tablename']}"
                    last = self.last_fired.get(key, 0)
                    if time.time() - last < policy.cooldown_seconds:
                        continue
                    if policy.condition(signal):
                        logging.info(f"Policy '{policy.name}' fired on

```

```

{signal['tablename']})
        policy.action(signal, conn)
        self.last_fired[key] = time.time()

    finally:
        conn.close()

# Example policy definition
def fragmentation_pressure_condition(signal: dict) -> bool:
    return (signal["avg_fragmentation_in_percent"] > 30
            and signal["page_count"] > 10000)

def trigger_index_remediation(signal: dict, conn) -> None:
    table = f"[{signal['dbname']}]dbo.[{signal['tablename']}]"
    frag = signal["avg_fragmentation_in_percent"]
    # REORGANIZE for moderate fragmentation, REBUILD for severe
    op = "REBUILD WITH (ONLINE = ON, SORT_IN_TEMPDB = ON)" if frag >= 40 else
    "REORGANIZE"
    cur = conn.cursor()
    cur.execute(f"ALTER INDEX ALL ON {table} {op}")
    logging.info(f"ALTER INDEX {op} executed on {table} (fragmentation was {frag:.1f}%)")

fragmentation_policy = Policy(
    name="fragmentation_pressure",
    condition=fragmentation_pressure_condition,
    action=trigger_index_remediation,
    cooldown_seconds=600
)

```

The cooldown mechanism in the example above is not optional — it is the circuit breaker that prevents action storms, where a runaway policy fires continuously on a table that is not responding to the corrective action. Production implementations add exponential backoff and a maximum escalation depth before human handoff.

How Recommendation Pipelines Become Self-Executing

Most teams that have completed the AI-powered DBA stage have recommendation pipelines already in place. These pipelines collect workload data, pass it through an advisor (whether AWS Database Advisor, Azure SQL Advisor, or a custom model built on `sys.dm_exec_query_stats` and Query Store), and emit recommendations as structured output. The autonomous upgrade to this infrastructure is often smaller than teams expect: the missing piece is an executor that consumes the recommendation output and applies it — with governance gates that encode acceptable risk.

The governance gate is the architectural element that distinguishes production autonomous systems from reckless automation. A governance gate defines, for each class of action, the conditions under which the system may act autonomously versus requiring human approval. A well-designed gate might specify:

- → **Autonomous execution:** Adding a non-clustered index during off-peak hours when estimated row count is below 50 million and the table is not part of an active replication subscription.
- → **Asynchronous approval required:** Dropping an unused index identified by a 30-day zero-usage window, requiring acknowledgment within 24 hours before execution.
- → **Synchronous approval required:** Any DDL change affecting a table with a foreign key relationship to a table owned by a different team.

```
-- Identifying index candidates for autonomous creation
-- Output is consumed by the governance gate for classification
SELECT TOP 25
    DB_NAME(mid.database_id)                AS database_name,
    OBJECT_NAME(mid.object_id, mid.database_id) AS table_name,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.avg_user_impact,
    migs.user_seeks,
    migs.user_scans,
    migs.last_user_seek,
    -- Estimated DDL for autonomous executor
    'CREATE INDEX [IX_AUTO_' + CONVERT(VARCHAR,mid.object_id) + '_' +
    CONVERT(VARCHAR,mid.index_handle) + '] ON ' +
    OBJECT_NAME(mid.object_id, mid.database_id) +
    ' (' + ISNULL(mid.equality_columns,'') +
    CASE WHEN mid.inequality_columns IS NOT NULL
        THEN ',' + mid.inequality_columns ELSE '' END + ')' +
    CASE WHEN mid.included_columns IS NOT NULL
        THEN ' INCLUDE (' + mid.included_columns + ')' ELSE '' END
    AS proposed_ddl
FROM sys.dm_db_missing_index_details mid
JOIN sys.dm_db_missing_index_groups mig
    ON mid.index_handle = mig.index_handle
JOIN sys.dm_db_missing_index_group_stats migs
    ON mig.index_group_handle = migs.group_handle
WHERE migs.avg_user_impact > 40
    AND migs.user_seeks > 1000
ORDER BY migs.avg_user_impact * migs.user_seeks DESC;
```

The proposed DDL column in the SQL Server query is significant — the recommendation pipeline emits not just a suggestion but executable code. The autonomous executor does not interpret natural language; it executes structured, pre-validated DDL that has passed through the governance gate. This is the same pattern used by Azure SQL Automatic Tuning, which has been executing autonomous index creation and deletion in production since 2017.

When integrating LLMs (OpenAI, Claude, or local models via Ollama) into this pipeline, they belong at the recommendation generation stage, not at the execution stage. An LLM can translate a Query Store fingerprint into a human-readable explanation or generate index DDL with proper naming conventions — but execution decisions must flow through deterministic policy engines, not probabilistic models. The risk profile of a model hallucinating an `ALTER TABLE DROP COLUMN` is not acceptable in a production autonomous system.

Autonomous Failover and Topology Management

Failover automation is where the distance between AI-assisted and fully autonomous systems is most consequential. A system that detects a primary failure and pages an engineer is helpful. A system that detects a primary failure, validates the replica is current, promotes the replica, updates DNS, redirects application connections, and notifies the team — all within 30 seconds — is autonomous.

Always On Availability Groups are the reference implementation for SQL Server autonomous failover. The Windows Server Failover Cluster quorum acts as the arbiter: when the primary fails its health checks within the configured thresholds, the cluster promotes a synchronous-commit secondary without human input, preventing split-brain promotion. The entire process is governed by the availability group's failover mode, health-check timeout, session-timeout, and quorum voting configuration.

```

# Evaluate-AGFailoverEligibility.ps1 – autonomous failover readiness reference
# Runs on the control-plane host; output feeds the failover policy gate.
# Production tuning for low-latency promotion decisions.

$Policy = @{
    MaxLogSendQueueKB = 10240      # 10 MB – max send queue before exclusion
    MaxRedoQueueKB     = 10240      # 10 MB – max redo queue before exclusion
    RequireSynchronized = $true     # Only SYNCHRONIZED secondaries are eligible
}

function Get-FailoverCandidates {
    param([string]$PrimaryInstance, [string]$AgName)

    $q = @"
SELECT ar.replica_server_name,
       ars.synchronization_health_desc,
       ars.connected_state_desc,
       drs.synchronization_state_desc,
       ISNULL(drs.log_send_queue_size, 0) AS send_queue_kb,
       ISNULL(drs.redo_queue_size, 0)    AS redo_queue_kb,
       ISNULL(drs.redo_rate, 0)          AS redo_rate_kbps
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
     ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
     ON ar.replica_id = ars.replica_id
LEFT JOIN sys.dm_hadr_database_replica_states drs
     ON ar.replica_id = drs.replica_id
WHERE ag.name = '$AgName'
      AND ars.role_desc = 'SECONDARY';
"@

    $rows = Invoke-Sqlcmd -ServerInstance $PrimaryInstance -Query $q -Database
master

    $eligible = foreach ($r in $rows) {
        $ok = $r.connected_state_desc -eq 'CONNECTED' `
            -and $r.send_queue_kb -lt $Policy.MaxLogSendQueueKB `
            -and $r.redo_queue_kb -lt $Policy.MaxRedoQueueKB
        if ($Policy.RequireSynchronized) {
            $ok = $ok -and $r.synchronization_state_desc -eq 'SYNCHRONIZED'
        }
        if ($ok) { [pscustomobject]@{
            Replica = $r.replica_server_name
            RedoQueueKB = $r.redo_queue_kb
            RedoRateKBps = $r.redo_rate_kbps
        }}
    }

    # Promote the eligible replica with the smallest redo queue first
    $eligible | Sort-Object RedoQueueKB
}

$candidates = Get-FailoverCandidates -PrimaryInstance 'SQL-PROD-01' -AgName

```

```
'AG_Orders'  
$candidates | Format-Table -AutoSize
```

Azure SQL auto-failover groups extend the same pattern to the cloud: failover policy, grace periods, and read-write listener updates are all managed by the platform, giving the autonomous control plane a single policy-driven API for topology decisions across regions.

The key production lesson is that autonomous failover decisions require pre-validated replica readiness checks that go beyond simple lag measurement. A replica whose `log_send_queue_size` and `redo_queue_size` are within thresholds but is experiencing high I/O queue depth due to a concurrent checkpoint is not a safe failover target. Production autonomous systems add secondary health checks — verifying redo queue drain rate, CPU saturation, and connection headroom — before any promotion decision executes. These checks are encoded in the cluster health probes and in SQL Server's availability group health-check logic.

LLMs as Decision Advisors in the Autonomous Control Plane

The integration of large language models into an autonomous control plane is not science fiction, but it requires careful architectural positioning. LLMs are probabilistic pattern matchers — they excel at generating context-aware explanations, synthesizing multiple signals into a coherent narrative, and suggesting actions from a knowledge corpus that no rule engine could encode. They are not reliable executors. This distinction determines where they sit in the architecture.

The practical pattern that has emerged in production autonomous systems is the **advisory layer**: an LLM sits between the policy evaluation layer and the human notification layer, and its job is to annotate autonomous decisions and escalations with context that makes them immediately actionable by the on-call engineer. When the circuit breaker fires and a human must be paged, the page includes not just raw metric data but a synthesized explanation — "Autonomous index maintenance has run 4 times on `orders` in the last 30 minutes without reducing fragmentation below 35%. The table is receiving approximately 12,000 inserts/second and the maintenance window is constrained to 15 minutes. Recommended escalation: switch from REORGANIZE to REBUILD with `SORT_IN_TEMPDB` and raise MAXDOP for the maintenance job from 2 to 4."

```

# LLM advisory layer integration
# Annotates escalation events before human notification

import openai
import json
from typing import Optional

def generate_escalation_advisory(
    event: dict,
    metric_history: list,
    actions_taken: list,
    model: str = "gpt-4o"
) -> Optional[str]:
    """
    Generates a human-readable escalation advisory from structured event data.
    Called only when the circuit breaker escalates to human notification.
    The LLM never initiates or approves actions – it only annotates them.
    """
    client = openai.OpenAI()

    context_payload = {
        "event_type": event.get("type"),
        "affected_object": event.get("object"),
        "current_metrics": event.get("metrics"),
        "metric_trend_last_30min": metric_history[-6:] if metric_history else [],
        "autonomous_actions_attempted": actions_taken,
        "autonomous_actions_failed_to_resolve": True
    }

    prompt = f"""You are a senior SQL Server DBA assistant embedded in an autonomous
database
control plane. The system has exhausted its automated remediation steps and is
escalating
to a human engineer.

Analyze the following event context and provide:
1. A one-sentence summary of what is happening
2. Why the autonomous remediation did not resolve it
3. The two most likely root causes
4. The single most important action the on-call engineer should take first

Be specific. Use metric values. Do not suggest basic steps the autonomous system has
already taken.
Limit your response to 200 words.

Context:
{json.dumps(context_payload, indent=2, default=str)}"""

    try:
        response = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.1, # Low temperature for consistent, factual output

```

```

        max_tokens=300
    )
    return response.choices[0].message.content
except Exception as exc:
    # Advisory generation failure must never block the escalation itself
    return f"[Advisory generation failed: {exc}. Review raw metrics attached.]"

```

The `temperature=0.1` setting is deliberate. Autonomous system advisories are not creative writing — they need to be consistent, factual, and tightly coupled to the input data. Higher temperature values introduce variability that erodes engineer trust in the advisory output over time. Trust erosion is the single largest threat to autonomous system adoption in production organizations: once engineers begin second-guessing the advisory layer, they begin second-guessing the entire control plane.

The broader implication of this pattern is that LLMs in autonomous database systems are most valuable as **interface translators** — converting machine-readable decision logs into human-readable context during the rare moments when human judgment is actually required. They should not be embedded in the hot path of action decisions, they should not have write access to database systems, and their outputs should never be passed directly to an executor without a deterministic validation layer in between.

Key Takeaways

- → The transition from AI-powered to autonomous database engineering is a shift from recommendation latency to action latency — the system does not wait for human acknowledgment to close the observe–decide–act loop.
- → A production autonomous control plane requires four separated layers: signal aggregation, policy evaluation, action execution, and feedback with circuit breaking; conflating these layers is the primary cause of runaway automation incidents.
- → Governance gates are the architectural mechanism that encodes acceptable risk into the autonomous system — every action class must have a declared autonomy level before the system goes to production, and that policy must be versioned and tested like application code.
- → Autonomous failover via Always On Availability Groups requires secondary health validation beyond lag measurement; replica readiness checks must include I/O queue depth, redo drain rate, and connection headroom to avoid promoting an unhealthy standby.
- → LLMs belong in the advisory layer of the autonomous control plane — annotating escalations with synthesized context for human engineers — not in the execution path, where probabilistic output creates unacceptable risk in high-stakes database operations.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 3: Core Principles of Autonomous Database Systems

Autonomous database systems don't emerge from bolting automation scripts onto a traditional DBA workflow. They emerge from a deliberate architectural philosophy — a set of first principles that govern how a system observes its own state, reasons about what that state means, decides what action to take, and executes that action without waiting for a human to approve it. This chapter establishes those principles with precision, because every autonomous pattern in the chapters that follow is built on top of them. Engineers who skip this foundation tend to build brittle automation: systems that act fast in known scenarios and fail catastrophically in unknown ones. The ADE builds systems that are correct, bounded, and recoverable across the full spectrum of conditions.

The Observe-Decide-Act Loop: The Engine of Autonomous Behavior

Every self-managing database system, regardless of how sophisticated its ML models or how rich its tooling stack, executes some variant of a three-phase loop: it observes its environment, decides what response is appropriate, and acts on that decision. The quality of an autonomous system is measured by how accurately it observes, how soundly it decides, and how safely it acts. Weakness in any phase propagates failure through the entire loop.

Observation is not simply collecting metrics. A SQL Server instance emitting 5,000 metrics per minute to Prometheus is not an observing system — it's a recording system. Observation requires context enrichment: knowing that a spike in `sys.dm_exec_requests` blocking waits coincides with a batch job that runs at 03:00 every Tuesday, and that this combination historically resolves within 90 seconds without intervention, is observational intelligence. Raw telemetry becomes observation when it is correlated, timestamped relative to known change events, and compared against historical baselines rather than static thresholds.

The following query illustrates a structured observation layer — not a simple metric read, but a contextual snapshot that combines wait events, query age, and lock relationships in a single pull:

```
-- Contextual observation: long-running queries with blocking chain
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000.0 AS wait_seconds,
    r.total_elapsed_time / 1000.0 AS elapsed_seconds,
    r.logical_reads,
    r.cpu_time,
    s.login_name,
    s.program_name,
    s.host_name,
    t.text AS query_text,
    qp.query_plan
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
CROSS APPLY sys.dm_exec_query_plan(r.plan_handle) qp
WHERE r.total_elapsed_time > 30000 -- 30 seconds
    AND r.session_id != @@SPID
ORDER BY r.total_elapsed_time DESC;
```

Decision is where most teams underinvest. They build clear observation pipelines and solid execution mechanisms, but the decision layer is a flat if-else tree: "if CPU > 80%, send alert." That is not a decision — it is a threshold check. A genuine decision layer considers multiple signals simultaneously, weights them against known system state (maintenance window? replication lag? recent schema change?), and selects from a bounded set of actions ranked by risk. The decision layer in a mature autonomous system resembles a rule engine or a policy evaluator more than a monitoring alert.

Action must always be bounded and auditable. Autonomous action without bounded scope is the architectural equivalent of `DROP TABLE` in a trigger — technically possible, catastrophically dangerous. Every action the system takes should be logged with the full decision context that produced it, reversible where possible, and subject to a circuit breaker that halts automated action when a defined safety condition is violated.

Self-Configuration: Letting the System Know Itself

Self-configuration is the capacity of a database system to adjust its own operational parameters in response to workload change — without a DBA manually running `sp_configure` or scheduling maintenance by hand. This is not about setting `max server memory` once at startup and forgetting it. It is about continuous, workload-aware parameter management that understands the current query mix, memory pressure, and concurrency level, and adjusts accordingly.

SQL Server's `sp_configure` model distinguishes dynamic options from those requiring a service restart. An autonomous system accounts for this by maintaining a configuration change queue and applying changes at the appropriate time — immediately for dynamic settings via `RECONFIGURE`, and during a planned maintenance window for options requiring a restart. The system must know which bucket each option falls into; `sys.configurations` exposes `is_dynamic` and `is_advanced` for exactly this classification.

A well-structured self-configuration engine operates on a scoring model. Parameters like `work_mem` can be tuned dynamically at the session level for specific query classes without any system-wide impact:

```
-- Self-configuration: adjust max degree of parallelism based on active workload
DECLARE @active_parallel_queries INT;
DECLARE @current_maxdop INT;

SELECT @active_parallel_queries = COUNT(*)
FROM sys.dm_exec_requests
WHERE dop > 1 AND status = 'running';

SELECT @current_maxdop = CAST(value_in_use AS INT)
FROM sys.configurations
WHERE name = 'max degree of parallelism';

-- If parallelism is saturating CPUs and MAXDOP is unconstrained, reduce it
IF @active_parallel_queries > 8 AND @current_maxdop = 0
BEGIN
    EXEC sp_configure 'max degree of parallelism', 4;
    RECONFIGURE WITH OVERRIDE;

    INSERT INTO dba_autonomous_log (event_type, old_value, new_value, reason,
    event_time)
    VALUES ('MAXDOP_ADJUST', '0', '4',
            CONCAT('Active parallel queries: ', @active_parallel_queries),
            GETUTCDATE());
END;
```

Self-configuration also extends to infrastructure parameters managed outside the database process itself: kernel `vm.overcommit_memory`, `huge_pages`, TCP socket buffer sizes, and OS scheduler settings all affect database performance. An autonomous system managing SQL Server on Windows or Linux should include a host-configuration layer — often implemented as PowerShell DSC or Ansible playbooks that read system metrics, compare them to tuning profiles, and apply OS settings such as Lock Pages in Memory, max server memory alignment, and storage layout checks within a defined safe envelope.

The key constraint of self-configuration: **changes must be idempotent and auditable**. Every parameter the system changes must be recorded with its previous value, the signal that triggered the change, the timestamp, and the rollback command. This is not optional instrumentation — it is the mechanism by which the system can undo its own decisions when they produce unintended effects.

Self-Healing: Detecting Failure and Restoring State Without Human Intervention

Self-healing is the most operationally visible autonomous capability — it is what gets demoed to CTOs and what eliminates 3 AM pages. But it is also the capability most likely to be built incorrectly, because engineers conflate "automatic failover" with self-healing. Failover is one action within a self-healing system. A complete self-healing architecture handles the full failure taxonomy: process crashes, data corruption, replication divergence, connection pool exhaustion, runaway queries, storage saturation, and schema-level lock accumulation.

For SQL Server high availability, the most production-hardened pattern is Always On Availability Groups with automatic failover, arbitrated by the Windows Server Failover Cluster quorum. The cluster is inherently autonomous: it runs health checks on every node, detects primary failure through missed heartbeats and failed `sp_server_diagnostics` responses, and promotes the best synchronous secondary without human input. The "best" secondary is not chosen at random — it is based on synchronization state, configured failover priority, and quorum voting.

An availability group configured for minimal autonomous intervention encodes its promotion policy in the group definition itself:

```

-- Availability group configured for autonomous failover
-- Health-check timeout and sync requirements encode the promotion policy
CREATE AVAILABILITY GROUP [AG_Orders]
WITH (
    AUTOMATED_BACKUP_PREFERENCE = SECONDARY,
    DB_FAILOVER = ON,
    REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT = 1
)
FOR DATABASE [OrdersDB]
REPLICA ON
    'SQL-PROD-01' WITH (
        ENDPOINT_URL = 'TCP://SQL-PROD-01:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SEEDING_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY),
        SESSION_TIMEOUT = 10          -- 10s health-check timeout; primary must
respond
    ),
    'SQL-PROD-02' WITH (
        ENDPOINT_URL = 'TCP://SQL-PROD-02:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SEEDING_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY),
        SESSION_TIMEOUT = 10
    );

```

The cluster resource monitor in Windows Server Failover Clustering handles the arbitration — detecting primary failure, validating quorum, and promoting the designated automatic failover partner. For environments running SQL Server on Linux or in Kubernetes, Pacemaker with the `mssql-server-ha` resource agent replicates this quorum-arbitrated behavior on Linux-native infrastructure.

Beyond process-level failover, self-healing must address **query-level failures** — specifically, runaway queries that consume resources beyond their operational budget. A self-healing system has a defined policy for what happens when a query exceeds a resource budget, not just a monitoring alert. In SQL Server, `KILL <session_id>` (granted via `ALTER ANY CONNECTION`) is the action primitive; the autonomous system wraps it in a policy engine:

```

# autonomous_query_governor.ps1
# Runs on a 15-second loop via SQL Server Agent; enforces query resource policies

$Policies = @{
    default      = @{ MaxAgeSeconds = 300; ExemptLogins = @('sa','NT
SERVICE\SQLSERVERAGENT','backup_agent'); Action = 'Kill' }
    batch_window = @{ MaxAgeSeconds = 7200; ExemptLogins = @('sa','NT
SERVICE\SQLSERVERAGENT','backup_agent','etl_user'); Action = 'LogOnly' }
}

function Invoke-QueryGovernor {
    param([string]$Instance, [hashtable]$Policy)

    $exempt = ($Policy.ExemptLogins | ForEach-Object { "$$_" }) -join ','
    $budgetMs = $Policy.MaxAgeSeconds * 1000
    $q = @"
SELECT r.session_id, s.login_name, r.total_elapsed_time,
       SUBSTRING(t.text, 1, 2000) AS query_text
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50
      AND r.total_elapsed_time > $budgetMs
      AND s.login_name NOT IN ($exempt);
"@
    foreach ($row in (Invoke-Sqlcmd -ServerInstance $Instance -Database master -
Query $q)) {
        # Audit first, act second – every autonomous kill is recorded
        $safeText = $row.query_text -replace '"', '""'
        Invoke-Sqlcmd -ServerInstance $Instance -Database DBAOps -Query @"
INSERT INTO dbo.autonomous_actions (action_type, target_session, target_login,
reason, query_snapshot, ts)
VALUES ('$(($Policy.Action.ToUpper()))', $($row.session_id), '$($row.login_name)',
      'elapsed_ms=$(($row.total_elapsed_time) > budget_ms=$budgetMs',
      '$safeText', SYSDATETIME());
"@
        if ($Policy.Action -eq 'Kill') {
            Invoke-Sqlcmd -ServerInstance $Instance -Database master `
                -Query "KILL $($row.session_id);"
            Write-Warning "Terminated session $($row.session_id) (login=$
($row.login_name), elapsed_ms=$(($row.total_elapsed_time))"
        }
    }
}

Invoke-QueryGovernor -Instance 'SQL-PROD-01' -Policy $Policies.default

```

Self-healing systems must also incorporate **self-testing**. A system that claims it will heal itself when the primary fails should periodically verify that claim — by executing controlled chaos against non-production replicas, checking that the failover path resolves within SLO, and validating that application connection strings correctly reroute after a leader change. This is the discipline of chaos engineering applied to autonomous database infrastructure.

Self-Optimization: Continuous Performance Without Manual Tuning

Self-optimization is what separates an autonomous system from a well-monitored one. A well-monitored system tells the DBA that table `orders` is missing an index on `(customer_id, created_at)`. An autonomous system identifies the missing index, evaluates the impact of creating it — including its write amplification cost and the estimated query improvement — decides whether to create it immediately, schedule it during low-traffic hours, or defer it pending further data collection, and then executes the DDL with zero DBA involvement.

Oracle's Automatic Indexing and Azure SQL's Automatic Tuning operate on variants of this principle. SQL Server teams building equivalent capability on-premises construct it from Query Store, the missing-index DMVs, and a policy engine — the same composable parts this chapter's examples assemble.

For SQL Server, `sys.dm_exec_query_stats` and `sys.dm_db_missing_index_details` provide the raw signal. The autonomous optimizer reads these DMVs, identifies queries with high total worker time and low selectivity on table scans, and proposes index candidates — the engine's own cost estimates (via `avg_user_impact`) quantify the expected benefit before anything is created:


```
-- Self-optimization: surface missing index candidates ranked by improvement
potential
SELECT TOP 20
    mid.database_id,
    migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans)
        AS improvement_score,
    migs.avg_total_user_cost,
    migs.avg_user_impact,
    migs.user_seeks,
    migs.user_scans,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    OBJECT_NAME(mid.object_id, mid.database_id) AS table_name,
    -- Generate the DDL that the autonomous engine would execute
    CONCAT(
        'CREATE INDEX [IX_auto_', OBJECT_NAME(mid.object_id, mid.database_id), '_',
        REPLACE(ISNULL(mid.equality_columns, '') +
            ISNULL(mid.inequality_columns, ''), ', ', '_'), ']' ON ',
        OBJECT_NAME(mid.object_id, mid.database_id), ' (',
        ISNULL(mid.equality_columns, ''),
        CASE WHEN mid.inequality_columns IS NOT NULL
            THEN ', ' + mid.inequality_columns ELSE '' END, ')',
        CASE WHEN mid.included_columns IS NOT NULL
            THEN ' INCLUDE (' + mid.included_columns + ')' ELSE '' END
    ) AS proposed_ddl
FROM sys.dm_db_missing_index_groups mig
JOIN sys.dm_db_missing_index_group_stats migs
    ON mig.index_group_handle = migs.group_handle
JOIN sys.dm_db_missing_index_details mid
    ON mig.index_handle = mid.index_handle
WHERE migs.avg_user_impact > 30 -- Only consider if estimated impact > 30%
ORDER BY improvement_score DESC;
```

Self-optimization extends beyond indexes into query plan stability. SQL Server's plan cache is volatile by design — statistics changes cause plan flips that can degrade production queries overnight. An autonomous optimization system monitors plan changes via Query Store: when a query's average duration increases more than 3x between consecutive observation windows, it triggers a plan investigation and compares the current plan to the last known good plan preserved in Query Store's plan history.

SQL Server's Query Store provides this natively. When automatic plan correction is enabled, SQL Server detects plan regressions and forces the last known good plan without DBA intervention — this is an example of self-optimization built into the engine that ADE-level engineers should rely on rather than replicate:

```
-- Enable automatic plan correction via Query Store on SQL Server
ALTER DATABASE [production_db] SET QUERY_STORE = ON;

ALTER DATABASE [production_db] SET QUERY_STORE
(
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    INTERVAL_LENGTH_MINUTES = 60,
    MAX_STORAGE_SIZE_MB = 1000,
    QUERY_CAPTURE_MODE = AUTO,
    SIZE_BASED_CLEANUP_MODE = AUTO
);

-- Enable automatic tuning – plan forcing is applied autonomously
ALTER DATABASE [production_db] SET AUTOMATIC_TUNING (
    FORCE_LAST_GOOD_PLAN = ON
);

-- Verify autonomous corrections in effect
SELECT
    name,
    reason,
    score,
    JSON_VALUE(details, '$.implementationDetails.script') AS forced_plan_action,
    last_updated
FROM sys.dm_db_tuning_recommendations
WHERE state = 'Verifying' OR state = 'Success'
ORDER BY score DESC;
```

Governance, Guardrails, and the Autonomous Safety Contract

The single most dangerous misconception in autonomous database engineering is that more autonomy is always better. Unconstrained autonomous action is not a feature — it is a liability. The ADE's job is not to build a system that does everything automatically; it is to build a system that does the right things automatically, within a precisely defined safety boundary, and that halts and escalates when it encounters a situation outside that boundary.

This is the **autonomous safety contract**: a formal specification of what the system is permitted to do without human approval, what requires human confirmation before execution, and what is permanently prohibited regardless of any signal. Every production autonomous database system should have this contract written down, versioned, and enforced in code — not assumed.

A typical safety contract for a SQL Server autonomous system might be structured as follows:

Action Class	Permitted Autonomously	Requires Approval	Prohibited

|---|---|---|---|

| Session-level parameter changes | ✓ | — | — |

| CREATE INDEX ... WITH (ONLINE = ON) | During maintenance window | Outside window if impact score < 50 | Never on replicated tables without replication check |

| UPDATE STATISTICS | ✓ | — | — |

| KILL session_id | Queries > 5min over budget | Queries 2-5min over budget | sa / service sessions |

| sp_configure + RECONFIGURE | ✓ | — | — |

| Availability group automatic failover | Detected primary failure | Planned manual failover | — |

| DROP INDEX | — | Always | Indexes protecting FK constraints |

| Schema DDL other than indexes | — | — | Prohibited (human only) |

This contract is enforced programmatically. The autonomous engine's action dispatcher validates every proposed action against the safety contract before execution:

```

# safety_contract.py – action dispatcher with autonomous safety validation

from enum import Enum
from typing import Callable, Optional
import logging

class ApprovalLevel(Enum):
    AUTONOMOUS = "autonomous"
    REQUIRES_APPROVAL = "requires_approval"
    PROHIBITED = "prohibited"

class SafetyContract:
    def __init__(self, contract_definition: dict, approval_callback:
Optional[Callable] = None):
        self.contract = contract_definition
        self.approval_callback = approval_callback
        self.audit_log = []

    def evaluate(self, action_type: str, context: dict) -> ApprovalLevel:
        rule = self.contract.get(action_type)
        if rule is None:
            # Unknown actions default to prohibited – fail closed
            return ApprovalLevel.PROHIBITED

        level = rule["default"]

        # Apply context overrides
        for condition, override_level in rule.get("conditions", []):
            if condition(context):
                level = override_level
                break

        return ApprovalLevel(level)

    def dispatch(self, action_type: str, action_fn: Callable, context: dict) ->
bool:
        level = self.evaluate(action_type, context)

        self.audit_log.append({
            "action": action_type,
            "context": context,
            "approval_level": level.value
        })

        if level == ApprovalLevel.AUTONOMOUS:
            logging.info(f"[AUTONOMOUS] Executing {action_type} |
context={context}")
            action_fn(context)
            return True

        elif level == ApprovalLevel.REQUIRES_APPROVAL:
            logging.info(f"[APPROVAL REQUIRED] {action_type} | context={context}")
            if self.approval_callback and self.approval_callback(action_type,

```

```

context):
    action_fn(context)
    logging.info(f"[APPROVED] Executing {action_type} |
context={context}")
    return True
    logging.warning(f"[NOT APPROVED] {action_type} | context={context}")
    return False

    else: # PROHIBITED
        logging.error(f"[PROHIBITED] Refusing to execute {action_type} |
context={context}")
        return False

```

The `evaluate` method's fail-closed default — unknown action types default to `PROHIBITED` — is the single most important line in the contract. As the system encounters new action types over time (new tooling, new extensions, new failure modes), the safe default is always to require human review rather than assume autonomy is appropriate. This is the same posture Tier 3 classification takes in a taxonomy-based safety model: silence in the specification means "ask a human," never "proceed."

```

graph TD
    O[Observe  
context-enriched telemetry] --> D[Decide  
policy engine evaluates signals]
    D --> G["G{Safety Contract  
evaluate action_type}"]
    G -->|Autonomous| A[Act: execute directly]
    G -->|Requires Approval| H[Notify + wait for approval]
    G -->|Prohibited / unknown| X[Refuse + log – fail closed]
    H -->|approved| A
    H -->|denied or timeout| X
    A --> O

```

The Observe-Decide-Act loop as gated by the autonomous safety contract: every action is classified before it executes, and unknown action types default to prohibited rather than permitted.

The autonomous safety contract, the taxonomy of autonomous actions, and the governance gate are three names for the same underlying discipline: autonomy is not a binary switch, it is a graduated, auditable, and reversible boundary that the engineering team defines explicitly rather than discovers by incident.

Key Takeaways

- → Every autonomous database system is built on the Observe–Decide–Act loop, and the quality of the system is bounded by the weakest of the three phases — rich observation with a shallow decision layer produces a system that reacts fast to the wrong things.
- → Self-configuration requires the system to know which operational parameters can change immediately, which require a reload, and which require a restart — and to queue changes accordingly, with every change recorded as an idempotent, reversible, auditable event.
- → Self-healing is broader than automatic failover: a complete implementation covers process crashes, runaway queries, connection exhaustion, and storage saturation, and it periodically verifies its own recovery paths through controlled testing rather than assuming they still work.
- → Self-optimization tools like Query Store automatic plan correction let the system evaluate the impact of a change — an index, a forced plan — before committing to it, which is what separates autonomous optimization from blind automation.
- → The autonomous safety contract is the governing artifact of the entire system: a versioned, code-enforced specification of what the system may do alone, what needs human approval, and what is permanently off-limits, with unknown actions defaulting to prohibited rather than permitted.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 4: Telemetry, Metrics, and Data Required for Autonomy

An autonomous database system is only as intelligent as the data it can see about itself. Before any self-healing loop executes, before any workload gets rerouted, before any index gets dropped or any connection pool gets resized — the system must first observe its own state with enough fidelity to make a sound decision. This chapter is about building that observational foundation: the telemetry pipelines, metric taxonomies, and contextual data structures that transform a passively monitored database into a fully aware autonomous engine. Getting this layer right is the prerequisite for everything else in this book. Getting it wrong means your autonomous actions will be based on incomplete or misleading signals — and autonomous actions taken on bad data are worse than no action at all.

4.1 The Telemetry Taxonomy: What Autonomous Systems Actually Need to See

Most monitoring stacks are built for humans — dashboards, alerts, on-call pages. Autonomous systems need something different: structured, timestamped, low-latency signal feeds that can be ingested by decision engines without human interpretation. Before touching a single query or Prometheus scrape config, the autonomous database architect needs a taxonomy of what categories of data actually drive autonomous decisions.

The four layers of database telemetry are:

1. Resource telemetry — CPU, memory, disk I/O, network throughput, and storage capacity at both the OS and database engine level. These are necessary but not sufficient. An autonomous system that reacts only to CPU spikes is a basic autoscaler, not an autonomous database.

2. Engine-internal metrics — buffer cache hit ratios, lock wait chains, transaction throughput, index maintenance progress, checkpoint frequency, transaction log flush throughput, query plan cache hit rates, and replication lag. These metrics describe what the database engine itself is experiencing, independent of what the hardware is doing.

3. Workload telemetry — query-level execution data: individual query durations, I/O reads per query, wait events, plan changes over time, and parameter distributions. This is the layer most monitoring systems underinvest in, and it's where the highest-value autonomous decisions live.

4. State and topology telemetry — cluster membership, replica health, failover readiness, connection pool saturation, and schema version state. Autonomous systems cannot safely act without understanding the current topology — promoting a standby without knowing whether it's actually current is a path to split-brain.

Each layer feeds different decision classes. Resource telemetry drives scaling and capacity decisions. Engine-internal metrics drive tuning and maintenance scheduling. Workload telemetry drives query optimization and plan pinning. State telemetry drives failover and topology management. A production autonomous system maintains all four layers simultaneously, with different collection frequencies appropriate to each.

The engine-internal layer lives in Dynamic Management Views (DMVs), the `sys` catalog views, the Query Store, and the Extended Events system — plus the transaction log subsystem for durability telemetry. These are your primary observational surfaces — everything else is derived from them.

4.2 Instrumenting SQL Server for Machine-Readable Metric Collection

The distinction between a monitoring-friendly database and an autonomy-ready database comes down to whether your metric collection is designed for human dashboards or for programmatic consumption. Autonomous systems need metrics that are timestamped precisely, labeled with enough dimensional context to drive decisions, and collected at frequencies that match the dynamics of the phenomena they describe.

SQL Server: Core Metric Surfaces

The `sys.dm_exec_requests` and `sys.dm_exec_sessions` DMVs are the primary source for real-time workload visibility. For autonomous systems, the critical fields are `wait_type`, `wait_time`, `status`, `start_time`, and `query_hash`. Querying these DMVs every 5–10 seconds and persisting the results gives your decision engine a timeline of what the database was doing at any moment.


```

-- Equivalent workload snapshot using DMVs
-- Designed for programmatic collection; include in a scheduled job or monitoring
agent
SELECT
    GETUTCDATE()                                AS collected_at,
    r.session_id,
    s.login_name,
    s.program_name,
    s.host_name,
    r.status,
    r.wait_type,
    r.wait_time / 1000.0                        AS wait_time_seconds,
    r.total_elapsed_time / 1000.0              AS elapsed_seconds,
    r.logical_reads,
    r.writes,
    r.cpu_time / 1000.0                         AS cpu_seconds,
    qt.text                                     AS query_text,
    qp.query_plan
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(r.plan_handle) qp
WHERE r.session_id != @@SPID
      AND s.is_user_process = 1
ORDER BY r.total_elapsed_time DESC;

```

These snapshots are the raw feed. They become useful to an autonomous system only when they're persisted with timestamps and queryable across time. A clustered columnstore table in SQL Server is appropriate for storing these at scale. The autonomous decision engine then queries the history, not just the present-moment snapshot.

Structured Metric Emission with DMVs and Query Store

For workload telemetry at the query level, `sys.dm_exec_query_stats` and the Query Store are the authoritative sources. These surfaces aggregate execution statistics per normalized query, giving you mean execution time, standard deviation, total I/O, and call count — exactly the inputs needed to identify regression candidates for autonomous tuning.

```

-- Query Store equivalent for regression detection
-- Surfaces queries with high average duration and significant variability
SELECT TOP 50
    GETUTCDATE()                                AS collected_at,
    qs.query_id,
    qt.query_sql_text,
    qp.plan_id,
    rs.count_executions,
    ROUND(rs.avg_duration / 1000.0, 3)           AS avg_duration_ms,
    ROUND(rs.stdev_duration / 1000.0, 3)         AS stdev_duration_ms,
    ROUND(rs.avg_logical_io_reads, 0)            AS avg_logical_reads,
    ROUND(rs.avg_cpu_time / 1000.0, 3)           AS avg_cpu_ms,
    rs.last_execution_time
FROM sys.query_store_query qs
JOIN sys.query_store_query_text qt ON qs.query_text_id = qt.query_text_id
JOIN sys.query_store_plan qp      ON qs.query_id = qp.query_id
JOIN sys.query_store_runtime_stats rs ON qp.plan_id = rs.plan_id
WHERE rs.last_execution_time > DATEADD(MINUTE, -10, GETUTCDATE())
    AND rs.count_executions > 5
ORDER BY rs.avg_duration DESC;

```

The delta pattern is important here: you capture a snapshot, store it, capture the next snapshot 60 seconds later, and compute the difference in `calls`, `total_exec_time`, and I/O. This gives you per-interval throughput rates rather than cumulative totals, which is what decision logic actually needs. A query whose `avg_ms` jumps from 12ms to 340ms between two consecutive snapshots is a regression event — and that event should trigger autonomous investigation without waiting for a human to notice a dashboard spike.

4.3 Building a Real-Time Telemetry Pipeline with Prometheus, OpenTelemetry, and SQL Server Columnstore

Raw metric collection from database views is only half the architecture. Those metrics need to flow into a storage and query layer that the autonomous decision engine can interrogate in real time. The canonical stack is Prometheus + `sql_exporter` for scraping, combined with a SQL Server columnstore staging database for long-retention analytical queries and Alertmanager as a trigger surface for autonomous actions.

Prometheus Exporter Configuration

The `sql_exporter` from the Prometheus community exposes DMV output as Prometheus metrics. Its custom query capability is what makes it useful for autonomous systems — you can expose any DMV query you need, not just the defaults.

```
# custom_queries.yaml – extend sql_exporter for autonomy-relevant metrics
mssql_ag_lag:
  query: |
    SELECT ag.name AS ag_name,
           ar.replica_server_name,
           ISNULL(drs.log_send_queue_size, 0) AS send_queue_kb,
           ISNULL(drs.redo_queue_size, 0) AS redo_queue_kb
    FROM sys.availability_groups ag
    JOIN sys.availability_replicas ar
      ON ag.group_id = ar.group_id
    LEFT JOIN sys.dm_hadr_database_replica_states drs
      ON ar.replica_id = drs.replica_id
  metrics:
    - ag_name:
        usage: "LABEL"
    - replica_server_name:
        usage: "LABEL"
    - send_queue_kb:
        usage: "GAUGE"
        description: "Log send queue in KB"
    - redo_queue_kb:
        usage: "GAUGE"
        description: "Redo queue in KB"

mssql_long_running_queries:
  query: |
    SELECT COUNT(*) AS count
    FROM sys.dm_exec_requests
    WHERE session_id > 50
      AND total_elapsed_time > 30000
  metrics:
    - count:
        usage: "GAUGE"
        description: "Number of queries running longer than 30 seconds"

mssql_index_fragmentation:
  query: |
    SELECT DB_NAME(ips.database_id) AS database_name,
           OBJECT_NAME(ips.object_id, ips.database_id) AS table_name,
           MAX(ips.avg_fragmentation_in_percent) AS max_fragmentation_pct
    FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
    WHERE ips.page_count > 10000
    GROUP BY ips.database_id, ips.object_id
  metrics:
    - database_name:
        usage: "LABEL"
    - table_name:
        usage: "LABEL"
    - max_fragmentation_pct:
        usage: "GAUGE"
        description: "Worst index fragmentation; trigger autonomous maintenance at threshold"
```

This YAML feeds directly into the `sql_exporter` collector configuration. The index fragmentation metric is particularly important: when `max_fragmentation_pct` exceeds 30% on an index with more than 10,000 pages, the autonomous maintenance engine should schedule a targeted `ALTER INDEX ... REORGANIZE` or `REBUILD` for that specific index — not a database-wide rebuild, and not an alert to a human. This is what machine-readable telemetry enables: surgical, targeted autonomous action.

Alertmanager as an Autonomous Trigger Surface

Most teams use Alertmanager to page engineers. In an autonomous architecture, Alertmanager fires webhooks that trigger autonomous action scripts, not pagers. The alert becomes an event in the autonomous decision pipeline, not a notification.

```
# alertmanager.yml – route database alerts to autonomous action endpoint
route:
  group_by: ['alertname', 'database', 'instance']
  group_wait: 10s
  group_interval: 30s
  repeat_interval: 5m
  receiver: 'autonomous-action-handler'
  routes:
    - match:
        alertname: MSSQLAGLagHigh
        receiver: 'autonomous-failover-handler'
        continue: false
    - match:
        alertname: MSSQLLongRunningQueryDetected
        receiver: 'autonomous-query-killer'
        continue: false
    - match:
        alertname: MSSQLIndexFragmentationHigh
        receiver: 'autonomous-index-maintenance'
        continue: false

receivers:
  - name: 'autonomous-action-handler'
    webhook_configs:
      - url: 'http://ade-controller:8080/api/v1/action'
        send_resolved: true
        http_config:
          bearer_token: "${ADE_WEBHOOK_TOKEN}"

  - name: 'autonomous-failover-handler'
    webhook_configs:
      - url: 'http://ade-controller:8080/api/v1/failover'
        send_resolved: false

  - name: 'autonomous-query-killer'
    webhook_configs:
      - url: 'http://ade-controller:8080/api/v1/terminate-query'
        send_resolved: false

  - name: 'autonomous-index-maintenance'
    webhook_configs:
      - url: 'http://ade-controller:8080/api/v1/schedule-index-maintenance'
        send_resolved: true
```

```
graph LR
    T[Resource / Engine / Workload / State Telemetry] --> E[sql_exporter]
    E --> P[Prometheus TSDB]
    P --> AM[Alertmanager]
    AM -->|MSQLAGLagHigh| F[autonomous-failover-handler]
    AM -->|MSQLLongRunningQueryDetected| K[autonomous-query-killer]
    AM -->|MSQLIndexFragmentationHigh| V[autonomous-index-maintenance]
    F --> C[ADE Controller]
    K --> C
    V --> C
```

The telemetry pipeline: four layers of raw signal flow through exporters into Prometheus, where Alertmanager routes classified alerts to webhook handlers instead of a pager.

The ADE controller at `http://ade-controller:8080` is the autonomous decision engine — a service (covered in detail in Chapter 7) that receives webhook payloads from Alertmanager, validates the trigger conditions against a second source of truth, and executes the appropriate action. The double-check against a second source is non-negotiable: acting on a single metric signal without validation is how autonomous systems cause outages.

4.4 Wait Event Analysis: The Highest-Signal Metric Category for Autonomous Diagnosis

Wait events are the most information-dense signal category in any relational database engine. A CPU spike tells you that something is consuming CPU. A wait event tells you *why* the database is blocked — whether it's waiting for a lock held by another transaction, waiting for a buffer to be read from disk, waiting for network I/O to complete, or waiting for a transaction log write to harden. This distinction is what separates a monitoring system from an autonomous diagnostic system.

In SQL Server, wait events are surfaced in `sys.dm_exec_requests.wait_type` and aggregated in `sys.dm_os_wait_stats`. The taxonomy includes `LCK_M_*` (locking), `PAGEIOLATCH_*` (data file I/O), `CXPACKET / CXCONSUMER` (parallelism), `SOS_SCHEDULER_YIELD` (CPU pressure), `WRITELOG` (transaction log throughput), and `HADR_*` (availability replica sync) categories. Each category implies a different autonomous response.

The autonomous decision tree for SQL Server wait events looks like this:

- → ** `LCK_M_*` (locks): **Sessions are blocked on locks. Autonomous action: identify the blocker via `blocking_session_id` in `sys.dm_exec_requests`, evaluate its age and importance, and if it exceeds the threshold (e.g., >120 seconds with >5 blocked waiters), kill the blocking session.**

- → `PAGEIOLATCH_*` / `PAGEIO_*` (I/O): Sessions are waiting for data to be read from disk. **Autonomous action:** warm the buffer pool for the hot table, evaluate max server memory headroom, and flag for capacity review if sustained.
- → `WRITELOG` : High transaction log write contention. **Autonomous action:** verify the log file sits on dedicated low-latency storage, check for autogrowth stalls, and consider delayed durability for workloads that tolerate it — without compromising durability for committed transactions.
- → `THREADPOOL` / worker starvation**: Worker threads are exhausted. **Autonomous action:** check `max worker threads` against current utilization and hunt the blocking chains consuming workers before raising any ceiling.

```

-- Wait type aggregation with autonomous action classification
-- Run on a schedule; compare delta between snapshots for rate-of-change analysis
WITH wait_stats AS (
    SELECT
        wait_type,
        waiting_tasks_count,
        wait_time_ms,
        max_wait_time_ms,
        signal_wait_time_ms,
        wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms
    FROM sys.dm_os_wait_stats
    WHERE wait_type NOT IN (
        -- Exclude benign background waits
        'SLEEP_TASK', 'BROKER_TO_FLUSH', 'BROKER_TASK_STOP',
        'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE',
        'FT_IFTS_SCHEDULER_IDLE_WAIT', 'HADR_WORK_QUEUE',
        'HADR_FILESTREAM_IOMGR_IOCOMPLETION', 'ONDEMAND_TASK_QUEUE',
        'REQUEST_FOR_DEADLOCK_SEARCH', 'RESOURCE_QUEUE',
        'SERVER_IDLE_CHECK', 'SLEEP_DBSTARTUP', 'SLEEP_DBRECOVER',
        'SLEEP_MASTERDBREADY', 'SLEEP_MASTERMDREADY',
        'SLEEP_MASTERUPGRADED', 'SLEEP_MSDBSTARTUP',
        'SLEEP_SYSTEMTASK', 'SLEEP_TEMPDBSTARTUP',
        'SNI_HTTP_ACCEPT', 'SP_SERVER_DIAGNOSTICS_SLEEP',
        'SQLTRACE_BUFFER_FLUSH', 'WAITFOR', 'XE_DISPATCHER_WAIT',
        'XE_TIMER_EVENT', 'BROKER_EVENTHANDLER', 'CHECKPOINT_QUEUE',
        'DBMIRROR_EVENTS_QUEUE', 'SQLTRACE_INCREMENTAL_FLUSH_SLEEP',
        'WAIT_XTP_OFFLINE_CKPT_NEW_LOG'
    )
)
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    ROUND(wait_time_ms / 1000.0, 2) AS wait_time_seconds,
    ROUND(resource_wait_time_ms / 1000.0, 2) AS
resource_wait_seconds,
    ROUND(
        CASE WHEN waiting_tasks_count > 0
            THEN wait_time_ms * 1.0 / waiting_tasks_count
            ELSE 0 END, 2) AS
avg_wait_ms_per_task,
    CASE
        WHEN wait_type LIKE 'LCK_%'
            AND waiting_tasks_count > 20 THEN
'INVESTIGATE_LOCK_CHAIN'
        WHEN wait_type = 'PAGEIOLATCH_SH'
            AND wait_time_ms > 30000 THEN
'REVIEW_BUFFER_POOL_AND_IO'
        WHEN wait_type = 'WRITELOG'
            AND wait_time_ms > 10000 THEN
'REVIEW_LOG_IO_SUBSYSTEM'
        WHEN wait_type = 'RESOURCE_SEMAPHORE' THEN
'EVALUATE_MEMORY_GRANT'
        WHEN wait_type = 'THREADPOOL' THEN

```



```

'EVALUATE_MAX_WORKER_THREADS'
    WHEN wait_type LIKE 'HADR_%'
        AND wait_time_ms > 5000
    THEN
'EVALUATE_MAX_WORKER_THREADS'
    ELSE 'MONITOR'
END
AS recommended_action
FROM wait_stats
ORDER BY wait_time_ms DESC;

```

The `recommended_action` column in these queries is the bridge between metric and decision. An autonomous system ingests this output, reads the action class, and dispatches to the appropriate handler. This pattern — embedding decision logic directly in metric queries — is a valid approach for low-complexity decision rules, and it keeps the logic close to the data where it belongs.

4.5 Contextual State Data: Topology, Schema, and Configuration Telemetry

Metrics about query performance and wait events are meaningless without context about the system's current topology and configuration state. An autonomous action that's perfectly correct for a primary with three healthy replicas may be catastrophically wrong during a failover event. The autonomous system must always know the current state of the world before acting — and that state includes cluster topology, current role (primary or replica), schema version, and the effective value of every configuration parameter the decision logic depends on.

```

-- Availability Group role and replica-count snapshot
SELECT
    ars.role_desc,
    (SELECT COUNT(*) FROM sys.dm_hadr_availability_replica_states) AS
    replica_state_rows,
    SERVERPROPERTY('ProductVersion') AS product_version
FROM sys.dm_hadr_availability_replica_states ars
WHERE ars.is_local = 1;

```

Configuration state — the effective, running values of options like `max server memory`, `max degree of parallelism`, or `cost threshold for parallelism` — must be polled from `sys.configurations` rather than assumed from a baseline document, because a pending `RECONFIGURE` or a Resource Governor override can make the baseline and the running state diverge. An autonomous system that reads its configuration baseline instead of `sys.configurations` is reasoning about a state that may no longer be true.

Schema version telemetry — tracking which migration has been applied, via a migrations table or a tool like Flyway or Sqitch — matters because an autonomous system evaluating whether to create an index or adjust a query plan needs to know it's looking at the schema it

thinks it's looking at, not a version mid-migration. Combining topology, configuration, and schema-version telemetry with the performance and wait-event data from earlier sections gives the decision engine the complete picture it needs before it acts.

Key Takeaways

- → Autonomous systems need telemetry across four layers — resource, engine-internal, workload, and state/topology — and each layer feeds a different class of decision; a system missing any one layer has a blind spot in its corresponding decision class.
- → `sys.dm_exec_requests`, `sys.dm_exec_query_stats`, and Query Store are the primary machine-readable surfaces; snapshotting them on a schedule and computing deltas between snapshots is what turns cumulative counters into actionable rate-of-change signals.
- → Wait event data is the highest-signal category for autonomous diagnosis because it explains *why* a session is blocked, not just that resource utilization is elevated — and it maps directly to a bounded set of recommended actions per wait category.
- → Prometheus with custom exporter queries, and Alertmanager routed to webhook-driven action handlers rather than pagers, form a practical reference pipeline for turning raw telemetry into autonomous triggers — with a second-source validation step before any action executes.
- → No metric is safe to act on in isolation from topology and configuration context; the same signal can call for opposite actions depending on whether the target is a primary or a replica, or whether a schema migration is in flight.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 5: Self-Monitoring Database Architectures

A database that cannot accurately observe itself cannot govern itself. Before any autonomous system can tune indexes, reroute traffic, or initiate a failover, it must first possess a complete, real-time model of its own operational state — not a snapshot delivered by a human running a query, but a continuous, structured stream of telemetry that flows into decision engines without manual intervention. This chapter builds the architectural foundation for that capability: how to instrument SQL Server for autonomous observability, how to design metric pipelines that survive the failures they are meant to detect, and how to move from passive dashboards to systems that interpret their own signals and act on them.

Instrumentation Architecture: What Autonomous Systems Must Observe

The gap between a monitored database and a self-monitoring one is not a matter of how many metrics you collect — it is a matter of what those metrics are connected to. Passive monitoring tells a human that something is wrong. Autonomous instrumentation provides structured signals to a decision layer that determines what to do without human involvement.

For autonomous operation, database telemetry must satisfy four properties:

1. **Completeness** — every failure mode the system might encounter has a measurable precursor signal.
2. **Timeliness** — signals arrive at the decision engine before the failure window closes.
3. **Structure** — metrics are labeled with enough context (database, instance, workload class, query fingerprint) that automated logic can act on them without ambiguity.
4. **Self-referential integrity** — the monitoring pipeline itself is monitored; a dead metrics agent must not leave the autonomous engine flying blind.

SQL Server exposes its internal state through a rich set of system catalog views and Dynamic Management Views (DMVs). The critical surfaces for autonomous instrumentation are

```
sys.dm_exec_requests, sys.dm_exec_sessions, sys.dm_os_wait_stats,
sys.dm_db_index_usage_stats, sys.dm_exec_query_stats,
sys.dm_hadr_availability_replica_states, and the Query Store.
```

SQL Server does not surface all operationally relevant signals by default. The following query illustrates a foundation metric pull that an autonomous agent executes on a scheduled interval — producing a structured record suitable for ingestion into a time-series store like Prometheus, InfluxDB, or a columnstore staging database.

```
-- Autonomous baseline telemetry snapshot: connection pressure, blocking, AG lag,
-- and buffer cache efficiency
SELECT
    GETUTCDATE() AS collected_at,
    (SELECT COUNT(*) FROM sys.dm_exec_requests
     WHERE status = 'running') AS active_requests,
    (SELECT COUNT(*) FROM sys.dm_exec_requests
     WHERE blocking_session_id > 0) AS blocked_requests,
    (SELECT COUNT(*) FROM sys.dm_exec_sessions
     WHERE is_user_process = 1
        AND status = 'sleeping'
        AND DATEDIFF(SECOND, last_request_end_time, GETUTCDATE()) > 30) AS
idle_sessions_over_30s,
    (SELECT MAX(secondary_lag_seconds)
     FROM sys.dm_hadr_database_replica_states
     WHERE is_primary_replica = 0) AS
ag_replication_lag_seconds,
    (SELECT CAST(
        (SUM(CASE WHEN physical_reads = 0 THEN 1.0 ELSE 0.0 END) /
         COUNT(*)) * 100 AS DECIMAL(5,2))
     FROM sys.dm_exec_requests
     WHERE status = 'running') AS
buffer_hit_proxy_pct,
    (SELECT SUM(cntr_value)
     FROM sys.dm_os_performance_counters
     WHERE counter_name = 'Number of Deadlocks/sec'
        AND instance_name = '_Total') AS
deadlocks_per_sec;
```

These snapshots are insufficient on their own. Autonomous systems require delta calculations, moving averages, and anomaly baselines computed over rolling windows — which is why raw metric collection must be paired with a time-series backend. The instrumentation queries provide the raw material; the architecture around them determines whether that material becomes autonomous intelligence.

Building Metric Pipelines That Survive the Failures They Detect

The most dangerous design flaw in any autonomous monitoring architecture is a single-path metric pipeline. If the agent that collects replication lag data runs on the primary node, and the primary node crashes, the autonomous engine loses its replication signal at exactly the moment it is most needed. Every metric pipeline in an autonomous database architecture must be designed with failure-mode independence as a first-order constraint.

The production pattern is a **sidecar-plus-external-aggregator topology**:

- → A lightweight collector (Prometheus `sql_exporter` or a custom PowerShell/Python agent) runs as a sidecar process co-located with each database node — not inside it — and pushes metrics to a remote aggregator.
- → The aggregator (Prometheus with remote write, VictoriaMetrics, or a columnstore staging database) lives on independent infrastructure with no database dependency.
- → The decision engine (Alertmanager rules, a custom Python daemon, or a Kubernetes operator) reads from the aggregator and never talks directly to the database node for telemetry.

```
graph TD
    subgraph Primary Node
        DBA[(Database)]
        SA[Sidecar Collector]
    end
    subgraph Replica Node
        DBB[(Database)]
        SB[Sidecar Collector]
    end
    SA -->|push metrics| AGG[External Aggregator  
Prometheus / VictoriaMetrics / Columnstore]
    SB -->|push metrics| AGG
    AGG --> DE[Decision Engine]
    DE -->|primary metrics go silent at T+30s| FA[Trigger failover evaluation]
```

Sidecar-plus-external-aggregator topology: the collector runs alongside each node but the aggregator and decision engine live on independent infrastructure, so a failed primary is still observable.

This topology means a failed primary is still observable from the external aggregator, because the replica's sidecar continues emitting. The decision engine sees the primary's metrics go silent at `T+30s` (scrape interval expiry), which is itself a signal that triggers the autonomous failover evaluation path.

For SQL Server Always On Availability Groups, this pattern maps onto a distributed collector that queries `sys.dm_hadr_availability_replica_states` from every replica — not just the primary — so the system maintains a consistent view of the AG topology even during role transitions.

The following Python agent demonstrates a resilient collection loop with self-healing behavior: if the database connection is lost, the agent emits a `db_unreachable` metric rather than silently dying.

```

import pyodbc
import time
import prometheus_client as prom
from prometheus_client import Gauge, Counter

# --- Metric definitions ---
DB_ACTIVE_CONNECTIONS = Gauge('mssql_active_connections', 'Active user sessions',
                                ['host', 'database'])
DB_AG_SEND_QUEUE = Gauge('mssql_ag_send_queue_kb', 'Max AG log send queue in
KB',
                           ['host'])
DB_UNREACHABLE = Gauge('mssql_unreachable', '1 if database is unreachable, 0
if healthy',
                        ['host'])
COLLECTION_ERRORS = Counter('mssql_collection_errors_total', 'Total metric
collection failures',
                             ['host'])

COLLECTION_QUERY = """
SELECT
    (SELECT count(*) FROM sys.dm_exec_sessions
     WHERE is_user_process = 1 AND status = 'running') AS active_conn,
    COALESCE(
        (SELECT MAX(log_send_queue_size)
         FROM sys.dm_hadr_database_replica_states), 0
    ) AS send_queue_kb
"""

def collect(host: str, conn_string: str, interval: int = 15):
    """Resilient metric collection loop. Emits mssql_unreachable on failure."""
    prom.start_http_server(9187)
    conn = None

    while True:
        try:
            if conn is None:
                conn = pyodbc.connect(conn_string, timeout=5)
                DB_UNREACHABLE.labels(host=host).set(0)

            with conn.cursor() as cur:
                cur.execute(COLLECTION_QUERY)
                row = cur.fetchone()
                DB_ACTIVE_CONNECTIONS.labels(host=host,
                                                database=conn.getinfo(pyodbc.SQL_DATABASE_NAME)).set(row[0])
                DB_AG_SEND_QUEUE.labels(host=host).set(row[1])
                DB_UNREACHABLE.labels(host=host).set(0)

        except pyodbc.Error:
            # Connection lost – mark unreachable so decision engine can act
            DB_UNREACHABLE.labels(host=host).set(1)
            COLLECTION_ERRORS.labels(host=host).inc()
            try:
                conn.close()

```

```

except Exception:
    pass
    conn = None # Force reconnect on next cycle

except Exception:
    COLLECTION_ERRORS.labels(host=host).inc()

time.sleep(interval)

```

The `mssql_unreachable` gauge is not decorative. The autonomous decision engine subscribes to it via Alertmanager and uses it as a trigger condition for failover evaluation — but only after correlating it with replica health signals to distinguish a true primary failure from a network partition affecting only the collector.

On Windows clusters or Kubernetes, the same pattern uses the community `sql_exporter` with a `job_name: sqlserver` target configuration, with separate scrape endpoints per replica in the AG set. The key architectural rule is identical: **the metric pipeline must be independent of the components it monitors.**

Autonomous Alerting Logic: From Thresholds to Conditional Decision Trees

Static thresholds are the lowest form of monitoring intelligence. Setting `alert when connections > 200` works until the day you provision a larger instance, add a connection pooler, or change application behavior — at which point the threshold is wrong and nobody updates it. Autonomous systems replace static thresholds with **dynamic baselines** and **conditional decision trees** that route different signals to different response actions.

The architecture has three tiers:

Tier 1 — Absolute violation alerts (respond immediately, no learning required): AG replica reporting as disconnected, log send queue stalled, disk at > 95% capacity, database engine down. These have no legitimate normal state and the autonomous response is always the same action.

Tier 2 — Baseline deviation alerts (respond based on deviation from a rolling average): query latency at P99 exceeds 2× the 7-day baseline, connection count exceeds the rolling mean by 3 standard deviations, lock wait time rising at > 20% per minute. The response depends on the magnitude and direction of deviation.

Tier 3 — Trend projection alerts (act before a threshold is hit): at the current rate of transaction log growth, the log send queue will exceed 10GB in 45 minutes; at the current growth rate, the data volume will exhaust disk in 6 hours. These fire before anything is broken, giving the autonomous system time to act gracefully.

The following Prometheus alerting rules implement all three tiers for a SQL Server deployment:

```
# prometheus/alerts/mssql_autonomous.yml
groups:
- name: mssql_autonomous_tier1
  rules:
    - alert: MSSQLReplicaDisconnected
      expr: mssql_ag_send_queue_kb > 10485760
        OR absent(mssql_ag_send_queue_kb)
      for: 30s
      labels:
        severity: critical
        autonomous_action: evaluate_failover
      annotations:
        summary: "AG send queue critical or lag signal lost on {{ $labels.host }}"
        action: "Trigger autonomous failover evaluation pipeline"

- name: mssql_autonomous_tier2
  rules:
    - alert: ConnectionSurgeAboveBaseline
      expr: >
        mssql_active_connections
        > (avg_over_time(mssql_active_connections[7d]) +
          3 * stddev_over_time(mssql_active_connections[7d]))
      for: 2m
      labels:
        severity: warning
        autonomous_action: scale_connection_pool
      annotations:
        summary: "Connection count {{ $value }} exceeds 3σ baseline on
{{ $labels.host }}"

- name: mssql_autonomous_tier3
  rules:
    - alert: AGLogSendQueueTrendCritical
      expr: >
        predict_linear(mssql_ag_send_queue_kb[30m], 2700) > 10737418240
      for: 0s
      labels:
        severity: warning
        autonomous_action: evaluate_replica_health
      annotations:
        summary: >
          "AG send queue on {{ $labels.host }} projected to exceed 10GB
          in 45 minutes"
```

The `autonomous_action` label is not a documentation field — it is read by the Alertmanager webhook receiver and used to route the firing alert to the correct autonomous action handler. When `ConnectionSurgeAboveBaseline` fires, the webhook calls the connection-pool

scaling function. When `AGLogSendQueueTrendCritical` fires, the handler queries `sys.dm_hadr_database_replica_states` to identify the lagging replica and evaluates failover or replica removal.

This label-driven routing means the alert definition and the action it triggers are coupled in a structured, auditable way — which matters enormously when a CTO asks why the system dropped a replication slot at 3am.

For SQL Server, the equivalent conditional logic lives in SQL Server Agent jobs triggered by WMI alerts, or — in more modern architectures — in Azure Monitor alert action groups that call Azure Automation runbooks. The autonomous action label pattern translates directly: an action group tag `autonomous_action: rebuild_fragmented_indexes` routes to a runbook that evaluates fragmentation levels and executes `ALTER INDEX ... REBUILD` with `ONLINE = ON` during the next maintenance window.

Columnstore History and SQL Server Temporal Tables as the Autonomous Memory Layer

Prometheus and Alertmanager handle real-time signal processing well, but they are poor historians. The autonomous decision engine needs more than a 15-second snapshot — it needs to ask: **was this exact query pattern responsible for a similar latency spike six weeks ago, and what resolved it?** That requires a queryable, relational history of database state, and the production solution pairs a columnstore metric-history table with SQL Server temporal tables.

SQL Server columnstore indexes provide segment elimination and high compression ratios that make multi-month telemetry analytics practical. By routing metric snapshots into a time-partitioned columnstore staging table, the autonomous engine gains the ability to run arbitrary SQL analytics over months of telemetry — and to do so on the same database engine it already operates.

SQL Server (columnstore table for metric storage):

```
-- Schema for autonomous metric history (columnstore, partitioned by time)
CREATE TABLE dbo.db_metrics (
    collected_at          DATETIME2      NOT NULL,
    host                  NVARCHAR(128) NOT NULL,
    database_name         NVARCHAR(128) NOT NULL,
    active_connections    INT,
    ag_send_queue_kb      BIGINT,
    deadlocks             BIGINT,
    idle_session_count    INT,
    p99_query_latency_ms  NUMERIC(10, 3),
    INDEX CCI_db_metrics CLUSTERED COLUMNSTORE
);

-- Hourly P99 latency baseline per host (maintained by a scheduled aggregate job)
CREATE TABLE dbo.db_metrics_hourly (
    bucket                DATETIME2      NOT NULL,
    host                  NVARCHAR(128) NOT NULL,
    avg_connections       FLOAT,
    p99_latency_ms        NUMERIC(10, 3),
    max_send_queue_kb     BIGINT,
    CONSTRAINT PK_db_metrics_hourly PRIMARY KEY (bucket, host)
);

-- Populate hourly baselines from the columnstore history
INSERT INTO dbo.db_metrics_hourly (bucket, host, avg_connections, p99_latency_ms,
max_send_queue_kb)
SELECT DATEADD(HOUR, DATEDIFF(HOUR, 0, collected_at), 0),
    host,
    AVG(CAST(active_connections AS FLOAT)),
    PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY p99_query_latency_ms),
    MAX(ag_send_queue_kb)
FROM dbo.db_metrics
WHERE collected_at >= DATEADD(HOUR, -1, SYSDATETIME())
GROUP BY DATEADD(HOUR, DATEDIFF(HOUR, 0, collected_at), 0), host;
```

For SQL Server, temporal tables provide system-versioned history of any row-level state change. The autonomous engine uses this to maintain a versioned record of configuration decisions — every time an index is rebuilt, a query plan is forced, or a connection pool limit is adjusted, the change is recorded in a temporal table with a `valid_from` / `valid_to` timestamp pair. This creates a complete audit trail and enables the system to compare "current configuration" against "configuration at last healthy state."

```
-- Temporal table for autonomous configuration change history
CREATE TABLE dbo.AutonomousConfigHistory
(
    config_id          INT          NOT NULL PRIMARY KEY,
    host_name          NVARCHAR(128) NOT NULL,
    database_name       NVARCHAR(128) NOT NULL,
    config_key          NVARCHAR(256) NOT NULL,
    config_value        NVARCHAR(MAX) NOT NULL,
    changed_by          NVARCHAR(128) NOT NULL DEFAULT 'autonomous_engine',
    change_reason       NVARCHAR(512),
    valid_from          DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL,
    valid_to            DATETIME2 GENERATED ALWAYS AS ROW END   NOT NULL,
    PERIOD FOR SYSTEM_TIME (valid_from, valid_to)
)
WITH (SYSTEM_VERSIONING = ON (
    HISTORY_TABLE = dbo.AutonomousConfigHistory_History
));

-- Query: what was the max_worker_threads setting during the last major latency
incident?
SELECT
    config_key,
    config_value,
    valid_from,
    valid_to,
    change_reason
FROM dbo.AutonomousConfigHistory
FOR SYSTEM_TIME BETWEEN '2024-09-15 02:00:00' AND '2024-09-15 04:00:00'
WHERE host_name = 'sqlprod-01'
    AND config_key = 'max_worker_threads'
ORDER BY valid_from;
```

The combination of a columnstore metric-history table and SQL Server temporal tables (for configuration change history) gives the autonomous engine both dimensions of memory it requires: *what was the system doing*, and *what was the system configured to do*. Without both, the engine cannot distinguish between a recurring problem and a newly introduced configuration error.

Closing the Loop: Autonomous Health Scoring and Self-Assessment

All the instrumentation, pipeline resilience, and alert routing described so far feeds into what is, architecturally, the most important component: the **autonomous health score** — a single, composite number that the decision engine uses to determine whether the database is in a state where autonomous action is safe to take.

This is not a vanity dashboard metric. It is the gate that controls whether the autonomous engine operates in **observe mode** (collect data, take no action), **recommend mode** (propose actions for human approval), or **act mode** (execute changes without human sign-off). Getting this wrong in either direction is costly: too conservative and the system never acts; too aggressive and it takes destructive action during a degraded state.

The health score is computed as a weighted sum of normalized sub-scores across five dimensions:

Dimension	Weight	SQL Server Signal
Availability	30%	AG sync state, <code>sys.dm_hadr_availability_replica_states</code>
Replication health	25%	AG log send queue, redo queue, <code>synchronization_health_desc</code>
Resource pressure	20%	Connection utilization, wait stats, CPU pressure, <code>sys.dm_os_ring_buffers</code>
Query performance	15%	P99 from <code>sys.dm_exec_query_stats</code> vs. 7-day baseline
Configuration drift	10%	<code>sys.configurations</code> vs. desired-state manifest

Each sub-score is a value between 0 and 1. A sub-score of 1.0 means that dimension is operating within its autonomous action bounds. A sub-score below 0.5 suppresses autonomous action for the affected class of operations. A composite score below 0.6 puts the engine in recommend mode. Below 0.4 it alerts a human and halts all autonomous action except safety-critical responses (emergency failover, disk pressure mitigation).

The following Python function implements this scoring model against a running SQL Server instance:

```

import pyodbc
from dataclasses import dataclass
from typing import Dict

@dataclass
class HealthScore:
    composite: float
    sub_scores: Dict[str, float]
    autonomous_mode: str # 'act', 'recommend', or 'observe'

def compute_mssql_health_score(conn_string: str, baseline: Dict) -> HealthScore:
    """
    Compute a weighted autonomous health score for a SQL Server instance.
    baseline: dict of 7-day rolling averages {
        'p99_latency_ms': float, 'avg_connections': float,
        'connection_budget': float
    }
    """
    scores = {}

    with pyodbc.connect(conn_string) as conn:
        cur = conn.cursor()

        # --- Availability (30%) ---
        cur.execute("""
            SELECT COUNT(*)
            FROM sys.dm_hadr_availability_replica_states
            WHERE role_desc = 'SECONDARY'
            AND synchronization_health_desc = 'HEALTHY'
        """)
        healthy_replicas = cur.fetchone()[0]
        scores['availability'] = min(1.0, healthy_replicas / 1.0) # Expect >=1

        # --- Replication health (25%) ---
        cur.execute("""
            SELECT COALESCE(MAX(log_send_queue_size + redo_queue_size), 0)
            FROM sys.dm_hadr_database_replica_states
        """)
        lag_kb = cur.fetchone()[0] or 0
        # Score: 1.0 at 0 KB queued, decays linearly to 0 at 10 GB queued
        scores['replication'] = max(0.0, 1.0 - (lag_kb / 10485760.0))

        # --- Resource pressure (20%) ---
        cur.execute("""
            SELECT COUNT(*) FROM sys.dm_exec_sessions
            WHERE is_user_process = 1 AND status = 'running'
        """)
        active = cur.fetchone()[0]
        budget = baseline.get('connection_budget', 500)
        scores['resource_pressure'] = max(0.0, 1.0 - (active / budget))

        # --- Query performance (15%) ---
        cur.execute("""

```

```

        SELECT COALESCE(
            MAX(total_elapsed_time * 1.0 / NULLIF(execution_count, 0)) / 1000.0,
0
        )
    FROM sys.dm_exec_query_stats
    """
    p99 = cur.fetchone()[0] or 0.0
    baseline_p99 = baseline.get('p99_latency_ms', p99 or 1.0)
    ratio = p99 / baseline_p99 if baseline_p99 > 0 else 1.0
    # Score: 1.0 at or below baseline, decays linearly to 0 at 2x baseline
    scores['query_performance'] = max(0.0, min(1.0, 1.0 - (ratio - 1.0)))

    # --- Configuration drift (10%) ---
    cur.execute("""
        SELECT COUNT(*) FROM sys.configurations
        WHERE value <> value_in_use
    """)
    drift_count = cur.fetchone()[0]
    scores['configuration_drift'] = max(0.0, 1.0 - (drift_count / 10.0))

weights = {
    'availability': 0.30,
    'replication': 0.25,
    'resource_pressure': 0.20,
    'query_performance': 0.15,
    'configuration_drift': 0.10,
}
composite = sum(scores[k] * weights[k] for k in weights)

if composite >= 0.6:
    mode = 'act'
elif composite >= 0.4:
    mode = 'recommend'
else:
    mode = 'observe'

return HealthScore(composite=composite, sub_scores=scores, autonomous_mode=mode)

```

The scoring thresholds above (0.6 for act mode, 0.4 for recommend mode) are starting points, not fixed constants — they must be tuned against a specific environment's actual incident history. A team adopting this model should treat the initial thresholds as a hypothesis and validate them against months of historical health-score-versus-incident data before trusting the gate in production.

The health score's real value is not the number itself but what it gates: it is the single decision point that separates "the system is confident enough to act" from "the system should defer to a human." Every autonomous action described elsewhere in this book should, in a mature implementation, check this gate — or an equivalent one — before executing.

Key Takeaways

- → Self-monitoring is not more metrics — it is metrics with structure, timeliness, and self-referential integrity, so a dead collector doesn't leave the autonomous engine reasoning from stale or absent data.
- → Metric pipelines must be architecturally independent of the components they observe: a sidecar-plus-external-aggregator topology ensures a failed primary is still visible to the decision engine, because the signal path doesn't run through the primary itself.
- → Static thresholds degrade as workloads change; dynamic baselines (rolling averages, standard deviation bands) and trend-projection alerts (`predict_linear`) let the system detect both current violations and problems that haven't happened yet.
- → Columnstore metric history and SQL Server temporal tables give the autonomous engine two distinct kinds of memory — what the system was doing, and what it was configured to do — and both are necessary to distinguish a recurring problem from a newly introduced misconfiguration.
- → A composite, weighted health score is the gate that determines whether the system operates in observe, recommend, or act mode; the weights and thresholds must be tuned to the specific environment rather than treated as universal defaults.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 6: Autonomous Query Optimization

Query performance is the most visible surface area of database operations — slow queries surface as customer complaints, SLA breaches, and on-call pages. Yet in most organizations, query tuning remains a manual, reactive, expert-dependent process: a DBA receives an alert, opens a query plan, adds an index, and deploys a fix hours later. Autonomous query optimization replaces this loop entirely. The system continuously observes execution plans, detects regressions and anti-patterns, tests optimization candidates, and deploys changes — without waiting for a human to notice something is wrong. This chapter covers the full architecture of an autonomous query optimization layer: how queries are captured and classified, how the system reasons about plan stability and index opportunities, how candidate optimizations are validated safely before deployment, and how a feedback loop ensures that changes persist only when they demonstrably improve production workloads.

Continuous Query Telemetry: Building the Observation Layer

Autonomous optimization is only as good as its observability. Before any decision can be made, the system needs a continuous, structured feed of query behavior — not sampled logs, not manual EXPLAIN runs, but a persistent, queryable record of every workload pattern the database encounters.

SQL Server provides Query Store as the foundation. It captures query text, plans, and runtime statistics per query, normalizing parameterized variants into a single query entry — so `SELECT * FROM orders WHERE id = 42` and `SELECT * FROM orders WHERE id = 99` collapse into one entry. This normalization is essential for pattern-level analysis. Query Store must be enabled on each database, with capture mode and retention tuned for high-volume systems.

SQL Server ships Query Store — a far richer structure that persists plan history, resource consumption over time, and supports plan forcing directly from T-SQL. It must be enabled per database and configured for appropriate retention.


```

-- Enable Query Store with production-appropriate settings
ALTER DATABASE [YourDatabase] SET QUERY_STORE = ON;
ALTER DATABASE [YourDatabase] SET QUERY_STORE (
    OPERATION_MODE           = READ_WRITE,
    CLEANUP_POLICY           = (STALE_QUERY_THRESHOLD_DAYS = 30),
    DATA_FLUSH_INTERVAL_SECONDS = 60,
    MAX_STORAGE_SIZE_MB      = 4096,
    INTERVAL_LENGTH_MINUTES  = 15,      -- granularity of runtime stats
    SIZE_BASED_CLEANUP_MODE  = AUTO,
    QUERY_CAPTURE_MODE       = AUTO,    -- filters low-impact noise
    MAX_PLANS_PER_QUERY      = 200,
    WAIT_STATS_CAPTURE_MODE  = ON       -- SQL Server 2017+
);

-- Find regressing queries: recent runtime 50%+ worse than historical baseline
SELECT TOP 50
    qt.query_sql_text,
    q.query_id,
    rs_recent.avg_duration / 1000.0 AS recent_avg_ms,
    rs_hist.avg_duration   / 1000.0 AS hist_avg_ms,
    ROUND(
        100.0 * (rs_recent.avg_duration - rs_hist.avg_duration)
        / NULLIF(rs_hist.avg_duration, 0), 2
    ) AS pct_regression,
    rs_recent.count_executions AS recent_executions,
    p_recent.plan_id AS recent_plan_id,
    p_hist.plan_id AS historical_plan_id
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p_recent ON p_recent.query_id = q.query_id
JOIN sys.query_store_runtime_stats rs_recent
    ON rs_recent.plan_id = p_recent.plan_id
JOIN sys.query_store_plan p_hist ON p_hist.query_id = q.query_id
    AND p_hist.plan_id <> p_recent.plan_id
JOIN sys.query_store_runtime_stats rs_hist
    ON rs_hist.plan_id = p_hist.plan_id
WHERE rs_recent.last_execution_time > DATEADD(HOUR, -1, GETUTCDATE())
    AND rs_hist.last_execution_time < DATEADD(HOUR, -2, GETUTCDATE())
    AND rs_hist.avg_duration > 0
    AND rs_recent.avg_duration > rs_hist.avg_duration * 1.5
ORDER BY pct_regression DESC;

```

Beyond these built-in views, the autonomous system needs a time-series telemetry store — not just a point-in-time snapshot. The practical architecture uses a Prometheus scraper that exports Query Store and DMV metrics on a 30-second interval and writes them to a time-series database (Victoria Metrics, Thanos, or Prometheus itself with long retention). A Python collector handles the normalization and emission.

```

import pyodbc
import time
import requests
from datetime import datetime

PUSHGATEWAY_URL = "http://prometheus-pushgateway:9091"
DB_CONN = ("DRIVER={ODBC Driver 18 for SQL Server};"
           "SERVER=sql-prod-01;DATABASE=appdb;UID=monitor;PWD=secret")

def collect_query_stats():
    conn = pyodbc.connect(DB_CONN)
    cur = conn.cursor()
    cur.execute("""
        SELECT TOP 500
            CONVERT(VARCHAR(64), qs.query_hash, 1) AS query_hash,
            LEFT(qt.text, 200) AS query_preview,
            qs.execution_count AS calls,
            qs.total_elapsed_time / 1000.0 / NULLIF(qs.execution_count, 0)
                AS mean_exec_ms,
            qs.total_logical_reads AS logical_reads,
            qs.total_logical_writes AS logical_writes,
            qs.total_worker_time / 1000.0 AS total_cpu_ms
        FROM sys.dm_exec_query_stats qs
        CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
        WHERE qs.execution_count > 10
        ORDER BY mean_exec_ms DESC
    """)
    rows = cur.fetchall()
    cols = [d[0] for d in cur.description]
    cur.close()
    conn.close()

    metrics = []
    ts = int(time.time() * 1000)

    for row in rows:
        record = dict(zip(cols, row))
        qid = record["query_hash"]
        label = f'query_hash="{qid}"'
        for metric in ["mean_exec_ms", "calls",
                      "logical_reads", "logical_writes", "total_cpu_ms"]:
            val = record.get(metric) or 0
            metrics.append(
                f'mssql_query_{metric}{{{label}}} {val} {ts}'
            )

    payload = "\n".join(metrics) + "\n"
    resp = requests.post(
        f"{PUSHGATEWAY_URL}/metrics/job/query_telemetry",
        data=payload,
        headers={"Content-Type": "text/plain"}
    )
    resp.raise_for_status()

```

```

print(f"[{datetime.utcnow().isoformat()}] Pushed {len(rows)} query records")

if __name__ == "__main__":
    while True:
        try:
            collect_query_stats()
        except Exception as e:
            print(f"Error: {e}")
            time.sleep(30)

```

With this telemetry layer operational, the system has the raw material it needs to make decisions. The next step is classifying what kind of problem each query signature represents.

Plan Regression Detection and Automatic Plan Stabilization

Plan regression is one of the most insidious database failure modes. A query that ran in 12 ms for months suddenly takes 4.2 seconds — not because the query changed, but because a statistics update, a new index, or an automatic statistics update shifted the optimizer's cost estimates enough to select a different join strategy or access path. In a manually operated system, this surfaces as a P2 incident. In an autonomous system, it is detected within one collection interval and remediated before most users notice.

The detection model is straightforward: for each normalized query ID, maintain a rolling baseline of `mean_exec_time` and `stddev_exec_time`. When the observed mean exceeds `baseline_mean + N * baseline_stddev` (where N is a tunable sensitivity, typically 3), flag the query as a regression candidate. But raw statistical deviation is not sufficient — call volume matters. A query called twice is not a meaningful regression signal. The autonomous rule is:

> When `calls_in_window >= 20` AND `mean_exec_time > baseline_mean + 3 * baseline_stddev` AND the trend is sustained for at least two consecutive collection intervals, classify the query as a confirmed regression and trigger the remediation pipeline.

SQL Server remediation is the cleaner case because Query Store supports plan forcing — pinning a query to a known-good historical plan with a single T-SQL statement.

```

-- Autonomous plan stabilization: force the last known-good plan
-- Called by the remediation engine when a regression is confirmed

DECLARE @query_id    BIGINT = 1042;          -- detected regressing query
DECLARE @good_plan_id BIGINT;

-- Find the plan with lowest avg_duration in the prior stable window
SELECT TOP 1
    @good_plan_id = p.plan_id
FROM sys.query_store_plan p
JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
WHERE p.query_id = @query_id
    AND rs.last_execution_time BETWEEN
        DATEADD(DAY, -7, GETUTCDATE()) AND
        DATEADD(HOUR, -2, GETUTCDATE()) -- exclude the regression window
ORDER BY rs.avg_duration ASC;

-- Force the good plan
IF @good_plan_id IS NOT NULL
BEGIN
    EXEC sys.sp_query_store_force_plan
        @query_id    = @query_id,
        @plan_id     = @good_plan_id;

    -- Log the action for the autonomous audit trail
    INSERT INTO dba_autonomous_actions
        (action_type, target_id, target_plan_id, reason, acted_at)
    VALUES
        ('PLAN_FORCE', @query_id, @good_plan_id,
        'Regression detected: mean_exec_time exceeded 3-sigma baseline',
        GETUTCDATE());
END

```

SQL Server provides native plan forcing through Query Store (`sp_query_store_force_plan`) and plan guides. For the autonomous layer, Query Store plan forcing is the practical choice: the system identifies the last known good plan from Query Store history and forces it without touching application code — the same mechanism behind Automatic Tuning's automatic plan correction.

For SQL Server, once a plan is forced, the autonomous system must monitor whether the forced plan is still the correct choice after conditions change — for example, after a major data load that legitimately shifts the optimal join order. The system schedules a re-evaluation every 24 hours: it temporarily unforces the plan, observes runtime for one collection window, and forces again if the regression persists, or permanently unforces if the new plan is now stable and comparable in performance. This feedback loop prevents the system from accumulating stale plan forces indefinitely.

Autonomous Index Management: Discovery, Validation, and Lifecycle

Indexes are the highest-leverage knob in query optimization, and they are also the easiest source of configuration drift — redundant indexes consuming write overhead, missing indexes causing sequential scans at scale, and bloated indexes that were once useful but now serve dead workloads. An autonomous index management system handles all three dimensions continuously.

Discovery starts with execution plan analysis. SQL Server exposes whether a query performed a full table scan in a context where an index could have served it through `sys.dm_db_index_usage_stats` (user_scans against heaps and clustered indexes), actual execution plans, and `sys.dm_db_missing_index_details`.

```
-- Surface high-impact missing index recommendations from DMVs
-- The autonomous engine queries this on every 15-minute cycle
SELECT TOP 30
    ROUND(migs.avg_total_user_cost
        * migs.avg_user_impact
        * (migs.user_seeks + migs.user_scans), 0) AS estimated_improvement_score,
    migs.user_seeks,
    migs.user_scans,
    ROUND(migs.avg_user_impact, 1) AS avg_pct_benefit,
    mid.statement AS table_name,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    -- Construct the index DDL automatically
    'CREATE INDEX [ADE_IDX_' +
        REPLACE(REPLACE(mid.statement, '[', ''), ']', '') + '_' +
        CAST(migs.group_handle AS VARCHAR) + ']'
    + ' ON ' + mid.statement
    + ' (' + ISNULL(mid.equality_columns, '')
    + CASE WHEN mid.equality_columns IS NOT NULL
        AND mid.inequality_columns IS NOT NULL THEN ', ' ELSE '' END
    + ISNULL(mid.inequality_columns, '') + ')'
    + ISNULL(' INCLUDE (' + mid.included_columns + '), ''')
    + ' WITH (ONLINE = ON, FILLFACTOR = 90)' AS proposed_ddl,
    migs.last_user_seek
FROM sys.dm_db_missing_index_groups mig
JOIN sys.dm_db_missing_index_group_stats migs ON migs.group_handle =
mig.index_group_handle
JOIN sys.dm_db_missing_index_details mid ON mid.index_handle =
mig.index_handle
WHERE migs.avg_total_user_cost * migs.avg_user_impact
    * (migs.user_seeks + migs.user_scans) > 10000 -- impact threshold
    AND migs.last_user_seek > DATEADD(HOUR, -4,
GETUTCDATE())
ORDER BY estimated_improvement_score DESC;
```

For SQL Server, the autonomous system uses a combination of `sys.dm_db_index_usage_stats`, `sys.dm_db_missing_index_details`, and hypothetical index analysis via `AUTOPILOT`.

`AUTOPILOT` lets the optimizer reason about an index that does not yet exist — the system defines a hypothetical index, requests the estimated plan (the query is never executed), checks whether the optimizer would use it, then rolls back the hypothesis without any write to disk.

Validation before deployment is non-negotiable. The autonomous system must not create indexes on production tables without a safety gate. The gate checks:

- → Table size: if the table exceeds a configured threshold (e.g., 100 GB), the index build is scheduled during the maintenance window with `WITH (ONLINE = ON)` in SQL Server.
- → Write amplification: the system estimates the additional write cost by querying DML rates from `sys.dm_db_index_operational_stats`. If the table sustains more than a configured number of inserts/updates per second, the estimated overhead is computed against available I/O headroom.
- → Redundancy check: before any new index is created, the engine checks whether an existing index already covers the proposed columns, possibly with a superset of keys.

Lifecycle management closes the loop. An index that was useful during a past workload pattern becomes a pure write tax when that pattern disappears. The autonomous system tracks `user_seeks` and `user_scans` counts from `sys.dm_db_index_usage_stats`. When an index has recorded zero scans over a rolling 30-day window and the database has been active throughout that window, it is classified as a dead index and queued for removal — first disabled (SQL Server supports disabling without dropping), then dropped in the next maintenance window after a configurable hold period.

Workload-Aware Statistics Management and Cardinality Feedback

The SQL Server query optimizer is cost-based — it relies on statistics that describe the distribution of values in each column to estimate the number of rows a predicate will match. When those statistics are stale or insufficiently granular for the actual workload, the planner produces bad cardinality estimates, which cascade into bad plan choices. Autonomous statistics management means the system continuously monitors cardinality error rates and triggers targeted statistics updates rather than waiting for the auto-update statistics threshold to fire.

The core signal is the **cardinality error ratio**: the ratio of estimated rows in the plan vs. actual rows returned at runtime. SQL Server exposes this after execution through Query Store's estimated vs. actual row counts and the `last_rows` / `avg_rows` columns in `sys.dm_exec_query_stats`.

When cardinality error ratio exceeds a threshold (typically > 10x under-estimate or > 5x over-estimate on a high-frequency query), the autonomous system triggers targeted statistics refresh:

```
-- Targeted statistics update with full scan for high-variance columns
-- SQL Server by default samples; FULLSCAN is expensive but accurate

UPDATE STATISTICS [dbo].[orders] [stat_orders_customer_segment]
WITH FULLSCAN, NORECOMPUTE;  -- NORECOMPUTE: we manage the schedule autonomously

-- For new multi-column statistics where none exist
IF NOT EXISTS (
    SELECT 1 FROM sys.stats s
    JOIN sys.stats_columns sc ON sc.object_id = s.object_id AND sc.stats_id =
s.stats_id
    JOIN sys.columns c ON c.object_id = sc.object_id AND c.column_id = sc.column_id
    WHERE s.object_id = OBJECT_ID('dbo.orders')
    AND c.name = 'customer_segment'
    AND (SELECT COUNT(*) FROM sys.stats_columns sc2 WHERE sc2.stats_id =
s.stats_id) > 1
)
BEGIN
    CREATE STATISTICS [stat_orders_status_segment]
    ON [dbo].[orders] ([status], [customer_segment])
    WITH FULLSCAN;
END

-- Record action
INSERT INTO dba_autonomous_actions
    (action_type, object_name, reason, acted_at)
VALUES (
    'STATISTICS_FULLSCAN',
    'dbo.orders(status, customer_segment)',
    'Cardinality error ratio 15x detected on QueryID 1042, plan switched to nested
loop',
    GETUTCDATE()
);
```

Beyond reactive statistics refresh, the autonomous system must also manage statistics for **skewed distributions** — columns where a small number of values dominate (e.g., an `order_status` column where 95% of rows have status `completed`). Standard histogram statistics handle these poorly. The pattern is to identify high-skew columns from `sys.dm_db_stats_histogram` (SQL Server 2016+), and increase the statistics target or create filtered indexes that serve the minority predicate values where skew causes plan failures.

The Autonomous Validation and Rollback Feedback Loop

None of the remediation actions described above — plan forcing, hint injection, index creation, statistics refresh — are safe to leave unverified. An autonomous optimization system is not a fire-and-forget script; it is a closed loop that measures whether its own actions produced the intended effect, and reverses them when they did not.

The validation pattern is consistent across action types:

1. **Record a pre-action baseline.** Before executing any change, snapshot the current `mean_exec_time`, `calls`, and I/O counters for the affected query or queries, alongside the exact action being taken and its rationale.
2. **Apply the change.** Execute the plan force, index creation, hint injection, or statistics refresh, and log it in the `dba_autonomous_actions` audit table with a unique action ID.
3. **Observe the post-action window.** Wait for a defined evaluation window (typically two to three collection intervals) to accumulate enough calls to be statistically meaningful, then re-measure the same metrics.
4. **Compare and decide.** If the post-action metrics show sustained improvement relative to the pre-action baseline, the action is marked confirmed and left in place. If the metrics are unchanged or worse, the action is automatically reverted — the index is dropped, the plan is unforced, the hint is removed, or the statistics target is rolled back — and the audit record is updated with the outcome.

```
graph TD
    A[Record pre-action baseline  
mean_exec_time, calls, I/O] --> B[Apply change  
plan force / index / hint / stats refresh]
    B --> C[Observe post-action window  
2-3 collection intervals]
    C --> D{Sustained improvement  
vs. baseline?}
    D -->|Yes| E[Mark confirmed, leave in place]
    D -->|No / worse| F[Auto-revert: drop index,  
unforce plan, remove hint, roll back stats]
```

The autonomous validation and rollback loop applied consistently across every remediation type in this chapter.

This loop is what prevents an autonomous optimizer from accumulating stale or counterproductive changes over time. An index created in response to a transient reporting workload that never recurs, or a plan force that was correct at the time but has since become suboptimal as data distributions shifted, both get cleaned up automatically rather than persisting as silent technical debt that only a human audit would eventually catch.

The evaluation window and improvement threshold are the two parameters that most determine how conservative or aggressive the loop behaves. A short window with a low improvement bar creates noisy, thrashing behavior — the system reverting and reapplying changes as normal variance crosses the threshold. A long window with a high bar makes the system slow to correct clearly bad decisions. Tuning these parameters against the specific workload's variance characteristics, rather than adopting a fixed industry default, is part of operating this system responsibly.

Key Takeaways

- → Autonomous query optimization depends on continuous, structured telemetry — Query Store and `sys.dm_exec_query_stats` — where Query Store provides native time windows through its runtime stats intervals, while DMV-based collectors need external snapshot-and-diff tooling to compute windowed rates from cumulative counters.
- → Plan regression detection should combine statistical deviation (mean exceeding a multiple of baseline standard deviation) with a minimum call-volume threshold, so the system does not react to noise from low-frequency queries.
- → SQL Server's Query Store supports native plan forcing via `sp_query_store_force_plan`, and Automatic Tuning can apply it autonomously — giving the autonomous layer an engine-native plan-pinning mechanism.
- → Autonomous index management must validate candidates with a hypothetical-index tool (missing-index DMVs and `AUTOPILOT` hypothetical indexes on SQL Server) before committing to a real `CREATE INDEX`, and must track index usage over time to retire indexes that no longer serve live workloads.
- → Every autonomous optimization action needs a validation and rollback loop: a recorded pre-action baseline, a defined observation window, and an automatic reversal path when the action does not produce the expected improvement — without this loop, the system accumulates unverified changes as technical debt.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 7: Self-Indexing and Schema Adaptation

Schema and index management represent two of the most labor-intensive responsibilities in traditional database operations — and two of the highest-leverage targets for autonomous systems. A DBA who manually reviews missing-index DMVs or watches

`sys.dm_db_index_usage_stats` for unused indexes and `sys.dm_db_index_physical_stats` for fragmentation is performing work that a well-designed autonomous layer can execute continuously, with finer granularity and faster response time than any human rotation schedule allows. This chapter builds the full architecture: how autonomous systems observe workload patterns, infer optimal index structures, apply changes safely under live traffic, and adapt the physical schema as access patterns evolve — without waiting for a human to file a ticket.

```
graph LR
    A[Observe  
Query Store / missing-index DMVs] --> B[Score & Recommend  
confidence + risk tier]
    B --> C[Governance Gate]
    C -->|score >= 0.70| D[Auto-apply  
ONLINE=ON]
    C -->|0.40-0.70| H[Human review]
    H -->|approved| D
    D --> F[Monitor usage stats]
    F -->|zero scans 14-21+ days| G[Disable / Retire / Drop]
```

The full self-indexing lifecycle this chapter builds: observation, scored recommendation, governance-gated apply, and usage-driven retirement.

7.1 Workload Observation: Building the Signal Pipeline for Index Intelligence

Autonomous index management lives or dies on the quality of its input signal. Before any decision logic can fire, the system must continuously capture what queries are running, how often, at what cost, and which predicates drive the most scan overhead. This is not a once-a-day snapshot job — it is a streaming observation pipeline.

On SQL Server, Query Store is the foundation. It records query text, plans, execution count, duration, logical reads, and estimated vs. actual rows per plan. However, raw Query Store data is insufficient on its own for index inference. For index inference, you need predicate-level visibility — which columns appear in `WHERE`, `JOIN ON`, `ORDER BY`, and `GROUP BY` clauses — and that requires parsing query text or capturing actual execution plans via Extended Events with the query plan profile action enabled.

```
-- Query the plan cache DMVs for sequential scan and high-cost query signals
SELECT TOP 50
    qs.total_elapsed_time / qs.execution_count          AS avg_elapsed_us,
    qs.total_logical_reads / qs.execution_count         AS avg_logical_reads,
    qs.execution_count,
    qs.total_elapsed_time,
    SUBSTRING(qt.text, (qs.statement_start_offset/2)+1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(qt.text)
            ELSE qs.statement_end_offset END
            - qs.statement_start_offset)/2)+1)          AS statement_text,
    qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle)        AS qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle)     AS qp
WHERE
    qs.execution_count > 50
    AND qs.total_logical_reads / qs.execution_count > 1000
ORDER BY avg_logical_reads DESC;
```

On SQL Server, the Missing Index DMVs provide a pre-computed signal layer that should be consumed directly. `sys.dm_db_missing_index_details`, `sys.dm_db_missing_index_groups`, and `sys.dm_db_missing_index_group_stats` together give you column-level index recommendations weighted by estimated impact. The critical discipline is interpreting these signals correctly rather than naively applying everything the engine suggests — the DMVs do not account for write amplification, duplicate near-miss indexes, or the cost of maintaining a large index catalog on write-heavy tables.

The observation pipeline feeds a structured fact store — a dedicated schema, typically in a monitoring database or a columnstore staging database, that retains:

- → Query fingerprint (normalized hash)
- → Predicate columns and their selectivity estimates
- → Execution frequency per hour and per day
- → Estimated and actual row counts (to catch plan regressions)
- → Current plan type (index scan, bitmap scan, seq scan, nested loop, hash join)
- → Block I/O cost

This fact store is the working memory of your autonomous index advisor. Without retention and trend analysis, the system cannot distinguish a transient spike from a structural workload shift — and those two cases demand completely different responses.

7.2 Index Recommendation Logic: From Raw Signal to Actionable Proposals

A signal pipeline produces observations. A recommendation engine converts observations into specific, prioritized index proposals with confidence scores and risk assessments. This is where most autonomous index systems either add real value or generate noise.

The recommendation logic must enforce several hard constraints before surfacing any proposal:

1. **Duplicate suppression:** If an index on `(customer_id, created_at)` already exists, do not propose `(customer_id)` as a separate index unless cardinality analysis justifies it.
2. **Write-cost awareness:** On a table with an insert rate above a threshold (say, 5,000 rows/second), each additional index adds measurable write overhead. The scoring function must penalize proposals on high-write tables.
3. **Partial index preference:** If the workload filter consistently includes `WHERE status = 'active'` and active rows represent less than 15% of the table, a partial index almost always outperforms a full-column index. The recommendation engine should detect this pattern automatically.
4. **Covering index consolidation:** Multiple proposals touching overlapping column sets on the same table should be consolidated into a single covering index candidate before surfacing to the action layer.

The scoring function used in production at several platform teams looks roughly like this:

```

import hashlib
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class IndexProposal:
    table_name: str
    columns: list[str]
    include_columns: list[str] = field(default_factory=list)
    partial_predicate: Optional[str] = None
    confidence_score: float = 0.0
    estimated_benefit_ms: float = 0.0
    estimated_write_overhead_pct: float = 0.0
    risk_tier: str = "low" # low | medium | high

def score_index_proposal(
    proposal: IndexProposal,
    avg_query_time_ms: float,
    daily_executions: int,
    table_write_rate_per_sec: float,
    existing_index_count: int
) -> float:
    """
    Scoring model: benefit weighted against write cost and catalog overhead.
    Scores above 0.70 are auto-apply candidates.
    Scores 0.40–0.70 require async human review before apply.
    Scores below 0.40 are suppressed.
    """

    # Estimated time saved per day (normalized to 0-1 range, cap at 10min)
    time_saved_daily = (avg_query_time_ms * 0.65 * daily_executions) / 1000
    benefit_score = min(time_saved_daily / 600_000, 1.0)

    # Write overhead penalty: each index on a hot-write table is expensive
    write_penalty = min(table_write_rate_per_sec / 10_000, 0.5)

    # Catalog bloat penalty: diminishing returns after 10 indexes
    catalog_penalty = min(max(existing_index_count - 10, 0) * 0.03, 0.30)

    # Partial index bonus
    partial_bonus = 0.15 if proposal.partial_predicate else 0.0

    raw_score = benefit_score - write_penalty - catalog_penalty + partial_bonus
    proposal.confidence_score = round(max(0.0, min(raw_score, 1.0)), 4)

    if proposal.confidence_score >= 0.70:
        proposal.risk_tier = "low"
    elif proposal.confidence_score >= 0.40:
        proposal.risk_tier = "medium"
    else:
        proposal.risk_tier = "high"

    return proposal.confidence_score

```

SQL Server has no supported, documented mechanism for testing a hypothetical index against the optimizer without actually building it (the undocumented AUTOPILOT commands from Chapter 6 are not production-safe). The Database Engine Tuning Advisor performs a related workload analysis internally, but it is a separate tool, not an API the autonomous layer calls inline. In practice, teams get most of the same signal from the missing-index DMVs' pre-computed cost and impact estimates (`sys.dm_db_missing_index_group_stats`), combined with creating updated column statistics to sharpen cardinality estimates before deciding whether a proposal is worth a real, online-built index. Creating a statistics object alone does not make the optimizer generate a plan against an index that doesn't exist — it only improves the estimates the optimizer uses for the indexes and access paths that are actually present. Genuine validation of a candidate index on SQL Server means building it, typically first in a staging environment or during a low-traffic window, and measuring the real effect.

```
-- Refresh statistics to sharpen cardinality estimates before deciding
-- whether a proposed index is worth building for real
UPDATE STATISTICS dbo.orders WITH FULLSCAN;

-- Baseline the current plan's cost before committing to an index build
SET STATISTICS IO ON;
SELECT order_id, order_total
FROM dbo.orders
WHERE customer_id = 10042
    AND created_at >= '2024-01-01'
ORDER BY created_at DESC;
SET STATISTICS IO OFF;
```

The recommendation engine should also integrate with the LLM layer introduced in Book 2's AIOps patterns. When a proposal's confidence score falls in the 0.40–0.70 range, routing the query workload context, current table statistics, and the proposal details to an OpenAI or Claude API call can generate a human-readable risk summary and a recommendation to approve or defer — without requiring a DBA to perform the analysis manually. The autonomous system isn't guessing blindly; it's delegating ambiguous cases to a reasoning layer that produces auditable output.

7.3 Zero-Downtime Index Application and the Concurrent Build Safety Net

Generating a good index recommendation is straightforward relative to applying it safely under production load. Index creation on large tables is a blocking operation in its default form — and on a system processing thousands of transactions per second, even a one-second lock on a critical table triggers cascading timeouts. The autonomous apply layer must enforce non-blocking build patterns as an absolute invariant.

SQL Server's `CREATE INDEX ... WITH (ONLINE = ON)` is the mechanism. It builds the index while holding only brief locks at the start and end of the operation, allowing reads and writes to continue throughout. The cost is higher resource consumption, and on Standard Edition online index operations are unavailable — the autonomous apply layer checks edition support and falls back to an offline build inside a maintenance window when online is not available.

```
-- Non-blocking index creation with ONLINE = ON
-- Requires Enterprise or Developer edition; validate edition before attempting

CREATE INDEX idx_orders_customer_created
ON dbo.orders (customer_id, created_at DESC)
INCLUDE (order_total, status)
WHERE status = 'active'
WITH (
    ONLINE = ON,           -- non-blocking, allows concurrent DML
    SORT_IN_TEMPDB = ON,   -- reduces contention on the primary filegroup
    DATA_COMPRESSION = ROW, -- optional: compress the index pages
    FILLFACTOR = 85,       -- leave 15% free for future inserts
    MAXDOP = 4             -- cap parallelism to limit system impact
);
```

The autonomous apply layer in a production system adds several safety controls around these primitives:

Maintenance window gating: Even with concurrent/online builds, large index operations consume I/O bandwidth. The orchestration layer checks a configurable maintenance schedule before starting any build — or consults a real-time load signal from Prometheus to determine if the current CPU and I/O utilization permits a background operation.

Rollback via drop: If a build completes but query performance does not improve (measured by comparing the rolling average execution time of affected queries before and after), the system drops the index automatically. This is the index equivalent of a feature flag rollback — and it requires retaining the pre-build query timing baseline in the fact store.

Locking circuit breaker: The autonomous apply script must monitor `sys.dm_exec_requests` before and during the build for blocked sessions accumulating against the target table. If the blocker count exceeds a threshold — typically three or more sessions blocked for more than five seconds — the build is aborted and rescheduled.

7.4 Autonomous Index Retirement: Detecting and Removing the Dead Weight

Every team builds indexes they later forget about. The autonomous system that only adds indexes without retiring unused ones eventually produces the opposite of its intended effect: a swollen index catalog that adds write overhead on every INSERT, UPDATE, and DELETE with no corresponding read benefit. Index retirement is not a nice-to-have — it is a necessary counterbalance to the recommendation engine.

Unused index detection relies on usage statistics reset cycles. SQL Server accumulates index usage counts in `sys.dm_db_index_usage_stats`, which resets on service restart. For reliable detection, usage data must be persisted externally across restarts — another function of the observation fact store.


```

-- Identify unused indexes with write cost visibility
-- Note: usage stats are per-instance, reset on SQL Server restart
SELECT
    OBJECT_SCHEMA_NAME(i.object_id)          AS schema_name,
    OBJECT_NAME(i.object_id)                 AS table_name,
    i.name                                   AS index_name,
    i.type_desc,
    ISNULL(s.user_seeks, 0)                  AS user_seeks,
    ISNULL(s.user_scans, 0)                  AS user_scans,
    ISNULL(s.user_lookups, 0)                AS user_lookups,
    ISNULL(s.user_updates, 0)                AS user_updates,
    p.rows                                   AS table_rows,
    (SUM(a.total_pages) * 8) / 1024          AS index_size_mb
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats s
    ON s.object_id = i.object_id
    AND s.index_id = i.index_id
    AND s.database_id = DB_ID()
JOIN sys.partitions p
    ON p.object_id = i.object_id
    AND p.index_id = i.index_id
JOIN sys.allocation_units a
    ON a.container_id = p.partition_id
WHERE
    i.type_desc <> 'HEAP'
    AND ISNULL(s.user_seeks, 0) = 0
    AND ISNULL(s.user_scans, 0) = 0
    AND ISNULL(s.user_lookups, 0) = 0
    AND ISNULL(s.user_updates, 0) > 100      -- being maintained but never read
    AND i.is_primary_key = 0
    AND i.is_unique_constraint = 0          -- never retire PKs or unique
constraints
GROUP BY
    i.object_id, i.name, i.type_desc, s.user_seeks, s.user_scans,
    s.user_lookups, s.user_updates, p.rows
ORDER BY index_size_mb DESC;

```

The retirement pipeline applies graduated action rather than immediate deletion. The standard state machine is:

1. **Observation:** Index has zero reads for 14+ consecutive days across at least three stat-reset cycles (confirmed via the external fact store).
2. **Disable** (SQL Server only): `ALTER INDEX idx_name ON table DISABLE` — a reversible operation that drops the index pages but retains the metadata. If workload changes cause a performance regression within 72 hours, re-enable without a full rebuild.

3. **Index disable testing:** SQL Server supports `ALTER INDEX ... DISABLE`, which keeps the index definition while making it invisible to the optimizer. Disable the candidate and monitor for 24 hours — if no query regresses, proceed to DROP; if something breaks, `REBUILD` brings it back instantly. This is the safest production validation path for index removal.

4. **Drop:** Issue the DROP after confirming no blocking sessions and that the observation hold period has elapsed.

5. **Post-drop monitoring:** Retain a suppression record in the fact store so the recommendation engine does not immediately propose recreating the same index without evidence of workload change.

7.5 Schema Adaptation Pipelines: Aligning Physical Structure to Evolving Access Patterns

Self-indexing is one layer of schema adaptation. The deeper layer — modifying column definitions, partition boundaries, table structures, and data types in response to workload evolution — is where autonomous systems move from reactive optimization to proactive design. This is still an emerging practice, but the building blocks are well-defined.

Partition boundary adaptation is the most mature use case. On a time-series table partitioned by month, if the system detects that 80% of queries land on the last two partitions while earlier partitions are accessed only by archival processes, the schema adaptation layer should:

- → Verify the partition access distribution using execution plan analysis
- → Compress or archive cold partitions automatically (SQL Server partition switching to a compressed filegroup or archive table)
- → Pre-create future partitions on a rolling schedule without DBA intervention

Column-level type evolution is riskier but actionable. When the observation layer detects consistent implicit casts in query plans — for example, a `WHERE user_id = $1` where `user_id` is `VARCHAR(36)` storing UUIDs and `$1` is always bound as a proper UUID — the system can flag a type migration proposal. The actual migration follows the expand/contract pattern:

1. Add `user_id_new` `UUID` column with a default and a trigger that dual-writes.
2. Backfill in small batches using an autonomous migration runner.
3. Update the application's index bindings to reference the new column.
4. Swap column names (or update application queries) and drop the old column.

This pattern is implemented in expand/contract migration frameworks and integrated into CI/CD pipelines, but the autonomous layer can initiate the sequence when it detects a persistent type mismatch penalty in execution plans — without waiting for a developer to file a migration ticket.

The schema adaptation pipeline also governs **statistics freshness**. Stale statistics produce incorrect cardinality estimates, which produce bad plans, which look to the observation layer like unindexed queries — generating false positive index proposals. The autonomous system avoids this by monitoring the statistics modification counter and update date via `sys.stats` and `STATS_DATE()`, triggering `UPDATE STATISTICS` when modification thresholds are breached rather than relying on the auto-update schedule alone.

7.6 Governance, Auditability, and the Human Override Layer

Autonomous systems that act without any governance trail are operationally dangerous regardless of how well the decision logic is tuned. At the ADE level, every index created, modified, or dropped by the autonomous layer must be logged with full context: the signal that triggered the proposal, the scoring rationale, the time of application, the pre- and post-apply performance delta, and the identity of the executing service account. This is not optional overhead — it is the mechanism that lets teams trust the system and the evidence that justifies expanding its operating envelope over time.

The governance layer also defines the **authority tiers** that determine when the system acts versus when it surfaces a recommendation for human approval:

Action	Condition for Auto-Apply	Escalation Path
---	---	---
Create non-unique index	Score ≥ 0.70 , table not in freeze list, maintenance window active	Slack/webhook alert with approve/deny button
Drop unused index	Zero reads for 21+ days, not PK or unique constraint	Ticket created in Jira/Linear with 48h approval window
Modify partition boundary	No active transactions spanning boundary	Requires manual approval regardless of score
Column type migration	Expand/contract phase 1 only (additive)	Phase 3 (removal) always requires human sign-off

Human override is implemented at every tier of this table, not just as an emergency stop. Any table can be placed on a freeze list that suppresses autonomous DDL entirely, regardless of proposal score — a necessary control during migrations, major releases, or incident

response, when an autonomous index build is the last thing an on-call engineer wants competing for I/O. The freeze list, the scoring thresholds, and the escalation paths are themselves configuration, versioned alongside the rest of the autonomous system, so a change to the safety posture is reviewable the same way a change to application code is reviewable.

The broader principle underlying this chapter is that self-indexing and schema adaptation are not about removing humans from the loop — they are about moving humans from the execution path to the policy-design path. The DBA who once ran `CREATE INDEX` manually now defines the scoring thresholds, the freeze conditions, and the escalation rules that govern when the system may act on its own. That is a higher-leverage use of expertise, and it is also the only way index and schema management can keep pace with workloads that change faster than any manual review cadence could track.

Key Takeaways

- → Autonomous index management requires a persistent observation fact store — query fingerprints, predicate columns, execution frequency, and plan type over time — because a single point-in-time snapshot cannot distinguish a transient spike from a structural workload shift.
- → A sound recommendation engine enforces hard constraints before surfacing a proposal: duplicate suppression, write-cost penalties on high-insert tables, partial-index preference for low-selectivity predicates, and consolidation of overlapping column sets into covering indexes.
- → Index creation must default to non-blocking mechanisms — `WITH (ONLINE = ON)` on SQL Server — with automated validity checks, a locking circuit breaker, and a rollback-via-drop path when a build does not improve the queries it targeted.
- → Index retirement is a necessary counterbalance to index creation: usage statistics reset on service restart, so reliable dead-index detection requires persisting usage data externally across multiple observation windows before acting.
- → Schema adaptation — partition pre-creation, cold-partition archival, and expand/contract type migrations — extends the same observe-score-apply-verify discipline beyond indexes, but destructive phases (column removal, boundary changes affecting in-flight transactions) should remain human-approved regardless of confidence score.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 8: Autonomous Execution Plan Management

Query execution plans are the contracts between your application's intent and the database engine's physical execution strategy. When those contracts degrade — through statistics drift, parameter sensitivity, schema evolution, or optimizer regressions introduced by version upgrades — performance collapses in ways that are rarely caught by traditional monitoring until users are already impacted. An autonomous execution plan management system detects plan instability before it manifests as user-visible latency, evaluates candidate plans against production workloads, promotes or pins the best-performing plan, and rolls back regressions — all without a DBA touching a keyboard. This chapter constructs that system in full, covering the detection infrastructure, the decision engine, the execution and feedback loop, and the production patterns that make it safe to run unsupervised in regulated, high-traffic environments.

8.1 Anatomy of Plan Instability: What the Autonomous System Must Observe

Before automation can act, it must observe the right signals. Plan instability does not always announce itself as a slow query. More often it presents as variance — the same query running in 8ms one minute and 800ms the next. The autonomous system must distinguish between three categories of instability that require different responses.

Regression is the simplest case: a plan that was stable is replaced by a worse plan. This happens when statistics are stale, when a new index is created and the optimizer picks it incorrectly, or when a minor schema change shifts cardinality estimates past a threshold that flips a join strategy from nested loop to hash join with the wrong side as the build input.

Parameter sensitivity (parameter sniffing) describes a plan compiled for one parameter value that performs poorly for a different distribution. A stored procedure compiled on a date range of one day serves a query asking for three months of data with the same nested loop plan — and collapses.

Optimizer regression occurs during version upgrades. SQL Server cumulative updates regularly adjust cardinality estimator behavior. A plan that was correct under the prior optimizer model becomes wrong under the new one without any data or schema change.

The observability infrastructure must capture plan identifiers, execution statistics, and plan text at a granularity that makes these three categories distinguishable.

```
-- Identify plan instability via execution count variance and plan count per query
hash
SELECT
    qs.query_hash,
    COUNT(DISTINCT qs.plan_handle) AS plan_variants,
    SUM(qs.execution_count) AS total_executions,
    AVG(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)) AS avg_elapsed_us,
    MAX(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)) AS max_elapsed_us,
    MIN(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)) AS min_elapsed_us,
    -- Coefficient of variation proxy using max/avg ratio
    MAX(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)) /
        NULLIF(AVG(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)), 0) AS
volatility_ratio
FROM sys.dm_exec_query_stats qs
GROUP BY qs.query_hash
HAVING COUNT(DISTINCT qs.plan_handle) > 1
    OR MAX(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)) /
        NULLIF(AVG(qs.total_elapsed_time / NULLIF(qs.execution_count, 0)), 0) > 3.0
ORDER BY avg_elapsed_us DESC;
```

The coefficient of variation (stddev/mean) and the volatility ratio (max/avg elapsed) are your primary instability signals. A CV above 1.5 or a volatility ratio above 3.0 flags a query as a candidate for automated plan investigation. These thresholds are calibrated starting points — your production system will tune them over time based on false-positive rates in your workload.

For SQL Server, the Query Store provides the full plan-per-query-hash history that makes plan-level analysis native — no extensions or custom collection tables are required.

The autonomous collector runs this detection query on a configurable interval — typically every five minutes for OLTP systems — and writes results to a plan instability log table. This table is the input queue for the decision engine described in section 8.3.

8.2 Plan Baseline Architecture: Building the Ground Truth Store

An autonomous plan management system needs a reference point — a "known good" state to measure against and to roll back to. This is the plan baseline. The architecture of your baseline store determines what the automation can do: a shallow baseline supports only regression detection, while a deep baseline with execution context supports full plan pinning, forced plan rotation, and A/B evaluation.

The autonomous baseline store is a thin management layer on top of Query Store — the forcing mechanism itself is native, so the overlay only needs to track which plans were forced, when, why, and what the performance outcome was.

Baseline store built on Query Store forced plans:

The overlay below is the only custom schema required:

```
-- Autonomous plan management overlay for Query Store
CREATE TABLE dbo.ade_plan_baselines (
    baseline_id      INT IDENTITY(1,1) PRIMARY KEY,
    query_id         BIGINT NOT NULL,      -- sys.query_store_query.query_id
    plan_id          BIGINT NOT NULL,      -- sys.query_store_plan.plan_id
    query_hash       BINARY(8),
    query_sql_text   NVARCHAR(MAX),
    avg_duration_us  BIGINT,              -- when baseline was captured
    captured_at      DATETIME2 DEFAULT SYSUTCDATETIME(),
    last_validated_at DATETIME2,
    force_status     NVARCHAR(20) DEFAULT 'candidate'
    CHECK (force_status IN ('candidate','forced','rejected','retired')),
    forced_by        NVARCHAR(50),        -- 'autonomous' | 'manual'
    regression_threshold_pct INT DEFAULT 20,
    notes            NVARCHAR(1000)
);

CREATE INDEX IX_ade_plan_baselines_queryid
ON dbo.ade_plan_baselines (query_id, force_status);
```

The key architectural decision is the **promotion pipeline**: candidate → accepted → pinned. A plan enters as a candidate when the autonomous system captures it during a clean performance window. It advances to accepted after surviving validation runs (section 8.3). It becomes pinned when the system actively forces it in response to a detected regression. This state machine prevents the automation from forcing plans that were only observed once under favorable conditions.

```
graph LR
    C[Candidate  
captured in clean window] -->|passes shadow validation| A[Accepted]
    A -->|regression detected| P[Pinned  
forced in production]
    P -->|fails weekly re-validation| C
    C -->|fails validation| R[Rejected]
    P -->|second regression within 24h| ESC[Escalate to human]
```

The plan baseline state machine: no plan is forced without first surviving validation, and repeated regression on a pinned plan escalates rather than triggers another automatic force.

For SQL Server, the native Query Store forced plan mechanism is invoked with:

```
EXEC sys.sp_query_store_force_plan @query_id = ?, @plan_id = ?;
```

At plan promotion time, the automation forces the accepted plan in Query Store and records the decision in the overlay table:

```
-- Promotion: force the validated plan, then audit the decision
EXEC sys.sp_query_store_force_plan @query_id = @query_id, @plan_id = @plan_id;

INSERT INTO dbo.ade_plan_baselines
    (query_id, plan_id, query_hash, query_sql_text, avg_duration_us,
     force_status, forced_by, regression_threshold_pct, notes)
VALUES
    (@query_id, @plan_id, @query_hash, @query_text, @avg_duration_us,
     'forced', 'autonomous', 20, @rationale);
```

The baseline store also integrates with your version control system. Each promotion event generates a migration record — the forced plan identifier and rationale — that can be tracked in Git alongside schema changes. This audit trail is non-negotiable in regulated environments.

8.3 The Autonomous Decision Engine: From Signal to Action

The decision engine is the intelligence layer. It consumes instability signals from section 8.1, evaluates candidate plans from the baseline store, and makes promotion or rollback decisions according to a policy model. The policy model must be explicit and auditable — "when X, do Y" — because autonomous systems that act on implicit logic cannot be reviewed, tested, or trusted in production.

The decision policy for autonomous plan management has four rules, evaluated in priority order:

1. **Regression rule:** If mean execution time for a query has increased by more than `regression_threshold_pct` (default 20%) compared to the accepted baseline, and more than `min_regression_samples` (default 100 executions) have been observed in the degraded state, evaluate whether a pinned plan exists. If yes, force the pinned plan immediately. If no, escalate to human review.
2. **Instability rule:** If the coefficient of variation exceeds the configured threshold for a query with no pinned plan, capture the current plan as a candidate baseline during the next clean performance window (CV < 0.3 and mean time within 10% of 30-day low).
3. **Validation rule:** Before promoting a candidate to accepted, the engine runs it with SET STATISTICS TIME, IO ON against a readable secondary with production-representative data and records the measured execution time.

4. **Escalation rule:** Any action that would affect a query running more than 500 times per second, or any query touching more than 10 tables, requires human confirmation before proceeding. The system opens a Slack/PagerDuty alert and waits up to 4 hours before timing out and escalating to critical.

```

# Autonomous plan decision engine – core evaluation loop
# Runs as a scheduled task every 5 minutes (Kubernetes CronJob or systemd timer)

import pyodbc
import json
import logging
from datetime import datetime, timedelta
from dataclasses import dataclass
from enum import Enum
from typing import Optional
import requests # for Slack alerting

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger('ade.plan_mgmt')

class PlanAction(Enum):
    FORCE_EXISTING_BASELINE = "force_existing_baseline"
    CAPTURE_NEW_CANDIDATE = "capture_new_candidate"
    ESCALATE_HUMAN = "escalate_human"
    MONITOR_ONLY = "monitor_only"

@dataclass
class QuerySignal:
    queryid: int
    query_hash: str
    mean_exec_time_ms: float
    coefficient_of_variation: float
    calls_per_minute: float
    table_count: int
    baseline_mean_ms: Optional[float]
    pinned_plan_id: Optional[int]
    regression_threshold_pct: int = 20

def evaluate_plan_action(signal: QuerySignal) -> tuple[PlanAction, str]:
    """
    Applies the four-rule policy model in priority order.
    Returns (action, rationale) for audit logging.
    """
    # Safety gate: high-frequency or complex queries require human confirmation
    if signal.calls_per_minute > 500 or signal.table_count > 10:
        return (
            PlanAction.ESCALATE_HUMAN,
            f"Safety gate: {signal.calls_per_minute:.0f} calls/min or "
            f"{signal.table_count} tables exceeds autonomous threshold"
        )

    # Rule 1: Regression with existing baseline
    if signal.baseline_mean_ms is not None:
        regression_pct = (

```

```

        (signal.mean_exec_time_ms - signal.baseline_mean_ms)
        / signal.baseline_mean_ms * 100
    )
    if regression_pct > signal.regression_threshold_pct:
        if signal.pinned_plan_id is not None:
            return (
                PlanAction.FORCE_EXISTING_BASELINE,
                f"Regression detected: {regression_pct:.1f}% above baseline "
                f"({signal.mean_exec_time_ms:.1f}ms vs "
                f"{signal.baseline_mean_ms:.1f}ms baseline). "
                f"Forcing plan_id={signal.pinned_plan_id}"
            )
        else:
            return (
                PlanAction.ESCALATE_HUMAN,
                f"Regression of {regression_pct:.1f}% detected but no pinned "
                f"baseline available. Human review required."
            )

# Rule 2: Instability – capture candidate if in clean window
if signal.coefficient_of_variation > 1.5:
    if signal.coefficient_of_variation < 0.3: # currently stable – clean window
        return (
            PlanAction.CAPTURE_NEW_CANDIDATE,
            f"Instability pattern detected
(CV={signal.coefficient_of_variation:.2f}). "
            f"Current window is stable – capturing candidate baseline."
        )
    return (
        PlanAction.MONITOR_ONLY,
        f"Instability detected (CV={signal.coefficient_of_variation:.2f}) "
        f"but no clean window available. Continuing observation."
    )

return (PlanAction.MONITOR_ONLY, "No action criteria met.")

def dispatch_action(action: PlanAction, signal: QuerySignal,
                    rationale: str, db_conn, slack_webhook: str):
    """Execute the decided action and log the outcome."""
    logger.info(f"QueryID {signal.queryid}: {action.value} – {rationale}")

    if action == PlanAction.FORCE_EXISTING_BASELINE:
        # SQL Server: sp_query_store_force_plan via force_plan()
        force_plan(signal.queryid, signal.pinned_plan_id, db_conn)
        log_action_audit(signal.queryid, action, rationale, db_conn)

    elif action == PlanAction.CAPTURE_NEW_CANDIDATE:
        capture_candidate_plan(signal.queryid, db_conn)
        log_action_audit(signal.queryid, action, rationale, db_conn)

    elif action == PlanAction.ESCALATE_HUMAN:
        send_escalation_alert(signal, rationale, slack_webhook)

```

```
log_action_audit(signal.queryid, action, rationale, db_conn)

# MONITOR_ONLY: log only, no structural change
else:
    log_action_audit(signal.queryid, action, rationale, db_conn)
```

The decision engine is intentionally stateless between runs — all context comes from the database tables. This means it can be restarted, replaced, or scaled horizontally without losing decision continuity. The audit log table records every evaluation, the signal values at decision time, the action taken, and the outcome measured on the next evaluation cycle. This retrospective data is what you use to tune thresholds.

8.4 Plan Validation Before Promotion: The Shadow Execution Framework

Autonomous systems that force execution plans without validating them first will cause outages. The validation framework is the safety net between the decision engine's recommendation and the actual plan forcing action. It runs candidate plans against production-representative data in a controlled environment before they advance to accepted status.

The shadow execution framework has three components: a read replica or isolated test database with current statistics, a workload replay mechanism, and a comparison evaluator.

Setting up the validation environment:

The validation target is a readable secondary in the availability group. The automation connects to the secondary, runs the candidate workload with `SET STATISTICS TIME, IO ON`, captures the actual execution plan, and records the real execution metrics. This gives you real statistics and real data distributions without risk to the primary.

```

-- Validation via USE PLAN hint with Query Store plan XML
-- Forces a specific plan from Query Store during testing
DECLARE @plan_xml NVARCHAR(MAX);

SELECT @plan_xml = CAST(qsp.query_plan AS NVARCHAR(MAX))
FROM sys.query_store_plan qsp
WHERE qsp.plan_id = ?; -- candidate plan_id from baseline store

-- Execute with plan forcing on the validation workload
-- (run against a secondary read-only AG replica)
EXEC sp_executesql
    N'SELECT o.OrderID, o.TotalAmount, c.Email
      FROM dbo.Orders o
     JOIN dbo.Customers c ON o.CustomerID = c.CustomerID
    WHERE o.CreatedDate >= DATEADD(DAY, -30, GETUTCDATE())
          AND o.Status = ''pending''
    OPTION (USE PLAN @hint_plan)',
    N'@hint_plan NVARCHAR(MAX)',
    @hint_plan = @plan_xml;

```

The comparison evaluator applies statistical significance testing to the validation results. A candidate plan must demonstrate at minimum a 10% improvement over the current plan's mean execution time, with the improvement being statistically significant at $p < 0.05$ across at least 30 validation executions. A plan that is merely equivalent to the current plan is not promoted — it offers no benefit and introducing any plan change carries operational risk.

```

# Plan validation comparison with statistical significance testing
import numpy as np
from scipy import stats

def evaluate_candidate_plan(
    baseline_times_ms: list[float],
    candidate_times_ms: list[float],
    improvement_threshold_pct: float = 10.0,
    alpha: float = 0.05
) -> dict:
    """
    Compare baseline vs. candidate plan execution times.
    Returns promotion decision with full statistical context.
    """

    baseline = np.array(baseline_times_ms)
    candidate = np.array(candidate_times_ms)

    baseline_mean = np.mean(baseline)
    candidate_mean = np.mean(candidate)
    improvement_pct = (baseline_mean - candidate_mean) / baseline_mean * 100

    # Welch's t-test – does not assume equal variance
    t_stat, p_value = stats.ttest_ind(baseline, candidate, equal_var=False)

    # Mann-Whitney U test for non-normal distributions (common in DB latency)
    u_stat, u_p_value = stats.mannwhitneyu(
        baseline, candidate, alternative='greater'
    )

    meets_improvement_threshold = improvement_pct >= improvement_threshold_pct
    statistically_significant = p_value < alpha and u_p_value < alpha
    should_promote = meets_improvement_threshold and statistically_significant

    return {
        "decision": "PROMOTE" if should_promote else "REJECT",
        "baseline_mean_ms": round(baseline_mean, 3),
        "candidate_mean_ms": round(candidate_mean, 3),
        "improvement_pct": round(improvement_pct, 2),
        "t_test_p_value": round(p_value, 6),
        "mannwhitney_p_value": round(u_p_value, 6),
        "statistically_significant": statistically_significant,
        "meets_improvement_threshold": meets_improvement_threshold,
        "sample_size": len(candidate_times_ms),
        "evaluated_at": datetime.utcnow().isoformat()
    }

```

A plan that passes validation gets its status updated to `accepted` in the baseline store and its measured performance characteristics recorded. If a regression is subsequently detected, the system now has a validated plan it can force with confidence.

The shadow execution framework should also run continuously against accepted baselines — not just candidates — on a weekly cycle. Statistics drift can make yesterday's validated baseline wrong for today's data distribution. Baselines that fail re-validation are demoted to `candidate` status automatically, and a new capture cycle begins.

8.5 Regression Rollback Automation: Closing the Control Loop

A plan management system that only detects regressions without acting on them is a monitoring system. The autonomous system closes the control loop by executing rollbacks without waiting for human intervention. This section describes the rollback execution path, the safeguards that prevent rollback cascades, and the feedback mechanism that makes each rollback event improve the system's future decisions.

SQL Server — Automated plan forcing via Query Store:

```

-- Autonomous rollback procedure: forces the accepted baseline plan
-- Called by the decision engine when regression rule fires
CREATE OR ALTER PROCEDURE dbo.usp_ade_force_baseline_plan
    @query_id    BIGINT,
    @reason       NVARCHAR(500) = 'Autonomous regression rollback'
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @plan_id    BIGINT;
    DECLARE @current_status NVARCHAR(20);

    -- Retrieve the accepted baseline plan for this query
    SELECT TOP 1
        @plan_id    = b.plan_id,
        @current_status = b.force_status
    FROM dbo.ade_plan_baselines b
    WHERE b.query_id    = @query_id
        AND b.force_status IN ('accepted', 'pinned')
    ORDER BY b.captured_at DESC;

    IF @plan_id IS NULL
    BEGIN
        RAISERROR('No accepted baseline available for query_id %d; escalating to
human review.', 16, 1, @query_id);
        RETURN;
    END

    -- Force the plan via Query Store
    EXEC sys.sp_query_store_force_plan @query_id = @query_id, @plan_id = @plan_id;

    UPDATE dbo.ade_plan_baselines
    SET force_status    = 'pinned',
        forced_by       = 'autonomous',
        last_validated_at = SYSUTCDATETIME(),
        notes           = @reason
    WHERE query_id = @query_id
        AND plan_id = @plan_id;

    INSERT INTO dbo.ade_plan_action_log (query_id, plan_id, action, reason,
acted_at)
    VALUES (@query_id, @plan_id, 'FORCE_PLAN', @reason, SYSUTCDATETIME());
END

```

Rollback is a native Query Store operation: unforcing the plan returns the optimizer to free choice, and the overlay records the reversal:


```
-- Autonomous rollback: unforce the regressed plan
-- Called by the decision engine when the regression rule fires
EXEC sys.sp_query_store_unforce_plan @query_id = @query_id, @plan_id = @plan_id;

UPDATE dbo.ade_plan_baselines
SET force_status = 'retired',
    notes = CONCAT('Rolled back: ', @reason)
WHERE query_id = @query_id
    AND plan_id = @plan_id
    AND force_status = 'forced';

INSERT INTO dbo.ade_plan_action_log (query_id, plan_id, action, reason, acted_at)
VALUES (@query_id, @plan_id, 'UNFORCE_PLAN', @reason, SYSUTCDATETIME());
```

Rollback cascade prevention is the safeguard that keeps this loop from oscillating. If a forced plan itself starts regressing — because the underlying data distribution has shifted again — the naive response would be to force yet another plan, and potentially flip back and forth indefinitely as conditions change. The autonomous system enforces a cooldown: once a plan is forced, no further forcing action is taken for that query for a minimum window (typically one hour), and if a second regression is detected within 24 hours of a prior force, the action escalates to human review instead of forcing again automatically. Two autonomous corrections in one day on the same query is treated as evidence that the underlying cause is not a simple plan flip but something the decision policy doesn't understand yet.

The feedback mechanism records every rollback's outcome — did execution time actually return to baseline after the force? — back into the audit log. This closes the loop: a decision engine that forces a plan and never checks whether it worked is not meaningfully different from a human who guesses and walks away.

Key Takeaways

- → Plan instability falls into three distinguishable categories — regression, parameter sensitivity, and optimizer-version regression — and the observability layer must capture enough plan-level detail (via Query Store) to tell them apart before any automated action fires.
- → A plan baseline store with an explicit state machine (candidate → accepted → pinned) prevents the system from forcing a plan that was only ever observed once under favorable conditions; forcing is implemented natively via Query Store's `sp_query_store_force_plan`.
- → The decision engine's policy rules must be explicit, ordered, and auditable — regression detection, instability capture, pre-promotion validation, and escalation thresholds for high-frequency or structurally complex queries — because implicit logic cannot be reviewed or trusted in production.

- → No candidate plan should be promoted without shadow validation against production-representative data and a real statistical test (not just an eyeballed percentage difference); a plan that is merely equivalent to the current one should be rejected, since any plan change carries operational risk without corresponding benefit.
- → Rollback automation needs its own safeguard against oscillation: a cooldown window after forcing a plan, and escalation to human review if the same query regresses again shortly after a prior autonomous correction, rather than repeatedly re-forcing plans in response to a cause the system doesn't understand.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 9: Self-Healing Database Operations

Self-healing is the operational core of any autonomous database system. While previous chapters addressed detection and decision-making in isolation, this chapter synthesizes those capabilities into end-to-end remediation loops — systems that not only observe failure conditions but act on them without waiting for a human to read a PagerDuty alert at 2 AM. The patterns here draw from production architectures running SQL Server Always On Availability Groups under Windows Server Failover Clustering and Kubernetes operators, Azure SQL failover groups, and orchestration layers that treat remediation as a deterministic state machine rather than a collection of runbooks. The goal is not to automate existing manual procedures. It is to design systems whose default state is recovery — systems where the question is never "will it heal?" but "how fast, and at what cost?"

Anatomy of a Remediation Loop: Detect, Classify, Act, Verify

A self-healing system is architecturally identical to a control system in industrial engineering. There is a sensor layer, a comparator that measures deviation from desired state, an actuator that drives corrective action, and a feedback loop that confirms the system returned to setpoint. Every remediation loop in a production database platform must implement all four stages, or it is not self-healing — it is just automated alerting with extra steps.

Detection at the ADE level is not threshold crossing. Threshold crossing is the 2015 version of monitoring. Real detection uses composite signals: a connection pool saturation event that correlates with a blocker query age exceeding 30 seconds and a transaction rate drop of more than 15% within the same 90-second window means something different than any of those signals alone. The detection layer must produce classified events, not raw metrics.

Classification determines which remediation playbook applies. A primary replica that fails a health check because its log-send queue is 500MB behind while the secondary is still connected and network latency is normal is in a different failure class than a primary that fails a health check because the OS ran out of file descriptors. One requires a controlled manual failover. The other requires a forced failover with possible data loss, plus an investigation of the host before the old primary is ever allowed back.

Action must be implemented as a state machine. The actuator doesn't execute a single command — it drives the system through a sequence of states with defined transitions, rollback conditions, and timeouts at each stage.

Verification closes the loop. After remediation, the system must re-measure the originating signal and confirm it has returned to a healthy range before marking the incident closed. Without this, you get cascading automation: a loop that fires the same remediation action repeatedly because it never confirmed the first execution succeeded.

The following Python class implements a minimal remediation loop skeleton that production systems can extend:

```

import time
import logging
from enum import Enum, auto
from dataclasses import dataclass, field
from typing import Callable, Optional

logger = logging.getLogger("remediation_loop")

class RemediationState(Enum):
    IDLE = auto()
    DETECTED = auto()
    CLASSIFIED = auto()
    ACTING = auto()
    VERIFYING = auto()
    RESOLVED = auto()
    FAILED = auto()
    ESCALATED = auto()

@dataclass
class RemediationContext:
    incident_id: str
    signal: dict
    classification: Optional[str] = None
    playbook: Optional[str] = None
    action_result: Optional[dict] = None
    attempt_count: int = 0
    max_attempts: int = 3
    state: RemediationState = RemediationState.IDLE
    history: list = field(default_factory=list)

class RemediationLoop:
    def __init__(
        self,
        classifier: Callable,
        playbook_registry: dict,
        verifier: Callable,
        escalation_hook: Callable,
    ):
        self.classifier = classifier
        self.playbook_registry = playbook_registry
        self.verifier = verifier
        self.escalation_hook = escalation_hook

    def run(self, ctx: RemediationContext) -> RemediationState:
        ctx.state = RemediationState.DETECTED
        ctx.history.append(("DETECTED", time.time()))

        # Classify
        ctx.classification = self.classifier(ctx.signal)
        if ctx.classification not in self.playbook_registry:
            logger.error(f"No playbook for classification: {ctx.classification}")
            ctx.state = RemediationState.ESCALATED
            self.escalation_hook(ctx)

```

```

        return ctx.state

    ctx.state = RemediationState.CLASSIFIED
    ctx.playbook = self.playbook_registry[ctx.classification]
    logger.info(f"[{ctx.incident_id}] Classified as {ctx.classification}, "
               f"playbook: {ctx.playbook.__name__}")

    # Act with retry
    while ctx.attempt_count < ctx.max_attempts:
        ctx.state = RemediationState.ACTING
        ctx.attempt_count += 1
        try:
            ctx.action_result = ctx.playbook(ctx)
            break
        except Exception as e:
            logger.warning(f"[{ctx.incident_id}] Attempt {ctx.attempt_count}
failed: {e}")
            if ctx.attempt_count >= ctx.max_attempts:
                ctx.state = RemediationState.ESCALATED
                self.escalation_hook(ctx)
                return ctx.state
            time.sleep(2 ** ctx.attempt_count) # exponential backoff

    # Verify
    ctx.state = RemediationState.VERIFYING
    for attempt in range(5):
        time.sleep(10)
        if self.verifier(ctx):
            ctx.state = RemediationState.RESOLVED
            ctx.history.append(("RESOLVED", time.time()))
            logger.info(f"[{ctx.incident_id}] Resolved after "
                       f"{ctx.attempt_count} action attempt(s)")
            return ctx.state
        logger.debug(f"[{ctx.incident_id}] Verification attempt {attempt+1}/5
pending")

    # Verification failed
    ctx.state = RemediationState.FAILED
    self.escalation_hook(ctx)
    return ctx.state

```

```

graph TD
    Idle --> Detected
    Detected --> Classified
    Classified -->|no playbook found| Escalated
    Classified --> Acting
    Acting -->|exception, retries exhausted| Escalated
    Acting --> Verifying
    Verifying -->|condition healthy| Resolved
    Verifying -->|still unhealthy after 5 checks| Failed
    Failed --> Escalated

```

The remediation loop's state machine: detect, classify, act, verify — with every failure path routing to escalation rather than silently retrying forever.

This structure enforces the invariant that no action is taken without a classification, no incident is closed without verification, and repeated failures escalate rather than loop indefinitely. Every production self-healing system must enforce these three invariants. Systems that don't eventually create automation storms — a well-documented failure mode where remediation logic runs in a tight loop and makes the outage worse.

Connection and Session Healing: Killing Blockers, Recycling Pools, and Recovering from Lock Storms

Lock storms and connection exhaustion are the two most common causes of database brownouts that are not hardware failures. They are also the easiest class of problem to self-heal because the corrective actions are low-risk, reversible, and fast. A self-healing system should handle both classes autonomously, with human notification rather than human intervention.

Detecting and killing blockers requires more than identifying the longest-running query. The system must identify the root blocker in a blocking chain — the session holding the lock that all others are waiting on — and evaluate whether terminating it is safe given the current workload context.

```

-- Identify root blockers with wait chain and resource details
WITH BlockingChain AS (
    SELECT
        r.session_id,
        r.blocking_session_id,
        r.wait_type,
        r.wait_time / 1000.0 AS wait_seconds,
        r.status,
        DB_NAME(r.database_id) AS database_name,
        s.login_name,
        s.program_name,
        s.host_name,
        st.text AS current_sql,
        0 AS depth
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
    CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
    WHERE r.blocking_session_id = 0
        AND r.session_id IN (
            SELECT DISTINCT blocking_session_id
            FROM sys.dm_exec_requests
            WHERE blocking_session_id > 0
        )

    UNION ALL

    SELECT
        r.session_id,
        r.blocking_session_id,
        r.wait_type,
        r.wait_time / 1000.0,
        r.status,
        DB_NAME(r.database_id),
        s.login_name,
        s.program_name,
        s.host_name,
        st.text,
        bc.depth + 1
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
    CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
    JOIN BlockingChain bc ON r.blocking_session_id = bc.session_id
    WHERE bc.depth < 10 -- cap recursion depth
)
SELECT
    session_id,
    blocking_session_id,
    depth,
    wait_type,
    wait_seconds,
    database_name,
    login_name,
    program_name,

```



```

LEFT(current_sql, 200) AS sql_snippet
FROM BlockingChain
ORDER BY depth, wait_seconds DESC;

```

Once a root blocker is identified, the system applies a kill decision policy. The policy must evaluate: is this session running a known scheduled job that should not be interrupted? Is the blocking duration below the configured tolerance (e.g., 60 seconds for OLTP, 300 seconds for batch windows)? Is the application name on a protected list? Only after all three conditions pass does the system issue the termination.

Connection exhaustion — SQL Server connection counts at max — requires a different healing path. The system first checks whether the exhaustion is caused by a blocker (which is handled above) or by a genuine connection leak. A leak manifests as a large number of idle connections with increasing age and no active transaction. The healing action is to terminate idle connections beyond a configurable age, then recycle the pool.

```

-- Kill idle sessions exceeding age threshold
DECLARE @session_id INT;
DECLARE @kill_sql NVARCHAR(50);

DECLARE idle_cursor CURSOR FOR
    SELECT session_id
    FROM sys.dm_exec_sessions
    WHERE status = 'sleeping'
        AND login_time < DATEADD(MINUTE, -10, GETDATE())
        AND is_user_process = 1
        AND session_id > 50 -- exclude system sessions
        AND program_name NOT LIKE 'SQL Agent%'
        AND login_name NOT IN ('sa', 'monitoring_user');

OPEN idle_cursor;
FETCH NEXT FROM idle_cursor INTO @session_id;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @kill_sql = N'KILL ' + CAST(@session_id AS NVARCHAR(10));
    EXEC sp_executesql @kill_sql;
    FETCH NEXT FROM idle_cursor INTO @session_id;
END

CLOSE idle_cursor;
DEALLOCATE idle_cursor;

```

After terminations execute, the verification step must confirm that pool availability has recovered: SQL Server's connection count should drop below 80% of the configured maximum. If that condition is not met within 60 seconds, the incident escalates — there is an application-side leak that cannot be healed at the database layer.

Automated Failover Architecture: Always On Availability Groups on WSFC and Kubernetes

Failover is the highest-stakes self-healing action a database system takes. The cost of an incorrect failover decision — promoting a stale replica, failing over when the primary is actually healthy, or triggering split-brain — is frequently worse than the original failure. The architecture of an autonomous failover system must therefore be conservative by design: it should require convergent evidence from multiple independent observers before acting, and it must enforce a fencing protocol that guarantees the old primary cannot accept writes after the new primary is promoted.

Always On automatic failover is the native self-healing mechanism. When a primary replica is configured for synchronous commit with automatic failover, the Windows Server Failover Cluster (or the Kubernetes operator) promotes a synchronized secondary if the primary becomes unavailable — no human decision is involved. The self-healing capability is inherent to the design, but production deployments require careful tuning of three conditions:

- → `synchronization_health` : automatic failover only fires when the secondary is in a SYNCHRONIZED state. A secondary that has fallen behind to SYNCHRONIZING will not be promoted automatically — the cluster waits or requires manual intervention, which is the correct conservative behavior.
- → `quorum` : the WSFC must retain quorum to make a failover decision. Losing quorum freezes automatic failover entirely — this is the fencing guarantee that prevents split-brain, and it is why quorum witness placement (disk, file share, or cloud witness) is a first-class design decision.
- → `flexible failover policy` : the failure-condition level (from transient network blips to full server unresponsiveness) determines what counts as a failure. Too sensitive causes false failovers under load; too lenient delays legitimate failovers. Level 3 with customized timeout multipliers is a common production starting point for low-latency OLTP.

Quorum witnesses determine whether the cluster can reach a failover decision at all. A file-share witness on a third site, a disk witness on shared storage, or a cloud witness in Azure Blob Storage each keep quorum achievable when one data-center side goes dark. The autonomous layer monitors witness reachability as a first-class health signal: if the witness becomes unreachable, automatic failover is effectively suspended, and the system must alert loudly rather than assume failover protection is intact.

For **SQL Server Always On** running on Kubernetes, the SQL Server Operator (either the Microsoft SQL Server Operator or community operators) manages AG health through Kubernetes health probes and custom resource definitions. The operator pattern here is

significant: the operator reconciliation loop is itself a self-healing construct. Any deviation between the desired state declared in the `SqlServer` CRD and the observed state triggers the operator's reconcile function, which drives corrective action.

A production Kubernetes-based AG self-healing architecture uses three components:

1. **Liveness probe** at the SQL Server pod level — if the pod fails the liveness probe, Kubernetes restarts it. The probe must be scoped to verify AG listener responsiveness, not just TCP port availability.
2. **Operator reconciler** — detects AG replica role divergence (e.g., a secondary that has been in `RESOLVING` state for more than 60 seconds) and triggers a manual failover via T-SQL.
3. **External DNS or load balancer controller** — updates the listener IP binding after a successful failover so application connection strings remain stable.

The reconcile function for a simplified AG failover:

```

import subprocess
import pyodbc
import logging
from kubernetes import client, config

logger = logging.getLogger("ag_reconciler")

def get_ag_status(conn_str: str, ag_name: str) -> dict:
    """Query AG replica status from the current primary."""
    conn = pyodbc.connect(conn_str, autocommit=True)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT
            ar.replica_server_name,
            ars.role_desc,
            ars.synchronization_health_desc,
            ars.connected_state_desc,
            ars.last_commit_time
        FROM sys.dm_hadr_availability_replica_states ars
        JOIN sys.availability_replicas ar
            ON ars.replica_id = ar.replica_id
        JOIN sys.availability_groups ag
            ON ar.group_id = ag.group_id
        WHERE ag.name = ?
    """, ag_name)
    rows = cursor.fetchall()
    conn.close()
    return {
        row.replica_server_name: {
            "role": row.role_desc,
            "sync_health": row.synchronization_health_desc,
            "connected": row.connected_state_desc,
            "last_commit": row.last_commit_time,
        }
        for row in rows
    }

def trigger_failover(conn_str: str, ag_name: str, target_replica: str):
    """Execute controlled failover to target replica."""
    # Connect to the TARGET replica, not the primary
    conn = pyodbc.connect(conn_str, autocommit=True)
    cursor = conn.cursor()
    logger.warning(f"Initiating failover of {ag_name} to {target_replica}")
    cursor.execute(f"ALTER AVAILABILITY GROUP [{ag_name}] FAILOVER;")
    conn.close()
    logger.info(f"Failover command issued to {target_replica}")

def reconcile_ag(ag_name: str, primary_conn: str, replica_conns: dict):
    status = get_ag_status(primary_conn, ag_name)

    primary_count = sum(1 for r in status.values() if r["role"] == "PRIMARY")
    healthy_secondary = [
        name for name, r in status.items()
    ]

```

```

        if r["role"] == "SECONDARY"
        and r["sync_health"] == "HEALTHY"
        and r["connected"] == "CONNECTED"
    ]

    if primary_count == 0:
        logger.error(f"No primary detected for AG {ag_name}. "
                    f"Attempting failover to best available replica.")
    if healthy_secondary:
        target = healthy_secondary[0]
        target_conn = replica_conns.get(target)
        if target_conn:
            trigger_failover(target_conn, ag_name, target)
        else:
            logger.error(f"No connection string for target replica {target}.
Escalating.")
    else:
        logger.critical(f"No healthy secondary available for {ag_name}. "
                      f"Manual intervention required.")
    elif primary_count > 1:
        logger.critical(f"Split-brain detected for {ag_name}: "
                      f"{primary_count} nodes reporting PRIMARY role. "
                      f"Escalating immediately – no automated action.")
    else:
        logger.debug(f"AG {ag_name} healthy. Primary confirmed, "
                    f"{len(healthy_secondary)} healthy secondaries.")

```

The critical design choice in this reconciler is the `primary_count > 1` branch: the system deliberately refuses to take automated action on split-brain and escalates immediately. This is the correct behavior. Automated systems must know which failure classes require human judgment, and split-brain is categorically one of them.

Autonomous Space Management: Ghost Cleanup, Fragmentation Control, and Filegroup Rebalancing

Space management failures — table bloat causing query plan degradation, transaction ID wraparound approaching, or a SQL Server filegroup filling to capacity — are slow-moving incidents that are ideal for autonomous remediation. They are predictable, the corrective actions are well-understood, and the window for action is measured in hours or days rather than seconds.

Ghost records and heap forwarding are SQL Server's slow-burn space problems. When rows are deleted, they become ghost records that the background ghost cleanup task reclaims — but under sustained heavy delete workloads, ghost cleanup can fall behind, leaving pages that are logically empty yet physically full. Heaps accumulate forwarded

records when variable-length rows grow beyond their original slot, turning single-row reads into multi-page chases. A self-healing system monitors this trajectory and intervenes before the waste materializes as plan degradation.

The detection query identifies indexes where ghost records or fragmentation are losing the race:

```
-- Indexes where ghost records or fragmentation are accumulating
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) + '.' + OBJECT_NAME(ips.object_id) AS
table_name,
    i.name AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.ghost_record_count,
    ips.forwarded_record_count,
    ips.page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'DETAILED') ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE ips.page_count > 1000
    AND (ips.avg_fragmentation_in_percent > 10
        OR ips.ghost_record_count > 1000
        OR ips.forwarded_record_count > 1000)
ORDER BY ips.ghost_record_count DESC;
```

When this query identifies candidates, the self-healing system applies a graduated response. If fragmentation is between 10% and 30%, it schedules an `ALTER INDEX ... REORGANIZE` — a lightweight, online operation that compacts pages and reclaims ghost records. If fragmentation exceeds 30% or ghost records are accumulating faster than cleanup reclaims them, it schedules an `ALTER INDEX ... REBUILD WITH (ONLINE = ON)` during the next maintenance window. If a heap shows a high forwarded-record count, it schedules a clustered index creation (or a table rebuild) — the only durable fix for forwarded-record chains — during a low-traffic window.

For **transaction log emergencies**, the autonomous system monitors log percent-used via `sys.dm_db_log_space_usage` and enforces a two-stage response: when log usage exceeds 80%, the system checks the `log_reuse_wait` reason — an open transaction, a stalled AG log-send queue, or a delayed log backup — and acts on the specific blocker. When it exceeds 90%, it takes emergency action: forcing a log backup on the primary (or the preferred backup replica), and if the blocker is an orphaned open transaction, alerting with the session details for human kill approval.

Filegroup capacity management, index fragmentation, and log file growth are the primary targets. The autonomous system monitors filegroup utilization and pre-emptively extends data files before they hit autogrowth events — because autogrowth during peak traffic causes latency spikes:

```
-- Monitor filegroup utilization and project time-to-full based on recent growth
rate
SELECT
    DB_NAME(mf.database_id)                AS database_name,
    mf.name                                AS file_name,
    mf.type_desc,
    CAST(fs.size / 128.0 AS DECIMAL(15,2)) AS current_size_mb,
    CAST(FILEPROPERTY(mf.name, 'SpaceUsed') / 128.0 AS DECIMAL(15,2)) AS used_mb,
    CAST(
        (fs.size - FILEPROPERTY(mf.name, 'SpaceUsed')) / 128.0
        AS DECIMAL(15,2)
    )                                       AS free_mb,
    mf.growth,
    mf.is_percent_growth
FROM sys.master_files mf
JOIN sys.database_files fs
    ON mf.database_id = DB_ID() AND fs.name = mf.name
WHERE mf.database_id = DB_ID()
ORDER BY free_mb ASC;
```

When free space in a data file drops below a configured floor (for example, less than the volume of one day's typical growth), the autonomous system proactively grows the file during a low-traffic window using an explicit `ALTER DATABASE ... MODIFY FILE` with a fixed-size increment, rather than allowing autogrowth to fire unplanned mid-transaction. The same pattern applies to transaction log files, where an unplanned autogrowth event is especially costly because it briefly halts all write activity while the log is zero-initialized (unless instant file initialization is enabled for data files — a setting that does not extend to log files).

Index fragmentation is handled with the same graduated-response pattern: a scheduled scan of `sys.dm_db_index_physical_stats` identifies indexes above a fragmentation threshold, and the system chooses `REORGANIZE` for moderate fragmentation (10–30%) or `REBUILD WITH (ONLINE = ON)` for severe fragmentation (>30%), scheduling the rebuild during a maintenance window if the table is large enough that even an online rebuild would add meaningful I/O load.

Key Takeaways

- → A self-healing loop is not complete unless it implements all four control-system stages — detect, classify, act, verify — and every production remediation loop should enforce bounded retries with exponential backoff and mandatory escalation on repeated failure, to prevent automation storms.
- → Connection and lock-storm healing (killing root blockers, recycling idle connections) is the highest-value, lowest-risk starting point for self-healing because the actions are fast, reversible, and easy to verify — but the kill decision must still check protected workloads and tolerance windows before acting.
- → Automated failover is the highest-stakes self-healing action; Always On availability groups converge on one design principle — require corroborating evidence before promoting, and never automate a decision when split-brain is detected. That case always escalates to a human.
- → Space management problems (fragmentation, ghost record accumulation, filegroup exhaustion) are slow-moving and well-suited to graduated autonomous response: escalating intervention tiers tied to how close the system is to the actual emergency, rather than a single binary trigger.
- → Every self-healing action must be idempotent and paired with verification against the original signal — an incident is not resolved because an action executed successfully, it is resolved because the measured condition that triggered the incident has returned to a healthy range.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 10: Automated Root Cause Analysis

When a production database degrades at 2 AM, the difference between a five-minute recovery and a two-hour war room comes down to one thing: how fast the system understands *why* something broke. Automated root cause analysis (ARCA) is the discipline of building systems that move from symptom detection to causal diagnosis without waiting for a human to open a terminal. At the ADE level, this means architecting a pipeline that ingests telemetry across multiple signal layers, applies causal reasoning models, generates testable hypotheses, and either acts autonomously or presents the on-call engineer with a ranked, evidence-backed cause list before they finish reading the alert. This chapter covers the signal taxonomy, correlation engine design, causal graph construction, LLM-augmented diagnosis, and the production patterns that make this work reliably across SQL Server environments.

Signal Taxonomy: Building the Evidence Hierarchy for Database Failures

Root cause analysis fails when the system tries to reason from a single signal layer. A spike in `avg_query_time` is a symptom. The lock event underneath it is a contributing factor. The schema change that introduced a missing index is the cause. An ARCA system must model all three levels simultaneously.

The practical signal hierarchy for database environments has five layers:

Layer 1 — User-visible symptoms: Query latency (p50/p95/p99), error rates, connection refusals, transaction throughput drops. These are what alerts fire on. They are never root causes.

Layer 2 — Database execution signals: Lock waits, blocked queries, plan cache invalidations, wait event distributions, tempdb spills, ghost cleanup backlog, PAGELATCH/PAGEIOLATCH waits.

Layer 3 — Resource signals: CPU saturation, IOPS ceiling, memory pressure, network bandwidth to storage, log flush latency, VLF fragmentation, or log growth.

Layer 4 — Configuration and state signals: Parameter changes, endpoint and firewall modifications, statistics staleness (modification-counter drift), fragmentation ratio, ghost record accumulation, AG synchronization lag, WSFC quorum history.

Layer 5 — External causation signals: Deployment events, schema migrations, batch job start times, connection pool reconfiguration, upstream service traffic spikes, cloud provider maintenance windows.

Each layer has different collection mechanisms and different temporal characteristics. Resource signals move in seconds. Configuration signals move in minutes to hours. Deployment signals are discrete events. A proper ARCA schema stores all five layers in a unified timeline, keyed to wall-clock time with enough resolution to establish sequence — because *sequence is the foundation of causation*.

The collection architecture relies on DMVs: `sys.dm_exec_requests`, `sys.dm_os_wait_stats`, `sys.dm_exec_query_stats`, `sys.dm_db_index_usage_stats`, and Extended Events sessions for high-resolution capture.

```
-- Equivalent Layer 2 snapshot for SQL Server: active requests with blocking chain
-- Deployed as a SQL Agent job step running every 10 seconds
```

```
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000.0          AS wait_sec,
    r.total_elapsed_time / 1000.0 AS elapsed_sec,
    r.cpu_time / 1000.0          AS cpu_sec,
    r.logical_reads,
    r.writes,
    r.granted_query_memory        AS memory_grant_pages,
    DB_NAME(r.database_id)        AS database_name,
    s.login_name,
    s.program_name,
    SUBSTRING(t.text, (r.statement_start_offset/2)+1,
        ((CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(t.text)
            ELSE r.statement_end_offset
            END - r.statement_start_offset)/2)+1) AS current_statement,
    qp.query_plan
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
OUTER APPLY sys.dm_exec_query_plan(r.plan_handle) qp
WHERE r.session_id != @@SPID
ORDER BY r.wait_time DESC;
```

Both snapshots are pushed into an InfluxDB measurement by the collector, tagged with `db_host`, `db_name`, `snapshot_ts`, and `collection_layer`. The `collection_layer` tag is what allows the correlation engine to traverse layers during causal chain reconstruction.

Causal Graph Construction: From Correlated Events to Directed Causation

Correlation is not causation — but in ARCA, correlation with correct temporal ordering and physical dependency modeling *is* strong evidence of causation. The architecture that makes this practical is a **causal graph**: a directed acyclic graph (DAG) where nodes represent observable events and edges represent either known physical dependencies or statistically inferred temporal precedence.

Physical dependency edges are hardcoded from database internals knowledge:

- → Lock acquisition → lock wait → query timeout
- → Ghost cleanup blocked by long transaction → page density loss → scan amplification → slow query
- → Log writer stall → replication lag → standby read query degradation
- → Log file autogrowth event → I/O spike → all query latencies increase
- → Plan cache eviction (SQL Server) → recompilation storm → CPU spike

Statistical edges are derived at runtime by analyzing whether event type A consistently precedes event type B within a configurable time window (typically 30–300 seconds, tuned per event class).

The causal graph implementation uses Python's `networkx` library with a custom edge-scoring function that combines:

1. **Temporal precedence score** — how reliably does A precede B in historical incidents?
2. **Physical dependency score** — does database engine documentation confirm A can cause B?
3. **Recency weight** — events in the last 5 minutes score higher than events 30 minutes ago
4. **Severity amplification** — high-magnitude events on critical paths get edge weight boosted

```

import networkx as nx
import pandas as pd
from datetime import datetime, timedelta
from typing import List, Dict, Tuple

class CausalGraphBuilder:
    """
    Builds a directed causal graph from multi-layer telemetry events.
    Physical dependency edges are seeded from the knowledge base.
    Statistical edges are derived from temporal precedence analysis.
    """

    PHYSICAL_DEPENDENCIES: List[Tuple[str, str, float]] = [
        ("ghost_cleanup_blocked", "page_density_loss", 0.95),
        ("table_bloat_increase", "seq_scan_preference", 0.80),
        ("seq_scan_preference", "query_latency_spike", 0.85),
        ("lock_acquisition_spike", "lock_wait_queue", 0.99),
        ("lock_wait_queue", "query_timeout_increase", 0.90),
        ("log_writer_stall", "replication_lag_increase", 0.95),
        ("replication_lag_increase", "standby_query_degradation", 0.75),
        ("log_autogrowth_event", "io_stall_spike", 0.90),
        ("io_stall_spike", "global_latency_increase", 0.85),
        ("stats_staleness", "plan_regression", 0.70),
        ("plan_regression", "cpu_spike", 0.65),
        ("plan_regression", "query_latency_spike", 0.80),
        ("connection_pool_exhaust", "new_conn_refusal", 0.99),
        ("schema_change_event", "stats_staleness", 0.60),
        ("schema_change_event", "plan_regression", 0.55),
        ("deployment_event", "schema_change_event", 0.85),
    ]

    def __init__(self, lookback_window_sec: int = 300):
        self.graph = nx.DiGraph()
        self.lookback_window = timedelta(seconds=lookback_window_sec)
        self._seed_physical_edges()

    def _seed_physical_edges(self):
        for source, target, weight in self.PHYSICAL_DEPENDENCIES:
            self.graph.add_edge(source, target,
                                edge_type="physical",
                                weight=weight)

    def ingest_events(self, events: List[Dict]) -> None:
        """
        events: list of dicts with keys:
            event_type, severity (0-1), timestamp (datetime), host, metadata
        """
        now = datetime.utcnow()
        relevant = [e for e in events
                    if e["timestamp"] >= now - self.lookback_window]

        for event in relevant:
            node = event["event_type"]

```

```

        if node not in self.graph:
            self.graph.add_node(node,
                                severity=event["severity"],
                                timestamp=event["timestamp"],
                                host=event["host"],
                                metadata=event.get("metadata", {}))
        else:
            # Update severity to max observed
            existing = self.graph.nodes[node].get("severity", 0)
            self.graph.nodes[node]["severity"] = max(existing,
                                                       event["severity"])

def compute_root_causes(self, target_symptom: str) -> List[Dict]:
    """
    Walk backwards from target_symptom through the causal graph.
    Returns ranked list of root cause candidates with confidence scores.
    """
    if target_symptom not in self.graph:
        return []

    candidates = []
    # Find all ancestors of the symptom node
    ancestors = nx.ancestors(self.graph, target_symptom)

    for ancestor in ancestors:
        if not self.graph.nodes[ancestor]:
            continue

        # Compute path strength: product of edge weights along shortest path
        try:
            path = nx.shortest_path(self.graph, ancestor, target_symptom,
                                    weight=lambda u, v, d: 1 - d["weight"])
            path_edges = list(zip(path[:-1], path[1:]))
            path_confidence = 1.0
            for u, v in path_edges:
                path_confidence *= self.graph[u][v]["weight"]

            node_data = self.graph.nodes[ancestor]
            candidates.append({
                "event_type": ancestor,
                "confidence": round(path_confidence, 4),
                "causal_path": path,
                "severity": node_data.get("severity", 0),
                "timestamp": node_data.get("timestamp"),
                "host": node_data.get("host"),
                "metadata": node_data.get("metadata", {}),
            })
        except nx.NetworkXNoPath:
            continue

    # Rank by confidence x severity
    candidates.sort(
        key=lambda x: x["confidence"] * x["severity"],

```

```

        reverse=True
    )
    return candidates

```

```

graph LR
    SC[schema_change_event] --> SS[stats_staleness]
    SS --> PR[plan_regression]
    PR --> QL[query_latency_spike]
    PR --> CPU[cpu_spike]
    AV[ghost_cleanup_blocked] --> TB[page_density_loss]
    TB --> SP[seq_scan_preference]
    SP --> QL
    LA[lock_acquisition_spike] --> LW[lock_wait_queue]
    LW --> QT[query_timeout_increase]
    WS[log_writer_stall] --> RL[replication_lag_increase]
    RL --> SD[standby_query_degradation]

```

A sample slice of the causal DAG's seeded physical-dependency edges — the topology `compute_root_causes` walks backward through from an observed symptom to its ranked ancestor causes.

This graph is rebuilt every 60 seconds during an active incident. The `compute_root_causes` method walks backwards from the observed symptom — say, `query_latency_spike` — through every physical and statistical dependency edge, scoring each ancestor node by the cumulative product of edge weights along the causal path. The highest-scoring ancestor that has an observed event attached to it becomes the top root cause candidate.

The key architectural decision is separating graph *topology* (which is pre-seeded from engine knowledge and updated slowly) from graph *node state* (which is populated in real time from live telemetry). This means the reasoning structure does not have to be re-derived during an incident — only the evidence layer changes.

Wait Event Fingerprinting: Identifying Failure Modes from Execution Signatures

Wait event distributions are one of the most reliable fingerprints for database failure modes. Rather than treating a `PAGELATCH_EX` wait or a `Lock:relation` wait as a single data point, an ARCA system learns the characteristic wait event *composition* of known failure modes and uses signature matching to accelerate diagnosis.

Each failure mode has a signature: a probability distribution over wait event types observed during confirmed incidents of that type. The ARCA engine maintains a library of these signatures, built from historical incidents and augmented with known database engine behavior.

For example, the "missing index causing full table scan under load" failure mode shows a characteristic distribution: `PAGEIOLATCH_SH` climbing while `CXPACKET` (parallelism) waits either spike (if the optimizer attempts a parallel scan) or remain flat (if DOP is forced down by memory pressure).

```
-- Wait event distribution snapshot for SQL Server signature matching
-- Captures delta from baseline (cleared at analysis window start)

WITH current_waits AS (
    SELECT
        wait_type,
        waiting_tasks_count,
        wait_time_ms,
        max_wait_time_ms,
        signal_wait_time_ms,
        wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms
    FROM sys.dm_os_wait_stats
    WHERE wait_type NOT IN (
        -- Filter benign background waits
        'SLEEP_TASK', 'WAITFOR', 'BROKER_TO_FLUSH', 'BROKER_TASK_STOP',
        'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE', 'FT_IFTS_SCHEDULER_IDLE_WAIT',
        'HADR_WORK_QUEUE', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION',
        'SQLTRACE_BUFFER_FLUSH', 'WAIT_XTP_OFFLINE_CKPT_NEW_LOG',
        'XE_DISPATCHER_WAIT', 'XE_TIMER_EVENT', 'SLEEP_DBSTARTUP',
        'SLEEP_DCOMSTARTUP', 'SLEEP_MASTERMDREADY', 'SLEEP_MASTERUPGRADED',
        'SLEEP_MSDBSTARTUP', 'SLEEP_SYSTEMTASK', 'SLEEP_TEMPDBSTARTUP',
        'SNI_HTTP_ACCEPT', 'SP_SERVER_DIAGNOSTICS_SLEEP'
    )
)
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    ROUND(wait_time_ms / 1000.0, 2) AS wait_time_sec,
    ROUND(max_wait_time_ms / 1000.0, 2) AS max_wait_sec,
    ROUND(resource_wait_time_ms / 1000.0, 2) AS resource_wait_sec,
    ROUND(signal_wait_time_ms / 1000.0, 2) AS signal_wait_sec,
    ROUND(
        wait_time_ms * 100.0 / NULLIF(SUM(wait_time_ms) OVER (), 0),
        2) AS pct_total_wait
FROM current_waits
ORDER BY wait_time_ms DESC;
```

The signature matching algorithm computes cosine similarity between the observed wait distribution vector and each stored failure mode vector. A match above 0.82 cosine similarity triggers a `HIGH_CONFIDENCE` classification for that failure mode; 0.65–0.82 triggers `MODERATE_CONFIDENCE`. Below 0.65, the system falls back to the causal graph traversal described in the previous section.

This approach handles the common case where a failure mode is already known — it bypasses multi-layer correlation and delivers a diagnosis in one scoring pass. The causal graph is reserved for novel or compound failure modes where the signature library has no strong match.

The signature library is self-expanding: every confirmed incident where a human DBA records the root cause is automatically converted into a new signature vector and merged into the library via a background reconciliation job. Over enough months of operation to accumulate a meaningful incident history, the hit rate against known failure-mode signatures rises and time-to-diagnosis for previously-seen incident types drops substantially — though the actual hit rate and diagnosis latency depend heavily on incident diversity in a given environment and should be measured locally rather than assumed from a generic figure.

LLM-Augmented Diagnosis: Structured Context Injection for Causal Reasoning

The causal graph and wait event fingerprinting handle the *known* failure space well. What they cannot handle is novel compound failures — situations where three events interact in a way that has never appeared in training data. This is where large language model integration provides genuine value, not as a replacement for structured analysis but as a reasoning layer over structured evidence.

The pattern is **context injection with structured output enforcement**. The ARCA pipeline constructs a detailed incident context object containing the ranked causal candidates from the graph engine, the top wait event signatures, recent configuration changes, active query plans for blocked sessions, and the relevant metric time series. This context is serialized into a structured prompt and submitted to an LLM — a current-generation, high-capability model from a provider like Anthropic or OpenAI is suitable for this pattern. The LLM is instructed to reason over the provided evidence, propose a root cause hypothesis, and return a structured JSON response — never free text.

The enforced output schema is:


```

{
  "primary_cause": {
    "category": "lock_contention | resource_exhaustion | plan_regression | ...",
    "description": "string",
    "confidence": 0.0–1.0,
    "supporting_evidence": ["string", ...]
  },
  "contributing_factors": [...],
  "recommended_actions": [
    {
      "action_type": "terminate_query | reindex | update_stats | ...",
      "target": "string",
      "priority": "immediate | deferred",
      "estimated_impact": "string",
      "requires_human_approval": true | false
    }
  ],
  "confidence_in_diagnosis": 0.0–1.0,
  "requires_escalation": true | false,
  "escalation_reason": "string | null"
}

```

The `requires_human_approval` flag on each recommended action is where the autonomous boundary is enforced. Actions that are statistically low-risk (killing a single idle-in-transaction session, running `ANALYZE` on a table, flushing plan cache for a specific query) are executed autonomously. Actions above the risk threshold (failover initiation, index creation on tables >50GB, parameter changes) are queued for human approval with a 15-minute timeout — if no response arrives, the system either escalates further or falls back to a pre-approved safe action.

```

import anthropic
import json
from dataclasses import dataclass, asdict
from typing import Optional

@dataclass
class IncidentContext:
    db_type: str # "sqlserver"
    db_host: str
    incident_start_utc: str
    symptom_description: str
    causal_candidates: list # from CausalGraphBuilder.compute_root_causes()
    wait_event_snapshot: list # from wait event query
    top_blocked_queries: list
    recent_config_changes: list # last 2 hours from audit log
    deployment_events: list # last 24 hours from CI/CD webhook
    active_query_plans: list # explain output for top blocked queries
    metric_summary: dict # p95 latency, error rate, tps delta

def build_diagnosis_prompt(ctx: IncidentContext) -> str:
    context_json = json.dumps(asdict(ctx), indent=2, default=str)
    return f"""You are a senior database reliability engineer performing root cause
analysis.
You have been provided with structured telemetry from a {ctx.db_type} database
cluster
experiencing degraded performance. Analyze the evidence and return ONLY a valid JSON
object
conforming to the required schema.

Do not speculate beyond the provided evidence. If evidence is insufficient for high
confidence,
set confidence_in_diagnosis below 0.6 and set requires_escalation to true.

INCIDENT CONTEXT:
{context_json}

Return ONLY valid JSON. No explanatory text before or after the JSON object."""

def get_llm_diagnosis(ctx: IncidentContext,
                      model: str = "<configured-model-id>") -> dict: # set to your
provider's current model
    client = anthropic.Anthropic()
    prompt = build_diagnosis_prompt(ctx)

    message = client.messages.create(
        model=model,
        max_tokens=2048,
        messages=[{"role": "user", "content": prompt}]
    )

    raw_response = message.content[0].text.strip()

    # Strip markdown fences if present (some models add them despite instructions)

```

```

if raw_response.startswith("```"):
    raw_response = raw_response.split("```")[1]
    if raw_response.startswith("json"):
        raw_response = raw_response[4:]

try:
    diagnosis = json.loads(raw_response)
except json.JSONDecodeError as e:
    # Fallback: log the raw response and return a minimum safe diagnosis
    return {
        "primary_cause": {"category": "unknown", "confidence": 0.0,
                          "description": "LLM response parse failure",
                          "supporting_evidence": []},
        "contributing_factors": [],
        "recommended_actions": [],
        "confidence_in_diagnosis": 0.0,
        "requires_escalation": True,
        "escalation_reason": f"LLM parse error: {str(e)}"
    }

return diagnosis

```

The LLM layer is not on the critical path. The causal graph and signature matching run synchronously and produce a diagnosis within the same collection interval that detected the symptom, so any autonomous low-risk action described earlier in this book can proceed without waiting on a network call to an external model. The LLM call is invoked asynchronously, alongside or immediately after the structured diagnosis, specifically for the residual class of incidents the structured methods could not confidently classify — and its output augments the on-call engineer's context rather than gating the system's fast-path response. This ordering matters: an architecture where autonomous remediation waits on an LLM response introduces both unnecessary latency and a new failure mode (a slow or unavailable model API stalling incident response) into what should be the fastest part of the system.

Key Takeaways

- → Root cause analysis requires modeling telemetry across at least five distinct signal layers — user-visible symptoms, database execution signals, resource signals, configuration/state signals, and external causation events — because reasoning from a single layer conflates symptoms with causes.
- → A causal graph separates stable topology (physical dependencies seeded from engine internals knowledge) from live node state (real-time telemetry), which lets the reasoning structure persist across incidents while only the evidence changes.

- → Wait event fingerprinting — matching the observed distribution of wait types against a library of known failure-mode signatures — resolves previously-seen incidents in a single scoring pass and should be tried before falling back to full causal graph traversal.
- → LLM-augmented diagnosis belongs off the critical path: it is valuable for novel, compound failures the structured methods can't confidently classify, but it must return schema-enforced structured output, and any action it recommends above a defined risk threshold still requires human approval.
- → Claims about hit rates, time-to-diagnosis improvements, or other outcome statistics for a system like this should be measured against your own incident history rather than assumed from a generic industry figure — the actual numbers vary enormously with incident diversity and signature library maturity.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 11: Predictive Failure Detection

Reactive alerting is the floor, not the ceiling. When a database crashes, a disk fills, or a replication lag spikes to the point of data loss, the system has already failed — the alert is an obituary. Autonomous systems operate in a fundamentally different mode: they maintain continuous predictive models of component health and trigger remediation before the failure state is reached. This chapter covers the architecture, signal collection, model design, and automated response systems required to build genuine predictive failure detection into SQL Server environments — not as a monitoring dashboard that a human reads, but as a closed-loop system that detects, decides, and acts without waiting for a page.

From Threshold Alerts to Failure Probability Scores

Traditional monitoring assigns static thresholds: disk usage above 85% triggers a warning, above 95% triggers a critical. This model fails in production in two directions simultaneously. It fires too late — 95% disk is often already a crisis — and it fires too often on noise, causing alert fatigue that makes engineers ignore the signal. The ADE replaces static thresholds with failure probability scores derived from trend analysis, rate-of-change detection, and historical failure correlation.

The conceptual shift is from *state* to *trajectory*. A disk at 60% full that is growing at 4GB per hour is more dangerous than a disk at 80% that has been stable for three months. A probability score encodes both current state and projected state at some future horizon (typically T+1h, T+6h, T+24h).

The minimum viable predictive signal set for a database node includes:

- → **Storage**: current utilization, fill rate (bytes/hour), projected exhaustion time
- → **Memory**: buffer pool hit ratio trend, free memory rate-of-change, OOM kill history
- → **Replication**: lag in bytes and seconds, lag acceleration, sender/receiver heartbeat delta
- → **Connection pressure**: active vs. max connections ratio, wait event accumulation rate
- → **I/O**: read/write latency percentiles (p50, p95, p99), IOPS saturation coefficient
- → **Lock contention**: lock wait time trends, deadlock frequency (rolling 1h, 24h)
- → **Query execution**: plan regression score (new execution time / baseline execution time)

Each signal is not evaluated in isolation but fed into a composite health score. The most effective early production implementation is a weighted linear model with hand-tuned weights derived from your historical incident data. More sophisticated deployments use gradient boosting models (XGBoost, LightGBM) trained on labeled failure events. Both approaches require the same foundation: a reliable, high-resolution signal pipeline.

A signal pipeline at ADE scale looks like this: database-side metrics → Prometheus exporters (sql_exporter) → Prometheus TSDB → feature extraction layer → scoring model → decision engine → action executor. Every component except the database itself is replaceable; what cannot change is the requirement for sub-minute metric resolution and durable storage of at least 90 days of history for model training.

```
graph LR
    M[DB Metrics] --> E[Prometheus Exporters]
    E --> R[Recording Rules  
rates, trends, projections]
    R --> S[Scoring Model  
P failure, time-to-failure]
    S --> D["D{Confidence Window  
N consecutive cycles > threshold}"]
    D --> L["|0.50-0.75 ambiguous| L[LLM Diagnosis  
async, off critical path]"]
    L --> A["|threshold met| A[Action Executor]"]
    A --> N[Notify / Mitigate / Escalate / Failover]
```

The predictive pipeline: trends are derived at collection time, the confidence window guards against acting on a single noisy cycle, and LLM reasoning is reserved for the ambiguous middle band rather than every cycle.

Building the Failure Signal Pipeline for SQL Server

Before prediction, you need instrumentation. The sql_exporter alone is insufficient for predictive work — it exposes point-in-time snapshots. You need derived metrics: rates, deltas, moving averages, and percentiles computed over time windows. Prometheus recording rules are the right place to compute these at collection time rather than at query time.

The signal surface below is what the scoring model consumes. Each query is designed to be cheap enough to run every 60 seconds via sql_exporter custom collectors:

SQL Server — base signal queries:

```

-- Replication/AG lag and synchronization health
SELECT
    ag.name                                AS ag_name,
    ar.replica_server_name,
    drs.database_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.log_send_queue_size                AS log_send_queue_kb,
    drs.log_send_rate                     AS log_send_rate_kbps,
    drs.redo_queue_size                   AS redo_queue_kb,
    drs.redo_rate                         AS redo_rate_kbps,
    -- Derived: seconds of data at risk
    CASE WHEN drs.redo_rate > 0
        THEN drs.redo_queue_size / drs.redo_rate
        ELSE NULL
    END                                  AS estimated_lag_seconds,
    drs.last_redone_time,
    drs.last_commit_time
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas          ar ON ar.replica_id = drs.replica_id
JOIN sys.availability_groups            ag ON ag.group_id   = ar.group_id
WHERE drs.is_local = 0;

-- Buffer pool pressure and memory grant signal
SELECT
    counter_name,
    cntr_value
FROM sys.dm_os_performance_counters
WHERE object_name LIKE '%Buffer Manager%'
   OR object_name LIKE '%Memory Manager%'
   OR counter_name IN (
        'Buffer cache hit ratio',
        'Page life expectancy',
        'Memory Grants Pending',
        'Target Server Memory (KB)',
        'Total Server Memory (KB)'
    );

```

The Prometheus recording rules that derive rate and trend signals belong in a dedicated `predictive_rules.yml` file, not mixed with alerting rules:

```
# /etc/prometheus/rules/predictive_rules.yml
groups:
- name: predictive_signals
  interval: 60s
  rules:
    # Disk fill rate: bytes per second, averaged over 30 minutes
    - record: db:disk_fill_rate_bytes_per_second:avg30m
      expr: >
        rate(node_filesystem_size_bytes{mountpoint="/var/opt/mssql"}[30m])
        - rate(node_filesystem_free_bytes{mountpoint="/var/opt/mssql"}[30m])

    # Projected seconds until disk exhaustion
    - record: db:disk_exhaustion_seconds:predicted
      expr: >
        node_filesystem_free_bytes{mountpoint="/var/opt/mssql"}
        / clamp_min(db:disk_fill_rate_bytes_per_second:avg30m, 0.001)

    # AG redo queue acceleration: is the redo queue growing faster than last hour?
    # (metric name depends on the sql_exporter HADR collector in use)
    - record: sqlserver:ag_redo_queue_acceleration:ratio
      expr: >
        rate(sqlserver_hadr_redo_queue_kb[10m]) /
        clamp_min(rate(sqlserver_hadr_redo_queue_kb[1h]), 0.001)

    # PLE trend for SQL Server nodes (via sql_exporter)
    - record: sqlserver:ple_trend:delta15m
      expr: >
        sqlserver_performance{counter="Page life expectancy"}
        - sqlserver_performance{counter="Page life expectancy"} offset 15m
```

With these signals flowing, the prediction layer has the raw material it needs. The critical design principle: **derive at collection time, not at alert evaluation time**. Recording rules precompute trends continuously; alerting rules and prediction models consume them. This separates signal quality from alert latency.

Failure Pattern Classification and Model Architecture

Historical failure events are the training data. Every production incident — disk exhaustion, OOM kill, AG failover, replication disconnect, deadlock storm — leaves a signature in the metric stream for the 30–120 minutes preceding failure. The predictive model's job is to recognize that signature early enough to act.

The practical model taxonomy for database failure prediction:

Time-to-failure regression: Given current signal values and their recent trends, predict the number of minutes until a failure condition. Used for disk exhaustion, connection pool saturation, tablespace fill. Straightforward to implement with linear regression on rate signals.

Anomaly-based classification: Build a baseline of "normal" metric distributions during healthy periods. Use statistical distance (Z-score, Mahalanobis distance) or isolation forest to score how far current behavior deviates from normal. Used for query plan regressions, lock pattern changes, unusual wait event distributions.

Event sequence matching: Learn which sequences of events precede failures. "Checkpoint warnings followed by connection spike followed by lock escalation" precedes a specific class of write storm failure. Implemented with sequence pattern matching or lightweight LSTM on sliding windows.

For most production deployments starting on this path, a Python-based scoring service using scikit-learn or XGBoost is the right balance of capability and operational simplicity:

```

# failure_predictor.py – runs as a sidecar service, consumes Prometheus metrics
import time
import logging
import requests
import numpy as np
import joblib
from dataclasses import dataclass
from typing import Optional
from prometheus_client import Gauge, start_http_server

PROMETHEUS_URL = "http://prometheus:9090"
MODEL_PATH = "/models/db_failure_model.pkl"
SCORE_INTERVAL = 60 # seconds

failure_score = Gauge("db_failure_probability", "Predicted failure probability [0-1]",
                      ["instance", "failure_class"])
time_to_failure = Gauge("db_time_to_failure_minutes", "Predicted minutes to failure",
                        ["instance", "failure_class"])

@dataclass
class InstanceFeatures:
    instance: str
    disk_fill_rate: float
    disk_exhaustion_secs: float
    replication_lag_secs: float
    lag_acceleration: float
    buffer_hit_ratio: float
    active_connections_pct: float
    deadlocks_per_hour: float
    ple_trend: float # SQL Server: Page Life Expectancy delta

def query_prometheus(promql: str) -> Optional[float]:
    try:
        r = requests.get(f"{PROMETHEUS_URL}/api/v1/query",
                        params={"query": promql}, timeout=5)
        r.raise_for_status()
        result = r.json()["data"]["result"]
        return float(result[0]["value"][1]) if result else None
    except Exception as e:
        logging.warning(f"Prometheus query failed: {promql[:60]} – {e}")
        return None

def collect_features(instance: str) -> Optional[InstanceFeatures]:
    label = f'instance="{instance}"'
    disk_rate =
    query_prometheus(f'db:disk_fill_rate_bytes_per_second:avg30m{{{label}}}')
    disk_exhaust =
    query_prometheus(f'db:disk_exhaustion_seconds:predicted{{{label}}}')
    rep_lag = query_prometheus(f'sqlserver_hadr_redo_queue_kb{{{label}}}')
    lag_accel =
    query_prometheus(f'sqlserver:ag_redo_queue_acceleration:ratio{{{label}}}')

```

```

hit_ratio      = query_prometheus(
    f'sqlserver_performance{{{label}},counter="Buffer cache hit ratio"}} / 100')
# configured_user_connections: static metric or recording rule sourced from
# the instance's 'user connections' configuration option
conn_pct       = query_prometheus(
    f'sqlserver_performance{{{label}},counter="User Connections"}} /
sqlserver:configured_user_connections{{{label}}}')
deadlocks      = query_prometheus(
    f'rate(sqlserver_performance{{{label}},counter="Number of Deadlocks/sec"}}
[1h]) * 3600')
ple_trend      = query_prometheus(f'sqlserver:ple_trend:delta15m{{{label}}}')

# Require core signals; impute optional ones
if any(v is None for v in [disk_rate, disk_exhaust, conn_pct]):
    return None

return InstanceFeatures(
    instance      = instance,
    disk_fill_rate = disk_rate      or 0.0,
    disk_exhaustion_secs = disk_exhaust or 86400.0,
    replication_lag_secs = rep_lag    or 0.0,
    lag_acceleration  = lag_accel     or 1.0,
    buffer_hit_ratio   = hit_ratio    or 0.99,
    active_connections_pct = conn_pct or 0.0,
    deadlocks_per_hour  = deadlocks   or 0.0,
    ple_trend          = ple_trend    or 0.0,
)

def score_instance(model, features: InstanceFeatures) -> dict:
    X = np.array([
        features.disk_fill_rate,
        features.disk_exhaustion_secs,
        features.replication_lag_secs,
        features.lag_acceleration,
        features.buffer_hit_ratio,
        features.active_connections_pct,
        features.deadlocks_per_hour,
        features.ple_trend,
    ])
    proba = model.predict_proba(X)[0][1]      # P(failure within horizon)
    ttf   = model.predict(X)[0]                # regression head: minutes
    return {"probability": float(proba), "time_to_failure_minutes": float(ttf)}

def run_prediction_loop(instances: list[str]):
    model = joblib.load(MODEL_PATH)
    start_http_server(9200)      # expose metrics for Prometheus scrape
    logging.info("Failure predictor started, exposing metrics on :9200")

    while True:
        for instance in instances:
            features = collect_features(instance)
            if features is None:
                logging.warning(f"Incomplete features for {instance}, skipping

```

```

cycle")
    continue
    scores = score_instance(model, features)
    failure_score.labels(instance=instance,

failure_class="composite").set(scores["probability"])
    time_to_failure.labels(instance=instance,
                           failure_class="composite").set(
                           scores["time_to_failure_minutes"])
    logging.info(f"{instance}: P(failure)={scores['probability']:.3f}  "
                f"TTF={scores['time_to_failure_minutes']:.1f}m")
    time.sleep(SCORE_INTERVAL)

if __name__ == "__main__":
    import sys
    logging.basicConfig(level=logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")
    instances = sys.argv[1:] # pass instance labels as CLI args
    run_prediction_loop(instances)

```

This service runs as a Kubernetes sidecar or a systemd unit alongside each database cluster. The scores it emits are first-class Prometheus metrics — they can trigger Alertmanager rules, feed Grafana dashboards, and most importantly, feed the autonomous action layer.

Autonomous Remediation Triggers and Decision Logic

A failure probability score is only useful if it connects to action. The action architecture must be explicit about three parameters for every failure class: the **trigger threshold**, the **confidence window** (how many consecutive scoring cycles must exceed the threshold before action fires), and the **action class** (notify, mitigate, escalate, failover).

Failure Class	Trigger Threshold	Confidence Window	Action
Disk exhaustion < 2h	P > 0.85	3 consecutive cycles	Automated cleanup + alert
AG redo queue acceleration	P > 0.70	5 cycles	Throttle writes + investigate
Connection saturation	P > 0.75	3 cycles	Kill idle sessions + app-tier scale-out
Memory pressure (PLE collapse)	P > 0.80	2 cycles	Query kill + cache flush
AG sync health degraded	P > 0.90	1 cycle	

Failure Class	Trigger Threshold	Confidence Window	Action
			Pre-position failover candidate

The confidence window prevents action storms from model noise. A single anomalous scoring cycle should not trigger a failover. The combination of threshold and window is your system's risk tolerance encoded as code.

The Alertmanager configuration that bridges prediction scores to the action layer:

```
# alertmanager.yml – predictive failure routing
route:
  receiver: 'null'
  routes:
    - match:
        alertname: PredictedDiskExhaustion
      receiver: autonomous_remediation
      group_wait: 0s
      repeat_interval: 10m

    - match:
        alertname: PredictedReplicationFailure
      receiver: autonomous_remediation
      group_wait: 0s
      repeat_interval: 5m

    - match:
        alertname: PredictedAGFailover
      receiver: dba_escalation
      continue: true # also fire autonomous prep action
    - match:
        alertname: PredictedAGFailover
      receiver: autonomous_remediation

receivers:
  - name: autonomous_remediation
    webhook_configs:
      - url: 'http://action-executor:8080/remediate'
        send_resolved: true
        max_alerts: 1

  - name: dba_escalation
    pagerduty_configs:
      - service_key: '<PD_KEY>'
        severity: warning
        description: '{{ .CommonAnnotations.summary }}'
```

The Prometheus alert rules that consume the prediction metrics:

SQL Server (Prometheus rules — engine-agnostic):

```
# /etc/prometheus/rules/predictive_alerts.yml
groups:
- name: predictive_failure_alerts
  rules:
    - alert: PredictedDiskExhaustion
      expr: >
        db:disk_exhaustion_seconds:predicted < 7200
        AND db:disk_fill_rate_bytes_per_second:avg30m > 0
      for: 3m
      labels:
        severity: critical
        action_class: autonomous
      annotations:
        summary: "Disk exhaustion predicted within 2h on {{ $labels.instance }}"
        runbook: "https://runbooks.internal/db/disk-exhaustion"
        ttf_minutes: "{{ $value | humanizeDuration }}"

    - alert: PredictedReplicationFailure
      expr: >
        db_failure_probability{failure_class="composite"} > 0.75
        AND pg:replication_lag_acceleration:ratio > 2.0
      for: 5m
      labels:
        severity: critical
        action_class: autonomous
      annotations:
        summary: "Replication failure predicted on {{ $labels.instance }}"

    - alert: PredictedAGFailover
      expr: >
        db_failure_probability{failure_class="composite"} > 0.90
        AND on(instance) sqlserver_hadr_replica_health == 0
      for: 1m
      labels:
        severity: critical
        action_class: pre_position_failover
      annotations:
        summary: "AG failover predicted, pre-positioning secondary on
        {{ $labels.instance }}"
```

When the action executor webhook fires, it dispatches to the appropriate remediation handler. For disk pressure, this means querying for reclaimable space (fragmented indexes, old transaction log backups, temporary files) and executing the cleanup autonomously. For replication lag under acceleration, it means identifying the heavy write workload and either throttling it or routing new write connections to a secondary. For an AG in pre-failure state, it means ensuring the target secondary is caught up, its TCP keepalive settings are tuned for

fast failover, and the monitoring system is primed for the transition — so that when the failover fires (either autonomously or by operator), it completes in seconds rather than minutes.

Integrating LLM-Based Diagnosis into the Prediction Loop

The quantitative prediction layer handles well-characterized failure modes. But production databases fail in novel ways — a new query pattern that stress-tests a previously safe code path, a schema migration that creates a lock chain topology the model has never seen, a cloud provider degrading a specific IOPS tier in a way that does not match historical disk failure signatures. For these cases, integrating an LLM reasoning layer into the prediction loop adds qualitative diagnostic depth.

The pattern is not to replace the quantitative model with an LLM. It is to escalate to LLM reasoning when the quantitative model fires with moderate confidence (P between 0.50 and 0.75) and the signals are ambiguous. At this confidence range, the human DBA would normally investigate manually. The autonomous system instead queries an LLM with the full anomalous signal context and asks it to reason about likely failure class, estimated time horizon, and recommended action.

```

# llm_diagnosis.py – escalation path for ambiguous failure signals
import os
import json
import logging
from anthropic import Anthropic
from typing import Optional

client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

SYSTEM_PROMPT = """
You are an expert database reliability engineer analyzing production database
telemetry. You will receive a JSON object containing metric signals from a
database instance over the last 30 minutes. Your job is to:

1. Identify the most likely failure mode (disk, memory, replication, connection,
   locking, or none).
2. Estimate time to failure in minutes (or 'none' if no failure is predicted).
3. Recommend the single most important autonomous action to take right now.
4. Assign an overall confidence score from 0.0 to 1.0.

Respond ONLY with valid JSON matching this schema:
{
  "failure_mode": "<disk|memory|replication|connection|locking|none>",
  "time_to_failure_minutes": <number or null>,
  "recommended_action": "<string, max 100 chars>",
  "confidence": <0.0 to 1.0>,
  "reasoning": "<string, max 300 chars>"
}
"""

def llm_diagnose(signal_context: dict) -> Optional[dict]:
    """
    Send ambiguous signal context to Claude for qualitative diagnosis.
    Returns structured diagnosis or None on failure.
    """
    try:
        message = client.messages.create(
            model = os.environ.get("ANTHROPIC_MODEL", "<configured-model-id>"), #
            set to your provider's current model
            max_tokens = 512,
            system = SYSTEM_PROMPT,
            messages = [{
                "role": "user",
                "content": json.dumps(signal_context, indent=2)
            }]
        )
        raw = message.content[0].text.strip()
        diagnosis = json.loads(raw)

        # Validate required fields before trusting the response
        required = {"failure_mode", "recommended_action", "confidence"}
        if not required.issubset(diagnosis.keys()):
            logging.warning(f"LLM response missing required fields: {raw[:200]}")
    
```



```

        return None

    return diagnosis

except json.JSONDecodeError as e:
    logging.error(f"LLM returned non-JSON response: {e}")
    return None
except Exception as e:
    logging.error(f"LLM diagnosis call failed: {e}")
    return None

def build_signal_context(instance: str, features: dict,
                        recent_events: list[str]) -> dict:
    """
    Package features and recent events into LLM context payload.
    recent_events: list of human-readable event strings from audit log,
        e.g. ["Index maintenance ran on orders table 12m ago",
              "Index rebuild started on inventory 25m ago"]
    """
    return {
        "instance":      instance,
        "timestamp_utc": features.get("timestamp"),
        "signals":       features,
        "recent_events": recent_events,
    }

```

The confidence-band escalation pattern — only invoking the LLM when the quantitative model is genuinely ambiguous — keeps the reasoning layer off the critical path for the majority of predictions, where the statistical model is confident and fast. It also keeps the system's cost and latency profile bounded: a system that calls an LLM on every scoring cycle for every instance does not scale, and most of those calls would add no diagnostic value over the quantitative score alone.

The LLM diagnosis output is treated the same way as any Tier 2 or Tier 3 action from the autonomy envelope established earlier in this book: `recommended_action` is surfaced to the action executor as a proposal, not a command. Low-risk recommended actions that match a pre-approved catalog entry (killing a specific session, running an ANALYZE, throttling a specific application's connection pool) may execute autonomously. Anything else queues for human approval with the full reasoning context attached, which is precisely the situation where a synthesized explanation saves the on-call engineer the ten minutes they would otherwise spend reconstructing the same picture from raw dashboards.

Key Takeaways

- → Predictive failure detection replaces static thresholds with trajectory-aware scoring — trend, rate of change, and projected time-to-failure — because a stable system near a threshold is a different risk than an unstable system approaching one from a distance.
- → A durable signal pipeline (metrics → recording rules computed at collection time → a scoring model → a decision engine → an action executor) is the prerequisite; deriving trends at alert-evaluation time instead of collection time couples signal quality to alert latency in ways that degrade both.
- → Every failure class needs an explicit trigger threshold, a confidence window (consecutive scoring cycles above threshold), and a bounded action class — the confidence window is what prevents a single noisy scoring cycle from triggering an irreversible action like a failover.
- → LLM reasoning belongs in the ambiguous middle band of model confidence, not on every scoring cycle — it augments the quantitative model for novel failure modes the statistical approach hasn't seen before, and its recommendations still flow through the same autonomy-tier approval gates as any other proposed action.
- → Avoid citing specific accuracy percentages, false-positive rates, or time-to-detection figures for a predictive system without your own measured data behind them — these numbers are entirely dependent on the quality and volume of labeled historical incidents in a given environment.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 12: Autonomous Backup & Restore Systems

Backup and restore have traditionally been the most human-dependent operations in database management — schedules set manually, retention policies enforced inconsistently, restore testing performed quarterly at best, and recovery procedures that exist as PDF documents nobody updates. An autonomous backup system inverts this entirely: it observes the database's change rate, transaction patterns, and business criticality in real time; decides what to back up, how frequently, and to which tiers of storage; executes and validates those backups without operator input; and can initiate restore operations — including full environment reconstruction — based on defined policy or detected corruption. This chapter covers the full architecture of autonomous backup systems: decision logic for adaptive scheduling, continuous validation pipelines, policy-driven restore automation, and the operational patterns that make these systems production-safe at scale.

Adaptive Backup Scheduling Based on Change Velocity

Static backup schedules are an artifact of the manual era. A database that processes 500 transactions per hour at 2 AM and 50,000 transactions per hour at noon has a fundamentally different RPO risk profile across those windows, yet most organizations apply a uniform schedule to both. An autonomous system observes write velocity continuously and adjusts backup frequency accordingly.

The core metric is **modified page rate** and **log generation rate**. Both serve as proxies for how quickly the database is diverging from its last known-good backup. When the divergence rate is high, the cost of data loss per unit time is proportionally high, and backup frequency should increase. When the database is quiescent, backup frequency can safely decrease to preserve storage and I/O budget.

```

-- Track modified extent rate using dm_io_virtual_file_stats
-- Proxy for change velocity per database
SELECT
    DB_NAME(vfs.database_id)                AS database_name,
    SUM(vfs.io_stall_write_ms)              AS write_stall_ms,
    SUM(vfs.num_of_bytes_written)           AS bytes_written,
    SUM(vfs.num_of_writes)                  AS write_ops,
    CASE
        WHEN SUM(vfs.num_of_bytes_written) > 5368709120 THEN 'HIGH'
        WHEN SUM(vfs.num_of_bytes_written) > 1073741824 THEN 'MEDIUM'
        WHEN SUM(vfs.num_of_bytes_written) > 104857600  THEN 'LOW'
        ELSE 'QUIESCENT'
    END                                     AS velocity_band,
    GETUTCDATE()                           AS sampled_at
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
GROUP BY vfs.database_id
ORDER BY bytes_written DESC;

```

The scheduler agent consumes this velocity data and maps it to a backup tier decision table:

Velocity Band	Full Backup	Incremental / Differential	Log Backup
HIGH	Weekly	Every 2 hours	Every 5 minutes
MEDIUM	Weekly	Every 6 hours	Every 15 minutes
LOW	Weekly	Daily	Every 30 minutes
QUIESCENT	Bi-weekly	Weekly	Hourly

This table is not static configuration — the scheduler agent recalculates it against observed RPO breach probability. If a HIGH-velocity period has been sustained for more than four hours (suggesting a bulk load or migration rather than a transient spike), the scheduler may promote to a more aggressive log backup cadence or trigger an opportunistic incremental.

The Python scheduler that drives this loop runs as a Kubernetes CronJob with a 1-minute resolution. It reads the velocity band, compares against the last backup timestamp for each tier, and emits backup jobs to a task queue:

```

import pyodbc
import redis
import json
from datetime import datetime, timezone, timedelta
from dataclasses import dataclass, asdict
from enum import Enum

class VelocityBand(Enum):
    HIGH = "HIGH"
    MEDIUM = "MEDIUM"
    LOW = "LOW"
    QUIESCENT = "QUIESCENT"

@dataclass
class BackupDecision:
    database: str
    backup_type: str          # full | incremental | log
    priority: int             # 1=immediate, 2=scheduled, 3=deferred
    reason: str
    triggered_at: str

POLICY = {
    VelocityBand.HIGH:      {"log_interval_min": 5, "incr_interval_hr": 2},
    VelocityBand.MEDIUM:   {"log_interval_min": 15, "incr_interval_hr": 6},
    VelocityBand.LOW:       {"log_interval_min": 30, "incr_interval_hr": 24},
    VelocityBand.QUIESCENT: {"log_interval_min": 60, "incr_interval_hr": 168},
}

def get_velocity_band(conn, database: str) -> VelocityBand:
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT TOP 1 velocity_band FROM backup_monitor.log_velocity
            WHERE recorded_at > DATEADD(MINUTE, -10, SYSUTCDATETIME())
            ORDER BY recorded_at DESC
            """
        )
        row = cur.fetchone()
        return VelocityBand(row[0]) if row else VelocityBand.QUIESCENT

def get_last_backup_time(redis_client, database: str, backup_type: str) -> datetime:
    key = f"backup:last:{database}:{backup_type}"
    val = redis_client.get(key)
    if val:
        return datetime.fromisoformat(val.decode())
    return datetime.min.replace(tzinfo=timezone.utc)

def evaluate_backup_need(
    database: str,
    band: VelocityBand,
    last_log: datetime,
    last_incr: datetime,
) -> list[BackupDecision]:

```

```

decisions = []
policy = POLICY[band]
now = datetime.now(timezone.utc)

if (now - last_log) > timedelta(minutes=policy["log_interval_min"]):
    decisions.append(BackupDecision(
        database=database,
        backup_type="log",
        priority=1 if band == VelocityBand.HIGH else 2,
        reason=f"Log interval exceeded for band {band.value}",
        triggered_at=now.isoformat(),
    ))

if (now - last_incr) > timedelta(hours=policy["incr_interval_hr"]):
    decisions.append(BackupDecision(
        database=database,
        backup_type="incremental",
        priority=2,
        reason=f"Incremental interval exceeded for band {band.value}",
        triggered_at=now.isoformat(),
    ))

return decisions

def run_scheduler(mssql_dsn: str, redis_url: str, databases: list[str]):
    conn = pyodbc.connect(mssql_dsn)
    r = redis.from_url(redis_url)
    queue = r.pipeline()

    for db in databases:
        band = get_velocity_band(conn, db)
        last_log = get_last_backup_time(r, db, "log")
        last_incr = get_last_backup_time(r, db, "incremental")
        decisions = evaluate_backup_need(db, band, last_log, last_incr)

        for decision in decisions:
            queue.rpush("backup:jobs", json.dumps(asdict(decision)))

    queue.execute()
    conn.close()

```

This loop runs on a 1-minute Kubernetes CronJob schedule. The actual backup execution is decoupled into a separate worker pool, allowing the scheduler to remain lightweight and the backup workers to scale independently under load.

Continuous Backup Validation Without Human Review

A backup that has never been tested is not a backup — it is a hope. Most organizations test restore capability infrequently because the process requires human coordination: spin up a test environment, restore, verify, tear down. An autonomous system eliminates that coordination dependency entirely by running a continuous validation pipeline in the background, generating an auditable truth record of every backup's restorability.

The validation pipeline has three stages:

1. **Structural integrity check** — immediately after backup completion, verify that the backup files are not corrupted and that the catalog metadata is consistent.
2. **Restore smoke test** — periodically, restore the backup to an isolated environment (a throwaway container or VM) and run a lightweight probe query suite to confirm the data is readable and internally consistent.
3. **Application-level consistency check** — run a set of business logic assertions against the restored database to confirm that referential integrity, row counts in key tables, and sequence values are within expected bounds.

The first stage runs synchronously after every backup. The second and third stages run asynchronously in a dedicated validation namespace in Kubernetes.

```
graph LR
    A[Backup Completes] --> B[Structural Integrity Check  
synchronous]
    B -->|catalog: INTEGRITY_OK| C[Restore Smoke Test  
ephemeral container]
    C -->|catalog: RESTORE_OK| D[Application Consistency Check  
business assertions]
    D -->|catalog: APP_OK| E[Backup Marked Trusted]
    B -->|checksum mismatch| F[FAILED: emergency backup + page on-call]
    C -->|probe query fails| F
    D -->|assertion fails| F
```

The three-stage validation pipeline: each backup advances through synchronous integrity checking and two asynchronous restore-based checks before it is trusted for use in a restore plan.

```

-- SQL Server backup validation catalog
-- Track RESTORE VERIFYONLY results and restore smoke test outcomes
CREATE TABLE backup_catalog.BackupValidations (
    ValidationId    UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    BackupSetId     INT                NOT NULL,    -- from msdb.dbo.backupset
    DatabaseName    SYSNAME            NOT NULL,
    BackupType      VARCHAR(20)        NOT NULL,    -- Full | Differential | Log
    BackupStartDate DATETIME2          NOT NULL,
    BackupFinishDate DATETIME2         NOT NULL,
    PhysicalDeviceName NVARCHAR(512) NOT NULL,
    ValidationState VARCHAR(20)        NOT NULL DEFAULT 'PENDING',
    VerifyOnly_Passed BIT              NULL,
    RestoreSmoke_Passed BIT            NULL,
    AppConsistency_Passed BIT          NULL,
    ValidatedAt     DATETIME2          NULL,
    ValidationMessage NVARCHAR(MAX)    NULL
);

-- Populate from msdb on backup completion (called by SQL Agent job)
INSERT INTO backup_catalog.BackupValidations (
    BackupSetId, DatabaseName, BackupType,
    BackupStartDate, BackupFinishDate, PhysicalDeviceName
)
SELECT
    bs.backup_set_id,
    bs.database_name,
    CASE bs.type
        WHEN 'D' THEN 'Full'
        WHEN 'I' THEN 'Differential'
        WHEN 'L' THEN 'Log'
    END,
    bs.backup_start_date,
    bs.backup_finish_date,
    bmf.physical_device_name
FROM msdb.dbo.backupset bs
JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.backup_finish_date > DATEADD(MINUTE, -5, GETUTCDATE())
    AND bs.database_name = @DatabaseName;

```

The restore smoke test uses ephemeral containers managed by a Kubernetes operator. The operator watches the `backup_catalog.backups` table (or its SQL Server equivalent) for entries in `INTEGRITY_OK` state and schedules a restore pod:


```

import subprocess
import pyodbc
import logging

logger = logging.getLogger("backup_validator")

SMOKE_TEST_QUERIES = [
    # Verify system catalog is readable
    "SELECT COUNT(*) FROM sys.tables;",
    # Verify physical stats are readable (catches page-level corruption)
    "SELECT COUNT(*) FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED');",
    # Verify user data is present
    "SELECT SUM(row_count) FROM sys.dm_db_partition_stats WHERE index_id IN (0, 1);",
]

# Host of the ephemeral SQL Server used for restore validation
# (containerized instance managed by the Kubernetes operator)
EPHEMERAL_SQL_HOST = "sql-verify.default.svc.cluster.local"

def restore_to_ephemeral(backup_path: str, verify_db: str) -> bool:
    """Restore a native backup to an ephemeral SQL Server database, then CHECKDB it."""
    result = subprocess.run(
        [
            "sqlcmd", "-S", EPHEMERAL_SQL_HOST, "-E", "-b",
            "-Q",
            f"RESTORE DATABASE [{verify_db}] FROM DISK = N'{backup_path}' "
            f"WITH MOVE 'data' TO '/var/opt/mssql/data/{verify_db}.mdf', "
            f"MOVE 'log' TO '/var/opt/mssql/data/{verify_db}_log.ldf', "
            "REPLACE, STATS = 10;",
        ],
        capture_output=True,
        text=True,
        timeout=900,
    )

    if result.returncode != 0:
        logger.error("Restore failed: %s", result.stderr)
        return False

    # DBCC CHECKDB catches page-level corruption the smoke tests cannot see
    dbcc = subprocess.run(
        [
            "sqlcmd", "-S", EPHEMERAL_SQL_HOST, "-E", "-b",
            "-Q", f"DBCC CHECKDB ([{verify_db}]) WITH NO_INFOMSGS;",
        ],
        capture_output=True,
        text=True,
        timeout=900,
    )

    if dbcc.returncode != 0:
        logger.error("CHECKDB failed: %s", dbcc.stderr)
    return dbcc.returncode == 0

```

```

def run_smoke_tests(verify_db: str, database: str) -> tuple[bool, str]:
    """Execute smoke test queries against the restored database."""
    try:
        conn = pyodbc.connect(
            "DRIVER={ODBC Driver 18 for SQL Server};"
            f"SERVER={EPHEMERAL_SQL_HOST};DATABASE={verify_db};"
            "Trusted_Connection=yes;",
            timeout=10,
        )
        with conn.cursor() as cur:
            for query in SMOKE_TEST_QUERIES:
                cur.execute(query)
                result = cur.fetchone()
                if result is None or result[0] == 0:
                    return False, f"Smoke test failed: zero rows from [{query}]"
        conn.close()
        return True, "All smoke tests passed"
    except Exception as e:
        return False, f"Connection or query error: {e}"

def drop_ephemeral(verify_db: str):
    subprocess.run(
        ["sqlcmd", "-S", EPHEMERAL_SQL_HOST, "-E", "-b",
        "-Q", f"ALTER DATABASE [{verify_db}] SET SINGLE_USER WITH ROLLBACK
IMMEDIATE; ",
        f"DROP DATABASE [{verify_db}];"],
        capture_output=True,
        timeout=300,
    )

def validate_backup(backup_id: str, backup_path: str, database: str,
                    catalog_dsn: str):
    verify_db = f"{database}_verify_{backup_id[:8]}"
    try:
        restored = restore_to_ephemeral(backup_path, verify_db)
        if not restored:
            _update_catalog(catalog_dsn, backup_id, "FAILED", "Restore failed")
            return

        passed, message = run_smoke_tests(verify_db, database)
        state = "RESTORE_OK" if passed else "FAILED"
        _update_catalog(catalog_dsn, backup_id, state, message)
    finally:
        drop_ephemeral(verify_db)

def _update_catalog(dsn: str, backup_id: str, state: str, message: str):
    conn = pyodbc.connect(dsn)
    with conn.cursor() as cur:
        cur.execute(
            """
            UPDATE backup_catalog.backups
            SET validation_state = ?,

```

```

        validation_message = ?,
        validated_at = SYSUTCDATETIME()
    WHERE backup_id = ?
    """
    (state, message, backup_id),
)
conn.commit()
conn.close()

```

When a backup fails validation, the system does not simply log the failure. It immediately triggers a fresh backup of the same type, alerts the on-call channel through PagerDuty or Alertmanager, and suppresses all automated restore operations that would have consumed the failed backup. The decision tree is: **if validation_state = 'FAILED' and backup_type = 'full', trigger emergency full backup and page; if backup_type = 'log', trigger log backup retry and record the gap in the catalog for PITR range invalidation.**

Policy-Driven Retention and Tiered Storage Lifecycle

Retention management is where backup systems accumulate the most hidden cost and compliance risk simultaneously. Organizations over-retain backups on expensive primary storage because the rules for deletion are unclear or the deletion process requires human approval. An autonomous system encodes retention policy as machine-executable rules and enforces them on a continuous basis.

Retention policy in an autonomous system has four dimensions:

- → **Time-based retention:** Keep backups for N days regardless of count
- → **Count-based retention:** Keep the last N full backups regardless of age
- → **RPO-coverage retention:** Keep enough logs to satisfy any point-in-time restore within the defined RPO window
- → **Compliance anchor retention:** Certain backups (e.g., end-of-month full backups) are locked and exempt from normal expiry rules

The storage lifecycle manager evaluates each backup against all four dimensions. A backup is eligible for deletion only when it fails all four retention tests. The tier migration logic runs separately: backups older than 7 days move from NVMe-backed primary storage to object storage (S3, Azure Blob, or GCS); backups older than 30 days move to archive storage classes (Glacier, Archive).

```

-- SQL Server: identify backup sets eligible for deletion
-- Respects time, count, and log chain continuity constraints
WITH RetentionPolicy AS (
    SELECT
        retain_days          = 30,
        retain_full_count    = 4,
        pitr_window_hours    = 72
),
BackupRanked AS (
    SELECT
        bs.backup_set_id,
        bs.database_name,
        bs.backup_start_date,
        bs.backup_finish_date,
        bs.type               AS backup_type,
        bmf.physical_device_name,
        ROW_NUMBER() OVER (
            PARTITION BY bs.database_name, bs.type
            ORDER BY bs.backup_finish_date DESC
        )                     AS recency_rank
    FROM msdb.dbo.backupset bs
    JOIN msdb.dbo.backupmediafamily bmf
        ON bs.media_set_id = bmf.media_set_id
    WHERE bs.database_name = @DatabaseName
        AND bs.is_copy_only = 0
),
RetentionDecisions AS (
    SELECT
        br.*,
        CASE
            WHEN br.backup_finish_date > DATEADD(
                DAY, -rp.retain_days, GETUTCDATE()
            )
            THEN 'RETAINED_TIME'
            WHEN br.recency_rank <= rp.retain_full_count THEN 'RETAINED_COUNT'
            WHEN br.backup_type = 'L'
                AND br.backup_finish_date > DATEADD(
                    HOUR, -rp.pitr_window_hours, GETUTCDATE()
                )
            THEN 'RETAINED_PITR'
            ELSE 'ELIGIBLE_FOR_DELETION'
        END AS RetentionDecision
    FROM BackupRanked br
    CROSS JOIN RetentionPolicy rp
)
SELECT
    backup_set_id,
    database_name,
    backup_type,
    backup_finish_date,
    physical_device_name,
    RetentionDecision
FROM RetentionDecisions

```

```
WHERE RetentionDecision = 'ELIGIBLE_FOR_DELETION'  
ORDER BY backup_finish_date ASC;
```

The storage lifecycle manager runs this query, then for each eligible backup calls the appropriate object storage API to delete the backing files and removes the catalog entry. Deletion is always two-phase: the catalog entry is marked `DELETION_PENDING` first, then the storage objects are removed, then the catalog entry is hard-deleted. If the storage deletion fails, the catalog entry reverts to its previous state and an alert is raised. This prevents catalog-storage divergence — the failure mode where the catalog believes a backup has been deleted but the storage objects remain, consuming cost invisibly.

Tier migration follows a complementary pattern:

```

import boto3
import pyodbc
from datetime import datetime, timezone, timedelta
from dataclasses import dataclass

@dataclass
class TierMigrationRule:
    min_age_days: int
    target_storage_class: str    # e.g. "STANDARD_IA", "GLACIER", "DEEP_ARCHIVE"
    source_storage_class: str

TIER_RULES = [
    TierMigrationRule(7, "STANDARD_IA", "STANDARD"),
    TierMigrationRule(30, "GLACIER", "STANDARD_IA"),
    TierMigrationRule(90, "DEEP_ARCHIVE", "GLACIER"),
]

def migrate_backup_tier(
    s3_client,
    bucket: str,
    key: str,
    target_class: str,
    backup_id: str,
    catalog_dsn: str,
):
    """Move a backup object to a cheaper S3 storage class via copy-in-place."""
    s3_client.copy_object(
        Bucket=bucket,
        CopySource={"Bucket": bucket, "Key": key},
        Key=key,
        StorageClass=target_class,
        MetadataDirective="COPY",
    )
    # Update catalog to reflect new tier
    conn = pyodbc.connect(catalog_dsn)
    with conn.cursor() as cur:
        cur.execute(
            """
            UPDATE backup_catalog.backups
            SET storage_tier = ?,
                tier_migrated_at = SYSUTCDATETIME()
            WHERE backup_id = ?
            """,
            (target_class, backup_id),
        )
    conn.commit()
    conn.close()

def run_tier_lifecycle(catalog_dsn: str, bucket: str, aws_region: str):
    s3 = boto3.client("s3", region_name=aws_region)
    conn = pyodbc.connect(catalog_dsn)
    now = datetime.now(timezone.utc)

```

```

for rule in TIER_RULES:
    cutoff = now - timedelta(days=rule.min_age_days)
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT backup_id, storage_path, storage_tier
            FROM backup_catalog.backups
            WHERE completed_at < ?
              AND storage_tier = ?
              AND validation_state NOT IN ('FAILED', 'PENDING',
'DELETION_PENDING')
            """,
            (cutoff, rule.source_storage_class),
        )
        rows = cur.fetchall()

    for backup_id, storage_path, current_tier in rows:
        # storage_path is stored as s3://bucket/key
        key = storage_path.replace(f"s3://{bucket}/", "")
        try:
            migrate_backup_tier(
                s3, bucket, key,
                rule.target_storage_class,
                backup_id, catalog_dsn,
            )
        except Exception as e:
            # Log and continue – don't let one failure block the rest
            logger.error(
                "Tier migration failed for backup %s: %s", backup_id, e
            )

conn.close()

```

This lifecycle runner executes daily. Because it is idempotent — re-running it on a backup already in the correct tier results in a no-op from S3's perspective and a no-change update in the catalog — it is safe to run more frequently during catch-up scenarios without risk of double-migration charges.

Automated Point-in-Time Restore Orchestration

Restore operations are where the gap between backup capability and operational confidence is widest. Teams that have good backups often still take hours to recover because the restore procedure involves manual decisions: which backup set to use, how far forward to replay logs, whether to restore in place or to a parallel environment, and how to validate completeness before cutting over traffic. An autonomous restore system encodes all of those decisions as policy and executes them without human input for declared emergency scenarios, or with a single human approval step for routine restores.

The restore orchestrator maintains a **restore plan catalog** — pre-computed restore recipes for common scenarios. These plans are re-evaluated every time a new backup completes, so the plan is always current. When a restore is triggered, the orchestrator selects the appropriate plan rather than computing it under pressure.


```

-- SQL Server: compute restore sequence for a given target time
-- Returns ordered list of backup sets to apply
WITH FullBackups AS (
    SELECT TOP 1
        bs.backup_set_id,
        bs.backup_start_date,
        bs.backup_finish_date,
        bs.first_lsn,
        bs.last_lsn
    FROM msdb.dbo.backupset bs
    WHERE bs.database_name      = @DatabaseName
        AND bs.type              = 'D'    -- Full
        AND bs.backup_finish_date <= @TargetTime
        AND bs.is_copy_only      = 0
    ORDER BY bs.backup_finish_date DESC
),
DifferentialBackups AS (
    SELECT TOP 1
        bs.backup_set_id,
        bs.backup_start_date,
        bs.backup_finish_date,
        bs.first_lsn,
        bs.last_lsn,
        bs.database_backup_lsn
    FROM msdb.dbo.backupset bs
    JOIN FullBackups fb
        ON bs.database_backup_lsn BETWEEN fb.first_lsn AND fb.last_lsn
    WHERE bs.database_name      = @DatabaseName
        AND bs.type              = 'I'    -- Differential
        AND bs.backup_finish_date <= @TargetTime
        AND bs.is_copy_only      = 0
    ORDER BY bs.backup_finish_date DESC
),
LogChain AS (
    SELECT
        bs.backup_set_id,
        bs.backup_start_date,
        bs.backup_finish_date,
        bs.first_lsn,
        bs.last_lsn,
        ROW_NUMBER() OVER (ORDER BY bs.first_lsn) AS apply_order
    FROM msdb.dbo.backupset bs
    CROSS JOIN (
        SELECT COALESCE(
            (SELECT last_lsn FROM DifferentialBackups),
            (SELECT last_lsn FROM FullBackups)
        ) AS chain_start_lsn
    ) base
    WHERE bs.database_name      = @DatabaseName
        AND bs.type              = 'L'    -- Log
        AND bs.first_lsn        >= base.chain_start_lsn
        AND bs.backup_finish_date <= @TargetTime
        AND bs.is_copy_only      = 0

```

```

        ORDER BY bs.first_lsn
    )
    -- Return the complete ordered restore sequence
    SELECT 'Full'          AS step_type, backup_set_id, backup_finish_date, 1 AS
    apply_order
    FROM FullBackups
    UNION ALL
    SELECT 'Differential', backup_set_id, backup_finish_date, 2
    FROM DifferentialBackups
    UNION ALL
    SELECT 'Log',          backup_set_id, backup_finish_date, 2 + apply_order
    FROM LogChain
    ORDER BY apply_order;

```

The orchestrator consumes the restore plan and drives the actual restore execution. The execution wrapper below drives native RESTORE statements through sqlcmd:

```

import subprocess
import pyodbc
import time
import logging
from dataclasses import dataclass
from typing import Optional

logger = logging.getLogger("restore_orchestrator")

# SQL Server instance that receives restores (staging target)
TARGET_SQL_HOST = "sql-primary.default.svc.cluster.local"

@dataclass
class RestoreRequest:
    database: str
    plan_type: str          # 'latest' | 'pitr' | 'labeled'
    target_time: Optional[str] = None    # ISO 8601 for pitr
    target_lsn: Optional[str] = None
    restore_db_name: Optional[str] = None # defaults to _restore
    dry_run: bool = False

def fetch_restore_plan(catalog_dsn: str, request: RestoreRequest) -> dict:
    conn = pyodbc.connect(catalog_dsn)
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT TOP 1
                rp.plan_id,
                rp.base_backup_id,
                rp.log_sequence,
                rp.target_time,
                rp.target_lsn,
                b.storage_path AS base_storage_path
            FROM backup_catalog.restore_plans rp
            JOIN backup_catalog.backups b ON b.backup_id = rp.base_backup_id
            WHERE rp.database_name = ?
                AND rp.plan_type    = ?
                AND rp.is_current    = 1
            ORDER BY rp.computed_at DESC
            """,
            (request.database, request.plan_type),
        )
        row = cur.fetchone()
    conn.close()

    if not row:
        raise RuntimeError(
            f"No current restore plan found for {request.database} / {request.plan_type}"
        )

    return {
        "plan_id": str(row[0]),
    }

```

```

        "base_backup_id": str(row[1]),
        "log_sequence": row[2],
        "target_time": row[3],
        "target_lsn": row[4],
        "base_storage_path": row[5],
    }

def execute_native_restore(
    plan: dict,
    request: RestoreRequest,
) -> bool:
    restore_db = request.restore_db_name or f"{request.database}_restore"
    statements = [
        # Base full backup lands with NORECOVERY so the log chain can follow
        f"RESTORE DATABASE [{restore_db}] FROM DISK = N'{plan['base_storage_path']}'"
        "
        "WITH NORECOVERY, REPLACE, STATS = 10;",
    ]

    # Apply the log chain captured in the plan (pre-computed at backup time)
    for log_path in plan.get("log_sequence", []):
        statements.append(
            f"RESTORE LOG [{restore_db}] FROM DISK = N'{log_path}' WITH NORECOVERY,
STATS = 10;"
        )

    # Final recovery – STOPAT pins the point-in-time target
    if request.plan_type == "pitr" and request.target_time:
        statements.append(
            f"RESTORE LOG [{restore_db}] WITH RECOVERY, STOPAT =
'{request.target_time}';"
        )
    else:
        statements.append(f"RESTORE DATABASE [{restore_db}] WITH RECOVERY;")

    if request.dry_run:
        logger.info("DRY RUN – would execute %d statements", len(statements))
        for s in statements:
            logger.info("  %s", s)
        return True

    for stmt in statements:
        logger.info("Executing: %.80s...", stmt)
        result = subprocess.run(
            ["sqlcmd", "-S", TARGET_SQL_HOST, "-E", "-b", "-Q", stmt],
            capture_output=True, text=True, timeout=3600,
        )
        if result.returncode != 0:
            logger.error("Native restore failed:
%s", result.stderr)
            return False

    logger.info("Restore completed successfully")

```

```

    return True

def record_restore_event(
    catalog_dsn: str,
    plan_id: str,
    request: RestoreRequest,
    success: bool,
    duration_seconds: float,
    message: str,
):
    conn = pyodbc.connect(catalog_dsn)
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO backup_catalog.restore_events (
                plan_id, database_name, plan_type, target_time,
                restore_db_name, success, duration_seconds, message, executed_at
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, SYSUTCDATETIME())
            """,
            (
                plan_id, request.database, request.plan_type,
                request.target_time,
                request.restore_db_name or f"{request.database}_restore",
                success, duration_seconds, message,
            ),
        )
    conn.commit()
    conn.close()

def orchestrate_restore(
    catalog_dsn: str,
    request: RestoreRequest,
):
    start = time.monotonic()
    plan = fetch_restore_plan(catalog_dsn, request)
    success = False
    message = ""

    try:
        success = execute_native_restore(plan, request)
        message = "Restore completed" if success else "Restore execution failed"
    except Exception as e:
        message = f"Restore orchestration error: {e}"
        logger.exception("Restore failed for %s", request.database)
    finally:
        duration = time.monotonic() - start
        record_restore_event(
            catalog_dsn, plan["plan_id"], request,
            success, duration, message,
        )

    return success

```

The key design choice here is the separation of **plan computation** from **plan execution**. Plans are computed continuously as backups complete, so when a restore is triggered — whether by an operator, an automated corruption detector, or a failover controller — the system is not computing a restore sequence under pressure. It is simply retrieving a pre-validated recipe and executing it.

Corruption Detection and Autonomous Recovery Triggers

Backup systems that wait for humans to declare a disaster will always respond more slowly than systems that monitor for corruption signals continuously and trigger recovery automatically. Autonomous corruption detection runs at two levels: **storage-level checksums** (detecting silent data corruption in backup files) and **application-level anomaly detection** (detecting logical corruption in the live database before it propagates into backups).

At the storage level, every backup object is stored with a SHA-256 checksum recorded in the backup catalog. The integrity monitor re-verifies checksums on a rolling schedule — not just at backup creation time. This catches bit-rot in object storage, failed disk sectors on backup media, and silent data modification by malicious processes.

At the application level, the corruption detector watches for anomalies in live database metrics that suggest logical corruption: sudden drops in row counts in critical tables, index-to-table count divergence, and failed constraint checks. These signals arrive faster than backup validation cycles and allow the system to freeze backup activity — preventing a corrupt state from being captured in a fresh backup — while alerting operators and queuing a restore evaluation.

```

-- Application-level integrity monitor: detect row count anomalies
-- Compares current counts against a rolling baseline
WITH BaselineCounts AS (
    SELECT
        table_name,
        AVG(row_count)                                AS baseline_avg,
        STDEV(row_count)                              AS baseline_stddev
    FROM backup_catalog.RowCountHistory
    WHERE sampled_at > DATEADD(DAY, -7, GETUTCDATE())
    GROUP BY table_name
),
CurrentCounts AS (
    SELECT
        OBJECT_NAME(i.object_id)                      AS table_name,
        SUM(p.rows)                                    AS current_rows
    FROM sys.indexes i
    JOIN sys.partitions p ON i.object_id = p.object_id
                        AND i.index_id = p.index_id
    WHERE i.index_id <= 1 -- heap or clustered index only
        AND OBJECTPROPERTY(i.object_id, 'IsUserTable') = 1
    GROUP BY i.object_id
)
SELECT
    cc.table_name,
    cc.current_rows,
    bc.baseline_avg,
    bc.baseline_stddev,
    ABS(cc.current_rows - bc.baseline_avg)
    / NULLIF(bc.baseline_stddev, 0)                    AS z_score,
    CASE
        WHEN ABS(cc.current_rows - bc.baseline_avg)
            / NULLIF(bc.baseline_stddev, 0) > 5        THEN 'ANOMALY_CRITICAL'
        WHEN ABS(cc.current_rows - bc.baseline_avg)
            / NULLIF(bc.baseline_stddev, 0) > 3        THEN 'ANOMALY_WARNING'
        ELSE 'NORMAL'
    END                                                AS anomaly_status
FROM CurrentCounts cc
JOIN BaselineCounts bc ON cc.table_name = bc.table_name
WHERE ABS(cc.current_rows - bc.baseline_avg)
    / NULLIF(bc.baseline_stddev, 0) > 3
ORDER BY z_score DESC;

```

When the anomaly detector fires at `ANOMALY_CRITICAL` level, the autonomous recovery trigger evaluates two branches: if the anomaly appeared within the last log backup interval, it attempts a PITR restore to the timestamp just before the anomaly; if the anomaly has propagated across multiple log backups, it escalates to human review before proceeding. The system never performs an automated restore when it cannot confidently identify a clean recovery point — that is a safety invariant that must be preserved to prevent restore loops.

```

from enum import Enum
from datetime import datetime, timezone, timedelta
import pyodbc
import logging

logger = logging.getLogger("corruption_trigger")

class AnomalyLevel(Enum):
    NORMAL = "NORMAL"
    WARNING = "ANOMALY_WARNING"
    CRITICAL = "ANOMALY_CRITICAL"

def assess_recovery_action(
    catalog_dsn: str,
    database: str,
    anomaly_detected_at: datetime,
    anomaly_level: AnomalyLevel,
) -> dict:
    """
    Decide whether autonomous restore is safe, requires escalation,
    or should be deferred.
    Returns a dict with 'action', 'target_time', and 'reason'.
    """
    conn = pyodbc.connect(catalog_dsn)

    # Find the last clean log backup before the anomaly
    with conn.cursor() as cur:
        cur.execute(
            """
            SELECT backup_id, completed_at, last_lsn
            FROM backup_catalog.backups
            WHERE database_name = ?
            AND backup_type = 'log'
            AND completed_at < ?
            AND validation_state IN ('INTEGRITY_OK', 'RESTORE_OK', 'APP_OK')
            ORDER BY completed_at DESC
            LIMIT 1
            """,
            (database, anomaly_detected_at),
        )
        last_clean = cur.fetchone()

    conn.close()

    if last_clean is None:
        return {
            "action": "ESCALATE",
            "target_time": None,
            "reason": "No clean log backup found before anomaly – cannot determine
safe recovery point",
        }

    last_clean_time: datetime = last_clean[1]

```



```

gap = anomaly_detected_at - last_clean_time

if anomaly_level == AnomalyLevel.CRITICAL and gap < timedelta(hours=2):
    # Anomaly is recent and a clean point is available – safe to auto-restore
    return {
        "action": "AUTO_RESTORE",
        "target_time": last_clean_time.isoformat(),
        "reason": (
            f"Critical anomaly detected {gap.total_seconds()/60:.1f} minutes "
            f"after last clean backup. Restoring to "
            f"{last_clean_time.isoformat()}."
        ),
    }
elif anomaly_level == AnomalyLevel.CRITICAL and gap >= timedelta(hours=2):
    # Anomaly may have propagated into multiple log backups
    return {
        "action": "ESCALATE",
        "target_time": last_clean_time.isoformat(),
        "reason": (
            f"Critical anomaly detected but gap to clean backup is "
            f"{gap.total_seconds()/3600:.1f} hours. Manual review required."
        ),
    }
else:
    # Warning level – alert but don't restore automatically
    return {
        "action": "ALERT",
        "target_time": None,
        "reason": f"Warning-level anomaly detected. Monitoring for escalation.",
    }

```

This decision logic enforces two safety properties: the system will not restore autonomously when the clean recovery point is ambiguous, and it will not perform an in-place restore when it cannot verify that the restore will land in a state newer than the anomaly detection time. Both properties prevent the worst failure mode in automated recovery: a restore loop where corruption is repeatedly restored back into the live database.

Observability: The Backup Health Dashboard

An autonomous backup system is only as trustworthy as its observability layer. Operators need to be able to answer five questions at any time without digging through logs:

1. When was the last successful backup for each database, and is it within policy?
2. What is the current recovery point for each database?
3. Are any backups in a failed or unknown validation state?
4. What is the current storage consumption by tier, and what is the projected cost trajectory?

5. Have any restore operations occurred recently, and what was the outcome?

These five questions map directly to a health dashboard query set. The queries below are designed to run in under one second against the backup catalog, making them suitable for a Grafana dashboard with a 30-second refresh interval.

```
-- SQL Server equivalent dashboard queries
-- Query 1: backup recency and policy compliance
SELECT
    bs.database_name,
    CASE bs.type
        WHEN 'D' THEN 'full'
        WHEN 'I' THEN 'incremental'
        WHEN 'L' THEN 'log'
    END AS backup_type,
    MAX(bs.backup_finish_date) AS last_backup_at,
    DATEDIFF(MINUTE, MAX(bs.backup_finish_date), GETUTCDATE()) AS age_minutes,
    CASE
        WHEN bs.type = 'L'
            AND DATEDIFF(MINUTE, MAX(bs.backup_finish_date), GETUTCDATE()) > 30
            THEN 'POLICY_BREACH'
        WHEN bs.type = 'I'
            AND DATEDIFF(HOUR, MAX(bs.backup_finish_date), GETUTCDATE()) > 6
            THEN 'POLICY_BREACH'
        WHEN bs.type = 'D'
            AND DATEDIFF(DAY, MAX(bs.backup_finish_date), GETUTCDATE()) > 7
            THEN 'POLICY_BREACH'
        ELSE 'COMPLIANT'
    END AS policy_status
FROM msdb.dbo.backupset bs
WHERE bs.backup_finish_date > DATEADD(DAY, -30, GETUTCDATE())
    AND bs.is_copy_only = 0
GROUP BY bs.database_name, bs.type
ORDER BY bs.database_name, bs.type;

-- Query 2: current recovery point per database
SELECT
    database_name,
    MAX(backup_finish_date) AS recovery_point,
    DATEDIFF(MINUTE, MAX(backup_finish_date), GETUTCDATE()) AS rpo_lag_minutes
FROM msdb.dbo.backupset
WHERE type = 'L'
    AND is_copy_only = 0
GROUP BY database_name
ORDER BY rpo_lag_minutes DESC;
```

These dashboard queries feed directly into alerting rules. When `policy_status = 'POLICY_BREACH'` appears for any database, the alerting layer pages the on-call team. When `rpo_lag_minutes` exceeds the database's defined RPO threshold, a separate high-priority

alert fires. The observability layer is the interface through which operators maintain confidence in the autonomous system — it does not replace human judgment, but it ensures that human judgment is applied to the right signals at the right time.

Disaster Recovery Runbooks as Code

The final component of a mature autonomous backup system is the replacement of static PDF runbooks with executable recovery definitions. A runbook-as-code is a versioned, testable script that encodes the complete recovery procedure for a specific failure scenario. It is stored in source control alongside application code, reviewed in pull requests, and executed by the recovery orchestrator rather than read by a panicking engineer.

Each step carries an `on_failure` disposition: `ABORT` halts the runbook and pages the team; `ESCALATE` halts and pages with the current context attached; `ALERT_AND_CONTINUE` logs the failure and proceeds; `WARN` records the condition but does not interrupt flow. This disposition model is the key structural difference between a runbook-as-code and a simple shell script — it encodes the decision logic about what is a hard stop versus an acceptable degraded path.

The recovery framework itself maintains an execution log that captures every step's start time, duration, inputs, outputs, and final state. This log is written to the backup catalog as a `recovery_events` record and is queryable by the observability dashboard. Post-incident reviews can replay the exact sequence of actions taken, including the values of every context variable at each decision point.

Runbooks are versioned semantically: the version number in `PRIMARY_FAILURE_RUNBOOK (2.4.1)` corresponds to a git tag in the runbook repository. When a new version is deployed, the recovery framework validates that the new runbook's preconditions and postconditions are a superset of the previous version's — a regression in safety invariants requires an explicit override with a documented justification. This is the same principle applied to database schema migrations in Chapter 25, transposed to operational procedure management.

A minimal runbook-as-code for an Always On availability group failover looks like:

```

# runbooks/sqlserver_ag_failover.py
# Triggered by: AG primary replica health check fails for > 90 seconds
# Preconditions: secondary replica synchronization state is SYNCHRONIZED
# Postconditions: AG primary role moved to designated secondary

from recovery_framework import (
    Step, Precondition, Postcondition, RecoveryRunbook,
    execute_tsql, update_listener_dns, verify_ag_health,
    send_notification,
)

AG_FAILOVER_RUNBOOK = RecoveryRunbook(
    name="sqlserver_ag_failover",
    version="1.3.0",
    description="Automated AG failover when primary replica is unavailable",
    max_execution_minutes=15,

    preconditions=[
        Precondition(
            name="secondary_is_synchronized",
            check=lambda ctx: ctx.secondary_synchronization_state == "SYNCHRONIZED",
            failure_action="ESCALATE",
        ),
        Precondition(
            name="quorum_is_healthy",
            check=lambda ctx: ctx.wsfc_quorum_state in ("Normal", "NormalQuorum"),
            failure_action="ESCALATE",
        ),
    ],

    steps=[
        Step(
            name="force_failover_with_data_loss_check",
            action=execute_tsql,
            args={
                "server": "{{secondary_replica_server}}",
                "sql": (
                    "ALTER AVAILABILITY GROUP [{ag_name}] "
                    "FAILOVER;"
                ),
            },
            timeout_seconds=120,
            on_failure="ABORT",
        ),
        Step(
            name="verify_new_primary_role",
            action=execute_tsql,
            args={
                "server": "{{secondary_replica_server}}",
                "sql": (
                    "SELECT role_desc FROM sys.dm_hadr_availability_replica_states "
                    "WHERE is_local = 1;"
                ),
            },
        ),
    ],
)

```

```

        "expected_result": "PRIMARY",
    },
    timeout_seconds=60,
    on_failure="ESCALATE",
),
Step(
    name="verify_ag_health",
    action=verify_ag_health,
    args={
        "ag_name": "{{ag_name}}",
        "server": "{{secondary_replica_server}}",
    },
    timeout_seconds=60,
    on_failure="ALERT_AND_CONTINUE",
),
Step(
    name="notify_stakeholders",
    action=send_notification,
    args={
        "channel": "{{alert_channel}}",
        "severity": "P1",
        "message": (
            "AG failover completed for {{ag_name}}. "
            "New primary: {{secondary_replica_server}}. "
            "Duration: {{elapsed_seconds}}s."
        ),
    },
    timeout_seconds=10,
    on_failure="WARN",
),
],
postconditions=[
    Postcondition(
        name="ag_databases_online",
        check=lambda ctx: ctx.ag_database_states_all_online is True,
        failure_action="ESCALATE",
    ),
    Postcondition(
        name="listener_resolves_to_new_primary",
        check=lambda ctx: ctx.listener_target == ctx.new_primary_server,
        failure_action="ALERT_AND_CONTINUE",
    ),
],
)

```

The value of this approach compounds over time. Each time a runbook executes — whether in a test environment, a chaos engineering drill, or a real incident — the execution log provides evidence about which steps succeed reliably, which steps occasionally time out, and which preconditions are frequently violated. The autonomous system uses this historical execution data to tune timeouts, reorder steps for faster resolution, and flag runbooks that

have not been exercised recently for mandatory drill scheduling. Runbooks that go untested for longer than a defined period are automatically queued for a chaos-engineering dry run in the staging environment, ensuring that the entire procedure library remains exercised and current.

Key Takeaways

- → **Backup scheduling should be demand-driven, not time-driven.** Measuring log generation rate (write byte throughput) and mapping it to a velocity band allows the scheduler to increase log backup cadence during high-change periods and reduce it during quiescent periods, keeping RPO tight where it matters without wasteful constant-rate scheduling.
- → **Every backup must be tested to the point of data readability before it is trusted.** A three-stage validation pipeline — structural integrity check, restore smoke test, and application-level consistency assertions — run automatically after every backup provides an auditable truth record of restorability and eliminates the silent failure mode where backups exist but cannot actually be restored.
- → **Retention policy is a machine-executable rule set, not a human judgment call.** Encoding time-based, count-based, PITR-coverage, and compliance-anchor dimensions into a query-driven retention engine, with two-phase deletion and catalog-storage consistency checks, removes both over-retention cost and under-retention compliance risk simultaneously.
- → **Restore plans should be pre-computed at backup time, not calculated at incident time.** Maintaining a continuously updated restore plan catalog — which base backup to use, which log sequence to apply, what the estimated restore duration is — means the orchestrator can begin execution immediately when a restore is triggered, rather than computing the sequence under operational pressure when time is most scarce.
- → **Runbooks-as-code with explicit failure dispositions are a prerequisite for safe automation.** Encoding recovery procedures as versioned, testable Python objects with per-step `on_failure` behaviors (`ABORT` , `ESCALATE` , `ALERT_AND_CONTINUE` , `WARN`) transforms static documentation into executable policy, creates an auditable execution history for post-incident review, and enables the system to automatically schedule drills for runbooks that have not been exercised within a defined window.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 13: Self-Optimizing Storage and I/O

Storage is where autonomous database engineering gets physical. Query optimizers, connection poolers, and caching layers all operate in memory and CPU — but eventually, every read and write touches a storage medium, and the behavior of that medium under load determines whether your system survives a crisis or collapses under it. The systems examined in this chapter don't wait for a storage bottleneck to appear on a dashboard. They continuously monitor I/O saturation, file system layout, tablespace utilization, buffer hit ratios, and write amplification — then act. They move hot data closer to compute, tier cold data to cheaper media, rebalance writes across devices, and restructure on-disk layout without a human ever opening a terminal. This is autonomous storage management: a discipline that sits at the intersection of operating system I/O scheduling, database internals, and feedback-driven automation.

Instrumentation Architecture: Building the I/O Observability Layer

No autonomous action is sound without high-fidelity data. Before a system can self-optimize storage, it must build a complete picture of where I/O time is actually going — across SQL Server's I/O completion ports, VDI subsystem, and filegroup allocation patterns.

The observability layer must collect at three levels simultaneously: the OS block device level, the database engine level, and the logical object level. Conflating these three creates misleading signals. A table with poor sequential scan performance might be fine at the block device level but suffering from page fragmentation. A transaction log device might show 95% utilization while the data device sits idle — indicating checkpoint tuning as the correct intervention, not storage provisioning.

```

-- Capture I/O stall distribution per file to identify hot files vs. idle files
SELECT
  DB_NAME(vfs.database_id)          AS database_name,
  mf.physical_name,
  mf.type_desc,
  vfs.io_stall_read_ms,
  vfs.io_stall_write_ms,
  vfs.io_stall,
  vfs.num_of_reads,
  vfs.num_of_writes,
  CASE
    WHEN vfs.num_of_reads = 0 THEN 0
    ELSE ROUND(vfs.io_stall_read_ms * 1.0 / vfs.num_of_reads, 2)
  END AS avg_read_ms,
  CASE
    WHEN vfs.num_of_writes = 0 THEN 0
    ELSE ROUND(vfs.io_stall_write_ms * 1.0 / vfs.num_of_writes, 2)
  END AS avg_write_ms
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf
  ON vfs.database_id = mf.database_id
  AND vfs.file_id = mf.file_id
ORDER BY vfs.io_stall DESC;

```

These queries feed a time-series collection loop. In production, a Python agent runs on each database host at 15-second intervals, writing delta values (not cumulative totals) to a Prometheus Pushgateway. The delta calculation is important — raw cumulative counters are useless for detecting transient I/O spikes that resolve within a scrape window.


```

import pyodbc
import time
import requests

PUSHGATEWAY_URL = "http://prometheus-pushgateway:9091"
SCRAPE_INTERVAL = 15 # seconds
MSSQL_DSN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=sql-primary;DATABASE=master;Trusted_Connection=yes;"
)

def collect_io_deltas(conn, prev_snapshot):
    cursor = conn.cursor()
    cursor.execute("""
        SELECT
            DB_NAME(vfs.database_id) AS database_name,
            mf.physical_name,
            vfs.num_of_reads,
            vfs.num_of_writes,
            vfs.num_of_bytes_read,
            vfs.num_of_bytes_written,
            vfs.io_stall_read_ms,
            vfs.io_stall_write_ms
        FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
        JOIN sys.master_files mf
            ON vfs.database_id = mf.database_id
            AND vfs.file_id = mf.file_id
    """)
    current = {(row[0], row[1]): row for row in cursor.fetchall()}

    metrics = []
    for key, row in current.items():
        if key in prev_snapshot:
            prev = prev_snapshot[key]
            delta_reads = row[2] - prev[2]
            delta_writes = row[3] - prev[3]
            delta_stall = (row[6] + row[7]) - (prev[6] + prev[7])
            total_io = delta_reads + delta_writes
            if total_io > 0:
                db, phys = key
                avg_stall = delta_stall / total_io
                metrics.append(
                    f'sqlserver_file_io_ops{{database="{db}",file="{phys}"}}
{total_io}'
                )
                metrics.append(
                    f'sqlserver_file_io_stall_ms{{database="{db}",file="{phys}"}}
{delta_stall:.0f}'
                )
                metrics.append(
                    f'sqlserver_file_avg_stall_ms{{database="{db}",file="{phys}"}}
{avg_stall:.2f}'
                )

```

```

    push_metrics(metrics)
    return current

def push_metrics(metrics):
    payload = "
".join(metrics) + "
"
    requests.post(
        f"{PUSHGATEWAY_URL}/metrics/job/sqlserver_io_agent",
        data=payload,
        headers={"Content-Type": "text/plain"}
    )

def run_collector():
    conn = pyodbc.connect(MSSQL_DSN)
    snapshot = {}
    while True:
        snapshot = collect_io_deltas(conn, snapshot)
        time.sleep(SCRAPE_INTERVAL)

if __name__ == "__main__":
    run_collector()

```

At the OS level, `iostat -x`, `blktrace`, and `nvme smart-log` provide device-level saturation metrics. These are collected separately and joined to database-level metrics in Grafana using host labels. The full picture — object-level, engine-level, and device-level — is what allows the autonomous decision engine to distinguish between a bad query plan, a full checkpoint storm, and a failing NVMe drive.

Autonomous Filegroup Rebalancing

Once instrumentation is in place, the first class of autonomous action involves routing data to the right storage tier. Production environments routinely have heterogeneous storage: a fast NVMe tier for active working data, a mid-tier SAS or network-attached volume for warm data, and object storage or compressed block storage for cold or archival data. Manual tiering policies degrade over time because access patterns shift — yesterday's cold archive becomes today's compliance reporting target. Autonomous tiering systems react to this drift continuously.

In SQL Server, filegroup-based tiering is the mechanism. Each filegroup maps to one or more files on specific mount points. Tables and indexes can be moved between filegroups online using `CREATE INDEX ... WITH (DROP_EXISTING = ON) ON [ColdFilegroup]` for clustered indexes, or partition switching for partitioned tables — operations the autonomous system schedules during low-traffic windows when they would contend with active workloads.

The tiering decision logic follows a straightforward heat score model. For each relation, the system computes a heat score based on recent access frequency, write volume, and age of last modification:

```
heat_score = (reads_per_hour * 0.4) + (writes_per_hour * 0.5) + (1 /
days_since_modified * 0.1)
```

Relations scoring below a configured cold threshold for seven consecutive measurement periods become candidates for demotion. Relations scoring above a hot threshold that currently reside on a slower tablespace become candidates for promotion.

```
-- Identify filegroups with disproportionate I/O load relative to data volume
SELECT
    fg.name                                AS filegroup_name,
    fg.type_desc,
    SUM(mf.size) * 8 / 1024                AS total_size_mb,
    SUM(vfs.num_of_reads)                  AS total_reads,
    SUM(vfs.num_of_writes)                 AS total_writes,
    SUM(vfs.io_stall)                      AS total_stall_ms,
    ROUND(
        SUM(vfs.io_stall) * 1.0 /
        NULLIF(SUM(vfs.num_of_reads + vfs.num_of_writes), 0),
        2)                                AS avg_stall_ms_per_op
FROM sys.filegroups fg
JOIN sys.database_files mf ON mf.data_space_id = fg.data_space_id
JOIN sys.dm_io_virtual_file_stats(DB_ID(), NULL) vfs
    ON vfs.file_id = mf.file_id
GROUP BY fg.name, fg.type_desc
ORDER BY avg_stall_ms_per_op DESC;
```

In SQL Server, the equivalent of tablespace tiering is filegroup migration. Tables can be moved between filegroups by rebuilding their clustered index on the target filegroup: `CREATE CLUSTERED INDEX ... ON table(col) WITH (DROP_EXISTING = ON) ON new_filegroup`. This operation can be performed online in Enterprise Edition, making it appropriate for autonomous execution during low-traffic windows. The autonomous agent identifies candidate objects, schedules the rebuild during a detected low-traffic window (typically identified by a rolling average of `@CPU_BUSY` and `dm_os_wait_stats` for `PAGEIOLATCH_*` waits), and executes the move.

The rebalancing agent runs as a Kubernetes CronJob with a configurable schedule. It pulls the heat score analysis from the metrics database, generates a list of recommended moves, applies a safety budget (maximum N GB of data movement per execution window), and issues the appropriate DDL. Completion is logged to an audit table, and the metrics pipeline recalculates heat scores in the next collection cycle.

Checkpoint and Write Amplification Control in SQL Server

SQL Server's write path is dominated by the checkpoint process and the lazy writer. Checkpoints flush dirty pages to disk; under automatic checkpoints their frequency is driven by log activity since the last checkpoint, while indirect checkpoints bound recovery time through the per-database target recovery time setting. When checkpoints fire too aggressively, they produce write amplification — the same data being written to disk multiple times unnecessarily — or checkpoint storms: sudden bursts of dirty page flushes that saturate the I/O subsystem and spike latency for every concurrent query.

An autonomous checkpoint controller observes `sys.dm_os_performance_counters` (checkpoint pages/sec) and per-database recovery settings continuously, and tunes `TARGET_RECOVERY_TIME` based on current write load and observed checkpoint duration. The key signal is checkpoint write volume versus I/O stall time from `sys.dm_io_virtual_file_stats`. When checkpoint I/O dominates total write I/O during burst windows, the I/O subsystem is being asked to flush dirty buffers faster than it comfortably can. The correct autonomous response is to raise `TARGET_RECOVERY_TIME` — spreading dirty-page flushes over a longer interval and trading a bounded increase in recovery time for smoother I/O.

When dirty pages accumulate faster than checkpoints drain them — visible as a rising checkpoint pages/sec rate alongside growing lazy-writes pressure — the buffer pool is holding too many modified pages between checkpoints. The correct response is to lower `TARGET_RECOVERY_TIME` so checkpoints fire more frequently and keep the dirty-page backlog bounded.

```

import pyodbc
import logging

THRESHOLDS = {
    "checkpoint_io_share": 0.60, # checkpoint writes / total writes during
burst
    "checkpoint_stall_ms": 20.0, # avg write stall ms that signals I/O pressure
}

# target_recovery_time bounds, in seconds (SQL Server default is 60)
RECOVERY_TIME_BOUNDS = {"min": 30, "max": 600, "step": 30}

def analyze_checkpoints(conn):
    cur = conn.cursor()
    cur.execute("""
        SELECT DB_NAME(vfs.database_id) AS database_name,
               SUM(vfs.num_of_bytes_written) AS bytes_written,
               SUM(vfs.io_stall_write_ms) AS write_stall_ms,
               SUM(vfs.num_of_writes) AS writes
        FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
        GROUP BY vfs.database_id
    """)
    io_by_db = {row[0]: row for row in cur.fetchall()}

    cur.execute("""
        SELECT name, target_recovery_time_in_seconds
        FROM sys.databases
        WHERE database_id > 4
    """)
    recovery_targets = {row[0]: row[1] for row in cur.fetchall()}
    return io_by_db, recovery_targets

def recommend_adjustments(io_by_db, recovery_targets, prev_io_by_db):
    recommendations = []
    for db, row in io_by_db.items():
        if db not in prev_io_by_db:
            continue
        prev = prev_io_by_db[db]
        delta_writes = row[3] - prev[3]
        delta_stall = row[2] - prev[2]
        if delta_writes == 0:
            continue
        avg_stall = delta_stall / delta_writes
        current_target = recovery_targets.get(db, 60)
        # High average write stall during a write burst = checkpoint pressure.
        # Raising target_recovery_time spreads checkpoint flushes out.
        if (avg_stall > THRESHOLDS["checkpoint_stall_ms"]
            and current_target < RECOVERY_TIME_BOUNDS["max"]):
            recommendations.append({
                "database": db,
                "parameter": "target_recovery_time",
                "direction": "increase",
                "reason": f"avg_write_stall={avg_stall:.1f}ms exceeds threshold",
            })

```

```

    })
    return recommendations

def apply_adjustment(conn, database, direction):
    cur = conn.cursor()
    cur.execute(
        "SELECT target_recovery_time_in_seconds FROM sys.databases WHERE name = ?",
        (database,),
    )
    current = cur.fetchone()[0]
    step = RECOVERY_TIME_BOUNDS["step"]
    new_value = current + step if direction == "increase" else current - step
    new_value = max(RECOVERY_TIME_BOUNDS["min"],
                    min(RECOVERY_TIME_BOUNDS["max"], new_value))
    logging.info(f"Adjusting {database}: target_recovery_time {current}s ->
{new_value}s")
    # In production: ALTER DATABASE [db] SET TARGET_RECOVERY_TIME = new_value
    SECONDS
    # Omitted here for brevity – see the configuration management module.

```

The autonomous controller does not apply all recommendations simultaneously. It implements a one-parameter-per-cycle rule: change one variable, wait two checkpoint cycles, observe the new checkpoint pages/sec rate and I/O stall trend, and decide whether to apply the next recommendation. This is a primitive form of causal isolation — changing one parameter at a time and waiting for the system to reach steady state before making another change. Violating this principle produces configuration drift where the cause of a performance improvement or regression is impossible to identify.

Autonomous Index Storage Reorganization and Bloat Remediation

Index and table bloat is one of the most predictable sources of I/O waste in long-running SQL Server deployments. Deleted rows leave ghost records in pages. Index entries accumulate for rows that no longer exist. Over time, a table that logically contains 10 million rows may span pages sized for far more — and a large share of every I/O operation retrieves ghost records that the cleanup task has not yet reclaimed.

Autonomous bloat remediation differs from a fixed maintenance schedule in a critical way: it is reactive to measured waste, not a calendar. The system continuously estimates waste using the `sys.dm_db_index_physical_stats` DMV, and it acts when waste crosses a defined threshold relative to the cost of remediation.

```
-- Identify indexes with significant fragmentation eligible for autonomous rebuild
or reorganize
SELECT
    OBJECT_NAME(ips.object_id)          AS table_name,
    i.name                              AS index_name,
    ips.index_type_desc,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.avg_page_space_used_in_percent,
    ips.record_count,
    ips.ghost_record_count,
    CASE
        WHEN ips.avg_fragmentation_in_percent > 30 THEN 'REBUILD'
        WHEN ips.avg_fragmentation_in_percent > 10 THEN 'REORGANIZE'
        ELSE 'NONE'
    END AS recommended_action
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'SAMPLED'
) ips
JOIN sys.indexes i
    ON i.object_id = ips.object_id
    AND i.index_id = ips.index_id
WHERE ips.page_count > 1000
    AND ips.avg_fragmentation_in_percent > 10
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

The remediation decision tree is more nuanced than fragmentation percentage alone. `ALTER INDEX ... REORGANIZE` is the first-line response — it compacts pages and reclaims ghost records without requiring an offline window. `ALTER INDEX ... REBUILD WITH (ONLINE = ON)` is the second-line response, reserved for cases where fragmentation has grown so severe that page count has expanded significantly beyond the logical data size. The autonomous agent calculates a waste ratio: $(\text{page_count} * 8\text{KB}) / (\text{record_count} * \text{avg_record_size_in_bytes})$. When this ratio exceeds 2.0 and REORGANIZE has not reduced it within a configured window, an online REBUILD is scheduled.

For SQL Server, the `REORGANIZE` vs `REBUILD` decision is made per-index based on fragmentation percentage and whether the index supports online rebuild. The autonomous agent constructs and executes the appropriate DDL, wrapped in a resource governor classification that limits I/O to a configurable percentage of device throughput during execution, preventing remediation from becoming its own source of I/O saturation.

A safety interlock prevents the autonomous bloat remediation system from executing during periods of high I/O utilization. The agent checks `iostat` device utilization and waits if any data device exceeds 70% utilization — using an exponential backoff with a maximum wait of 4 hours before escalating to an alert.

Partition-Based I/O Shaping and Autonomous Data Lifecycle Management

Partitioning is the most powerful tool available for shaping I/O patterns at the schema level. When a time-series table is partitioned by date, queries that filter on recent dates touch only the most recent partitions — the working set fits in buffer pool, sequential scan performance is predictable, and I/O pressure on older partitions drops to zero unless explicitly accessed. Autonomous data lifecycle management takes this further: it creates new partitions before they are needed, migrates aging partitions to cold storage, compresses or archives them on a schedule derived from observed access patterns, and detaches them from the parent table when they become permanently dormant.

The lifecycle engine manages five partition states: **active** (receiving writes and serving reads), **warm** (serving reads, no writes), **cold** (rarely accessed, eligible for compression), **archived** (moved to cold tablespace or object storage), and **detached** (removed from the partition tree, accessible only via direct query).

Autonomous partition creation is the most time-sensitive operation. A system that allows a partition to be missing during a high-write period will fail with partition scheme exhaustion. The creation agent runs hourly and ensures partitions exist for the current period plus two future periods.

```
-- Extend a partition scheme to accommodate future periods
-- Assumes partition function pfn_daily_range already exists
DECLARE @next_boundary DATE = CAST(DATEADD(DAY, 7, GETDATE()) AS DATE);

-- Add boundary points for each missing future day
DECLARE @d DATE = CAST(DATEADD(DAY, 1, GETDATE()) AS DATE);
WHILE @d <= @next_boundary
BEGIN
    -- Only add if boundary does not already exist
    IF NOT EXISTS (
        SELECT 1 FROM sys.partition_range_values prv
        JOIN sys.partition_functions pf ON pf.function_id = prv.function_id
        WHERE pf.name = 'pfn_daily_range'
            AND CAST(prv.value AS DATE) = @d
    )
    BEGIN
        -- Partition scheme must have NEXT USED filegroup designated
        ALTER PARTITION SCHEME psc_daily_range NEXT USED [warm_filegroup];
        ALTER PARTITION FUNCTION pfn_daily_range() SPLIT RANGE (@d);
        RAISERROR('Added partition boundary: %s', 0, 1, @d) WITH NOWAIT;
    END
    SET @d = DATEADD(DAY, 1, @d);
END
```


The partition lifecycle agent also handles the opposite end of the spectrum: identifying partitions that have transitioned from warm to cold based on observed access counts, and triggering compression or tablespace migration. This means switching the partition out to a staging table on the cold filegroup and applying `DATA_COMPRESSION = PAGE` before switching it back in.

```

import pyodbc
from datetime import datetime, timedelta
import logging

COLD_THRESHOLD_DAYS = 30      # partitions older than this are candidates for cold
tier
ACCESS_THRESHOLD = 10        # avg reads per collection interval below which =
cold
COLD_FILEGROUP = "ColdFG"
HOT_FILEGROUP = "HotFG"

def get_partition_access_stats(conn, partitioned_table: str, schema: str = "dbo"):
    """Return recent I/O stats for each partition of a partitioned table."""
    cur = conn.cursor()
    cur.execute("""
        SELECT
            p.partition_number,
            fg.name AS current_filegroup,
            COALESCE(AVG(m.reads_delta), 0) AS avg_reads,
            MAX(m.collected_at)           AS last_access
        FROM sys.partitions p
        JOIN sys.indexes i
            ON i.object_id = p.object_id AND i.index_id = p.index_id
        JOIN sys.partition_schemes ps
            ON ps.data_space_id = i.data_space_id
        JOIN sys.destination_data_spaces dds
            ON dds.partition_scheme_id = ps.data_space_id
            AND dds.destination_id = p.partition_number
        JOIN sys.filegroups fg
            ON fg.data_space_id = dds.data_space_id
        JOIN sys.tables t ON t.object_id = p.object_id
        JOIN sys.schemas s ON s.schema_id = t.schema_id
        LEFT JOIN io_metrics_history m
            ON m.partition_number = p.partition_number
            AND m.collected_at >= DATEADD(DAY, -7, SYSUTCDATETIME())
        WHERE t.name = ? AND s.name = ?
            AND i.index_id IN (0, 1)
        GROUP BY p.partition_number, fg.name
    """, (partitioned_table, schema))
    return cur.fetchall()

def demote_cold_partitions(conn, partitioned_table: str, schema: str = "dbo",
                           dry_run: bool = True):
    rows = get_partition_access_stats(conn, partitioned_table, schema)
    cur = conn.cursor()

    for partition_number, current_fg, avg_reads, last_access in rows:
        # Partition age is derived from the boundary values of the partition
    function
        cur.execute("""
            SELECT CAST(rv.value AS DATE)
            FROM sys.partition_range_values rv
            JOIN sys.partition_functions pf ON pf.function_id = rv.function_id

```

```

JOIN sys.indexes i ON i.data_space_id IN (
    SELECT ps.data_space_id FROM sys.partition_schemes ps
)
WHERE i.object_id = OBJECT_ID(?, 'U')
    AND rv.boundary_id = ?
"", (f"{schema}.{partitioned_table}", partition_number))
brow = cur.fetchone()
if not brow:
    continue
age_days = (datetime.utcnow().date() - brow[0]).days

if age_days >= COLD_THRESHOLD_DAYS and avg_reads < ACCESS_THRESHOLD:
    if current_fg == HOT_FILEGROUP:
        logging.info(
            f"Demoting partition {partition_number}: "
            f"age={age_days}d, avg_reads={avg_reads:.1f}"
        )
        if not dry_run:
            # SWITCH out to a staging table on the cold filegroup,
            # apply PAGE compression, SWITCH back in.
            cur.execute(
                f"ALTER TABLE {schema}.{partitioned_table} "
                f"SWITCH PARTITION {partition_number} "
                f"TO {schema}.{partitioned_table}_cold_stage "
                f"PARTITION {partition_number}"
            )
            cur.execute(
                f"ALTER INDEX ALL ON {schema}.{partitioned_table}_cold_stage "
                f" "
                f"REBUILD PARTITION = ALL "
                f"WITH (DATA_COMPRESSION = PAGE) ON [{COLD_FILEGROUP}]"
            )
            conn.commit()
            logging.info(f"Moved partition {partition_number} to
{COLD_FILEGROUP}")
        else:
            logging.debug(f"Partition {partition_number} already on
{current_fg}")
        else:
            logging.debug(
                f"Partition {partition_number}: age={age_days}d, "
                f"avg_reads={avg_reads:.1f} – no action"
            )

```

The combination of forward partition creation and backward partition demotion forms a complete data lifecycle pipeline. The system maintains a predictable working set on fast storage, prevents partition exhaustion failures, and continuously reclaims expensive NVMe capacity by moving dormant data to cheaper tiers — all without human scheduling.

Autonomous Buffer Pool Management and Prefetch Tuning

The database buffer pool is the primary mechanism for keeping frequently accessed pages in memory and avoiding physical I/O. SQL Server's buffer pool operates on an LRU-K-variant eviction policy. The engine already resizes the buffer pool dynamically within the max server memory bound, so autonomous buffer pool management focuses on shaping what competes for that memory. Autonomous buffer pool management bridges this gap.

In SQL Server, the buffer pool is managed dynamically by SQLOS within the `max server memory` bound. The autonomous layer adjusts max server memory within guardrails in response to page life expectancy trends and memory grant pressure, configures Buffer Pool Extension on fast SSD tiers as a second-level cache, and uses Resource Governor memory limits to keep runaway workloads from monopolizing the buffer pool.

```
-- SQL Server: warm the buffer pool after a partition migration by scanning
-- the newly promoted partition into memory ahead of the workload
SELECT COUNT_BIG(*)
FROM dbo.Orders_2024_11_15 WITH (INDEX(1)); -- clustered scan pulls pages into the
buffer pool
```

In SQL Server, the buffer pool is managed dynamically by the memory manager (SQLOS), and the autonomous layer can influence it through Resource Governor memory limits, `max server memory` adjustments, and Buffer Pool Extension (BPE) configuration on systems with fast SSD tiers. The more powerful lever in SQL Server is identifying which buffer pages are being evicted and why, using `sys.dm_os_buffer_descriptors`.

```

-- Identify the distribution of buffer pool pages by database and object
-- to understand what is consuming the working set
SELECT
    DB_NAME(bd.database_id)          AS database_name,
    OBJECT_NAME(p.object_id, bd.database_id) AS object_name,
    i.name                          AS index_name,
    COUNT(*)                        AS pages_in_buffer,
    COUNT(*) * 8 / 1024             AS size_in_buffer_mb,
    SUM(CAST(bd.is_modified AS INT)) AS dirty_pages
FROM sys.dm_os_buffer_descriptors bd
JOIN sys.allocation_units au
    ON bd.allocation_unit_id = au.allocation_unit_id
JOIN sys.partitions p
    ON au.container_id = CASE
        WHEN au.type IN (1, 3) THEN p.hobt_id
        WHEN au.type = 2      THEN p.partition_id
    END
JOIN sys.indexes i
    ON i.object_id = p.object_id
    AND i.index_id = p.index_id
WHERE bd.database_id NOT IN (1, 2, 3, 4) -- exclude system databases
    AND bd.page_type = 'DATA_PAGE'
GROUP BY bd.database_id, p.object_id, i.name
HAVING COUNT(*) > 100
ORDER BY pages_in_buffer DESC;

```

The autonomous prefetch tuner operates at a different level. Rather than reacting to buffer pool contents, it predicts which pages will be needed and initiates reads before the query arrives. This is useful for scheduled workloads with known access patterns — nightly reporting jobs that always scan the same partition ranges, or ETL pipelines that process data in a known order.

The prefetch system maintains a workload schedule: a table of recurring queries with their historical execution start times, observed I/O patterns (which table and partition ranges they read), and typical page counts. Before a scheduled job runs, the prefetch agent issues covering scans over the expected partition ranges ahead of the job, pulling pages into the buffer pool before the workload arrives.

```

from datetime import datetime, timedelta
import pyodbc
import schedule
import time
import logging

PREWARM_LEAD_TIME_MINUTES = 10
MSSQL_DSN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=sql-primary;DATABASE=Sales;Trusted_Connection=yes;"
)

WORKLOAD_SCHEDULE = [
    {
        "name": "nightly_revenue_report",
        "cron_time": "02:00",
        "tables_to_prewarm": [
            "dbo.Orders_2024_11_14",
            "dbo.Orders_2024_11_15",
            "dbo.Orders_2024_11_16",
        ],
    },
    {
        "name": "weekly_churn_analysis",
        "cron_time": "03:30",
        "tables_to_prewarm": [
            "dbo.CustomerEvents_2024_10",
            "dbo.CustomerEvents_2024_11",
        ],
    },
]

def prewarm_tables(conn, tables):
    cur = conn.cursor()
    for table in tables:
        try:
            # Covering scan pulls the table's pages into the buffer pool
            cur.execute(f"SELECT COUNT_BIG(*) FROM {table} WITH (INDEX(1))")
            pages = cur.fetchone()[0]
            conn.commit()
            logging.info(f"Prewarmed {table}: scan touched {pages} rows")
        except Exception as e:
            conn.rollback()
            logging.error(f"Failed to prewarm {table}: {e}")

def schedule_prewarm_jobs(conn):
    for job in WORKLOAD_SCHEDULE:
        job_hour, job_minute = map(int, job["cron_time"].split(":"))
        prewarm_dt = datetime.now().replace(
            hour=job_hour, minute=job_minute, second=0
        ) - timedelta(minutes=PREWARM_LEAD_TIME_MINUTES)
        prewarm_time_str = prewarm_dt.strftime("%H:%M")

```

```

        schedule.every().day.at(rewarm_time_str).do(
            rewarm_tables, conn=conn, tables=job["tables_to_rewarm"]
        )
        logging.info(
            f"Scheduled rewarm for '{job['name']}' at {rewarm_time_str}"
        )

def run():
    conn = pyodbc.connect(MSSQL_DSN)
    schedule_rewarm_jobs(conn)
    while True:
        schedule.run_pending()
        time.sleep(30)

if __name__ == "__main__":
    run()

```

This predictive prefetch approach reduces the latency spike that occurs when a large reporting job starts cold. Instead of spending the first several minutes of execution time loading pages from disk, the job finds its data already resident in the buffer pool and begins processing at full speed.

Transaction Log I/O Optimization

The transaction log is the highest-priority I/O path in SQL Server. Every committed transaction must have its log record durably written before the commit acknowledgment is returned to the client. Any contention, latency spike, or device saturation on the log device directly increases transaction commit latency for every session simultaneously.

Autonomous log optimization has three concerns: device placement, write batching behavior, and log size management.

Device placement is handled at the infrastructure level: The log should live on a dedicated device separate from data files, ideally one optimized for sequential write throughput rather than random read IOPS. The autonomous agent validates this configuration by checking the mount point of the log files via `sys.master_files` against the list of known device paths and their I/O characteristics. When log and data share a device, an alert is raised and, if a suitable dedicated device is available in the device inventory, an automated migration is initiated.

Write batching in SQL Server is controlled by delayed durability. The `DELAYED_DURABILITY = ALLOWED` database option lets transactions return before their log records are flushed — a group-commit pattern that batches multiple commits into a single log flush. The autonomous controller enables delayed durability selectively for non-critical workloads based on observed `WRITELOG` wait time in `sys.dm_os_wait_stats` and the ratio of active sessions to commits per second.

```
-- SQL Server: transaction log I/O telemetry per database
SELECT
    DB_NAME(vfs.database_id)                AS database_name,
    vfs.num_of_bytes_written                 AS log_bytes_written,
    vfs.io_stall_write_ms                    AS log_write_stall_ms,
    CASE WHEN vfs.num_of_writes = 0 THEN 0
         ELSE vfs.io_stall_write_ms * 1.0 / vfs.num_of_writes END AS
avg_write_stall_ms,
    lu.used_log_space_pct                    AS used_log_space_pct
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf
    ON vfs.database_id = mf.database_id AND vfs.file_id = mf.file_id
JOIN sys.dm_db_log_space_usage lu ON lu.database_id = vfs.database_id
WHERE mf.type_desc = 'LOG';
```

When `WRITELOG` waits dominate `sys.dm_os_wait_stats` and average log write stall climbs, the underlying device cannot sustain the flush rate — the correct intervention is either device upgrade or, as a temporary measure, enabling delayed durability for non-critical workloads. The autonomous system never sets `DELAYED_DURABILITY = FORCED` without explicit operator permission, since this sacrifices crash safety. The decision is encoded as a policy flag:

```
DELAYED_DURABILITY_POLICY = {
    "allow_delayed": True,  # may enable delayed durability for non-critical
workloads
    "allow_forced": False,  # never force delayed durability without human approval
}
```

In SQL Server, the transaction log is managed through VLF (virtual log file) architecture. A log file that has been grown many times in small increments accumulates hundreds of small VLFs, which degrades log scan performance during recovery. The autonomous agent monitors VLF count using `DBCC LOGINFO` and proactively grows the log in larger increments to consolidate VLFs when the count exceeds a configurable threshold.


```
-- SQL Server: Count VLFs per database log file
CREATE TABLE #vlf_info (
    RecoveryUnitId INT,
    FileId          INT,
    FileSize         BIGINT,
    StartOffset      BIGINT,
    FSeqNo           BIGINT,
    Status           INT,
    Parity           INT,
    CreateLSN        NUMERIC(25,0)
);

EXEC sp_MSforeachdb '
    USE [?];
    INSERT INTO #vlf_info
    EXEC(''DBCC LOGINFO WITH NO_INFOMSGS'');
';

SELECT
    DB_NAME()      AS database_name,
    COUNT(*)       AS vlf_count,
    SUM(FileSize) / 1024 / 1024 AS total_log_mb,
    SUM(CASE WHEN Status = 2 THEN 1 ELSE 0 END) AS active_vlfs
FROM #vlf_info;

DROP TABLE #vlf_info;
```

When VLF count exceeds the threshold (commonly set to 200 for databases under 100GB), the remediation path is to shrink the log file to minimum size (which forces VLF consolidation), then immediately grow it in a single large allocation. This is a brief operation that causes no data loss but does require the log to not be actively written — the agent schedules it during the low-traffic window identified by the I/O utilization monitor.

Integrating Storage Autonomy with the Broader Control Loop

Each of the subsystems described in this chapter — I/O instrumentation, tablespace tiering, checkpoint control, bloat remediation, partition lifecycle management, buffer pool prefetch, and transaction log optimization — can function independently. But maximum effectiveness comes from integration: a shared signal bus where each subsystem both reads from and writes to a common state model.

The integration point is the **storage control plane**: a lightweight service that aggregates signals from all subsystems and arbitrates competing actions. Without arbitration, subsystems can conflict. An index rebuild and a filegroup migration might both attempt to take conflicting locks on the same table simultaneously. A checkpoint control adjustment

that increases `TARGET_RECOVERY_TIME` might interact adversely with a log device migration in progress. A partition prewarm job might run during peak I/O utilization and compete with active queries.

```
graph TD
  A[Filegroup Tiering] -->|register intent| G[Storage Control Plane]
  B[Bloat Remediation] -->|register intent| G
  C[Checkpoint Control] -->|register intent| G
  D[Partition Lifecycle] -->|register intent| G
  E[Buffer Pool / Prefetch] -->|register intent| G
  F[Log Optimization] -->|register intent| G
  G -->|conflict check + I/O budget check| H{Grant or Defer}
  H -->|granted| I[Action Executes]
  H -->|denied: object locked or budget exceeded| J[Deferred / Retried Next Cycle]
  G --> K[Audit Log]
```

Every autonomous storage subsystem registers its intent with the storage control plane before acting; the plane grants or defers based on object-level conflicts and the global I/O budget.

The control plane maintains a global I/O budget: a maximum number of concurrent I/O-intensive operations, a ceiling on MB/s consumed by background tasks, and an exclusion list of objects currently undergoing any autonomous action. Before any subsystem initiates an action, it registers an intent with the control plane, which checks for conflicts and grants or denies the action based on current system state.

```

import threading
import time
from dataclasses import dataclass, field
from typing import Dict, Optional
from enum import Enum

class ActionType(Enum):
    FILEGROUP_MOVE = "filegroup_move"
    BLOAT_REMEDIATION = "bloat_remediation"
    PARTITION_CREATE = "partition_create"
    PARTITION_DEMOTE = "partition_demote"
    CHECKPOINT_TUNE = "checkpoint_tune"
    LOG_TUNE = "log_tune"
    PREWARM = "prewarm"

@dataclass
class ActionIntent:
    action_type: ActionType
    target_object: str # schema.table or partition name
    estimated_io_mb: float
    requester: str
    granted: bool = False
    started_at: Optional[float] = None

class StorageControlPlane:
    MAX_CONCURRENT_IO_OPS = 2
    MAX_BACKGROUND_IO_MBS = 200.0

    def __init__(self):
        self._lock = threading.Lock()
        self._active: Dict[str, ActionIntent] = {}
        self._io_budget_used = 0.0

    def request_action(self, intent: ActionIntent) -> bool:
        with self._lock:
            # Check object-level conflict
            if intent.target_object in self._active:
                existing = self._active[intent.target_object]
                print(
                    f"CONFLICT: {intent.action_type.value} on {intent.target_object}"
                    f"blocked by {existing.action_type.value} from"
                    f"{existing.requester}"
                )
                return False

            # Check global I/O concurrency
            if len(self._active) >= self.MAX_CONCURRENT_IO_OPS:
                print(f"I/O_BUDGET: max concurrent ops reached, deferring"
                    f"{intent.action_type.value}")
                return False

            # Check I/O bandwidth budget

```

```

        if self._io_budget_used + intent.estimated_io_mb >
self.MAX_BACKGROUND_IO_MBS:
            print(f"IO_BUDGET: bandwidth ceiling reached, deferring
{intent.action_type.value}")
            return False

        intent.granted = True
        intent.started_at = time.time()
        self._active[intent.target_object] = intent
        self._io_budget_used += intent.estimated_io_mb
        return True

    def release_action(self, target_object: str):
        with self._lock:
            if target_object in self._active:
                released = self._active.pop(target_object)
                self._io_budget_used = max(0.0, self._io_budget_used -
released.estimated_io_mb)

    def active_actions(self):
        with self._lock:
            return dict(self._active)

```

This control plane is deliberately simple — a threading lock over an in-memory dictionary. In production deployments across multiple database hosts, it would be backed by a distributed coordination service (etcd, Redis with Lua transactions, or a purpose-built sidecar). But the core contract is the same regardless of implementation: every background storage action must declare its intent, every execution is gated by a conflict check and a budget check, and every completion releases the held resources.

The control plane also serves as the audit log entry point. Every granted and denied action is written to a structured log that the observability layer reads for post-hoc analysis. When storage performance degrades unexpectedly, the audit log shows precisely which autonomous operations were active at the time, making root cause analysis tractable rather than speculative.

Key Takeaways

- → **Observability at three levels is non-negotiable:** object-level, engine-level, and block-device-level I/O metrics must be collected simultaneously and at sub-minute granularity. Conflating these three levels produces misleading signals that drive incorrect autonomous actions.
- → **Tiering decisions must be based on measured heat scores, not age alone:** access frequency, write volume, and recency of modification together determine whether a relation belongs on fast NVMe or cold-tier storage. Age-based policies fail when access patterns shift and yesterday's archive becomes today's active dataset.

- → **Checkpoint and log tuning must follow a one-parameter-per-cycle discipline:** changing `TARGET_RECOVERY_TIME` and related settings simultaneously makes causal attribution impossible. Autonomous controllers must change one variable, observe two full checkpoint cycles, and evaluate before proceeding.
- → **Partition lifecycle automation closes the gap between theoretical tiering policy and operational reality:** forward partition creation prevents exhaustion failures; backward partition demotion continuously reclaims fast-tier capacity. Neither is sufficient without the other, and both must integrate with the broader I/O control plane to avoid lock conflicts and bandwidth saturation.
- → **A storage control plane with I/O budgeting and object-level conflict detection is required before deploying multiple autonomous storage subsystems concurrently:** without arbitration, independent subsystems will schedule conflicting operations, exhaust I/O bandwidth, and create the exact instability they were designed to prevent.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 14: Autonomous Capacity Planning

Capacity planning in traditional database operations is a quarterly ritual — a DBA pulls growth trends, builds a spreadsheet, presents a forecast, and waits for procurement. By the time new hardware arrives, the situation has already changed. Autonomous capacity planning replaces this cycle with a continuously running feedback system that observes resource consumption, projects future demand using statistical and ML-based models, triggers provisioning actions through infrastructure APIs, and validates outcomes — all without a human in the scheduling loop. This chapter walks through building that system end-to-end: the signal collection layer, the forecasting engine, the decision rules that translate forecasts into actions, and the integration with Kubernetes, Terraform, and cloud-native scaling APIs that make those actions real.

14.1 Measuring What Actually Constrains Your Database

Before any forecasting model is worth building, the signal collection layer must be honest about which resources are actually binding constraints. Storage exhaustion is obvious and easy to measure. What kills production systems more often is a subtler constellation of pressures: buffer pool saturation, transaction log write amplification, connection exhaustion at the proxy layer, I/O queue depth at the block device level, and CPU contention during checkpoint storms. An autonomous system that only watches disk is dangerous — it will declare the environment healthy while the query engine is thrashing.

The baseline metrics collection strategy needs to capture five resource dimensions at short intervals (30 seconds to 2 minutes depending on the resource): storage, memory, compute, I/O, and connection throughput. These are not server-level metrics; they must be captured at the database instance level, correlated with workload identity.

```

-- Equivalent resource pressure snapshot for SQL Server
-- Designed to be executed by a SQL Agent job every 60 seconds

SELECT
    GETUTCDATE()                                AS captured_at,
    (SELECT SUM(size) * 8192 FROM sys.master_files
     WHERE database_id = DB_ID())                AS db_size_bytes,
    (SELECT COUNT(*) FROM sys.dm_exec_sessions
     WHERE is_user_process = 1 AND status = 'running') AS
active_connections,
    (SELECT COUNT(*) FROM sys.dm_os_waiting_tasks
     WHERE wait_type LIKE 'LCK%')                AS lock_waiters,
    (SELECT cntr_value FROM sys.dm_os_performance_counters
     WHERE counter_name = 'Buffer cache hit ratio'
     AND object_name LIKE '%Buffer Manager%')    AS buffer_hit_ratio,
    (SELECT physical_memory_in_use_kb * 1024
     FROM sys.dm_os_process_memory)              AS
memory_in_use_bytes,
    (SELECT cntr_value FROM sys.dm_os_performance_counters
     WHERE counter_name = 'Log Bytes Flushed/sec'
     AND instance_name = DB_NAME())              AS log_flush_rate,
    (SELECT SUM(log_send_queue_size) FROM sys.dm_hadr_database_replica_states
     WHERE is_local = 0)                          AS
max_replication_lag_bytes,
    (SELECT physical_memory_kb * 1024
     FROM sys.dm_os_sys_info)                     AS
total_host_memory_bytes;

```

Store these snapshots in a dedicated metrics database — not in the production instance you are monitoring. Use a dedicated SQL Server instance as the metrics database, or push to Prometheus with a scrape interval of 30 seconds and long-term retention via Thanos or Cortex. The key architectural decision here is **separation of the observer from the observed**: if your capacity system depends on the health of the system it monitors, you lose visibility exactly when you need it most.

For I/O, do not rely on database-layer counters alone. Instrument at the OS level using `node_exporter` for Linux hosts (or the Windows Exporter for SQL Server on Windows), and correlate `diskstats` metrics — specifically `io_time_ms` and `await` — with the database-layer write rates. A rising `await` with stable write volume signals block device saturation before any database-level metric reflects it.

Connection pressure is particularly insidious. SQL Server exposes live connection counts through `sys.dm_exec_connections` and the `User Connections` performance counter. When the connection count approaches the configured `user connections` limit — or when the rate of new connections spikes without a matching rise in active requests — the system is already degraded: a capacity event has effectively started. Capture this metric explicitly:

```
-- SQL Server connection pressure – run every 30s, export to Prometheus via
sql_exporter
SELECT
    COUNT(*) AS total_connections,
    SUM(CASE WHEN s.status = 'running' THEN 1 ELSE 0 END) AS active_requests,
    SUM(CASE WHEN DATEDIFF(MINUTE, c.connect_time, GETUTCDATE()) > 60
        AND s.status = 'sleeping' THEN 1 ELSE 0 END) AS idle_over_60min,
    -- Connections waiting on a worker = pressure signal
    SUM(CASE WHEN w.wait_type IS NOT NULL THEN 1 ELSE 0 END) AS waiting_connections
FROM sys.dm_exec_connections c
LEFT JOIN sys.dm_exec_sessions s ON s.session_id = c.session_id
LEFT JOIN sys.dm_os_waiting_tasks w
    ON w.session_id = c.session_id AND w.wait_type LIKE 'THREADPOOL%';
```

Push these metrics into the same time-series store as your database-level snapshots. The forecasting engine needs a unified view.

14.2 Forecasting Models Calibrated for Database Workload Patterns

Database resource consumption has properties that make naive forecasting models perform poorly. Storage growth is often smooth but punctuated by bulk loads, schema migrations, and index rebuilds. Connection counts show strong diurnal seasonality — business applications spike at 9 AM and taper off overnight — but also carry irregular spikes from batch jobs, reporting windows, and end-of-month processing. Memory pressure follows query complexity, not time. A good forecasting layer handles all three patterns: smooth trends, seasonal cycles, and irregular spikes.

The practical forecasting stack for an autonomous system combines three approaches depending on the resource dimension:

- → **Linear regression with drift detection** for storage (growth is mostly monotonic; the anomaly is when it isn't)
- → **Prophet or SARIMA** for connection counts and CPU (strong seasonality, predictable cycles)
- → **Percentile-based headroom analysis** for buffer pool and memory (you care about P95/P99, not mean)

The following Python script implements the forecasting engine. It pulls raw metrics from Prometheus, applies Prophet for connection forecasting, and emits a structured capacity recommendation. This runs as a Kubernetes CronJob every four hours.


```
#!/usr/bin/env python3
"""
autonomous_capacity_forecast.py
Pulls metrics from Prometheus, applies statistical forecasting,
and emits capacity recommendations to a Redis-backed action queue.
"""

import json
import logging
from datetime import datetime, timedelta

import pandas as pd
import redis
import requests
from prophet import Prophet

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("capacity_forecast")

PROMETHEUS_URL = "http://prometheus:9090"
REDIS_URL = "redis://redis:6379"
FORECAST_HORIZON_HOURS = 72
RECOMMENDATION_THRESHOLD_PCT = 0.85 # trigger if forecast hits 85% of limit

RESOURCE_LIMITS = {
    "connections": 500,          # user connections for this instance class
    "storage_bytes": 1_099_511_627_776, # 1 TB provisioned
    "memory_bytes": 68_719_476_736,    # 64 GB RAM
}

def query_prometheus_range(metric: str, hours_back: int = 168) -> pd.DataFrame:
    end = datetime.utcnow()
    start = end - timedelta(hours=hours_back)
    params = {
        "query": metric,
        "start": start.isoformat() + "Z",
        "end": end.isoformat() + "Z",
        "step": "300", # 5-minute resolution
    }
    response = requests.get(f"{PROMETHEUS_URL}/api/v1/query_range",
                           params=params, timeout=30)
    response.raise_for_status()
    results = response.json()["data"]["result"]
    if not results:
        return pd.DataFrame()

    values = results[0]["values"]
    df = pd.DataFrame(values, columns=["timestamp", "value"])
    df["ds"] = pd.to_datetime(df["timestamp"], unit="s", utc=True)
    df["y"] = df["value"].astype(float)
    return df[["ds", "y"]]
```

```

def run_prophet_forecast(df: pd.DataFrame,
                        horizon_hours: int) -> pd.DataFrame:
    model = Prophet(
        yearly_seasonality=False,
        weekly_seasonality=True,
        daily_seasonality=True,
        changepoint_prior_scale=0.05,
        interval_width=0.95,
    )
    df_local = df.copy()
    df_local["ds"] = df_local["ds"].dt.tz_localize(None)
    model.fit(df_local)

    future = model.make_future_dataframe(
        periods=horizon_hours * 12, # 5-min intervals
        freq="5min"
    )
    forecast = model.predict(future)
    return forecast[["ds", "yhat", "yhat_lower", "yhat_upper"]].tail(
        horizon_hours * 12
    )

def evaluate_capacity(resource: str,
                    forecast: pd.DataFrame,
                    limit: float) -> dict:
    peak_forecast = forecast["yhat_upper"].max()
    peak_at = forecast.loc[forecast["yhat_upper"].idxmax(), "ds"]
    utilization = peak_forecast / limit

    return {
        "resource": resource,
        "forecast_peak_value": round(peak_forecast, 2),
        "forecast_peak_at": peak_at.isoformat(),
        "limit": limit,
        "forecast_utilization_pct": round(utilization * 100, 2),
        "action_required": utilization >= RECOMMENDATION_THRESHOLD_PCT,
        "evaluated_at": datetime.utcnow().isoformat(),
    }

def push_recommendation(client: redis.Redis, recommendation: dict):
    if recommendation["action_required"]:
        client.lpush("capacity:actions:pending",
                    json.dumps(recommendation))
        log.info("Pushed capacity action: %s utilization %.1f%%",
                recommendation["resource"],
                recommendation["forecast_utilization_pct"])
    else:
        log.info("Resource %s within bounds: %.1f%%",
                recommendation["resource"],

```

```

recommendation["forecast_utilization_pct"])

def main():
    r = redis.from_url(REDIS_URL)

    # Metric names depend on the sql_exporter collectors enabled;
    # adjust to match your deployment's collector configuration.
    metric_map = {
        "connections": 'sqlserver_performance{counter="User Connections"}',
        "storage_bytes": "sqlserver_database_size_bytes",
        "memory_bytes": 'sqlserver_performance{counter="Total Server Memory (KB)"} *
1024',
    }

    for resource, prometheus_metric in metric_map.items():
        log.info("Forecasting: %s", resource)
        df = query_prometheus_range(prometheus_metric)
        if df.empty or len(df) < 50:
            log.warning("Insufficient data for %s, skipping", resource)
            continue

        forecast = run_prophet_forecast(df, FORECAST_HORIZON_HOURS)
        rec = evaluate_capacity(resource, forecast,
                                RESOURCE_LIMITS[resource])
        push_recommendation(r, rec)

if __name__ == "__main__":
    main()

```

For storage specifically, augment Prophet with a simple linear regression over the trailing 30-day trend. Storage growth rates rarely change suddenly, and Prophet's seasonality machinery adds noise where none exists. When the linear model and Prophet disagree by more than 20%, escalate to human review rather than acting autonomously — forecast disagreement is itself a signal that something unusual is happening in the workload.

Calibrate `RECOMMENDATION_THRESHOLD_PCT` per resource type. Connections should trigger at 80% because connection exhaustion is instantaneous — there is no graceful degradation. Storage can trigger at 85% because growth is observable and provisioning has lead time. Memory should trigger at 75% because you need buffer to absorb a reporting query spike before the OOM killer becomes involved.

14.3 Translating Forecasts into Provisioning Actions via Infrastructure APIs

A capacity forecast that produces a report is still manual capacity planning. The autonomous layer converts forecast outputs into concrete provisioning actions executed against infrastructure APIs without waiting for human approval, subject to safety guardrails that

define the action envelope. The architecture here follows the same observe-decide-act loop described in earlier chapters, but the decision layer is more conservative than in performance tuning because provisioning actions are typically irreversible or expensive to reverse.

The action hierarchy for capacity events, in ascending order of impact and required confidence:

1. **Rebalance connection governance** — tighten Resource Governor workload group caps (`MAX_DOP` , `REQUEST_MAX_MEMORY_GRANT_PERCENT`) for reporting workloads and raise the application connection pool ceiling via config reload. Zero risk, zero cost, immediate effect.
2. **Expand vertical resources** — increase instance size via cloud provider API (RDS `ModifyDBInstance` , Azure SQL service tier change). Requires maintenance window awareness.
3. **Add read replicas** — provision a new read replica via Terraform or cloud API to offload read workload. Increases cost, takes 15–30 minutes for large databases.
4. **Expand storage** — increase allocated storage or tablespace via cloud API. Usually non-disruptive on modern cloud platforms; can be disruptive on self-managed storage.
5. **Trigger archival pipelines** — move cold data to object storage or a separate archive instance to reclaim space. Complex; requires data classification to be pre-configured.

```
graph LR
    A[Forecast Engine] -->|utilization >= threshold| B[Redis Recommendation Queue]
    B --> C{Action Hierarchy}
    C -->|Tier 1| D[Rebalance Connection Pools]
    C -->|Tier 2| E[Expand Vertical Resources]
    C -->|Tier 3| F[Add Read Replicas]
    C -->|Tier 4| G[Expand Storage]
    C -->|Tier 5| H[Trigger Archival Pipeline]
    D --> I[Validate Outcome]
    E --> I
    F --> I
    G --> I
    H --> I
    I -->|failed| J[Escalate to Human]
    I -->|succeeded| K[Feed Back to Forecast Model]
```

Forecasts drive a ranked action hierarchy, from low-risk connection pool rebalancing to high-impact archival pipelines; every executed action is validated and fed back into the forecasting loop.

The decision engine reads from the Redis recommendation queue and routes actions through the appropriate handler:

```
#!/usr/bin/env python3
"""
capacity_action_executor.py
Reads capacity recommendations from the action queue and executes
the appropriate provisioning response through the correct API.
"""

import json
import logging
import os
import time

import boto3
import redis

log = logging.getLogger("capacity_executor")
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

REDIS_URL = os.environ["REDIS_URL"]
RDS_INSTANCE_ID = os.environ["RDS_INSTANCE_ID"]
AWS_REGION = os.environ.get("AWS_REGION", "us-east-1")

# Action cooldown: don't act on the same resource more than once per window
COOLDOWN_SECONDS = {
    "connections": 1800,      # 30 minutes
    "storage_bytes": 14400,   # 4 hours
    "memory_bytes": 7200,     # 2 hours
}

rds = boto3.client("rds", region_name=AWS_REGION)
r = redis.from_url(REDIS_URL)

def is_in_cooldown(resource: str) -> bool:
    key = f"capacity:cooldown:{resource}"
    return r.exists(key) == 1

def set_cooldown(resource: str):
    key = f"capacity:cooldown:{resource}"
    r.setex(key, COOLDOWN_SECONDS[resource], "1")

def expand_rds_storage(current_gb: int, increment_pct: float = 0.30) -> dict:
    new_size = int(current_gb * (1 + increment_pct))
    new_size = max(new_size, current_gb + 50) # minimum 50 GB increment

    log.info("Expanding RDS storage: %d GB -> %d GB", current_gb, new_size)
    response = rds.modify_db_instance(
        DBInstanceIdentifier=RDS_INSTANCE_ID,
        AllocatedStorage=new_size,
        ApplyImmediately=True,

```

```

    )
    return {
        "action": "storage_expansion",
        "old_size_gb": current_gb,
        "new_size_gb": new_size,
        "rds_status": response["DBInstance"]["DBInstanceStatus"],
    }

MSSQL_DSN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=sql-primary;DATABASE=master;Trusted_Connection=yes;"
)

def scale_connection_governance(utilization_pct: float) -> dict:
    """
    Shed load under connection pressure by tightening Resource Governor
    limits on the reporting workload group, protecting the OLTP group.
    """
    import pyodbc

    if utilization_pct >= 90:
        new_max_dop, new_grant_pct = 2, 10
    elif utilization_pct >= 85:
        new_max_dop, new_grant_pct = 4, 15
    else:
        return {"action": "governor_noop"}

    conn = pyodbc.connect(MSSQL_DSN, autocommit=True)
    cur = conn.cursor()
    cur.execute(
        "ALTER WORKLOAD GROUP reporting_group WITH "
        f"(MAX_DOP = {new_max_dop}, "
        f"REQUEST_MAX_MEMORY_GRANT_PERCENT = {new_grant_pct});"
    )
    cur.execute("ALTER RESOURCE GOVERNOR RECONFIGURE;")
    conn.close()
    log.info("Resource Governor tightened: MAX_DOP=%d grant_pct=%d",
            new_max_dop, new_grant_pct)

    return {
        "action": "governor_adjustment",
        "max_dop": new_max_dop,
        "grant_pct": new_grant_pct,
    }

def handle_recommendation(rec: dict) -> dict | None:
    resource = rec["resource"]
    utilization = rec["forecast_utilization_pct"]

    if is_in_cooldown(resource):

```

```

        log.info("Resource %s is in cooldown, skipping", resource)
        return None

    result = None

    if resource == "connections":
        result = scale_connection_governance(utilization)

    elif resource == "storage_bytes":
        current_gb = int(rec["limit"] / (1024 ** 3))
        result = expand_rds_storage(current_gb)

    elif resource == "memory_bytes":
        # Memory scaling requires instance resize – emit for human approval
        # above a certain threshold; auto-act only for minor adjustments
        log.warning("Memory pressure at %.1f%% – emitting for orchestrator review",
                    utilization)
        r.lpush("capacity:escalations", json.dumps(rec))
        return {"action": "escalated_to_orchestrator", "resource": resource}

    if result:
        set_cooldown(resource)
        r.lpush("capacity:actions:completed",
                json.dumps(**rec, "result": result,
                           "executed_at": time.time()))

    return result

def main():
    log.info("Capacity action executor starting")
    while True:
        item = r.brpop("capacity:actions:pending", timeout=30)
        if item is None:
            continue
        _, payload = item
        try:
            rec = json.loads(payload)
            handle_recommendation(rec)
        except Exception as e:
            log.error("Failed to handle recommendation: %s", e)
            r.lpush("capacity:actions:failed", payload)

if __name__ == "__main__":
    main()

```

The memory escalation path in this executor is intentional. Vertical scaling of an RDS instance or an Azure SQL tier is not instantaneous and carries a brief failover window (typically 30–60 seconds on modern multi-AZ deployments, but still observable). The autonomous system acts immediately on soft resources (connection pool limits, storage expansion) and escalates

hard resource decisions — instance resizes — to the orchestrator layer, which can apply maintenance window awareness and rollback logic. This is not a limitation of the autonomous design; it is the appropriate confidence boundary.

For SQL Server on Azure SQL, the equivalent scaling action uses the Azure SDK:

```
# Azure SQL service tier scaling via azure-mgmt-sql SDK
from azure.mgmt.sql import SqlManagementClient
from azure.identity import DefaultAzureCredential

credential = DefaultAzureCredential()
client = SqlManagementClient(credential, subscription_id)

# Scale Azure SQL Database to next service tier
client.databases.begin_create_or_update(
    resource_group_name=resource_group,
    server_name=server_name,
    database_name=database_name,
    parameters={
        "location": location,
        "sku": {
            "name": "GP_Gen5",
            "tier": "GeneralPurpose",
            "capacity": 16 # vCores – was 8
        }
    }
).result()
```

14.4 Kubernetes-Native Capacity Scaling for Database Workloads

When databases run inside Kubernetes — whether as StatefulSets managed by the CloudNativePG operator, CrunchyData PGO, or the SQL Server on Kubernetes operator — capacity planning integrates with the Kubernetes resource model. This introduces both tighter feedback loops and new failure modes. PersistentVolumeClaims can be expanded online (on supporting storage classes) by patching the PVC spec. CPU and memory requests can be adjusted by updating the StatefulSet, which triggers a rolling pod restart. Connection scaling is handled by the operator's pooler abstraction.

CloudNativePG exposes a `Cluster` CRD that the autonomous system can patch directly when capacity thresholds are breached. The following shows a Kubernetes operator-style patch applied by the capacity executor:


```
#!/usr/bin/env python3
"""
k8s_capacity_patch.py
Applies capacity changes to CloudNativePG Cluster CRDs via the Kubernetes API.
"""

from kubernetes import client, config
import json
import logging

log = logging.getLogger("k8s_capacity")
config.load_incluster_config() # running inside the cluster

custom_api = client.CustomObjectsApi()
core_api = client.CoreV1Api()

NAMESPACE = "databases"
STATEFULSET_NAME = "mssql-primary"
PVC_NAME = "mssql-data-mssql-primary-0"

def expand_pvc_storage(new_size_gi: int):
    """
    Patch the data PVC – the CSI driver handles online volume expansion.
    Requires the storage class to allow volume expansion.
    """
    patch = {"spec": {"resources": {"requests": {"storage": f"{new_size_gi}Gi"}}}}
    core_api.patch_namespaced_persistent_volume_claim(
        name=PVC_NAME,
        namespace=NAMESPACE,
        body=patch,
    )
    log.info("Patched PVC %s storage to %dGi", PVC_NAME, new_size_gi)

def scale_read_replicas(new_replicas: int):
    """
    Scale the AG secondary StatefulSet. The operator's reconcile loop
    joins new pods to the availability group automatically.
    """
    apps_api = client.AppsV1Api()
    apps_api.patch_namespaced_stateful_set_scale(
        name="mssql-secondary",
        namespace=NAMESPACE,
        body={"spec": {"replicas": new_replicas}},
    )
    log.info("Scaled AG secondary StatefulSet to %d replicas", new_replicas)

def update_resource_requests(cpu_request: str,
                              memory_request: str,
                              cpu_limit: str,
                              memory_limit: str):
```

```

"""
Update CPU and memory resource requests on the primary StatefulSet.
This triggers a rolling restart – use with care.
"""

apps_api = client.AppsV1Api()
sts = apps_api.read_namespaced_stateful_set(
    name=STATEFULSET_NAME, namespace=NAMESPACE)
container = sts.spec.template.spec.containers[0]
container.resources.requests = {"cpu": cpu_request, "memory": memory_request}
container.resources.limits = {"cpu": cpu_limit, "memory": memory_limit}
apps_api.patch_namespaced_stateful_set(
    name=STATEFULSET_NAME,
    namespace=NAMESPACE,
    body={"spec": {"template": {"spec": {"containers": [
        {"name": container.name, "resources": container.resources.to_dict()}
    ]}}}},
)
log.info("Patched StatefulSet resources: cpu=%s mem=%s",
        cpu_request, memory_request)

def get_current_statefulset_spec() -> dict:
    apps_api = client.AppsV1Api()
    sts = apps_api.read_namespaced_stateful_set(
        name=STATEFULSET_NAME, namespace=NAMESPACE)
    container = sts.spec.template.spec.containers[0]
    return {
        "replicas": sts.spec.replicas,
        "resources": {
            "requests": dict(container.resources.requests or {}),
            "limits": dict(container.resources.limits or {}),
        },
    }

def autonomous_scale_decision(recommendation: dict) -> None:
    spec = get_current_statefulset_spec()
    resource = recommendation["resource"]
    utilization = recommendation["forecast_utilization_pct"]

    if resource == "storage_bytes":
        # PVC size is tracked separately from the pod spec
        pvc = core_api.read_namespaced_persistent_volume_claim(
            name=PVC_NAME, namespace=NAMESPACE)
        current_size_str = pvc.spec.resources.requests["storage"] # e.g. "500Gi"
        current_gi = int(current_size_str.replace("Gi", ""))
        new_gi = int(current_gi * 1.35) # 35% expansion
        new_gi = max(new_gi, current_gi + 100) # floor increment
        expand_pvc_storage(new_gi)

    elif resource == "connections":
        current_replicas = spec.get("replicas", 1)
        if utilization >= 90 and current_replicas < 5:

```

```

        scale_read_replicas(current_replicas + 1)
        log.info("Added read replica: %d -> %d replicas",
                current_replicas, current_replicas + 1)

elif resource == "memory_bytes":
    # Only auto-scale memory if headroom is extremely tight
    # and the cluster is running below the node's available capacity.
    # Otherwise escalate.
    if utilization >= 95:
        current_mem = spec.get("resources", {}).get(
            "requests", {}).get("memory", "8Gi")
        current_gi = int(current_mem.replace("Gi", ""))
        new_gi = current_gi + 8
        update_resource_requests(
            cpu_request=spec.get("resources", {}).get(
                "requests", {}).get("cpu", "2"),
            memory_request=f"{new_gi}Gi",
            cpu_limit=spec.get("resources", {}).get(
                "limits", {}).get("cpu", "4"),
            memory_limit=f"{int(new_gi * 1.25)}Gi",
        )
    else:
        log.info("Memory at %.1f%% – below auto-scale threshold, "
                "escalating", utilization)

```

The key behavioral rule embedded in `autonomous_scale_decision` is that storage and read-replica counts are scaled automatically, while memory resource updates are gated at a higher utilization threshold because they force a pod restart (and an AG failover sequence). This asymmetry reflects the actual cost profile of each operation: PVC expansion is non-disruptive on most CSI drivers (Ceph RBD, EBS, GCE PD), but pod restarts always carry a brief promotion or failover sequence, even with proper readiness probes.

The functions above use the operator-agnostic approach: patch the StatefulSet for memory and CPU, patch the PVC for storage, and manage secondary replicas through the availability group configuration — no operator-specific CRD required.

Horizontal Pod Autoscaler integration for connection proxies:

The database itself cannot be HPA-scaled (it is stateful), but the connection proxy layer in front of it can be. The following HPA manifest scales the proxy deployment based on connection wait pressure as a custom metric:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: sqlproxy-hpa
  namespace: databases
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: sqlproxy
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: External
    external:
      metric:
        name: sqlserver_proxy_connection_waits
        selector:
          matchLabels:
            database: mssql-primary
        target:
          type: AverageValue
          averageValue: "0" # any nonzero waiting clients triggers scale-out
  - type: External
    external:
      metric:
        name: sqlserver_proxy_idle_connections
        selector:
          matchLabels:
            database: mssql-primary
        target:
          type: AverageValue
          averageValue: "5" # scale in when avg idle > 5 per pod
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
      - type: Pods
        value: 2
        periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 600 # 10 minutes before scaling in
      policies:
      - type: Pods
        value: 1
        periodSeconds: 120

```

The `cl_waiting` metric is binary in intent — any sustained nonzero value means clients are being throttled. Setting the target to zero means any waiting client detected above the stabilization window triggers scale-out. The scale-in stabilization window of ten minutes prevents oscillation after a burst subsides.

14.5 Terraform-Driven Capacity Automation with State Drift Detection

For environments managed through Terraform, the autonomous capacity system must interact with Terraform state rather than calling APIs directly. Calling the AWS or Azure SDK directly from the executor bypasses Terraform's state model, which creates drift: Terraform state says the instance is `db.r6g.2xlarge` but the API has moved it to `db.r6g.4xlarge`. The next `terraform apply` will revert the autonomous change and potentially cause an unplanned resize mid-day.

The correct integration pattern is to update the Terraform variable store (an S3-backed variable file, Terraform Cloud workspace variables, or a Vault-backed Terraform input) and trigger a controlled `terraform apply` rather than calling the infrastructure API directly. The autonomous system becomes a variable writer, not an API caller.

```
#!/usr/bin/env python3
"""
terraform_capacity_updater.py
Updates Terraform variables in S3 and triggers a Terraform Cloud run
in response to capacity recommendations. Terraform performs the actual
API call, keeping state consistent.
"""

import json
import logging
import os

import boto3
import requests

log = logging.getLogger("terraform_capacity")

S3_BUCKET = os.environ["CAPACITY_VARS_BUCKET"]
S3_KEY = "capacity/rds_variables.json"
TFC_TOKEN = os.environ["TFC_TOKEN"]
TFC_WORKSPACE_ID = os.environ["TFC_WORKSPACE_ID"]

s3 = boto3.client("s3")

def read_current_vars() -> dict:
    try:
        obj = s3.get_object(Bucket=S3_BUCKET, Key=S3_KEY)
        return json.loads(obj["Body"].read())
    except s3.exceptions.NoSuchKey:
        return {}

def write_updated_vars(vars_dict: dict):
    s3.put_object(
        Bucket=S3_BUCKET,
        Key=S3_KEY,
        Body=json.dumps(vars_dict, indent=2),
        ContentType="application/json",
    )
    log.info("Updated capacity variables in S3: %s", vars_dict)

def trigger_tfc_run(message: str):
    """
    Trigger a Terraform Cloud plan+apply run for the capacity workspace.
    """
    url = f"https://app.terraform.io/api/v2/runs"
    headers = {
        "Authorization": f"Bearer {TFC_TOKEN}",
        "Content-Type": "application/vnd.api+json",
    }
    payload = {
```

```

        "data": {
            "attributes": {
                "message": message,
                "auto-apply": True,
                "refresh": True,
            },
            "type": "runs",
            "relationships": {
                "workspace": {
                    "data": {
                        "type": "workspaces",
                        "id": TFC_WORKSPACE_ID
                    }
                }
            }
        }
    }
}

response = requests.post(url, headers=headers,
                        json=payload, timeout=30)
response.raise_for_status()
run_id = response.json()["data"]["id"]
log.info("Triggered Terraform Cloud run: %s", run_id)
return run_id

```

```

INSTANCE_CLASS_LADDER = [
    "db.t4g.medium",
    "db.t4g.large",
    "db.r6g.large",
    "db.r6g.xlarge",
    "db.r6g.2xlarge",
    "db.r6g.4xlarge",
    "db.r6g.8xlarge",
    "db.r6g.16xlarge",
]

```

```

def next_instance_class(current: str) -> str | None:
    try:
        idx = INSTANCE_CLASS_LADDER.index(current)
        if idx + 1 < len(INSTANCE_CLASS_LADDER):
            return INSTANCE_CLASS_LADDER[idx + 1]
    except ValueError:
        pass
    return None

```

```

def apply_capacity_recommendation(recommendation: dict):
    resource = recommendation["resource"]
    current_vars = read_current_vars()

    if resource == "storage_bytes":
        current_gb = current_vars.get("rds_allocated_storage_gb", 500)

```

```

new_gb = max(int(current_gb * 1.35), current_gb + 100)
current_vars["rds_allocated_storage_gb"] = new_gb
write_updated_vars(current_vars)
trigger_tfc_run(
    f"Autonomous capacity: storage {current_gb}GB -> {new_gb}GB "
    f"(forecast {recommendation['forecast_utilization_pct']:.1f}%)"
)

elif resource == "memory_bytes":
    current_class = current_vars.get("rds_instance_class", "db.r6g.large")
    new_class = next_instance_class(current_class)
    if new_class:
        current_vars["rds_instance_class"] = new_class
        write_updated_vars(current_vars)
        trigger_tfc_run(
            f"Autonomous capacity: instance {current_class} -> {new_class} "
            f"(memory forecast {recommendation['forecast_utilization_pct']:.1f}%)"
        )
    else:
        log.error("Already at maximum instance class: %s", current_class)
        # Escalate to human – no more ladder rungs available

log.info("Capacity recommendation applied via Terraform: resource=%s",
        resource)

```

This pattern ensures that `terraform plan` outputs remain auditable and predictable. Every autonomous capacity change is a Terraform run with a message string that identifies the triggering recommendation and the forecast percentage that drove it. The run log becomes the capacity audit trail.

Drift detection runs as a separate process that periodically compares the live infrastructure state against Terraform state and alerts when they diverge by more than a defined tolerance:


```
def detect_capacity_drift():
    """
    Compare actual RDS instance class against Terraform state.
    Flags if they diverge, indicating an out-of-band change.
    """
    current_vars = read_current_vars()
    tf_instance_class = current_vars.get("rds_instance_class")

    rds = boto3.client("rds")
    instance = rds.describe_db_instances(
        DBInstanceIdentifier=os.environ["RDS_INSTANCE_ID"]
    )["DBInstances"][0]
    actual_class = instance["DBInstanceClass"]

    if tf_instance_class and actual_class != tf_instance_class:
        log.warning(
            "DRIFT DETECTED: Terraform expects %s, actual is %s. "
            "An out-of-band change may have occurred.",
            tf_instance_class, actual_class
        )
        # Push drift alert to monitoring channel
        return {"drift": True,
                "expected": tf_instance_class,
                "actual": actual_class}

    return {"drift": False, "instance_class": actual_class}
```

14.6 Validating That Capacity Actions Worked

Executing a provisioning action is not the end of the loop. The autonomous system must validate that the action achieved its intended effect and that the validated outcome is fed back into the forecasting model as a signal. Without this feedback, the system will re-forecast against stale assumptions and potentially trigger redundant or contradictory actions.

The validation cycle runs after a resource-specific settlement period (storage: 15 minutes for the cloud API to reflect the new size; connection governance: 5 minutes for the Resource Governor reconfigure to take effect; instance resize: 30 minutes including failover and warm-up).

Post-action storage validation:

```

-- Verify storage expansion outcome in SQL Server environment
-- Run after the settlement window following Azure SQL storage tier update

SELECT
    GETUTCDATE()                AS validated_at,
    DB_NAME()                   AS database_name,
    SUM(size * 8192)            AS actual_db_size_bytes,
    SUM(size * 8192) / 1073741824.0 AS actual_db_size_gb,
    (
        SELECT max_size * 8192
        FROM sys.master_files
        WHERE database_id = DB_ID()
        AND type = 0 -- data files only
    )                            AS max_allocated_bytes,
    (
        (SELECT max_size * 8192
         FROM sys.master_files
         WHERE database_id = DB_ID() AND type = 0)
        - SUM(size * 8192)
    )                            AS
remaining_headroom_bytes
FROM
    sys.master_files
WHERE
    database_id = DB_ID();

```

The Python validation loop reads the completed action record from Redis, waits the settlement period, queries the database, and compares the outcome against the expected post-action state:

```

def validate_action_outcome(completed_action: dict) -> dict:
    resource = completed_action["resource"]
    result = completed_action["result"]
    executed_at = completed_action["executed_at"]

    settlement_seconds = {
        "storage_bytes": 900,      # 15 minutes
        "connections": 300,        # 5 minutes
        "memory_bytes": 1800,      # 30 minutes
    }

    # Wait for settlement if needed
    elapsed = time.time() - executed_at
    remaining_wait = settlement_seconds[resource] - elapsed
    if remaining_wait > 0:
        log.info("Waiting %.0fs for %s action to settle",
                remaining_wait, resource)
        time.sleep(remaining_wait)

    # Query actual current state
    if resource == "storage_bytes":
        actual = query_current_db_size_bytes() # sqlserver
        expected = result.get("new_size_gb", 0) * (1024 ** 3)
        # Storage expansion is validated if actual db_size < 85% of new limit
        success = actual < (expected * 0.85)

    elif resource == "connections":
        actual_waiting = query_sqlserver_waiting_connections()
        success = actual_waiting == 0

    elif resource == "memory_bytes":
        actual_mem_pct = query_current_memory_utilization_pct()
        success = actual_mem_pct < 75.0

    else:
        success = None # unknown resource type

    validation_record = {
        "resource": resource,
        "action_result": result,
        "validation_success": success,
        "validated_at": time.time(),
        "actual_state": actual if resource == "storage_bytes" else None,
    }

    if not success:
        log.warning("Action validation FAILED for %s – pushing to escalation queue",
                    resource)
        r.lpush("capacity:validations:failed", json.dumps(validation_record))
    else:
        log.info("Action validation SUCCEEDED for %s", resource)
        r.lpush("capacity:validations:succeeded", json.dumps(validation_record))

```

```
return validation_record
```

Failed validations are escalated, not retried automatically. Automatic retry of a failed provisioning action risks compounding the error — if an RDS storage expansion failed because the instance was in a `modifying` state, retrying immediately will fail again and potentially leave the instance in a confused state. Human review of failed validations is the correct response. The autonomous system has done its job by detecting the failure and escalating it with full context.

14.7 Capacity Planning for Multi-Tenant Database Architectures

Multi-tenant architectures add a layer of complexity: resources are shared across tenants, but tenant growth rates are independent and often uncorrelated. A single large tenant onboarding a new product line can exhaust storage allocated to dozens of smaller tenants. The autonomous capacity system must track resource consumption per tenant and forecast at the tenant level, not just the aggregate.

In a shared-schema multi-tenant SQL Server deployment, tenant isolation typically uses Row-Level Security with a `tenant_id` column. Resource attribution requires estimating storage per tenant from row counts and average record size:

```
-- Per-tenant storage consumption (shared schema) – run nightly and store in
-- the capacity metrics database. Assumes a tenant_table_registry mapping
-- table names to tenant_id.
SELECT
    t.tenant_id,
    COUNT(DISTINCT st.name)                                AS table_count,
    SUM(CAST(p.rows AS BIGINT) * ips.avg_record_size_in_bytes) AS estimated_bytes,
    GETUTCDATE()                                           AS measured_at
FROM sys.tables st
JOIN sys.partitions p
    ON st.object_id = p.object_id
JOIN sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'SAMPLED') ips
    ON ips.object_id = st.object_id AND ips.index_id = p.index_id
JOIN tenant_table_registry t
    ON st.name = t.table_name
WHERE p.index_id IN (0, 1) -- heap or clustered index only
GROUP BY t.tenant_id
ORDER BY estimated_bytes DESC;
```

For a schema-per-tenant deployment, attribution is simpler — the schema size is directly attributable:

```
-- Schema-per-tenant storage attribution
SELECT
    s.name                                AS tenant_schema,
    SUM(pa.used_pages * 8192)            AS used_bytes,
    GETUTCDATE()                        AS measured_at
FROM sys.schemas s
JOIN sys.tables st
    ON st.schema_id = s.schema_id
JOIN sys.partitions p
    ON p.object_id = st.object_id
JOIN sys.allocation_units pa
    ON pa.container_id = p.partition_id
WHERE s.name NOT IN ('dbo', 'sys', 'information_schema', 'guest')
    AND p.index_id IN (0, 1)  -- heap or clustered index only
GROUP BY s.name
ORDER BY used_bytes DESC;
```

SQL Server — per-tenant storage in a shared database:

```
-- Per-tenant table size attribution in shared SQL Server database
-- Assumes tenant_id-prefixed tables or a tenant_id column mapped in a registry

SELECT
    t.tenant_id,
    COUNT(DISTINCT st.name)                AS table_count,
    SUM(pa.used_pages * 8192)              AS used_bytes,
    SUM(pa.total_pages * 8192)             AS allocated_bytes,
    GETUTCDATE()                          AS measured_at
FROM
    sys.tables st
JOIN sys.partitions p
    ON st.object_id = p.object_id
JOIN sys.allocation_units pa
    ON p.partition_id = pa.container_id
JOIN tenant_table_registry t      -- your mapping table
    ON st.name = t.table_name
WHERE
    p.index_id IN (0, 1)              -- heap or clustered index only
GROUP BY
    t.tenant_id
ORDER BY
    used_bytes DESC;
```

The per-tenant forecast then runs Prophet independently per tenant, and the aggregate forecast is built by summing tenant forecasts. This prevents the "average is fine but one tenant is exploding" failure mode. When a single tenant's forecast exceeds a per-tenant

quota threshold, the action is tenant-specific: notify the tenant, apply storage limits if the architecture supports them, or accelerate the migration of that tenant to a dedicated instance.

The tenant-level capacity data also feeds churn modeling. A tenant whose storage growth rate is decelerating sharply and whose connection count is falling is showing pre-churn signal. That is not a capacity problem — it is a business problem. The capacity system surfaces it; the business system acts on it. Keeping these concerns separated at the data layer but unified at the metrics layer is what makes the autonomous system genuinely useful beyond pure infrastructure management.

14.8 Failure Modes and Safety Boundaries

Every autonomous provisioning system will eventually act on a bad forecast. The question is not whether it will happen but what the blast radius looks like when it does. The safety architecture has three layers: pre-action validation, action envelopes, and a circuit breaker.

Pre-action validation checks that the action is coherent before execution:

- → The new storage size is larger than the current size (never shrink autonomously)
- → The new instance class exists in the approved ladder and is available in the target AZ
- → The action does not conflict with a maintenance window defined in the operations calendar
- → The recommendation is not older than the cooldown period (stale recommendations can be hazardous)

Action envelopes bound what the system can do autonomously:

- → Maximum single-step storage increase: 50% of current size
- → Maximum single-step instance class change: one rung on the ladder
- → Maximum read replicas per autonomous decision: one
- → No autonomous action between 08:00–09:30 or 17:00–18:00 local business time (peak risk windows)

The maintenance window check deserves its own implementation because it must be reliable even when the capacity system's dependencies are partially degraded:

```

from datetime import datetime, time as dttime
import pytz

BUSINESS_TZ = pytz.timezone("America/New_York")
BLACKOUT_WINDOWS = [
    (dttime(8, 0), dttime(9, 30)),    # morning ramp-up
    (dttime(17, 0), dttime(18, 0)),   # evening peak
    (dttime(23, 30), dttime(0, 30)), # midnight batch jobs -- wraps midnight
]

def is_in_blackout_window() -> bool:
    now_local = datetime.now(BUSINESS_TZ).time()
    for start, end in BLACKOUT_WINDOWS:
        if start <= end:
            if start <= now_local <= end:
                return True
        else:
            # Window wraps midnight
            if now_local >= start or now_local <= end:
                return True
    return False

```

Circuit breaker: If three consecutive capacity actions within a 24-hour window result in validation failures, the circuit opens and the system stops acting autonomously. It continues to forecast and queue recommendations but routes them all to the escalation queue instead of executing. The circuit resets only when a human operator explicitly closes it after reviewing the failure history. This pattern borrows from distributed systems resilience design — the circuit breaker prevents a malfunctioning autonomous component from repeatedly hammering infrastructure in a degraded state.

```

CIRCUIT_BREAKER_KEY = "capacity:circuit_breaker:open"
FAILURE_COUNTER_KEY = "capacity:circuit_breaker:failure_count"
FAILURE_THRESHOLD = 3
FAILURE_WINDOW_SECONDS = 86400 # 24 hours

def record_action_failure(r: redis.Redis):
    count = r.incr(FAILURE_COUNTER_KEY)
    r.expire(FAILURE_COUNTER_KEY, FAILURE_WINDOW_SECONDS)
    if count >= FAILURE_THRESHOLD:
        r.set(CIRCUIT_BREAKER_KEY, "open")
        log.error("CIRCUIT BREAKER OPENED: %d consecutive failures. "
                  "All capacity actions paused pending human review.", count)

def is_circuit_open(r: redis.Redis) -> bool:
    return r.exists(CIRCUIT_BREAKER_KEY) == 1

def close_circuit(r: redis.Redis):
    """Called by human operator after reviewing failure history."""
    r.delete(CIRCUIT_BREAKER_KEY)
    r.delete(FAILURE_COUNTER_KEY)
    log.info("Circuit breaker closed by operator.")

```

The combination of pre-action validation, bounded envelopes, blackout windows, and a circuit breaker gives the autonomous system a defined safety surface. It will not fix every problem perfectly, but it will not make a bad situation dramatically worse through unbounded autonomous action.

Key Takeaways

- → **Signal honesty before forecasting accuracy:** An autonomous capacity system is only as reliable as its observability layer. Measure all five resource dimensions — storage, memory, compute, I/O, and connection throughput — at the database instance level, separated from the systems being monitored, before investing in forecasting sophistication.
- → **Match the forecasting model to the resource pattern:** Use linear regression for monotonic resources like storage, seasonal models such as Prophet or SARIMA for cyclical resources like connection counts and CPU, and percentile-based headroom analysis for memory pressure. Applying a single model across all resource types produces forecasts that are right on average and wrong at the moments that matter.

- → **Preserve Terraform state integrity:** When Terraform manages your infrastructure, the autonomous system must write to the variable layer and trigger controlled Terraform runs rather than calling cloud APIs directly. Out-of-band API calls create state drift that the next planned `terraform apply` will silently revert, potentially causing unplanned changes at the worst possible moment.
- → **Calibrate the action hierarchy to reversibility:** Not all capacity actions carry the same risk. Connection pool adjustments are immediate and costless to reverse; instance resizes force failover windows; storage contractions are irreversible on most platforms. The autonomous system should act freely at the low-risk end of the hierarchy and escalate — not act autonomously — as the cost and irreversibility of an action increase.
- → **Close the loop with validated outcomes and a circuit breaker:** Executing a provisioning action and moving on is incomplete automation. The system must validate that each action achieved its intended effect after a resource-specific settlement period, feed validated outcomes back into the forecasting model, and trip a circuit breaker after a defined number of consecutive validation failures to prevent a malfunctioning component from compounding infrastructure problems through repeated unchecked action.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 15: Intelligent Workload Classification

Autonomous database systems can only act intelligently when they understand *what* they're running. Query tuning, resource allocation, connection routing, and scheduling decisions all depend on knowing whether a given workload is OLTP, analytical, batch, ETL, or something in between — and that classification must happen continuously, at runtime, without human annotation. This chapter builds the full stack of intelligent workload classification: from raw signal extraction through feature engineering, model-based classification, and the downstream control loops that consume those labels to make autonomous decisions. The architecture described here runs in production today, using SQL Server's Query Store and DMV telemetry as primary data sources, augmented by custom instrumentation and ML inference layers that operate with sub-second latency for hot decisions and background model refresh cycles for drift correction.

```
graph LR
    A[Query Store / DMV Telemetry] --> B[Signal Extraction]
    B --> C[Feature Engineering  
burst coef, plan instability, etc.]
    C --> D[Classification Model  
LightGBM/XGBoost]
    D --> E{Confidence >= 0.65?}
    E -->|yes| F[Hysteresis Filter]
    E -->|no| G[Human Review Queue]
    F -->|stable 3 intervals| H[Routing & Resource Policy  
pool, timeout, work_mem]
    D -.drift monitored.-> I[Concept Drift Detector]
    I -->|PSI > 0.2| J[Retraining Pipeline]
    J -.promotes new model.-> D
```

The end-to-end classification loop: raw telemetry becomes feature vectors, a model assigns a workload class, a hysteresis filter prevents thrashing before routing changes are applied, and drift detection keeps the model calibrated over time.

15.1 Signal Extraction: Reading Workload DNA from Native Instrumentation

Classification begins with signals. SQL Server exposes rich runtime telemetry, but the signals that matter for workload classification are not the obvious ones like CPU or memory — they are behavioral fingerprints embedded in execution statistics, wait profiles, and access patterns.

The core features for workload classification fall into four categories:

- → **Execution profile:** rows examined vs. rows returned ratio, execution frequency, plan stability
- → **Resource signature:** buffer hits vs. disk reads, temp space usage, parallelism degree
- → **Wait topology:** the distribution of wait types encountered during execution
- → **Temporal pattern:** time-of-day clustering, burst coefficient, inter-arrival time variance

SQL Server exposes this through `sys.query_store_runtime_stats`, `sys.dm_exec_query_stats`, and `sys.dm_exec_sessions`. The challenge is that Query Store normalizes query text and loses some instance-level variance. Augmenting it with lightweight Extended Events sampling (`sql_batch_completed`) and a collector that snapshots `sys.dm_exec_requests` at 1-second intervals gives you the full picture.

```
-- Extract equivalent workload signals from Query Store
-- Maps to the same feature vector schema used for classification
SELECT
    qsq.query_id,
    qsqt.query_sql_text,
    SUM(qsrs.count_executions)                                AS
total_calls,
    AVG(qsrs.avg_duration / 1000.0)                          AS
avg_exec_ms,
    STDEV(qsrs.avg_duration) / NULLIF(AVG(qsrs.avg_duration), 0) AS
exec_time_cv,
    AVG(qsrs.avg_rowcount)                                    AS
avg_rows_returned,
    AVG(qsrs.avg_logical_io_reads)                            AS
avg_logical_reads,
    AVG(qsrs.avg_physical_io_reads)                            AS
avg_physical_reads,
    AVG(qsrs.avg_tempdb_space_used)                            AS
avg_temp_writes,
    AVG(qsrs.avg_log_bytes_used)                              AS
avg_log_bytes,
    SUM(qsrs.count_executions) /
        NULLIF(DATEDIFF(HOUR,
            MIN(qsrs.first_execution_time),
            MAX(qsrs.last_execution_time)), 0)                AS
calls_per_hour,
    AVG(qsrs.avg_dop)                                          AS
avg_parallelism
FROM sys.query_store_query          qsq
JOIN sys.query_store_query_text    qsqt ON qsq.query_text_id =
qsqt.query_text_id
JOIN sys.query_store_plan          qspp ON qsq.query_id      = qspp.query_id
JOIN sys.query_store_runtime_stats qsrs ON qspp.plan_id      = qsrs.plan_id
WHERE qsrs.count_executions > 10
GROUP BY qsq.query_id, qsqt.query_sql_text
ORDER BY SUM(qsrs.avg_duration * qsrs.count_executions) DESC;
```

Wait profiles require a separate collection path. `sys.dm_os_wait_stats` gives cumulative waits; per-query waits come from `sys.dm_exec_query_stats` joined with `sys.dm_exec_query_plan` for per-plan wait attribution. A 1-second polling collector that snapshots `sys.dm_exec_requests` builds a wait histogram per query pattern over time.

The wait topology feature is then encoded as a distribution vector: what fraction of total wait time falls into Lock, LWLock/Buffer, IO, CPU, Network, and Idle categories. OLTP workloads show high Lock and short CPU spikes. Analytical workloads show sustained CPU with sequential IO. Batch ETL workloads show high IO and Log write waits with low Lock contention.

15.2 Feature Engineering for Runtime Classification

Raw telemetry numbers are not directly usable by a classifier. The feature engineering layer transforms them into scale-invariant, semantically meaningful representations that generalize across databases of different sizes and query volumes.

The derived features that carry the most discriminative signal are:

Feature	Formula	Interpretation
---------	---------	----------------

---	---	---
-----	-----	-----

<code>scan_dominance</code>	<code>seq_scan_rows / total_rows_examined</code>	High = analytical
-----------------------------	--	-------------------

<code>write_amplification</code>	<code>log_bytes / (rows_modified + 1)</code>	High = batch/ETL
----------------------------------	--	------------------

<code>result_compression</code>	<code>rows_returned / rows_examined</code>	Low = aggregation-heavy
---------------------------------	--	-------------------------

<code>temporal_burst_coef</code>	<code>calls_p99 / calls_median</code> over 5-min windows	High = bursty OLTP
----------------------------------	--	--------------------

<code>plan_instability</code>	<code>distinct_plans / total_calls</code>	High = parameter sensitivity
-------------------------------	---	------------------------------

<code>parallelism_affinity</code>	<code>avg_dop / max_dop</code>	High = analytical intent
-----------------------------------	--------------------------------	--------------------------

The following Python feature engineering pipeline runs as a scheduled collector — pulling from both databases, computing derived features, and publishing to a Redis time-series store that feeds the classification inference layer.

```

import pyodbc
import redis
import json
import time
from dataclasses import dataclass, asdict
from typing import Optional
import numpy as np

@dataclass
class WorkloadFeatureVector:
    query_id: str
    avg_exec_ms: float
    exec_time_cv: float          # coefficient of variation
    avg_rows_returned: float
    buffer_hit_ratio: float
    avg_physical_reads: float
    avg_temp_writes: float
    avg_log_bytes: float
    calls_per_hour: float
    avg_parallelism: float
    # Derived features
    result_compression: float    # rows_returned / rows_examined
    temporal_burst_coef: float
    plan_instability: float
    write_amplification: float

def compute_derived_features(raw: dict, burst_history: list[float]) -> dict:
    """
    Compute scale-invariant derived features from raw telemetry.
    burst_history: list of calls_per_hour values from last 12 x 5-min windows
    """
    rows_examined = raw['avg_rows_returned'] / max(raw['buffer_hit_ratio'], 0.001)

    result_compression = raw['avg_rows_returned'] / max(rows_examined, 1)

    # Burst coefficient: ratio of 95th percentile to median over recent windows
    if len(burst_history) >= 4:
        p95 = float(np.percentile(burst_history, 95))
        median = float(np.median(burst_history))
        burst_coef = p95 / max(median, 0.1)
    else:
        burst_coef = 1.0

    write_amplification = raw['avg_log_bytes'] / max(raw['avg_rows_returned'], 1)

    return {
        'result_compression': result_compression,
        'temporal_burst_coef': burst_coef,
        'write_amplification': write_amplification,
    }

class FeatureCollector:
    def __init__(self, mssql_dsn: str, redis_url: str):

```

```

self.mssql_conn = pyodbc.connect(mssql_dsn)
self.redis = redis.from_url(redis_url)
self.burst_cache: dict[str, list[float]] = {}

def collect_and_publish(self):
    ms_features = self._collect_sqlserver()

    for fv in ms_features:
        key = f"wl:features:sqlserver:{fv.query_id}"
        self.redis.setex(key, 3600, json.dumps(asdict(fv)))
        # Append to time-series for burst calculation
        ts_key = f"wl:timeseries:sqlserver:{fv.query_id}"
        self.redis.lpush(ts_key, fv.calls_per_hour)
        self.redis.ltrim(ts_key, 0, 11) # keep 12 windows = 60 minutes

def _collect_sqlserver(self) -> list[WorkloadFeatureVector]:
    cur = self.mssql_conn.cursor()
    cur.execute("""
        SELECT
            CAST(qsq.query_id AS VARCHAR(20)) AS query_id,
            AVG(qsrs.avg_duration / 1000.0) AS avg_exec_ms,
            STDEV(qsrs.avg_duration) / NULLIF(AVG(qsrs.avg_duration),0) AS
exec_time_cv,
            AVG(qsrs.avg_rowcount) AS avg_rows,
            AVG(qsrs.avg_logical_io_reads) AS avg_logical,
            AVG(qsrs.avg_physical_io_reads) AS avg_physical,
            AVG(qsrs.avg_tempdb_space_used) AS avg_temp,
            AVG(qsrs.avg_log_bytes_used) AS avg_log,
            SUM(qsrs.count_executions) /
            NULLIF(DATEDIFF(HOUR,
                MIN(qsrs.first_execution_time),
                GETUTCDATE()), 0) AS calls_per_hour,
            AVG(qsrs.avg_dop) AS avg_dop,
            COUNT(DISTINCT qspp.plan_id) * 1.0 /
            NULLIF(SUM(qsrs.count_executions), 0) AS plan_instability
        FROM sys.query_store_query sq
        JOIN sys.query_store_plan qspp ON sq.query_id =
qspp.query_id
        JOIN sys.query_store_runtime_stats qsrs ON qspp.plan_id = qsrs.plan_id
        WHERE qsrs.count_executions > 10
        AND qsrs.last_execution_time > DATEADD(MINUTE, -60, GETUTCDATE())
        GROUP BY sq.query_id
    """)
    results = []
    for row in cur.fetchall():
        burst_key = f"wl:timeseries:sqlserver:{row.query_id}"
        history = [float(v) for v in self.redis.lrange(burst_key, 0, 11)]
        derived = compute_derived_features({
            'avg_rows_returned': row.avg_rows or 0,
            'buffer_hit_ratio': (row.avg_logical - (row.avg_physical or 0)) /
                max(row.avg_logical or 1, 1),
            'avg_log_bytes': row.avg_log or 0,
        }, history)

```

```

        fv = WorkloadFeatureVector(
            query_id=row.query_id,
            database_type='sqlserver',
            avg_exec_ms=row.avg_exec_ms or 0,
            exec_time_cv=row.exec_time_cv or 0,
            avg_rows_returned=row.avg_rows or 0,
            buffer_hit_ratio=(row.avg_logical - (row.avg_physical or 0)) /
                             max(row.avg_logical or 1, 1),
            avg_physical_reads=row.avg_physical or 0,
            avg_temp_writes=row.avg_temp or 0,
            avg_log_bytes=row.avg_log or 0,
            calls_per_hour=row.calls_per_hour or 0,
            avg_parallelism=row.avg_dop or 1,
            plan_instability=row.plan_instability or 0,
            **derived
        )
        results.append(fv)
    return results

```

15.3 The Classification Model: Architecture and Label Taxonomy

The classification model sits at the core of the autonomous control loop. It receives feature vectors and produces workload labels that drive downstream decisions. The label taxonomy needs to be precise enough to guide action but stable enough to avoid thrashing.

A five-class taxonomy works well across all production scenarios:

- → **OLTP_POINT**: Short-duration, high-frequency, index-bound, single-digit row access. Classic transactional. Action: prioritize in connection pools, protect from query plan regression.
- → **OLTP_MODERATE**: Slightly wider queries — small range scans, multi-table joins, under 100ms. Action: standard SLA tier.
- → **ANALYTICAL**: Long-running, high-row-count aggregations, sequential scans, parallel execution. Action: route to read replica, assign lower CPU priority, allow longer lock waits.
- → **BATCH_ETL**: High write amplification, sequential access, predictable schedule, large tempdb usage. Action: schedule during maintenance window, pre-warm buffers, expand the index maintenance budget post-run.
- → **MAINTENANCE**: index rebuilds and reorganizations, statistics updates, DBCC operations. Action: track separately, do not apply SLA enforcement, use as calibration signal.

The classifier itself is a gradient-boosted tree model (XGBoost or LightGBM) trained on labeled historical data from Query Store. The labeling phase is the only place human judgment enters: a DBA manually labels 500–1000 representative query patterns once, and the model generalizes from there. Semi-supervised bootstrapping can extend coverage to new query patterns by treating high-confidence predictions as pseudo-labels for retraining.

In production, the classifier runs as a FastAPI microservice with a 5ms SLA for batch classification of up to 200 feature vectors per request. The model is loaded into memory and refreshed weekly via a background retraining pipeline triggered by concept drift detection.


```

import lightgbm as lgb
import numpy as np
import joblib
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
from enum import Enum

class WorkloadClass(str, Enum):
    OLTP_POINT = "OLTP_POINT"
    OLTP_MODERATE = "OLTP_MODERATE"
    ANALYTICAL = "ANALYTICAL"
    BATCH_ETL = "BATCH_ETL"
    MAINTENANCE = "MAINTENANCE"

FEATURE_COLUMNS = [
    'avg_exec_ms', 'exec_time_cv', 'avg_rows_returned',
    'buffer_hit_ratio', 'avg_physical_reads', 'avg_temp_writes',
    'avg_log_bytes', 'calls_per_hour', 'avg_parallelism',
    'result_compression', 'temporal_burst_coef',
    'plan_instability', 'write_amplification'
]

LABEL_MAP = {0: WorkloadClass.OLTP_POINT,
              1: WorkloadClass.OLTP_MODERATE,
              2: WorkloadClass.ANALYTICAL,
              3: WorkloadClass.BATCH_ETL,
              4: WorkloadClass.MAINTENANCE}

class ClassificationRequest(BaseModel):
    query_id: str
    features: dict

class ClassificationResponse(BaseModel):
    query_id: str
    workload_class: WorkloadClass
    confidence: float
    probabilities: dict

app = FastAPI()
model: lgb.Booster = joblib.load("/models/workload_classifier_v3.pkl")

@app.post("/classify", response_model=List[ClassificationResponse])
def classify_batch(requests: List[ClassificationRequest]):
    if not requests:
        return []

    feature_matrix = np.array([
        [r.features.get(col, 0.0) for col in FEATURE_COLUMNS]
        for r in requests
    ])

    # LightGBM returns probability matrix (n_samples x n_classes)

```

```

proba_matrix = model.predict(feature_matrix)

results = []
for req, probas in zip(requests, proba_matrix):
    predicted_idx = int(np.argmax(probas))
    predicted_class = LABEL_MAP[predicted_idx]
    confidence = float(probas[predicted_idx])

    # Refuse to classify if confidence is below threshold –
    # route to human review queue instead
    if confidence < 0.65:
        predicted_class = WorkloadClass.OLTP_MODERATE # safe default
        confidence = 0.0 # signals uncertainty to downstream consumers

    results.append(ClassificationResponse(
        query_id=req.query_id,
        workload_class=predicted_class,
        confidence=confidence,
        probabilities={LABEL_MAP[i].value: float(p)
                      for i, p in enumerate(probas)}
    ))
return results

```

The confidence threshold of 0.65 is not arbitrary. Below this level, empirical testing shows that misclassifications cause more harm (wrong resource allocation, incorrect routing) than the safe default of OLTP_MODERATE. Queries below threshold are published to a review queue consumed weekly by the DBA for manual labeling — feeding the retraining loop.

15.4 Real-Time Classification Routing and Resource Policy Enforcement

Classification labels are only valuable when they drive action at runtime. The downstream control loop connects workload labels to three enforcement mechanisms: connection pool routing, resource group assignment, and query timeout policies.

Connection Routing via Resource Governor Classification

SQL Server has no external pooler tier to reconfigure — routing happens in the Resource Governor classifier function and the AG read-only routing list. When the classification service reclassifies a query pattern (e.g., a formerly OLTP query has migrated to ANALYTICAL due to data growth), the orchestration agent updates the `dbo.workload_routing_rules` table below; the classifier function picks up the new routing at the next connection without any restart.

```

-- SQL Server: Resource Governor workload group assignment
-- Classification function called by Resource Governor at connection time
-- Updated dynamically based on classification service output

-- Create/update the classifier function
CREATE OR ALTER FUNCTION dbo.fn_workload_classifier()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @workload_group SYSNAME;
    DECLARE @app_name        NVARCHAR(256) = APP_NAME();
    DECLARE @host_name        NVARCHAR(128) = HOST_NAME();
    DECLARE @login_name       SYSNAME      = SUSER_SNAME();

    -- Dynamic routing table updated by classification agent
    SELECT @workload_group = resource_group_name
    FROM dbo.workload_routing_rules
    WHERE (app_name_pattern IS NULL OR @app_name LIKE app_name_pattern)
        AND (login_pattern IS NULL OR @login_name LIKE login_pattern)
        AND is_active = 1
    ORDER BY priority ASC;

    -- Fall back to default if no rule matches
    IF @workload_group IS NULL
        SET @workload_group = N'WG_OLTP_MODERATE';

    RETURN @workload_group;
END;
GO

-- Supporting routing rules table (populated by classification agent)
CREATE TABLE IF NOT EXISTS dbo.workload_routing_rules (
    rule_id          INT IDENTITY PRIMARY KEY,
    app_name_pattern NVARCHAR(256) NULL,
    login_pattern     SYSNAME        NULL,
    resource_group_name SYSNAME      NOT NULL,
    priority          INT            NOT NULL DEFAULT 100,
    is_active         BIT            NOT NULL DEFAULT 1,
    updated_at        DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME()
);

-- Resource Governor groups matching the five-class taxonomy
ALTER RESOURCE POOL RP_OLTP
    WITH (MIN_CPU_PERCENT = 20, MAX_CPU_PERCENT = 80,
        MIN_MEMORY_PERCENT = 10, MAX_MEMORY_PERCENT = 40);

ALTER RESOURCE POOL RP_ANALYTICAL
    WITH (MIN_CPU_PERCENT = 0, MAX_CPU_PERCENT = 60,
        MIN_MEMORY_PERCENT = 20, MAX_MEMORY_PERCENT = 60);

ALTER RESOURCE POOL RP_BATCH
    WITH (MIN_CPU_PERCENT = 0, MAX_CPU_PERCENT = 40,

```

```

        MIN_MEMORY_PERCENT = 5, MAX_MEMORY_PERCENT = 30);

ALTER WORKLOAD GROUP WG_OLTP_POINT
    WITH (IMPORTANCE = HIGH, REQUEST_MAX_CPU_TIME_SEC = 5,
          REQUEST_MAX_MEMORY_GRANT_PERCENT = 5,
          GROUP_MAX_REQUESTS = 500)
    USING RP_OLTP;

ALTER WORKLOAD GROUP WG_OLTP_MODERATE
    WITH (IMPORTANCE = MEDIUM, REQUEST_MAX_CPU_TIME_SEC = 30,
          REQUEST_MAX_MEMORY_GRANT_PERCENT = 10,
          GROUP_MAX_REQUESTS = 200)
    USING RP_OLTP;

ALTER WORKLOAD GROUP WG_ANALYTICAL
    WITH (IMPORTANCE = LOW, REQUEST_MAX_CPU_TIME_SEC = 300,
          REQUEST_MAX_MEMORY_GRANT_PERCENT = 25,
          GROUP_MAX_REQUESTS = 20)
    USING RP_ANALYTICAL;

ALTER WORKLOAD GROUP WG_BATCH_ETL
    WITH (IMPORTANCE = LOW, REQUEST_MAX_CPU_TIME_SEC = 3600,
          REQUEST_MAX_MEMORY_GRANT_PERCENT = 20,
          GROUP_MAX_REQUESTS = 10)
    USING RP_BATCH;

ALTER RESOURCE GOVERNOR
    WITH (CLASSIFIER_FUNCTION = dbo.fn_workload_classifier);

ALTER RESOURCE GOVERNOR RECONFIGURE;

```

The key design tension here is that Resource Governor applies its classification at connection time, not at query execution time. A connection from a reporting application that also issues occasional OLTP point lookups will be assigned the ANALYTICAL group for the entire session. The mitigation is to route clients through application-layer connection proxies that maintain separate connection pools per workload class, so the Resource Governor classifier function sees clean, purpose-specific connections rather than mixed-use sessions.

Dynamic Reclassification and Hysteresis

Reclassification events — when a query pattern transitions from one workload class to another — can cause resource thrashing if acted upon immediately. A query experiencing a temporary data skew might look ANALYTICAL for a few minutes before settling back to OLTP. Acting on every reclassification signal wastes reconfiguration cycles and destabilizes steady state.

The solution is a hysteresis filter. A reclassification is only committed to the routing table when the new classification has been stable for a configurable number of consecutive scoring intervals. The following agent implements this pattern:

```

import time
import collections
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class ClassificationState:
    current_class: str
    candidate_class: str
    candidate_streak: int
    last_changed: float # epoch seconds

    def should_reclassify(self, new_class: str,
                        hysteresis_periods: int = 3) -> bool:
        """
        Return True only when the new class has been consistently
        observed for `hysteresis_periods` consecutive scoring intervals.
        """
        if new_class == self.current_class:
            self.candidate_class = new_class
            self.candidate_streak = 0
            return False

        if new_class == self.candidate_class:
            self.candidate_streak += 1
        else:
            self.candidate_class = new_class
            self.candidate_streak = 1

        return self.candidate_streak >= hysteresis_periods

class ReclassificationAgent:
    def __init__(self, classifier_url: str, mssql_dsn: str,
                scoring_interval_sec: int = 60,
                hysteresis_periods: int = 3):
        self.classifier_url = classifier_url
        self.mssql_dsn = mssql_dsn
        self.interval = scoring_interval_sec
        self.hysteresis = hysteresis_periods
        self.states: dict[str, ClassificationState] = {}

    def run(self):
        import requests, pyodbc
        conn = pyodbc.connect(self.mssql_dsn)
        while True:
            features = self._fetch_features(conn)
            if not features:
                time.sleep(self.interval)
                continue

            response = requests.post(
                f"{self.classifier_url}/classify",
                json=features,

```

```

        timeout=10
    )
    classifications = response.json()

    for result in classifications:
        qid          = result['query_id']
        new_class     = result['workload_class']
        confidence    = result['confidence']

        # Skip uncertain predictions entirely
        if confidence == 0.0:
            continue

        if qid not in self.states:
            self.states[qid] = ClassificationState(
                current_class=new_class,
                candidate_class=new_class,
                candidate_streak=0,
                last_changed=time.time()
            )
            self._apply_routing(conn, qid, new_class)
            continue

        state = self.states[qid]
        if state.should_reclassify(new_class, self.hysteresis):
            old_class = state.current_class
            state.current_class = new_class
            state.candidate_streak = 0
            state.last_changed = time.time()
            self._apply_routing(conn, qid, new_class)
            self._emit_reclassification_event(
                conn, qid, old_class, new_class
            )

    time.sleep(self.interval)

def _apply_routing(self, conn, query_id: str, workload_class: str):
    with conn.cursor() as cur:
        cur.execute(
            "EXEC dbo.assign_workload_resource_group @query_id = ?,
@workload_class = ?",
            int(query_id), workload_class
        )
    conn.commit()

def _emit_reclassification_event(self, conn, query_id: str,
                                old_class: str, new_class: str):
    with conn.cursor() as cur:
        cur.execute("""
            INSERT INTO dbo.workload_reclassification_log
                (query_id, old_class, new_class, event_time)
            VALUES (?, ?, ?, SYSUTCDATETIME())
            """, int(query_id), old_class, new_class)

```

```
conn.commit()

def _fetch_features(self, conn) -> list:
    # Fetch from Redis or directly from Query Store
    # Returns list of {query_id, features: {...}} dicts
    pass
```

Three consecutive scoring intervals — with a 60-second default interval — means a reclassification event must persist for three minutes before action is taken. This eliminates transient noise while still responding to genuine workload character changes within a few minutes.

15.5 Concept Drift Detection and Model Refresh

Workload classifiers degrade over time. Schema changes, new application releases, data volume growth, and seasonal patterns all shift the underlying feature distributions. A model trained on last quarter's telemetry may produce subtly wrong classifications on today's workload without any obvious failure signal — queries are still classified, resource policies are still applied, but the assignments are increasingly misaligned with actual behavior.

Detecting this drift without ground-truth labels requires statistical process control applied to the model's own output distribution. Three signals are monitored continuously:

1. **Class distribution drift:** The proportion of queries assigned to each class should be stable over time. A sudden shift — say, ANALYTICAL rising from 15% to 35% without a known schema or application change — indicates either a genuine workload shift or model degradation.
2. **Confidence distribution collapse:** If the average confidence score across all classifications drops below a threshold, the model is encountering feature vectors that it has not seen before. The distribution of `max_proba` values should be stable; a leftward shift signals drift.
3. **Feature distribution shift:** The input feature distributions themselves can be monitored using a Population Stability Index (PSI) computed between the current 7-day window and the training baseline. $PSI > 0.2$ on any feature is a trigger for retraining.


```

import numpy as np
from scipy import stats
from dataclasses import dataclass
from typing import Optional

@dataclass
class DriftReport:
    feature_name: str
    psi_score: float
    ks_statistic: float
    ks_pvalue: float
    drift_detected: bool

def compute_psi(baseline: np.ndarray, current: np.ndarray,
                n_bins: int = 10) -> float:
    """
    Population Stability Index.
    PSI < 0.1 : no significant drift
    PSI 0.1-0.2: moderate drift, monitor
    PSI > 0.2 : significant drift, retrain
    """
    baseline = baseline[np.isfinite(baseline)]
    current = current[np.isfinite(current)]
    if len(baseline) == 0 or len(current) == 0:
        return 0.0

    # Build bins from baseline distribution
    bin_edges = np.percentile(baseline, np.linspace(0, 100, n_bins + 1))
    bin_edges = np.unique(bin_edges) # dedup if percentiles collapse
    if len(bin_edges) < 2:
        return 0.0

    baseline_counts, _ = np.histogram(baseline, bins=bin_edges)
    current_counts, _ = np.histogram(current, bins=bin_edges)

    # Avoid division by zero with Laplace smoothing
    baseline_pct = (baseline_counts + 0.5) / (len(baseline) + 0.5 * n_bins)
    current_pct = (current_counts + 0.5) / (len(current) + 0.5 * n_bins)

    psi = np.sum((current_pct - baseline_pct) * np.log(current_pct / baseline_pct))
    return float(psi)

def detect_feature_drift(baseline_features: dict[str, np.ndarray],
                        current_features: dict[str, np.ndarray],
                        psi_threshold: float = 0.2) -> list[DriftReport]:
    reports = []
    for feature_name in baseline_features:
        if feature_name not in current_features:
            continue
        baseline = baseline_features[feature_name]
        current = current_features[feature_name]

        psi = compute_psi(baseline, current)

```

```

ks_stat, ks_pval = stats.ks_2samp(baseline, current)

# Drift declared when PSI exceeds threshold OR KS test rejects at 1%
drift = psi > psi_threshold or ks_pval < 0.01

reports.append(DriftReport(
    feature_name=feature_name,
    psi_score=psi,
    ks_statistic=ks_stat,
    ks_pvalue=ks_pval,
    drift_detected=drift
))
return reports

class DriftMonitor:
    def __init__(self, redis_client, retraining_trigger_callback):
        self.redis = redis_client
        self.trigger = retraining_trigger_callback
        self.baseline: Optional[dict[str, np.ndarray]] = None

    def load_baseline(self, model_version: str):
        """Load feature statistics captured at training time."""
        raw = self.redis.get(f"model:baseline_features:{model_version}")
        if raw:
            import json
            data = json.loads(raw)
            self.baseline = {k: np.array(v) for k, v in data.items()}

    def check(self, current_features: dict[str, np.ndarray],
              model_version: str):
        if self.baseline is None:
            self.load_baseline(model_version)
        if self.baseline is None:
            return # no baseline to compare against

        reports = detect_feature_drift(self.baseline, current_features)
        drifted = [r for r in reports if r.drift_detected]

        if drifted:
            # Publish drift report for observability
            import json
            drift_summary = {
                r.feature_name: {'psi': r.psi_score, 'ks_p': r.ks_pvalue}
                for r in drifted
            }
            self.redis.setex(
                f"model:drift_report:{model_version}",
                86400,
                json.dumps(drift_summary)
            )
            # Fire retraining if any feature shows significant drift
            self.trigger(reason="feature_drift", drifted_features=drifted)

```

Retraining Pipeline

When drift is detected, the retraining pipeline executes in four stages:

1. **Data extraction:** Pull the last 90 days of feature vectors from the time-series store, filtered to queries with at least 50 executions for statistical reliability.
2. **Pseudo-label generation:** High-confidence predictions from the current model (confidence ≥ 0.85) on the new data become training labels. This is the semi-supervised step — it allows the model to adapt to new query patterns without requiring fresh human annotation.
3. **Model training and validation:** The new model is trained on the combined dataset (original labeled data + pseudo-labeled new data) with a held-out validation set from the most recent 14 days. The new model must exceed the current model's F1 score on the validation set by at least 0.01 before it is promoted.
4. **Shadow deployment:** The new model runs in parallel with the existing model for 24 hours, its classifications logged but not applied. If the classification agreement rate between old and new model drops below 80% on OLTP_POINT queries — the highest-stakes class — the deployment is halted and the divergence is flagged for human review.

```

import lightgbm as lgb
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
import joblib

def retrain_classifier(
    labeled_X: np.ndarray,
    labeled_y: np.ndarray,
    unlabeled_X: np.ndarray,
    current_model: lgb.Booster,
    pseudo_label_threshold: float = 0.85,
    min_f1_improvement: float = 0.01
) -> Optional[lgb.Booster]:

    # Stage 2: generate pseudo-labels from current model
    if len(unlabeled_X) > 0:
        proba = current_model.predict(unlabeled_X)
        max_proba = proba.max(axis=1)
        pseudo_labels = proba.argmax(axis=1)

        high_conf_mask = max_proba >= pseudo_label_threshold
        pseudo_X = unlabeled_X[high_conf_mask]
        pseudo_y = pseudo_labels[high_conf_mask]

        combined_X = np.vstack([labeled_X, pseudo_X])
        combined_y = np.concatenate([labeled_y, pseudo_y])
    else:
        combined_X = labeled_X
        combined_y = labeled_y

    # Stage 3: train and validate
    X_train, X_val, y_train, y_val = train_test_split(
        combined_X, combined_y, test_size=0.2,
        stratify=combined_y, random_state=42
    )

    params = {
        'objective': 'multiclass',
        'num_class': 5,
        'learning_rate': 0.05,
        'num_leaves': 63,
        'min_child_samples': 30,
        'feature_fraction': 0.8,
        'bagging_fraction': 0.8,
        'bagging_freq': 5,
        'verbose': -1
    }

    dtrain = lgb.Dataset(X_train, label=y_train)
    dval = lgb.Dataset(X_val, label=y_val, reference=dtrain)

    new_model = lgb.train(

```

```

    params,
    dtrain,
    num_boost_round=500,
    valid_sets=[dval],
    callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)]
)

# Evaluate both models on validation set
new_preds = new_model.predict(X_val).argmax(axis=1)
current_preds = current_model.predict(X_val).argmax(axis=1)

new_f1 = f1_score(y_val, new_preds, average='weighted')
current_f1 = f1_score(y_val, current_preds, average='weighted')

if new_f1 < current_f1 + min_f1_improvement:
    return None # not good enough – keep existing model

return new_model

```

15.6 Workload Fingerprinting for Query Plan Stability

Classification labels serve a second purpose beyond resource allocation: they anchor query plan stability enforcement. A query classified as OLTP_POINT should never suddenly acquire a parallel plan or a hash join it did not have before — such changes are almost always regressions for this workload type. An ANALYTICAL query, conversely, should be allowed — encouraged — to use new parallel strategies as the optimizer sees them.

This insight leads to class-specific plan enforcement policies:

Workload Class	Plan Enforcement Policy
OLTP_POINT	Freeze plan on first stable execution; force via SQL Server plan guide or Query Store plan forcing
OLTP_MODERATE	Soft lock: alert on plan change, auto-test new plan before promoting
ANALYTICAL	No plan locking; allow optimizer freedom; monitor regression by execution time
BATCH_ETL	Lock to validated plan for each ETL step; rebuild plan on schema changes
MAINTENANCE	No plan management; use default optimizer behavior

SQL Server: Plan Fingerprinting and Stability Monitoring

```

-- Track plan stability per query class
CREATE TABLE dbo.query_plan_registry (
    query_id          INT          NOT NULL,
    workload_class    NVARCHAR(32) NOT NULL,
    plan_hash         BINARY(8)    NOT NULL, -- query_plan_hash from Query Store
    first_seen        DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME(),
    last_seen         DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME(),
    execution_count    BIGINT       NOT NULL DEFAULT 1,
    avg_exec_ms       FLOAT        NOT NULL,
    is_locked         BIT          NOT NULL DEFAULT 0,
    CONSTRAINT PK_query_plan_registry PRIMARY KEY (query_id, plan_hash)
);
GO

-- Called by the classification agent when a new plan hash
-- is observed for a locked query
CREATE OR ALTER PROCEDURE dbo.handle_plan_change
    @query_id          INT,
    @new_plan_hash     BINARY(8),
    @avg_exec_ms       FLOAT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @class NVARCHAR(32), @locked_plan BINARY(8),
            @locked_exec_ms FLOAT, @regression_pct FLOAT;

    SELECT @class = workload_class, @locked_plan = plan_hash,
           @locked_exec_ms = avg_exec_ms
    FROM dbo.query_plan_registry
    WHERE query_id = @query_id AND is_locked = 1;

    IF @@ROWCOUNT = 0
    BEGIN
        -- No locked plan yet; record this as baseline
        INSERT INTO dbo.query_plan_registry
            (query_id, workload_class, plan_hash, avg_exec_ms, is_locked)
        SELECT @query_id, workload_class, @new_plan_hash, @avg_exec_ms,
            CASE WHEN workload_class IN (N'OLTP_POINT', N'BATCH_ETL')
                THEN 1 ELSE 0 END
        FROM dbo.workload_routing_map
        WHERE query_id = @query_id;
        RETURN;
    END;

    -- If the plan changed on a locked class, evaluate the regression
    IF @locked_plan <> @new_plan_hash
    BEGIN
        SET @regression_pct = (@avg_exec_ms - @locked_exec_ms)
            / NULLIF(@locked_exec_ms, 0) * 100;

        INSERT INTO dbo.plan_change_alerts
            (query_id, workload_class, old_plan_hash, new_plan_hash,

```

```

        old_exec_ms, new_exec_ms, regression_pct, detected_at)
VALUES
    (@query_id, @class, @locked_plan, @new_plan_hash,
     @locked_exec_ms, @avg_exec_ms, @regression_pct, SYSUTCDATETIME());

-- For OLTP_POINT: flag for plan forcing via the locked plan hash.
-- The orchestration layer forces the known-good plan through
-- sys.sp_query_store_force_plan.
IF @class = N'OLTP_POINT' AND @regression_pct > 10
BEGIN
    UPDATE dbo.query_plan_registry
    SET last_seen = SYSUTCDATETIME()
    WHERE query_id = @query_id AND plan_hash = @locked_plan;

    INSERT INTO dbo.autonomous_actions_log
        (action_type, query_id, detail, performed_at)
    VALUES
        ('PLAN_FORCE_REQUESTED', @query_id,
         'Force locked plan hash for regressed OLTP_POINT query',
         SYSUTCDATETIME());
END;
END;
END;
GO

```

SQL Server: Plan Guide Automation

```

-- Create or refresh a plan guide for a regressed OLTP_POINT query
-- Called by the autonomous agent when a plan regression is detected
CREATE OR ALTER PROCEDURE dbo.enforce_plan_guide
    @query_id      INT,
    @statement_text NVARCHAR(MAX),
    @plan_handle    VARBINARY(64) -- handle of the known-good plan
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @guide_name NVARCHAR(256) = N'ADE_AUTO_' + CAST(@query_id AS NVARCHAR);
    DECLARE @hints      NVARCHAR(MAX);

    -- Extract USE PLAN hint XML from the plan handle
    SELECT @hints = N'OPTION (USE PLAN N'''
                + CAST(query_plan AS NVARCHAR(MAX))
                + N''')'
    FROM sys.dm_exec_query_plan(@plan_handle);

    -- Drop existing guide if present
    IF EXISTS (SELECT 1 FROM sys.plan_guides WHERE name = @guide_name)
        EXEC sp_control_plan_guide N'DROP', @guide_name;

    -- Create new guide forcing the known-good plan
    EXEC sp_create_plan_guide
        @name      = @guide_name,
        @stmt      = @statement_text,
        @type      = N'SQL',
        @module_or_batch = NULL,
        @params    = NULL,
        @hints     = @hints;

    -- Record the enforcement action
    INSERT INTO dbo.autonomous_actions_log
        (action_type, query_id, detail, performed_at)
    VALUES
        ('PLAN_GUIDE_ENFORCED', @query_id,
        N'Guide: ' + @guide_name, SYSUTCDATETIME());
END;
GO

```

The plan guide enforcement path completes the feedback loop: the classifier identifies a query's workload class, the plan registry tracks its execution plan history, and when a regression is detected on a locked class, the autonomous agent restores the known-good plan without waiting for DBA intervention.

15.7 Observability: Making the Classification System Legible

An autonomous classification system that operates as a black box will not survive contact with production operations teams. DBAs and on-call engineers need to understand why a query was assigned to a particular class, what resource policy is being applied, and what recent reclassification events have occurred. Without this visibility, the system will be disabled at the first sign of unexpected behavior.

Three observability surfaces are required:

Classification explanation table — a human-readable record of the dominant features that drove each classification decision.

```
-- Dashboard view: current classification state for all active queries
CREATE OR ALTER VIEW dbo.vw_workload_classification_dashboard AS
SELECT
    qsq.query_id,
    LEFT(qsqt.query_sql_text, 120)                AS query_preview,
    wrm.workload_class,
    wrm.target_pool,
    wrm.statement_timeout_ms,
    wrm.work_mem,
    wrm.updated_at                                AS last_classified,
    ce.confidence,
    ce.explanation_text,
    -- Recent performance context
    AVG(qsrs.avg_duration / 1000.0)              AS avg_exec_ms_recent,
    SUM(qsrs.count_executions)                   AS executions_last_hour,
    MAX(qsrs.last_execution_time)                AS last_execution
FROM dbo.workload_routing_map                    wrm
JOIN sys.query_store_query                      qsq  ON wrm.query_id      = qsq.query_id
JOIN sys.query_store_query_text                qsqt ON qsq.query_text_id =
qsqt.query_text_id
JOIN sys.query_store_plan                      qspp ON qsq.query_id      = qspp.query_id
JOIN sys.query_store_runtime_stats             qsrs ON qspp.plan_id      = qsrs.plan_id
LEFT JOIN dbo.classification_explanations      ce   ON wrm.query_id      = ce.query_id
WHERE qsrs.last_execution_time > DATEADD(HOUR, -1, SYSUTCDATETIME())
GROUP BY
    qsq.query_id, qsqt.query_sql_text, wrm.workload_class,
    wrm.target_pool, wrm.statement_timeout_ms, wrm.work_mem,
    wrm.updated_at, ce.confidence, ce.explanation_text;
```

Reclassification history and audit trail — DBAs need to see the timeline of classification changes for any query and the reason each change was made. This is both an operational tool and an audit record for compliance environments where resource policy decisions must be explainable.

```
-- Reclassification history with feature deltas
CREATE OR ALTER VIEW dbo.workload_reclassification_history AS
SELECT
    rl.query_id,
    rl.old_class,
    rl.new_class,
    rl.event_time,
    -- Feature snapshot at time of reclassification
    ce_new.top_features AS features_at_change,
    ce_new.confidence AS confidence_at_change,
    -- Time in previous class
    DATEDIFF(SECOND,
        LAG(rl.event_time) OVER
            (PARTITION BY rl.query_id ORDER BY rl.event_time),
        rl.event_time) AS seconds_in_prior_class
FROM dbo.workload_reclassification_log rl
LEFT JOIN dbo.classification_explanations ce_new
    ON rl.query_id = ce_new.query_id
```

The three observability layers — current classification state, feature explanations, and reclassification history — together provide the operational transparency required for DBAs to trust the autonomous system. When something unexpected occurs, the first question — "why did this query get classified as ANALYTICAL?" — has an immediate, data-backed answer in `classification_explanations`. The second question — "when did that change, and what triggered it?" — is answered by `workload_reclassification_history`.

Key Takeaways

- → **Workload classification requires behavioral fingerprints, not just resource metrics.** Features like result compression ratio, temporal burst coefficient, write amplification, and plan instability carry far more discriminative signal than raw CPU or memory consumption figures. Extracting these features from Query Store requires derived computation, not direct column reads.
- → **A five-class taxonomy — OLTP_POINT, OLTP_MODERATE, ANALYTICAL, BATCH_ETL, MAINTENANCE — is sufficient to drive all meaningful autonomous control decisions,** including connection pool routing, resource group assignment, query timeout policy, and plan stability enforcement. Finer-grained taxonomies introduce instability without corresponding control benefit.
- → **Hysteresis filtering is non-negotiable in reclassification pipelines.** Acting on every classification signal change causes resource thrashing that is more harmful than a slightly stale classification. Requiring three consecutive scoring intervals of agreement before committing a reclassification balances responsiveness against stability.

- → **Concept drift degrades classifiers silently.** Monitoring the Population Stability Index across all input features and the confidence score distribution of model outputs provides early warning of drift before misclassifications accumulate to operational impact. Semi-supervised pseudo-labeling with a high confidence gate (≥ 0.85) allows continuous model adaptation without requiring ongoing human annotation effort.
- → **Autonomous classification systems require layered observability to earn operational trust.** The classification decision itself, the feature contributions that drove it, and the full reclassification history for each query must all be queryable in real time. Without this transparency, operations teams will override or disable the system at the first unexpected outcome, defeating the purpose of autonomy.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 16: Autonomous Scaling in Cloud Databases

Scaling decisions made by humans are slow, reactive, and constrained by organizational bandwidth. A senior DBA wakes up to an on-call alert, interprets a spike, provisions additional capacity, and reruns workloads — by which time the damage is done. Autonomous scaling flips this model entirely: the system observes its own resource consumption, predicts demand trajectories, and executes capacity changes without human approval. This chapter covers the architecture and implementation of autonomous vertical and horizontal scaling for SQL Server in cloud environments, including the signal pipelines that drive decisions, the execution layers that implement them, and the guardrails that prevent runaway automation from causing worse problems than it solves.

The Signal Pipeline: From Metric Collection to Scaling Decisions

Autonomous scaling begins not with the scaling action itself but with the quality of the signals feeding the decision engine. A system that scales on raw CPU percentage will thrash. A system that scales on composite signals — query latency percentiles, buffer hit ratios, replication lag, active connection depth, and queue depth simultaneously — makes durable decisions.

The canonical metric stack for autonomous scaling uses Prometheus for collection, a time-series anomaly model (ARIMA, Prophet, or a lightweight LSTM) for prediction, and a rule engine that translates predictions into scaling intents.

The baseline signals come from SQL Server Dynamic Management Views:

`sys.dm_exec_requests`, `sys.dm_os_wait_stats`, `sys.dm_os_ring_buffers`,
`sys.dm_os_performance_counters`, and `sys.dm_db_resource_stats` on Azure SQL.

```
-- Composite health signal query for the scaling decision engine
SELECT
  (SELECT COUNT(*) FROM sys.dm_exec_requests
   WHERE blocking_session_id <> 0) AS blocked_queries,
  (SELECT COUNT(*) FROM sys.dm_exec_requests
   WHERE status = 'running') AS active_queries,
  (SELECT
    CAST(100.0 * SUM(CASE WHEN physical_reads = 0 THEN 1 ELSE 0 END)
    / NULLIF(COUNT(*), 0) AS DECIMAL(5,2))
   FROM sys.dm_exec_requests
   WHERE status = 'running') AS logical_read_ratio,
  (SELECT TOP 1 DATEDIFF(SECOND, last_commit_time,
    secondary_last_commit_time)
   FROM sys.dm_hadr_database_replica_states
   WHERE is_local = 0) AS replica_lag_seconds,
  (SELECT MAX(DATEDIFF(SECOND, start_time, GETDATE()))
   FROM sys.dm_exec_requests
   WHERE status IN ('running','suspended')) AS max_query_duration_sec;
```

These two queries form the heartbeat of the scaling signal pipeline. A Python collector runs them on a 15-second cadence, pushes results to Prometheus via the `prometheus_client` library, and feeds a sliding window evaluator that determines whether observed values represent a transient spike or a sustained load pattern requiring a scaling response.

The distinction between a spike and a trend is the most important judgment the scaling engine must make. Spikes warrant connection pool adjustments or query prioritization changes. Sustained trends warrant compute scaling. The window evaluator applies a simple rule: if the composite score (a weighted average of the normalized metric deviations) exceeds 0.75 for three consecutive five-minute evaluation windows, a scaling intent is raised. A single window breach does not trigger scaling; it resets a cooldown timer and increases monitoring frequency.

Vertical Scaling Automation: Compute Resize Without Downtime

Vertical scaling — adding vCPUs or memory to an existing instance — is the first lever most cloud database architectures pull because it requires no application topology changes. The challenge is executing it with zero or near-zero downtime while maintaining ACID guarantees throughout the resize window.

On Azure SQL Database and Azure SQL Managed Instance, compute tier changes are API-driven and can be automated via Terraform or direct REST calls. On AWS RDS, instance class modifications support the `apply-immediately` flag for minor version changes but require a maintenance window for major compute class changes unless using RDS Multi-AZ, where the standby is upgraded first and a failover is executed transparently.

The autonomous scaling controller must understand these platform-specific behaviors. A platform-agnostic abstraction layer is dangerous here; the controller needs to know whether the target platform supports online resize before issuing the command.

```

# Autonomous vertical scaling controller
# Handles Azure SQL Database / Managed Instance and AWS RDS for SQL Server

import boto3
import requests
import json
import time
import logging
from dataclasses import dataclass
from enum import Enum

logger = logging.getLogger("autonomous_scaler")

class Platform(Enum):
    AWS_RDS = "aws_rds"
    AZURE_SQL_DB = "azure_sql_db"
    AZURE_SQL_MI = "azure_sql_mi"

@dataclass
class ScalingIntent:
    platform: Platform
    resource_id: str
    current_sku: str
    target_sku: str
    trigger_metric: str
    trigger_value: float
    threshold: float
    cooldown_minutes: int = 30

class VerticalScalingController:
    def __init__(self, azure_token: str, aws_region: str):
        self.azure_token = azure_token
        self.rds_client = boto3.client('rds', region_name=aws_region)
        self.last_scale_time = {}

    def _is_in_cooldown(self, resource_id: str, cooldown_minutes: int) -> bool:
        if resource_id not in self.last_scale_time:
            return False
        elapsed = (time.time() - self.last_scale_time[resource_id]) / 60
        return elapsed < cooldown_minutes

    def execute_scaling(self, intent: ScalingIntent) -> dict:
        if self._is_in_cooldown(intent.resource_id, intent.cooldown_minutes):
            logger.info(f"Resource {intent.resource_id} in cooldown. Skipping.")
            return {"status": "skipped", "reason": "cooldown"}

        logger.warning(
            f"Scaling {intent.resource_id}: {intent.current_sku} ->
{intent.target_sku} "
            f"[trigger: {intent.trigger_metric}={intent.trigger_value:.2f}, "
            f"threshold={intent.threshold}]"
        )

```

```

if intent.platform == Platform.AWS_RDS:
    return self._scale_rds(intent)
elif intent.platform in (Platform.AZURE_SQL_DB, Platform.AZURE_SQL_MI):
    return self._scale_azure_sql(intent)

def _scale_rds(self, intent: ScalingIntent) -> dict:
    try:
        response = self.rds_client.modify_db_instance(
            DBInstanceIdentifier=intent.resource_id,
            DBInstanceClass=intent.target_sku,
            ApplyImmediately=True # safe for Multi-AZ; failover handles HA
        )
        self.last_scale_time[intent.resource_id] = time.time()
        return {
            "status": "initiated",
            "db_instance_status": response['DBInstance']['DBInstanceStatus']
        }
    except Exception as e:
        logger.error(f"RDS scaling failed: {e}")
        return {"status": "failed", "error": str(e)}

def _scale_azure_sql(self, intent: ScalingIntent) -> dict:
    # Azure SQL Database or Managed Instance SKU update
    # resource_id: full ARM id of the database or managed instance
    parts = intent.resource_id.split("/")
    subscription_id = parts[2]
    resource_group = parts[4]
    if intent.platform == Platform.AZURE_SQL_MI:
        provider_path = f"/providers/Microsoft.Sql/managedInstances/{parts[-1]}"
    else:
        provider_path = (
            f"/providers/Microsoft.Sql/servers/{parts[8]}"
            f"/databases/{parts[-1]}"
        )

    url = (
        f"https://management.azure.com/subscriptions/{subscription_id}"
        f"/resourceGroups/{resource_group}"
        f"{provider_path}"
        f"?api-version=2023-05-01-preview"
    )
    payload = {"sku": {"name": intent.target_sku, "tier": "GeneralPurpose"}}
    headers = {
        "Authorization": f"Bearer {self.azure_token}",
        "Content-Type": "application/json"
    }
    response = requests.patch(url, headers=headers, data=json.dumps(payload))
    if response.status_code in (200, 202):
        self.last_scale_time[intent.resource_id] = time.time()
        return {"status": "initiated", "http_status": response.status_code}
    return {"status": "failed", "http_status": response.status_code,
            "body": response.text}

```


The cooldown mechanism is non-negotiable. Without it, a scaling controller can enter a feedback loop: scale up triggers a brief latency increase during the resize, which looks like continued stress, which triggers another scale-up, until the instance reaches the maximum available SKU. Thirty minutes is a sensible default; production systems may tune this based on the platform's typical resize completion time plus a safety buffer.

On AWS RDS for SQL Server or Azure SQL Managed Instance, vertical scaling follows the same controller pattern but operates on different SKU nomenclature. Azure SQL Elastic Pools support dynamic DTU or vCore allocation within a pool without instance restarts — making pool-level scaling the preferred lever when multiple databases share infrastructure.

Horizontal Scaling: Read Replica Automation and Sharding Orchestration

Vertical scaling has a ceiling. When the workload outgrows the largest available instance class, or when write latency targets require distributing read traffic, horizontal scaling becomes the operative architecture. Autonomous horizontal scaling for SQL Server takes two distinct forms: replica fleet management and, for write scalability, shard orchestration.

Autonomous Read Replica Management

The replica fleet should grow and shrink based on read queue depth and replica lag. When replica lag exceeds a defined threshold — say, 5 seconds — adding another replica reduces the per-replica replication fan-out, indirectly reducing lag on existing replicas while absorbing additional read traffic. When replica lag returns to acceptable levels and the replica connection utilization drops below 30% for two consecutive evaluation windows, the controller decommissions the youngest replica.

On Kubernetes, this is expressed through an operator pattern. The operator watches a `DatabaseScalingPolicy` custom resource and controls the replica count of the availability group secondary StatefulSet (via the SQL Server Kubernetes operator or a plain StatefulSet scale).

Horizontal scaling of the connection proxy layer accompanies every replica addition. Each new replica gets its own proxy sidecar, and the application-layer load balancer (HAProxy or Envoy) is updated to include the new replica endpoint with `read-only` routing tags.

```
-- Replica health for Availability Group autoscaler
SELECT
    ar.replica_server_name,
    drs.database_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.log_send_queue_size AS log_send_queue_kb,
    drs.redo_queue_size AS redo_queue_kb,
    drs.log_send_rate AS log_send_rate_kb_sec,
    drs.redo_rate AS redo_rate_kb_sec,
    CASE
        WHEN drs.log_send_rate > 0
        THEN CAST(drs.log_send_queue_size * 1.0
                  / drs.log_send_rate AS DECIMAL(10,2))
        ELSE NULL
    END AS estimated_lag_seconds
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
WHERE drs.is_local = 0
ORDER BY estimated_lag_seconds DESC NULLS LAST;
```

Write Scalability and Shard Orchestration

For write-bound workloads, SQL Server scales horizontally through application-layer sharding backed by a shard map manager database. The autonomous layer orchestrates shard rebalancing in response to per-shard row count skew and per-shard query latency divergence. When a shard's query p99 exceeds 2× the fleet median for two evaluation periods, the rebalancer splits or moves that shard to a less-loaded node — transparently to the application.

```

# Autonomous shard rebalancer for SQL Server sharded deployments
# Triggers a shard split/move when a shard's p99 latency diverges beyond threshold

import pyodbc
import logging

logger = logging.getLogger("shard_rebalancer")

SHARDMAP_DSN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=shardmap-primary;DATABASE=ShardMapManager;Trusted_Connection=yes;"
)
P99_DIVERGENCE_THRESHOLD = 2.0 # target shard p99 must not exceed 2x fleet median
MIN_SHARD_ROWS_REBALANCE = 500000 # avoid rebalancing tiny shards

def get_shard_latency_profile(conn):
    cur = conn.cursor()
    cur.execute("""
        SELECT
            s.shard_id,
            s.database_name,
            s.server_name,
            SUM(p.rows) AS row_count
        FROM dbo.shard_registry s
        JOIN sys.partitions p ON p.object_id = OBJECT_ID(s.fact_table)
        WHERE p.index_id IN (0, 1)
        GROUP BY s.shard_id, s.database_name, s.server_name
        ORDER BY row_count DESC
    """)
    return cur.fetchall()

def trigger_rebalance_if_needed(conn, shard_profile):
    counts = [row[3] for row in shard_profile]
    if not counts:
        return

    median_count = sorted(counts)[len(counts) // 2]
    oversized_shards = [
        row for row in shard_profile
        if row[3] > median_count * P99_DIVERGENCE_THRESHOLD
        and row[3] > MIN_SHARD_ROWS_REBALANCE
    ]

    if oversized_shards:
        logger.warning(
            f"Triggering shard rebalance: {len(oversized_shards)} oversized shards "
            f"detected (median={median_count:,} rows)"
        )
        cur = conn.cursor()
        for shard in oversized_shards:
            # Queue an elastic-database split/move job for this shard
            cur.execute(
                "EXEC dbo.queue_shard_split @shard_id = ?",

```

```

        shard[0],
    )
    conn.commit()
    logger.info("Shard rebalance jobs queued successfully.")
else:
    logger.debug("Shard distribution within acceptable bounds. No action
required.")

if __name__ == "__main__":
    conn = pyodbc.connect(SHARDMAP_DSN)
    profile = get_shard_latency_profile(conn)
    trigger_rebalance_if_needed(conn, profile)
    conn.close()

```

Availability Groups with readable secondaries are the standard read-scale mechanism; for write scale, application-layer sharding with elastic pools on Azure SQL is the target. The autonomous controller for SQL Server targets pool elasticity — dynamically moving databases between pool service tiers based on DTU consumption patterns observed over rolling 24-hour windows.

Kubernetes-Native Scaling: Operators, HPA, and KEDA Integration

When SQL Server runs in containers on Kubernetes, the scaling surface expands to include Kubernetes-native primitives: Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA), and KEDA (Kubernetes Event-Driven Autoscaling). The challenge is that databases are stateful workloads — naive HPA configurations that treat database pods like stateless microservices will corrupt data. Autonomous scaling on Kubernetes requires an operator-aware approach that understands the difference between scaling the connection proxy layer (always safe to scale horizontally) and scaling the database server layer (requires careful leader election and persistent volume management).

The recommended architecture separates three independently scalable layers:

1. **Connection Proxy** — scaled horizontally via standard HPA on CPU and connection queue depth metrics from Prometheus. Safe to scale aggressively.
2. **Read Replicas** — scaled via the SQL Server operator's secondary replica count field, gated by the redo-queue health checks described in the previous section.
3. **Primary Compute** — scaled vertically via VPA (for SQL Server pods) or via cloud API calls for managed services. Never scaled horizontally without sharding.

KEDA enables event-driven scaling triggers that go beyond the metrics available natively to Kubernetes. A KEDA `ScaledObject` targeting the connection proxy deployment can use a Prometheus query as its scaling trigger — for example, scaling proxy replicas when the active connection count as a fraction of the proxy's configured client connection limit exceeds 0.70.

```
# KEDA ScaledObject for the SQL Server connection proxy – scales proxy tier on
connection pressure
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: sqlproxy-connection-scaler
  namespace: database
spec:
  scaleTargetRef:
    name: sqlproxy-deployment
  pollingInterval: 15
  cooldownPeriod: 120
  minReplicaCount: 2
  maxReplicaCount: 12
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring.svc.cluster.local:9090
        metricName: sqlproxy_active_connections_ratio
        query: |
          sum(sqlproxy_pools_cl_active) /
          sum(sqlproxy_config_max_client_conn)
        threshold: "0.70"
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring.svc.cluster.local:9090
        metricName: sqlproxy_wait_time_p95
        query: |
          histogram_quantile(0.95,
            rate(sqlproxy_pool_wait_seconds_bucket[2m]))
        threshold: "0.5"
```

For the SQL Server Kubernetes operator, secondary scaling is expressed as a patch to the availability group secondary StatefulSet scale subresource. The autonomous controller generates and applies this patch programmatically:

```

# Kubernetes operator patch to scale SQL Server AG read replicas
# Scales the secondary StatefulSet; the operator joins new pods to the AG

from kubernetes import client, config
import json
import logging

logger = logging.getLogger("k8s_sql_scaler")

def scale_sqlserver_secondaries(
    namespace: str,
    secondary_sts_name: str,
    target_replica_count: int,
    current_replica_count: int
):
    config.load_incluster_config() # use in-cluster ServiceAccount
    apps_api = client.AppsV1Api()

    logger.info(
        f"Scaling {secondary_sts_name}: secondary replicas "
        f"{current_replica_count} -> {target_replica_count}"
    )

    try:
        apps_api.patch_namespaced_stateful_set_scale(
            name=secondary_sts_name,
            namespace=namespace,
            body={"spec": {"replicas": target_replica_count}},
        )
        logger.info(
            f"Replica scale operation accepted for {secondary_sts_name}")
    except client.ApiException as e:
        logger.error(
            f"Kubernetes API error during replica scaling: {e.status} {e.reason}")
        raise

```

The controller invokes `scale_sqlserver_secondaries` only after passing the replication health gate (redo queue under threshold, no pending AG join, no failed data movement on any replica). These preconditions are checked synchronously before the patch is applied; if any fail, the scale-up is deferred by one evaluation cycle with an alert raised so the operator can investigate if the gate remains blocked beyond the defined timeout.

Scale-Down Safety: Draining, Guardrails, and Reversibility

Scale-up is operationally simple compared to scale-down. Scaling down a database tier risks connection drops, in-flight transaction loss, and replication topology instability. Autonomous scale-down requires a disciplined drain-before-terminate sequence and a set of hard guardrails that prevent the controller from scaling below the minimum viable topology for the current workload.

The drain sequence for an AG readable secondary before decommissioning:

1. Remove the secondary from the AG read-only routing list. New connections stop routing to this replica.
2. Wait for the secondary's active session count (from `sys.dm_exec_sessions`) to reach zero or the drain timeout (120 seconds default) to expire.
3. If the drain timeout expires with sessions remaining, force-disconnect idle sessions with `KILL` only (see the script below). Running queries are allowed to complete.
4. Verify the redo queue on remaining secondaries has not increased beyond the acceptable threshold before proceeding.
5. Issue the Kubernetes delete or cloud API decommission.

```

-- Check active sessions on an AG secondary being drained
-- Run on the secondary instance
SELECT
    s.session_id,
    s.login_name,
    s.host_name,
    s.program_name,
    s.status,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000.0 AS wait_seconds,
    DATEDIFF(SECOND, s.last_request_start_time, GETDATE()) AS seconds_since_request,
    LEFT(t.text, 80) AS query_preview
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.is_user_process = 1
    AND s.session_id <> @@SPID
ORDER BY seconds_since_request DESC;

-- Force-disconnect idle sessions after drain timeout
-- Uses a cursor to terminate sessions one at a time with audit logging
DECLARE @session_id INT;
DECLARE @kill_cmd NVARCHAR(20);

DECLARE drain_cursor CURSOR FOR
    SELECT session_id
    FROM sys.dm_exec_sessions
    WHERE is_user_process = 1
        AND session_id <> @@SPID
        AND status = 'sleeping'
        AND program_name NOT LIKE '%replication%'
        AND program_name NOT LIKE '%hadr%';

OPEN drain_cursor;
FETCH NEXT FROM drain_cursor INTO @session_id;
WHILE @@FETCH_STATUS = 0
BEGIN
    SET @kill_cmd = N'KILL ' + CAST(@session_id AS NVARCHAR(10));
    EXEC sp_executesql @kill_cmd;
    FETCH NEXT FROM drain_cursor INTO @session_id;
END;
CLOSE drain_cursor;
DEALLOCATE drain_cursor;

```

Beyond the drain sequence, the scale-down controller enforces a set of hard guardrails that cannot be overridden by the evaluation engine — only by a human operator with explicit override authorization:

Minimum replica floor. The cluster must retain at least two replicas at all times if the primary is a single-AZ deployment, or at least one replica if the primary is Multi-AZ. Scaling below the floor is blocked unconditionally.

Replication lag gate. If aggregate replica lag across the surviving replicas exceeds the defined SLA threshold at the time the scale-down is proposed, the action is deferred. Removing a replica while surviving replicas are already lagging compounds the problem.

Active backup exclusion. If a native `BACKUP` or RDS snapshot is in progress against any instance in the cluster, scale-down is deferred until the backup completes. Terminating a replica mid-backup produces an incomplete backup that may be silently useless.

Recent failover exclusion. If a primary failover occurred within the preceding 60 minutes, the scaling controller enters a mandatory observation window and takes no scale-down action. Failovers transiently inflate replica lag metrics and can make the cluster appear under-utilized when it is in fact recovering.

These guardrails are implemented as precondition checks in the scaling decision function, evaluated before any decommissioning API call is issued:

```

# Scale-down guardrail evaluator

from dataclasses import dataclass
from typing import List, Optional
import time

@dataclass
class ClusterState:
    replica_count: int
    primary_is_multi_az: bool
    max_replica_lag_seconds: float
    lag_sla_seconds: float
    backup_in_progress: bool
    last_failover_epoch: Optional[float]
    failover_observation_window_minutes: int = 60

class ScaleDownGuardrails:

    def evaluate(self, state: ClusterState) -> tuple[bool, str]:
        """
        Returns (allowed: bool, reason: str).
        Scale-down proceeds only if allowed is True.
        """
        min_replicas = 1 if state.primary_is_multi_az else 2
        if state.replica_count <= min_replicas:
            return False, (
                f"Replica count ({state.replica_count}) at or below minimum "
                f"floor ({min_replicas}) for this topology."
            )

        if state.max_replica_lag_seconds > state.lag_sla_seconds:
            return False, (
                f"Replica lag ({state.max_replica_lag_seconds:.1f}s) exceeds "
                f"SLA ({state.lag_sla_seconds:.1f}s). Deferring scale-down."
            )

        if state.backup_in_progress:
            return False, "Backup in progress. Scale-down deferred until
completion."

        if state.last_failover_epoch is not None:
            elapsed_minutes = (time.time() - state.last_failover_epoch) / 60.0
            if elapsed_minutes < state.failover_observation_window_minutes:
                remaining = state.failover_observation_window_minutes -
elapsed_minutes
                return False, (
                    f"Recent failover detected ({elapsed_minutes:.1f} minutes ago).
"
                    f"Observation window active for {remaining:.1f} more minutes."
                )

        return True, "All guardrails passed. Scale-down approved."

```

The `evaluate` method returns a structured reason alongside the boolean decision so that every blocked scale-down is logged with full context. This audit trail becomes the primary diagnostic tool when the scaling system appears to be stuck — engineers can read the log and understand exactly which guardrail is holding the action.

Predictive Scaling: Demand Forecasting Ahead of Load Events

Reactive scaling — responding to observed degradation — introduces an unavoidable lag between demand onset and capacity availability. For predictable load patterns, predictive scaling closes this gap by provisioning capacity before demand materializes. The most common patterns amenable to prediction are: daily business-hour load cycles, weekly batch processing windows, end-of-month reporting runs, and marketing campaign spikes with known start times.

The predictive layer uses a lightweight Prophet model trained on the historical composite signal from the Prometheus time series. Prophet handles weekly and daily seasonality natively and tolerates missing data, making it practical for production environments where the signal pipeline occasionally has gaps.

```

# Predictive scaling using Facebook Prophet
# Trains on rolling 30-day history from Prometheus and generates 4-hour lookahead

import pandas as pd
from prophet import Prophet
from prometheus_api_client import PrometheusConnect
import logging
from datetime import datetime, timedelta

logger = logging.getLogger("predictive_scaler")

PROMETHEUS_URL = "http://prometheus.monitoring.svc.cluster.local:9090"
LOOKAHEAD_HOURS = 4
SCALE_UP_THRESHOLD = 0.75 # predicted composite score that triggers preemptive
scale
TRAINING_DAYS = 30

def fetch_composite_signal(prom: PrometheusConnect) -> pd.DataFrame:
    end = datetime.utcnow()
    start = end - timedelta(days=TRAINING_DAYS)

    result = prom.get_metric_range_data(
        metric_name='db_composite_load_score',
        start_time=start,
        end_time=end,
        chunk_size=timedelta(hours=6)
    )

    if not result:
        raise ValueError("No data returned from Prometheus for composite load
score.")

    records = []
    for series in result:
        for ts, val in series['values']:
            records.append({'ds': pd.to_datetime(ts, unit='s', utc=True), 'y':
float(val)})

    df = pd.DataFrame(records).sort_values('ds').drop_duplicates('ds')
    df['ds'] = df['ds'].dt.tz_localize(None) # Prophet requires tz-naive timestamps
    return df

def generate_scaling_forecast(df: pd.DataFrame) -> pd.DataFrame:
    model = Prophet(
        changepoint_prior_scale=0.05, # conservative – avoid overfitting spikes
        seasonality_mode='multiplicative',
        daily_seasonality=True,
        weekly_seasonality=True,
        yearly_seasonality=False # 30 days insufficient for annual patterns
    )
    model.fit(df)

    future = model.make_future_dataframe(

```

```

        periods=LOOKAHEAD_HOURS * 4, # 15-minute granularity
        freq='15min',
        include_history=False
    )
    forecast = model.predict(future)
    return forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]

def evaluate_predictive_intent(forecast: pd.DataFrame) -> dict:
    upcoming_peak = forecast['yhat'].max()
    peak_time = forecast.loc[forecast['yhat'].idxmax(), 'ds']
    minutes_until_peak = (peak_time - datetime.utcnow()).total_seconds() / 60.0

    if upcoming_peak >= SCALE_UP_THRESHOLD and minutes_until_peak > 0:
        logger.warning(
            f"Predictive scale-up intent: peak score {upcoming_peak:.3f} "
            f"predicted at {peak_time} ({minutes_until_peak:.0f} minutes from now)"
        )
        return {
            "action": "preemptive_scale_up",
            "predicted_peak": upcoming_peak,
            "peak_time": str(peak_time),
            "minutes_until_peak": round(minutes_until_peak, 1)
        }

    return {"action": "none", "predicted_peak": upcoming_peak}

if __name__ == "__main__":
    prom = PrometheusConnect(url=PROMETHEUS_URL, disable_ssl=True)
    df = fetch_composite_signal(prom)
    forecast = generate_scaling_forecast(df)
    intent = evaluate_predictive_intent(forecast)
    logger.info(f"Predictive scaling evaluation complete: {intent}")

```

The predictive controller runs on a 15-minute cron schedule, separate from the reactive controller. Its outputs are intents — structured JSON records written to a shared intent queue — not direct API calls. The execution controller consumes intents from both reactive and predictive paths, applies deduplication (so a predictive scale-up and a reactive scale-up targeting the same resource within the same cooldown window result in a single operation), and issues the actual cloud API call.

This separation of intent generation from execution is architecturally important. It allows the predictive and reactive paths to be tuned, disabled, and audited independently. A runaway predictive model (one that has drifted from reality after a business pattern change) can be disabled without affecting reactive scaling behavior.

Cost Guardrails: Preventing Runaway Scaling Expenditure

Autonomous scaling that operates without cost visibility is a liability. A misconfigured threshold or a model that drifts after an unusual traffic event can scale an instance to the maximum available SKU and hold it there through the cooldown period, generating significant unplanned expenditure. Cost guardrails are the final layer of the autonomous scaling architecture.

The cost guardrail system maintains a per-resource monthly budget cap expressed in cloud provider currency units, a rolling expenditure estimate updated from cloud billing APIs, and a maximum SKU ceiling that the scaler is permitted to target regardless of the scaling intent's recommendation.

```

# Cost guardrail enforcer for the autonomous scaling controller

from dataclasses import dataclass
from typing import Dict
import logging

logger = logging.getLogger("cost_guardrail")

# SKU cost map – populated from cloud provider price list API at startup
# Values are approximate hourly costs in USD; replace with live API data
SKU_HOURLY_COST_USD: Dict[str, float] = {
    # AWS RDS for SQL Server (example tier names)
    "db.t3.medium": 0.120,
    "db.r6i.large": 0.430,
    "db.r6i.xlarge": 0.860,
    "db.r6i.2xlarge": 1.720,
    "db.r6i.4xlarge": 3.440,
    # Azure SQL Database (example SKU names)
    "GP_Gen5_2": 0.504,
    "GP_Gen5_4": 1.008,
    "GP_Gen5_8": 2.016,
    "GP_Gen5_16": 4.032,
}

@dataclass
class ResourceBudget:
    resource_id: str
    monthly_budget_usd: float
    current_month_spend_usd: float
    max_sku_ceiling: str # controller may never exceed this SKU

class CostGuardrailEnforcer:

    def __init__(self, budgets: Dict[str, ResourceBudget]):
        self.budgets = budgets

    def _hours_remaining_in_month(self) -> float:
        from datetime import datetime, timezone
        import calendar
        now = datetime.now(timezone.utc)
        days_in_month = calendar.monthrange(now.year, now.month)[1]
        end_of_month = datetime(now.year, now.month, days_in_month, 23, 59, 59,
                                tzinfo=timezone.utc)
        return (end_of_month - now).total_seconds() / 3600.0

    def approve_scaling_intent(
        self,
        resource_id: str,
        current_sku: str,
        target_sku: str
    ) -> tuple[bool, str]:

        budget = self.budgets.get(resource_id)

```

```

    if budget is None:
        # No budget defined – allow scaling but log warning
        logger.warning(f"No budget configured for {resource_id}. Allowing scaling.")
        return True, "No budget constraint configured."

    # SKU ceiling check
    current_cost = SKU_HOURLY_COST_USD.get(current_sku, 0.0)
    target_cost = SKU_HOURLY_COST_USD.get(target_sku, 0.0)
    ceiling_cost = SKU_HOURLY_COST_USD.get(budget.max_sku_ceiling, float('inf'))

    if target_cost > ceiling_cost:
        return False, (
            f"Target SKU {target_sku} (${target_cost:.3f}/hr) exceeds "
            f"configured ceiling {budget.max_sku_ceiling} "
            f"(${ceiling_cost:.3f}/hr) for {resource_id}."
        )

    # Projected spend check
    hours_remaining = self._hours_remaining_in_month()
    incremental_hourly = target_cost - current_cost
    projected_additional_spend = incremental_hourly * hours_remaining
    projected_total = budget.current_month_spend_usd +
    projected_additional_spend

    if projected_total > budget.monthly_budget_usd:
        return False, (
            f"Projected month-end spend ${projected_total:.2f} would exceed "
            f"budget ${budget.monthly_budget_usd:.2f} for {resource_id}. "
            f"Scaling from {current_sku} to {target_sku} blocked."
        )

    return True, (
        f"Cost guardrail approved. Projected month-end spend: "
        f"${projected_total:.2f} of ${budget.monthly_budget_usd:.2f} budget."
    )

```

When a scaling intent is blocked by the cost guardrail, the controller raises a `COST_GUARDRAIL_BLOCKED` alert to the on-call channel. This alert carries the full context: resource ID, blocked target SKU, projected overspend amount, and remaining budget. The human operator can choose to approve a one-time budget override, raise the monthly cap, or accept the degraded performance. The system does not silently absorb blocked scale-up events — each one is surfaced as an operational signal requiring a human judgment call on the cost-performance trade-off.

End-to-End Autonomous Scaling Workflow

Bringing together all the components described in this chapter, the end-to-end autonomous scaling workflow operates as a continuous loop with clearly separated phases:

Phase 1 — Observe (15-second cadence): The metric collector queries

`sys.dm_exec_requests`, `sys.dm_hadr_database_replica_states`, and related DMVs and pushes results to Prometheus. The composite load score is computed and stored as a time series.

Phase 2 — Evaluate (5-minute windows): The reactive evaluator computes the sliding-window composite score. If the score exceeds the scale-up threshold for three consecutive windows, a reactive scale-up intent is generated. If the score is below the scale-down threshold for six consecutive windows and all guardrails pass, a scale-down intent is generated.

Phase 3 — Predict (15-minute cadence): The Prophet-based predictive evaluator loads 30 days of composite score history, generates a 4-hour forecast, and produces a predictive scale-up intent if the forecast predicts a peak above threshold within the lookahead window.

Phase 4 — Gate (per intent): Each intent — reactive or predictive — passes through the guardrail stack: cooldown check, topology floor check, replication lag gate, backup exclusion, recent failover exclusion, and cost guardrail. Any failed gate defers the intent and generates an audit log entry.

Phase 5 — Execute (per approved intent): The execution controller calls the appropriate cloud API (RDS `modify_db_instance`, Azure REST PATCH, Kubernetes CRD patch) and records the scale event with full metadata to the audit log.

Phase 6 — Verify (5-minute post-scale window): After execution, the controller enters a heightened monitoring mode, sampling at 30-second intervals. If the composite score does not improve within 10 minutes of a scale-up completing, a `SCALING_INEFFECTIVE` alert is raised, prompting investigation of non-resource bottlenecks (lock contention, schema design, query plan regression) that scaling alone cannot resolve.

```

graph TD
    A[Phase 1: Observe  
15s cadence] --> B[Phase 2: Evaluate  
5-min sliding windows]
    B --> C[Phase 3: Predict  
Prophet, 15-min cadence]
    C --> D[Shared Intent Queue]
    D --> E[Phase 4: Gate  
cooldown, floor, lag,  
backup, failover, cost]
    E -->|approved| F[Phase 5: Execute  
cloud API / CRD patch]
    F -->|blocked| G[Deferred + Audit Log]
    G --> H[Phase 6: Verify  
30s sampling, 10-min window]
    H -->|no improvement| I[SCALING_INEFFECTIVE alert]
    I --> A
    H -->|improved| A

```

The six-phase autonomous scaling loop: reactive and predictive evaluators both feed a shared intent queue, every intent passes through the guardrail gate before execution, and post-scale verification closes the loop.

This six-phase loop runs continuously in production. The phases are independent processes connected through the shared intent queue and Prometheus metric store, so any individual phase can be paused, reconfigured, or replaced without affecting the others.

Key Takeaways

- → Autonomous scaling requires composite signal pipelines — weighted combinations of query latency percentiles, buffer hit ratios, replication lag, and connection depth — rather than single-metric triggers like raw CPU percentage, which produce thrashing and false positive scale events.
- → Vertical and horizontal scaling require platform-specific execution paths: AWS RDS Multi-AZ, Azure Flexible Server PATCH APIs, and Kubernetes CRD patches each carry distinct availability semantics that a platform-agnostic abstraction layer will obscure to the point of causing outages.
- → Scale-down is categorically more dangerous than scale-up; a disciplined drain-before-terminate sequence combined with hard guardrails — replica floor, replication lag gate, active backup exclusion, and post-failover observation window — must be enforced unconditionally and cannot be overridden by the automated evaluation engine without a human approval path.
- → Separating intent generation from execution — with reactive and predictive controllers both writing to a shared intent queue consumed by a single execution controller — allows each path to be tuned, disabled, and audited independently, preventing a drifted forecasting model from affecting reactive scaling behavior and vice versa.

- → Cost guardrails are a structural requirement, not an optional add-on: every blocked scaling intent must surface as an explicit operational alert with projected overspend context, forcing a human judgment call on the cost-performance trade-off rather than allowing the system to either silently overspend or silently degrade under load.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 17: Zero-Touch High Availability

High availability in autonomous systems is not a feature you configure once and monitor manually — it is a continuous, self-executing contract between your infrastructure and your uptime SLA. The moment a human must intervene to restore service, you have failed the ADE standard. This chapter covers the architecture and implementation of HA systems that detect failure, make promotion decisions, reroute traffic, reintegrate replicas, and validate their own health — all without a pager going off at 2 a.m. The patterns here apply across SQL Server Always On Availability Groups on WSFC, AGs orchestrated by Kubernetes operators, and hybrid topologies that span on-premises and cloud. The goal is not just fast failover — it is *correct*, *reversible*, and *observable* failover that a system can execute and explain without human involvement.

17.1 The Anatomy of a Zero-Touch Failover Event

Before automating failover, you must model every failure mode precisely. Autonomous systems do not fail over on a hunch — they fail over when a constellation of signals crosses a decision threshold. The difference between autonomous HA and naive auto-failover is the difference between a surgeon and a reflex: one evaluates context, the other reacts to a single stimulus.

The canonical failure taxonomy for HA systems has three tiers:

Tier 1 — Node failure. The primary database process is unreachable. This is unambiguous at the OS level: no TCP response on the database port, no heartbeat from the HA agent, process not running.

Tier 2 — Network partition. The primary is alive but cannot be reached by replicas or the load balancer. This is the most dangerous case for autonomous systems — a false positive here causes a split-brain condition where two nodes believe they are primary.

Tier 3 — Performance degradation masquerading as failure. The primary is responding, but query latency is so high that applications are timing out. Connections are queuing. Replication lag is climbing. This is not a hard failure — it is a soft failure that an autonomous system must distinguish from a real outage.

A production autonomous HA system evaluates signals across all three tiers before acting. The decision matrix looks like this:

Signal	Source	Threshold	Weight
TCP connectivity	AG listener / load balancer health check	3 consecutive failures	High
Replication lag	sys.dm_hadr_database_replica_states redo queue	> 30 seconds	Medium
Replica quorum	WSFC quorum votes	Majority unreachable	High
Transaction throughput	Prometheus sql_exporter batch requests/sec	Drop > 80% in 60s	Medium
OS-level process	WSFC resource monitor / Kubernetes liveness probe	Process absent	High

An autonomous system acts only when the aggregate signal exceeds a confidence threshold — typically, a combination of at least two High-weight signals or one High plus two Medium signals. This prevents failover storms triggered by transient network blips.

Querying AG synchronization state for decision input:

```
-- Run on any replica; exposes per-replica health for the orchestration layer
SELECT
    ag.name                                AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    ars.synchronization_health_desc,
    ars.connected_state_desc,
    ars.operational_state_desc,
    drs.synchronization_state_desc,
    drs.log_send_queue_size                AS send_queue_kb,
    drs.redo_queue_size                    AS redo_queue_kb,
    drs.last_commit_time
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id
ORDER BY ag.name, ar.replica_server_name;
```

These queries are not run interactively — they are ingested every 10 seconds by the HA orchestration layer, stored as time-series metrics, and evaluated against the decision matrix. The HA agent is a consumer of this data, not a human.

17.2 WSFC Quorum and the Lease Contract: Autonomous Promotion Architecture

Windows Server Failover Cluster (WSFC) quorum is the autonomous HA framework for SQL Server at scale. Its architecture is worth studying not just as a tool but as a design pattern: it externalizes consensus to a vote-based quorum model, making the cluster's leadership state a shared, durable, observable fact rather than an assumption held by individual nodes.

The WSFC architecture for zero-touch HA rests on three contracts:

The Quorum Vote. Every node (and optional witness) holds votes. A cluster can only bring resources online — including the availability group — while it holds a majority of votes. If a partition splits the cluster, the side without majority stops rather than risking split-brain. No external signal is needed — the absence of quorum is itself the trigger that halts the minority side.

The Candidate Election. When the primary replica fails, automatic failover requires both `SYNCHRONOUS_COMMIT` availability mode and `AUTOMATIC` failover mode on the target secondary. The AG promotes the synchronous secondary with the most hardened log sequence, minimizing data loss. With `REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT` set, a transaction cannot commit until the required number of secondaries has hardened it — giving a zero-data-loss guarantee under automatic failover.

The Health Lease. SQL Server reports instance health to the cluster through `sp_server_diagnostics`, which runs on a repeating interval (by default one third of the lease timeout). If diagnostics report unhealthy — or stop reporting before the lease timeout (default 20 seconds) expires — the cluster initiates failover. This prevents a scenario where a frozen SQL Server holds cluster resources while being unable to serve writes — a subtle split-brain vector.

The AG configuration for autonomous failover behavior is dense with tunable parameters that directly affect the decision logic:

```
-- AG-level health and timeout tuning for autonomous failover
ALTER AVAILABILITY GROUP [ProductionAG]
    SET (HEALTH_CHECK_TIMEOUT = 30000);    -- ms before cluster declares replica
unhealthy

-- Per-replica session timeout controls how long the primary waits
-- for a secondary's ping before marking it disconnected
ALTER AVAILABILITY GROUP [ProductionAG]
    MODIFY REPLICA ON N'sqlag-replica1'
    WITH (SESSION_TIMEOUT = 10);          -- seconds

-- Lease timeout lives in the cluster resource, not in T-SQL:
-- Get-ClusterResource "ProductionAG" | Set-ClusterParameter LeaseTimeout 20000
```

The `HEALTH_CHECK_TIMEOUT` parameter is critical for autonomous operation. It bounds how long the cluster waits for `sp_server_diagnostics` to return before declaring the replica unhealthy and triggering failover. Too short, and transient diagnostic stalls cause failover storms; too long, and real outages go unremediated.

When a synchronous secondary falls behind or disconnects, the primary's `REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT` setting determines whether writes block or continue. An autonomous HA controller watches the count of synchronized secondaries and can dynamically relax this setting during a degraded window — trading a bounded RPO increase for continued write availability — then restore it when the secondary recovers.

Python — AG DMV integration for autonomous monitoring:

```

import pyodbc
import time
import logging
from dataclasses import dataclass
from typing import Optional

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("ag_monitor")

@dataclass
class AGClusterState:
    primary: str
    members: list
    failover_in_progress: bool
    sync_secondaries: list

AG_ENDPOINTS = [
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlag-  
primary;Trusted_Connection=yes;",
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlag-  
replica1;Trusted_Connection=yes;",
    "DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlag-  
replica2;Trusted_Connection=yes;",
]

CLUSTER_QUERY = """
SELECT
    ar.replica_server_name,
    ars.role_desc,
    ar.availability_mode_desc,
    ar.failover_mode_desc,
    ars.synchronization_health_desc,
    drs.redo_queue_size
FROM sys.availability_replicas ar
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
LEFT JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id AND drs.is_local = 1
"""

def fetch_cluster_state() -> Optional[AGClusterState]:
    """
    Query each replica until one responds.
    In a real split-brain scenario, replicas may disagree on role –
    the quorum-holding side is authoritative, not any single node.
    """
    for dsn in AG_ENDPOINTS:
        try:
            conn = pyodbc.connect(dsn, timeout=5)
            cur = conn.cursor()
            cur.execute(CLUSTER_QUERY)
            members = [
                {

```



```

        "name": row[0], "role": row[1], "avail_mode": row[2],
        "failover_mode": row[3], "health": row[4],
        "redo_queue_kb": row[5] or 0,
    }
    for row in cur.fetchall()
]
conn.close()
primary = next(
    (m["name"] for m in members if m["role"] == "PRIMARY"), None
)
sync = [
    m["name"] for m in members
    if m["role"] == "SECONDARY"
    and m["avail_mode"] == "SYNCHRONOUS_COMMIT"
]
return AGClusterState(
    primary=primary,
    members=members,
    failover_in_progress=primary is None,
    sync_secondaries=sync,
)
except pyodbc.Error as e:
    logger.warning(f"Could not reach endpoint: {e}")
return None

def evaluate_and_alert(state: AGClusterState):
    lagging = [
        m for m in state.members
        if m["redo_queue_kb"] > 1024 and m["role"] != "PRIMARY"
    ]
    if lagging:
        logger.warning(
            f"Secondaries exceeding redo-queue threshold: "
            f"{[m['name'] for m in lagging]}"
        )
    if not state.sync_secondaries:
        logger.error(
            "No synchronous secondary present – cluster is operating in async mode.
            "
            "RPO guarantee is degraded."
        )

if __name__ == "__main__":
    while True:
        state = fetch_cluster_state()
        if state:
            evaluate_and_alert(state)
            time.sleep(15)

```

This script is not a replacement for WSFC's own decision-making — it is a secondary observer that feeds into an alerting and audit trail, allowing the autonomous system to record *why* it made a failover decision in language that a post-incident review can parse.

17.3 SQL Server Always On and the Kubernetes Operator Model

SQL Server's Always On Availability Groups provide quorum-based leadership-election semantics through the cluster service. In Windows Server Failover Cluster (WSFC) deployments, quorum is managed by a cluster quorum resource — a vote-based lease that makes leadership a shared, observable fact. In containerized deployments (SQL Server on Kubernetes), the SQL Server Operator from the `mssql-operator` or community `sql-server-k8s-operator` replaces WSFC entirely.

The Kubernetes operator model is significant because it brings SQL Server HA into the same orchestration plane as the rest of the application stack. An operator is a custom Kubernetes controller that watches a Custom Resource Definition (CRD) representing the AG's desired state and continuously reconciles actual state to match it. When a primary pod dies, Kubernetes detects the pod failure, the operator observes the AG state change via the DMVs, elects a new primary from the healthy replicas, and updates the Service object that application connections point to — all within the reconciliation loop.

The AG CRD for an autonomous deployment looks conceptually like this in YAML:

```

apiVersion: sql.microsoft.com/v1
kind: SqlServer
metadata:
  name: sqlag-primary
spec:
  saPassword: "$(SA_PASSWORD)"
  edition: Developer
---
apiVersion: sql.microsoft.com/v1
kind: AvailabilityGroup
metadata:
  name: production-ag
spec:
  name: ProductionAG
  databases:
    - orders
    - inventory
  replicas:
    - server: sqlag-primary
      availabilityMode: SYNCHRONOUS_COMMIT
      failoverMode: AUTOMATIC
    - server: sqlag-replica1
      availabilityMode: SYNCHRONOUS_COMMIT
      failoverMode: AUTOMATIC
    - server: sqlag-replica2
      availabilityMode: ASYNCHRONOUS_COMMIT
      failoverMode: MANUAL
  listenerPort: 1433

```

The `AUTOMATIC` failover mode on synchronous replicas is what enables zero-touch behavior. When the operator detects that the primary replica is in a `NOT_SYNCHRONIZING` or `DISCONNECTED` state for longer than the configured health check threshold, it executes the equivalent of `ALTER AVAILABILITY GROUP ... FORCE_FAILOVER_ALLOW_DATA_LOSS` or a clean automatic failover depending on whether data loss is acceptable.

For the operator to make this decision autonomously, it must have clear policy guidance baked into the CRD:

SQL Server — verifying AG readiness for automatic failover:

```
-- Validates that both synchronous replicas are healthy and within failover policy
SELECT
    ag.name                                AS availability_group,
    ar.replica_server_name,
    ar.availability_mode_desc,
    ar.failover_mode_desc,
    ars.synchronization_health_desc,
    ars.is_local,
    ars.role_desc,
    CASE
        WHEN ar.failover_mode_desc = 'AUTOMATIC'
            AND ars.synchronization_health_desc = 'HEALTHY'
        THEN 'ELIGIBLE_FOR_AUTO_FAILOVER'
        ELSE 'NOT_ELIGIBLE'
    END                                    AS failover_eligibility
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
WHERE ag.name = 'ProductionAG';
```

A critical architectural consideration in zero-touch SQL Server HA is the handling of the AG listener after failover. The listener is a virtual network name (VNN) that clients connect to. In WSFC, the cluster updates DNS automatically. In Kubernetes, the operator updates the ClusterIP Service. In both cases, the connection rerouting is transparent to application code — but only if applications are using the AG listener or Service name, never the direct node IP. Autonomous HA systems enforce this at the infrastructure layer: direct-node connection strings are blocked by network policy, forcing all traffic through the abstraction layer that the operator controls.

17.4 Traffic Rerouting Without Human Intervention: AG Listeners and Read-Only Routing

Promoting a replica to primary is half the problem. The other half is getting client connections to the new primary before application-level timeouts cascade into a service outage. In autonomous systems, connection rerouting must happen in under 5 seconds in most SLA frameworks — which rules out DNS TTL-based approaches for latency-sensitive workloads.

The standard architecture for zero-touch traffic rerouting in SQL Server uses a layered approach:

Application → AG Listener (virtual network name) → Primary replica / Read-only routing list

The availability group listener is a virtual network name (VNN) with its own IP that always points at the current primary. On failover, the cluster moves the listener to the new primary and updates DNS. Applications connect to the listener name — never to a node IP — so rerouting is transparent. Read-only routing extends this: connections specifying `ApplicationIntent=ReadOnly` are redirected by the primary to a readable secondary from the routing list, offloading reporting traffic automatically:

```
-- Read-only routing list: reporting connections land on secondaries automatically
ALTER AVAILABILITY GROUP [ProductionAG]
    MODIFY REPLICHA ON N'sqlag-primary'
    WITH (SECONDARY_ROLE (READ_ONLY_ROUTING_LIST = (N'sqlag-replica1', N'sqlag-replica2')));

ALTER AVAILABILITY GROUP [ProductionAG]
    MODIFY REPLICHA ON N'sqlag-replica1'
    WITH (SECONDARY_ROLE (READ_ONLY_ROUTING_LIST = (N'sqlag-replica2', N'sqlag-primary')));
```

In Kubernetes, the operator updates the ClusterIP Service that application connections point to, achieving the same effect as the WSFC listener move. In both cases, autonomous HA systems enforce listener-based connectivity at the infrastructure layer: direct-node connection strings are blocked by network policy, forcing all traffic through the abstraction layer that the operator controls.

Where an extra TCP proxy tier is required in front of the listener, HAProxy performs active port health checks against each replica and routes write traffic only to the node currently holding the primary role:

```
# haproxy.cfg – relevant sections for autonomous primary routing
frontend sql_write
    bind *:1433
    mode tcp
    default_backend sql_primary

backend sql_primary
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2 on-marked-down shutdown-sessions
    server sqlag-primary 10.0.1.1:1433 check
    server sqlag-replica1 10.0.1.2:1433 check backup
    server sqlag-replica2 10.0.1.3:1433 check backup
```

The key directive here is `on-marked-down shutdown-sessions`. When HAProxy's health check marks the current primary as down, it immediately tears down all existing connections to that node. This forces the application's connection pool to re-establish connections, which then hit the backend pool and land on the new primary. The total rerouting time is bounded by: HAProxy check interval (2s) × fall threshold (3) = 6 seconds worst case.

For workloads that cannot tolerate even 6 seconds of connection disruption, a service mesh approach using Envoy or Istio reduces this to sub-second. An Envoy TCP health check against port 1433 runs at 500ms intervals, and active request draining ensures in-flight transactions are completed before the connection is terminated. The trade-off is operational complexity — service mesh HA configuration is significantly more involved than HAProxy.

The application's connection string is the other half of failover resilience. `SqlClient's ConnectRetryCount` and `ConnectRetryInterval` parameters absorb the reconnection burst that follows a failover, so the application retries internally instead of surfacing errors:

```
-- Connection string parameters relevant to failover resilience
Server=ag-listener;Database=orders;Trusted_Connection=True;
ConnectRetryCount=3;           -- retry the open attempt 3 times
ConnectRetryInterval=10;       -- wait 10s between retries
ApplicationIntent=ReadWrite;   -- route to the primary replica
MultiSubnetFailover=True      -- parallel connection attempts across listener IPs
```

The `ConnectRetryCount = 3` setting is particularly important: it means that during the 6-second HAProxy switchover window, client connection attempts are retried by the driver rather than failing with connection errors. From the application's perspective, queries slow down for a few seconds rather than returning errors. This is the correct behavior for a zero-touch system — transient slowness is acceptable; connection errors are not.

Verifying connection routing post-failover:

```
-- Run against the new primary after promotion
-- Confirms that application traffic has successfully rerouted
SELECT
    c.client_net_address,
    s.program_name,
    s.status,
    COUNT(*) AS connection_count,
    MAX(DATEDIFF(SECOND, s.last_request_start_time, GETUTCDATE()))
        AS max_seconds_since_request
FROM sys.dm_exec_connections c
JOIN sys.dm_exec_sessions s
    ON c.session_id = s.session_id
WHERE s.is_user_process = 1
    AND s.status <> 'sleeping'
GROUP BY c.client_net_address, s.program_name, s.status
ORDER BY connection_count DESC;
```

This query is executed by the post-failover validation routine — not a human — to confirm that application connections have landed on the new primary and are in `running` rather than `suspended` state, which would indicate stuck requests from the failed primary.

17.5 Self-Healing Replica Reintegration After Failover

Failover is a one-way event unless the system also handles replica reintegration autonomously. After a primary fails and is promoted away, the former primary — once recovered — must rejoin the cluster as a secondary. The AG automatically re-synchronizes a rejoined replica through log shipping catch-up: the former primary rolls back any uncommitted work from its divergent tail and applies the new primary's log stream until it is synchronized.

The AG rejoin flow works as follows: when the former primary comes back online, it contacts the new primary and negotiates the common log sequence. If the divergence is within the available log on the primary, the secondary rolls back and catches up automatically. If the divergence exceeds the available log — because the primary has already truncated past the divergence point — the replica must be reseeded: the autonomous system removes it from the AG, restores a fresh backup with `NORECOVERY`, and rejoins it.

Monitoring the reintegration process autonomously requires tracking data movement state and redo queue from the moment the former primary reconnects:

The autonomous reintegration policy is: if `estimated_seconds_to_sync` keeps growing over three consecutive evaluations during reintegration, suspend and resume data movement on the replica once and retry. If three resume attempts fail, quarantine the node — remove it from the read-only routing list, suspend its AG data movement, and generate a human-

escalation alert. This is the one scenario where zero-touch fails gracefully into a managed-intervention state: the system has exhausted its automated recovery options and is explicitly communicating that human attention is required.

Replica reintegration after an AG failover is observable through the DMV layer:

Monitoring replica reintegration and data movement:

```
-- Tracks the progress of a replica rejoining the AG after primary failure
SELECT
    db_name(drs.database_id)                AS database_name,
    ar.replica_server_name,
    drs.synchronization_state_desc,
    drs.is_suspended,
    drs.suspend_reason_desc,
    drs.log_send_queue_size                 AS pending_send_kb,
    drs.redo_queue_size                     AS pending_redo_kb,
    drs.log_send_rate                       AS send_rate_kb_per_sec,
    drs.redo_rate                           AS redo_rate_kb_per_sec,
    CASE
        WHEN drs.redo_rate > 0
        THEN (drs.redo_queue_size / drs.redo_rate)
        ELSE NULL
    END                                    AS estimated_seconds_to_sync
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
WHERE drs.is_local = 0
ORDER BY drs.log_send_queue_size DESC;
```

The `estimated_seconds_to_sync` calculated column is what the autonomous system uses to decide whether to mark the rejoining replica as `pending` (expected to finish within SLA) or `degraded` (reintegration is taking longer than expected, possibly indicating storage or network issues on the rejoining node). This metric feeds into the cluster health dashboard and the replica promotion eligibility gate.

17.6 Autonomous Runbook Execution and Post-Failover Validation

The final layer of zero-touch HA is not the failover itself — it is the automated validation that follows. A failover that succeeds technically but leaves the system in a degraded or undiscovered state is a failure in the autonomous contract. Post-failover validation is the system's self-assessment: it must confirm that the new primary is healthy, that replication has resumed, that connections are routed correctly, that any dependent services have reconnected, and that the cluster is in a state where a second failover — if required — could succeed.

Autonomous runbook execution treats the post-failover sequence as a directed acyclic graph of checks, each with a pass/fail condition, a timeout, and an escalation policy:

- [1] New primary responds on port 1433 and reports PRIMARY role in DMVs
 - PASS: continue
 - FAIL after 10s: re-evaluate candidate replicas, attempt second promotion
- [2] sys.dm_hadr_database_replica_states shows at least one synchronizing secondary
 - PASS: continue
 - FAIL after 60s: alert, degrade RPO guarantee flag in cluster metadata
- [3] Application connection count matches pre-failover baseline ($\pm 20\%$)
 - PASS: continue
 - FAIL after 30s: check AG listener DNS, force connection pool recycle
- [4] Write throughput returns to baseline ($\pm 30\%$)
 - PASS: mark failover complete
 - FAIL after 120s: escalate to on-call – performance anomaly on new primary
- [5] Former primary rejoins as secondary
 - PASS within 300s: full cluster health restored
 - FAIL after 300s: quarantine former primary, alert for manual inspection

This runbook is implemented as an orchestration script that reads cluster state at each step rather than assuming the previous step's side effects persist:

```

import pyodbc
import time
import logging
from enum import Enum
from typing import Callable, Optional

logger = logging.getLogger("failover_validator")

class CheckResult(Enum):
    PASS = "PASS"
    FAIL = "FAIL"
    TIMEOUT = "TIMEOUT"

def run_check_with_timeout(
    check_fn: Callable[[], bool],
    timeout_seconds: int,
    poll_interval: int = 5,
    name: str = "check",
) -> CheckResult:
    deadline = time.monotonic() + timeout_seconds
    while time.monotonic() < deadline:
        try:
            if check_fn():
                logger.info(f"[{name}] PASS")
                return CheckResult.PASS
        except Exception as e:
            logger.warning(f"[{name}] exception during check: {e}")
            time.sleep(poll_interval)
    logger.error(f"[{name}] TIMEOUT after {timeout_seconds}s")
    return CheckResult.TIMEOUT

def check_primary_reachable(dsn: str) -> bool:
    conn = pyodbc.connect(dsn, timeout=5)
    cur = conn.cursor()
    cur.execute("""
        SELECT ars.role_desc
        FROM sys.dm_hadr_availability_replica_states ars
        WHERE ars.is_local = 1
    """)
    role = cur.fetchone()[0]
    conn.close()
    return role == "PRIMARY" # True only if this node believes it is primary

def check_replication_active(dsn: str, min_replicas: int = 1) -> bool:
    conn = pyodbc.connect(dsn, timeout=5)
    cur = conn.cursor()
    cur.execute("""
        SELECT COUNT(*)
        FROM sys.dm_hadr_database_replica_states
        WHERE synchronization_state_desc IN ('SYNCHRONIZED', 'SYNCHRONIZING')
        AND is_local = 0
    """)
    count = cur.fetchone()[0]

```

```

conn.close()
return count >= min_replicas

def check_cluster_complete(dsn: str, expected_members: int) -> bool:
    conn = pyodbc.connect(dsn, timeout=5)
    cur = conn.cursor()
    cur.execute("""
        SELECT COUNT(*)
        FROM sys.dm_hadr_availability_replica_states
        WHERE connected_state_desc = 'CONNECTED'
    """)
    count = cur.fetchone()[0]
    conn.close()
    return count >= expected_members

def execute_post_failover_runbook(
    new_primary_dsn: str,
    expected_cluster_size: int,
    escalation_fn: Callable[[str], None],
):
    steps = [
        (
            "primary_reachable",
            lambda: check_primary_reachable(new_primary_dsn),
            10,
            True, # fatal if fails – cannot continue without primary
        ),
        (
            "replication_active",
            lambda: check_replication_active(new_primary_dsn),
            60,
            False, # non-fatal – alert but continue
        ),
        (
            "cluster_complete",
            lambda: check_cluster_complete(new_primary_dsn, expected_cluster_size),
            300,
            False,
        ),
    ]

for name, fn, timeout, is_fatal in steps:
    result = run_check_with_timeout(fn, timeout_seconds=timeout, name=name)
    if result != CheckResult.PASS:
        escalation_fn(
            f"Post-failover check '{name}' returned {result.value}. "
            f"Manual inspection required."
        )
    if is_fatal:
        logger.critical(
            f"Fatal check '{name}' failed – halting runbook. "
            f"Cluster may be in unsafe state."
        )

```

```
return
```

```
logger.info("Post-failover runbook complete. Cluster health validated.")
```

The escalation function passed into `execute_post_failover_runbook` is deliberately abstract — in practice it dispatches to a PagerDuty API, a Slack webhook, or an incident management system. The point is that escalation is the *last resort* in the autonomous contract: the runbook exhausts every automated recovery option before producing a human-visible alert.

A companion post-failover validation script runs T-SQL checks against the DMVs and posts results to a centralized monitoring table that the operator controller reads:

```
-- Post-failover health summary for autonomous AG validation
-- Insert into a shared audit table read by the orchestration layer
INSERT INTO dba.ag_failover_audit (
    failover_time,
    new_primary,
    ag_name,
    replica_count_healthy,
    max_redo_queue_kb,
    validation_status
)
SELECT
    SYSDATETIMEOFFSET()                AS failover_time,
    SERVERNAME                          AS new_primary,
    ag.name                             AS ag_name,
    SUM(CASE WHEN ars.synchronization_health_desc = 'HEALTHY'
              THEN 1 ELSE 0 END)         AS
replica_count_healthy,
    MAX(drs.redo_queue_size)             AS max_redo_queue_kb,
    CASE
        WHEN SUM(CASE WHEN ars.synchronization_health_desc = 'HEALTHY'
                        THEN 1 ELSE 0 END) >= 2
            AND MAX(drs.redo_queue_size) < 102400  -- under 100MB redo backlog
        THEN 'VALIDATED'
        ELSE 'DEGRADED'
    END                                  AS validation_status
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id
WHERE ag.name = 'ProductionAG'
GROUP BY ag.name;
```

The `dba.ag_failover_audit` table becomes the authoritative record of every failover event: what triggered it, when it occurred, who became primary, how long reintegration took, and whether post-failover validation passed. This table is read by the automated reporting layer to produce post-incident summaries without human authorship — the system describes what happened to itself.

17.7 Split-Brain Prevention: Fencing, STONITH, and Consensus Guarantees

Every automated HA system faces one adversarial scenario that cannot be resolved by checking metrics: the network partition where both the primary and a promoted replica believe they are the authoritative write endpoint. This is split-brain, and it is the failure mode that turns an HA system into a data consistency catastrophe.

Zero-touch HA does not just detect split-brain — it prevents it architecturally. The prevention mechanisms operate at three levels.

Level 1: Quorum-enforced primary exclusivity. In WSFC, the cluster can only bring the availability group online on the side holding a majority of quorum votes. A replica cannot become primary without quorum. A primary that loses quorum connectivity is taken offline rather than continuing to serve writes without a valid majority. The quorum vote is the arbiter — no quorum, no primary designation.

Level 2: STONITH (Shoot The Other Node In The Head). When a replica is promoted, it must ensure the former primary cannot continue accepting writes. STONITH accomplishes this by issuing a fencing command — typically a power cycle or network isolation command — through an out-of-band management interface (IPMI, AWS EC2 stop-instance API, Azure VM deallocate). The promotion is not considered complete until the fencing operation confirms the former primary is offline.

In cloud environments, STONITH is implemented through the provider's control plane API:

```

import boto3
import logging

logger = logging.getLogger("stonith")

def fence_instance(instance_id: str, region: str) -> bool:
    """
    Hard fence an EC2 instance by stopping it via the AWS API.
    Called by the HA orchestrator when a replica is promoted
    and the former primary's cluster membership has not been gracefully released.
    Used as a STONITH mechanism to guarantee the former primary cannot write.
    """
    ec2 = boto3.client("ec2", region_name=region)
    try:
        response = ec2.stop_instances(
            InstanceIds=[instance_id],
            Force=True # Force=True bypasses graceful shutdown
        )
        state = response["StoppingInstances"][0]["CurrentState"]["Name"]
        logger.info(f"Fencing {instance_id}: instance state is now '{state}'")
        return state in ("stopping", "stopped")
    except Exception as e:
        logger.critical(
            f"STONITH FAILED for {instance_id}: {e}. "
            f"Human intervention required – potential split-brain."
        )
        return False

```

If STONITH fails — if the fencing operation cannot reach the management API, or the API returns an error — the autonomous system must not complete the promotion. Instead, it must halt the election and generate a critical alert. A failed fence is worse than a failed failover: a failed failover leaves you with no primary; a failed fence with a completed promotion leaves you with two primaries.

Level 3: Forced-failover sequencing. When the orchestrator issues the promotion command, it does so only after confirming the fencing operation succeeded and quorum majority is held. The sequencing is:

1. Confirm quorum majority on the surviving side
2. Confirm fencing of former primary (API acknowledgment)
3. Execute ALTER AVAILABILITY GROUP ... FORCE_FAILOVER_ALLOW_DATA_LOSS on the winning replica
4. Move the AG listener to the new primary and update DNS
5. Confirm sp_server_diagnostics reports healthy before opening client traffic

Step 2 must complete before step 3. This is not a best-effort sequence — it is enforced in the orchestrator's promotion code path. Any HA framework that promotes first and fences second is architecturally unsafe.

```
sequenceDiagram
    participant Quorum as WSFC Quorum
    participant Replica as Candidate Replica
    participant Fence as Fencing API (IPMI/Cloud)
    participant OldPrimary as Former Primary
    participant Listener as AG Listener

    Replica->>Quorum: Confirm majority held
    Quorum-->>Replica: Majority confirmed
    Replica->>Fence: Fence former primary
    Fence->>OldPrimary: Power cycle / network isolate
    Fence-->>Replica: Fence confirmed
    Replica->>Replica: ALTER AVAILABILITY GROUP ... FORCE_FAILOVER
    Replica->>Listener: Move listener to new primary
    Listener->>Replica: Route write traffic
```

Promotion only proceeds after the fence is confirmed — a failed fence halts the sequence rather than risking dual primaries.

The quorum configuration itself deserves periodic autonomous review — `Node Majority` for odd-node clusters, `Node` and `File Share Majority` for even-node clusters, plus dynamic quorum and dynamic witness behavior. An autonomous controller validates that the effective vote count still yields a strict majority after every topology change, because misconfigured quorum is the most common cause of split-brain in production WSFC environments.

17.8 Observable Failover: The Audit Trail That Explains Itself

An autonomous system that cannot explain its own decisions is unauditably and, in regulated environments, non-compliant. Zero-touch HA must generate a structured, timestamped, causal audit trail for every failover event: what signals triggered the decision, what threshold was crossed, what actions were taken in what sequence, and what the outcome of each action was.

The audit trail is not a log file — it is a structured database record that can be queried, compared across failover events, and used to improve the decision logic over time. Each failover event is a first-class entity:

```

-- Failover audit schema for autonomous HA
CREATE TABLE dba.failover_events (
    event_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    event_time        DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    ag_name            NVARCHAR(128) NOT NULL,
    trigger_signal     NVARCHAR(256) NOT NULL, -- e.g., 'TCP_HEALTH_CHECK_FAILED'
    trigger_details    NVARCHAR(MAX),          -- raw signal data at decision time
    (JSON)
    former_primary     NVARCHAR(128) NOT NULL,
    new_primary        NVARCHAR(128) NOT NULL,
    candidate_set      NVARCHAR(MAX),          -- all replicas evaluated, with
hardened LSN
    winning_candidate  NVARCHAR(128) NOT NULL,
    fence_attempted    BIT NOT NULL,
    fence_succeeded    BIT,
    promotion_time     DATETIMEOFFSET,
    listener_switch_time DATETIMEOFFSET,
    first_client_conn  DATETIMEOFFSET,
    replication_resumed DATETIMEOFFSET,
    total_downtime_ms  INT,
    validation_status  NVARCHAR(32),
    escalated          BIT NOT NULL DEFAULT 0,
    escalation_reason  NVARCHAR(MAX)
);

-- Indexes for post-incident analysis
CREATE INDEX IX_failover_events_time ON dba.failover_events (event_time DESC);
CREATE INDEX IX_failover_events_primary ON dba.failover_events (former_primary,
event_time DESC);
GO

-- View for SLA reporting: downtime per calendar month
CREATE OR ALTER VIEW dba.monthly_ha_summary AS
SELECT
    DATEFROMPARTS(YEAR(event_time), MONTH(event_time), 1) AS month,
    COUNT(*) AS failover_count,
    AVG(CAST(total_downtime_ms AS FLOAT)) AS avg_downtime_ms,
    MAX(total_downtime_ms) AS max_downtime_ms,
    SUM(CAST(total_downtime_ms AS BIGINT)) / 1000.0 AS
total_downtime_seconds,
    SUM(CASE WHEN escalated = 1 THEN 1 ELSE 0 END) AS escalated_count,
    SUM(CASE WHEN validation_status = N'VALIDATED' THEN 1 ELSE 0 END) AS
clean_failovers
FROM dba.failover_events
GROUP BY DATEFROMPARTS(YEAR(event_time), MONTH(event_time), 1);

```

The `trigger_details` JSON column stores the raw signal values at the moment the decision was made — the redo queue in KB, the seconds since the last `sp_server_diagnostics` heartbeat, the load balancer health check failure count. This is the evidence record that answers the post-incident question: "Why did the system fail over at 03:47?" The answer is

not "because the cluster said so" — it is a structured document showing that the redo queue was 2.3 GB, the last diagnostics heartbeat was 38 seconds ago (above the 20-second lease timeout), and TCP connectivity had failed three consecutive checks in the preceding 6 seconds.

The equivalent SQL Server audit table captures the same causal chain but reads from DMV snapshots captured at the time of the operator's reconciliation loop:

```
-- SQL Server: failover audit schema in the DBA utility database
CREATE TABLE dba.ag_failover_events (
    event_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    event_time        DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    ag_name           NVARCHAR(128) NOT NULL,
    former_primary    NVARCHAR(128) NOT NULL,
    new_primary       NVARCHAR(128) NOT NULL,
    trigger_signal     NVARCHAR(256) NOT NULL,
    send_queue_at_trigger BIGINT,      -- log_send_queue_size KB at trigger time
    redo_queue_at_trigger BIGINT,      -- redo_queue_size KB at trigger time
    failover_type      NVARCHAR(64),   -- 'AUTOMATIC' or 'FORCED'
    data_loss_detected BIT             NOT NULL DEFAULT 0,
    validation_status  NVARCHAR(32),
    total_downtime_ms INT,
    operator_version   NVARCHAR(64),  -- operator image version that performed
failover
    notes              NVARCHAR(MAX)
);
```

The `operator_version` column is subtle but important: it records which version of the Kubernetes operator or cluster service executed the failover. When a failover behaves unexpectedly — too slow, incorrect candidate selection, fence failure — the operator version is the first variable to investigate. Autonomous systems evolve, and their audit trails must capture the version of the autonomous agent that made each decision, not just the decision itself.

Key Takeaways

- → **Zero-touch failover requires a multi-signal decision matrix, not a single trip-wire.** Autonomous HA systems act on the aggregate of TCP connectivity, replication lag, replica quorum, and process health signals — preventing failover storms from transient network events while catching real outages with high confidence.
- → **WSFC quorum serves a single architectural role:** it externalizes leadership state into a shared, durable medium so that no individual node can unilaterally claim primary status. Any HA framework that does not anchor leadership in external consensus is architecturally unsafe for autonomous operation.

- → **Traffic rerouting speed is determined by the health-check stack, not DNS.** The combination of the AG listener move, HAProxy's interval-based TCP health checks with `on-marked-down shutdown-sessions`, and the driver's `ConnectRetryCount` delivers sub-10-second connection rerouting without application code changes or DNS TTL dependency.
- → **Split-brain prevention is non-negotiable and must be enforced by sequencing:** acquire consensus key, confirm fencing, then promote. An HA system that promotes first and fences later — or that treats fencing as optional — exposes the cluster to dual-primary data corruption. Failed fencing must halt the promotion, not ignore the error.
- → **The failover audit trail is as important as the failover itself.** A structured, queryable record of every trigger signal, candidate evaluation, fencing outcome, and validation result turns an autonomous HA system from a black box into an accountable agent — one that can explain its decisions to post-incident reviews, compliance auditors, and the next iteration of its own decision logic.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 18: Autonomous Disaster Recovery

Disaster recovery has always been the discipline where human judgment mattered most — the 3 a.m. call, the war-room decision, the tense minutes choosing between data loss and extended downtime. Autonomous systems change that contract entirely. When your DR pipeline self-monitors replication health, self-diagnoses failure mode, self-selects a recovery path, and self-executes a tested runbook without waiting for a human to wake up, your RTO shrinks from hours to seconds and your RPO approaches zero. This chapter covers the architecture and implementation of fully autonomous DR systems for SQL Server — systems that observe failure signals at the infrastructure layer, reason about failure type and blast radius, execute tiered recovery actions with circuit breakers, and validate recovery integrity before re-admitting traffic, all without manual intervention.

Failure Taxonomy and Autonomous Signal Acquisition

Before an autonomous system can recover from a disaster, it must correctly classify what it is recovering from. Treating a network partition the same as a storage corruption or an application-level connection flood produces wrong recovery actions and can worsen the situation. A taxonomy-first architecture forces the system to gate all recovery paths through a classification layer before execution.

The five failure classes that autonomous DR must handle independently are:

1. **Primary node loss** — the database process or host becomes unreachable
2. **Replication lag breach** — standby is alive but diverging beyond RPO threshold
3. **Split-brain ambiguity** — multiple nodes believe they are primary
4. **Data corruption** — block-level, page-level, or logical corruption on disk or in transit
5. **Region/AZ failure** — an entire failure domain becomes unavailable

Signal acquisition for each class requires distinct sensor types. For node loss, TCP health checks and SQL Server @@SERVERNAME reachability form the first layer. For replication lag, you instrument the replication stream directly.

```
-- Replication lag sensor for Always On AG
SELECT
    ag.name AS ag_name,
    ar.replica_server_name,
    drs.database_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.log_send_queue_size,
    drs.redo_queue_size,
    drs.last_commit_time,
    DATEDIFF(SECOND, drs.last_commit_time, GETUTCDATE()) AS lag_seconds
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
WHERE drs.is_local = 0;
```

These queries run on a 5-second scrape interval from a Prometheus exporter. When `replication_lag_bytes` exceeds your RPO threshold (converted to bytes using average transaction log generation rate) or `heartbeat_age_seconds` exceeds 15 seconds, the signal is emitted to the classification engine.

The classification engine is a Python process that aggregates signals across layers — network, OS, database process, replication, and application — before assigning a failure class. It deliberately waits 10–30 seconds before acting, using a voting window to distinguish transient blips from real events:

```

import time
import collections
from dataclasses import dataclass, field
from enum import Enum
from typing import Optional

class FailureClass(Enum):
    NODE_LOSS = "node_loss"
    REPLICATION_LAG = "replication_lag"
    SPLIT_BRAIN = "split_brain"
    DATA_CORRUPTION = "data_corruption"
    AZ_FAILURE = "az_failure"
    UNKNOWN = "unknown"

@dataclass
class FailureSignal:
    source: str
    signal_type: str
    severity: float # 0.0 to 1.0
    timestamp: float = field(default_factory=time.time)

class FailureClassifier:
    VOTE_WINDOW_SECONDS = 20
    CONFIDENCE_THRESHOLD = 0.70

    def __init__(self):
        self.signal_buffer: list[FailureSignal] = []

    def ingest(self, signal: FailureSignal):
        self.signal_buffer.append(signal)
        self._evict_stale()

    def classify(self) -> tuple[FailureClass, float]:
        if not self.signal_buffer:
            return FailureClass.UNKNOWN, 0.0

        votes: dict[FailureClass, float] = collections.defaultdict(float)

        for sig in self.signal_buffer:
            if sig.signal_type == "tcp_unreachable" and sig.severity > 0.8:
                votes[FailureClass.NODE_LOSS] += sig.severity
            elif sig.signal_type == "replication_lag_bytes" and sig.severity > 0.6:
                votes[FailureClass.REPLICATION_LAG] += sig.severity
            elif sig.signal_type == "dual_primary_detected":
                votes[FailureClass.SPLIT_BRAIN] += sig.severity * 2 # escalate
weight
            elif sig.signal_type == "checksum_failure":
                votes[FailureClass.DATA_CORRUPTION] += sig.severity
            elif sig.signal_type == "az_health_degraded" and sig.severity > 0.9:
                votes[FailureClass.AZ_FAILURE] += sig.severity

        if not votes:
            return FailureClass.UNKNOWN, 0.0

```

```

top_class = max(votes, key=votes.__getitem__)
total = sum(votes.values())
confidence = votes[top_class] / total if total > 0 else 0.0

if confidence >= self.CONFIDENCE_THRESHOLD:
    return top_class, confidence
return FailureClass.UNKNOWN, confidence

def _evict_stale(self):
    cutoff = time.time() - self.VOTE_WINDOW_SECONDS
    self.signal_buffer = [s for s in self.signal_buffer if s.timestamp >=
cutoff]

```

The classifier only fires a recovery action when confidence exceeds 70% within the vote window. This is not a soft threshold — it is a hard gate. Below 70%, the system escalates to a human via PagerDuty while continuing to collect signals. Above 70%, the recovery pipeline fires autonomously.

Autonomous Failover Architecture: WSFC Quorum and SQL Server Always On

Once failure is classified and confidence is sufficient, the autonomous system hands off to the failover execution layer. For SQL Server, **Always On Availability Groups** with the Windows Server Failover Cluster on Windows or a Pacemaker cluster on Linux provide the failover substrate. Two quorum architectures represent distinct operational philosophies: **WSFC quorum** (vote-based, witness-backed) and **Pacemaker** (resource-agent-based, common on Linux).

WSFC Quorum: Consensus-Driven Autonomous Failover

WSFC uses a vote-based quorum as its consensus layer. Each node holds a vote, and an optional witness (file share, disk, or cloud witness) breaks ties. The cluster stays online only while it holds a majority of votes; when the primary's side loses majority, resources — including the availability group — go offline there rather than risking split-brain. For fully autonomous operation, the key configuration parameters are:

```
# PowerShell – production autonomous quorum configuration
# Review current quorum votes
Get-ClusterNode | Format-Table Name, NodeWeight, DynamicWeight

# Set a file-share witness (even-node clusters)
Set-ClusterQuorum -NodeAndFileShareMajority '\\fileserver\quorum$'

# Or a cloud witness (Azure-hosted clusters)
Set-ClusterQuorum -NodeAndCloudMajority -AccountName 'quorumstore'

# Verify the effective vote count still yields a strict majority
# after every topology change
Test-Cluster -Node sql-primary, sql-secondary, sql-replica1
```

The `REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT = 1` setting on the availability group is your RPO gate. If a candidate secondary has not hardened the transaction log, the commit does not succeed — preventing an autonomous failover that would lose committed data. Combined with `SYNCHRONOUS_COMMIT` availability mode, the AG ensures at least one secondary is always synchronous, so automatic failover never loses acknowledged writes.

Pacemaker on Linux: Resource-Agent-Based Simplicity

For SQL Server on Linux, `Pacemaker` provides cluster management with less operational overhead than a full WSFC deployment. The availability group is managed as a Pacemaker resource via the `ocf:mssql:ag` resource agent:

```
# Create the AG cluster resource on each Linux node
sudo pcs resource create ag_production ocf:mssql:ag \
  ag_name=AG_Production \
  meta failure-timeout=60s \
  op start timeout=60s \
  op stop timeout=60s \
  op promote timeout=60s \
  op demote timeout=60s \
  op monitor timeout=60s interval=10s

# Constrain the virtual IP to follow the primary
sudo pcs resource create virtualip ocf:heartbeat:IPaddr2 \
  ip=10.0.1.50 cidr_netmask=24

sudo pcs constraint colocation add virtualip ag_production-master \
  INFINITY with-rsc-role=Master
```

The resource agent runs a monitor operation every 10 seconds against the AG DMVs. When the primary node stops responding and the standby's data movement is healthy, Pacemaker promotes the standby and moves the virtual IP with it — autonomously. No human action required.

When the monitor operation fails for the primary node across consecutive intervals and the standby's data movement is healthy, Pacemaker transitions the standby's role to `Master` autonomously. No human action required.

SQL Server Always On with Automatic Failover

SQL Server AG automatic failover requires WSFC quorum on Windows or Pacemaker on Linux. The key autonomous configuration is the `FAILOVER_MODE = AUTOMATIC` clause combined with health check thresholds:

```
-- Create AG with autonomous failover configuration
CREATE AVAILABILITY GROUP [AG_Production]
WITH (
    AUTOMATED_BACKUP_PREFERENCE = SECONDARY,
    HEALTH_CHECK_TIMEOUT = 30000,          -- 30 seconds before declaring unhealthy
    DB_FAILOVER = ON,                      -- fail over on database-level health
issues
    DTC_SUPPORT = NONE,
    REQUIRED_SYNCHRONIZED_SECONDARIES_TO_COMMIT = 1 -- synchronous commit gate
)
FOR DATABASE [AppDB], [ReportDB]
REPLICA ON
    N'sql-primary' WITH (
        ENDPOINT_URL = N'TCP://sql-primary.internal:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SEEDING_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    ),
    N'sql-secondary' WITH (
        ENDPOINT_URL = N'TCP://sql-secondary.internal:5022',
        AVAILABILITY_MODE = SYNCHRONOUS_COMMIT,
        FAILOVER_MODE = AUTOMATIC,
        SEEDING_MODE = AUTOMATIC,
        SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
    );

-- Set flexible failover policy (SQL Server 2019+)
ALTER AVAILABILITY GROUP [AG_Production]
    MODIFY REPLICA ON N'sql-primary'
        WITH (FAILURE_CONDITION_LEVEL = 3);
-- Level 3: fail on critical SQL Server errors, not just process crashes
```

`FAILURE_CONDITION_LEVEL = 3` is the recommended production setting. Level 1 triggers on server-down only; Level 3 adds SQL Server internal critical errors and orphaned spinlocks. Level 5 (any qualified failure condition) produces too many false positives in multi-tenant environments.

Connection string design matters enormously for autonomous failover. Applications must use the AG listener, not the instance name, and must set `MultiSubnetFailover=True` and `ConnectRetryCount=3` to survive the ~15–30 second WSFC transition window without returning errors to the application layer.

RPO-Aware Autonomous Recovery Decision Engine

Automatic failover tools (AG automatic failover) handle the happy path — clean primary failure, healthy synchronous replica, quorum intact. Production disasters rarely follow this path. Your autonomous system needs a decision engine that reasons about recovery options when conditions are degraded: the synchronous replica is also down, the lag breach is severe, quorum is lost, or you face a true split-brain.

The recovery decision engine is a state machine that maps (failure class, current topology state) → recovery action. The topology state is pulled from the cluster's own metadata store:

```

import pyodbc
import subprocess
import logging
from enum import Enum
from typing import NamedTuple

logger = logging.getLogger("autonomous_dr")

class RecoveryAction(Enum):
    PROMOTE_SYNC_REPLICA = "promote_sync_replica"
    PROMOTE_ASYNC_REPLICA = "promote_async_replica_with_data_loss_ack"
    RESTORE_FROM_BACKUP = "restore_from_backup"
    WAIT_FOR_PRIMARY = "wait_for_primary_recovery"
    ESCALATE_TO_HUMAN = "escalate_to_human"
    FENCE_OLD_PRIMARY = "fence_old_primary"

class TopologyState(NamedTuple):
    sync_replica_available: bool
    sync_replica_lag_bytes: int
    async_replica_available: bool
    async_replica_lag_bytes: int
    cluster_quorum_healthy: bool
    old_primary_network_reachable: bool
    rpo_bytes: int # configured RPO converted to bytes

class RecoveryDecisionEngine:

    def decide(self, failure_class, topo: TopologyState) -> RecoveryAction:
        # Split-brain: fence before anything else
        if failure_class == "split_brain":
            if topo.old_primary_network_reachable:
                return RecoveryAction.FENCE_OLD_PRIMARY
            return RecoveryAction.ESCALATE_TO_HUMAN

        # Primary lost, cluster quorum unhealthy – can't safely elect
        if not topo.cluster_quorum_healthy:
            logger.warning("Cluster quorum unhealthy – cannot safely elect new
primary")
            return RecoveryAction.ESCALATE_TO_HUMAN

        # Primary lost, sync replica available and within RPO
        if (failure_class == "node_loss"
            and topo.sync_replica_available
            and topo.sync_replica_lag_bytes <= topo.rpo_bytes):
            return RecoveryAction.PROMOTE_SYNC_REPLICA

        # Primary lost, sync replica down or lagging, async within RPO
        if (failure_class == "node_loss"
            and not topo.sync_replica_available
            and topo.async_replica_available
            and topo.async_replica_lag_bytes <= topo.rpo_bytes):
            logger.warning(
                "Promoting async replica – lag=%d bytes within RPO=%d bytes",

```

```

        topo.async_replica_lag_bytes, topo.rpo_bytes
    )
    return RecoveryAction.PROMOTE_ASYNC_REPLICA

    # Primary lost, all replicas beyond RPO
    if failure_class == "node_loss" and not topo.sync_replica_available:
        logger.error("All replicas exceed RPO – data loss promotion requires
human sign-off")
        return RecoveryAction.ESCALATE_TO_HUMAN

    # Replication lag breach on running primary – don't failover, investigate
    if failure_class == "replication_lag":
        return RecoveryAction.WAIT_FOR_PRIMARY # triggers lag remediation
pipeline

    # Data corruption detected
    if failure_class == "data_corruption":
        return RecoveryAction.RESTORE_FROM_BACKUP

    return RecoveryAction.ESCALATE_TO_HUMAN

```

```

graph TD
    A[Failure classified] --> B{split_brain?}
    B -->|reachable| C[FENCE_OLD_PRIMARY]
    B -->|unreachable| H[ESCALATE_TO_HUMAN]
    B -->|no| D{cluster quorum healthy?}
    D -->|no| H
    D -->|yes| E{sync replica available
and within RPO?}
    E -->|yes| F[PROMOTE_SYNC_REPLICA]
    E -->|no| G{async replica available
and within RPO?}
    G -->|yes| I[PROMOTE_ASYNC_REPLICA
with data-loss ack]
    G -->|no| H

```

The recovery decision engine's mapping from failure class and topology state to action — every path that would exceed RPO without pre-authorization terminates in human escalation, never a silent promotion.

Notice the deliberate conservatism: the engine escalates to a human whenever data loss would exceed the RPO and a human sign-off was not pre-authorized. This reflects a core ADE principle — **autonomous systems must have explicit permission to cause data loss, not implicit permission by default**. In practice, you can configure a `data_loss_authorized: true` flag per environment (acceptable in a dev/staging cluster, never in production without an SLA change).

The fencing action deserves special attention. Promoting a replica while the original primary is still reachable and accepting writes produces split-brain. The fence step must shoot the old primary in the head (STONITH — Shoot The Other Node In The Head) before completing promotion. On bare metal or VM infrastructure this is a power fence via IPMI/BMC. On cloud infrastructure it is an API call:

```
import boto3

def fence_ec2_instance(instance_id: str, region: str) -> bool:
    """
    Hard fence an EC2 instance by stopping it via AWS API.
    Returns True if fence succeeded, False if API call failed.
    The caller must verify instance state before proceeding with promotion.
    """
    ec2 = boto3.client("ec2", region_name=region)
    try:
        response = ec2.stop_instances(
            InstanceIds=[instance_id],
            Force=True # equivalent to power cut – no graceful shutdown
        )
        state = response["StoppingInstances"][0]["CurrentState"]["Name"]
        logger.info("Fence initiated for %s – current state: %s", instance_id,
state)

        # Poll for stopped state – max 60 seconds
        waiter = ec2.get_waiter("instance_stopped")
        waiter.wait(
            InstanceIds=[instance_id],
            WaiterConfig={"Delay": 5, "MaxAttempts": 12}
        )
        logger.info("Fence confirmed for %s", instance_id)
        return True
    except Exception as exc:
        logger.error("Fence failed for %s: %s", instance_id, exc)
        return False
```

If the fence fails, promotion must not proceed. A failed fence is an escalation event.

Automated Backup Validation and Point-in-Time Recovery Pipelines

Autonomous DR is only as strong as the backups it can restore from. Most organizations discover their backups are corrupted or incomplete during the actual disaster, not before. The autonomous pattern inverts this: **restores are executed continuously in a shadow environment, and any failure in shadow restoration is treated as a current-severity alert.**

The continuous restore pipeline operates on a schedule (typically every 4–8 hours for production databases) and validates three properties: restore completeness, data integrity, and RPO alignment.

For SQL Server, the autonomous validation pipeline runs on an isolated restore host every 6 hours (SQL Agent job or Kubernetes CronJob). Every backup cycle starts with rapid `RESTORE VERIFYONLY` verification; on a scheduled basis the pipeline performs a full shadow restore, replays the log chain, validates row counts, and pushes duration/RPO metrics to Prometheus:

Rapid backup file verification (run every backup cycle):

```

-- Rapid backup file verification (run every backup cycle)
RESTORE VERIFYONLY
FROM DISK = N'\\backup-share\AppDB\AppDB_FULL_20240115.bak'
WITH
    CHECKSUM,
    STATS = 10;

-- Full shadow restore to validation instance (scheduled, separate from production)
RESTORE DATABASE [AppDB_Validate]
FROM DISK = N'\\backup-share\AppDB\AppDB_FULL_20240115.bak'
WITH
    MOVE N'AppDB_data' TO N'D:\ValidateData\AppDB_validate.mdf',
    MOVE N'AppDB_log' TO N'D:\ValidateData\AppDB_validate.ldf',
    NORECOVERY,
    STATS = 5;

-- Apply latest differential
RESTORE DATABASE [AppDB_Validate]
FROM DISK = N'\\backup-share\AppDB\AppDB_DIFF_20240115.bak'
WITH
    NORECOVERY,
    STATS = 5;

-- Apply log chain to bring within RPO
RESTORE DATABASE [AppDB_Validate]
FROM DISK = N'\\backup-share\AppDB\AppDB_LOG_20240115_2300.trn'
WITH
    RECOVERY,
    STOPAT = '2024-01-15T23:59:59',
    STATS = 5;

-- Validate table counts and row counts post-restore
USE [AppDB_Validate];
SELECT
    t.name AS table_name,
    p.rows AS estimated_row_count
FROM sys.tables t
JOIN sys.partitions p
    ON t.object_id = p.object_id
WHERE p.index_id IN (0, 1)
ORDER BY p.rows DESC;

```

The shadow restore must run on a separate isolated instance — not a test schema on the production server. Restoring to the same instance risks corrupting production configuration, and the performance overhead of a large restore can degrade production query throughput.

Point-in-Time Recovery Automation

When data corruption or logical error is detected, the recovery action from the decision engine is `RESTORE_FROM_BACKUP`. Autonomous PITR must select the correct recovery target — not simply "restore the latest backup" but "restore to the point immediately before the corruption event was first detected."

The autonomous PITR pipeline needs three inputs:

1. The corruption detection timestamp (from the monitoring system)
2. The backup catalog (SQL Server msdb history)
3. The log archive to replay forward from the backup

Autonomous PITR selection via msdb:

```

import pyodbc
import datetime
import logging

logger = logging.getLogger("autonomous_pitr")

MSSQL_DSN = (
    "DRIVER={ODBC Driver 18 for SQL Server};"
    "SERVER=sql-primary;DATABASE=msdb;Trusted_Connection=yes;"
)

def select_pitr_target(corruption_detected_at: datetime.datetime,
                      database_name: str) -> dict:
    """
    Select the backup chain and PITR target for autonomous restoration.
    Restores to 5 minutes before detection to avoid already-corrupt data.
    """
    # Add safety margin – restore to 5 minutes before detection
    target_time = corruption_detected_at - datetime.timedelta(minutes=5)

    conn = pyodbc.connect(MSSQL_DSN)
    cur = conn.cursor()

    # Most recent full backup before the PITR target
    cur.execute("""
        SELECT TOP 1
            bs.backup_set_id,
            bs.backup_finish_date,
            bmf.physical_device_name,
            bs.compressed_backup_size,
            bs.first_lsn,
            bs.last_lsn
        FROM dbo.backupset bs
        JOIN dbo.backupmediafamily bmf
            ON bs.media_set_id = bmf.media_set_id
        WHERE bs.database_name = ?
            AND bs.type = 'D'
            AND bs.backup_finish_date <= ?
        ORDER BY bs.backup_finish_date DESC
    """, database_name, target_time)
    full = cur.fetchone()
    if not full:
        raise RuntimeError("No full backup found before PITR target")

    # Differential + log chain after that full backup, up to the target
    cur.execute("""
        SELECT
            bs.backup_set_id,
            bs.type,
            bs.backup_finish_date,
            bmf.physical_device_name,
            bs.first_lsn,
            bs.last_lsn
    """)

```



```

FROM dbo.backupset bs
JOIN dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = ?
    AND bs.type IN ('I', 'L')
    AND bs.backup_finish_date > ?
    AND bs.backup_start_date <= ?
ORDER BY bs.backup_start_date
"", database_name, full.backup_finish_date, target_time)
chain = cur.fetchall()
conn.close()

estimated_minutes = (full.compressed_backup_size or 0) / (1024**3) / 2.0
logger.info("PITR target %s: full backup %s + %d chain members",
            target_time.isoformat(), full.backup_set_id, len(chain))

return {
    "pitr_target": target_time.isoformat(),
    "full_backup_set_id": full.backup_set_id,
    "chain_device_names": [row.physical_device_name for row in chain],
    "estimated_restore_minutes": round(estimated_minutes, 1),
}

```

```

def execute_pitr(conn, database_name: str, pitr: dict) -> bool:
    """
    Execute the restore chain: full WITH NORECOVERY, then each chain
    member WITH NORECOVERY, final log WITH RECOVERY, STOPAT = pitr_target.
    Returns True on success, False on failure.
    """
    cur = conn.cursor()
    try:
        logger.info("Restoring full backup for %s", database_name)
        cur.execute(
            f"RESTORE DATABASE [{database_name}] FROM DISK = ? "
            f"WITH NORECOVERY, STATS = 5",
            pitr["full_device_name"],
        )
        for device in pitr["chain_device_names"][:-1]:
            logger.info("Applying chain member %s", device)
            cur.execute(
                f"RESTORE DATABASE [{database_name}] FROM DISK = ? "
                f"WITH NORECOVERY, STATS = 5",
                device,
            )
        logger.info("Applying final log to PITR target %s", pitr["pitr_target"])
        cur.execute(
            f"RESTORE DATABASE [{database_name}] FROM DISK = ? "
            f"WITH RECOVERY, STOPAT = ?",
            pitr["chain_device_names"][-1], pitr["pitr_target"],
        )
        conn.commit()
        logger.info("PITR restore completed successfully")
    
```

```
        return True
    except pyodbc.Error as exc:
        logger.error("PITR restore failed: %s", exc)
    return False
```

Backup chain identification in T-SQL:

```

-- Identify the correct backup set for autonomous PITR
-- @corruption_time: the timestamp of first detected corruption
DECLARE @corruption_time DATETIME2 = '2024-01-15T22:47:00';
DECLARE @pitr_target      DATETIME2 = DATEADD(MINUTE, -5, @corruption_time);
DECLARE @database_name    NVARCHAR(128) = N'AppDB';

-- Find most recent full backup before PITR target
SELECT TOP 1
    bs.backup_set_id,
    bs.backup_start_date,
    bs.backup_finish_date,
    bs.backup_size,
    bs.compressed_backup_size,
    bmf.physical_device_name,
    bs.first_lsn,
    bs.last_lsn,
    bs.database_backup_lsn,
    'FULL' AS backup_type
FROM msdb.dbo.backupset bs
JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = @database_name
    AND bs.type = 'D' -- Full backup
    AND bs.backup_finish_date <= @pitr_target
ORDER BY bs.backup_finish_date DESC;

-- Then find the differential and log chain to bring to PITR target
SELECT
    bs.backup_set_id,
    bs.type AS backup_type,
    bs.backup_start_date,
    bs.backup_finish_date,
    bmf.physical_device_name,
    bs.first_lsn,
    bs.last_lsn
FROM msdb.dbo.backupset bs
JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = @database_name
    AND bs.type IN ('I', 'L') -- Differential and Log
    AND bs.backup_finish_date > (
        SELECT TOP 1 backup_finish_date
        FROM msdb.dbo.backupset
        WHERE database_name = @database_name
            AND type = 'D'
            AND backup_finish_date <= @pitr_target
        ORDER BY backup_finish_date DESC
    )
    AND bs.backup_start_date <= @pitr_target
ORDER BY bs.backup_start_date;

```

Once the backup chain is identified, the automated restore script iterates through the results, issuing `RESTORE DATABASE ... WITH NORECOVERY` for each backup in the chain and a final `RESTORE DATABASE ... WITH RECOVERY, STOPAT = @pitr_target` for the last log backup.

Circuit Breakers and Recovery Runbook Orchestration

Autonomous DR without circuit breakers is dangerous. An incorrectly classified failure can trigger a chain of recovery actions that deepens the incident — for example, repeatedly promoting replicas in a loop, or initiating PITR on a database that was actually only experiencing a network hiccup. Circuit breakers impose a hard limit on autonomous action frequency and allow the system to pause and escalate rather than spiral.

The circuit breaker pattern for DR has three states: **Closed** (normal operation, recovery actions allowed), **Open** (recovery actions suspended, human required), and **Half-Open** (a limited test action is permitted to probe whether conditions have normalized).

```

import time
import threading
from enum import Enum

class CBState(Enum):
    CLOSED    = "closed"
    OPEN      = "open"
    HALF_OPEN = "half_open"

class DRCircuitBreaker:
    """
    Circuit breaker for autonomous DR action execution.

    failure_threshold: number of failed recovery attempts within window to open
    recovery_timeout:   seconds in OPEN state before moving to HALF_OPEN
    action_window:      rolling window in seconds for counting failures
    """

    def __init__(
        self,
        failure_threshold: int = 3,
        recovery_timeout: int = 300,
        action_window: int = 600
    ):
        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.action_window = action_window

        self._state = CBState.CLOSED
        self._failure_times: list[float] = []
        self._opened_at: float = 0.0
        self._lock = threading.Lock()

    @property
    def state(self) -> CBState:
        with self._lock:
            if self._state == CBState.OPEN:
                if time.time() - self._opened_at >= self.recovery_timeout:
                    self._state = CBState.HALF_OPEN
                    return CBState.HALF_OPEN
            return self._state

    def allow_action(self) -> bool:
        """Returns True if an autonomous recovery action is permitted."""
        s = self.state
        return s in (CBState.CLOSED, CBState.HALF_OPEN)

    def record_success(self):
        with self._lock:
            self._state = CBState.CLOSED
            self._failure_times.clear()

    def record_failure(self):

```

```

now = time.time()
with self._lock:
    # Evict old failures outside the window
    cutoff = now - self.action_window
    self._failure_times = [t for t in self._failure_times if t >= cutoff]
    self._failure_times.append(now)

    if (self._state == CBState.HALF_OPEN
        or len(self._failure_times) >= self.failure_threshold):
        self._state = CBState.OPEN
        self._opened_at = now

def status_dict(self) -> dict:
    return {
        "state": self.state.value,
        "failure_count_in_window": len(self._failure_times),
        "opened_at": self._opened_at if self._opened_at else None,
    }

```

The runbook orchestrator wraps every recovery action in the circuit breaker and emits the result to both the metrics pipeline and the incident management system:

```

import asyncio
import logging
from typing import Callable, Awaitable

logger = logging.getLogger("runbook_orchestrator")

class RunbookOrchestrator:

    def __init__(self, circuit_breaker: DRCircuitBreaker, alert_client):
        self.cb = circuit_breaker
        self.alert_client = alert_client

    async def execute(
        self,
        action_name: str,
        action_fn: Callable[[], Awaitable[bool]],
        context: dict
    ) -> bool:
        """
        Execute a DR recovery action with circuit breaker protection.
        action_fn must return True on success, False on failure.
        """
        if not self.cb.allow_action():
            logger.error(
                "Circuit breaker OPEN – autonomous action %s blocked. "
                "State: %s", action_name, self.cb.status_dict()
            )
            await self.alert_client.page(
                severity="critical",
                summary=f"DR circuit breaker open – {action_name} requires human
execution",
                context=context
            )
            return False

        logger.info("Executing autonomous DR action: %s | context: %s", action_name,
context)
        try:
            success = await action_fn()
            if success:
                self.cb.record_success()
                logger.info("Action %s succeeded", action_name)
                await self.alert_client.notify(
                    severity="info",
                    summary=f"Autonomous DR action completed: {action_name}",
                    context=context
                )
            else:
                self.cb.record_failure()
                logger.error("Action %s returned failure", action_name)
                await self.alert_client.page(
                    severity="high",
                    summary=f"Autonomous DR action failed: {action_name}",

```

```

        context=context
    )
    return success
except Exception as exc:
    self.cb.record_failure()
    logger.exception("Action %s raised exception: %s", action_name, exc)
    await self.alert_client.page(
        severity="critical",
        summary=f"Autonomous DR action raised exception: {action_name}",
        context={**context, "exception": str(exc)}
    )
    return False

```

Three failures within a 10-minute window opens the circuit breaker for 5 minutes. During that open window, the orchestrator blocks all autonomous actions and pages a human. This prevents the system from executing six failed promotions in a row while the on-call engineer is being woken up. Once the circuit re-enters Half-Open after 5 minutes, it permits one probe action; if that succeeds, the circuit closes and autonomous operation resumes.

Post-Recovery Integrity Validation

Completing a failover or restore is not recovery — validated recovery is recovery. Autonomous DR systems must execute a post-recovery validation suite before re-admitting application traffic. Traffic re-admission is gated on validation passing, not simply on the database process being reachable.

The validation suite checks four categories: connectivity, data integrity, replication topology restoration, and application-level smoke tests.

Post-recovery validation:


```

-- 1. AG health check after failover
SELECT
    ag.name AS ag_name,
    ar.replica_server_name,
    ars.role_desc,
    ars.operational_state_desc,
    ars.connected_state_desc,
    ars.synchronization_health_desc,
    drs.synchronization_state_desc,
    drs.log_send_queue_size,
    drs.redo_queue_size
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id
ORDER BY ars.role_desc, ar.replica_server_name;

-- 2. Database consistency check on newly promoted primary
-- Run after promotion, before re-admitting traffic
DBCC CHECKDB (N'AppDB') WITH NO_INFOMSGS, ALL_ERRORMSG;

-- 3. Identity and sequence object integrity
SELECT
    OBJECT_SCHEMA_NAME(object_id) AS schema_name,
    OBJECT_NAME(object_id) AS table_name,
    last_value,
    increment_by,
    is_exhausted
FROM sys.sequences
ORDER BY schema_name, table_name;

-- 4. Agent jobs re-enabled check (SQL Agent jobs are disabled on AG secondary)
SELECT
    job_id,
    name,
    enabled,
    date_modified
FROM msdb.dbo.sysjobs
WHERE enabled = 0
    AND name NOT LIKE '%_DO_NOT_ENABLE%'
ORDER BY name;
-- Jobs that were disabled on secondary must be re-enabled on new primary

```

The SQL Agent job re-enablement check is one of the most commonly missed post-failover steps in manual DR processes. When a SQL Server AG secondary becomes primary, SQL Agent jobs that were suppressed to prevent double-execution must be automatically re-enabled. This is typically handled by a post-failover alert policy or an automated script that watches for AG role change events and triggers the re-enablement:

```
-- Autonomous job re-enablement after AG promotion
-- Triggered by SQL Agent alert on AG role change event (event ID 1480)
DECLARE @new_role NVARCHAR(60);
SET @new_role = (
    SELECT TOP 1 role_desc
    FROM sys.dm_hadr_availability_replica_states
    WHERE is_local = 1
);

IF @new_role = N'PRIMARY'
BEGIN
    -- Re-enable all jobs that carry the auto-manage tag in their description
    UPDATE msdb.dbo.sysjobs
    SET enabled = 1
    WHERE enabled = 0
        AND description LIKE '%AG_AUTO_MANAGE%';

    -- Log the re-enablement
    EXEC msdb.dbo.sp_add_jobhistory
        @job_id = NULL,
        @step_id = 0,
        @sql_message_id = 0,
        @sql_severity = 0,
        @message = 'Autonomous AG failover: re-enabled tagged SQL Agent jobs on new
primary',
        @run_status = 1,
        @run_date = CONVERT(INT, CONVERT(VARCHAR, GETDATE(), 112)),
        @run_time = 0;
END
```

The traffic admission gate is a separate component — typically a load balancer health check endpoint or a DNS TTL expiry — that the validation suite controls. Only when all four validation categories pass does the suite signal the load balancer to re-route connections to the new primary. In Kubernetes environments this is a readiness probe endpoint implemented in the database sidecar container; in traditional environments it is a script that updates a HAProxy backend weight or a Route 53 health check status.

Geo-Redundant and Multi-Region Autonomous DR

Single-region autonomous DR handles the majority of failure scenarios. Region-level failures — AZ outages, cloud region degradation, network backbone events — require a distinct architecture: geo-redundant replication with an autonomous cross-region election mechanism.

The fundamental challenge with cross-region autonomous failover is network latency. Synchronous replication across regions introduces write latency equal to the round-trip time between regions. For most regions this is 20–80 ms, which is unacceptable for OLTP workloads with tight SLAs. The production pattern is therefore:

- → **Intra-region:** synchronous replication, automatic failover, RPO = 0
- → **Cross-region:** asynchronous replication, autonomous failover with explicit data-loss acknowledgment, RPO = replication lag at time of failure

This asymmetric replication design requires the recovery decision engine to know which tier it is operating in. A cross-region failover is always a data-loss event and must be logged as such in the audit trail.

One critical operational detail: an unreachable async secondary lets the log send queue grow on the primary — and a growing send queue can hold log truncation, filling the transaction log. Autonomous DR systems must monitor the send queue and, past a configured threshold, remove the stalled replica from the AG to release truncation:

```
-- Autonomous log send queue monitor – run every 5 minutes
-- Removes a stalled async replica that is holding log truncation
DECLARE @max_queue_kb BIGINT = 10485760; -- 10 GB send queue limit

IF EXISTS (
    SELECT 1
    FROM sys.dm_hadr_database_replica_states drs
    JOIN sys.availability_replicas ar
        ON drs.replica_id = ar.replica_id
    WHERE ar.replica_server_name = N'sql-dr-west'
        AND drs.is_local = 0
        AND drs.log_send_queue_size > @max_queue_kb
        AND DATEDIFF(MINUTE, drs.last_commit_time, GETUTCDATE()) > 30
)
BEGIN
    RAISERROR('DR replica send queue exceeded limit – removing replica to release
log truncation', 16, 1);
    ALTER AVAILABILITY GROUP [AG_Production]
        REMOVE REPLICA ON N'sql-dr-west';
END
```

Cross-region AG with asynchronous DR replica:

```
-- Add async DR replica to existing synchronous AG
ALTER AVAILABILITY GROUP [AG_Production]
ADD REPLICA ON N'sql-dr-west' WITH (
    ENDPOINT_URL = N'TCP://sql-dr-west.us-west-2.internal:5022',
    AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT,      -- async – no latency impact on
primary
    FAILOVER_MODE = MANUAL,                        -- cross-region failover requires
human or explicit script
    SEEDING_MODE = AUTOMATIC,
    SECONDARY_ROLE (ALLOW_CONNECTIONS = READ_ONLY)
);

-- Monitor cross-region replica lag
SELECT
    ar.replica_server_name,
    drs.database_name,
    drs.synchronization_state_desc,
    drs.log_send_queue_size          AS unsent_log_kb,
    drs.redo_queue_size              AS unredone_log_kb,
    drs.log_send_rate                AS send_rate_kb_per_sec,
    drs.redo_rate                    AS redo_rate_kb_per_sec,
    DATEDIFF(SECOND, drs.last_commit_time, GETUTCDATE()) AS estimated_lag_seconds
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
WHERE ar.replica_server_name = N'sql-dr-west'
    AND drs.is_local = 0;
```

For cross-region autonomous failover in SQL Server, `FAILOVER_MODE = MANUAL` means the failover cluster will not automatically trigger the role change. The autonomous DR script must explicitly issue the failover command after passing the decision engine's data-loss acknowledgment gate:

```

-- Autonomous cross-region failover script (executed on DR replica)
-- Requires prior data-loss acknowledgment from decision engine

-- Step 1: Verify this is the DR replica and it is secondary
IF (SELECT role_desc FROM sys.dm_hadr_availability_replica_states WHERE is_local =
1) != 'SECONDARY'
BEGIN
    RAISERROR('This script must run on the secondary DR replica', 16, 1);
    RETURN;
END

-- Step 2: Record pre-failover state for audit
INSERT INTO dbo.dr_audit_log (
    event_time, event_type, ag_name, replica_name,
    lag_seconds_at_failover, initiated_by
)
SELECT
    GETUTCDATE(), 'CROSS_REGION_FAILOVER_INITIATED',
    ag.name, ar.replica_server_name,
    DATEDIFF(SECOND, drs.last_commit_time, GETUTCDATE()),
    SYSTEM_USER
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_database_replica_states drs ON ar.replica_id = drs.replica_id
WHERE drs.is_local = 1;

-- Step 3: Execute forced failover with data loss (explicit acknowledgment)
ALTER AVAILABILITY GROUP [AG_Production] FORCE_FAILOVER_ALLOW_DATA_LOSS;

-- Step 4: Update listener DNS to point to new primary (via external script call)
-- xp_cmdshell to route53-update.ps1 or equivalent
EXEC xp_cmdshell 'powershell.exe -File C:\DR\update-dns-listener.ps1 -AgName
AG_Production';

```

The DNS update step after cross-region failover is frequently overlooked. AG listeners are cluster-aware within a WSFC; across regions they require a separate DNS entry or global load balancer update. The autonomous pipeline must handle this as part of the runbook, not as an afterthought.

Observability and the DR Simulation Framework

An autonomous DR system that has never been tested is a liability. The simulation framework executes DR scenarios against the production topology on a schedule — not against a separate test environment — but with application traffic isolated to a shadow copy. This is analogous to chaos engineering but with a specific DR focus.

The DR simulation framework maintains a catalog of scenarios and their expected outcomes:

```

from dataclasses import dataclass
from typing import Callable, Awaitable

@dataclass
class DRScenario:
    name: str
    description: str
    inject_fn: Callable[[], Awaitable[None]] # fault injection
    expected_recovery_action: str # from RecoveryAction enum
    max_recovery_seconds: int # SLO for autonomous recovery
    validation_fn: Callable[[], Awaitable[bool]] # post-recovery check

DR_SCENARIOS = [
    DRScenario(
        name="primary_process_kill",
        description="Kill SQL Server process on primary – expect sync replica
promotion within 30s",
        inject_fn=kill_primary_process,
        expected_recovery_action="promote_sync_replica",
        max_recovery_seconds=30,
        validation_fn=validate_new_primary_accepting_writes
    ),
    DRScenario(
        name="network_partition_primary",
        description="Block all network traffic to/from primary – expect fencing and
promotion",
        inject_fn=apply_iptables_blackhole_to_primary,
        expected_recovery_action="fence_old_primary",
        max_recovery_seconds=60,
        validation_fn=validate_new_primary_accepting_writes
    ),
    DRScenario(
        name="backup_restore_integrity",
        description="Execute full PITR to 1 hour ago – validate data matches
expected state",
        inject_fn=no_injection_required,
        expected_recovery_action="restore_from_backup",
        max_recovery_seconds=1800,
        validation_fn=validate_pitr_data_integrity
    ),
    DRScenario(
        name="log_send_queue_bloat",
        description="Pause DR replica for 30 minutes – expect replica removal at
send queue limit",
        inject_fn=pause_dr_replica_replication,
        expected_recovery_action="replica_auto_removed",
        max_recovery_seconds=300,
        validation_fn=validate_replica_removed_and_log_truncated
    ),
]

```

Each simulation run produces a DR readiness report: which scenarios passed, which failed, what the actual recovery time was versus the SLO, and whether any circuit breakers opened during the run. This report feeds directly into the incident management system's DR readiness dashboard.

The observability layer for autonomous DR should expose the following metrics as a minimum:

Metric	Type	Purpose
<code>dr_failover_total</code>	Counter	Count of autonomous failovers by type
<code>dr_failover_duration_seconds</code>	Histogram	Time from failure detection to traffic restored
<code>dr_classification_confidence</code>	Gauge	Current classifier confidence on active signal
<code>dr_circuit_breaker_state</code>	Gauge	0=closed, 1=half-open, 2=open
<code>dr_backup_rpo_seconds</code>	Gauge	Age of most recently validated backup restore
<code>dr_replication_lag_bytes</code>	Gauge	Current replication lag per replica
<code>dr_simulation_last_pass_timestamp</code>	Gauge	Unix timestamp of last passing simulation run

When `dr_simulation_last_pass_timestamp` has not updated in more than 7 days, the system automatically files an incident. A DR system that hasn't been validated recently is not a DR system you can trust for autonomous execution.

Key Takeaways

- → **Failure classification precedes recovery action.** Treating all failures as "failover" produces wrong recovery paths; a voting-window classifier that waits 10–30 seconds for signal convergence prevents false positives and wrong-class recoveries while keeping RTO within SLO.
- → **RPO is an explicit gate in the decision engine, not an aspiration.** The recovery decision engine must compare actual replication lag bytes against the configured RPO threshold and refuse to promote replicas that exceed it without an explicit pre-authorized data-loss acknowledgment. This transforms RPO from a metric you measure after a disaster into a constraint you enforce before one occurs.

- → **Circuit breakers prevent autonomous systems from deepening incidents.** Three failed recovery attempts within a rolling window should open the circuit breaker, block further autonomous action, and escalate to a human — stopping the system from executing a cascade of failed promotions while the on-call engineer is responding.
- → **Continuous shadow restores are the only reliable backup validation.** Verifying backup files with checksums is necessary but not sufficient; only a full autonomous restore to an isolated instance, followed by integrity checks and RPO age measurement, can confirm that your backup chain is actually recoverable when the decision engine selects `RESTORE_FROM_BACKUP`.
- → **Post-recovery validation gates traffic re-admission.** The database process being reachable is not recovery; validated recovery requires confirming primary role, replication topology restoration, data integrity spot checks, and application-level smoke tests — and only after all four pass should the load balancer or DNS be updated to re-admit production traffic.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 19: Self-Tuning Security Policies

Security policy management in traditional database operations is a human-gated process: a developer requests access, a DBA reviews it, a security officer approves it, and the permission is provisioned—sometimes days later. At the ADE level, that model is untenable. Schemas evolve hourly, microservices spin up and down on Kubernetes, and threat surfaces shift in real time. This chapter builds the architecture for security systems that observe access patterns, detect policy drift, enforce least-privilege dynamically, and respond to anomalies without waiting for a human in the loop. The goal is a security posture that tightens itself—one that becomes more precise over time rather than accumulating stale grants and dormant superusers.

19.1 The Policy Drift Problem: Why Static Security Decays

Every production database accumulates security debt. A developer gets `db_owner` during an incident, the emergency never gets cleaned up. A service account that once owned a batch job retains `SELECT` on 40 tables after the job was decommissioned. An application team migrates off a schema but its grants remain for three years. This is policy drift—the divergence between the access your security posture intends and the access that actually exists.

Static security audits catch drift episodically. The autonomous approach catches it continuously by maintaining a **desired-state security manifest** and reconciling reality against it on a scheduled cadence, the same way Kubernetes reconciles pod specs against running containers.

The desired-state manifest is not a text document. It is a machine-readable specification—stored in a configuration repository, versioned in Git, and enforced by an automation layer. Each entry describes an entity (a role, a login, a service account), the objects it should have access to, the privilege level, and the expiry conditions.

```
# security-manifest.yaml (excerpt)
roles:
- name: reporting_svc
  type: application
  grants:
    - object: analytics.fact_sales
      privilege: SELECT
      valid_until: null          # permanent, reviewed quarterly
    - object: analytics.dim_customer
      privilege: SELECT
      valid_until: null
  deny:
    - privilege: INSERT
    - privilege: UPDATE
    - privilege: DELETE
  max_connection_limit: 10
  row_security_enabled: true
  policy: reporting_rls_v2

- name: etl_loader
  type: service
  grants:
    - object: staging.*
      privilege: INSERT, SELECT, TRUNCATE
      valid_until: "2025-06-30"
  superuser: false
  connection_limit: 5
```

The reconciliation engine reads this manifest, queries the live catalog, computes the delta, and emits either a remediation script or executes changes directly depending on the confidence level of the operation.

```

-- Drift detection against desired-state manifest
-- Manifest data is injected via a staging table populated by the automation layer

CREATE TABLE #manifest_grants (
    principal_name SYSNAME,
    schema_name SYSNAME,
    object_name SYSNAME,
    permission_name SYSNAME
);

INSERT INTO #manifest_grants VALUES
    ('reporting_svc', 'analytics', 'fact_sales', 'SELECT'),
    ('reporting_svc', 'analytics', 'dim_customer', 'SELECT'),
    ('etl_loader', 'staging', 'orders_stg', 'INSERT');

WITH live_grants AS (
    SELECT
        dp.name AS principal_name,
        OBJECT_SCHEMA_NAME(p.major_id) AS schema_name,
        OBJECT_NAME(p.major_id) AS object_name,
        p.permission_name
    FROM sys.database_permissions p
    JOIN sys.database_principals dp
        ON p.grantee_principal_id = dp.principal_id
    WHERE p.class = 1 -- object-level permissions
        AND dp.type IN ('S', 'U', 'R') -- SQL users, Windows users, roles
        AND dp.name NOT IN ('dbo', 'guest', 'INFORMATION_SCHEMA', 'sys')
),
drift AS (
    SELECT
        l.principal_name,
        l.schema_name,
        l.object_name,
        l.permission_name,
        'REVOKE_CANDIDATE' AS action
    FROM live_grants l
    LEFT JOIN #manifest_grants m
        ON l.principal_name = m.principal_name
        AND l.schema_name = m.schema_name
        AND l.object_name = m.object_name
        AND l.permission_name = m.permission_name
    WHERE m.principal_name IS NULL

    UNION ALL

    SELECT
        m.principal_name,
        m.schema_name,
        m.object_name,
        m.permission_name,
        'GRANT_MISSING' AS action
    FROM #manifest_grants m
    LEFT JOIN live_grants l

```

```

        ON l.principal_name = m.principal_name
        AND l.schema_name   = m.schema_name
        AND l.object_name    = m.object_name
        AND l.permission_name = m.permission_name
    WHERE l.principal_name IS NULL
)
SELECT * FROM drift ORDER BY action, principal_name, schema_name, object_name;

DROP TABLE #manifest_grants;

```

The reconciliation loop runs on a configurable interval—typically every 15 minutes for high-churn environments and every 4 hours for stable production systems. Findings are classified into three categories:

- → **Auto-remediate:** Revocation of a privilege not in the manifest for a service account with no active connections in the last 72 hours. Risk is low; the system executes the `REVOKE` and logs it.
- → **Stage for review:** New grants found that weren't in the manifest. The system opens a GitOps pull request against the manifest repository, tagging the security team.
- → **Alert immediately:** Superuser or `sysadmin` elevation detected on any account not explicitly designated as a DBA principal. Pages on-call regardless of time.

This three-tier decision model is the foundation of autonomous security: not everything is automated equally. The confidence threshold for automated action scales inversely with the blast radius of the change.

19.2 Dynamic Row-Level Security: Policies That Adapt to Context

Row-Level Security (RLS) in its basic form is a static filter: a policy is written once, attached to a table, and applied uniformly. At the ADE level, RLS becomes a dynamic control surface that adapts based on runtime context—the application role, the time of day, the sensitivity classification of the data being touched, and the geographic origin of the connection.

The architecture has three layers:

1. **A sensitivity classification store** — a metadata table that records per-column data classifications (PII, PHI, financial, internal, public).
2. **A context propagation mechanism** — session variables or application-name conventions that carry runtime claims (tenant ID, clearance level, access reason) into the policy evaluation.
3. **A policy generation engine** — automation that reads classification metadata and emits or updates RLS policies when classifications change.

The policy generation engine monitors `security.column_classifications` for changes and re-emits policies when the classification of a table's columns shifts. This is particularly important after schema migrations: a developer adds a `ssn` column to an existing table, the classification scanner (discussed in section 19.3) detects it, marks it `PII`, and the policy engine automatically appends a masking layer to any existing `SELECT` policies for that table without manual intervention.

```
-- SQL Server RLS with context-aware predicates
-- Uses SESSION_CONTEXT() for runtime claim propagation

CREATE SCHEMA security;
GO

-- Security predicate function reading tenant and clearance from session context
CREATE OR REPLACE FUNCTION security.fn_tenant_clearance_predicate
(
    @tenant_id      NVARCHAR(50),
    @sensitivity    INT
)
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS predicate_result
    WHERE
        @tenant_id = CAST(SESSION_CONTEXT(N'app_tenant_id') AS NVARCHAR(50))
        AND @sensitivity <= CAST(
            ISNULL(SESSION_CONTEXT(N'app_clearance_level'), 0)
            AS INT
        );
GO

-- Apply to accounts table
CREATE SECURITY POLICY security.accounts_tenant_policy
    ADD FILTER PREDICATE
        security.fn_tenant_clearance_predicate(tenant_id, sensitivity_tier)
    ON customer.accounts,
    ADD BLOCK PREDICATE
        security.fn_tenant_clearance_predicate(tenant_id, sensitivity_tier)
    ON customer.accounts AFTER INSERT
WITH (STATE = ON, SCHEMABINDING = ON);
GO

-- Application sets context at connection time via the connection wrapper
-- EXEC sp_set_session_context N'app_tenant_id', N'tenant_42';
-- EXEC sp_set_session_context N'app_clearance_level', 3;
```

The dynamic element comes from the policy versioning system. Each policy carries a version hash. When the policy generator computes a new policy body (because a classification changed or a new column was added), it compares hashes. If they diverge, it executes the policy replacement atomically within a transaction, logs the change to the `security.policy_audit_log` table, and pushes a notification to the security team's Slack channel via webhook.

This pattern eliminates the class of incident where a developer migrates a table, adds a sensitive column, and it is inadvertently exposed to roles that should not see it—because the system closes that gap automatically within the next reconciliation cycle.

19.3 Automated Privilege Escalation Detection and Response

Privilege escalation is the database attacker's primary goal and the defender's most time-sensitive concern. In autonomous security architecture, detection and response must operate on a timeline measured in seconds, not the hours that human-in-the-loop processes require.

The detection pipeline has four components operating as a continuous stream:

Signal collection: The database audit log, an Extended Events session, and connection metadata are consumed in real time. An Extended Events session targeting `sql_statement_completed`, `error_reported`, and `security_error` provides the raw stream.

Behavioral baseline: For each principal, a rolling 30-day baseline tracks: typical connection hours, typical source IPs, typical object access patterns, typical statement mix (% SELECT vs. DML vs. DDL). Deviation from baseline is the primary signal—not raw behavior in isolation.

Escalation fingerprints: A set of hard patterns that bypass the baseline and trigger immediate response. These include: any session executing `CREATE ROLE`, `ALTER ROLE ... SUPERUSER`, `sp_addsrvrolemember`, or `GRANT CONTROL` from an account not in the designated DBA principal list; any bulk extraction event (single query returning more than N rows from a classified table); any login from an IP not in the allowlist for a privileged account.

Response executor: Tiered automated response that acts proportionally—soft actions (session kill, lock account) for moderate signals; hard actions (revoke privileges, force password reset, network ACL block) for confirmed escalation events.

```

# escalation_detector.py
# Autonomous privilege escalation detection and response
# Runs as a Kubernetes sidecar alongside the SQL Server connection proxy

import pyodbc
import time
import json
import requests
from datetime import datetime, timedelta
from dataclasses import dataclass, field
from enum import Enum
from collections import defaultdict

class ThreatLevel(Enum):
    LOW = 1
    MODERATE = 2
    HIGH = 3
    CRITICAL = 4

@dataclass
class EscalationEvent:
    principal: str
    source_ip: str
    statement: str
    database: str
    detected_at: datetime
    threat_level: ThreatLevel
    signals: list[str] = field(default_factory=list)
    response_taken: str = ""

# Hard-pattern fingerprints: matched against normalized statement text
ESCALATION_PATTERNS = [
    ("SUPERUSER_GRANT", ThreatLevel.CRITICAL,
     ["alter role", "superuser", "createdb", "createrole"]),
    ("CONTROL_GRANT", ThreatLevel.CRITICAL,
     ["grant control"]),
    ("SYSADMIN_ADD", ThreatLevel.CRITICAL,
     ["sp_addsrvrolemember", "sysadmin"]),
    ("BULK_EXTRACT", ThreatLevel.HIGH,
     ["select", "into outfile", "copy to", "bulk"]),
    ("ROLE_CREATION", ThreatLevel.MODERATE,
     ["create role", "create user", "create login"]),
    ("PERMISSION_GRANT", ThreatLevel.MODERATE,
     ["grant", "all privileges", "with grant option"]),
]

DESIGNATED_DBA_PRINCIPALS = {"sa", "sa_dba", "platform_admin"}

class EscalationDetector:
    def __init__(self, mssql_dsn: str,
                  alert_webhook: str, auto_respond: bool = True):
        self.mssql_dsn = mssql_dsn
        self.alert_webhook = alert_webhook

```

```

self.auto_respond = auto_respond
self.baseline = defaultdict(lambda: {
    "typical_hours": set(range(8, 19)), # seeded; updated from 30-day stats
    "trusted_ips": set(),
    "max_rows_threshold": 50_000,
})
self.event_log: list[EscalationEvent] = []

def analyze_statement(self, principal: str, source_ip: str,
                      statement: str, database: str) -> EscalationEvent | None:
    normalized = statement.lower().strip()
    signals = []
    max_level = ThreatLevel.LOW

    # Hard-pattern matching
    for pattern_name, level, tokens in ESCALATION_PATTERNS:
        if all(t in normalized for t in tokens):
            if principal not in DESIGNATED_DBA_PRINCIPALS:
                signals.append(f"PATTERN:{pattern_name}")
                if level.value > max_level.value:
                    max_level = level

    # Behavioral anomaly: off-hours access
    current_hour = datetime.utcnow().hour
    user_baseline = self.baseline[principal]
    if current_hour not in user_baseline["typical_hours"]:
        signals.append("OFF_HOURS_ACCESS")
        if max_level.value < ThreatLevel.MODERATE.value:
            max_level = ThreatLevel.MODERATE

    # IP anomaly
    if (user_baseline["trusted_ips"]
        and source_ip not in user_baseline["trusted_ips"]):
        signals.append(f"UNTRUSTED_IP:{source_ip}")
        if max_level.value < ThreatLevel.MODERATE.value:
            max_level = ThreatLevel.MODERATE

    if not signals:
        return None

    return EscalationEvent(
        principal = principal,
        source_ip = source_ip,
        statement = statement[:500], # truncate for storage
        database = database,
        detected_at = datetime.utcnow(),
        threat_level = max_level,
        signals = signals,
    )

def respond(self, event: EscalationEvent, mssql_conn):
    """Execute tiered automated response proportional to threat level."""
    if event.threat_level == ThreatLevel.CRITICAL:

```



```

        # Terminate all sessions for principal immediately
        self._terminate_mssql_sessions(event.principal, mssql_conn)
        # Disable login
        self._disable_mssql_login(event.principal, mssql_conn)
        event.response_taken = "SESSION_TERMINATED|LOGIN_DISABLED"

    elif event.threat_level == ThreatLevel.HIGH:
        # Terminate active sessions but leave account enabled for investigation
        self._terminate_mssql_sessions(event.principal, mssql_conn)
        event.response_taken = "SESSION_TERMINATED"

    elif event.threat_level == ThreatLevel.MODERATE:
        # Log and alert; no automated kill unless pattern repeats
        event.response_taken = "ALERT_ONLY"

    self._send_alert(event)
    self.event_log.append(event)

def _terminate_mssql_sessions(self, principal: str, conn):
    cursor = conn.cursor()
    cursor.execute("""
        DECLARE @spid INT;
        DECLARE kill_cursor CURSOR FOR
            SELECT session_id FROM sys.dm_exec_sessions
            WHERE login_name = ? AND is_user_process = 1;
        OPEN kill_cursor;
        FETCH NEXT FROM kill_cursor INTO @spid;
        WHILE @@FETCH_STATUS = 0 BEGIN
            EXEC('KILL ' + @spid);
            FETCH NEXT FROM kill_cursor INTO @spid;
        END;
        CLOSE kill_cursor;
        DEALLOCATE kill_cursor;
    """, principal)
    conn.commit()

def _disable_mssql_login(self, principal: str, conn):
    cursor = conn.cursor()
    cursor.execute("ALTER LOGIN [?] DISABLE", principal)
    conn.commit()

def _send_alert(self, event: EscalationEvent):
    payload = {
        "text": (
            f"🚨 *Security Alert* [{event.threat_level.name}]\n"
            f"Principal: `{event.principal}` | IP: `{event.source_ip}`\n"
            f"Database: `{event.database}` | Time: {event.detected_at.isoformat()}\n"
            f"Signals: {' | '.join(event.signals)}\n"
            f"Response: `{event.response_taken}`\n"
            f"Statement: ``{event.statement[:200]}``"
        )
    }

```

```
try:
    requests.post(self.alert_webhook, json=payload, timeout=5)
except requests.RequestException:
    pass # Alert delivery failure must not block the response pipeline
```

The detector runs as a long-polling loop against the audit log tables, consuming new rows since its last checkpoint. In high-throughput environments, it runs as a Kafka consumer reading from a topic that the database audit plugin writes to directly—removing the polling latency and enabling sub-second response times.

19.4 Temporal Access Grants and Just-in-Time Privilege Provisioning

Permanent privilege grants are a security liability. An account that has `SELECT` on a financial table for an ad-hoc query from six months ago still holds that privilege today, long after the need expired. Just-in-Time (JIT) privilege provisioning eliminates standing privileges: access is granted for a specific duration tied to a specific work item, and it is revoked automatically when the window closes.

The JIT system has four components:

1. **Access request API** — service accounts or humans request access, specifying purpose, duration, and required privileges. Requests are evaluated against policy; approved requests generate a time-bounded grant.
2. **Grant executor** — issues the grant with a recorded expiry timestamp.
3. **Expiry enforcer** — a background worker that polls for expired grants and revokes them.
4. **Audit trail** — every grant and every revocation is recorded with request metadata, approver identity, and actual usage statistics.

```

-- JIT privilege management for SQL Server
CREATE SCHEMA jit;
GO

CREATE TABLE jit.access_requests (
    request_id        UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
    requested_by      SYSNAME           NOT NULL,
    principal_name    SYSNAME           NOT NULL,
    target_schema     SYSNAME           NOT NULL,
    target_object     SYSNAME           NOT NULL,
    privilege_type    NVARCHAR(50)     NOT NULL,
    reason            NVARCHAR(500)    NOT NULL,
    ticket_ref        NVARCHAR(100)    NULL,
    requested_at      DATETIME2         NOT NULL DEFAULT SYSUTCDATETIME(),
    approved_by       SYSNAME           NULL,
    approved_at       DATETIME2         NULL,
    expires_at        DATETIME2         NULL,
    status            NVARCHAR(20)     NOT NULL DEFAULT 'PENDING'
    CONSTRAINT chk_jit_status CHECK (
        status IN ('PENDING','APPROVED','ACTIVE','EXPIRED','REVOKED','DENIED')
    )
);

CREATE TABLE jit.active_grants (
    grant_id          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
    request_id        UNIQUEIDENTIFIER NOT NULL
        REFERENCES jit.access_requests(request_id),
    principal_name    SYSNAME           NOT NULL,
    target_schema     SYSNAME           NOT NULL,
    target_object     SYSNAME           NOT NULL,
    privilege_type    NVARCHAR(50)     NOT NULL,
    granted_at        DATETIME2         NOT NULL DEFAULT SYSUTCDATETIME(),
    expires_at        DATETIME2         NOT NULL,
    revoked_at        DATETIME2         NULL,
    revoke_reason     NVARCHAR(100)    NULL
);

CREATE INDEX idx_jit_active_grants_expiry
    ON jit.active_grants (expires_at)
    WHERE revoked_at IS NULL;
GO

-- Grant executor stored procedure
CREATE OR ALTER PROCEDURE jit.usp_execute_grant
    @request_id        UNIQUEIDENTIFIER,
    @approver          SYSNAME,
    @duration_mins     INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @principal_name SYSNAME,

```

```

        @target_schema SYSNAME,
        @target_object SYSNAME,
        @privilege_type NVARCHAR(50),
        @expires_at DATETIME2,
        @grant_sql NVARCHAR(500),
        @grant_id UNIQUEIDENTIFIER;

BEGIN TRANSACTION;

SELECT
    @principal_name = principal_name,
    @target_schema = target_schema,
    @target_object = target_object,
    @privilege_type = privilege_type
FROM jit.access_requests WITH (UPDLCK, ROWLOCK)
WHERE request_id = @request_id
    AND status = 'PENDING';

IF @principal_name IS NULL
BEGIN
    ROLLBACK;
    RAISERROR('Request not found or not in PENDING state.', 16, 1);
    RETURN;
END;

-- Validate object exists
IF OBJECT_ID(QUOTENAME(@target_schema) + '.' + QUOTENAME(@target_object)) IS
NULL
BEGIN
    ROLLBACK;
    RAISERROR('Target object does not exist.', 16, 1);
    RETURN;
END;

SET @expires_at = DATEADD(MINUTE, @duration_mins, SYSUTCDATETIME());

SET @grant_sql = N'GRANT ' + @privilege_type
    + N' ON ' + QUOTENAME(@target_schema) + N'.' + QUOTENAME(@target_object)
    + N' TO ' + QUOTENAME(@principal_name);
EXEC sp_executesql @grant_sql;

SET @grant_id = NEWID();
INSERT INTO jit.active_grants (
    grant_id, request_id, principal_name,
    target_schema, target_object, privilege_type, expires_at
)
VALUES (
    @grant_id, @request_id, @principal_name,
    @target_schema, @target_object, @privilege_type, @expires_at
);

UPDATE jit.access_requests
SET status = 'ACTIVE',

```

```

        approved_by = @approver,
        approved_at = SYSUTCDATETIME(),
        expires_at = @expires_at
WHERE request_id = @request_id;

COMMIT;

SELECT @grant_id AS grant_id, @expires_at AS expires_at;
END;
GO

-- Expiry enforcer: scheduled via SQL Server Agent every minute
CREATE OR ALTER PROCEDURE jit.usp_enforce_expiry
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @grant_id          UNIQUEIDENTIFIER,
        @principal_name    SYSNAME,
        @target_schema     SYSNAME,
        @target_object     SYSNAME,
        @privilege_type     NVARCHAR(50),
        @request_id        UNIQUEIDENTIFIER,
        @revoke_sql        NVARCHAR(500);

    DECLARE expiry_cursor CURSOR FAST_FORWARD FOR
        SELECT grant_id, principal_name, target_schema,
               target_object, privilege_type, request_id
        FROM    jit.active_grants
        WHERE   expires_at <= SYSUTCDATETIME()
               AND revoked_at IS NULL;

    OPEN expiry_cursor;
    FETCH NEXT FROM expiry_cursor INTO
        @grant_id, @principal_name, @target_schema,
        @target_object, @privilege_type, @request_id;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @revoke_sql = N'REVOKE ' + @privilege_type
            + N' ON ' + QUOTENAME(@target_schema) + N'.' + QUOTENAME(@target_object)
            + N' FROM ' + QUOTENAME(@principal_name);

        BEGIN TRY
            EXEC sp_executesql @revoke_sql;

            UPDATE jit.active_grants
            SET revoked_at = SYSUTCDATETIME(), revoke_reason = 'EXPIRED'
            WHERE grant_id = @grant_id;

            UPDATE jit.access_requests
            SET status = 'EXPIRED'

```

```

        WHERE request_id = @request_id;
    END TRY
    BEGIN CATCH
        UPDATE jit.active_grants
        SET revoke_reason = 'ERROR:' + ERROR_MESSAGE()
        WHERE grant_id = @grant_id;
    END CATCH;

    FETCH NEXT FROM expiry_cursor INTO
        @grant_id, @principal_name, @target_schema,
        @target_object, @privilege_type, @request_id;
END;

CLOSE expiry_cursor;
DEALLOCATE expiry_cursor;
END;
GO

```

The JIT system integrates with your service desk via webhook: when a developer submits an access request through their ticketing system, the webhook fires the approval API, the approver's `+approve` comment in Jira triggers the grant executor, and the grant is live within seconds. The expiry enforcer does not care whether the developer remembers to return access—it revokes on schedule regardless. This transforms privilege hygiene from a discipline that depends on human memory into a property that is structurally guaranteed.

A usage telemetry layer closes the feedback loop. When a JIT grant expires, the enforcer queries the audit log to determine whether the granted privilege was ever used. Grants that expire unused are flagged and fed back into the access request policy: if a particular role consistently requests access it never uses, the policy engine can automatically shorten the default grant window or require elevated justification for future requests.

```
sequenceDiagram
    participant Dev as Requester
    participant API as Access Request API
    participant App as Approver (ticket +approve)
    participant Exec as Grant Executor
    participant DB as Database
    participant Enf as Expiry Enforcer

    Dev->>API: Request access (object, duration, reason)
    API->>App: Route for approval
    App-->>API: Approve
    API->>Exec: execute_grant(request_id, approver, duration)
    Exec->>DB: GRANT ... (time-bounded)
    Exec->>DB: Insert active_grants row (expires_at)
    loop Every minute
        Enf->>DB: Find grants past expires_at
        Enf->>DB: REVOKE + mark revoked
    end
end
```

Just-in-time privilege lifecycle: no standing grant survives past its recorded expiry without the enforcer revoking it.

19.5 Automated Sensitive Data Discovery and Classification

The security controls in sections 19.1 through 19.4 all depend on knowing which data is sensitive. Manual classification is episodic and incomplete: it runs at project inception and is never revisited when columns are added, tables are renamed, or data patterns shift. Automated classification runs continuously, scanning for patterns that indicate sensitive data and updating the `security.column_classifications` table that drives RLS policy generation and masking rules.

The scanner operates in two modes:

Pattern-based classification uses regular expression matching and column-name heuristics to detect common sensitive data types. Column names like `ssn`, `social_security`, `credit_card`, `dob`, `date_of_birth`, `passport`, `tax_id` are strong signals. Data-value scanning uses regex against sampled column contents— `\d{3}-\d{2}-\d{4}` for SSNs, Luhn-validated 13–19 digit sequences for card numbers, and so on.

Lineage-based classification propagates labels through derived columns: if

`raw.customer.ssn` is classified `PII`, then a view column `SELECT ssn AS social_sec_num FROM raw.customer` should inherit that classification automatically.

```

# data_classifier.py
# Automated sensitive data discovery for SQL Server

import re
import pyodbc
from dataclasses import dataclass
from typing import Optional

@dataclass
class ColumnClassification:
    schema_name: str
    table_name: str
    column_name: str
    classification: str
    confidence: float # 0.0 - 1.0
    detection_method: str # 'COLUMN_NAME', 'DATA_PATTERN', 'LINEAGE'
    sample_match: Optional[str] = None

# Column-name heuristics: (pattern, classification, confidence)
COLUMN_NAME_RULES = [
    (re.compile(r'\b(ssn|social_sec|taxpayer_id|tin)\b', re.I),
    'PII', 0.95),
    (re.compile(r'\b(credit_card|card_number|pan|cc_num)\b', re.I),
    'FINANCIAL', 0.95),
    (re.compile(r'\b(dob|date_of_birth|birthdate|birth_date)\b', re.I),
    'PII', 0.90),
    (re.compile(r'\b(passport|passport_no|passport_number)\b', re.I),
    'PII', 0.95),
    (re.compile(r'\b(salary|compensation|bonus|wage)\b', re.I),
    'FINANCIAL', 0.85),
    (re.compile(r'\b(email|email_addr|email_address)\b', re.I),
    'PII', 0.80),
    (re.compile(r'\b(phone|mobile|cell|telephone)\b', re.I),
    'PII', 0.75),
    (re.compile(r'\b(address|street|postal_code|zip_code)\b', re.I),
    'PII', 0.70),
    (re.compile(r'\b(diagnosis|icd_code|mrn|patient_id)\b', re.I),
    'PHI', 0.90),
    (re.compile(r'\b(password|pwd|passwd|secret|api_key|token)\b', re.I),
    'INTERNAL', 0.95),
]

# Data-value regex patterns for content scanning
DATA_PATTERN_RULES = [
    (re.compile(r'^\d{3}-\d{2}-\d{4}$'), 'PII', 0.90, 'SSN'),
    (re.compile(r'^[0-9]{13,19}$'), 'FINANCIAL', 0.75,
    'PAN_CANDIDATE'),
    (re.compile(r'^[A-Z0-9]{6,9}$'), 'PII', 0.60,
    'PASSPORT_CANDIDATE'),
    (re.compile(r'^\d{4}[-/]\d{2}[-/]\d{2}$'), 'PII', 0.55,
    'DOB_CANDIDATE'),
    (re.compile(r'^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$'),
    'PII', 0.85,

```



```

'EMAIL'),
]

SAMPLE_SIZE = 200 # rows to sample per column for data-pattern scanning

def luhn_check(number: str) -> bool:
    """Validate a candidate card number using the Luhn algorithm."""
    digits = [int(d) for d in number if d.isdigit()]
    odd_digits = digits[-1::-2]
    even_digits = digits[-2::-2]
    total = sum(odd_digits)
    for d in even_digits:
        total += sum(divmod(d * 2, 10))
    return total % 10 == 0

class SqlServerClassifier:
    """Sensitive data discovery for SQL Server via pyodbc."""

    TEXT_TYPES = {'nvarchar', 'varchar', 'nchar', 'char', 'text', 'ntext'}

    def __init__(self, conn_str: str):
        self.conn_str = conn_str

    @staticmethod
    def _quote(name: str) -> str:
        """Bracket-quote an identifier (catalog values only)."""
        return '[' + name.replace(']', ']]') + ']'

    def _query(self, sql: str, params: tuple = ()):
        with pyodbc.connect(self.conn_str) as conn:
            cur = conn.cursor()
            cur.execute(sql, params)
            return cur.fetchall()

    def discover_columns(self) -> list[tuple]:
        """Return all user-table columns from the catalog views."""
        return self._query("""
            SELECT s.name, t.name, c.name, ty.name
            FROM sys.columns c
            JOIN sys.tables t ON t.object_id = c.object_id
            JOIN sys.schemas s ON s.schema_id = t.schema_id
            JOIN sys.types ty ON ty.user_type_id = c.user_type_id
            WHERE t.is_ms_shipped = 0
            AND s.name NOT IN ('security', 'jit')
            ORDER BY s.name, t.name, c.column_id
        """)

    def sample_column(self, schema: str, table: str, column: str,
                      data_type: str) -> list[str]:
        """Sample up to SAMPLE_SIZE non-null text values from a column."""
        if data_type.lower() not in self.TEXT_TYPES:
            return []
        rows = self._query(

```

```

        f"SELECT TOP ({SAMPLE_SIZE}) {self._quote(column)} "
        f"FROM {self._quote(schema)}.{self._quote(table)} "
        f"WHERE {self._quote(column)} IS NOT NULL"
    )
    return [row[0] for row in rows if row[0]]

def classify_column(self, schema: str, table: str, column: str,
                    data_type: str) -> Optional[ColumnClassification]:
    # 1. Column-name heuristics
    for pattern, classification, confidence in COLUMN_NAME_RULES:
        if pattern.search(column):
            return ColumnClassification(
                schema_name=schema, table_name=table,
                column_name=column, classification=classification,
                confidence=confidence, detection_method='COLUMN_NAME'
            )

    # 2. Data-value scanning
    samples = self.sample_column(schema, table, column, data_type)
    if not samples:
        return None

    match_counts: dict[str, int] = {}
    for value in samples:
        for pattern, classification, confidence, label in DATA_PATTERN_RULES:
            if pattern.match(value):
                # Upgrade PAN_CANDIDATE confidence with Luhn check
                if label == 'PAN_CANDIDATE' and not luhn_check(value):
                    continue
                match_counts[classification] = match_counts.get(classification,
0) + 1

    if not match_counts:
        return None

    top_class = max(match_counts, key=match_counts.__getitem__)
    hit_rate = match_counts[top_class] / len(samples)

    if hit_rate < 0.30: # require at least 30% of sample to match
        return None

    return ColumnClassification(
        schema_name=schema, table_name=table,
        column_name=column, classification=top_class,
        confidence=round(hit_rate * 0.9, 3),
        detection_method='DATA_PATTERN',
        sample_match=f"{match_counts[top_class]}/{len(samples)} samples matched"
    )

def run_scan(self) -> list[ColumnClassification]:
    results = []
    for schema, table, column, data_type in self.discover_columns():
        classification = self.classify_column(schema, table, column, data_type)

```

```

        if classification:
            results.append(classification)
    return results

def persist_classifications(self, results: list[ColumnClassification]):
    """Upsert findings into security.column_classifications."""
    with pyodbc.connect(self.conn_str) as conn:
        cur = conn.cursor()
        for r in results:
            # MERGE with a human-review guard: never overwrite a
            # classification a security officer set manually
            cur.execute("""
                MERGE security.column_classifications AS target
                USING (SELECT ? AS schema_name, ? AS table_name,
                             ? AS column_name, ? AS classification,
                             ? AS rls_required) AS src
                ON target.schema_name = src.schema_name
                AND target.table_name = src.table_name
                AND target.column_name = src.column_name
                WHEN MATCHED AND target.reviewed_by = 'auto_classifier' THEN
                    UPDATE SET classification = src.classification,
                               rls_required   = src.rls_required,
                               last_reviewed_at = SYSUTCDATETIME(),
                               reviewed_by    = 'auto_classifier'
                WHEN NOT MATCHED THEN
                    INSERT (schema_name, table_name, column_name,
                           classification, rls_required,
                           last_reviewed_at, reviewed_by)
                    VALUES (src.schema_name, src.table_name,
                             src.column_name, src.classification,
                             src.rls_required, SYSUTCDATETIME(),
                             'auto_classifier');
            """, (
                r.schema_name, r.table_name, r.column_name,
                r.classification,
                r.classification in ('PII', 'PHI') # rls_required
            ))
        conn.commit()

```

The scanner is scheduled to run nightly on all schemas, and also triggered on-demand whenever the schema change detector records a DDL event of type `ALTER TABLE ... ADD COLUMN` or `CREATE TABLE`. This event-driven invocation means that new columns with sensitive data are classified within minutes of creation, not at the next scheduled overnight run.

The `persist_classifications` method contains one design principle worth highlighting: it will not overwrite a classification that was set by a human reviewer. The `WHERE reviewed_by = 'auto_classifier'` predicate ensures that if a security officer has manually marked a column as `PUBLIC` (perhaps because what looks like a phone number is actually a product SKU), the automation respects that decision and does not reverse it on the next scan.

19.6 Anomalous Query Detection and Automatic Containment

Beyond explicit privilege escalation, the more subtle threat is queries that are technically authorized but behaviorally anomalous—a service account that normally reads 500 rows per minute suddenly running full table scans and exfiltrating millions of rows. The privilege is valid; the behavior is not.

Anomalous query detection in the ADE architecture is built on a statistical model of per-principal access behavior. For each principal, the system maintains a behavioral profile:

- → **Hourly row throughput:** rolling mean and standard deviation of rows returned per hour.
- → **Table access entropy:** how many distinct tables the principal typically touches per session. High entropy suddenly appearing in a low-entropy account is anomalous.
- → **Query fingerprint distribution:** the normalized hash fingerprint of queries typically executed. A new fingerprint type appearing at volume is a signal.
- → **Result set size distribution:** the typical percentile distribution of rows per query. A query returning 100× the 99th percentile is anomalous.

```

-- Behavioral anomaly scoring for SQL Server
-- Behavioral baseline and telemetry tables

CREATE TABLE behavior.principal_baselines (
    principal_name          SYSNAME          PRIMARY KEY,
    typical_conn_hours      NVARCHAR(MAX)    NOT NULL DEFAULT '[]', -- JSON array of
ints
    mean_rows_per_hour      DECIMAL(12,2)    NULL,
    stddev_rows_per_hour    DECIMAL(12,2)    NULL,
    mean_query_count         DECIMAL(10,2)    NULL,
    typical_table_entropy    DECIMAL(6,4)     NULL,
    p99_result_rows         BIGINT            NULL,
    baseline_start           DATETIME2        NULL,
    baseline_end             DATETIME2        NULL,
    last_updated             DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    sample_sessions         INT              NULL
);

CREATE TABLE behavior.session_telemetry (
    telemetry_id            BIGINT            IDENTITY(1,1) PRIMARY KEY,
    principal_name          SYSNAME          NOT NULL,
    session_start           DATETIME2        NOT NULL,
    session_end             DATETIME2        NULL,
    source_ip              NVARCHAR(45)      NULL,
    rows_returned           BIGINT            NOT NULL DEFAULT 0,
    rows_modified           BIGINT            NOT NULL DEFAULT 0,
    distinct_tables         INT              NOT NULL DEFAULT 0,
    query_count             INT              NOT NULL DEFAULT 0,
    max_single_result       BIGINT            NOT NULL DEFAULT 0,
    new_fingerprints        INT              NOT NULL DEFAULT 0
);

CREATE INDEX idx_telemetry_principal_time
    ON behavior.session_telemetry (principal_name, session_start);
GO

-- Anomaly scoring procedure
CREATE OR ALTER PROCEDURE behavior.usp_score_session
    @principal            SYSNAME,
    @rows_returned        BIGINT,
    @distinct_tables      INT,
    @max_single_result    BIGINT,
    @new_fingerprints     INT,
    @session_hour         INT,
    @anomaly_score        DECIMAL(6,3) OUTPUT,
    @breach_components    NVARCHAR(MAX) OUTPUT -- comma-delimited
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @mean_rows         DECIMAL(12,2),
        @stddev_rows       DECIMAL(12,2),

```

```

        @p99_rows          BIGINT,
        @typical_hours      NVARCHAR(MAX),
        @typical_entropy    DECIMAL(6,4),
        @z_rows             DECIMAL(10,4),
        @score              DECIMAL(6,3) = 0,
        @breaches           NVARCHAR(MAX) = '';

SELECT
    @mean_rows             = mean_rows_per_hour,
    @stddev_rows           = stddev_rows_per_hour,
    @p99_rows              = p99_result_rows,
    @typical_hours         = typical_conn_hours,
    @typical_entropy       = typical_table_entropy
FROM behavior.principal_baselines
WHERE principal_name = @principal;

IF @mean_rows IS NULL
BEGIN
    SET @anomaly_score      = 0;
    SET @breach_components = 'NO_BASELINE';
    RETURN;
END;

-- Z-score on row throughput
IF ISNULL(@stddev_rows, 0) > 0
BEGIN
    SET @z_rows = (@rows_returned - @mean_rows) / @stddev_rows;
    IF @z_rows > 3
    BEGIN
        SET @score = @score + CASE WHEN @z_rows / 3.0 < 4.0
                                    THEN @z_rows / 3.0
                                    ELSE 4.0 END;

        SET @breaches = @breaches
            + 'HIGH_ROW_ZSCORE:' + CAST(ROUND(@z_rows,1) AS NVARCHAR(20)) + ',';
    END;
END;

-- Result set size spike (10x the p99)
IF ISNULL(@p99_rows, 0) > 0
    AND @max_single_result > @p99_rows * 10
BEGIN
    SET @score = @score + 3.0;
    SET @breaches = @breaches
        + 'RESULT_SPIKE:' + CAST(@max_single_result AS NVARCHAR(20))
        + '_vs_p99_' + CAST(@p99_rows AS NVARCHAR(20)) + ',';
END;

-- Off-hours check using JSON array stored in typical_conn_hours
IF NOT EXISTS (
    SELECT 1 FROM OPENJSON(@typical_hours)
    WHERE CAST(value AS INT) = @session_hour
)
BEGIN

```

```

        SET @score      = @score + 1.0;
        SET @breaches = @breaches
            + 'OFF_HOURS:' + CAST(@session_hour AS NVARCHAR(5)) + ',';
    END;

    -- New fingerprints volume
    IF @new_fingerprints > 5
    BEGIN
        SET @score = @score
            + CASE WHEN @new_fingerprints / 5.0 < 3.0
                THEN @new_fingerprints / 5.0
                ELSE 3.0 END;
        SET @breaches = @breaches
            + 'NEW_FINGERPRINTS:' + CAST(@new_fingerprints AS NVARCHAR(10)) + ',';
    END;

    -- Table entropy spike
    IF ISNULL(@typical_entropy, 0) > 0
        AND @distinct_tables > (@typical_entropy * 50 + 10)
    BEGIN
        SET @score      = @score + 2.0;
        SET @breaches = @breaches
            + 'TABLE_ENTROPY_SPIKE:' + CAST(@distinct_tables AS NVARCHAR(10)) + ',';
    END;

    SET @anomaly_score      = ROUND(@score, 3);
    SET @breach_components = LEFT(@breaches, LEN(@breaches) - 1); -- trim trailing
comma
END;
GO

```

The scoring function produces a dimensionless anomaly score. Scores are thresholded into response tiers:

Score Range	Classification	Automated Response
0 – 2.0	Normal	Log only
2.0 – 4.0	Elevated	Alert security channel; increase audit verbosity for principal
4.0 – 6.0	Suspicious	Throttle connection rate; require re-authentication on next query batch
> 6.0	Critical	Terminate sessions; quarantine principal account; page on-call

The threshold values are starting points, not fixed constants. The autonomic layer tracks the false-positive rate for each principal and adjusts thresholds upward when alerts consistently resolve as benign—a form of per-principal calibration that reduces alert fatigue without permanently suppressing genuine signals.

19.7 Security Policy as Code: GitOps Integration and Auditability

All the mechanisms in this chapter—desired-state manifests, JIT grants, RLS policy versions, classification metadata—are meaningless from a compliance perspective if they cannot be audited. Self-tuning security must be at least as auditable as human-managed security, and ideally more so, because every automated change is machine-generated and therefore completely recordable.

The audit architecture has three layers:

Immutable event log: Every automated security action—grant, revoke, policy update, account lock, session termination—is written to an append-only audit table. In SQL Server this is enforced via a ledger table (SQL Server 2022+) or a temporal table with restricted permissions on the history table.

SQL Server (Ledger Table, SQL Server 2022+):

```
-- Ledger table: cryptographically verifiable audit trail
CREATE TABLE security.audit_log (
    event_id          BIGINT          IDENTITY(1,1),
    event_time        DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    event_type        NVARCHAR(50)    NOT NULL,
    actor             NVARCHAR(128)   NOT NULL,
    target_principal  SYSNAME         NULL,
    target_object     NVARCHAR(500)   NULL,
    action_taken      NVARCHAR(200)   NOT NULL,
    sql_executed      NVARCHAR(MAX)   NULL,
    request_ref       UNIQUEIDENTIFIER NULL,
    threat_level      NVARCHAR(20)    NULL,
    outcome           NVARCHAR(20)    NOT NULL,
    detail            NVARCHAR(MAX)   NULL -- JSON string
)
WITH (LEDGER = ON (APPEND_ONLY = ON));
GO
```

GitOps integration: The desired-state manifest lives in a Git repository. Every automated reconciliation that produces a policy change commits a diff back to the repository with a machine-authored commit message that includes the timestamp, the reconciliation run ID,

and a summary of changes. Human engineers can review the history with `git log --oneline security-manifest.yaml` and see the complete evolution of the security posture—who granted what, when automation changed it, and why.

The reconciliation engine uses a write-back pattern: after executing changes, it serializes the new actual state back to YAML and opens a pull request against the manifest repository if the actual state diverges from the last committed manifest in any way that was not part of the current reconciliation run. This prevents silent divergence: if someone manually executes a `GRANT` outside the normal flow, the system detects it within the next reconciliation cycle and surfaces it as an open PR for review.

Compliance reporting: The audit log, combined with the classification metadata and the JIT access history, can generate compliance evidence automatically. A SOC 2 access review that would normally require a DBA to compile a spreadsheet of all access grants, their justifications, and their current status can be answered with a single query against the audit tables:

```
-- Access evidence report: all active grants with business justification
SELECT
  ag.principal_name,
  ag.target_schema + '.' + ag.target_table      AS object_name,
  ag.privilege_type,
  ar.reason                                    AS business_justification,
  ar.ticket_ref,
  ar.approved_by,
  ar.approved_at,
  ag.expires_at,
  CASE
    WHEN ag.expires_at > SYSUTCDATETIME() THEN 'ACTIVE'
    ELSE 'EXPIRED'
  END                                          AS grant_status,
  cc.classification                          AS data_sensitivity
FROM jit.active_grants ag
JOIN jit.access_requests ar
  ON ar.request_id = ag.request_id
LEFT JOIN security.column_classifications cc
  ON cc.schema_name = ag.target_schema
  AND cc.table_name = ag.target_table
WHERE ag.revoked_at IS NULL
  OR ag.expires_at > DATEADD(DAY, -90, SYSUTCDATETIME()) -- include recently
  expired for review period
ORDER BY cc.classification DESC, ag.principal_name, object_name;
```

This is the compounding advantage of autonomous security: not only does the system enforce policy, but it generates its own compliance evidence as a natural byproduct of operation, eliminating the audit preparation overhead that consumes significant DBA time in conventional environments.

Key Takeaways

- → **Policy drift is structural, not accidental.** Static grants accumulate because the processes that create them have no automatic expiry or reconciliation mechanism. The autonomous remedy is a desired-state security manifest paired with a continuous reconciliation engine that computes and closes the gap between intended and actual privilege state.
- → **Row-Level Security becomes a living control surface when driven by a classification metadata layer.** Automating the pipeline from data classification through policy generation ensures that newly added sensitive columns are protected within the next reconciliation cycle, not whenever a developer remembers to update the policy manually.
- → **Just-in-Time privilege provisioning eliminates standing access as a structural property, not a discipline.** When access grants carry enforced expiry timestamps and the revocation is executed automatically by a background enforcer, privilege hygiene no longer depends on human memory or process compliance—it is guaranteed by the system's mechanics.
- → **Anomalous query detection must be behavioral, not purely signature-based.** A query is authorized or it is not; authorization checks alone cannot detect a compromised legitimate account. Behavioral profiling—tracking row throughput, result-set size distributions, fingerprint entropy, and access hours per principal—provides the statistical layer that surfaces anomalies even when the credential is valid.
- → **Autonomous security is only trustworthy when it is fully auditable.** Every automated grant, revocation, session termination, and policy update must be recorded in an immutable, machine-readable audit log that feeds compliance reporting directly, and every policy change must be traceable through a GitOps history. The auditability of the system is not an add-on—it is the mechanism that makes autonomous action acceptable to governance and regulatory stakeholders.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 20: Autonomous Threat Detection

Security in a self-managing database platform cannot depend on a human analyst reviewing logs at the end of a shift. By the time an intrusion is confirmed, a privilege escalation has propagated, or an exfiltration query has run to completion, the damage is already bounded only by how long the attack went undetected. This chapter builds the architecture for a database system that detects, classifies, and responds to threats autonomously — closing the gap between signal and action from hours to seconds. The patterns here cover behavioral baselining, anomaly scoring, automated containment, and integration with broader security pipelines, all grounded in what SQL Server actually exposes at the engine level.

20.1 The Threat Surface at the Engine Layer: What Autonomous Systems Must Monitor

Most security tooling lives above the database — WAF rules, network ACLs, SIEM ingestion of application logs. These are necessary but insufficient. An autonomous threat detection system must instrument the database engine itself because the most dangerous attacks succeed after the perimeter has already been crossed: stolen credentials, compromised application accounts, malicious insiders, or injected queries that bypass all upstream filtering.

The threat surface that matters for autonomous detection divides into four observable planes:

Authentication and session plane. Who is connecting, from where, at what time, using which authentication method. SQL Server surfaces this through Extended Events and the `sys.dm_exec_sessions` DMV. An autonomous system tracks not just raw connection counts but behavioral fingerprints: which application users connect from which hosts, at what hour distribution, with what typical session duration.

Query execution plane. What SQL is running. This is where SQL injection, data exfiltration, and reconnaissance manifest. SQL Server Query Store captures normalized query text with execution statistics. The autonomous system must detect structural anomalies — queries with unusual `UNION` counts, bulk `SELECT *` against sensitive tables, absence of `WHERE` clauses on large tables, and `EXECUTE / xp_cmdshell` patterns.

Privilege and schema mutation plane. DDL changes, role grants, ownership transfers.

These are low-frequency operations in production that almost never happen outside of planned change windows. Any `CREATE USER`, `GRANT`, `ALTER TABLE`, `DROP` operation occurring outside of a sanctioned window is a high-confidence threat signal.

Data volume plane. Row counts and byte volumes returned by queries. A support engineer querying a customer table and returning 50 rows is normal. The same query returning 2 million rows is not. SQL Server does not expose per-query row return counts in a single DMV, so this plane requires instrumentation at the connection pooler or proxy layer — SQL Server Audit, or an eBPF-based collector at the network layer.

The autonomous threat detection system must monitor all four planes simultaneously, correlate signals across them, and produce a threat score that triggers graduated responses. A single anomalous signal is noise. The same session showing unusual connection time, a novel query pattern, and a high row-return volume simultaneously is an incident.

20.2 Behavioral Baselineing: Building the Normal Model Autonomously

Detection requires a model of normal. Static rule systems — blacklists of bad query patterns, IP allowlists — fail against sophisticated attackers who operate within the envelope of legitimate access. Behavioral baselineing builds a statistical model of normal operation that evolves continuously as the workload changes.

The baseline must be per-identity, not aggregate. The behavior of `app_user` is not the behavior of `reporting_user`. The autonomous system maintains a profile per database principal covering:

- → Typical connection hour distribution (histogram by hour-of-day)
- → Source host/IP distribution
- → Average and 99th-percentile query execution time
- → Query template frequency (normalized Query Store query hash)
- → Tables accessed per session
- → Average rows examined and returned

```

-- Baseline table for per-login query behavior
CREATE TABLE SecurityBaseline.QueryProfile (
    LoginName      NVARCHAR(128) NOT NULL,
    QueryHash      BINARY(8)      NOT NULL,
    QueryText      NVARCHAR(MAX),
    AvgRowsReturned  FLOAT,
    StdDevRows      FLOAT,
    AvgDurationMs   FLOAT,
    StdDevDurationMs  FLOAT,
    CallsPerHour    FLOAT,
    FirstSeen       DATETIME2,
    LastUpdated     DATETIME2 DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_QueryProfile PRIMARY KEY (LoginName, QueryHash)
);

-- Refresh from Query Store
MERGE SecurityBaseline.QueryProfile AS tgt
USING (
    SELECT
        es.login_name,
        qs.query_hash,
        qt.query_sql_text,
        AVG(CAST(rs.avg_rowcount AS FLOAT))          AS avg_rows,
        STDEV(CAST(rs.avg_rowcount AS FLOAT))         AS stddev_rows,
        AVG(rs.avg_duration / 1000.0)                AS avg_duration_ms,
        STDEV(rs.avg_duration / 1000.0)              AS stddev_duration_ms,
        COUNT(*) * 1.0 /
            NULLIF(DATEDIFF(HOUR,
                MIN(rs.last_execution_time),
                MAX(rs.last_execution_time)), 0)      AS calls_per_hour
    FROM sys.query_store_query      qs
    JOIN sys.query_store_query_text qt ON qt.query_text_id = qs.query_text_id
    JOIN sys.query_store_plan      qp ON qp.query_id      = qs.query_id
    JOIN sys.query_store_runtime_stats rs ON rs.plan_id    = qp.plan_id
    -- join to current sessions is best-effort; use audit log for persistent mapping
    CROSS JOIN sys.dm_exec_sessions es
    WHERE es.is_user_process = 1
    GROUP BY es.login_name, qs.query_hash, qt.query_sql_text
) AS src
ON tgt.LoginName = src.login_name
AND tgt.QueryHash = src.query_hash
WHEN MATCHED THEN UPDATE SET
    AvgRowsReturned = src.avg_rows,
    StdDevRows      = src.stddev_rows,
    AvgDurationMs   = src.avg_duration_ms,
    StdDevDurationMs = src.stddev_duration_ms,
    CallsPerHour    = src.calls_per_hour,
    LastUpdated     = SYSUTCDATETIME()
WHEN NOT MATCHED THEN INSERT (
    LoginName, QueryHash, QueryText, AvgRowsReturned,
    StdDevRows, AvgDurationMs, StdDevDurationMs,
    CallsPerHour, FirstSeen, LastUpdated
) VALUES (

```

```
src.login_name, src.query_hash, src.query_sql_text,  
src.avg_rows, src.stddev_rows,  
src.avg_duration_ms, src.stddev_duration_ms,  
src.calls_per_hour, SYSUTCDATETIME(), SYSUTCDATETIME()  
);
```

The baseline refresh runs on a scheduled job — every 6 hours during stable periods, paused during incident response so the attack behavior does not contaminate the model. The autonomous system tracks `first_seen` to exclude newly observed query templates from scoring until they have accumulated at least 7 days of data; new templates are flagged as *novel* but not scored as threats until a minimum observation window passes.

Connection-hour histograms are stored separately as JSON arrays — one cell per hour of the week (168 cells) — with exponential decay weighting so that recent behavior has higher influence than patterns from 90 days ago. The decay parameter ($\alpha = 0.05$ per day) means the effective half-life is about 14 days, which tracks seasonal workload changes without being so volatile that a week of unusual traffic permanently distorts the model.

20.3 Real-Time Anomaly Scoring at the Session Level

With a behavioral baseline in place, the scoring engine evaluates every active session continuously. Scoring runs as a Python process that polls both engines every 30 seconds, computes Z-scores for observed session behavior against the baseline, and produces a composite threat score per session.

The scoring formula weights four dimensions:

- → `Z_time` : How far outside the normal hour distribution this connection falls
- → `Z_rows` : Standard deviations above mean row return for the query being executed
- → `Z_query_novelty` : Binary flag (0/1) — is this query template absent from the baseline?
- → `Z_volume` : Aggregate data scanned in this session vs. the user's historical session average

```

import pyodbc
import json
import math
import time
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Dict, Optional
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
logger = logging.getLogger("threat_scorer")

THREAT_THRESHOLD_WARN = 2.5 # Z-score composite: raise alert
THREAT_THRESHOLD_ACT = 4.0 # Z-score composite: autonomous action
DECAY_ALPHA = 0.05 # per-day exponential decay for baseline

@dataclass
class SessionProfile:
    session_id: str
    username: str
    client_addr: str
    query_id: Optional[int]
    rows_returned: int
    exec_ms: float
    hour_of_week: int # 0-167

@dataclass
class ThreatScore:
    session_id: str
    username: str
    composite_score: float
    components: Dict[str, float]
    triggered_at: datetime = field(default_factory=lambda:
datetime.now(timezone.utc))

def z_score(value: float, mean: float, stddev: float) -> float:
    """Return Z-score clamped to [0, 10]; negative deviations are not threats."""
    if stddev is None or stddev < 1e-9:
        return 0.0 if abs(value - mean) < 1e-9 else 5.0 # no variance → novel
    return max(0.0, (value - mean) / stddev)

def hour_distribution_score(hour: int, histogram: list[float]) -> float:
    """
    Score how anomalous this hour is relative to the user's historical distribution.
    Returns 0 (normal) to 10 (never seen at this hour).
    """
    if not histogram or len(histogram) != 168:
        return 2.0 # no data: mildly suspicious
    freq = histogram[hour]
    total = sum(histogram)
    if total < 1e-9:
        return 2.0

```

```

expected = freq / total
if expected < 0.001:    # less than 0.1% of sessions at this hour
    return 8.0
if expected < 0.01:
    return 4.0
return 0.0

def score_session(session: SessionProfile, baseline_row: Optional[dict]) ->
ThreatScore:
    components = {}

    if baseline_row is None:
        # Novel user or query – no baseline to compare against
        components["novelty"] = 6.0
        components["rows"] = 0.0
        components["exec_ms"] = 0.0
        components["time"] = 0.0
    else:
        histogram = json.loads(baseline_row["hour_histogram"] or "[]")
        components["time"] = hour_distribution_score(session.hour_of_week,
histogram)
        components["rows"] = z_score(
            session.rows_returned,
            baseline_row["avg_rows"] or 0,
            baseline_row["stddev_rows"] or 0
        )
        components["exec_ms"] = z_score(
            session.exec_ms,
            baseline_row["avg_exec_ms"] or 0,
            baseline_row["stddev_exec_ms"] or 0
        )
        components["novelty"] = 0.0

    # Weighted composite
    composite = (
        components.get("rows", 0) * 0.40 +
        components.get("time", 0) * 0.25 +
        components.get("exec_ms", 0) * 0.20 +
        components.get("novelty", 0) * 0.15
    )

    return ThreatScore(
        session_id=session.session_id,
        username=session.username,
        composite_score=round(composite, 3),
        components=components
    )

def poll_and_score_mssql(conn, baseline_cache: dict) -> list[ThreatScore]:
    scores = []
    now_dow = datetime.now(timezone.utc).weekday()    # 0=Mon
    now_hour = datetime.now(timezone.utc).hour
    hour_of_week = now_dow * 24 + now_hour

```



```

with conn.cursor() as cur:
    cur.execute("""
        SELECT
            CAST(r.session_id AS VARCHAR(20)) AS session_id,
            s.login_name                        AS username,
            s.client_net_address                AS client_addr,
            qs.query_hash                       AS query_id,
            0                                  AS rows_returned,    -- enriched
separately
            DATEDIFF(MILLISECOND, r.start_time, SYSUTCDATETIME())
                                   AS exec_ms

        FROM sys.dm_exec_requests r
        JOIN sys.dm_exec_sessions s
            ON s.session_id = r.session_id
        LEFT JOIN sys.dm_exec_query_stats qs
            ON qs.sql_handle = r.sql_handle
        WHERE s.is_user_process = 1
            AND r.session_id > 50
            AND s.login_name NOT IN ('sa', '##MS_PolicyEventProcessingLogin##')
    """)
    cols = [d[0] for d in cur.description]
    rows = cur.fetchall()

for row in rows:
    r = dict(zip(cols, row))
    session = SessionProfile(
        session_id    = r["session_id"],
        username      = r["username"],
        client_addr   = r["client_addr"] or "unknown",
        query_id      = r["query_id"],
        rows_returned = r["rows_returned"],
        exec_ms       = r["exec_ms"] or 0.0,
        hour_of_week  = hour_of_week
    )
    baseline = baseline_cache.get((r["username"], r["query_id"]))
    scores.append(score_session(session, baseline))

return scores

def dispatch_threat_response(score: ThreatScore, severity: str):
    logger.warning(
        "THREAT %s | session=%s user=%s score=%.2f components=%s",
        severity, score.session_id, score.username,
        score.composite_score, score.components
    )
    # Downstream: push to Prometheus Alertmanager, PagerDuty, SIEM
    # For ACT-level threats: trigger terminate_session() in 20.4

if __name__ == "__main__":
    conn = pyodbc.connect("DRIVER={ODBC Driver 18 for SQL
Server};SERVER=localhost;DATABASE=prod;UID=monitor;PWD=*****")
    baseline_cache: dict = {}    # populated from security.query_baseline on startup

```

```

while True:
    scores = poll_and_score_mssql(conn, baseline_cache)
    for s in scores:
        if s.composite_score >= THREAT_THRESHOLD_ACT:
            dispatch_threat_response(s, "ACT")
        elif s.composite_score >= THREAT_THRESHOLD_WARN:
            dispatch_threat_response(s, "WARN")
    time.sleep(30)

```

The scoring loop runs every 30 seconds. It pulls from `sys.dm_exec_requests` and `sys.dm_exec_sessions`. The composite score crosses two thresholds: `WARN` (2.5) triggers alerting to the SIEM and on-call channel; `ACT` (4.0) triggers autonomous containment actions described in the next section.

Threshold calibration matters. In the first 30 days after deployment on a new cluster, the system operates in *shadow mode* — it scores and logs but does not act. The false positive rate is measured as alerts that an operator manually classifies as non-threats. The ACT threshold is tuned upward until the false positive rate drops below 1 per week on steady-state workloads, then autonomous action is enabled.

20.4 Autonomous Containment: From Detection to Response Without Human Intervention

Detection without response is just expensive logging. The autonomous containment layer closes the loop: when a threat score crosses the ACT threshold, the system takes a graduated sequence of actions proportional to confidence and risk. The escalation ladder has four levels:

Level 1 — Rate-limit the session. For scores between 4.0 and 5.5, the system does not terminate the session but reduces its resource access. In SQL Server, this means placing the login in a Resource Governor pool capped at 10% CPU and a low memory grant ratio.

Level 2 — Terminate the active query. For scores between 5.5 and 7.0, the specific query is killed. The session remains active; the system monitors for recurrence. Two Level-2 terminations within a 5-minute window auto-escalate to Level 3.

Level 3 — Terminate the session. For scores above 7.0, or after Level-2 escalation, the session is forcibly terminated and the user's ability to establish new connections is suspended for a configurable quarantine window (default: 15 minutes).

Level 4 — Revoke and isolate. For the highest-confidence threats — novel query + off-hours connection + bulk data access simultaneously — the system revokes the role's login privilege entirely and creates a forensic snapshot of what was accessed.

```

graph TD
    A[Composite threat score] --> B{Score below 4.0?}
    B -->|yes| M[MONITORING – log only]
    B -->|no| C{Protected principal?}
    C -->|yes| H[Escalate to human]
    C -->|no| D{Score 4.0-5.5?}
    D -->|yes| L1[Level 1: rate-limit session]
    D -->|no| E{Score 5.5-7.0?}
    E -->|yes| L2[Level 2: terminate query]
    L2 -->|2 terminations in 5 min| L3
    E -->|no| L3[Level 3: terminate session]
    L3 --> Q{Quorum check: session over 60% of app pool?}
    Q -->|yes| H
    Q -->|no| F{Novel + off-hours + bulk access?}
    F -->|yes| L4[Level 4: revoke login + forensic snapshot]
    F -->|no| L3done[Session quarantined]

```

The containment ladder proportions response to confidence — quorum and protected-principal checks are hard gates that route to a human before Level 3/4 actions fire.

```

-- Level 1: Move login to restricted Resource Governor pool
-- (assumes pool 'ThreatContainment' is pre-configured with max CPU 10%)
ALTER RESOURCE GOVERNOR RECONFIGURE; -- classifier function re-evaluated on next
connection

-- Level 2: Kill specific request
KILL :session_id; -- session_id from sys.dm_exec_sessions

-- Level 3: Disable the login entirely
ALTER LOGIN [suspicious_user] DISABLE;

-- Terminate all existing sessions for this login
DECLARE @sql NVARCHAR(512);
SELECT @sql = STRING_AGG('KILL ' + CAST(session_id AS VARCHAR), '; ')
FROM sys.dm_exec_sessions
WHERE login_name = 'suspicious_user'
AND is_user_process = 1;

IF @sql IS NOT NULL
    EXEC sp_executesql @sql;

-- Level 4 forensic snapshot using Query Store
INSERT INTO SecurityForensics.IncidentQueryLog (
    IncidentId, LoginName, QueryHash, QueryText,
    ExecutionCount, LastExecutionTime, CapturedAt
)
SELECT
    @IncidentId,
    'suspicious_user',
    qs.query_hash,
    qt.query_sql_text,
    SUM(rs.count_executions),
    MAX(rs.last_execution_time),
    SYSUTCDATETIME()
FROM sys.query_store_query      qs
JOIN sys.query_store_query_text qt ON qt.query_text_id = qs.query_text_id
JOIN sys.query_store_plan      qp ON qp.query_id      = qs.query_id
JOIN sys.query_store_runtime_stats rs ON rs.plan_id    = qp.plan_id
WHERE qs.last_execution_time > DATEADD(HOUR, -1, SYSUTCDATETIME())
-- filter to this login via audit correlation in production
GROUP BY qs.query_hash, qt.query_sql_text;

```

The autonomous containment logic integrates with the scoring engine through a simple state machine. Each session has a containment state (`CLEAR` , `MONITORING` , `RATE_LIMITED` , `TERMINATED` , `QUARANTINED`). State transitions are append-only to an audit table; no state is silently overwritten. This creates an immutable incident timeline that satisfies SOC-2 and ISO 27001 audit requirements — the autonomous system's every action is traceable.

Two safety mechanisms prevent the autonomous system from taking catastrophic actions:

1. **Protected principals list.** The accounts used by the monitoring agent itself, replication users, backup agents, and named DBA accounts are permanently excluded from autonomous containment. A compromised monitoring credential is still dangerous, but it should trigger a manual escalation path, not a self-terminating loop where the security agent kills itself.

2. **Quorum check before Level 3 and Level 4 actions.** Before terminating a session or revoking a login, the containment engine queries how many other sessions from the same application are currently active and healthy. If the target session represents more than 60% of an application's connection pool, autonomous termination is suppressed and the action is escalated to a human-approved queue instead. This prevents a miscalibrated score from causing an application outage by killing the majority of its connections.

```

def can_act_automously(
    username: str,
    session_id: str,
    level: int,
    conn
) -> bool:
    """
    Returns True if autonomous action at the given level is safe.
    Level 3+ requires quorum check.
    """
    PROTECTED = {"monitor", "backup_agent", "dba_monitor", "sa"}
    if username in PROTECTED:
        logger.warning("Skipping autonomous action: protected principal %s",
            username)
        return False

    if level < 3:
        return True

    # Quorum check: how many sessions does this user account for?
    cur = conn.cursor()
    cur.execute("""
        SELECT
            SUM(CASE WHEN login_name = ? THEN 1 ELSE 0 END) AS user_sessions,
            COUNT(*) AS total_sessions
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1
    """, username)
    user_sessions, total_sessions = cur.fetchone()

    if total_sessions == 0:
        return True

    ratio = user_sessions / total_sessions
    if ratio > 0.60:
        logger.warning(
            "Quorum check failed for %s: %.0f%% of active sessions – "
            "escalating to human queue instead of autonomous Level %d action",
            username, ratio * 100, level
        )
        return False

    return True

```

The containment actions are idempotent by design. Each `KILL` runs inside a `TRY/CATCH` that treats an already-disconnected session as a no-op, and `ALTER LOGIN ... DISABLE` on an already-disabled login is a no-op. This means the system can retry actions on the 30-second polling loop without accumulating side effects.

20.5 DDL and Privilege Escalation Detection

Schema mutations and privilege changes are categorically different from query anomalies. They do not require statistical modeling — any `GRANT`, `CREATE USER`, `ALTER TABLE`, or `DROP` operation that occurs outside a defined change window is an unconditional high-severity signal. The autonomous system treats them with a separate detection path that bypasses the scoring model entirely.

In SQL Server, the same out-of-band delivery is achieved with a `Service Broker` conversation or an Event Notification that fires the alerting pipeline within milliseconds of the DDL completing — meaningfully faster than waiting for the next 30-second polling cycle.

Server-scoped DDL trigger:

```

-- Forensic table
CREATE TABLE SecurityAudit.DdlEvents (
    EventId          BIGINT IDENTITY PRIMARY KEY,
    EventType        NVARCHAR(100),
    ObjectName       NVARCHAR(256),
    SchemaName       NVARCHAR(128),
    LoginName        NVARCHAR(128),
    HostName         NVARCHAR(128),
    AppName          NVARCHAR(256),
    OccurredAt       DATETIME2 DEFAULT SYSUTCDATETIME(),
    InChangeWindow   BIT,
    CommandText      NVARCHAR(MAX)
);

-- Server-scoped DDL trigger
CREATE OR ALTER TRIGGER SecurityAudit.CaptureServerDDL
ON ALL SERVER
FOR CREATE_USER, ALTER_USER, DROP_USER,
    GRANT_DATABASE, REVOKE_DATABASE, DENY_DATABASE,
    CREATE_TABLE, ALTER_TABLE, DROP_TABLE,
    CREATE_PROCEDURE, ALTER_PROCEDURE, DROP_PROCEDURE,
    CREATE_LOGIN, ALTER_LOGIN, DROP_LOGIN
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @EventData XML      = EVENTDATA();
    DECLARE @InWindow   BIT     = 0;
    DECLARE @Hour       TINYINT = DATEPART(HOUR, SYSUTCDATETIME());
    DECLARE @Dow        TINYINT = DATEPART(WEEKDAY, SYSUTCDATETIME()); -- 7=Saturday

    -- Change window: Saturday 02:00-04:00 UTC
    IF @Dow = 7 AND @Hour BETWEEN 2 AND 3
        SET @InWindow = 1;

    INSERT INTO SecurityAudit.DdlEvents (
        EventType, ObjectName, SchemaName,
        LoginName, HostName, AppName,
        InChangeWindow, CommandText
    )
    SELECT
        @EventData.value('(/EVENT_INSTANCE/EventType)[1]',      'NVARCHAR(100)'),
        @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]',    'NVARCHAR(256)'),
        @EventData.value('(/EVENT_INSTANCE/SchemaName)[1]',    'NVARCHAR(128)'),
        @EventData.value('(/EVENT_INSTANCE/LoginName)[1]',     'NVARCHAR(128)'),
        HOST_NAME(),
        APP_NAME(),
        @InWindow,
        @EventData.value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]',
            'NVARCHAR(MAX)');

    -- Outside change window: write to alert queue for autonomous handler
    IF @InWindow = 0

```



```

BEGIN
    INSERT INTO SecurityAudit.AlertQueue (
        Severity, AlertType, Details, QueuedAt
    )
    VALUES (
        'HIGH',
        'DDL_OUTSIDE_WINDOW',
        @EventData.value('/EVENT_INSTANCE/EventType)[1]', 'NVARCHAR(100)')
        + ' ON '
        + @EventData.value('/EVENT_INSTANCE/ObjectName)[1]',
        'NVARCHAR(256)'),
        SYSUTCDATETIME()
    );
END
END;

```

Privilege escalation detection extends beyond DDL triggers. The scoring engine also runs a scheduled differential check every 5 minutes comparing current role memberships against the last-known-good snapshot:

Any newly appeared row in this differential is an unconditional alert. A role granted between scheduled windows without a corresponding change-management ticket in the external ITSM system (validated via an HTTP callback to the ITSM API) is escalated immediately to Level 4 autonomous containment.

20.6 Integration with External Security Pipelines

An autonomous detection system that operates entirely within the database perimeter is incomplete. Threats that originate at the application layer, succeed in part through the database, and then pivot to other infrastructure require correlation across multiple signal sources. The database-layer detection system must emit structured events that SIEM platforms, SOAR orchestrators, and zero-trust policy engines can consume.

The canonical output format is structured JSON over a reliable transport. The scoring engine pushes events to three destinations simultaneously:

1. **Prometheus Alertmanager** for operational on-call routing (low latency, fire-and-forget)
2. **An append-only event log table** in the database itself (durable, queryable for forensics)
3. **A message queue** (Kafka topic or AWS SQS queue) for SIEM ingestion (ordered, replayable)

The event schema follows the OCSF (Open Cybersecurity Schema Framework) database/datastore activity category to maximize compatibility with Splunk, Microsoft Sentinel, and Elastic SIEM without requiring custom parsers. The exact `class_uid` and `category_uid` values are versioned by the OCSF schema itself, so confirm them against the schema release you target before wiring up production ingestion — the values below are illustrative:

```

import json
import datetime
from dataclasses import asdict

OCSF_CLASS_UID_DATABASE_ACTIVITY = 5001 # placeholder – verify against your OCSF
schema version

def build_ocsf_event(score: ThreatScore, action_taken: str) -> dict:
    """
    Construct an OCSF-compatible Database Activity event from a threat score.
    """
    return {
        "class_uid": OCSF_CLASS_UID_DATABASE_ACTIVITY,
        "class_name": "Database Activity",
        "category_uid": 5,
        "category_name": "Discovery",
        "activity_id": 2, # OCSF: Query
        "severity_id": (
            5 if score.composite_score >= 7.0 else
            4 if score.composite_score >= 5.5 else
            3 if score.composite_score >= 4.0 else
            2
        ),
        "severity": (
            "Critical" if score.composite_score >= 7.0 else
            "High" if score.composite_score >= 5.5 else
            "Medium" if score.composite_score >= 4.0 else
            "Low"
        ),
        "status_id": 1,
        "status": "Success",
        "time": int(score.triggered_at.timestamp() * 1000),
        "message": (
            f"Anomalous database session detected for user {score.username}. "
            f"Composite threat score: {score.composite_score:.2f}."
        ),
        "actor": {
            "user": {
                "name": score.username,
                "uid_alt": score.session_id
            }
        },
        "metadata": {
            "version": "1.1.0",
            "product": {
                "name": "Autonomous DB Threat Detection",
                "version": "2.0"
            }
        },
        "unmapped": {
            "score_components": score.components,
            "action_taken": action_taken,
            "composite_score": score.composite_score
        }
    }

```

```

    }
}

def emit_to_kafka(event: dict, producer) -> None:
    producer.produce(
        topic="db-security-events",
        value=json.dumps(event).encode("utf-8"),
        key=event["actor"]["user"]["name"].encode("utf-8")
    )
    producer.poll(0)    # non-blocking flush

def emit_to_alert_table(event: dict, conn) -> None:
    with conn.cursor() as cur:
        cur.execute("""
            INSERT INTO security.ocsf_event_log
                (class_uid, severity_id, event_time, actor_user, raw_json)
            VALUES (%s, %s, to_timestamp(%s / 1000.0), %s, %s)
        """, (
            event["class_uid"],
            event["severity_id"],
            event["time"],
            event["actor"]["user"]["name"],
            json.dumps(event)
        ))
    conn.commit()

```

The SOAR integration layer consumes from the Kafka topic and applies playbook logic. Playbooks at this layer handle tasks the database engine itself cannot: creating a security incident ticket, notifying the user's manager, coordinating with network teams to geo-block the source IP, or triggering an AWS GuardDuty finding. The database-layer autonomous system handles only what it can do with certainty and speed — query termination, session disconnect, login suspension — and delegates everything requiring broader organizational context to the SOAR.

One integration pattern worth implementing explicitly is the **feedback loop**: when a SOAR playbook resolves an incident as a false positive, it sends a callback to the database security system's API endpoint. That callback marks the incident in the event log as a false positive and triggers a baseline recalibration for the affected user — slightly widening the anomaly thresholds to account for legitimate-but-unusual behavior. Without this feedback loop, an analyst who regularly runs large batch queries off-hours will generate persistent noise until the baseline naturally absorbs their behavior over 14 days. The feedback loop collapses that to seconds.

20.7 Forensic Preservation and Post-Incident Analysis

When autonomous containment fires, forensic preservation must happen simultaneously — not afterward. Evidence that might survive in memory or in SQL Server's plan cache is volatile and can be lost as soon as the session ends and cache entries age out.

The containment engine triggers forensic capture as part of every Level 3 and Level 4 action:

Comprehensive session forensic capture:

```

-- Capture all available evidence before terminating the session
CREATE OR ALTER PROCEDURE security.usp_capture_session_forensics
    @incident_id UNIQUEIDENTIFIER,
    @target_sid INT,
    @target_user SYSNAME
AS
BEGIN
    SET NOCOUNT ON;

    -- 1. Snapshot of current session state
    INSERT INTO security.forensic_sessions (
        incident_id, session_id, login_name, host_name,
        program_name, client_net_address, login_time,
        last_request_start_time, status, captured_at
    )
    SELECT
        @incident_id, s.session_id, s.login_name, s.host_name,
        s.program_name, s.client_net_address, s.login_time,
        r.start_time, s.status, SYSUTCDATETIME()
    FROM sys.dm_exec_sessions s
    LEFT JOIN sys.dm_exec_requests r
        ON r.session_id = s.session_id
    WHERE s.session_id = @target_sid;

    -- 2. Currently executing statement text
    INSERT INTO security.forensic_query_history (
        incident_id, login_name, session_id, query_text,
        captured_at
    )
    SELECT
        @incident_id, s.login_name, r.session_id,
        SUBSTRING(
            qt.text,
            (r.statement_start_offset / 2) + 1,
            (CASE r.statement_end_offset
                WHEN -1 THEN DATALENGTH(qt.text)
                ELSE r.statement_end_offset
            END - r.statement_start_offset) / 2 + 1
        ),
        SYSUTCDATETIME()
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
    CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) qt
    WHERE r.session_id = @target_sid;

    -- 3. Any locks held by this session
    INSERT INTO security.forensic_locks (
        incident_id, session_id, resource_type, resource_database,
        request_mode, request_status, captured_at
    )
    SELECT
        @incident_id, l.request_session_id, l.resource_type,
        DB_NAME(l.resource_database_id), l.request_mode,

```

```

        l.request_status, SYSUTCDATETIME()
FROM sys.dm_tran_locks l
WHERE l.request_session_id = @target_sid;

-- 4. Record the containment action
INSERT INTO security.containment_actions (
    incident_id, action_type, target_user,
    target_session_id, executed_at, executed_by
)
VALUES (
    @incident_id, 'LEVEL3_TERMINATE', @target_user,
    @target_sid, SYSUTCDATETIME(), SYSTEM_USER
);
END;

```

After forensic capture, the post-incident analysis pipeline runs automatically within 2 minutes using the preserved evidence. It generates a structured incident report covering: timeline of the session, all queries executed during the incident window, tables accessed, data volume estimates, and the sequence of anomaly signals that triggered the response. This report goes directly to the SIEM as an OCSF Finding event and into a dedicated incident tracking table.

The post-incident analysis also feeds a longer-horizon threat hunting query that examines whether the same anomaly pattern appeared in other sessions in the preceding 30 days — potentially revealing earlier reconnaissance that did not individually cross the detection threshold:

```

-- Threat hunt: find sessions with similar anomaly fingerprint
-- within 30-day lookback window
DECLARE @confirmed_threat_user SYSNAME = N'compromised_svc';

SELECT
    el.actor_user,
    el.event_time,
    el.severity_id,
    JSON_VALUE(el.raw_json, '$.unmapped.score_components') AS components,
    JSON_VALUE(el.raw_json, '$.unmapped.composite_score') AS score
FROM security.ocsf_event_log el
WHERE el.event_time > DATEADD(DAY, -30, SYSUTCDATETIME())
    AND el.class_uid = 5001
    AND TRY_CAST(
        JSON_VALUE(el.raw_json, '$.unmapped.score_components.rows')
        AS FLOAT) > 2.0      -- similar row-volume anomaly
    AND el.actor_user <> @confirmed_threat_user
ORDER BY el.event_time DESC;

```

This pattern surfaces related actors that may be part of the same campaign — for instance, multiple application accounts compromised in a credential stuffing attack, each operating below the individual detection threshold but showing the same behavioral fingerprint.

Key Takeaways

- → **Autonomous threat detection requires simultaneous instrumentation of all four observable planes** — authentication, query execution, schema mutation, and data volume — because attackers who have already passed the perimeter leave signals that only become conclusive when correlated across multiple dimensions. A single-plane monitoring approach produces both excessive noise and dangerous blind spots.
- → **Behavioral baselining must be per-identity and decay-weighted**, not aggregate. A baseline that treats all users as a single population will miss insider threats and account compromise entirely. Exponential decay weighting ensures the model tracks genuine workload evolution without permanently absorbing attack behavior into the normal profile.
- → **The scoring-to-action pipeline must be graduated and reversible**. A binary detect-or-ignore system either generates too many false positives to remain credible or sets thresholds so high that real threats pass unimpeded. The four-level containment ladder — rate-limit, query kill, session terminate, login revoke — proportions the response to confidence, and protected principal lists with quorum checks prevent autonomous actions from causing application outages worse than the attack itself.
- → **DDL and privilege escalation events bypass the statistical scoring model entirely**. Schema changes and role grants in production outside of defined change windows are unconditional high-severity signals. Event triggers and server-scoped DDL triggers fire synchronously within the transaction, providing sub-second detection latency that polling-based approaches cannot match.
- → **Forensic preservation and SIEM emission must be atomic with the containment action**, not deferred. Evidence in plan caches, session state, and lock tables is volatile. Capturing it after termination means capturing nothing. Integrating forensics into the containment procedure, combined with OCSF-formatted event emission to a durable message queue, ensures that autonomous speed does not come at the cost of the post-incident visibility needed to understand the full scope of a threat.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 21: Policy-Driven Compliance Automation

Compliance frameworks — SOC 2, HIPAA, PCI-DSS, GDPR, ISO 27001 — share a common engineering problem: humans cannot reliably enforce policy at the speed and scale that modern database platforms demand. An autonomous compliance system doesn't wait for an audit cycle to discover that a staging database received production PII, that a service account accumulated excessive privileges over six months, or that an unencrypted backup landed in the wrong storage tier. It detects the drift, evaluates it against a machine-readable policy graph, and corrects or escalates the deviation without a ticket being opened. This chapter walks through how to build that system end-to-end — from encoding compliance rules as enforceable policy objects, to building the detection layer, to wiring autonomous remediation actions that leave a complete, auditor-readable record of every decision made.

From Compliance Checklists to Machine-Readable Policy Objects

The foundational shift in autonomous compliance is treating regulatory requirements not as PDF documents reviewed quarterly, but as versioned, structured objects that the platform can evaluate programmatically at any time. A policy object answers three questions: what condition must be true, what evidence proves it is true, and what action restores compliance when it is not.

The schema for a policy object should be stable enough to version in Git and expressive enough to generate both detection queries and remediation procedures from the same source.

```

# Policy object schema (Python dataclass, stored as YAML in Git)
from dataclasses import dataclass, field
from typing import List, Optional
from enum import Enum

class Severity(Enum):
    CRITICAL = "critical"
    HIGH      = "high"
    MEDIUM   = "medium"
    LOW       = "low"

class RemediationMode(Enum):
    AUTO      = "auto"          # system acts immediately
    APPROVE   = "approve"       # system proposes, human approves
    ALERT     = "alert"         # system only notifies

@dataclass
class CompliancePolicy:
    policy_id:      str
    framework:      str          # "PCI-DSS", "HIPAA", "SOC2", "GDPR"
    control_ref:    str          # e.g. "PCI-DSS 8.6.1"
    description:    str
    target_scope:   List[str]    # ["production", "staging-pii"]
    detection_query: str         # SQL or PromQL
    detection_platform: str      # "sqlserver", "prometheus"
    severity:       Severity
    remediation_mode: RemediationMode
    remediation_action: str      # function name in remediation registry
    evidence_query: str          # query that produces audit evidence
    sla_minutes:    int          # time to remediate before escalation
    owner:          str
    tags:           List[str] = field(default_factory=list)

```

A YAML representation of a real policy under PCI-DSS 8.6.1 (system account password management) might look like this:

```

policy_id: pci-8-6-1-sqlserver-default-logins
framework: PCI-DSS
control_ref: "PCI-DSS 8.6.1"
description: "No SQL Server login may retain a default, blank, or common password in production"
target_scope: ["production"]
detection_query: |
    SELECT name FROM sys.sql_logins
    WHERE is_disabled = 0
    AND (is_policy_checked = 0
        OR PWDCOMPARE(N'Password123', password_hash) = 1
        OR PWDCOMPARE(N'sa', password_hash) = 1)
detection_platform: sqlserver
severity: critical
remediation_mode: auto
remediation_action: rotate_default_password
evidence_query: |
    SELECT name, is_policy_checked, is_expiration_checked,
           SYSUTCDATETIME() as checked_at
    FROM sys.sql_logins WHERE is_disabled = 0
sla_minutes: 15
owner: platform-security@company.com
tags: ["pci", "authentication", "credentials"]

```

This structure means every compliance check is a first-class, testable artifact. It can be linted for completeness, loaded into a policy engine at startup, and every evaluation is traceable back to a specific version in your policy repository. When the auditor asks "how do you enforce PCI-DSS 8.6.1?", the answer is a Git commit hash.

The policy registry itself should live in a table structure that supports efficient querying by framework, scope, and severity — the compliance loop pulls active policies from this registry on each evaluation cycle.

```

-- Policy registry table
CREATE TABLE compliance_policies (
  policy_id          NVARCHAR(200) PRIMARY KEY,
  framework          NVARCHAR(50)  NOT NULL,
  control_ref        NVARCHAR(100) NOT NULL,
  description         NVARCHAR(MAX) NOT NULL,
  target_scope        NVARCHAR(MAX) NOT NULL, -- JSON array stored as string
  detection_query     NVARCHAR(MAX) NOT NULL,
  detection_platform  NVARCHAR(50)  NOT NULL
                                CHECK (detection_platform IN ('sqlserver','prometheus')),
  severity            NVARCHAR(20)  NOT NULL
                                CHECK (severity IN ('critical','high','medium','low')),
  remediation_mode    NVARCHAR(20)  NOT NULL
                                CHECK (remediation_mode IN ('auto','approve','alert')),
  remediation_action  NVARCHAR(200) NOT NULL,
  evidence_query      NVARCHAR(MAX) NOT NULL,
  sla_minutes         INT            NOT NULL DEFAULT 60,
  owner              NVARCHAR(200) NOT NULL,
  tags                NVARCHAR(MAX) NULL,      -- JSON array
  is_active           BIT            NOT NULL DEFAULT 1,
  version             INT            NOT NULL DEFAULT 1,
  last_modified       DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
  modified_by         NVARCHAR(200) NOT NULL DEFAULT SYSTEM_USER
);

CREATE TABLE compliance_findings (
  finding_id         BIGINT IDENTITY(1,1) PRIMARY KEY,
  policy_id          NVARCHAR(200) NOT NULL
                                REFERENCES compliance_policies (policy_id),
  detected_at        DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
  target_database     NVARCHAR(200)  NOT NULL,
  target_host        NVARCHAR(200)  NOT NULL,
  violation_detail    NVARCHAR(MAX)  NOT NULL, -- JSON
  severity           NVARCHAR(20)    NOT NULL,
  status             NVARCHAR(30)    NOT NULL DEFAULT 'open'
                                CHECK (status IN ('open','remediating','resolved',
                                                  'suppressed','escalated')),
  remediation_started DATETIMEOFFSET NULL,
  remediation_result  NVARCHAR(MAX)  NULL,      -- JSON
  resolved_at        DATETIMEOFFSET NULL,
  sla_deadline        AS (DATEADD(MINUTE, sla_minutes, detected_at)) PERSISTED,
  sla_minutes         INT            NOT NULL
);

CREATE INDEX IX_findings_status ON compliance_findings (status, detected_at DESC);
CREATE INDEX IX_findings_sla    ON compliance_findings (sla_deadline)
  WHERE status = 'open';

```

The separation of `detection_query` from `evidence_query` is intentional: the detection query is optimized for fast violation identification (it may return only violating rows), while the evidence query produces the full snapshot that goes into the audit report. Running the evidence query only at finding time and at resolution time keeps audit storage manageable without sacrificing completeness.

The Compliance Detection Loop: Continuous Evaluation Architecture

A compliance system that runs once per day is not autonomous — it is a scheduled report. An autonomous system evaluates policies on a cadence proportional to severity: critical policies scan every one to five minutes, high-severity policies every fifteen, medium and low every hour. The detection loop is the engine that makes this happen.

The loop architecture has four components: the **policy scheduler** (determines which policies are due for evaluation), the **query executor** (runs detection queries against each registered database), the **finding processor** (compares results against open findings to detect new violations and resolutions), and the **action dispatcher** (routes new findings to the remediation engine or approval workflow based on remediation mode).

```
graph LR
  A[Policy Registry] --> B[Policy Scheduler]
  B -->|due by cadence| C[Query Executor]
  C -->|violations| D[Finding Processor]
  D -->|new finding, fingerprinted| E[Action Dispatcher]
  D -->|no longer violating| F[Auto-resolve finding]
  E -->|mode: auto| G[Remediation Engine]
  E -->|mode: approve| H[Approval Workflow]
  E -->|mode: alert| I[Notify only]
  G --> J[Remediation Audit Log]
  H -->|approved or SLA breach| G
```

The continuous compliance loop: cadence is severity-proportional, every finding is fingerprinted for idempotency, and remediation mode — not technical capability — decides whether the system acts, asks, or just informs.

```

# compliance_loop.py – core detection loop
import asyncio
import aiodbc # async ODBC pool for SQL Server targets
import logging
from datetime import datetime, timezone
from typing import Dict, List, Any
from policy_registry import PolicyRegistry
from finding_processor import FindingProcessor
from action_dispatcher import ActionDispatcher

logger = logging.getLogger("compliance_loop")

CADENCE_MAP = {
    "critical": 60,      # seconds
    "high": 900,
    "medium": 3600,
    "low": 14400,
}

class ComplianceLoop:
    def __init__(self, registry: PolicyRegistry,
                  processor: FindingProcessor,
                  dispatcher: ActionDispatcher,
                  target_databases: List[Dict]):
        self.registry = registry
        self.processor = processor
        self.dispatcher = dispatcher
        self.targets = target_databases
        self._last_run: Dict[str, datetime] = {}

    async def run_forever(self):
        logger.info("Compliance detection loop started")
        while True:
            policies = await self.registry.get_active_policies()
            now = datetime.now(timezone.utc)

            tasks = []
            for policy in policies:
                last = self._last_run.get(policy.policy_id)
                cadence = CADENCE_MAP[policy.severity]
                if last is None or (now - last).total_seconds() >= cadence:
                    for target in self.targets:
                        if self._scope_matches(policy.target_scope, target["env"]):
                            tasks.append(
                                self._evaluate_policy(policy, target, now)
                            )
                    self._last_run[policy.policy_id] = now

            if tasks:
                results = await asyncio.gather(*tasks, return_exceptions=True)
                for r in results:
                    if isinstance(r, Exception):
                        logger.error("Evaluation error: %s", r)

```

```

        await asyncio.sleep(10) # tight loop; cadence enforcement is internal

    async def _evaluate_policy(self, policy, target: Dict, eval_time: datetime):
        try:
            violations = await self._run_detection_query(policy, target)
            await self.processor.process(
                policy=policy,
                target=target,
                violations=violations,
                eval_time=eval_time
            )
        except Exception as e:
            logger.error("Policy %s failed on %s: %s",
                        policy.policy_id, target["host"], e)
            raise

    async def _run_detection_query(self, policy, target: Dict) -> List[Any]:
        if policy.detection_platform == "sqlserver":
            # pyodbc path, called synchronously in executor
            return await asyncio.get_event_loop().run_in_executor(
                None, self._run_sqlserver_query,
                policy.detection_query, target
            )
        return []

    @staticmethod
    def _scope_matches(scope_list: List[str], env: str) -> bool:
        return env in scope_list or "all" in scope_list

```

The finding processor is the component responsible for idempotency — it must not create duplicate findings for a violation that is already open, and it must automatically resolve findings when the detection query no longer returns the violating rows.

```

-- Finding processor upsert – SQL Server equivalent
-- Insert new finding if not already open
INSERT INTO compliance_findings
    (policy_id, target_database, target_host,
     violation_detail, severity, status, sla_minutes)
SELECT
    @policy_id, @target_db, @target_host,
    @violation_detail, @severity, 'open', @sla_minutes
WHERE NOT EXISTS (
    SELECT 1 FROM compliance_findings
    WHERE policy_id      = @policy_id
      AND target_database = @target_db
      AND target_host    = @target_host
      AND JSON_VALUE(violation_detail, '$.fingerprint')
        = JSON_VALUE(@violation_detail, '$.fingerprint')
      AND status IN ('open', 'remediating')
);

-- Auto-resolve cleared findings
UPDATE compliance_findings
SET     status      = 'resolved',
      resolved_at = SYSDATETIMEOFFSET(),
      remediation_result = JSON_MODIFY('{}', '$.resolution', 'auto_resolved')
WHERE  policy_id    = @policy_id
      AND target_database = @target_db
      AND target_host    = @target_host
      AND status IN ('open', 'remediating')
      AND JSON_VALUE(violation_detail, '$.fingerprint')
        NOT IN (SELECT value FROM OPENJSON(@current_fingerprints));

```

The fingerprint approach deserves emphasis. Each violation row produced by a detection query must be reduced to a stable string key (typically a hash of the identifying fields — a username, a table name, a setting name) before comparison. Without fingerprinting, every evaluation cycle would generate spurious new findings for violations already in flight, and auto-resolution logic would fire incorrectly. The fingerprint is computed in the finding processor before writing to the database, not inside the SQL.

SLA enforcement runs as a separate, lightweight background coroutine that wakes every sixty seconds and promotes any open finding past its `sla_deadline` to `escalated`, then fires the escalation webhook:


```

async def sla_watchdog(pool, escalation_webhook: str):
    """Promote overdue open findings to escalated and notify."""
    import aiohttp, json
    async with pool.acquire() as conn:
        overdue = await conn.fetch("""
            UPDATE compliance_findings
            SET    status = 'escalated'
            WHERE  status = 'open'
            AND    sla_deadline < NOW()
            RETURNING finding_id, policy_id, target_host,
                      severity, violation_detail, detected_at, sla_deadline
        """)
    if overdue:
        payload = {
            "alert": "compliance_sla_breach",
            "count": len(overdue),
            "findings": [dict(r) for r in overdue]
        }
        async with aiohttp.ClientSession() as session:
            await session.post(escalation_webhook,
                              json=payload,
                              timeout=aiohttp.ClientTimeout(total=10))
        logger.warning("Escalated %d overdue compliance findings", len(overdue))

```

Autonomous Remediation: What Gets Auto-Fixed and What Doesn't

Not all compliance violations should be autonomously remediated. The remediation mode embedded in each policy object encodes the decision: `auto` means the system corrects the violation immediately and records what it did; `approve` means it opens an approval request and waits up to the SLA deadline before either escalating or self-approving based on risk tier; `alert` means it only notifies.

The remediation registry maps action names to Python callables. Each callable receives the finding, the target database connection parameters, and the full policy object. It must return a `RemediationResult` that includes success/failure, the SQL or commands executed, and enough context that an auditor could reconstruct the action from the result alone.

The set of actions that are safe to automate fully (`auto` mode) across production systems is narrower than intuition suggests. Good candidates for full automation:

- → Revoking excessive privileges added to service accounts outside the change window
- → Disabling login for accounts inactive beyond the policy threshold
- → Forcing TLS requirement at the session level for connections detected without encryption
- → Rotating credentials that match default or empty password patterns
- → Re-enabling audit logging extensions that have been dropped or disabled
- → Reclassifying data sensitivity labels on newly discovered columns containing PII patterns

Actions that warrant `approve` mode even when technically automatable:

- → Dropping database objects (even temp tables containing unmasked PII)
- → Changing authentication configuration at the cluster level
- → Modifying role membership for roles used by application service accounts
- → Encryption-at-rest changes that require restart or key rotation

The following shows a full remediation action for one of the highest-frequency PCI-DSS violations: an account with excessive login privileges that was provisioned outside the IAM workflow.

```

-- Remediation: disable unauthorized login – SQL Server equivalent
BEGIN TRANSACTION;

-- Capture current state
INSERT INTO compliance_remediation_audit
    (action, login_name, old_state, new_state, executed_at, policy_id)
SELECT
    'DISABLE_LOGIN',
    @login_name,
    (SELECT is_disabled, create_date, modify_date
     FROM sys.server_principals
     WHERE name = @login_name
     FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
    '{"is_disabled": true}',
    SYSDATETIMEOFFSET(),
    @policy_id
FROM sys.server_principals
WHERE name = @login_name;

-- Execute remediation
DECLARE @sql NVARCHAR(500) =
    N'ALTER LOGIN ' + QUOTENAME(@login_name) + N' DISABLE';
EXEC sp_executesql @sql;

-- Kill active sessions for this login
DECLARE @session_id INT;
DECLARE session_cursor CURSOR FOR
    SELECT session_id FROM sys.dm_exec_sessions
    WHERE login_name = @login_name;

OPEN session_cursor;
FETCH NEXT FROM session_cursor INTO @session_id;
WHILE @@FETCH_STATUS = 0
BEGIN
    DECLARE @kill_sql NVARCHAR(50) = N'KILL ' + CAST(@session_id AS NVARCHAR);
    EXEC sp_executesql @kill_sql;
    FETCH NEXT FROM session_cursor INTO @session_id;
END;
CLOSE session_cursor;
DEALLOCATE session_cursor;

COMMIT TRANSACTION;

```

Notice that both remediation scripts write to a `compliance_remediation_audit` table *before* executing the change. This is non-negotiable for autonomous systems: the audit record must exist even if the remediation command partially fails. The finding's `remediation_result` column then receives the full result including the audit record ID, so any auditor can trace a finding through its detection, approval, remediation, and evidence chain as a linked set of records rather than isolated log lines.

The `compliance_remediation_audit` table schema supports this linkage:

```
CREATE TABLE compliance_remediation_audit (  
  audit_id          BIGINT IDENTITY(1,1) PRIMARY KEY,  
  finding_id        BIGINT REFERENCES compliance_findings (finding_id),  
  policy_id         NVARCHAR(200) NOT NULL  
                    REFERENCES compliance_policies (policy_id),  
  action            NVARCHAR(100) NOT NULL,  
  login_name        NVARCHAR(200) NULL,  
  old_state          NVARCHAR(MAX) NULL,    -- JSON  
  new_state          NVARCHAR(MAX) NULL,    -- JSON  
  executed_at        DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),  
  executed_by        NVARCHAR(200) NOT NULL DEFAULT SYSTEM_USER,  
  sql_executed       NVARCHAR(MAX) NULL,  
  success            BIT NOT NULL DEFAULT 1,  
  error_detail       NVARCHAR(MAX) NULL  
);  
  
CREATE INDEX IX_remediation_audit_finding  
  ON compliance_remediation_audit (finding_id);  
CREATE INDEX IX_remediation_audit_policy  
  ON compliance_remediation_audit (policy_id, executed_at DESC);
```

PII Detection and Classification as a Continuous Policy

Data classification — knowing which columns contain personally identifiable information — is not a one-time labeling exercise. Schemas evolve, new tables appear during feature deployments, and developers occasionally copy a production column into a reporting view without triggering any security review. Autonomous compliance treats classification as a standing policy that runs on every schema change event and on a periodic sweep of all unlabeled columns.

The detection approach combines two techniques: pattern matching on column names and statistical sampling of column contents. Name-based detection is fast and catches the obvious cases (`email`, `ssn`, `dob`, `credit_card_number`). Content-based detection using regex sampling catches columns with opaque names (`col_7`, `field_b`, `attr_value`) that happen to hold PII.

PII column name detection query:

```

-- Detection query for compliance policy: pii-unlabeled-columns
SELECT
    SCHEMA_NAME(t.schema_id)          AS schema_name,
    t.name                            AS table_name,
    c.name                            AS column_name,
    tp.name                           AS data_type,
    CONVERT(NVARCHAR(64),
        HASHBYTES('MD5',
            SCHEMA_NAME(t.schema_id) + '.' + t.name + '.' + c.name),
        2)                            AS fingerprint
FROM sys.columns c
JOIN sys.tables t  ON t.object_id = c.object_id
JOIN sys.types tp  ON tp.user_type_id = c.user_type_id
WHERE
    -- Name-pattern match
    c.name LIKE '%email%'
    OR c.name LIKE '%phone%'
    OR c.name LIKE '%mobile%'
    OR c.name LIKE '%ssn%'
    OR c.name LIKE '%social%'
    OR c.name LIKE '%tax_id%'
    OR c.name LIKE '%passport%'
    OR c.name LIKE '%dob%'
    OR c.name LIKE '%birth%'
    OR c.name LIKE '%credit_card%'
    OR c.name LIKE '%card_number%'
    OR c.name LIKE '%cvv%'
    OR c.name LIKE '%first_name%'
    OR c.name LIKE '%last_name%'
    OR c.name LIKE '%full_name%'
    OR c.name LIKE '%salary%'
    OR c.name LIKE '%account_num%'
    OR c.name LIKE '%iban%'
    -- Not already labeled via SQL Server Data Classification
    AND NOT EXISTS (
        SELECT 1 FROM sys.sensitivity_classifications sc
        WHERE sc.object_id = c.object_id
              AND sc.column_id = c.column_id
    );

```

Content-based sampling runs as a Python callable in the remediation registry rather than as a pure SQL query, because it needs to issue per-table sample queries dynamically and apply regex matching in Python where the regex library is significantly more capable:

```

# pii_content_sampler.py
import re
import aioredis
from typing import List, Dict, Any

PII_PATTERNS = {
    "email": re.compile(r'\b[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Z]{2,}\b',
re.I),
    "ssn": re.compile(r'\b\d{3}-\d{2}-\d{4}\b'),
    "credit_card": re.compile(r'\b(?:4\d{12}(?:\d{3})?|5[1-5]\d{14}|3[47]\d{13})\b'),
    "ip_address": re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b'),
    "phone_us": re.compile(r'\b(?:\+1[-.\s]?)(?:\d{3}\s)?[-.\s]\d{3}[-.\s]\d{4}\b'),
    "iban": re.compile(r'\b[A-Z]{2}\d{2}[A-Z0-9]{4}\d{7}([A-Z0-9]?){0,16}\b'),
}

async def sample_column_for_pii(
    pool, # aioredis.Pool
    schema: str,
    table: str,
    column: str,
    sample_size: int = 200
) -> List[str]:
    """
    Return list of PII type names detected in a random column sample.
    Returns empty list if no PII patterns found.
    """
    try:
        async with pool.acquire() as conn:
            async with conn.cursor() as cur:
                await cur.execute(
                    f"SELECT TOP (?) CAST([{column}] AS NVARCHAR(MAX)) "
                    f"FROM [{schema}].[{table}] TABLESAMPLE SYSTEM (5)",
                    sample_size
                )
                rows = await cur.fetchall()
    except Exception:
        # Column may not be castable to NVARCHAR(MAX) or table may be empty
        return []

    values = [r[0] for r in rows if r[0] is not None]
    if not values:
        return []

    detected = []
    for pii_type, pattern in PII_PATTERNS.items():
        match_count = sum(1 for v in values if pattern.search(v))
        # Require at least 3% match rate to avoid false positives on
        # columns that happen to contain one incidental matching string
        if match_count / len(values) >= 0.03:

```

```
        detected.append(pii_type)
    return detected
```

When the classification policy finds an unlabeled PII column, the remediation action (in `approve` mode for schema-level changes) applies the appropriate sensitivity label and writes a classification record:

```
-- Apply sensitivity label via built-in Data Classification
ADD SENSITIVITY CLASSIFICATION TO
    [schema_name].[table_name].[column_name]
WITH (
    LABEL          = 'Highly Confidential',
    INFORMATION_TYPE = 'Personal',
    RANK           = HIGH
);

-- Write to classification registry
MERGE data_classification_registry AS tgt
USING (VALUES (@schema_name, @table_name, @column_name,
               @pii_type, 'high', @detection_method, SYSDATETIMEOFFSET(),
               @policy_id))
    AS src (schema_name, table_name, column_name,
            pii_type, sensitivity, detection_method, classified_at, policy_id)
ON  tgt.schema_name = src.schema_name
AND tgt.table_name  = src.table_name
AND tgt.column_name = src.column_name
WHEN MATCHED THEN
    UPDATE SET pii_type          = src.pii_type,
               sensitivity       = src.sensitivity,
               detection_method   = src.detection_method,
               classified_at      = src.classified_at
WHEN NOT MATCHED THEN
    INSERT (schema_name, table_name, column_name,
            pii_type, sensitivity, detection_method, classified_at, policy_id)
VALUES (src.schema_name, src.table_name, src.column_name,
        src.pii_type, src.sensitivity, src.detection_method,
        src.classified_at, src.policy_id);
```

Privilege Drift Detection and the Least-Privilege Enforcement Loop

Privilege accumulation is among the most common compliance findings in real database environments. Service accounts gain `SELECT` on a new table for a one-time migration and the grant is never revoked. A developer is given temporary `DBA` access to investigate an incident and the elevated role persists for months. The autonomous privilege drift engine maintains a desired-state model of every role and periodically compares it against actual state, generating findings for every deviation.

The desired state model is encoded in a privilege baseline table: the set of grants that are explicitly authorized for each role on each database object, with the authorization event that approved them.

```
CREATE TABLE privilege_baseline (
    baseline_id      BIGINT IDENTITY(1,1) PRIMARY KEY,
    role_name        NVARCHAR(200) NOT NULL,
    object_schema    NVARCHAR(200) NOT NULL,
    object_name      NVARCHAR(200) NOT NULL,
    object_type      NVARCHAR(50)  NOT NULL CHECK (object_type IN
        ('table','view','procedure','function','schema','database')),
    privilege_type   NVARCHAR(50)  NOT NULL,
    granted_by       NVARCHAR(200) NOT NULL,
    approved_by      NVARCHAR(200) NOT NULL,
    valid_from       DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    valid_until      DATETIMEOFFSET NULL,
    is_active        BIT NOT NULL DEFAULT 1,
    UNIQUE (role_name, object_schema, object_name, privilege_type)
);

-- Drift detection: object permissions present in sys.database_permissions
-- but absent from the baseline
CREATE OR ALTER VIEW privilege_drift AS
SELECT
    dp.grantee_principal_id,
    USER_NAME(dp.grantee_principal_id) AS role_name,
    OBJECT_SCHEMA_NAME(dp.major_id)   AS object_schema,
    OBJECT_NAME(dp.major_id)          AS object_name,
    dp.type_desc                      AS privilege_type,
    CONVERT(NVARCHAR(64),
        HASHBYTES('MD5',
            USER_NAME(dp.grantee_principal_id)
            + ISNULL(OBJECT_SCHEMA_NAME(dp.major_id), '')
            + ISNULL(OBJECT_NAME(dp.major_id), '')
            + dp.type_desc), 2)        AS fingerprint
FROM sys.database_permissions dp
WHERE dp.state IN ('G', 'W') -- GRANT or GRANT WITH GRANT OPTION
    AND NOT EXISTS (
        SELECT 1 FROM privilege_baseline pb
        WHERE pb.role_name      = USER_NAME(dp.grantee_principal_id)
            AND pb.object_schema = ISNULL(OBJECT_SCHEMA_NAME(dp.major_id), '')
            AND pb.object_name   = ISNULL(OBJECT_NAME(dp.major_id), '')
            AND pb.privilege_type = dp.type_desc
            AND pb.is_active     = 1
            AND (pb.valid_until IS NULL OR pb.valid_until > SYSDATETIMEOFFSET())
    );
```


The remediation action for privilege drift in `auto` mode revokes the unauthorized grant and records the pre-remediation state. For `approve` mode, the system writes a pending approval record and the remediation is held until a human approves or the SLA deadline triggers escalation.

One practical detail: the revocation must be idempotent. If the detection query finds a privilege, the remediation revokes it, but the next evaluation cycle runs before the revocation propagates fully (rare but possible in federated environments), the remediation must not error on a double-revoke. SQL Server's `REVOKE` statement is safe to run when the privilege does not exist — it simply has no effect.

Generating Audit Evidence Packages

Compliance auditors do not inspect your findings table directly. They expect evidence packages: timestamped, signed collections of queries and their outputs that demonstrate a control was operating continuously during the audit period. The autonomous system must be able to generate these on demand.

An evidence package for a single control contains:

1. The policy definition as it existed during the audit period (from the policy registry with version history)
2. A timeline of every evaluation: timestamp, query executed, rows returned, evaluation duration
3. All findings raised during the period with their status progression
4. All remediation actions taken with pre- and post-state
5. Any SLA breaches and their escalation records
6. The evidence query output at the time of each finding

The evaluation log table supports items 2 and 6:

```

CREATE TABLE compliance_evaluation_log (
    log_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    policy_id       NVARCHAR(200) NOT NULL
                    REFERENCES compliance_policies (policy_id),
    evaluated_at    DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    target_database NVARCHAR(200) NOT NULL,
    target_host     NVARCHAR(200) NOT NULL,
    rows_returned   INT NOT NULL,
    violation_count INT NOT NULL,
    duration_ms     INT NOT NULL,
    evidence_snapshot NVARCHAR(MAX) NULL, -- JSON
    error_detail    NVARCHAR(MAX) NULL
);

CREATE INDEX IX_eval_log_policy_time
    ON compliance_evaluation_log (policy_id, evaluated_at DESC);

```

The evidence package generator queries these tables and assembles a structured report. For a typical SOC 2 audit covering a ninety-day period, this function produces a JSON document that maps directly onto the auditor's control testing worksheet:

```

# evidence_package_generator.py
import aioredis
import json
from datetime import datetime, timezone
from typing import Dict, Any

def _dicts(cur, rows):
    cols = [c[0] for c in cur.description]
    return [dict(zip(cols, r)) for r in rows]

async def generate_evidence_package(
    pool, # aioredis.Pool
    policy_id: str,
    period_start: datetime,
    period_end: datetime
) -> Dict[str, Any]:
    async with pool.acquire() as conn:
        async with conn.cursor() as cur:
            # 1. Policy definition (latest version in effect during period)
            await cur.execute(
                "SELECT * FROM compliance_policies WHERE policy_id = ?",
                policy_id)
            policy_rows = _dicts(cur, await cur.fetchall())
            policy = policy_rows[0] if policy_rows else None

            # 2. Evaluation timeline
            await cur.execute("""
                SELECT evaluated_at, target_host, rows_returned,
                       violation_count, duration_ms, evidence_snapshot, error_detail
                FROM compliance_evaluation_log
                WHERE policy_id = ?
                       AND evaluated_at BETWEEN ? AND ?
                ORDER BY evaluated_at
            """, policy_id, period_start, period_end)
            evaluations = _dicts(cur, await cur.fetchall())

            # 3. Findings during the period
            await cur.execute("""
                SELECT finding_id, detected_at, target_host, violation_detail,
                       status, remediation_started, resolved_at,
                       sla_deadline, remediation_result
                FROM compliance_findings
                WHERE policy_id = ?
                       AND detected_at BETWEEN ? AND ?
                ORDER BY detected_at
            """, policy_id, period_start, period_end)
            findings = _dicts(cur, await cur.fetchall())

            # 4. Remediation actions
            await cur.execute("""
                SELECT ra.audit_id, ra.finding_id, ra.action,
                       ra.old_state, ra.new_state, ra.executed_at,
                       ra.executed_by, ra.success, ra.error_detail
            """)

```

```

        FROM compliance_remediation_audit ra
        JOIN compliance_findings f ON f.finding_id = ra.finding_id
        WHERE f.policy_id = ?
        AND ra.executed_at BETWEEN ? AND ?
        ORDER BY ra.executed_at
        """ , policy_id, period_start, period_end)
    remediations = _dicts(cur, await cur.fetchall())

    # 5. SLA summary
    await cur.execute("""
        SELECT
            COUNT(*) AS
total_findings,
            SUM(CASE WHEN status = 'resolved' THEN 1 END) AS resolved,
            SUM(CASE WHEN status = 'escalated' THEN 1 END) AS escalated,
            AVG(CAST(DATEDIFF(MINUTE, detected_at, resolved_at)
                AS FLOAT)) AS
avg_resolution_minutes,
            MAX(DATEDIFF(MINUTE, detected_at, resolved_at)) AS
max_resolution_minutes
        FROM compliance_findings
        WHERE policy_id = ?
        AND detected_at BETWEEN ? AND ?
        """ , policy_id, period_start, period_end)
    sla_rows = _dicts(cur, await cur.fetchall())
    sla_stats = sla_rows[0] if sla_rows else {}

    return {
        "package_generated_at": datetime.now(timezone.utc).isoformat(),
        "audit_period": {
            "start": period_start.isoformat(),
            "end": period_end.isoformat()
        },
        "policy": policy,
        "evaluation_summary": {
            "total_evaluations": len(evaluations),
            "evaluations_with_violations": sum(
                1 for e in evaluations if e["violation_count"] > 0
            ),
            "evaluation_errors": sum(
                1 for e in evaluations if e["error_detail"]
            ),
        },
        "evaluations": evaluations,
        "findings": findings,
        "remediations": remediations,
        "sla_summary": sla_stats,
    }

```

The output of `generate_evidence_package` can be signed with a platform key and stored in immutable object storage (S3 Object Lock, Azure Blob immutability policies) the moment it is generated. This produces a tamper-evident audit trail that can be presented to auditors without involving any engineer in the evidence collection process — which itself satisfies an independence requirement in several audit frameworks.

Cross-Framework Policy Inheritance and Deduplication

A mature compliance program covers multiple frameworks simultaneously. PCI-DSS, HIPAA, and SOC 2 all require activity logging, encryption in transit, and access control reviews, but they phrase the requirement differently and assign different SLA expectations. Maintaining separate policy objects for each framework's version of "enable TLS" leads to three detection queries running against the same target, generating three findings for the same violation, and creating confusion in the remediation workflow.

The policy inheritance model resolves this by distinguishing between the canonical technical control and the framework citations that reference it. A single canonical policy handles detection and remediation; framework mappings record which controls satisfy which framework requirements.

```
CREATE TABLE policy_framework_mappings (  
    mapping_id      BIGINT IDENTITY(1,1) PRIMARY KEY,  
    policy_id       NVARCHAR(200) NOT NULL  
                   REFERENCES compliance_policies (policy_id),  
    framework       NVARCHAR(50)  NOT NULL,  
    control_ref     NVARCHAR(100) NOT NULL,  
    requirement_text NVARCHAR(MAX) NOT NULL,  
    UNIQUE (policy_id, framework, control_ref)  
);  
  
INSERT INTO policy_framework_mappings  
    (policy_id, framework, control_ref, requirement_text)  
VALUES  
    ('tls-required-ss', 'PCI-DSS', 'PCI-DSS 4.2.1',  
     'Strong cryptography used to safeguard PAN during transmission'),  
    ('tls-required-ss', 'HIPAA',   'HIPAA 164.312(e)(1)',  
     'Technical security measures to guard against unauthorized access'),  
    ('tls-required-ss', 'SOC2',    'CC6.7',  
     'Transmits data using encryption and authenticated protocols'),  
    ('tls-required-ss', 'ISO27001', 'ISO27001 A.10.1',  
     'Cryptographic controls applied to protect information in transit');
```

When the evidence package generator runs for a specific framework, it queries the mapping table to find all canonical policies that contribute to that framework, then assembles evidence across all of them. A single finding on `tls-required-pg` surfaces in the PCI-DSS,

HIPAA, SOC 2, and ISO 27001 evidence packages simultaneously, because the mapping table links them. Engineers receive one finding, one remediation task, and one resolution event — the multi-framework coverage is a reporting-layer concern, not a detection-layer concern.

Change Window Enforcement as a Policy Class

Compliance frameworks that require change management (SOC 2 CC8.1, ITIL-aligned controls, certain PCI-DSS requirements) impose restrictions on *when* schema and configuration changes may occur. The autonomous system enforces change windows not only by blocking out-of-window DDL (covered in Chapter 28), but by detecting DDL that *did* execute outside an approved window and raising it as a compliance finding.

The DDL audit detection query requires that DDL event logging is already active:

```
-- Detection query: DDL executed outside approved change windows
-- Assumes SQL Server Audit or an event trigger writes to ddl_audit_log
SELECT
    event_time,
    user_name,
    object_type,
    object_name,
    statement,
    CONVERT(NVARCHAR(64),
        HASHBYTES('MD5',
            CONVERT(NVARCHAR(50), event_time, 126)
            + user_name + object_name), 2) AS fingerprint
FROM ddl_audit_log
WHERE event_time >= DATEADD(HOUR, -1, SYSDATETIMEOFFSET())
    AND NOT (
        -- Approved change windows: weekdays 22:00–02:00 UTC
        DATEPART(WEEKDAY, event_time AT TIME ZONE 'UTC') BETWEEN 2 AND 6
        AND (
            DATEPART(HOUR, event_time AT TIME ZONE 'UTC') >= 22
            OR DATEPART(HOUR, event_time AT TIME ZONE 'UTC') < 2
        )
    )
    AND user_name NOT IN (
        SELECT service_account FROM approved_automation_accounts
        WHERE ddl_allowed_anytime = 1
    );
```

Because DDL outside change windows cannot typically be undone (the schema change already happened), the remediation mode for this policy class is almost always `alert` or `approve`, with the remediation action being a forced review ticket rather than a database operation. The compliance finding serves as the immutable record that the deviation occurred, regardless of whether the resulting schema state is ultimately accepted or rolled back.

Key Takeaways

- → **Policy objects are the unit of compliance work.** Encoding each regulatory requirement as a versioned, machine-readable object with a detection query, evidence query, remediation action, and SLA creates a complete, auditable compliance control that can be tested, deployed, and updated through standard engineering workflows rather than manual procedures.
- → **Idempotent fingerprinting prevents finding noise.** Every violation row produced by a detection query must be reduced to a stable fingerprint before writing to the findings table. Without fingerprinting, repeated evaluation cycles generate duplicate findings for active violations and misfire auto-resolution logic, destroying the reliability of the detection layer.
- → **Remediation mode is a risk classification, not a technical limitation.** The distinction between `auto`, `approve`, and `alert` is a deliberate risk decision encoded in the policy object — not a reflection of what the system is technically capable of doing. Privilege revocations and credential rotations can be automated safely; object drops and cluster-level configuration changes warrant human review even when they are technically straightforward to script.
- → **Cross-framework deduplication belongs in the mapping layer, not the detection layer.** A single canonical policy detecting one technical condition satisfies multiple regulatory frameworks simultaneously. Maintaining separate detection queries per framework multiplies operational noise without adding control coverage. The evidence package generator, not the detection loop, is responsible for rendering a finding against each applicable framework.
- → **Evidence packages must be generated autonomously and stored immutably at the moment of collection.** An audit evidence system that requires an engineer to manually gather query outputs, sign them, and submit them introduces both latency and integrity risk. Evidence generated programmatically from the evaluation log, signed with a platform key, and committed to immutable object storage the moment it is produced satisfies the independence and tamper-evidence requirements that modern compliance auditors expect from automated control systems.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 22: Autonomous Patch Management

Patch management sits at the intersection of security posture, availability guarantees, and operational risk — and in most organizations, it remains the most stubbornly manual process in the DBA's portfolio. Security advisories arrive unpredictably, maintenance windows are negotiated weeks in advance, and the gap between a CVE publication and a patched production cluster is measured in weeks or months. An autonomous patch management system collapses that gap by treating patching as a pipeline: continuous vulnerability ingestion, automated risk scoring, staged rollout with self-validation, and rollback triggered without human escalation. This chapter builds that pipeline end to end for SQL Server environments, with the goal of reducing the human decision surface to policy definition rather than per-patch approval.

22.1 Vulnerability Intelligence Pipelines: From CVE Feed to Patch Priority Score

The first obligation of an autonomous patch system is awareness — knowing what patches exist, what vulnerabilities they address, and how urgently each matters in your specific environment. Manual monitoring of vendor security pages does not scale. An ADE-level system subscribes to structured vulnerability feeds and enriches them with local context automatically.

SQL Server security advisories arrive through the Microsoft Security Response Center (MSRC) API at <https://api.msrc.microsoft.com/cvrf/v2.0/>, and the Microsoft SQL Server release notes track every Cumulative Update with its fixed KBs. The MSRC feed is machine-consumable and should be polled daily by an ingestion service that stores results in a patch intelligence table. Cross-referencing advisories with the installed build of each instance tells the system exactly which machines are exposed.


```

import httpx
import json
from datetime import datetime, timedelta
import pyodbc

MSRC_BASE = "https://api.msrc.microsoft.com/cvrf/v2.0/updates"

def fetch_msrc_sqlserver_updates(days_back: int = 1) -> list[dict]:
    """Fetch MSRC security updates mentioning SQL Server products."""
    resp = httpx.get(MSRC_BASE, timeout=30,
                     params={"$filter": "cvrfID ge '2026-Jan'"})
    resp.raise_for_status()
    updates = resp.json().get("value", [])
    # Keep only updates whose product tree includes SQL Server
    return [u for u in updates
            if "SQL Server" in json.dumps(u.get("ProductTree", {}))]

def compute_priority_score(update: dict) -> float:
    """
    Weighted score combining CVSS base score, exploit availability,
    and local asset exposure. Returns 0.0–100.0.
    """
    cvss_score = 0.0
    for metric in update.get("cvss", []):
        if metric.get("cvssSource") in ("NVD", "MSRC"):
            cvss_score = max(cvss_score, metric.get("baseScore", 0.0))

    # Exploit maturity multiplier – check EPSS score from api.first.org
    epss_score = fetch_epss_score(update["DocumentTracking"]["ID"]) # returns 0.0–
1.0

    # Asset exposure: how many instances run the affected build?
    affected_instances = count_affected_instances(update)

    exposure_multiplier = min(1.5, 1.0 + (affected_instances / 10.0))
    raw = (cvss_score * 10) * (1 + epss_score) * exposure_multiplier
    return min(100.0, raw)

def ingest_updates_to_catalog(conn, updates: list[dict]):
    cur = conn.cursor()
    for update in updates:
        update_id = update["DocumentTracking"]["ID"]
        score = compute_priority_score(update)
        cur.execute("""
            MERGE patch_catalog AS target
            USING (SELECT ? AS update_id) AS src
            ON target.update_id = src.update_id
            WHEN MATCHED THEN
                UPDATE SET priority_score = ?, ingested_at = SYSUTCDATETIME()
            WHEN NOT MATCHED THEN
                INSERT (update_id, db_engine, priority_score,
                        published_at, raw_payload, ingested_at)
                VALUES (?, 'sqlserver', ?, ?, ?, SYSUTCDATETIME());
        """)

```

```
        "", (update_id, score, update_id, score,
            update.get("DocumentTracking", {}).get("CurrentReleaseDate"),
            json.dumps(update)))
    conn.commit()
```

The `priority_score` formula is intentional — it does not treat all CVEs equally. A CVSS 9.8 with no known exploit and zero affected clusters scores lower than a CVSS 7.5 with active exploitation and fifty clusters on the vulnerable version. The Exploit Prediction Scoring System (EPSS) from FIRST.org provides daily updated exploit probability scores that feed this calculation.

Once scores are assigned, the system classifies patches into three tiers that drive downstream automation behavior:

- → **Tier 1 (score $\geq$ 80):** Deploy within 24 hours, bypass standard change freeze
- → **Tier 2 (score 50–79):** Deploy within next scheduled maintenance window ($\leq$ 7 days)
- → **Tier 3 (score $<$ 50):** Batch with next planned minor version upgrade

This tiering is stored alongside the CVE record and queried by the patch orchestrator to determine rollout timing without human input.

22.2 Version Inventory and Drift Detection Across Fleet Clusters

Autonomous patching requires authoritative, real-time knowledge of what version every cluster is running. Version drift — where different clusters diverge from the approved baseline — is itself a risk signal and a scheduling input. Without automated inventory, the patch pipeline cannot know which clusters need action.

```
-- Central inventory table in a dedicated management database
CREATE TABLE dbo.SqlInstanceInventory (
    InstanceId          NVARCHAR(200) NOT NULL PRIMARY KEY,
    Environment         NVARCHAR(50)  NOT NULL,
    Region              NVARCHAR(100),
    SqlVersion          NVARCHAR(50)  NOT NULL,
    BuildNumber         NVARCHAR(50)  NOT NULL,
    PatchLevel          NVARCHAR(50),  -- CU or SP designation
    ApprovedBaseline    NVARCHAR(50),
    DriftDetected       AS CAST(CASE WHEN BuildNumber != ApprovedBaseline
                                   THEN 1 ELSE 0 END AS BIT) PERSISTED,
    LastSeen            DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME()
);

-- Surfaces instances behind the approved Cumulative Update
SELECT
    InstanceId,
    Environment,
    BuildNumber      AS CurrentBuild,
    ApprovedBaseline,
    PatchLevel,
    DATEDIFF(DAY, LastSeen, SYSUTCDATETIME()) AS DaysSinceCheckin
FROM dbo.SqlInstanceInventory
WHERE DriftDetected = 1
ORDER BY Environment DESC, DaysSinceCheckin DESC;
```

Inventory agents run as lightweight daemons on each node. On SQL Server, a simple Python agent connects locally and extracts version data:

```

import pyodbc
import httpx
import socket

LOCAL_CONN = "DRIVER={ODBC Driver 18 for SQL
Server};SERVER=localhost;Trusted_Connection=yes;"
MGMT_CONN = "DRIVER={ODBC Driver 18 for SQL Server};SERVER=mgmt-
db;DATABASE=patch_catalog;UID=patch_agent;PWD=*****;"

def collect_and_report():
    # Connect to local instance
    local_conn = pyodbc.connect(LOCAL_CONN)
    cur = local_conn.cursor()

    cur.execute("""
        SELECT @@VERSION,
               SERVERPROPERTY('ProductVersion'),
               SERVERPROPERTY('ProductLevel'),
               SERVERPROPERTY('Edition')
    """)
    full_version, build, level, edition = cur.fetchone()

    # Report to central management DB
    mgmt_conn = pyodbc.connect(MGMT_CONN)
    mgmt_cur = mgmt_conn.cursor()
    mgmt_cur.execute("""
        MERGE dbo.SqlInstanceInventory AS target
        USING (SELECT ? AS InstanceId) AS src
        ON target.InstanceId = src.InstanceId
        WHEN MATCHED THEN
            UPDATE SET SqlVersion = ?, BuildNumber = ?,
                       PatchLevel = ?, LastSeen = SYSUTCDATETIME()
        WHEN NOT MATCHED THEN
            INSERT (InstanceId, Environment, Region, SqlVersion,
                   BuildNumber, PatchLevel, ApprovedBaseline, LastSeen)
            VALUES (?, 'production', 'us-east-1', ?, ?, ?, NULL, SYSUTCDATETIME());
    """, (
        socket.gethostname(),
        full_version, build, level,
        socket.gethostname(), full_version, build, level
    ))
    mgmt_conn.commit()

if __name__ == "__main__":
    collect_and_report()

```

On Kubernetes-hosted SQL Server (StatefulSets or Azure Arc-enabled SQL Server), the same inventory function is expressed as a CronJob that connects to the instance over the service DNS name and writes results to the central catalog. The autonomous patch system subscribes to instance lifecycle events rather than implementing its own polling.

Drift detection feeds into the patch scheduler: clusters with `minor_versions_behind >= 3` are escalated to Tier 2 regardless of the CVE score, because version gaps introduce an operational complexity cost beyond security risk alone. The system auto-generates a patch work order for these clusters and assigns them to the next available maintenance window.

22.3 Staged Rollout Architecture: Canary, Ring, and Fleet Propagation

The single most important property of autonomous patching is that it must never apply a patch to all clusters simultaneously. Fleet-wide simultaneous patching is how autonomous systems cause outages at scale. The rollout architecture implements concentric rings of trust, where each ring provides validation evidence for the next.

The ring model for a typical enterprise fleet:

Ring	Scope	Dwell Time	Pass Criteria	
-----	-----	-----	-----	
0	Dev/test clusters	2 hours	Zero crash loop, schema migrations pass	
1	Staging clusters (10% of traffic)	24 hours	Error rate < baseline P95, query latency ≤ +5%	
2	Production canary (1 cluster)	48 hours	All Ring 1 metrics + replication lag < 500ms	
3	Production batch (25% per wave)	72 hours per wave	SLA metrics within tolerance	

```
graph LR
    R0["Ring 0: Dev/test  
2h dwell"] -->|pass| R1["Ring 1: Staging 10%  
24h dwell"]
    R1 -->|pass| R2["Ring 2: Prod canary  
48h dwell"]
    R2 -->|pass| R3["Ring 3: Prod batch 25%/wave  
72h per wave"]
    R3 -->|pass| Done["Fully deployed"]
    R0 -->|fail| RB0["Rollback ring 0 + escalate"]
    R1 -->|fail| RB1["Rollback ring 1 + escalate"]
    R2 -->|fail| RB2["Rollback ring 2 + escalate"]
    R3 -->|fail| RB3["Rollback ring 3 + escalate"]
```

A failing ring halts propagation entirely — later rings never see a patch that hasn't earned trust from every ring before it.

The orchestrator is responsible for advancing or halting rings. It does not ask for human confirmation to advance; it only escalates when a ring fails validation. The core orchestration loop:

```

from enum import Enum
from dataclasses import dataclass
from datetime import datetime, timedelta
import time
import logging

class RingStatus(Enum):
    PENDING      = "pending"
    IN_FLIGHT    = "in_flight"
    VALIDATING    = "validating"
    PASSED       = "passed"
    FAILED       = "failed"
    ROLLED_BACK  = "rolled_back"

@dataclass
class PatchRing:
    ring_id: int
    clusters: list[str]
    dwell_hours: int
    status: RingStatus = RingStatus.PENDING
    started_at: datetime = None
    completed_at: datetime = None

class PatchOrchestrator:
    def __init__(self, patch_id: str, rings: list[PatchRing],
                 validator, patcher, notifier):
        self.patch_id = patch_id
        self.rings     = rings
        self.validator = validator
        self.patcher   = patcher
        self.notifier   = notifier

    def run(self):
        for ring in self.rings:
            logging.info(f"Patch {self.patch_id}: starting ring {ring.ring_id}")
            self._apply_ring(ring)

            dwell_deadline = ring.started_at + timedelta(hours=ring.dwell_hours)
            while datetime.utcnow() < dwell_deadline:
                time.sleep(300) # Check every 5 minutes during dwell
                result = self.validator.evaluate(ring.clusters)
                if result.status == "failing":
                    self._halt_and_rollback(ring, result.reason)
                    return # Stop propagation entirely

            # Final gate check after full dwell period
            final = self.validator.evaluate(ring.clusters)
            if final.status != "healthy":
                self._halt_and_rollback(ring, final.reason)
                return

            ring.status = RingStatus.PASSED
            ring.completed_at = datetime.utcnow()

```

```

        logging.info(f"Ring {ring.ring_id} passed. Advancing.")

        self.notifier.send(f"Patch {self.patch_id} fully deployed across all
rings.")

    def _apply_ring(self, ring: PatchRing):
        ring.status = RingStatus.IN_FLIGHT
        ring.started_at = datetime.utcnow()
        for cluster in ring.clusters:
            self.patcher.apply(cluster, self.patch_id)
        ring.status = RingStatus.VALIDATING

    def _halt_and_rollback(self, ring: PatchRing, reason: str):
        ring.status = RingStatus.FAILED
        logging.error(f"Ring {ring.ring_id} failed: {reason}. Initiating rollback.")
        for cluster in ring.clusters:
            self.patcher.rollback(cluster, self.patch_id)
        ring.status = RingStatus.ROLLED_BACK
        self.notifier.escalate(
            f"PATCH HALTED: {self.patch_id} at ring {ring.ring_id}. "
            f"Reason: {reason}. Rollback complete. Human review required."
        )

```

The `validator.evaluate()` call queries Prometheus for a set of pre-defined SLIs: connection error rate, query latency P99, AG synchronization health, and active connection count. For SQL Server Always On Availability Groups, the validator checks the DMV `sys.dm_hadr_database_replica_states` for synchronization health and redo queue depth.

```

-- Validation query run by the patch orchestrator against each SQL Server AG replica
SELECT
    ag.name                                AS availability_group,
    drs.database_id,
    DB_NAME(drs.database_id)              AS db_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.redo_queue_size                    AS redo_queue_kb,
    drs.log_send_queue_size                AS send_queue_kb,
    drs.secondary_lag_seconds,
    CASE WHEN drs.redo_queue_size > 51200 THEN 'WARN'
         WHEN drs.secondary_lag_seconds > 30 THEN 'WARN'
         WHEN drs.synchronization_health_desc != 'HEALTHY' THEN 'FAIL'
         ELSE 'OK'
    END                                    AS patch_gate_status
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_groups ag
    ON drs.group_id = ag.group_id
WHERE drs.is_local = 0; -- secondary replicas only

```

When `patch_gate_status` returns any `FAIL` row, the orchestrator halts the ring immediately. A `WARN` during the dwell window triggers increased monitoring frequency (60-second checks instead of 5-minute) but does not by itself halt propagation — it must persist for 30 consecutive minutes before triggering a halt. This hysteresis prevents transient load spikes from stopping legitimate patch rollouts.

22.4 Zero-Downtime Patch Execution for SQL Server Always On

Applying a patch to a production database cluster without downtime requires understanding the AG topology and executing the patch sequence in the right order: patch secondary replicas first, verify they re-synchronize, then move the primary role to a patched replica, and finally patch the former primary.

AG patch sequencer: The safest sequence is replica-first, then controlled primary failover. The autonomous patcher queries the AG topology from the DMVs and drives failover itself:


```

import pyodbc
import time

def get_ag_topology(conn) -> tuple[str, list[str]]:
    """Return (primary_replica, secondary_replicas) for the production AG."""
    cur = conn.cursor()
    cur.execute("""
        SELECT ar.replica_server_name, ars.role_desc
        FROM sys.availability_groups ag
        JOIN sys.availability_replicas ar
            ON ag.group_id = ar.group_id
        JOIN sys.dm_hadr_availability_replica_states ars
            ON ar.replica_id = ars.replica_id
        WHERE ag.name = 'AG_Production'
    """)
    primary, secondaries = None, []
    for replica_name, role in cur.fetchall():
        if role == 'PRIMARY':
            primary = replica_name
        else:
            secondaries.append(replica_name)
    return primary, secondaries

def wait_for_sync(conn, timeout_seconds: int = 600):
    deadline = time.time() + timeout_seconds
    while time.time() < deadline:
        cur = conn.cursor()
        cur.execute("""
            SELECT COUNT(*)
            FROM sys.dm_hadr_database_replica_states
            WHERE is_local = 0
            AND synchronization_health_desc != 'HEALTHY'
        """)
        if cur.fetchone()[0] == 0:
            return
        time.sleep(15)
    raise RuntimeError("AG did not return to HEALTHY within timeout")

def patch_ag_cluster(admin_conn_str: str, patch_fn):
    """
    patch_fn: callable that accepts a hostname and performs the OS-level
    Cumulative Update install (e.g., via Azure Update Manager or
    AWS Systems Manager) and restarts the SQL Server service.
    """
    conn = pyodbc.connect(admin_conn_str, autocommit=True)

    # Step 1: Identify topology
    primary, secondaries = get_ag_topology(conn)

    # Step 2: Patch secondary replicas first (no traffic disruption)
    for replica in secondaries:
        patch_fn(replica)
        time.sleep(30) # Brief settle period

```

```

        wait_for_sync(conn) # Verify replica rejoined and re-synchronized

# Step 3: Move primary role to a now-patched replica
cur = conn.cursor()
cur.execute("ALTER AVAILABILITY GROUP [AG_Production] FAILOVER")
time.sleep(15) # Allow role transition to complete

# Step 4: Verify the new primary is the patched replica
new_primary, _ = get_ag_topology(conn)
assert new_primary != primary, "Failover did not complete successfully"

# Step 5: Patch the old primary (now a secondary)
patch_fn(primary)
wait_for_sync(conn)

```

The actual `patch_fn` submits the Cumulative Update through the platform's update orchestration — Azure Update Manager, AWS Systems Manager Patch Manager, or an Ansible playbook — and restarts the SQL Server service on the target host. The restart is coordinated so the AG role is already off the node being patched, which keeps the WSFC quorum intact throughout.

Pre-patch gate: Before any patch sequence begins, the autonomous patcher verifies the AG is fully synchronized.

```

-- Pre-patch validation: verify AG is fully synchronized before beginning
SELECT
    ag.name,
    agl.dns_name           AS listener,
    ar.replica_server_name,
    ar.availability_mode_desc,
    ars.role_desc,
    ars.connected_state_desc,
    ars.synchronization_health_desc,
    ars.operational_state_desc
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.availability_group_listeners agl
    ON ag.group_id = agl.group_id
WHERE ars.synchronization_health_desc != 'HEALTHY'
    OR ars.connected_state_desc != 'CONNECTED';
-- Zero rows returned = safe to proceed with patch

```

If any rows are returned, the autonomous patcher aborts with an escalation rather than proceeding — patching an already-degraded AG risks data loss on failover. This hard gate runs before every patch execution, not just as a pre-flight check.

For SQL Server, the patch sequence is: patch synchronous secondary replicas → verify they come back online and re-synchronize → move primary role to a patched replica using `ALTER AVAILABILITY GROUP ... FAILOVER` → patch the former primary. The Windows Update orchestration can be handled through Azure Update Manager, AWS Systems Manager Patch Manager, or a Terraform-driven Ansible playbook — the autonomous patcher generates and submits the job rather than executing it inline.

One non-obvious edge case: a Cumulative Update can change engine behavior gated behind a higher database compatibility level, and post-patch the optimizer may choose different plans. The autonomous patcher includes a post-patch SQL execution step that refreshes statistics and validates consistency on each patched replica before it is allowed to take the primary role:

This runs as part of the post-patch validation phase and its output is logged to the patch audit trail.

22.5 Autonomous Rollback: Decision Logic, Trigger Conditions, and Recovery Verification

Rollback is where autonomous systems either earn trust or destroy it. A system that patches but cannot reliably roll back is more dangerous than no automation at all. The rollback subsystem must be opinionated, fast, and verifiable.

Rollback triggers fall into three categories:

Hard triggers — immediate rollback, no dwell, no override:

- → SQL Server service exits within 60 seconds of service restart (crash loop)
- → AG secondary redo queue growing monotonically for 10 minutes post-patch
- → Connection rejection rate exceeds 5% over a 2-minute window
- → SQL Server AG replica enters `NOT_SYNCHRONIZING` state post-patch

Soft triggers — rollback if condition persists beyond hysteresis window (30 minutes):

- → Query latency P99 elevated > 20% above pre-patch baseline
- → Lock wait events per second > 2×pre-patch baseline
- → Buffer hit ratio drops > 5 percentage points below pre-patch baseline
- → TempDB allocation failures per minute > 0 (SQL Server specific)

Informational triggers — logged, escalated to human, no automatic rollback:

- → New query plan regressions detected (via Query Store)
- → Checkpoint duration increase > 50%

- → Index fragmentation increase in tracked tables

The distinction between hard and soft triggers is intentional. Hard triggers represent conditions where continued operation is objectively unsafe — the database is either crashing, disconnecting clients, or losing replication. Soft triggers represent degradation that could be transient (a batch job, a sudden traffic spike) and need the hysteresis window to distinguish noise from signal.

The rollback state machine tracks which tier of trigger fired and records it in the patch audit log:

```

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from typing import Optional

class TriggerType(Enum):
    HARD = "hard"
    SOFT = "soft"
    NONE = "none"

@dataclass
class RollbackDecision:
    trigger_type: TriggerType
    trigger_reason: str
    fired_at: datetime
    should_rollback: bool
    escalate_human: bool

@dataclass
class SoftTriggerState:
    condition: str
    first_seen: datetime
    last_seen: datetime
    breach_count: int = 0
    HYSTERESIS_MINUTES: int = 30

    def is_sustained(self) -> bool:
        duration = (self.last_seen - self.first_seen).total_seconds() / 60
        return duration >= self.HYSTERESIS_MINUTES

class RollbackEvaluator:
    def __init__(self, metrics_client, baseline: dict):
        self.metrics = metrics_client
        self.baseline = baseline
        self._soft_states: dict[str, SoftTriggerState] = {}

    def evaluate(self, cluster_id: str) -> RollbackDecision:
        m = self.metrics.fetch(cluster_id)

        # --- Hard trigger evaluation ---
        if m.get("process_crash_detected"):
            return RollbackDecision(
                trigger_type = TriggerType.HARD,
                trigger_reason = "SQL Server service crash detected",
                fired_at = datetime.utcnow(),
                should_rollback = True,
                escalate_human = True,
            )

        if m.get("connection_rejection_rate", 0) > 0.05:
            return RollbackDecision(
                trigger_type = TriggerType.HARD,
                trigger_reason = f"Connection rejection rate

```

```

{m['connection_rejection_rate']:.1%}",
    fired_at      = datetime.utcnow(),
    should_rollback = True,
    escalate_human = True,
)

    replication_lag_growth =
m.get("replication_lag_growth_rate_seconds_per_minute", 0)
    if replication_lag_growth > 0 and m.get("minutes_since_patch", 0) > 10:
        return RollbackDecision(
            trigger_type    = TriggerType.HARD,
            trigger_reason   = "Replication lag growing monotonically post-
patch",
            fired_at        = datetime.utcnow(),
            should_rollback = True,
            escalate_human  = True,
        )

    # --- Soft trigger evaluation ---
    latency_baseline = self.baseline.get("p99_latency_ms", 0)
    current_latency  = m.get("p99_latency_ms", 0)
    if latency_baseline > 0 and current_latency > latency_baseline * 1.20:
        decision = self._evaluate_soft(
            "latency_elevation",
            f"P99 latency {current_latency:.0f}ms vs baseline
{latency_baseline:.0f}ms"
        )
        if decision:
            return decision

    lock_baseline = self.baseline.get("lock_waits_per_second", 0)
    current_locks = m.get("lock_waits_per_second", 0)
    if lock_baseline > 0 and current_locks > lock_baseline * 2:
        decision = self._evaluate_soft(
            "lock_wait_elevation",
            f"Lock waits {current_locks:.1f}/s vs baseline {lock_baseline:.1f}/
s"
        )
        if decision:
            return decision

    # Clear any soft states that have recovered
    self._clear_recovered_states(m)

    return RollbackDecision(
        trigger_type    = TriggerType.NONE,
        trigger_reason   = "All metrics within tolerance",
        fired_at        = datetime.utcnow(),
        should_rollback = False,
        escalate_human  = False,
    )

def _evaluate_soft(self, condition_key: str, reason: str) ->

```

```

Optional[RollbackDecision]:
    now = datetime.utcnow()
    if condition_key not in self._soft_states:
        self._soft_states[condition_key] = SoftTriggerState(
            condition = reason,
            first_seen = now,
            last_seen = now,
        )
    else:
        state = self._soft_states[condition_key]
        state.last_seen = now
        state.breach_count += 1
        if state.is_sustained():
            return RollbackDecision(
                trigger_type = TriggerType.SOFT,
                trigger_reason = f"Sustained 30m: {reason}",
                fired_at = now,
                should_rollback = True,
                escalate_human = True,
            )
    return None

def _clear_recovered_states(self, metrics: dict):
    """Remove soft trigger states for conditions that have self-resolved."""
    latency_ok = (
        metrics.get("p99_latency_ms", 0)
        <= self.baseline.get("p99_latency_ms", 0) * 1.20
    )
    if latency_ok and "latency_elevation" in self._soft_states:
        del self._soft_states["latency_elevation"]

```

Once a rollback decision is made, the rollback executor must verify its own success. A rollback that silently fails is indistinguishable from a patch that was never rolled back. The verification sequence:

```

def execute_rollback_with_verification(cluster_id: str, patch_id: str,
                                      patcher, metrics_client,
                                      baseline: dict) -> bool:
    """
    Returns True if rollback succeeded and cluster returned to healthy baseline.
    Returns False if rollback itself failed – escalates immediately.
    """
    import logging, time

    logging.warning(f"Initiating rollback: cluster={cluster_id}, patch={patch_id}")

    try:
        patcher.rollback(cluster_id, patch_id)
    except Exception as exc:
        logging.error(f"Rollback execution error on {cluster_id}: {exc}")
        # Escalate immediately – this is a tier-1 incident
        escalate_incident(
            cluster_id = cluster_id,
            severity    = "P1",
            message     = f"Patch rollback FAILED on {cluster_id}: {exc}",
        )
        return False

    # Wait for service to stabilize after rollback
    time.sleep(60)

    # Verify the binary actually reverted
    current_version = patcher.get_version(cluster_id)
    expected_version = patcher.pre_patch_version(cluster_id, patch_id)
    if current_version != expected_version:
        logging.error(
            f"Version mismatch post-rollback: got {current_version}, "
            f"expected {expected_version}"
        )
        escalate_incident(
            cluster_id = cluster_id,
            severity    = "P1",
            message     = (
                f"Rollback version verification failed: {cluster_id} "
                f"still running {current_version}"
            ),
        )
        return False

    # Verify metric recovery – wait up to 10 minutes
    for attempt in range(20):
        m = metrics_client.fetch(cluster_id)
        latency_ok = (
            m.get("p99_latency_ms", 0)
            <= baseline.get("p99_latency_ms", 0) * 1.10
        )
        error_rate_ok = m.get("connection_rejection_rate", 0) < 0.01
        replication_ok = m.get("replication_lag_growth_rate_seconds_per_minute", 0)

```



```

<= 0

    if latency_ok and error_rate_ok and replication_ok:
        logging.info(f"Rollback verified successfully for {cluster_id}")
        return True

    time.sleep(30)

# Metrics did not recover in 10 minutes even after rollback
escalate_incident(
    cluster_id = cluster_id,
    severity    = "P1",
    message     = (
        f"Post-rollback metric recovery failed on {cluster_id}. "
        f"Manual intervention required."
    ),
)
return False

```

The distinction between a failed rollback and a slow recovery is time-bounded: ten minutes is sufficient for an AG to re-establish synchronization and drain in-flight queries after a binary revert. If the AG is still degraded at that point, the issue is not the patch reversal but something deeper — possibly a corrupted page, a config inconsistency, or an infrastructure-layer failure — and human judgment is genuinely needed.

22.6 Patch Audit Trails and Compliance Reporting

Autonomous systems that operate without human approval at each step face a higher evidentiary standard than manual processes. When an auditor asks "who approved patching this cluster at 2 AM?" the answer "the policy authorized it and the orchestrator executed autonomously" must be substantiated by a complete, tamper-evident record of every decision, every trigger condition evaluated, and every action taken.

The patch audit log is append-only by design. In SQL Server this is enforced using a ledger table (SQL Server 2022+) or an INSERT-only permission structure where the patch agent role is granted `INSERT` but never `UPDATE` or `DELETE` on the audit tables.

```

CREATE TABLE dbo.PatchAuditLog (
    LogId            BIGINT IDENTITY(1,1) PRIMARY KEY,
    EventTime        DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    PatchId          NVARCHAR(100) NOT NULL,
    ClusterId        NVARCHAR(200) NOT NULL,
    RingId           INT,
    EventType        NVARCHAR(100) NOT NULL,
    TriggerType      NVARCHAR(50),
    TriggerReason    NVARCHAR(MAX),
    Actor            NVARCHAR(200) NOT NULL DEFAULT SYSTEM_USER,
    MetricsSnapshot  NVARCHAR(MAX), -- JSON stored as NVARCHAR for SQL Server 2016
    compat
    Outcome          NVARCHAR(100),
    Notes            NVARCHAR(MAX)
);

-- Deny UPDATE and DELETE to the patch agent login
DENY UPDATE ON dbo.PatchAuditLog TO patch_agent_login;
DENY DELETE ON dbo.PatchAuditLog TO patch_agent_login;

-- Create a DDL trigger to prevent table alterations by non-sysadmin principals
CREATE TRIGGER PreventAuditTableAlter
ON DATABASE
FOR ALTER_TABLE, DROP_TABLE
AS
BEGIN
    DECLARE @EventData XML = EVENTDATA();
    IF @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]', 'NVARCHAR(200)')
        = 'PatchAuditLog'
        AND IS_SRVROLEMEMBER('sysadmin') = 0
    BEGIN
        RAISERROR('Modifications to PatchAuditLog are not permitted.', 16, 1);
        ROLLBACK;
    END
END;

```

Compliance reports query the audit log to produce evidence artifacts. A typical SOC 2 or PCI-DSS audit requires demonstrating that patches were applied within the required timeframe of CVE publication and that a complete approval chain exists. The autonomous model satisfies this by making the policy itself the auditable artifact:

```

-- Patch SLA compliance report
-- Shows MSRC updates published in the last quarter and whether they were patched
within tier SLA
SELECT
    pc.update_id,
    pc.priority_score,
    CASE
        WHEN pc.priority_score >= 80 THEN 'Tier 1 (24h)'
        WHEN pc.priority_score >= 50 THEN 'Tier 2 (7d)'
        ELSE 'Tier 3 (next upgrade)'
    END AS tier,
    pc.published_at,
    MIN(CASE WHEN pal.event_type = 'patch_applied'
        THEN pal.event_time END) AS first_applied_at,
    DATEDIFF(HOUR, pc.published_at,
        MIN(CASE WHEN pal.event_type = 'patch_applied'
            THEN pal.event_time END)) AS hours_to_first_patch,
    CASE
        WHEN pc.priority_score >= 80
            AND DATEDIFF(HOUR, pc.published_at,
                MIN(CASE WHEN pal.event_type = 'patch_applied'
                    THEN pal.event_time END)) <= 24
            THEN 'COMPLIANT'
        WHEN pc.priority_score >= 50
            AND DATEDIFF(HOUR, pc.published_at,
                MIN(CASE WHEN pal.event_type = 'patch_applied'
                    THEN pal.event_time END)) <= 168
            THEN 'COMPLIANT'
        WHEN pc.priority_score < 50
            THEN 'TIER3_DEFERRED'
        ELSE 'NON_COMPLIANT'
    END AS sla_status
FROM patch_catalog pc
LEFT JOIN patch_audit_log pal
    ON pal.patch_id = pc.update_id
GROUP BY pc.update_id, pc.priority_score, pc.published_at
ORDER BY pc.published_at DESC;

```

SQL Server equivalent:

```

SELECT
    pc.CveId,
    pc.PriorityScore,
    CASE
        WHEN pc.PriorityScore >= 80 THEN 'Tier 1 (24h)'
        WHEN pc.PriorityScore >= 50 THEN 'Tier 2 (7d)'
        ELSE 'Tier 3 (next upgrade)'
    END AS Tier,
    pc.PublishedAt,
    MIN(CASE WHEN pal.EventType = 'patch_applied'
        THEN pal.EventTime END) AS FirstAppliedAt,
    DATEDIFF(HOUR, pc.PublishedAt,
        MIN(CASE WHEN pal.EventType = 'patch_applied'
            THEN pal.EventTime END)) AS HoursToFirstPatch,
    CASE
        WHEN pc.PriorityScore >= 80
            AND DATEDIFF(HOUR, pc.PublishedAt,
                MIN(CASE WHEN pal.EventType = 'patch_applied'
                    THEN pal.EventTime END)) <= 24
            THEN 'COMPLIANT'
        WHEN pc.PriorityScore >= 50
            AND DATEDIFF(HOUR, pc.PublishedAt,
                MIN(CASE WHEN pal.EventType = 'patch_applied'
                    THEN pal.EventTime END)) <= 168
            THEN 'COMPLIANT'
        WHEN pc.PriorityScore < 50
            THEN 'TIER3_DEFERRED'
        ELSE 'NON_COMPLIANT'
    END AS SlaStatus
FROM dbo.PatchCatalog pc
LEFT JOIN dbo.PatchAuditLog pal
    ON pal.PatchId = pc.CveId
WHERE pc.PublishedAt >= DATEADD(DAY, -90, SYSUTCDATETIME())
GROUP BY pc.CveId, pc.PriorityScore, pc.PublishedAt
ORDER BY pc.PriorityScore DESC, pc.PublishedAt DESC;

```

Beyond the compliance report, the audit log surfaces operational metrics that feed continuous improvement of the patch policy itself. Rollback rates per patch tier, average dwell time before ring advancement, frequency of soft-trigger false positives — all of these are computable from the log and should be reviewed quarterly to tighten or relax thresholds as the fleet matures.

22.7 Integrating with Change Management Systems: Autonomous Tickets Without Human Approval

The autonomous patch pipeline does not operate in a vacuum. Most regulated environments require that every production change be associated with a change record — a ticket in ServiceNow, Jira, or a similar system. The key architectural decision is whether a human must approve that ticket before the patch executes, or whether the ticket is created and auto-approved by policy for patches that meet defined criteria.

For Tier 1 patches (CVSS-equivalent score ≥ 80 , 24-hour SLA), requiring human approval before patching begins is incompatible with the SLA. The system must be authorized to auto-approve via policy and to create the change record as an artifact of what it did, not as a gate to get permission. This requires a formal "pre-authorized change" category in the change management system — a concept that most ITSM platforms support but that must be explicitly negotiated with the change advisory board.

The integration with ServiceNow's REST API demonstrates the pattern:

```

import httpx
import json
from datetime import datetime

SNOW_BASE = "https://your-instance.service-now.com/api/now"
SNOW_AUTH = ("automation_user", "secret")

def create_patch_change_record(
    patch_id: str,
    cve_ids: list[str],
    tier: int,
    affected_clusters: list[str],
    planned_start: datetime,
    planned_end: datetime,
) -> str:
    """
    Creates a ServiceNow change request.
    Tier 1 patches are submitted as pre-authorized (change_type=standard).
    Tier 2 and 3 submit as normal changes and await CAB approval.
    Returns the change request sys_id.
    """

    change_type = "standard" if tier == 1 else "normal"
    approval = "pre_approved" if tier == 1 else "requested"

    payload = {
        "short_description": f"Autonomous DB Patch: {' '.join(cve_ids)}",
        "description": (
            f"Automated patch deployment initiated by ADE patch orchestrator.\n"
            f"Patch ID: {patch_id}\n"
            f"CVEs addressed: {' '.join(cve_ids)}\n"
            f"Tier: {tier}\n"
            f"Affected clusters: {' '.join(affected_clusters)}\n"
            f"Planned window: {planned_start.isoformat()} - "
            f"{planned_end.isoformat()}"
        ),
        "type": change_type,
        "approval": approval,
        "category": "Database",
        "risk": "low" if tier == 1 else "moderate",
        "impact": "2", # Medium impact
        "planned_start_date": planned_start.isoformat(),
        "planned_end_date": planned_end.isoformat(),
        "u_automation_driven": "true", # Custom field for audit filtering
    }

    resp = httpx.post(
        f"{SNOW_BASE}/table/change_request",
        auth = SNOW_AUTH,
        json = payload,
        headers = {"Content-Type": "application/json"},
    )
    resp.raise_for_status()
    sys_id = resp.json()["result"]["sys_id"]

```

```

return sys_id

def close_patch_change_record(sys_id: str, outcome: str, notes: str):
    """Closes the change record with actual outcome after patch completes."""
    close_code = "successful" if outcome == "completed" else "unsuccessful"
    httpx.patch(
        f"{SNOW_BASE}/table/change_request/{sys_id}",
        auth = SNOW_AUTH,
        json = {
            "state": "closed",
            "close_code": close_code,
            "close_notes": notes,
        },
        headers = {"Content-Type": "application/json"},
    )

```

The `u_automation_driven` custom field is load-bearing for compliance: it allows auditors to filter all autonomously-executed changes and review them as a category, rather than having to distinguish them from manually-executed changes by description text. The change management team gets a clean report of every autonomous action, and the CAB can tune the pre-authorized change criteria based on observed outcomes.

For Tier 2 changes that do require CAB approval, the orchestrator submits the ticket and then polls for approval state before proceeding:

```

def wait_for_cab_approval(sys_id: str, timeout_hours: int = 72) -> bool:
    """
    Polls the change record until approved or timeout.
    Returns True if approved, False if rejected or timed out.
    """
    import time
    deadline = time.time() + (timeout_hours * 3600)

    while time.time() < deadline:
        resp = httpx.get(
            f"{SNOW_BASE}/table/change_request/{sys_id}"
            "?sysparm_fields=approval,state",
            auth = SNOW_AUTH,
        )
        record = resp.json()["result"]
        if record["approval"] == "approved":
            return True
        if record["approval"] in ("rejected", "cancelled"):
            return False
        time.sleep(300) # Poll every 5 minutes

    return False # Timed out – treat as blocked, not rejected

```

A timeout is treated differently from a rejection: a timed-out approval means the CAB did not act within the window, which may itself be an SLA violation on the ITSM side. The orchestrator logs this separately and notifies both the DBA team and the ITSM administrators, rather than silently dropping the patch.

22.8 Testing the Patch Pipeline: Chaos Injection and Dry-Run Mode

An autonomous patch pipeline that has never been exercised under failure conditions will fail unpredictably when it encounters one in production. The patch pipeline requires its own test regime: scheduled dry runs that walk the full orchestration path without applying any package changes, and chaos injection tests that deliberately trigger rollback conditions to verify the rollback path executes correctly.

Dry-run mode is toggled by a single flag threaded through the orchestrator. In dry-run mode, every action that would cause a side effect is replaced by a logged simulation:

```
class DryRunPatcher:
    """Drop-in replacement for the real patcher during dry-run execution."""

    def __init__(self, real_patcher):
        self._real = real_patcher

    def apply(self, cluster_id: str, patch_id: str):
        import logging
        version_before = self._real.get_version(cluster_id)
        logging.info(
            f"[DRY-RUN] Would apply {patch_id} to {cluster_id} "
            f"(currently {version_before})"
        )

    def rollback(self, cluster_id: str, patch_id: str):
        import logging
        logging.info(f"[DRY-RUN] Would rollback {patch_id} on {cluster_id}")

    def get_version(self, cluster_id: str) -> str:
        return self._real.get_version(cluster_id)

    def pre_patch_version(self, cluster_id: str, patch_id: str) -> str:
        return self._real.pre_patch_version(cluster_id, patch_id)
```

Dry-run mode exercises the full decision logic, the ServiceNow ticket creation (in a non-production SNOW instance), the metrics evaluation, and the ring advancement logic — but touches no database binaries. It should be run weekly in a staging environment against the current patch backlog to verify the pipeline remains functional.

Chaos injection goes further: it deliberately injects failing conditions into the metrics stream to verify that the rollback evaluator fires correctly and that rollback execution completes within the expected time budget. A chaos test scenario for the hard-trigger path:

```
class ChaosMetricsClient:
    """
    Wraps a real metrics client and injects synthetic failures
    at a specified time offset after patch application.
    """
    def __init__(self, real_client, chaos_scenario: dict):
        self._real = real_client
        self._scenario = chaos_scenario
        self._patch_applied_at = None

    def mark_patch_applied(self):
        from datetime import datetime
        self._patch_applied_at = datetime.utcnow()

    def fetch(self, cluster_id: str) -> dict:
        from datetime import datetime
        real_metrics = self._real.fetch(cluster_id)

        if self._patch_applied_at is None:
            return real_metrics

        elapsed = (datetime.utcnow() - self._patch_applied_at).total_seconds()
        inject_after = self._scenario.get("inject_after_seconds", 120)

        if elapsed >= inject_after:
            for key, value in self._scenario.get("inject_metrics", {}).items():
                real_metrics[key] = value

        return real_metrics

# Example chaos scenario: inject a crash-loop signal 2 minutes after patch
crash_loop_scenario = {
    "inject_after_seconds": 120,
    "inject_metrics": {
        "process_crash_detected": True,
    },
}

# Example chaos scenario: inject sustained latency elevation
latency_scenario = {
    "inject_after_seconds": 60,
    "inject_metrics": {
        "p99_latency_ms": 4500, # Well above any reasonable baseline
    },
}
```

Chaos tests run in a dedicated test environment on a monthly cadence and their results are recorded in the audit log under `event_type = 'chaos_test'`. A chaos test that does not trigger the expected rollback within the specified time budget is treated as a pipeline defect and blocks the next production patch cycle until resolved. This is the test-as-gate model: the autonomous system must continuously prove its own safety mechanisms work before it is trusted to act autonomously in production.

Key Takeaways

- → **Patch priority scoring combines CVSS base score, exploit probability (EPSS), and local fleet exposure** — treating all CVEs as equal severity leads to alert fatigue and delays action on genuinely dangerous vulnerabilities while wasting resources on theoretical ones.
- → **The ring rollout model gates fleet-wide propagation on dwell-time validation** — no ring advances until the previous ring has operated within defined SLI tolerances for the full dwell period, converting patch deployment from a single high-risk event into a series of validated, low-radius steps.
- → **Rollback triggers are stratified into hard triggers (immediate, no override) and soft triggers (hysteresis-gated)** — this distinction prevents transient load spikes from causing unnecessary rollbacks while ensuring genuinely dangerous post-patch conditions receive an immediate response without waiting for human review.
- → **Append-only audit logs with privilege-enforced write restrictions provide the evidentiary foundation** for autonomous operation in regulated environments — the policy itself becomes the auditable approval artifact, replacing per-change human sign-off with documented, machine-verifiable policy adherence.
- → **Dry-run mode and monthly chaos injection tests verify the rollback path remains functional** — an untested autonomous rollback is operationally equivalent to no rollback at all, and the patch pipeline must treat its own safety mechanisms as first-class test subjects with gate conditions that block production use when tests fail.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 23: Self-Managing Maintenance Tasks

Maintenance is where autonomous systems prove their worth most visibly. Index rebuild and reorganize cycles, statistics refreshes, log archival, partition management — these are tasks that senior DBAs have spent decades scheduling manually, tuning reactively, and debugging at 2 a.m. when they collide with peak workload. An autonomous maintenance engine eliminates that pattern entirely: it observes the state of every object in the catalog, models the cost of action against the cost of inaction, and executes maintenance at the optimal moment with zero human scheduling overhead. This chapter builds that engine — from the feedback loops that drive autonomous statistics and index management, to the partition lifecycle systems that provision and retire data without operator involvement, to the production-grade governance layer that keeps autonomous actions auditable and reversible.

23.1 The Maintenance Decision Loop: Observation, Costing, and Scheduling

Before any autonomous system can manage maintenance, it needs a principled model for when maintenance is *worth doing* — not just whether it is overdue. Scheduled maintenance (weekly index rebuilds, manual statistics updates) fails in high-throughput environments because it ignores current workload, lock contention probability, and the actual degradation rate of the objects it targets. The autonomous alternative is a continuous decision loop: measure object health, estimate the cost of immediate action, compare it against projected degradation cost, and schedule or defer accordingly.

The loop runs on a tight polling cycle (typically 60–300 seconds) and feeds a priority queue. Each candidate maintenance action carries three attributes: **urgency score** (how degraded is the object now), **execution cost** (estimated I/O, lock wait, duration), and **opportunity window** (how clear is the current workload). The system acts when $\text{urgency} \times \text{opportunity}$ exceeds execution cost. This is not metaphor — it translates directly into evaluable expressions over catalog metrics.

Maintenance candidate scoring — using index fragmentation and page density:

```
-- SQL Server: maintenance candidate scoring
-- Combines index fragmentation, page density, and recent rebuild history
SELECT
    OBJECT_SCHEMA_NAME(i.object_id) AS schema_name,
    OBJECT_NAME(i.object_id) AS table_name,
    i.name AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.avg_page_space_used_in_percent,
    ips.page_count,
    -- Urgency score: fragmentation weighted by page count
    ROUND(
        ips.avg_fragmentation_in_percent
        * ips.page_count / 1000.0, 2
    ) AS urgency_score,
    -- Last maintenance from job history proxy
    maint.last_run_datetime AS last_rebuilt
FROM sys.indexes i
CROSS APPLY sys.dm_db_index_physical_stats(
    DB_ID(), i.object_id, i.index_id, NULL, 'LIMITED'
) ips
LEFT JOIN (
    SELECT
        j.name,
        MAX(CAST(h.run_date AS CHAR(8)) + ' ' +
            STUFF(STUFF(RIGHT('000000' + CAST(h.run_time AS VARCHAR), 6),
                5, 0, ':'), 3, 0, ':')) AS last_run_datetime
    FROM msdb.dbo.sysjobs j
    JOIN msdb.dbo.sysjobhistory h ON j.job_id = h.job_id
    WHERE h.step_id = 0 AND h.run_status = 1
    GROUP BY j.name
) maint ON maint.name LIKE '%' + OBJECT_NAME(i.object_id) + '%'
WHERE i.type > 0
    AND ips.page_count > 500
    AND ips.avg_fragmentation_in_percent > 5
ORDER BY urgency_score DESC;
```

The output from these views feeds a Python scheduler that maintains the priority queue. The scheduler enforces two constraints before acting: a **workload gate** (active sessions below threshold, no long-running transactions older than N minutes) and a **maintenance budget** (maximum concurrent maintenance operations, rolling I/O ceiling). When both constraints pass and the top-priority candidate clears the cost threshold, the action fires.

23.2 Autonomous Statistics Management in SQL Server

SQL Server's automatic statistics update is often mistaken for autonomous maintenance — it is actually a *reactive threshold mechanism* that fires when roughly 20% of rows (plus 500) have changed since the last update. On a billion-row table, that means 200 million modifications

before statistics refresh; on high-velocity OLTP tables the threshold perpetually lags behind the workload. A truly autonomous statistics engine replaces the static threshold with modification-velocity tracking driven by real-time catalog state.

The architecture has two layers. The first layer is a **catalog monitor** that tracks per-table modification velocity: the rate at which `modification_counter` grows per minute of wall clock time, read from `sys.dm_db_stats_properties`. Tables with high velocity need more frequent updates than stable reference tables. The second layer is a **statistics dispatcher** that issues targeted `UPDATE STATISTICS ... WITH FULLSCAN` (or a calibrated sample) on a computed schedule, ahead of the automatic threshold — effectively making auto-update statistics the fallback rather than the primary mechanism.

Statistics staleness detection — identifying statistics whose modification ratio has crossed the velocity-adjusted threshold:

```

-- Autonomous statistics staleness detection
-- Run by the maintenance engine; feeds the dispatch priority queue
SELECT
    OBJECT_SCHEMA_NAME(o.object_id)          AS schema_name,
    OBJECT_NAME(o.object_id)                 AS table_name,
    s.name                                    AS stats_name,
    sp.last_updated,
    sp.rows,
    sp.modification_counter,
    ROUND(sp.modification_counter * 1.0
          / NULLIF(sp.rows, 0) * 100, 2)     AS modification_pct,
    -- Velocity-adjusted threshold: large tables get a tighter band
    CASE
        WHEN sp.rows > 100000000 THEN 2.0    -- 100M+ rows: 2%
        WHEN sp.rows > 10000000  THEN 5.0    -- 10M+ rows: 5%
        ELSE 10.0                          -- smaller: 10%
    END                                       AS stale_threshold_pct,
    CASE
        WHEN sp.modification_counter * 1.0 / NULLIF(sp.rows, 0) * 100 >
            CASE
                WHEN sp.rows > 100000000 THEN 2.0
                WHEN sp.rows > 10000000  THEN 5.0
                ELSE 10.0
            END THEN 'UPDATE_REQUIRED'
        ELSE 'OK'
    END                                       AS dispatch_decision
FROM sys.stats s
JOIN sys.objects o ON o.object_id = s.object_id
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
WHERE o.type = 'U'
    AND s.auto_created = 0
    AND sp.modification_counter > 1000
ORDER BY sp.modification_counter * 1.0 / NULLIF(sp.rows, 0) DESC;

```

Beyond scheduled refreshes, the autonomous engine must also decide when to escalate from an online `REORGANIZE` to an offline-capable `REBUILD` — the most consequential maintenance decision in SQL Server. An offline rebuild holds a schema-modification lock that blocks all access to the table. The rule is: never escalate to a non-online rebuild during business hours, never on indexes serving queries averaging more than 100 concurrent sessions, and only when fragmentation exceeds a threshold that makes reorganization economically unjustified (typically >30% fragmentation with >1 GB of affected pages).

The Python component that implements this escalation gate queries active session counts before firing:

```

# Autonomous rebuild escalation engine
# Checks the workload gate before issuing REBUILD on high-fragmentation indexes

import pyodbc
from datetime import datetime, time as dt_time

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlprimary;"
            "DATABASE=AppDB;UID=maintenance_role;PWD=*****;")
FRAG_PCT_THRESHOLD = 30.0
PAGE_COUNT_MIN = 131072 # ~1 GB in 8 KB pages
MAX_ACTIVE_SESSIONS = 20
MAINTENANCE_WINDOW = (dt_time(1, 0), dt_time(5, 0)) # 01:00-05:00 UTC

def within_maintenance_window():
    now = datetime.utcnow().time()
    return MAINTENANCE_WINDOW[0] <= now <= MAINTENANCE_WINDOW[1]

def get_active_session_count(conn):
    cur = conn.cursor()
    cur.execute("""
        SELECT COUNT(*)
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1 AND status = 'running'
    """)
    return cur.fetchone()[0]

def get_rebuild_candidates(conn):
    cur = conn.cursor()
    cur.execute("""
        SELECT OBJECT_SCHEMA_NAME(ips.object_id) AS schema_name,
               OBJECT_NAME(ips.object_id) AS table_name,
               i.name AS index_name,
               ips.avg_fragmentation_in_percent,
               ips.page_count
        FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')
ips
        JOIN sys.indexes i
        ON ips.object_id = i.object_id AND ips.index_id = i.index_id
        WHERE ips.avg_fragmentation_in_percent > ?
        AND ips.page_count > ?
        AND i.type > 0
        ORDER BY ips.page_count DESC
    """, FRAG_PCT_THRESHOLD, PAGE_COUNT_MIN)
    return cur.fetchall()

def run_resumable_rebuild(conn, schema, table, index):
    stmt = (f"ALTER INDEX [{index}] ON [{schema}].[{table}] "
            "REBUILD WITH (ONLINE = ON, RESUMABLE = ON, MAX_DURATION = 120"
            "MINUTES)")
    conn.execute(stmt)
    print(f"[{datetime.utcnow().isoformat()}] Rebuild issued: {schema}.{table}."
          f"{index}")

```

```

def main():
    if not within_maintenance_window():
        print("Outside maintenance window – skipping escalation check.")
        return

    conn = pyodbc.connect(CONN_STR, autocommit=True)
    active = get_active_session_count(conn)
    if active > MAX_ACTIVE_SESSIONS:
        print(f"Active sessions={active} exceeds gate={MAX_ACTIVE_SESSIONS} –
deferring.")
        return

    for schema, table, index, frag, pages in get_rebuild_candidates(conn):
        print(f"Escalating to REBUILD: {schema}.{table}.{index} "
              f"({frag:.1f}% fragmented, {pages // 128} MB)")
        run_resumable_rebuild(conn, schema, table, index)

    conn.close()

if __name__ == "__main__":
    main()

```

The same autonomous statistics management pattern applies to plan regressions. High-throughput OLTP tables should receive statistics updates independently from index maintenance — particularly when Query Store detects a plan regression. A spike in mean execution time on a query that previously was stable indicates stale statistics before any system alert fires. The maintenance engine monitors the Query Store regression signal and issues targeted `UPDATE STATISTICS` on the tables involved in those queries.

23.3 Autonomous Index Lifecycle Management

Index management is the maintenance task most likely to accumulate technical debt in manual operations. DBAs add indexes reactively (fixing a slow query), never audit whether the index is still used, and rarely rebuild fragmented ones on a principled schedule. The autonomous index lifecycle engine handles all three phases — creation, fragmentation remediation, and retirement — without human initiation.

Index creation is covered in detail in the query optimization chapter, but the maintenance layer adds a constraint: any autonomously created index must be tracked in a `dba_autonomous_indexes` registry table, stamped with the query pattern that justified it, and subject to a 30-day usage review. If `sys.dm_db_index_usage_stats` shows zero user seeks or scans in 30 days, the index is a retirement candidate.

Fragmentation remediation is where the maintenance engine makes daily decisions. The threshold logic is explicit:

- → Fragmentation < 10%: no action
- → Fragmentation 10–30%: REORGANIZE on a background schedule
- → Fragmentation > 30%: REBUILD, subject to the workload gate

Autonomous index remediation — REORGANIZE vs REBUILD decision:

```
-- SQL Server: autonomous index maintenance decision
-- Emits the correct action (REORGANIZE vs REBUILD) per fragmentation band
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id)          AS schema_name,
    OBJECT_NAME(ips.object_id)                 AS table_name,
    i.name                                     AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    CASE
        WHEN ips.avg_fragmentation_in_percent BETWEEN 10 AND 30
        THEN
            'ALTER INDEX ' + QUOTENAME(i.name)
            + ' ON ' + QUOTENAME(OBJECT_SCHEMA_NAME(ips.object_id))
            + '.' + QUOTENAME(OBJECT_NAME(ips.object_id))
            + ' REORGANIZE;'
        WHEN ips.avg_fragmentation_in_percent > 30
        THEN
            'ALTER INDEX ' + QUOTENAME(i.name)
            + ' ON ' + QUOTENAME(OBJECT_SCHEMA_NAME(ips.object_id))
            + '.' + QUOTENAME(OBJECT_NAME(ips.object_id))
            + ' REBUILD WITH (ONLINE = ON, RESUMABLE = ON, '
            + ' MAX_DURATION = 120 MINUTES);'
        ELSE 'No action required.'
    END                                         AS maintenance_action
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'LIMITED'
) ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id
   AND ips.index_id = i.index_id
WHERE i.type > 0
   AND ips.page_count > 200
   AND ips.avg_fragmentation_in_percent > 10
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

The `RESUMABLE = ON` clause on SQL Server rebuilds is a requirement, not a nicety, in autonomous systems. A resumable rebuild that gets interrupted by a workload spike or failover does not lose its progress — it pauses and resumes at next opportunity. The autonomous scheduler checks `sys.index_resumable_operations` on each maintenance cycle and resumes any paused operations if the workload gate passes.

Index retirement follows a conservative pattern: the engine marks unused indexes as `DEFERRED_DROP` in the registry, disables (rather than drops) them for a 14-day observation window, verifies no plan regressions appear (via plan cache monitoring), and only then executes the drop. This staged approach means no autonomously managed index drop can produce an irreversible performance regression.

23.4 Automated Partition Lifecycle: Provisioning, Migrating, and Retiring Partitions

Partition management is one of the most operationally demanding maintenance tasks in large-scale databases — and one of the most amenable to full automation. A time-partitioned table growing at 50 GB/day needs new partitions created before the current period ends, old partitions detached and archived on retention schedule, and tablespace placement managed as storage tiers change. Done manually across dozens of tables, this is a multi-hour weekly DBA task. Done autonomously, it is a background process that requires no human attention until an exception condition fires.

The autonomous partition manager has four responsibilities:

1. **Provisioning:** Create future partitions ahead of the current period (typically 3–7 days ahead for daily partitions, 2 months ahead for monthly).
2. **Attachment validation:** Verify new partitions pass constraint checks before the period goes live.
3. **Archival offload:** Detach expired partitions, back up and export them to cold storage, and verify integrity before dropping.
4. **Filegroup migration:** Move warm partitions to compressed or lower-cost filegroups based on age rules.

```
graph LR
    A[Provision future partition  
3-7 days ahead] --> B[Validate constraints  
before period goes live]
    B --> C[Partition ages past  
hot-storage window]
    C --> D[Migrate to warm/cold  
filegroup]
    D --> E[Partition ages past  
retention threshold]
    E --> F[Switch out + export  
to cold storage]
    F --> G[Verify row count]
    G --> H[Drop staging table]
```

The four partition-manager responsibilities form one continuous lifecycle per table — every stage writes to `dba_maintenance_log` so a crashed run resumes from the last unconfirmed action.

Autonomous monthly partition provisioning — for a time-series table on a monthly partition scheme. Idempotent: safe to call on every maintenance cycle:

```
-- Provision the next 2 monthly partitions ahead of the current period
-- Partition function: pf_monthly_events (RANGE RIGHT on DATE)
DECLARE @i INT = 0;
DECLARE @boundary DATE;
DECLARE @existing_boundaries TABLE (value DATE);

INSERT INTO @existing_boundaries
SELECT CAST(value AS DATE)
FROM sys.partition_range_values prv
WHERE prv.function_id = (
    SELECT pf.function_id
    FROM sys.partition_functions pf
    WHERE pf.name = 'pf_monthly_events'
);

WHILE @i < 2
BEGIN
    -- First day of the month, @i months ahead of next month
    SET @boundary = DATEADD(MONTH, DATEDIFF(MONTH, 0, GETDATE()) + @i + 1, 0);

    IF NOT EXISTS (SELECT 1 FROM @existing_boundaries WHERE value = @boundary)
    BEGIN
        ALTER PARTITION SCHEME ps_monthly_events NEXT USED [PRIMARY];
        ALTER PARTITION FUNCTION pf_monthly_events() SPLIT RANGE (@boundary);

        INSERT INTO dba_maintenance_log
            (action, object_name, executed_at, details)
        VALUES
            ('PARTITION_SPLIT', 'pf_monthly_events', GETUTCDATE(),
            'Split boundary ' + CAST(@boundary AS NVARCHAR(20)));
    END;

    SET @i = @i + 1;
END
```

For partition archival on SQL Server, the autonomous engine uses partition switching — a metadata-only operation that detaches a partition into a staging table without data movement, then exports and drops the staging table. This is the correct production pattern because `ALTER TABLE ... SWITCH` is near-instantaneous and does not hold locks on the main table during export:

SQL Server — Autonomous partition archival via partition switching:

```

-- SQL Server: autonomous partition archival for a monthly partitioned table
-- Step 1: identify the oldest partition eligible for archival
DECLARE @oldest_boundary DATE;
DECLARE @partition_number INT;
DECLARE @staging_table NVARCHAR(128);
DECLARE @sql NVARCHAR(MAX);

-- Find the oldest partition boundary older than retention threshold (24 months)
SELECT TOP 1
    @oldest_boundary = CAST(prv.value AS DATE),
    @partition_number = p.partition_number
FROM sys.partitions p
JOIN sys.partition_range_values prv
    ON prv.function_id = (
        SELECT pf.function_id
        FROM sys.partition_functions pf
        WHERE pf.name = 'pf_monthly_events'
    )
    AND prv.boundary_id = p.partition_number - 1
WHERE p.object_id = OBJECT_ID('dbo.event_log')
    AND CAST(prv.value AS DATE) < DATEADD(MONTH, -24, GETDATE())
ORDER BY prv.value ASC;

IF @oldest_boundary IS NOT NULL
BEGIN
    SET @staging_table = 'dbo.event_log_archive_'
        + FORMAT(@oldest_boundary, 'yyyyMM');

    -- Create staging table with same schema
    SET @sql = '
        SELECT TOP 0 * INTO ' + @staging_table + '
        FROM dbo.event_log;
    ';
    EXEC sp_executesql @sql;

    -- Switch partition to staging table (metadata-only, instant)
    SET @sql = '
        ALTER TABLE dbo.event_log
        SWITCH PARTITION ' + CAST(@partition_number AS NVARCHAR)
        + ' TO ' + @staging_table + '
    ';
    EXEC sp_executesql @sql;

    -- Log the archival event
    INSERT INTO dba_maintenance_log
        (action, object_name, executed_at, details)
    VALUES
        ('PARTITION_SWITCH', @staging_table, GETUTCDATE(),
        'Switched partition ' + CAST(@partition_number AS NVARCHAR)
        + ' boundary ' + CAST(@oldest_boundary AS NVARCHAR(20)));

    -- At this point, the staging table is handed off to the
    -- async export job (bcp/BACPAC/Azure Blob) before DROP

```

```
PRINT 'Partition switched to staging: ' + @staging_table;  
END
```

The async export job that follows the partition switch is a separate process that reads `dba_maintenance_log` for `PARTITION_SWITCH` events not yet marked `ARCHIVED`, exports the staging table to cold storage (Azure Blob, S3, or NFS depending on deployment), verifies the row count against the pre-switch snapshot, and only then issues `DROP TABLE`. This separation of concerns is deliberate: the partition switch itself cannot fail the main table, and the drop can be retried independently if the export job is interrupted.

Filegroup migration completes the partition lifecycle. As partitions age from hot to warm to cold, the autonomous engine moves them across filegroups defined on progressively lower-cost storage. The migration uses an online index rebuild with `DROP_EXISTING = ON` onto the target filegroup — the autonomous engine operates on one partition at a time rather than the whole table: each move is brief and scoped, and the workload gate still governs when it is allowed to run:

```

-- Autonomous filegroup migration for aged partitions
-- Moves partitions older than 90 days to the WARM_DATA filegroup
-- using online index rebuilds (one partition at a time)
DECLARE @partition_number INT;
DECLARE @sql NVARCHAR(MAX);

DECLARE part_cursor CURSOR FOR
SELECT p.partition_number
FROM sys.partitions p
JOIN sys.partition_range_values prv
    ON prv.function_id = (
        SELECT pf.function_id
        FROM sys.partition_functions pf
        WHERE pf.name = 'pf_monthly_events'
    )
    AND prv.boundary_id = p.partition_number - 1
JOIN sys.indexes i
    ON i.object_id = p.object_id AND i.index_id = p.index_id
WHERE p.object_id = OBJECT_ID('dbo.event_log')
    AND p.index_id = 1
    AND CAST(prv.value AS DATE) < DATEADD(DAY, -90, GETDATE())
    AND p.data_compression_desc = 'NONE';

OPEN part_cursor;
FETCH NEXT FROM part_cursor INTO @partition_number;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Online rebuild of this partition onto the WARM_DATA filegroup
    SET @sql = 'ALTER INDEX PK_event_log ON dbo.event_log '
        + 'REBUILD PARTITION = ' + CAST(@partition_number AS NVARCHAR)
        + ' WITH (ONLINE = ON, DATA_COMPRESSION = PAGE) ON [WARM_DATA];';
    EXEC sp_executesql @sql;

    INSERT INTO dba_maintenance_log
        (action, object_name, executed_at, details)
    VALUES
        ('FILEGROUP_MIGRATE', 'dbo.event_log', GETUTCDATE(),
        'Moved partition ' + CAST(@partition_number AS NVARCHAR)
        + ' to WARM_DATA filegroup');

    FETCH NEXT FROM part_cursor INTO @partition_number;
END

CLOSE part_cursor;
DEALLOCATE part_cursor;

```

A complete partition lifecycle manager combines all four responsibilities into a single orchestration loop, typically implemented as a Python daemon with individual method calls for provision, validate, archive, and migrate. Each method is idempotent and logs every action to `dba_maintenance_log`, so a crashed run simply resumes from the last unconfirmed action on the next cycle.

23.5 Log and Audit File Management

Database engines generate maintenance overhead of their own: transaction log files, slow query logs, audit logs, and trace files. An autonomous maintenance engine that manages tables but ignores file-level growth creates a different class of 2 a.m. incident — disk exhaustion from an unmanaged audit log directory.

The log management layer of the autonomous engine handles three concerns: **Transaction log retention policy enforcement, log file rotation and compression**, and **alert escalation when growth rate suggests impending exhaustion**.

SQL Server transaction log management in autonomous systems centers on ensuring log backups run frequently enough under the FULL recovery model to prevent log file expansion. The autonomous engine monitors log space usage via `sys.dm_db_log_space_usage` and triggers an unscheduled log backup when used log space crosses a threshold, rather than waiting for the next scheduled backup window:

```

-- SQL Server: reactive log backup trigger
-- Fires an ad-hoc log backup when log space utilization exceeds threshold
DECLARE @log_used_pct    FLOAT;
DECLARE @db_name          SYSNAME = DB_NAME();
DECLARE @backup_path      NVARCHAR(512);
DECLARE @sql              NVARCHAR(1024);

SELECT @log_used_pct =
    used_log_space_in_percent
FROM sys.dm_db_log_space_usage;

IF @log_used_pct > 70.0
BEGIN
    SET @backup_path =
        N'\\backup-share\logs\'
        + @db_name + N'_log_'
        + FORMAT(GETUTCDATE(), N'yyyyMMdd_HH:mm:ss')
        + N'.bak';

    SET @sql = N'BACKUP LOG ' + QUOTENAME(@db_name)
        + N' TO DISK = ' + QUOTENAME(@backup_path, '''')
        + N' WITH COMPRESSION, STATS = 10;';

    EXEC sp_executesql @sql;

    INSERT INTO dba_maintenance_log
        (action, object_name, executed_at, details)
    VALUES
        ('LOG_BACKUP_REACTIVE', @db_name, GETUTCDATE(),
        'Log used ' + CAST(ROUND(@log_used_pct, 1) AS VARCHAR)
        + '% triggered unscheduled backup to ' + @backup_path);
END

```

Log file compression and rotation for operating-system-level logs (SQL Server ERRORLOG files) is handled by the Python maintenance daemon, which calls standard OS rotation utilities or implements rotation logic directly. The key autonomous behavior is that the daemon models disk consumption rate and projects time-to-full, alerting when the projection falls below a configurable horizon (for example, fewer than 48 hours to disk exhaustion at current growth rate) before the threshold is actually breached.

23.6 Governance, Auditing, and the Human Override Layer

Autonomous maintenance at production scale requires a governance layer that answers three questions at any moment: what did the system do, why did it do it, and how can a human stop it or reverse it? Without complete answers to all three, the autonomous engine is operationally dangerous — not because of what it does in normal operation, but because of what happens the first time it does something unexpected.

The governance layer has four components: the **maintenance audit log**, the **action registry with justification capture**, the **circuit breaker**, and the **dry-run mode**.

The Maintenance Audit Log

Every autonomous action — every index rebuild, every statistics refresh, every partition create or drop, every partition split — writes a structured record to `dba_maintenance_log`.

This table was referenced throughout the preceding sections; here is its complete definition:

```
-- Autonomous maintenance audit log (SQL Server)
CREATE TABLE dbo.dba_maintenance_log (
    log_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    action          VARCHAR(64) NOT NULL, -- e.g. 'INDEX_REBUILD',
    'PARTITION_SPLIT'
    object_schema   VARCHAR(128),
    object_name     VARCHAR(256) NOT NULL,
    executed_at     DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    duration_ms     INT, -- populated on completion
    rows_affected   BIGINT,
    status          VARCHAR(16) NOT NULL DEFAULT 'RUNNING',
    -- RUNNING, COMPLETE, FAILED,
    SKIPPED
    justification    NVARCHAR(MAX), -- why the engine chose this action
    initiated_by    VARCHAR(64) NOT NULL DEFAULT 'autonomous_engine',
    dry_run         BIT NOT NULL DEFAULT 0,
    details         NVARCHAR(MAX),
    error_message   NVARCHAR(MAX)
);

CREATE INDEX idx_maint_log_action_ts
ON dbo.dba_maintenance_log (action, executed_at DESC);

CREATE INDEX idx_maint_log_object
ON dbo.dba_maintenance_log (object_name, executed_at DESC);
```

The `justification` column is what separates a governance-grade audit log from a simple activity log. Every action written by the autonomous engine includes a human-readable explanation of the metric values that triggered it: `"frag_pct=42.1, pages=260000, last_rebuild=9d ago, active_sessions=3"`. A DBA reviewing the log the next morning can reconstruct exactly what the engine observed and why it acted, without needing to query historical catalog snapshots.

The Circuit Breaker

The circuit breaker is a single-table, single-row control record that the maintenance engine checks at the start of every decision loop iteration. If the breaker is tripped — either by the engine itself in response to an anomalous condition, or by a human operator — all autonomous actions halt immediately:

```
-- Circuit breaker control table
CREATE TABLE dbo.dba_maintenance_circuit_breaker (
    id            INT PRIMARY KEY DEFAULT 1,
    enabled       BIT NOT NULL DEFAULT 1,
    tripped_at    DATETIME2,
    tripped_by    VARCHAR(128),
    reason        NVARCHAR(MAX),
    CONSTRAINT single_row CHECK (id = 1)
);

INSERT INTO dbo.dba_maintenance_circuit_breaker (id, enabled)
VALUES (1, 1);

-- Human operator trips the breaker:
-- UPDATE dbo.dba_maintenance_circuit_breaker
-- SET enabled = 0, tripped_at = SYSUTCDATETIME(),
--     tripped_by = 'dba_oncall', reason = 'Pre-release freeze';

-- Engine checks at loop start:
-- SELECT enabled FROM dbo.dba_maintenance_circuit_breaker WHERE id = 1;
```

The Python maintenance daemon checks this record before every action dispatch. If `enabled = 0`, the daemon logs a `CIRCUIT_BREAKER_HALT` event and sleeps until the next cycle, at which point it checks again. This means a DBA can halt all autonomous maintenance with a single UPDATE statement from any SQL client — no SSH required, no process kill needed.

The engine also trips its own breaker under two self-protection conditions: if it detects that a previous autonomous action is still running beyond its expected duration (suggesting a lock wait or runaway operation), and if it encounters three consecutive failures on any single object (suggesting a systemic problem requiring human diagnosis).

Dry-Run Mode

Dry-run mode is the development and validation interface for the autonomous engine. When `dry_run = TRUE` is passed to any action method, the engine performs all analysis, writes the full justification record to `dba_maintenance_log` with `dry_run = TRUE` and `status = 'SKIPPED'`, but does not execute the maintenance statement. This mode is used during initial deployment to validate that the engine's decisions match operator intuitions, and during change-freeze periods when the circuit breaker would otherwise need to be tripped entirely.

The Python dispatcher implements dry-run as a flag threaded through all action methods:

```

# Maintenance dispatcher with dry-run and circuit breaker integration

import pyodbc
import logging
from datetime import datetime

logger = logging.getLogger("autonomous_maintenance")

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=sqlprimary;"
            "DATABASE=AppDB;UID=maintenance_role;PWD=*****;")

class MaintenanceDispatcher:

    def __init__(self, dry_run: bool = False):
        self.dry_run = dry_run
        self.conn = pyodbc.connect(CONN_STR, autocommit=True)

    def _circuit_breaker_enabled(self) -> bool:
        cur = self.conn.cursor()
        cur.execute(
            "SELECT enabled FROM dbo.dba_maintenance_circuit_breaker WHERE id = 1"
        )
        row = cur.fetchone()
        return bool(row[0]) if row else False

    def _log_action(self, action: str, object_name: str,
                    justification: str, status: str = "RUNNING",
                    duration_ms: int = None, error: str = None) -> int:
        cur = self.conn.cursor()
        cur.execute("""
            INSERT INTO dbo.dba_maintenance_log
                (action, object_name, executed_at, status,
                 justification, initiated_by, dry_run, error_message,
                 duration_ms)
            OUTPUT INSERTED.log_id
            VALUES (?, ?, SYSUTCDATETIME(), ?, ?, ?, ?, ?, ?)
        """, (action, object_name, status, justification,
              "autonomous_engine", int(self.dry_run), error, duration_ms))
        return cur.fetchone()[0]

    def _update_log(self, log_id: int, status: str,
                    duration_ms: int = None, error: str = None):
        cur = self.conn.cursor()
        cur.execute("""
            UPDATE dbo.dba_maintenance_log
            SET status = ?, duration_ms = ?, error_message = ?
            WHERE log_id = ?
        """, (status, duration_ms, error, log_id))

    def dispatch_stats_update(self, schema: str, table: str,
                              justification: str, fullscan: bool = False):
        if not self._circuit_breaker_enabled():
            logger.warning("Circuit breaker tripped – skipping UPDATE STATISTICS on

```

```

%S.%S",
                                schema, table)

    return

    action = "STATS_FULLSCAN" if fullscan else "STATS_SAMPLED"
    scan   = "WITH FULLSCAN" if fullscan else ""
    stmt   = f"UPDATE STATISTICS [{schema}].[{table}] {scan}".strip()
    log_id = self._log_action(action, f"{schema}.{table}", justification)

    if self.dry_run:
        logger.info("[DRY RUN] Would execute: %s", stmt)
        self._update_log(log_id, "SKIPPED")
        return

    start = datetime.utcnow()
    try:
        self.conn.execute(stmt)
        elapsed = int((datetime.utcnow() - start).total_seconds() * 1000)
        self._update_log(log_id, "COMPLETE", duration_ms=elapsed)
        logger.info("Completed %s on %s.%s in %dms",
                    action, schema, table, elapsed)
    except Exception as exc:
        elapsed = int((datetime.utcnow() - start).total_seconds() * 1000)
        self._update_log(log_id, "FAILED",
                        duration_ms=elapsed, error=str(exc))
        logger.error("FAILED %s on %s.%s: %s", action, schema, table, exc)
        raise

```

This pattern — circuit breaker check, pre-action log entry, dry-run branch, execution with timing, completion update — is applied uniformly to every action type in the dispatcher. A DBA can query `dba_maintenance_log` at any time and see the full history with justifications, durations, and outcomes for every autonomous decision the engine has ever made.

Operator Dashboard Query

The operational health of the autonomous maintenance engine is summarized by a single dashboard query that DBAs can pin to their monitoring interface:

Operational summary (last 24 hours):

```
-- Autonomous maintenance engine summary
SELECT
    action,
    status,
    COUNT(*)                                AS event_count,
    AVG(duration_ms)                        AS avg_duration_ms,
    MAX(duration_ms)                        AS max_duration_ms,
    SUM(CASE WHEN dry_run = 1 THEN 1 ELSE 0 END) AS dry_run_count,
    MIN(executed_at)                        AS first_event,
    MAX(executed_at)                        AS last_event
FROM dba_maintenance_log
WHERE executed_at > DATEADD(HOUR, -24, GETUTCDATE())
GROUP BY action, status
ORDER BY action, status;
```

The circuit breaker state, the count of `FAILED` status events, and the presence of any escalation entries in the last 24 hours are the three signals a DBA monitors for autonomous maintenance health. Everything else runs without human attention.

Key Takeaways

- → **The maintenance decision loop** replaces static cron schedules with a continuous urgency-cost-opportunity model: maintenance fires when object degradation cost exceeds execution cost and the workload gate is open, not because a clock expired.
- → **Autonomous statistics management** treats automatic statistics updates as a fallback, not the primary mechanism — using velocity-adjusted staleness thresholds and targeted `UPDATE STATISTICS` dispatch to keep the optimizer fed with fresh cardinality estimates without manual scheduling.
- → **Index lifecycle automation** covers all three phases — creation, fragmentation remediation (`REORGANIZE` vs. `REBUILD` based on fragmentation band), and staged retirement through a deferred-drop registry — eliminating index technical debt accumulation over time.
- → **Partition lifecycle management** handles provisioning, attachment validation, archival via metadata-only partition switching, and filegroup migration in a single idempotent orchestration loop, reducing multi-hour weekly DBA effort to a background process requiring no human attention during normal operation.
- → **The governance layer — audit log, justification capture, circuit breaker, and dry-run mode — is not optional scaffolding:** it is what makes autonomous maintenance safe to deploy in production, giving operators complete visibility into every decision, a single-statement halt mechanism, and a validation path for new environments before live execution begins.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 24: Autonomous Job Scheduling

Databases have always needed scheduled work — index rebuilds, statistics refreshes, partition rotations, archival jobs, ETL pipelines. Traditional schedulers treat these as fixed calendar entries: run at 2 AM, repeat nightly, alert a human when something fails. Autonomous job scheduling inverts this model entirely. Instead of humans deciding when and what to run, the system observes its own internal state, external load signals, and historical execution patterns to decide dynamically — not just *when* to run a job, but *whether* to run it at all, *which version* of it to execute, and *how to react* when conditions change mid-execution. This chapter builds that system from first principles through production-hardened implementation on SQL Server.

From Cron to Intent: Rethinking What a Job Schedule Actually Is

The cron model encodes a decision made by a human at a point in time: "I think Tuesday at 3 AM will be quiet." That assumption decays. Load patterns shift, data volumes grow, maintenance windows disappear as systems go global. The autonomous alternative is to encode *intent* rather than timing: "Rebuild bloated indexes whenever write throughput drops below a threshold and no active transactions exceed 30 seconds."

Intent-based scheduling requires three things the cron model lacks: a real-time signal source, a policy engine that converts signals into scheduling decisions, and a job executor that can be interrupted, retried, or throttled based on changing conditions mid-run.

Before building the scheduler, you need to formalize what a job definition looks like in this model. A job is no longer a command plus a cron expression. It is a tuple:

```
Job = (work_unit, preconditions, constraints, priority, estimated_cost)
```

- → **work_unit**: the actual SQL or procedure to execute
- → **preconditions**: signals that must be true before execution begins
- → **constraints**: conditions that, if violated during execution, trigger suspension or abort
- → **priority**: relative weight when multiple jobs compete for execution windows
- → **estimated_cost**: predicted impact on I/O, CPU, and lock contention

This formalization matters because the scheduler needs to make decisions that balance multiple jobs against each other and against live workload — a problem that cron cannot even represent, let alone solve.

```

-- Intent-based job registry in SQL Server
CREATE TABLE dbo.autonomous_jobs (
    job_id          INT IDENTITY(1,1) PRIMARY KEY,
    job_name        NVARCHAR(256) NOT NULL,
    work_unit       NVARCHAR(MAX) NOT NULL,    -- T-SQL batch or EXEC call
    preconditions   NVARCHAR(MAX) NOT NULL    -- JSON array
        CONSTRAINT chk_pre CHECK (ISJSON(preconditions) = 1),
    constraints_def NVARCHAR(MAX) NOT NULL
        CONSTRAINT chk_con CHECK (ISJSON(constraints_def) = 1),
    priority        INT NOT NULL DEFAULT 50,
    estimated_io_mb INT NULL,
    estimated_cpu_pct DECIMAL(5,2) NULL,
    max_duration_sec INT DEFAULT 3600,
    enabled         BIT DEFAULT 1,
    created_at      DATETIME2 DEFAULT SYSUTCDATETIME(),
    last_modified   DATETIME2 DEFAULT SYSUTCDATETIME()
);

-- Register an autonomous statistics update job
INSERT INTO dbo.autonomous_jobs (
    job_name, work_unit, preconditions, constraints_def, priority, estimated_cpu_pct
) VALUES (
    'auto_update_statistics',
    'EXEC maintenance.UpdateStaleStatistics',
    N'[
        {"signal": "cpu_utilization_pct", "operator": "<", "threshold": 35},
        {"signal": "stale_stats_object_count", "operator": ">", "threshold": 10}
    ]',
    N'[
        {"signal": "cpu_utilization_pct", "operator": "<", "threshold": 75},
        {"signal": "blocking_chain_depth", "operator": "<", "threshold": 3}
    ]',
    30,
    15.0
);

```

The precondition system means a job that has nothing meaningful to do tonight — because bloat hasn't accumulated or statistics are current — simply doesn't run. This is different from skipping: the system evaluated the state, found the precondition unmet, and logged a deferred decision. That log entry feeds back into the system's learning loop.

Building the Signal Pipeline: Real-Time State Collection for Scheduling Decisions

The scheduler is only as intelligent as its signal inputs. A signal pipeline continuously collects, normalizes, and stores database health metrics that precondition and constraint evaluators query at decision time. Building this pipeline correctly is where most autonomous scheduling implementations fail — they either query too slowly (stale signals lead to bad decisions) or query too aggressively (the monitoring overhead becomes its own workload problem).

The architecture uses a tiered collection model. High-frequency signals — active connections, CPU utilization, query duration — are sampled every 10–15 seconds and stored in a fixed-size ring buffer. Lower-frequency signals — replication lag, bloat estimates, stale statistics counts — are collected every 1–5 minutes. Expensive signals — actual bloat calculations requiring `pgstattuple`, or SQL Server's fragmentation analysis — are collected on-demand or once per hour and cached.

For SQL Server, the equivalent pipeline integrates with existing DMV infrastructure and can feed into a dedicated monitoring database or an external time-series store:

```

-- Signal collection stored procedure for SQL Server autonomous scheduler
CREATE OR ALTER PROCEDURE monitoring.CollectSchedulerSignals
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @cpu_pct          DECIMAL(5,2),
            @longest_txn      INT,
            @blocking_depth   INT;

    -- CPU utilization from ring buffer (last 30-second average)
    SELECT @cpu_pct = AVG(CAST(100 - record.value('(/Record/SchedulerMonitorEvent/
SystemHealth/ProcessUtilization)[1]','int') AS DECIMAL(5,2)))
    FROM (
        SELECT CAST(record AS XML) AS record
        FROM sys.dm_os_ring_buffers
        WHERE ring_buffer_type = N'RING_BUFFER_SCHEDULER_MONITOR'
        AND DATEADD(ms, -1 * (SELECT cpu_ticks / (cpu_ticks/ms_ticks) FROM
sys.dm_os_sys_info), GETDATE()) < GETDATE()
    ) AS ring_data;

    -- Longest active transaction
    SELECT @longest_txn = ISNULL(MAX(DATEDIFF(SECOND, at2.transaction_begin_time,
GETDATE()))), 0)
    FROM sys.dm_tran_active_transactions at2
    WHERE at2.transaction_type != 2; -- exclude system transactions

    -- Blocking chain depth
    SELECT @blocking_depth = ISNULL(MAX(blocking_depth.depth), 0)
    FROM (
        SELECT COUNT(*) AS depth
        FROM sys.dm_exec_requests r
        WHERE r.blocking_session_id > 0
        GROUP BY r.blocking_session_id
    ) AS blocking_depth;

    INSERT INTO monitoring.scheduler_signals (signal_name, signal_value,
collected_at)
    VALUES
        ('cpu_utilization_pct', @cpu_pct,          SYSUTCDATETIME()),
        ('longest_txn_seconds', @longest_txn,      SYSUTCDATETIME()),
        ('blocking_chain_depth', @blocking_depth,  SYSUTCDATETIME());

    -- Purge signals older than 24 hours
    DELETE FROM monitoring.scheduler_signals
    WHERE collected_at < DATEADD(HOUR, -24, SYSUTCDATETIME());
END;

```

The signal pipeline also needs to collect *semantic* signals — database-level facts like bloat ratios, partition aging, statistics staleness — that are not directly observable from DMVs. These are computed by separate collector jobs on longer intervals, stored in a

`semantic_signals` table with explicit TTLs, and consumed by the precondition evaluator the same way as real-time metrics. The separation matters: real-time and semantic signals have different freshness requirements and different collection costs, and conflating them creates either over-expensive polling or dangerously stale state.

Externally, the signal pipeline integrates with Prometheus for cluster-level signals. When the scheduler needs to know whether the AG is mid-failover, or whether connection counts are saturated, it queries Prometheus metrics via the HTTP API rather than trying to infer this from DMV data alone. A Python signal bridge handles this:

```

import requests
import pyodbc
from datetime import datetime, timezone

PROMETHEUS_URL = "http://prometheus:9090/api/v1/query"
CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=scheduler;UID=scheduler_svc;PWD=*****;")

PROMETHEUS_SIGNALS = {
    "ag_sync_health_pct":
        '100 * avg(ag_database_replica_health{ag="prod-ag"} == 1)',
    "connection_saturation_pct":
        '100 * mssql_user_connections / 32767',
    "batch_requests_per_sec":
        'rate(mssql_batch_requests{instance="primary"}[5m])',
}

def collect_external_signals(conn):
    cursor = conn.cursor()
    for signal_name, query in PROMETHEUS_SIGNALS.items():
        try:
            resp = requests.get(
                PROMETHEUS_URL,
                params={"query": query},
                timeout=5
            )
            result = resp.json()["data"]["result"]
            if result:
                value = float(result[0]["value"][1])
                cursor.execute("""
                    MERGE scheduler_signals AS target
                    USING (SELECT ? AS signal_name) AS src
                    ON target.signal_name = src.signal_name
                    WHEN MATCHED THEN
                        UPDATE SET signal_value = ?, collected_at = SYSUTCDATETIME()
                    WHEN NOT MATCHED THEN
                        INSERT (signal_name, signal_value, collected_at)
                        VALUES (?, ?, SYSUTCDATETIME());
                """, (signal_name, value, signal_name, value))
        except Exception as e:
            print(f"Signal collection failed for {signal_name}: {e}")
    conn.commit()

if __name__ == "__main__":
    with pyodbc.connect(CONN_STR) as conn:
        collect_external_signals(conn)

```

This bridge runs as a Kubernetes CronJob every 30 seconds, feeding cluster-level state into the same signal table that the database-internal collector populates. The scheduler has a unified signal query path regardless of whether a signal originated from a DMV, a catalog query, or a Prometheus query.

The Scheduling Engine: Decision Logic, Job Queuing, and Conflict Resolution

With a rich signal pipeline in place, the scheduling engine is the decision layer that sits between signals and execution. Its core loop runs on a tight interval — every 30 to 60 seconds — and performs four operations: evaluate all enabled jobs against their preconditions, score eligible jobs by priority and resource cost, resolve conflicts between competing jobs, and dispatch the winner to the execution layer.

The precondition evaluator is a signal-query function that reads the most recent values for each signal referenced in a job's precondition array and applies the operator-threshold comparison:

The conflict resolver handles a scenario the cron model cannot: two jobs are both eligible to run, but running both simultaneously would collectively breach a resource constraint. The resolver calculates the *combined estimated cost* of all eligible jobs sorted by priority, then selects a maximal subset whose aggregate I/O and CPU estimates stay within configurable limits. Lower-priority jobs that cannot fit into the current window are deferred with a `deferred_reason` log entry.

This approach has a secondary benefit: when a high-priority job finishes early and releases its resource reservation, the scheduler's next tick can immediately promote a previously deferred job without waiting for the next calendar window. The system fills available capacity continuously rather than wasting it.

Job execution state is tracked in a `job_runs` table, which the constraint evaluator monitors throughout execution. If a job is running and a constraint signal breaches its threshold — for example, active connection percentage spikes above 70% during an index rebuild — the executor receives a soft-stop signal. The job is given a grace period (configurable per job, default 60 seconds) to reach a safe checkpoint before being terminated. The interrupted run is logged with `status = 'suspended'` and rescheduled for the next eligible window, not silently dropped.

The suspension-and-resume pattern requires jobs themselves to be written with interruption awareness. Index rebuilds issued with `RESUMABLE = ON` can be paused and resumed safely. Statistics updates are inherently interruptible. For multi-step ETL pipelines, the work unit should be a stored procedure that implements checkpoint-based progress tracking, resuming from the last committed batch rather than restarting from scratch.

Workload-Aware Execution Windows and Load Forecasting

Static scheduling knows nothing about the future. Autonomous scheduling learns it. The system builds a rolling forecast of expected workload by analyzing historical signal data, identifying time-of-day and day-of-week patterns, and predicting when execution windows — periods of low contention — are likely to occur. This allows the scheduler to not just react to current conditions but to *anticipate* them.

The forecasting layer uses a straightforward but effective approach: for each hour-of-week combination (168 buckets), it calculates the 75th percentile of historical connection percentage and CPU utilization from the signal history. These percentile values become soft thresholds that shift the precondition evaluator's effective baselines. At 3 AM on a Tuesday that historically shows 12% connection utilization, the scheduler applies more lenient preconditions and accepts more concurrent jobs. At 9 AM on a Monday that historically shows 78% utilization, the scheduler suppresses all non-critical maintenance regardless of current signals, because the forecast says the window will close within minutes even if it looks clear right now.

```

import pyodbc
import pandas as pd
from scipy import stats

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=scheduler;UID=scheduler_svc;PWD=*****;")

def build_workload_forecast():
    """
    Compute 75th percentile baselines per hour-of-week bucket
    from 30 days of signal history. Outputs a forecast table
    used by the scheduler's precondition evaluator.
    """
    with pyodbc.connect(CONN_STR) as conn:
        df = pd.read_sql("""
            SELECT
                DATEPART(WEEKDAY, collected_at) AS dow,
                DATEPART(HOUR, collected_at) AS hour_of_day,
                signal_name,
                signal_value
            FROM scheduler_signals
            WHERE collected_at > DATEADD(DAY, -30, SYSUTCDATETIME())
            AND signal_name IN (
                'active_connections_pct',
                'cpu_utilization_pct'
            )
        """, conn)

        forecast = (
            df.groupby(['dow', 'hour_of_day', 'signal_name'])['signal_value']
              .quantile(0.75)
              .reset_index()
              .rename(columns={'signal_value': 'p75_value'})
        )

        cursor = conn.cursor()
        cursor.execute("TRUNCATE TABLE scheduler_workload_forecast")
        for _, row in forecast.iterrows():
            cursor.execute("""
                INSERT INTO scheduler_workload_forecast
                    (dow, hour_of_day, signal_name, p75_value, computed_at)
                VALUES (?, ?, ?, ?, SYSUTCDATETIME())
            """, (int(row.dow), int(row.hour_of_day),
                row.signal_name, float(row.p75_value)))
        conn.commit()
        print(f"Forecast updated: {len(forecast)} hour-of-week buckets written")

if __name__ == "__main__":
    build_workload_forecast()

```

This forecast rebuilds nightly, using the last 30 days of signal history. The rolling window means the forecast adapts automatically to workload shifts — a new ETL pipeline that starts hammering the database at 11 PM will, within a few days, be reflected in the forecast and cause the scheduler to shift maintenance windows away from that slot without any human intervention.

The forecast integrates into the precondition evaluator as an *additive strictness modifier*. When the forecast for the current hour predicts high load (above configurable quantile), precondition thresholds are tightened by a multiplier (e.g., require connections below 30% instead of 40%). When the forecast predicts low load, thresholds are relaxed and the scheduler becomes more aggressive about batching eligible jobs together.

Seasonal patterns — month-end reporting, quarterly data loads — are handled by a separate calendar-annotation table where teams can mark known high-load periods. The scheduler combines statistical forecasting with explicit annotations, treating calendar annotations as hard overrides that suppress non-critical maintenance regardless of what the signal or forecast says. This gives the autonomous system a safety valve: it can be overridden by intentional human input without breaking the automated decision path.

Failure Handling, Retry Intelligence, and Job Dependency Graphs

A scheduler that cannot handle failure autonomously is just cron with extra steps. Autonomous failure handling requires the system to distinguish between different failure classes, apply class-appropriate retry strategies, and propagate failure state through job dependency chains without human intervention.

Failure classification happens at execution time. When a job's work unit raises an exception, the scheduler catches it and assigns a failure class:

- → **Transient failures**: deadlocks, lock timeouts, connection pool exhaustion — retry with backoff after a short delay
- → **Resource failures**: out-of-disk during an index rebuild, query cancelled due to statement timeout — retry after the constraint that caused the failure resolves
- → **Logic failures**: division by zero in a statistics query, a missing table — do not retry, alert, suspend the job pending human review
- → **Partial failures**: a multi-step procedure that committed some batches before failing — resume from checkpoint, not full restart


```

-- Equivalent job run tracking for SQL Server
CREATE TABLE dbo.job_runs (
    run_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    job_id          INT REFERENCES dbo.autonomous_jobs(job_id),
    started_at      DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    finished_at     DATETIME2 NULL,
    status          NVARCHAR(30) NOT NULL
        CONSTRAINT chk_status CHECK (status IN (
            'running', 'completed', 'failed_transient',
            'failed_resource', 'failed_logic',
            'suspended', 'skipped'
        )),
    failure_class   NVARCHAR(30) NULL,
    error_message   NVARCHAR(MAX) NULL,
    rows_processed  BIGINT NULL,
    retry_count     INT NOT NULL DEFAULT 0,
    next_retry_at   DATETIME2 NULL,
    checkpoint_data NVARCHAR(MAX) NULL -- JSON checkpoint for resumable jobs
        CONSTRAINT chk_checkpoint CHECK (
            checkpoint_data IS NULL OR ISJSON(checkpoint_data) = 1
        )
);

-- SQL Server failure classifier using error number ranges
CREATE OR ALTER FUNCTION dbo.ClassifyJobFailure(
    @error_number INT,
    @error_message NVARCHAR(MAX)
)
RETURNS NVARCHAR(30)
AS
BEGIN
    -- Deadlock, lock timeout
    IF @error_number IN (1205, 1222)          RETURN 'transient';
    -- Connection/pool exhaustion
    IF @error_number IN (17142, 18456, 233) RETURN 'transient';
    -- Disk full, log full
    IF @error_number IN (1105, 9002)          RETURN 'resource';
    -- Query timeout (from application timeout)
    IF @error_number = 0 AND @error_message LIKE '%timeout%'
        RETURN 'resource';
    -- Object not found, syntax, type mismatch
    IF @error_number IN (208, 207, 245, 8144)
        RETURN 'logic';
    -- Default to transient
    RETURN 'transient';
END;

```

The retry engine calculates the next retry timestamp using exponential backoff, capped by the policy's `max_delay_sec`. For resource failures it adds a signal-aware component: instead of retrying after a fixed delay, it waits until the signal that caused the failure — disk

utilization, connection count — returns to a safe level, then adds the backoff delay on top. This prevents the pathological case where a job on a fixed retry timer repeatedly attempts to run, fails immediately because the resource constraint is still active, and floods the error log.

Job dependency graphs are declared in a separate edge table and enforced by the scheduling engine before dispatch. A partition-rotation job that must complete before the archival job touches the old partition, an index rebuild that should not run in parallel with a statistics update on the same table — these are first-class relationships in the schema, not comments in a runbook:

```

-- SQL Server job dependency graph
CREATE TABLE dbo.job_dependencies (
    dependency_id      INT IDENTITY(1,1) PRIMARY KEY,
    upstream_job_id    INT REFERENCES dbo.autonomous_jobs(job_id),
    downstream_job_id  INT REFERENCES dbo.autonomous_jobs(job_id),
    dependency_type     NVARCHAR(30) NOT NULL
                        CONSTRAINT chk_dep_type CHECK (dependency_type IN (
                            'sequential', 'exclusive', 'shared_table'
                        )),
    CONSTRAINT no_self_dep CHECK (upstream_job_id != downstream_job_id)
);

-- Dependency check procedure returns 1 if job may be dispatched
CREATE OR ALTER PROCEDURE dbo.CheckJobDependencies
    @job_id      INT,
    @can_run      BIT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET @can_run = 1;

    -- Sequential: upstream must have a completed run in the last 24 hours
    IF EXISTS (
        SELECT 1
        FROM dbo.job_dependencies d
        JOIN dbo.job_runs r ON r.job_id = d.upstream_job_id
        WHERE d.downstream_job_id = @job_id
              AND d.dependency_type = 'sequential'
              AND r.status != 'completed'
              AND r.started_at > DATEADD(HOUR, -24, SYSUTCDATETIME())
    )
    BEGIN
        SET @can_run = 0; RETURN;
    END

    -- Exclusive or shared_table: upstream must not be running
    IF EXISTS (
        SELECT 1
        FROM dbo.job_dependencies d
        JOIN dbo.job_runs r ON r.job_id = d.upstream_job_id
        WHERE d.downstream_job_id = @job_id
              AND d.dependency_type IN ('exclusive', 'shared_table')
              AND r.status = 'running'
    )
    BEGIN
        SET @can_run = 0; RETURN;
    END

    -- Reverse check: downstream sibling must not be running
    IF EXISTS (
        SELECT 1
        FROM dbo.job_dependencies d
        JOIN dbo.job_runs r ON r.job_id = d.downstream_job_id

```

```

        WHERE d.upstream_job_id = @job_id
              AND d.dependency_type IN ('exclusive', 'shared_table')
              AND r.status = 'running'
    )
    BEGIN
        SET @can_run = 0; RETURN;
    END
END;

```

The combination of failure classification, signal-aware retry scheduling, and dependency enforcement means that a cascade failure — where one job's failure would previously cause a chain of cron-triggered downstream jobs to also fail — is replaced by an ordered recovery: the upstream job retries autonomously, the downstream jobs wait in a dependency-blocked state, and once the upstream job completes successfully, the downstream jobs become eligible for dispatch without any human action.

The Master Scheduler Loop: Assembling the Full Decision Cycle

All the components built so far — signal pipeline, precondition evaluator, conflict resolver, dependency checker, retry engine — compose into a single master scheduler loop. This loop is the autonomous decision cycle, and it runs on a fixed short interval (30–60 seconds), treating each iteration as a fresh evaluation of the world state.

```

graph TD
    A["Cycle start: apply forecast-adjusted budgets"] --> B["Next eligible job by priority"]
    B --> C["Preconditions met?"]
    C -->|no| D["Log: skipped_precondition"]
    C -->|yes| E["Dependencies clear?"]
    E -->|no| F["Log: skipped_dependency"]
    E -->|yes| G["Fits resource budget?"]
    G -->|no| H["Log: skipped_budget"]
    G -->|yes| I["Insert running record, allocate budget"]
    I --> J["Async dispatch: Service Broker queue"]
    J --> B

```

Every job in the priority-ordered queue passes through the same four gates each cycle — nothing runs on a clock, only on cleared preconditions, dependencies, and budget.

The scheduler sends a message to a Service Broker queue, which dispatches to a Python worker pool listening on the `scheduler_dispatch` service. Each worker picks up one dispatch message, executes the work unit, and updates the `job_runs` record. The scheduler procedure itself never blocks on job execution — the 30-second loop runs cleanly regardless of how long individual jobs take.

```

-- SQL Server master scheduler cycle using Service Broker for async dispatch
CREATE OR ALTER PROCEDURE dbo.RunSchedulerCycle
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @job_id            INT,
        @job_name           NVARCHAR(256),
        @work_unit          NVARCHAR(MAX),
        @max_duration_sec   INT,
        @estimated_io       INT,
        @estimated_cpu       DECIMAL(5,2),
        @run_id             BIGINT,
        @pre_ok             BIT,
        @dep_ok             BIT,
        @budget_io          INT          = 4096,
        @budget_cpu         DECIMAL(5,2) = 60.0,
        @allocated_io       INT          = 0,
        @allocated_cpu       DECIMAL(5,2) = 0.0;

    -- Forecast-based budget tightening
    SELECT
        @budget_io = @budget_io * CASE WHEN p75_conn > 60 THEN 0.5 ELSE 1.0 END,
        @budget_cpu = @budget_cpu * CASE WHEN p75_cpu > 60 THEN 0.5 ELSE 1.0 END
    FROM (
        SELECT
            MAX(CASE WHEN signal_name = 'active_connections_pct'
                THEN p75_value ELSE 0 END) AS p75_conn,
            MAX(CASE WHEN signal_name = 'cpu_utilization_pct'
                THEN p75_value ELSE 0 END) AS p75_cpu
        FROM dbo.scheduler_workload_forecast
        WHERE dow = DATEPART(WEEKDAY, SYSUTCDATETIME()) - 1
            AND hour_of_day = DATEPART(HOUR, SYSUTCDATETIME())
    ) f;

    -- Cursor over eligible jobs in priority order
    DECLARE job_cursor CURSOR LOCAL FAST_FORWARD FOR
        SELECT j.job_id, j.job_name, j.work_unit,
            j.max_duration_sec, j.estimated_io_mb, j.estimated_cpu_pct
        FROM dbo.autonomous_jobs j
        OUTER APPLY (
            SELECT TOP 1 status, next_retry_at
            FROM dbo.job_runs
            WHERE job_id = j.job_id
            ORDER BY started_at DESC
        ) lr
        WHERE j.enabled = 1
            AND (lr.status IS NULL
                OR (lr.status NOT IN ('running')
                    AND (lr.next_retry_at IS NULL
                        OR lr.next_retry_at <= SYSUTCDATETIME()))
            )
        ORDER BY j.priority ASC, j.estimated_io_mb ASC;

```

```

OPEN job_cursor;
FETCH NEXT FROM job_cursor
INTO @job_id, @job_name, @work_unit,
    @max_duration_sec, @estimated_io, @estimated_cpu;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Evaluate preconditions (called as scalar function inline)
    SET @pre_ok = dbo.EvaluateJobPreconditions(@job_id);

    IF @pre_ok = 0
    BEGIN
        INSERT INTO dbo.job_runs (job_id, status, started_at, finished_at)
        VALUES (@job_id, 'skipped', SYSUTCDATETIME(), SYSUTCDATETIME());
        GOTO next_job;
    END

    -- Dependency check
    EXEC dbo.CheckJobDependencies @job_id = @job_id, @can_run = @dep_ok OUTPUT;
    IF @dep_ok = 0 GOTO next_job;

    -- Resource budget check
    IF (@allocated_io + ISNULL(@estimated_io, 0)) > @budget_io
    OR (@allocated_cpu + ISNULL(@estimated_cpu, 0)) > @budget_cpu
    BEGIN
        INSERT INTO dbo.job_runs (job_id, status, started_at, finished_at,
            error_message)
        VALUES (@job_id, 'skipped', SYSUTCDATETIME(), SYSUTCDATETIME(),
            'resource_budget_exhausted');
        GOTO next_job;
    END

    -- Insert running record
    INSERT INTO dbo.job_runs (job_id, status, started_at)
    VALUES (@job_id, 'running', SYSUTCDATETIME());
    SET @run_id = SCOPE_IDENTITY();

    -- Accumulate budget usage
    SET @allocated_io = @allocated_io + ISNULL(@estimated_io, 0);
    SET @allocated_cpu = @allocated_cpu + ISNULL(@estimated_cpu, 0);

    -- Dispatch via Service Broker for async execution
    DECLARE @msg NVARCHAR(MAX) = (
        SELECT @run_id AS run_id, @job_id AS job_id,
            @work_unit AS work_unit,
            @max_duration_sec AS max_duration_sec
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
    );
    EXEC dbo.DispatchJobToServiceBroker @message = @msg;

next_job:
    FETCH NEXT FROM job_cursor

```

```
        INTO @job_id, @job_name, @work_unit,  
            @max_duration_sec, @estimated_io, @estimated_cpu;  
    END  
  
    CLOSE job_cursor;  
    DEALLOCATE job_cursor;  
END;
```

Both implementations share the same decision structure: forecast-adjusted budgets → precondition evaluation → dependency check → resource budget gate → async dispatch. The async dispatch layer is critical. Synchronous execution inside the scheduler loop would cause the loop to block behind slow-running maintenance jobs, defeating the point of a dynamic decision cycle.

Observability: Audit Trails, Decision Logs, and Scheduler Introspection

An autonomous scheduler making decisions invisibly is an operations nightmare. Every decision — run, skip, defer, suspend, retry — must be logged with enough context to reconstruct why the system chose what it did. The `job_runs` table provides execution history, but the scheduler also needs a separate decision log that captures the signal values and precondition outcomes at the moment of each scheduling decision. When an operator asks "why didn't the index rebuild run last night?", they should get a precise answer from structured data, not a guess.

```

-- Scheduler decision audit log (SQL Server)
CREATE TABLE dbo.scheduler_decision_log (
    decision_id      BIGINT IDENTITY(1,1) PRIMARY KEY,
    decided_at       DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    job_id           INT REFERENCES dbo.autonomous_jobs(job_id),
    decision          NVARCHAR(30) NOT NULL CHECK (decision IN (
        'dispatched', 'skipped_precondition',
        'skipped_dependency', 'skipped_budget',
        'deferred_retry', 'suspended'
    )),
    signal_snapshot  NVARCHAR(MAX),      -- JSON: signal values at decision time
    reason_detail    NVARCHAR(MAX)
);

-- Helper to snapshot current signal values for a job's preconditions
CREATE OR ALTER FUNCTION dbo.fn_snapshot_precondition_signals(@p_job_id INT)
RETURNS NVARCHAR(MAX)
AS
BEGIN
    -- NOTE: preconditions are stored as JSON array; per-precondition signal
    -- lookup is handled row-wise by the caller via OPENJSON
    RETURN (
        SELECT (
            SELECT s.signal_name,
                   s.signal_value,
                   s.collected_at,
                   DATEDIFF(SECOND, s.collected_at, SYSUTCDATETIME()) AS age_seconds
            FROM dbo.autonomous_jobs j
            CROSS APPLY OPENJSON(j.preconditions)
                WITH (signal NVARCHAR(100) '$.signal') AS pc
            CROSS APPLY (
                SELECT TOP 1 signal_name, signal_value, collected_at
                FROM dbo.scheduler_signals
                WHERE signal_name = pc.signal
                ORDER BY collected_at DESC
            ) s
            WHERE j.job_id = @p_job_id
            FOR JSON PATH
        )
    );
END;

```

The `signal_snapshot` column stores the exact state the evaluator saw when it made its decision. When queried later, an operator can see: "At 02:14 UTC, `active_connections_pct` was 43.2 (threshold was 40), so the precondition failed by 3.2 percentage points." This kind of precision turns autonomous behavior from a black box into an auditable system that builds operational trust over time.

A live introspection view aggregates across the decision log and job run history to give a real-time picture of scheduler health:


```

-- Scheduler health introspection view (SQL Server)
CREATE OR ALTER VIEW dbo.v_scheduler_health AS
WITH recent_decisions AS (
    SELECT
        j.job_name,
        SUM(CASE WHEN dl.decision = 'dispatched'      THEN 1 ELSE 0 END) AS
dispatched_24h,
        SUM(CASE WHEN dl.decision LIKE 'skipped%'      THEN 1 ELSE 0 END) AS
skipped_24h,
        SUM(CASE WHEN dl.decision = 'deferred_retry' THEN 1 ELSE 0 END) AS
retries_24h,
        MAX(CASE WHEN dl.decision = 'dispatched' THEN dl.decided_at END) AS
last_dispatched
    FROM dbo.scheduler_decision_log dl
    JOIN dbo.autonomous_jobs j ON j.job_id = dl.job_id
    WHERE dl.decided_at > DATEADD(HOUR, -24, SYSUTCDATETIME())
    GROUP BY j.job_name
),
recent_runs AS (
    SELECT
        j.job_name,
        SUM(CASE WHEN r.status = 'completed'          THEN 1 ELSE 0 END) AS
completed_24h,
        SUM(CASE WHEN r.status LIKE 'failed%'          THEN 1 ELSE 0 END) AS failed_24h,
        AVG(CASE WHEN r.status = 'completed'
            THEN DATEDIFF(SECOND, r.started_at, r.finished_at) END)
            AS avg_duration_sec
    FROM dbo.job_runs r
    JOIN dbo.autonomous_jobs j ON j.job_id = r.job_id
    WHERE r.started_at > DATEADD(HOUR, -24, SYSUTCDATETIME())
    GROUP BY j.job_name
)
SELECT
    d.job_name,
    d.dispatched_24h,
    d.skipped_24h,
    d.retries_24h,
    r.completed_24h,
    r.failed_24h,
    ROUND(r.avg_duration_sec, 1) AS avg_duration_sec,
    d.last_dispatched,
    CASE
        WHEN r.failed_24h > 3                      THEN 'degraded'
        WHEN d.dispatched_24h = 0
            AND d.skipped_24h > 20                  THEN 'starved'
        WHEN d.dispatched_24h > 0
            AND r.completed_24h = 0                  THEN 'dispatch_stuck'
        ELSE 'healthy'
    END AS health_status
FROM recent_decisions d
LEFT JOIN recent_runs r ON r.job_name = d.job_name;

```

The `health_status` column automatically surfaces three pathological states: `degraded` (too many failures), `starved` (a job keeps getting skipped, suggesting its preconditions are chronically unmet or too tight), and `dispatch_stuck` (jobs are being dispatched but never completing, indicating a problem in the execution layer). These states can drive alerts through standard monitoring pipelines without requiring a human to manually review scheduler logs.

Production Hardening: Scheduler Self-Protection and Graceful Degradation

A production autonomous scheduler must protect itself from three failure modes that can cause more damage than the problems it is meant to solve: runaway jobs that consume excessive resources, scheduler loop failures that silently stop all maintenance, and feedback loops where the scheduler's own activity becomes a signal that suppresses further scheduling.

Runaway job protection is implemented as a watchdog that runs independently of the main scheduler loop. It periodically scans `job_runs` for entries that have been in `running` state longer than the job's `max_duration_sec`, and issues a cancellation:

```

-- Scheduler watchdog: suspend jobs exceeding their time budget (SQL Server)
CREATE OR ALTER PROCEDURE dbo.usp_scheduler_watchdog
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @run_id BIGINT, @job_id INT, @job_name NVARCHAR(200);
    DECLARE @max_duration_sec INT, @elapsed_sec BIGINT;

    DECLARE run_cursor CURSOR FOR
    SELECT r.run_id, r.job_id, j.job_name, j.max_duration_sec,
           DATEDIFF(SECOND, r.started_at, SYSUTCDATETIME())
    FROM dbo.job_runs r
    JOIN dbo.autonomous_jobs j ON j.job_id = r.job_id
    WHERE r.status = 'running'
           AND DATEDIFF(SECOND, r.started_at, SYSUTCDATETIME()) > j.max_duration_sec;

    OPEN run_cursor;
    FETCH NEXT FROM run_cursor
        INTO @run_id, @job_id, @job_name, @max_duration_sec, @elapsed_sec;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Attempt graceful cancellation: KILL the worker session
        -- The worker registers its session_id in job_runs.worker_session_id
        DECLARE @sid INT = (
            SELECT worker_session_id
            FROM dbo.job_runs
            WHERE run_id = @run_id
        );
        IF @sid IS NOT NULL
        BEGIN
            BEGIN TRY
                EXEC('KILL ' + @sid);
            END TRY
            BEGIN CATCH
                -- Session already gone: treat as already suspended
            END CATCH
        END

        UPDATE dbo.job_runs
        SET status = 'suspended',
            finished_at = SYSUTCDATETIME(),
            failure_class = 'resource',
            error_message = 'Exceeded max_duration_sec ('
                            + CAST(@max_duration_sec AS NVARCHAR)
                            + '). Elapsed: '
                            + CAST(@elapsed_sec AS NVARCHAR) + ' s'
        WHERE run_id = @run_id
            AND status = 'running';

        PRINT 'Watchdog suspended job ' + @job_name
            + ' (run_id=' + CAST(@run_id AS NVARCHAR)

```

```

        + ') after ' + CAST(@elapsed_sec AS NVARCHAR) + ' seconds';

    FETCH NEXT FROM run_cursor
    INTO @run_id, @job_id, @job_name, @max_duration_sec, @elapsed_sec;
END

CLOSE run_cursor;
DEALLOCATE run_cursor;
END;

```

Scheduler loop liveness is monitored by a heartbeat mechanism. The scheduler loop writes a timestamp to a `scheduler_heartbeat` table at the end of each successful cycle. A separate lightweight check — running as a true cron entry or a cloud health check endpoint — verifies that the heartbeat is recent. If the scheduler has been silent for more than three cycle intervals, it triggers an alert and attempts a self-restart through the process supervisor. The bootstrap cron entry that calls `run_scheduler_cycle()` every 60 seconds is thus reduced to exactly two responsibilities: collecting signals and verifying the scheduler's own heartbeat. If that minimal bootstrap cron fails, the impact is limited and recoverable.

Feedback loop prevention addresses a subtle failure mode: the scheduler running a noisy maintenance job that elevates connection counts or CPU, which causes the scheduler's own signal pipeline to read high utilization, which suppresses further scheduling, which allows maintenance debt to accumulate, which makes future maintenance more expensive when it eventually runs. The prevention mechanism exempts the scheduler's own signal collection and decision processes from the signal-based precondition evaluations, and tags scheduler-initiated connections in `sys.dm_exec_sessions` with an `program_name` prefix (`scheduler_worker_`) so the connection percentage calculation can exclude them from the utilization signal.

Key Takeaways

- → **Intent replaces timing**: autonomous job scheduling encodes what a job needs to be true before it runs — signal preconditions, dependency state, resource budgets — rather than when on the clock it should execute. This eliminates stale assumptions and enables the system to defer, batch, or suppress work based on real conditions.
- → **Signal quality determines decision quality**: the scheduler is only as intelligent as its inputs. A tiered signal pipeline — high-frequency DMV signals, lower-frequency semantic signals, and external cluster signals via Prometheus — gives the decision engine the context it needs to distinguish a genuinely idle window from a deceptively quiet moment before a load spike.

- → **Failure classification drives proportionate recovery:** treating all job failures as equivalent leads to either aggressive retrying of logic errors (which will never succeed) or under-retrying of transient lock conflicts (which resolve quickly). Classifying failures at catch time — transient, resource, logic, partial — and applying class-specific retry strategies with signal-aware backoff makes recovery both faster and safer.
- → **Async dispatch decouples decision cycles from execution time:** the scheduler loop must never block waiting for a job to finish. Using Service Broker keeps the 30-second decision cycle clean regardless of whether a triggered index rebuild takes 45 seconds or 45 minutes.
- → **Observability is the trust mechanism:** an autonomous system that makes decisions invisibly will be overridden by humans the moment something unexpected happens. Logging every scheduling decision with its signal snapshot, surfacing scheduler health through introspection views, and detecting pathological states like job starvation or dispatch-stuck conditions transforms autonomous behavior from opaque automation into a legible, auditable system that operators can reason about and trust.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 25: AI-Driven Change Management

Autonomous systems that can observe and decide are only as valuable as their ability to act — and acting on a production database system means managing change. Change management has historically been the domain of human judgment, CAB meetings, deployment windows, and rollback plans drafted at midnight. At the ADE level, the goal is to replace that human-in-the-loop bottleneck with a system that classifies risk, selects strategy, executes with precision, and rolls back autonomously when signals deteriorate. This chapter covers the full architecture: how an autonomous change management system ingests proposed changes, scores their risk against live telemetry, selects an execution strategy, instruments the rollout, and either completes or self-heals — without paging anyone at 2 AM.

Risk Scoring Engines: Classifying Change Before Execution

Every autonomous change pipeline begins with a classification gate. Before a DDL statement, configuration push, or schema migration touches a production instance, it must pass through a risk scoring engine that combines static analysis of the change itself with dynamic analysis of the current system state.

Static analysis examines the change in isolation. A `CREATE INDEX ... WITH (ONLINE = ON)` on a 50-million-row table is categorically different from the same operation on a 50-thousand-row table. An `ALTER TABLE ... ADD COLUMN` with a default value on SQL Server 2012+ is a metadata-only operation and trivially safe; the same statement on earlier versions can rewrite the table. A SQL Server `ALTER TABLE ... ALTER COLUMN` on a column with dependent indexed views is a blocking operation regardless of row count. The static analyzer must know these rules and apply them without human instruction.

Dynamic analysis examines what the system is doing right now. A change that scores "medium risk" at 3 AM may score "critical risk" at 2 PM when the instance is processing 40,000 transactions per second with 85% buffer cache utilization. The scoring engine must pull live metrics at decision time, not at submission time.

```
-- Equivalent live risk context for SQL Server change scoring
CREATE OR ALTER VIEW autonomous_change.live_risk_context AS
SELECT
    (SELECT COUNT(*) FROM sys.dm_exec_requests WHERE blocking_session_id <> 0) AS
active_blocked_sessions,
    (SELECT COUNT(*) FROM sys.dm_exec_sessions WHERE is_user_process = 1) AS
active_connections,
    (SELECT CAST(cntr_value AS FLOAT)
    FROM sys.dm_os_performance_counters
    WHERE counter_name = 'Buffer cache hit ratio'
    AND object_name LIKE '%Buffer Manager%') AS buffer_cache_hit_ratio,
    (SELECT MAX(DATEDIFF(SECOND, transaction_begin_time, GETDATE()))
    FROM sys.dm_tran_active_transactions
    WHERE transaction_type = 1) AS oldest_txn_seconds,
    (SELECT MAX(secondary_lag_seconds)
    FROM sys.dm_hadr_database_replica_states
    WHERE is_local = 0) AS max_replica_lag_seconds,
    (SELECT COUNT(*) FROM sys.dm_exec_requests WHERE wait_type LIKE 'LCK%') AS
lock_wait_count;
```

The scoring engine combines static weights with dynamic multipliers. A Python-based scorer might represent this as:

```

import pyodbc
import json
from dataclasses import dataclass
from enum import Enum
from typing import Optional

class RiskTier(Enum):
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"

@dataclass
class ChangeProposal:
    change_id: str
    change_type: str          # ddl, config, data_migration, index
    target_table: str
    estimated_row_count: int
    statement: str
    requires_lock: bool
    lock_level: str          # row, page, table, exclusive

def score_change_risk(proposal: ChangeProposal, db_conn) -> dict:
    # Static base score
    static_score = 0

    static_rules = {
        ("ddl", "exclusive"): 40,
        ("ddl", "table"): 30,
        ("index", "table"): 20,
        ("index", "row"): 5,
        ("config", "none"): 10,
        ("data_migration", "row"): 15,
    }

    static_score += static_rules.get((proposal.change_type, proposal.lock_level),
10)

    # Row count multiplier
    if proposal.estimated_row_count > 100_000_000:
        static_score *= 2.5
    elif proposal.estimated_row_count > 10_000_000:
        static_score *= 1.8
    elif proposal.estimated_row_count > 1_000_000:
        static_score *= 1.3

    # Dynamic context multipliers
    with db_conn.cursor() as cur:
        cur.execute("SELECT * FROM autonomous_change.live_risk_context")
        row = cur.fetchone()
        cols = [desc[0] for desc in cur.description]
        context = dict(zip(cols, row))

    dynamic_multiplier = 1.0

```



```

    if context["active_blocked_sessions"] > 5:
        dynamic_multiplier += 0.4
    if context["buffer_cache_hit_ratio"] and context["buffer_cache_hit_ratio"] <
90.0:
        dynamic_multiplier += 0.3
    if context["oldest_txn_seconds"] and context["oldest_txn_seconds"] > 30:
        dynamic_multiplier += 0.5
    if context["max_replica_lag_seconds"] and context["max_replica_lag_seconds"] >
60:
        dynamic_multiplier += 0.6
    if context["lock_wait_count"] and context["lock_wait_count"] > 10:
        dynamic_multiplier += 0.8 # Lock accumulation means DDL may queue badly

    final_score = static_score * dynamic_multiplier

    if final_score < 20:
        tier = RiskTier.LOW
    elif final_score < 45:
        tier = RiskTier.MEDIUM
    elif final_score < 75:
        tier = RiskTier.HIGH
    else:
        tier = RiskTier.CRITICAL

    return {
        "change_id": proposal.change_id,
        "static_score": static_score,
        "dynamic_multiplier": dynamic_multiplier,
        "final_score": final_score,
        "risk_tier": tier.value,
        "context_snapshot": context,
        "autonomous_action": _select_autonomous_action(tier, context)
    }

def _select_autonomous_action(tier: RiskTier, context: dict) -> str:
    if tier == RiskTier.LOW:
        return "execute_immediately"
    elif tier == RiskTier.MEDIUM:
        return "execute_with_monitoring"
    elif tier == RiskTier.HIGH:
        return "defer_to_maintenance_window"
    else: # CRITICAL
        if context.get("active_blocked_sessions", 0) > 10:
            return "block_and_escalate"
        return "block_and_defer_72h"

```

This scoring system is the gatekeeper. Every change proposal — whether generated by a CI/CD pipeline, a human DBA via a migration tool, or by the autonomous system itself as part of an optimization cycle — passes through this gate before touching production.

Autonomous Migration Execution: Online Schema Changes Without Human Oversight

Once a change clears the risk gate at an acceptable tier, the execution layer takes over. The most dangerous class of database change is DDL on large tables, and the autonomous execution layer must have first-class support for online schema change strategies — not as a fallback, but as the default path.

For SQL Server, online index operations (`WITH (ONLINE = ON)`), resumable index rebuilds, and online column operations serve this purpose. An online index build holds only brief schema locks at the start and end of the operation, so concurrent reads and writes continue throughout. The autonomous execution layer defaults to online variants for any change on a table with live traffic, falling back to offline execution only when the risk scorer approves a maintenance-window exception.

The SQL Server Agent or an external orchestrator manages the execution, with Always On Availability Group lag monitoring integrated into the execution loop.

The autonomous execution controller wraps these operations in an instrumented harness:

```

-- SQL Server execution audit equivalent
CREATE TABLE autonomous_change.execution_log (
    execution_id      UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    change_id         NVARCHAR(200) NOT NULL,
    statement         NVARCHAR(MAX) NOT NULL,
    risk_tier         NVARCHAR(20) NOT NULL,
    strategy          NVARCHAR(50) NOT NULL,
    started_at        DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
    completed_at      DATETIMEOFFSET NULL,
    status            NVARCHAR(20) DEFAULT 'running',
    pre_metrics       NVARCHAR(MAX) NULL,    -- JSON
    post_metrics      NVARCHAR(MAX) NULL,    -- JSON
    abort_reason      NVARCHAR(MAX) NULL,
    rows_affected     BIGINT NULL
);
GO

-- Metrics snapshot for SQL Server boundary capture
CREATE OR ALTER PROCEDURE autonomous_change.capture_metrics_snapshot
    @snapshot NVARCHAR(MAX) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT @snapshot = (
        SELECT
            SYSDATETIMEOFFSET() AS timestamp,
            (SELECT SUM(cnt_value) FROM sys.dm_os_performance_counters
             WHERE counter_name = 'Batch Requests/sec') AS batch_requests_sec,
            (SELECT COUNT(*) FROM sys.dm_exec_sessions WHERE is_user_process = 1) AS
active_connections,
            (SELECT MAX(secondary_lag_seconds) FROM
sys.dm_hadr_database_replica_states WHERE is_local = 0) AS max_ag_lag_seconds,
            (SELECT TOP 1 total_elapsed_time / 1000 FROM sys.dm_exec_requests ORDER
BY total_elapsed_time DESC) AS longest_request_ms
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER
    );
END;

```

The execution harness runs the change inside a monitoring loop. Every 10 seconds during a long-running operation, it samples progress, checks replica lag, checks for lock accumulation, and compares against pre-execution baselines. If any metric degrades beyond a configured threshold, the operation is aborted and the abort reason is logged.

For batched data migrations — moving data in chunks, backfilling columns, or archiving records — the autonomous executor uses a self-throttling loop that adjusts batch size based on observed write latency:

```

import time
import pyodbc
from typing import Callable

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=prod;UID=migration_svc;PWD=*****;")

def autonomous_batched_migration(
    batch_query: str,          # Must accept ? for batch_size and last_id
    count_query: str,
    max_lag_seconds: int = 60,
    target_batch_duration_ms: int = 500,
    initial_batch_size: int = 1000,
    max_batch_size: int = 50_000,
    min_batch_size: int = 100,
    execution_id: str = None
):
    """
    Self-throttling batched migration executor.
    Adjusts batch size dynamically based on actual execution duration
    and AG replica lag, ensuring the migration stays within safe thresholds.
    """
    conn = pyodbc.connect(CONN_STR, autocommit=True)

    batch_size = initial_batch_size
    last_id = 0
    total_rows = 0
    consecutive_lag_violations = 0

    cur = conn.cursor()
    cur.execute(count_query)
    total_estimated = cur.fetchone()[0]

    print(f"[{execution_id}] Starting migration. Estimated rows: {total_estimated}")

    while True:
        # Check AG replica lag before each batch
        cur.execute("""
            SELECT COALESCE(MAX(DATEDIFF(SECOND, last_commit_time, GETUTCDATE())),
0)
            FROM sys.dm_hadr_database_replica_states
            WHERE is_local = 0
        """)
        current_lag = cur.fetchone()[0]

        if current_lag > max_lag_seconds:
            consecutive_lag_violations += 1
            wait_seconds = min(2 ** consecutive_lag_violations, 60)
            print(f"[{execution_id}] Replica lag {current_lag:,} seconds exceeds
threshold. "
                f"Backing off {wait_seconds}s (violation
#{consecutive_lag_violations})")
            time.sleep(wait_seconds)

```

```

        # Shrink batch size aggressively when lag builds
        batch_size = max(min_batch_size, batch_size // 2)
        continue
    else:
        consecutive_lag_violations = 0

    # Execute batch and measure duration
    batch_start = time.monotonic()
    cur.execute(batch_query, (batch_size, last_id))
    rows_processed = cur.rowcount
    batch_duration_ms = (time.monotonic() - batch_start) * 1000

    if rows_processed == 0:
        print(f"[{execution_id}] Migration complete. Total rows processed:
{total_rows}")
        break

    total_rows += rows_processed
    last_id += rows_processed # Assumes sequential ID or adjust per schema

    # Self-tuning batch size based on duration feedback
    if batch_duration_ms > 0:
        adjustment_ratio = target_batch_duration_ms / batch_duration_ms
        new_batch_size = int(batch_size * adjustment_ratio)
        batch_size = max(min_batch_size, min(max_batch_size, new_batch_size))

    progress_pct = round(total_rows / total_estimated * 100, 1) if
total_estimated > 0 else 0
    print(f"[{execution_id}] Batch complete: {rows_processed} rows in
{batch_duration_ms:.0f}ms. "
          f"New batch size: {batch_size}. Progress: {progress_pct}%")

    # Brief yield between batches to reduce contention
    time.sleep(0.05)

conn.close()

```

This pattern means that a 500-million-row backfill migration runs without any human watching it. If the replica falls behind, the system slows down. If the replica catches up and the database is underloaded, the system speeds up. The batch size converges on the fastest rate that keeps lag within bounds.

AI-Assisted Change Plan Generation: From Intent to Executable Strategy

Risk scoring and execution harnesses handle the mechanics. But in a fully autonomous system, the change plan itself should not always originate from a human writing SQL. The system needs the ability to translate high-level intents — "this query is slow because it lacks an appropriate index" or "this table is bloated by 60% and needs a repack" — into concrete, sequenced, risk-scored change plans.

This is where large language model integration moves from a developer convenience into production infrastructure. The autonomous optimizer identifies a problem, describes it in structured natural language, and queries an LLM API to generate a candidate change plan. That plan is then parsed, statically analyzed by the risk scorer, and submitted through the same execution pipeline as any human-authored migration.

The integration requires strict guardrails. The LLM output is never executed directly. Every generated statement is passed through a SQL parser, validated against schema, compared against a catalog of known-dangerous patterns, and scored. Only then does it enter the execution queue.

```

import openai
import sqlglot
import re
from typing import Optional

DANGEROUS_PATTERNS = [
    r'\bDROP\s+TABLE\b',
    r'\bTRUNCATE\b',
    r'\bDELETE\s+FROM\b(?:!.*\bWHERE\b)', # DELETE without WHERE
    r'\bUPDATE\b(?:!.*\bWHERE\b)',        # UPDATE without WHERE
    r'\bDROP\s+DATABASE\b',
    r'\bDROP\s+SCHEMA\b',
]

def generate_change_plan_from_intent(
    intent: str,
    schema_context: str,
    db_platform: str = "sqlserver" # SQL Server only
    model: str = "gpt-4o"
) -> Optional[dict]:
    """
    Generates a structured change plan from a natural-language optimization intent.
    Output is a JSON object with statements, sequencing, and reversibility metadata.
    All output is validated before entering the execution pipeline.
    """

    system_prompt = f"""You are a database change plan generator for {db_platform}.

    Given an optimization intent and schema context, produce a JSON change plan with
    this exact structure:
    {{
      "intent_summary": "...",
      "statements": [
        {{
          "sequence": 1,
          "statement": "...",
          "change_type": "ddl|index|config|data_migration",
          "lock_level": "none|row|page|table|exclusive",
          "reversible": true|false,
          "reversal_statement": "..." or null,
          "online_safe": true|false,
          "estimated_duration_hint": "seconds|minutes|hours"
        }}
      ],
      "requires_maintenance_window": true|false,
      "dependencies": []
    }}

    Rules:
    - Prefer ONLINE = ON operations wherever SQL Server supports them
    - Always include reversal_statement for DDL that has a clean inverse
    - Never generate DROP TABLE, TRUNCATE, or DELETE without WHERE
    - For {db_platform}, use correct syntax precisely
    """

```

- Output ONLY valid JSON, no explanation text""

```
response = openai.chat.completions.create(
    model=model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": f"Intent: {intent}\n\nSchema context:
\n{schema_context}"}
    ],
    temperature=0.1, # Low temperature for deterministic, precise SQL output
    response_format={"type": "json_object"}
)

raw_plan = response.choices[0].message.content

try:
    import json
    plan = json.loads(raw_plan)
except json.JSONDecodeError as e:
    print(f"LLM returned unparseable JSON: {e}")
    return None

# Validate every statement before returning
validated_statements = []
for stmt_obj in plan.get("statements", []):
    statement = stmt_obj.get("statement", "")

    # Pattern-based safety check
    for pattern in DANGEROUS_PATTERNS:
        if re.search(pattern, statement, re.IGNORECASE):
            print(f"BLOCKED: Dangerous pattern detected in generated statement:
{statement[:100]}")
            return None

    # Syntax validation via sqlglot
    try:
        dialect = "tsql" # SQL Server dialect for sqlglot validation
        parsed = sqlglot.parse(statement, dialect=dialect)
        if not parsed:
            print(f"BLOCKED: Statement failed SQL parsing: {statement[:100]}")
            return None
    except sqlglot.errors.ParseError as e:
        print(f"BLOCKED: SQL parse error in generated statement: {e}")
        return None

    validated_statements.append(stmt_obj)

plan["statements"] = validated_statements
plan["generated_by"] = "llm"
plan["validated"] = True

return plan
```


The schema context passed to this function is not a raw dump of all DDL. It is a curated, minimal representation: the target table's column definitions, existing indexes, approximate row count, and the specific query or metric that triggered the optimization intent. Sending a full schema dump to an LLM API is both expensive and produces worse results than targeted context.

Rollback Orchestration: Autonomous Recovery When Deployments Go Wrong

A change management system without autonomous rollback is not autonomous — it is merely automated. The difference is consequential. Automation runs a script. Autonomy decides when the script has produced a bad outcome and reverses it without being told to.

Autonomous rollback requires three things: a pre-execution baseline, continuous signal monitoring during execution, and a reversibility map that knows how to undo every change type the system can apply.

Some changes are trivially reversible: creating an index can be reversed by dropping it; adding a column with a default can be reversed by dropping the column; changing a configuration parameter can be reversed by restoring the previous value. Other changes are not reversible in-place: once an online table rebuild completes, there is no "un-rebuild." For those, the rollback strategy is not reversal but rather pre-execution snapshot validation — ensuring a point-in-time restore target exists before the operation begins.

```

CREATE TABLE autonomous_change.rollback_registry (
    change_type          NVARCHAR(50) NOT NULL,
    lock_level           NVARCHAR(20) NOT NULL,
    reversal_strategy     NVARCHAR(50) NOT NULL,
    max_rollback_window_minutes INT,
    PRIMARY KEY (change_type, lock_level)
);

INSERT INTO autonomous_change.rollback_registry VALUES
    ('index',          'table',      'invert_ddl',      10080),
    ('index',          'row',        'invert_ddl',      10080),
    ('ddl',            'exclusive',  'invert_ddl',      60),
    ('ddl',            'table',      'invert_ddl',      60),
    ('config',         'none',       'restore_config',  43200),
    ('data_migration', 'row',        'restore_snapshot', 1440);

CREATE TABLE autonomous_change.execution_baselines (
    execution_id          UNIQUEIDENTIFIER NOT NULL
        REFERENCES autonomous_change.execution_log(execution_id),
    captured_at           DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
    tps_baseline           DECIMAL(18,2),
    p99_latency_ms        DECIMAL(18,2),
    error_rate_per_min     DECIMAL(18,4),
    cache_hit_ratio        DECIMAL(5,4),
    replication_lag_seconds INT,
    active_connections     INT,
    PRIMARY KEY (execution_id)
);

```

The rollback decision engine runs as a background monitor during every active execution. It samples current metrics on a short interval and compares them against the baseline using configurable degradation thresholds:

```

import time
import pyodbc
from dataclasses import dataclass
from typing import Optional

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=prod;UID=change_svc;PWD=*****;")

@dataclass
class RollbackSignal:
    triggered: bool
    reason: str
    severity: str # warning, abort
    metric_name: str
    baseline_value: float
    current_value: float
    threshold_pct: float

def evaluate_rollback_signals(
    execution_id: str,
    thresholds: dict = None
) -> Optional[RollbackSignal]:
    """
    Compares current system metrics against pre-execution baseline.
    Returns a RollbackSignal if any metric has degraded beyond threshold.
    """
    if thresholds is None:
        thresholds = {
            "tps_drop_pct": 30.0, # Abort if TPS drops more than 30%
            "p99_latency_increase_pct": 50.0, # Abort if P99 latency grows more
than 50%
            "cache_hit_ratio_drop_pct": 5.0, # Abort if cache hit ratio falls more
than 5 points
            "replica_lag_seconds_max": 120, # Abort if AG lag exceeds 2 minutes
            "error_rate_multiplier": 3.0, # Abort if error rate triples
        }

    conn = pyodbc.connect(CONN_STR)
    try:
        cur = conn.cursor()
        # Fetch baseline
        cur.execute("""
            SELECT tps_baseline, p99_latency_ms, error_rate_per_min,
                   cache_hit_ratio, replica_lag_seconds
            FROM autonomous_change.execution_baselines
            WHERE execution_id = ?
            """, (execution_id,))
        baseline_row = cur.fetchone()
        if not baseline_row:
            return None
        tps_base, p99_base, err_base, cache_base, lag_base = baseline_row

        # Sample current metrics from SQL Server DMVs

```

```

cur.execute("""
    SELECT
        (SELECT cntr_value
         FROM sys.dm_os_performance_counters
         WHERE counter_name = 'Batch Requests/sec') AS current_tps,
        (SELECT CAST(cntr_value AS FLOAT)
         FROM sys.dm_os_performance_counters
         WHERE counter_name = 'Buffer cache hit ratio'
          AND object_name LIKE '%Buffer Manager%') AS current_cache_hit,
        (SELECT COALESCE(MAX(DATEDIFF(SECOND, last_commit_time,
GETUTCDATE()))), 0)
        FROM sys.dm_hadr_database_replica_states
        WHERE is_local = 0) AS current_lag_seconds
""")
current = cur.fetchone()
current_tps, current_cache, current_lag = current

# Evaluate each signal
if tps_base and tps_base > 0 and current_tps is not None:
    drop_pct = ((tps_base - current_tps) / tps_base) * 100
    if drop_pct > thresholds["tps_drop_pct"]:
        return RollbackSignal(
            triggered=True,
            reason=f"TPS dropped {drop_pct:.1f}% from baseline ({tps_base:.
0f} → {current_tps:.0f})",
            severity="abort",
            metric_name="tps",
            baseline_value=float(tps_base),
            current_value=float(current_tps),
            threshold_pct=thresholds["tps_drop_pct"]
        )

    if current_lag is not None and current_lag >
thresholds["replica_lag_seconds_max"]:
        return RollbackSignal(
            triggered=True,
            reason=f"AG replica lag {current_lag:,} seconds exceeds abort
threshold",
            severity="abort",
            metric_name="replica_lag_seconds",
            baseline_value=float(lag_base or 0),
            current_value=float(current_lag),
            threshold_pct=0.0
        )

    if cache_base and current_cache is not None:
        cache_drop = float(cache_base) - float(current_cache)
        if cache_drop > thresholds["cache_hit_ratio_drop_pct"]:
            return RollbackSignal(
                triggered=True,
                reason=f"Buffer cache hit ratio dropped {cache_drop:.1f} points
from baseline",
                severity="abort",

```

```

        metric_name="cache_hit_ratio",
        baseline_value=float(cache_base),
        current_value=float(current_cache),
        threshold_pct=thresholds["cache_hit_ratio_drop_pct"]
    )

    return None # No rollback signal triggered

finally:
    conn.close()

def execute_rollback(
    execution_id: str,
    reversal_statement: str,
    signal: RollbackSignal
) -> bool:
    """
    Executes the reversal statement for a triggered rollback signal,
    updates the execution log, and emits a structured audit record.
    """
    conn = pyodbc.connect(CONN_STR, autocommit=True)
    try:
        cur = conn.cursor()
        # Execute the reversal
        cur.execute(reversal_statement)

        # Update execution log
        cur.execute("""
            UPDATE autonomous_change.execution_log
            SET status = 'rolled_back',
                completed_at = SYSUTCDATETIME(),
                abort_reason = ?
            WHERE execution_id = ?
            """, (signal.reason, execution_id))

        print(f"[ROLLBACK] execution_id={execution_id} | reason={signal.reason} | "
              f"metric={signal.metric_name} | "
              f"baseline={signal.baseline_value} | current={signal.current_value}")
        return True

    except Exception as e:
        print(f"[ROLLBACK FAILED] execution_id={execution_id} | error={e}")
        return False
    finally:
        conn.close()

```

The monitoring loop that drives this engine runs at the orchestrator level, calling `evaluate_rollback_signals` every few seconds for any in-flight execution. The moment a signal is returned, the loop calls `execute_rollback` with the pre-stored reversal statement.

For operations with no clean reversal — such as a completed partition merge — the abort path instead halts any remaining follow-on steps and flags the execution for human review, since the principal change cannot be undone by SQL alone.

Canary Deployment Patterns for Schema Changes

Not all schema changes are single-shot. In distributed application environments where application code and database schema evolve together, the autonomous change system must support the database side of a canary deployment — where a schema change is applied in a way that remains backward-compatible with both the current and next version of the application simultaneously.

This is the expand-contract (also called parallel change) pattern formalized into the execution engine. Rather than applying a breaking schema change in one step, the autonomous system sequences it across three phases:

Phase 1 – Expand: Add new structures (columns, tables, indexes) without removing or altering existing ones. The old application version continues to function; the new version can begin writing to the new structures.

Phase 2 – Migrate: Backfill existing data into the new structures using the self-throttling batched executor. Both application versions operate concurrently during this phase.

Phase 3 – Contract: Once the canary application deployment has stabilized and telemetry confirms correctness, remove the old structures.

```
graph LR
    A[Phase 1: Expand  
add new structures] -->|Gate: 15min telemetry,  
error rate + latency OK?| G1
    G1 -->|passed| B[Phase 2: Migrate  
self-throttled backfill]
    G1 -->|failed| RB1[Rollback phase 1]
    B -->|Gate: telemetry OK?| G2
    G2 -->|passed| C[Phase 3: Contract  
remove old structures]
    G2 -->|failed| RB2[Rollback phase 2]
    C -->|Gate: stricter thresholds| G3
    G3 -->|passed| Done[Deployment complete]
    G3 -->|failed| RB3[Rollback phase 3]
```

Expand-contract turns one breaking change into three reversible steps, each gated on live telemetry rather than a fixed calendar slot — the contract phase gets the tightest thresholds because it's the hardest to undo.

The autonomous system tracks canary deployments as multi-phase change plans with explicit phase gates:

```
CREATE TABLE autonomous_change.canary_deployments (
    deployment_id        UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    change_description    NVARCHAR(500) NOT NULL,
    current_phase         NVARCHAR(20) NOT NULL DEFAULT 'expand',
    phase_entered_at     DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
    expand_plan_id        NVARCHAR(200),
    migrate_plan_id       NVARCHAR(200),
    contract_plan_id      NVARCHAR(200),
    canary_traffic_pct    DECIMAL(5,2) DEFAULT 0,
    phase_gate_status     NVARCHAR(20) DEFAULT 'pending',
    phase_gate_metrics    NVARCHAR(MAX), -- JSON
    created_at            DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()
);
GO

CREATE OR ALTER PROCEDURE autonomous_change.evaluate_canary_phase_gate
    @deployment_id        UNIQUEIDENTIFIER,
    @error_rate_threshold DECIMAL(5,4) = 0.01,
    @latency_increase_pct DECIMAL(5,2) = 25.0,
    @result                NVARCHAR(20) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @phase NVARCHAR(20), @entered_at DATETIMEOFFSET;

    SELECT @phase = current_phase, @entered_at = phase_entered_at
    FROM autonomous_change.canary_deployments
    WHERE deployment_id = @deployment_id;

    -- Require 15 minutes of canary data
    IF DATEDIFF(MINUTE, @entered_at, SYSDATETIMEOFFSET()) < 15
    BEGIN
        SET @result = 'insufficient_data';
        RETURN;
    END

    -- Phase gate evaluation against telemetry store (illustrated as structure)
    -- Real implementation queries application observability data joined to
    deployment window
    SET @result = 'passed'; -- placeholder: replace with actual metric comparisons

    UPDATE autonomous_change.canary_deployments
    SET phase_gate_status = @result
    WHERE deployment_id = @deployment_id;
END;
```

The canary phase gate runs on a schedule — every five minutes once a phase has been active for the minimum observation window. If the gate returns `passed`, the orchestrator advances to the next phase automatically. If it returns `failed`, the system executes the rollback plan for the current phase and emits an alert. The contract phase is held to a stricter standard than the expand phase: removing old columns or tables is harder to reverse than adding new ones, so the gate thresholds for Phase 3 are intentionally tighter.

Integrating with CI/CD Pipelines: The Autonomous Change API

For the autonomous change management system to function at scale, it cannot operate as a standalone silo. It must integrate with the same CI/CD pipelines that development teams already use. The integration point is a change submission API that accepts a migration from a deployment pipeline — a Flyway migration file, a Django migration, a Liquibase changeset — scores it, and either approves it for autonomous execution or returns a structured rejection with the exact reason.

The API contract is simple but consequential: any change that returns `approved` will be executed by the autonomous system with no further human confirmation. Any change that returns `deferred` or `blocked` is held in queue or returned to the submitter with a reasoning payload. This means the API must be trustworthy enough that teams are willing to submit migrations to it rather than running them manually.


```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List
import uuid

app = FastAPI(title="Autonomous Change Management API")

class MigrationSubmission(BaseModel):
    pipeline_id: str
    change_type: str
    target_schema: str
    target_table: Optional[str]
    statements: List[str]
    estimated_row_count: Optional[int] = 0
    requires_lock: bool = False
    lock_level: str = "row"
    submitter: str
    urgency: str = "normal" # normal, urgent, maintenance_only

class ChangeDecision(BaseModel):
    submission_id: str
    decision: str # approved, deferred, blocked
    risk_tier: str
    risk_score: float
    scheduled_for: Optional[str]
    rejection_reasons: List[str]
    execution_strategy: str

@app.post("/v1/changes/submit", response_model=ChangeDecision)
async def submit_change(submission: MigrationSubmission):
    submission_id = str(uuid.uuid4())
    rejection_reasons = []

    # Build ChangeProposal from submission
    proposal = ChangeProposal(
        change_id=submission_id,
        change_type=submission.change_type,
        target_table=submission.target_table or "",
        estimated_row_count=submission.estimated_row_count or 0,
        statement="\n".join(submission.statements),
        requires_lock=submission.requires_lock,
        lock_level=submission.lock_level
    )

    # Score against live system state
    db_conn = get_db_connection() # connection pool, not shown
    score_result = score_change_risk(proposal, db_conn)
    risk_tier = score_result["risk_tier"]
    autonomous_action = score_result["autonomous_action"]

    # Map action to decision
    if autonomous_action == "execute_immediately":
        decision = "approved"

```

```

        execution_strategy = "immediate"
        scheduled_for = "now"
    elif autonomous_action == "execute_with_monitoring":
        decision = "approved"
        execution_strategy = "monitored"
        scheduled_for = "now"
    elif autonomous_action == "defer_to_maintenance_window":
        if submission.urgency == "urgent":
            decision = "approved"
            execution_strategy = "monitored_urgent"
            scheduled_for = "now"
        else:
            decision = "deferred"
            execution_strategy = "maintenance_window"
            scheduled_for = next_maintenance_window()
            rejection_reasons.append(
                f"Risk score {score_result['final_score']:.1f} requires maintenance
window execution"
            )
    else: # block_and_escalate / block_and_defer_72h
        decision = "blocked"
        execution_strategy = "none"
        scheduled_for = None
        rejection_reasons.append(
            f"Risk tier CRITICAL: {score_result['autonomous_action']}"
        )
        rejection_reasons.append(
            f"Dynamic risk multiplier: {score_result['dynamic_multiplier']:.2f}x "
            f"(live system conditions are unfavorable)"
        )

    return ChangeDecision(
        submission_id=submission_id,
        decision=decision,
        risk_tier=risk_tier,
        risk_score=score_result["final_score"],
        scheduled_for=scheduled_for,
        rejection_reasons=rejection_reasons,
        execution_strategy=execution_strategy
    )

```

Teams configure their CI/CD pipelines to call this endpoint before any migration step. A `blocked` response fails the pipeline stage with a structured error that includes the exact metrics that caused the block — making it trivially clear whether the issue is the change itself (static risk) or the current system state (dynamic risk). If it is the current system state, the same migration may be resubmitted an hour later and receive an `approved` decision.

This feedback loop is itself an educational mechanism. Engineers learn quickly which change patterns trigger which risk tiers, and over time they author migrations that are less likely to conflict with production conditions — writing `CREATE INDEX ... WITH (ONLINE = ON)` rather than an offline `CREATE INDEX`, partitioning large deletes into batches, and including reversal statements as standard practice.

Key Takeaways

- → **Risk scoring must be both static and dynamic.** A change's inherent danger — derived from its type, lock level, and affected row count — is only half the picture. The live system state at execution time determines the final risk tier and the appropriate autonomous action, and those two inputs must be combined at decision time, not at submission time.
- → **Online and concurrent execution strategies are not optional for autonomous systems.** When no human is watching, a blocking DDL operation that escalates lock contention has no one to cancel it. The autonomous execution layer must default to `ONLINE`, resumable, or batched strategies for every operation class that supports them, treating blocking operations as a last resort reserved for explicit maintenance windows.
- → **LLM-generated change plans require deterministic validation gates before execution.** Language model output is generative and can contain syntactically valid but semantically dangerous SQL. Every generated statement must pass pattern-based safety checks, a SQL parser, and the same risk scoring pipeline as any human-authored migration before it is eligible for execution.
- → **Autonomous rollback requires pre-execution baselines, not just post-execution comparisons.** Without capturing TPS, latency, cache hit ratio, and replica lag immediately before a change begins, the system has no meaningful reference point against which to evaluate degradation. A rollback signal is only as reliable as the baseline it compares against.
- → **The expand-contract pattern and canary phase gates make schema changes safe to automate fully.** By breaking changes into backward-compatible phases with automated telemetry gates between them, the autonomous system can advance — or retreat — through a schema migration without any human decision point, while maintaining compatibility with both old and new application versions throughout the entire deployment window.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 26: Autonomous Data Lifecycle Management

Data lifecycle management has always been a DBA's responsibility, but in traditional environments it lived entirely in documentation, calendar reminders, and ad hoc scripts run by someone who remembered the retention policy. In an autonomous system, lifecycle management is policy-as-code: the database platform observes its own data, classifies it by temperature and regulatory standing, and executes retention, archival, compression, tiering, and deletion decisions continuously — without a human initiating the workflow. This chapter builds that system from policy definition through execution engines, covering SQL Server with production-grade automation patterns.

Defining Data Temperature and Policy as Machine-Readable Contracts

Before any autonomous system can act on data lifecycle decisions, it needs a structured model of what "lifecycle" actually means for each dataset. The mistake most teams make is encoding lifecycle logic into stored procedures or cron jobs with no formal policy layer — so when retention requirements change, the changes scatter across dozens of scripts. The ADE approach is to treat lifecycle policy as a first-class contract: a structured schema that the automation engine reads at runtime.

A policy contract captures five dimensions for every dataset: **access temperature** (how frequently is it read or written), **regulatory retention floor** (minimum time data must be kept), **business retention ceiling** (maximum time data is operationally useful), **data classification** (PII, financial, operational metrics, audit), and **action chain** (what happens at each lifecycle stage — compress, tier, archive, anonymize, delete).

Store these contracts in a dedicated metadata schema that all lifecycle automation reads from:

```

CREATE SCHEMA lifecycle;
GO

CREATE TABLE lifecycle.Policy (
    PolicyId          INT IDENTITY(1,1) PRIMARY KEY,
    SchemaName        NVARCHAR(128) NOT NULL,
    TableName         NVARCHAR(128) NOT NULL,
    PartitionKey      NVARCHAR(128),
    DataClass         NVARCHAR(50) NOT NULL
                        CHECK (DataClass IN
('pii','financial','operational','audit')),
    TempHotDays       INT NOT NULL DEFAULT 30,
    TempWarmDays      INT NOT NULL DEFAULT 90,
    RetainMinDays     INT NOT NULL,
    RetainMaxDays     INT NULL,
    ActionWarm        NVARCHAR(100) NOT NULL DEFAULT 'compress',
    ActionCold        NVARCHAR(100) NOT NULL DEFAULT 'archive',
    ActionExpire      NVARCHAR(100) NOT NULL DEFAULT 'delete',
    AnonymizeBeforeDelete BIT NOT NULL DEFAULT 0,
    Enabled           BIT NOT NULL DEFAULT 1,
    LastEvaluated     DATETIMEOFFSET,
    CreatedAt         DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
    UpdatedAt         DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()
);
GO

-- Matching policy for SQL Server
INSERT INTO lifecycle.Policy (
    SchemaName, TableName, PartitionKey, DataClass,
    TempHotDays, TempWarmDays, RetainMinDays, RetainMaxDays,
    ActionWarm, ActionCold, ActionExpire, AnonymizeBeforeDelete
) VALUES (
    'app', 'UserEvents', 'EventTime', 'pii',
    30, 90, 2555, 2555,
    'compress', 'archive_to_blob', 'delete', 1
);
GO

```

With policy stored in schema, the automation layer has a single source of truth. When a GDPR request arrives demanding a shorter retention window for a specific table, one UPDATE to `lifecycle.policy` propagates to all downstream automation at next evaluation. This is the foundational shift: the human edits a contract, and the system executes the consequences.

Autonomous Partition Lifecycle: Rolling Windows and Automated Detach/Attach

Partitioning is the mechanical prerequisite for autonomous lifecycle management at scale. Without partition boundaries that map to time ranges, the engine cannot act on lifecycle stages without expensive full-table scans and row-by-row deletions. SQL Server partition switching gives the autonomous engine atomic, low-lock operations on complete data segments.

The autonomous partition manager has three jobs running continuously: **proactive creation** (ensure future partitions exist before they're needed), **lifecycle transition** (move partitions to warm/cold storage based on policy), and **expiry processing** (detach and drop partitions that have crossed their retention ceiling).

```
graph LR
    A[Hot  
within temp_hot_days] -->|age crosses threshold| B[Warm  
compress / columnstore]
    B -->|age crosses threshold| C[Cold  
archive to object storage]
    C -->|past retain_max_days| D[Expired  
anonymize + drop/detach]
    W[Late write detected] -. re-heats .-> A
    Gate[IO headroom gate] -. defers heavy ops .-> C
```

Every partition moves through the same policy-driven stages; a late write re-heats it back to hot, and the IO headroom gate can hold a partition at its current stage rather than risk a heavy operation under load.

In SQL Server, partition functions and partition schemes are the underlying engine, with the autonomous layer sitting above them as a policy executor that splits ranges, switches partitions to staging tables, and merges boundaries on schedule:

```

-- SQL Server: identify partition ranges and lifecycle stage
-- Uses partition function metadata joined against policy
WITH PartitionRanges AS (
    SELECT
        OBJECT_SCHEMA_NAME(i.object_id) AS SchemaName,
        OBJECT_NAME(i.object_id) AS TableName,
        p.partition_number,
        p.rows,
        SUM(a.total_pages) * 8 * 1024 AS SizeBytes,
        -- For range-right: lower boundary is the previous value
        LAG(rv.value) OVER (
            PARTITION BY i.object_id ORDER BY p.partition_number
        ) AS RangeStart,
        rv.value AS RangeEnd
    FROM sys.partitions p
    JOIN sys.indexes i
        ON i.object_id = p.object_id AND i.index_id = p.index_id
    JOIN sys.partition_schemes ps
        ON i.data_space_id = ps.data_space_id
    JOIN sys.partition_functions pf
        ON ps.function_id = pf.function_id
    LEFT JOIN sys.partition_range_values rv
        ON rv.function_id = pf.function_id
        AND rv.boundary_id = p.partition_number
    JOIN sys.allocation_units a
        ON a.container_id = p.hobt_id
    WHERE i.index_id <= 1
    GROUP BY
        i.object_id, p.partition_number, p.rows, rv.value
),
PolicyJoin AS (
    SELECT
        pr.*,
        pol.PolicyId,
        pol.TempHotDays,
        pol.TempWarmDays,
        pol.RetainMinDays,
        pol.RetainMaxDays,
        pol.ActionWarm,
        pol.ActionCold,
        pol.ActionExpire,
        pol.AnonymizeBeforeDelete,
        DATEDIFF(DAY, CAST(pr.RangeStart AS DATETIMEOFFSET), SYSDATETIMEOFFSET()) AS
DataAgeDays,
        CASE
            WHEN DATEDIFF(DAY, CAST(pr.RangeStart AS DATETIMEOFFSET),
SYSDATETIMEOFFSET())
                < pol.TempHotDays THEN 'hot'
            WHEN DATEDIFF(DAY, CAST(pr.RangeStart AS DATETIMEOFFSET),
SYSDATETIMEOFFSET())
                < pol.TempWarmDays THEN 'warm'
            WHEN pol.RetainMaxDays IS NOT NULL
                AND DATEDIFF(DAY, CAST(pr.RangeStart AS DATETIMEOFFSET),

```

```

SYSDATETIMEOFFSET())
        > pol.RetainMaxDays THEN 'expired'
        ELSE 'cold'
    END AS LifecycleStage
FROM PartitionRanges pr
JOIN lifecycle.Policy pol
    ON pol.SchemaName = pr.SchemaName
    AND pol.TableName = pr.TableName
WHERE pol.Enabled = 1
    AND pr.RangeStart IS NOT NULL
)
SELECT
    SchemaName,
    TableName,
    partition_number,
    RangeStart,
    RangeEnd,
    DataAgeDays,
    LifecycleStage,
    ActionWarm,
    ActionCold,
    ActionExpire,
    AnonymizeBeforeDelete,
    SizeBytes / 1048576.0 AS SizeMB
FROM PolicyJoin
WHERE LifecycleStage IN ('warm', 'cold', 'expired')
ORDER BY TableName, partition_number;

```

The Python orchestrator consumes these queries, builds an execution plan, and dispatches actions. The key engineering decision is **serialization**: lifecycle operations against large partitions are expensive, so the orchestrator must rate-limit itself against observed I/O headroom, never initiating a cold-tier operation during peak traffic windows.

The Autonomous Archival Engine: Tiered Storage Without Human Scheduling

Archival in an autonomous system is not a monthly job — it's a continuous pipeline triggered by policy evaluation. When a partition crosses into the "cold" stage, the archival engine extracts it to object storage (S3, Azure Blob, GCS), optionally compresses it with Parquet or columnar format, records the archive manifest, and either drops the partition or replaces it with a foreign table wrapper pointing at the archived location.

The following Python engine handles the full archival workflow for SQL Server. It reads from `lifecycle.Policy`, queries the partition inventory, and dispatches archival operations with controlled concurrency:


```

import pyodbc
import boto3
import pandas as pd
import pyarrow as pa
import pyarrow.parquet as pq
import logging
import time
import tempfile
import os
from datetime import datetime, timezone
from dataclasses import dataclass
from typing import Optional
from concurrent.futures import ThreadPoolExecutor, as_completed

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger("lifecycle.archival")

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=prod;UID=lifecycle_svc;PWD=*****;")

@dataclass
class ArchivalTask:
    policy_id: int
    schema_name: str
    table_name: str
    partition_number: int
    range_start: datetime
    size_bytes: int
    action: str
    anonymize: bool

class AutonomousArchivalEngine:
    def __init__(self, s3_bucket: str, s3_prefix: str,
                 max_workers: int = 3, io_headroom_pct: float = 0.30):
        self.s3_bucket = s3_bucket
        self.s3_prefix = s3_prefix
        self.max_workers = max_workers
        self.io_headroom_pct = io_headroom_pct
        self.s3 = boto3.client("s3")

    def _connect(self):
        return pyodbc.connect(CONN_STR)

    def _get_pending_tasks(self, conn) -> list:
        """Query the lifecycle engine view for cold/expired partitions needing
        action."""
        cur = conn.cursor()
        cur.execute("""
            SELECT
                lp.PolicyId,
                lp.SchemaName,
                lp.TableName,

```

```

        pi.partition_number,
        pi.RangeStart,
        pi.SizeBytes,
        CASE
            WHEN pi.LifecycleStage = 'expired' THEN lp.ActionExpire
            WHEN pi.LifecycleStage = 'cold' THEN lp.ActionCold
            ELSE lp.ActionWarm
        END AS action,
        lp.AnonymizeBeforeDelete
    FROM lifecycle.PartitionLifecycleView pi
    JOIN lifecycle.Policy lp
        ON lp.SchemaName = pi.SchemaName
        AND lp.TableName = pi.TableName
    WHERE pi.LifecycleStage IN ('cold', 'expired')
        AND lp.Enabled = 1
        AND NOT EXISTS (
            SELECT 1 FROM lifecycle.ArchivalLog al
            WHERE al.PartitionNumber = pi.partition_number
            AND al.Status = 'completed'
        )
    ORDER BY pi.SizeBytes ASC -- smallest first to validate before large
ops
    """
    rows = cur.fetchall()
    return [ArchivalTask(*r) for r in rows]

def _check_io_headroom(self, conn) -> bool:
    """Gate archival operations on current I/O utilization."""
    cur = conn.cursor()
    cur.execute("""
        SELECT CAST(cnr_value AS FLOAT)
        FROM sys.dm_os_performance_counters
        WHERE counter_name = 'Buffer cache hit ratio'
        AND object_name LIKE '%Buffer Manager%'
    """)
    row = cur.fetchone()
    cache_hit_ratio = row[0] or 0
    # Low cache hit ratio signals heavy disk I/O – defer archival
    if cache_hit_ratio < 90.0:
        logger.warning("Cache hit ratio %.2f below threshold – deferring
archival", cache_hit_ratio)
        return False
    return True

def _export_partition_to_parquet(self, conn, task: ArchivalTask) -> str:
    """Stream partition data to S3 as Parquet."""
    cur = conn.cursor()
    select_list = "*"
    if task.anonymize:
        # Anonymize known PII columns before export – column list from metadata
        cur.execute("""
            SELECT ColumnName FROM lifecycle.PiiColumns
            WHERE SchemaName = ? AND TableName = ?
        """)

```

```

        """
        (task.schema_name, task.table_name)
        pii_cols = {r[0] for r in cur.fetchall()}
        cur.execute("""
            SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS
            WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?
            ORDER BY ORDINAL_POSITION
        """, (task.schema_name, task.table_name))
        all_cols = [r[0] for r in cur.fetchall()]
        select_list = ", ".join(
            (f"HASHBYTES('SHA2_256', CAST([c] AS NVARCHAR(MAX))) AS [{c}]"
             if c in pii_cols else f"[{c}]")
            for c in all_cols
        )

    query = f"""
        SELECT {select_list}
        FROM [{task.schema_name}].[{task.table_name}]
        WHERE $PARTITION.[{task.table_name}_pf]([CreatedAt]) =
{task.partition_number}
    """

    df = pd.read_sql(query, conn)
    local_path = os.path.join(tempfile.gettempdir(),
                              f"{task.schema_name}_{task.table_name}"
                              _p{task.partition_number}.parquet")
    s3_key = (
        f"{self.s3_prefix}/{task.schema_name}/{task.table_name}/"
        f"{task.range_start.strftime('%Y/%m/%d')}/"
        f"partition_{task.partition_number}.parquet"
    )

    table = pa.Table.from_pandas(df)
    pq.write_table(table, local_path, compression='snappy')
    self.s3.upload_file(local_path, self.s3_bucket, s3_key)
    logger.info("Archived %s.%s partition %d → s3://%s/%s (%d rows)",
                task.schema_name, task.table_name, task.partition_number,
                self.s3_bucket, s3_key, len(df))
    return s3_key

def _record_archival(self, conn, task: ArchivalTask, s3_key: str, status: str):
    cur = conn.cursor()
    cur.execute("""
        MERGE lifecycle.ArchivalLog AS target
        USING (SELECT ? AS PolicyId, ? AS PartitionNumber) AS src
        ON target.PolicyId = src.PolicyId
        AND target.PartitionNumber = src.PartitionNumber
        WHEN MATCHED THEN
            UPDATE SET Status = ?, ArchivedAt = SYSUTCDATETIME(), S3Key = ?
        WHEN NOT MATCHED THEN
            INSERT (PolicyId, SchemaName, TableName, PartitionNumber,
                    RangeStart, S3Key, Status, ArchivedAt)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, SYSUTCDATETIME());
    """, (task.policy_id, task.partition_number, status, s3_key,

```

```

        task.policy_id, task.schema_name, task.table_name,
        task.partition_number, task.range_start, s3_key, status))
    conn.commit()

def _switch_out_partition(self, conn, task: ArchivalTask):
    # Atomic partition SWITCH to a staging table, then drop staging
    cur = conn.cursor()
    staging = f"[{task.schema_name}].[staging_{task.table_name}]"
    cur.execute(f"""
        ALTER TABLE [{task.schema_name}].[{task.table_name}]
        SWITCH PARTITION {task.partition_number} TO {staging}
    """)
    cur.execute(f"DROP TABLE {staging}")
    conn.commit()
    logger.info("Switched out and dropped partition %d of %s.%s after archival",
                task.partition_number, task.schema_name, task.table_name)

def _process_task(self, task: ArchivalTask):
    conn = self._connect()
    try:
        s3_key = self._export_partition_to_parquet(conn, task)
        self._record_archival(conn, task, s3_key, "completed")
        if task.action in ("archive_to_s3", "delete"):
            self._switch_out_partition(conn, task)
    except Exception as e:
        logger.error("Archival failed for %s partition %d: %s",
                    task.table_name, task.partition_number, e)
        self._record_archival(conn, task, "", "failed")
    finally:
        conn.close()

def run_cycle(self):
    conn = self._connect()
    try:
        if not self._check_io_headroom(conn):
            return
        tasks = self._get_pending_tasks(conn)
        logger.info("Found %d partitions pending lifecycle action", len(tasks))
    finally:
        conn.close()

    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        futures = {executor.submit(self._process_task, t): t for t in tasks}
        for future in as_completed(futures):
            task = futures[future]
            try:
                future.result()
            except Exception as e:
                logger.error("Unhandled error processing %s: %s",
                             task.table_name, e)

if __name__ == "__main__":
    engine = AutonomousArchivalEngine(

```

```

    s3_bucket="company-db-archives",
    s3_prefix="sqlserver/lifecycle",
    max_workers=3
)
while True:
    engine.run_cycle()
    time.sleep(3600) # evaluate every hour

```

The archival engine's IO gate is the key autonomous safeguard: it reads live cache hit ratio and defers all archival operations when the database is under storage pressure. This prevents the lifecycle engine itself from becoming a production incident. More sophisticated implementations replace the cache hit ratio check with a Prometheus query against `node_disk_io_time_seconds_total` to get true disk saturation metrics, and factor in replication lag to avoid running archival operations when replicas are already falling behind.

For SQL Server, cold-tiering uses partition switching to a staging table, followed by BCP export to Azure Blob Storage via SQL Server Agent or an Azure Data Factory pipeline triggered from T-SQL through `sp_invoke_external_rest_endpoint`.

Autonomous PII Anonymization and Compliance Enforcement at Lifecycle Boundaries

Regulatory compliance requirements — GDPR Article 17 (right to erasure), CCPA, HIPAA — create hard lifecycle boundaries where data must not only be deleted but provably anonymized or pseudonymized before any archival copy leaves the system. An autonomous lifecycle engine that cannot enforce these boundaries creates legal liability. The system must detect PII fields, apply consistent anonymization at the point of archival, and generate auditable proof of compliance.

The compliance enforcement layer maintains a PII column registry, applies deterministic or one-way transformations at lifecycle boundaries, and records signed attestations of the operations performed:

```
-- PII column registry
CREATE TABLE lifecycle.PiiColumns (
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    SchemaName        NVARCHAR(128) NOT NULL,
    TableName         NVARCHAR(128) NOT NULL,
    ColumnName        NVARCHAR(128) NOT NULL,
    PiiType           NVARCHAR(50)  NOT NULL
        CHECK (PiiType IN (
            'email', 'phone', 'ssn', 'name', 'address',
            'ip_address', 'payment_card', 'health_record'
        )),
    Transform         NVARCHAR(50)  NOT NULL
        CHECK (Transform IN (
            'hash_sha256', 'null_out', 'generalize',
            'pseudonymize', 'tokenize'
        )),
    SaltColumn        NVARCHAR(128),
    Enabled           BIT NOT NULL DEFAULT 1,
    CONSTRAINT UQ_PiiColumns UNIQUE (SchemaName, TableName, ColumnName)
);
GO
```

```
-- Compliance attestation log
CREATE TABLE lifecycle.ComplianceAttestation (
    AttestationId     UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    PolicyId          INT,
    PartitionName     NVARCHAR(256) NOT NULL,
    TableName         NVARCHAR(128) NOT NULL,
    Operation         NVARCHAR(100) NOT NULL,
    RowCount          BIGINT,
    ColumnsTouched    NVARCHAR(MAX), -- JSON array
    Checksum          NVARCHAR(64),
    PerformedBy       NVARCHAR(128) NOT NULL DEFAULT SUSER_SNAME(),
    PerformedAt       DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    Regulation        NVARCHAR(50)
);
GO
```

```
-- Stored procedure: anonymize a partition before archival
```

```
CREATE OR ALTER PROCEDURE lifecycle.AnonymizePartition
```

```
    @SchemaName      NVARCHAR(128),
    @TableName       NVARCHAR(128),
    @PartitionNum    INT,
    @PolicyId        INT,
    @Regulation      NVARCHAR(50) = 'GDPR'
```

```
AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @Sql      NVARCHAR(MAX) = N'';
    DECLARE @SetParts NVARCHAR(MAX) = N'';
    DECLARE @ColName   NVARCHAR(128);
    DECLARE @Transform NVARCHAR(50);
```

```

DECLARE @SaltCol          NVARCHAR(128);
DECLARE @RowCount         BIGINT;
DECLARE @AttestId        UNIQUEIDENTIFIER = NEWID();
DECLARE @ColumnList      NVARCHAR(MAX) = N'[]';
DECLARE @Checksum        NVARCHAR(64);
DECLARE @FullTable       NVARCHAR(256) = QUOTENAME(@SchemaName) + '.' +
QUOTENAME(@TableName);

-- Build SET clause dynamically from PII registry
DECLARE col_cursor CURSOR LOCAL FAST_FORWARD FOR
    SELECT ColumnName, Transform, SaltColumn
    FROM lifecycle.PiiColumns
    WHERE SchemaName = @SchemaName
        AND TableName = @TableName
        AND Enabled = 1;

OPEN col_cursor;
FETCH NEXT FROM col_cursor INTO @ColName, @Transform, @SaltCol;

WHILE @@FETCH_STATUS = 0
BEGIN
    IF @SetParts <> N'' SET @SetParts += N', ';

    IF @Transform = 'hash_sha256'
        SET @SetParts += QUOTENAME(@ColName) +
            N' = CONVERT(NVARCHAR(64), HASHBYTES(''SHA2_256'', ' +
            QUOTENAME(@ColName) + N'), 2)';
    ELSE IF @Transform = 'null_out'
        SET @SetParts += QUOTENAME(@ColName) + N' = NULL';
    ELSE IF @Transform = 'generalize'
        SET @SetParts += QUOTENAME(@ColName) +
            N' = DATEFROMPARTS(YEAR(' + QUOTENAME(@ColName) + N'), 1, 1)';
    ELSE IF @Transform = 'pseudonymize'
        SET @SetParts += QUOTENAME(@ColName) +
            N' = CONVERT(NVARCHAR(64), HASHBYTES(''SHA2_256'', ' +
            QUOTENAME(@ColName) + N' + COALESCE(' +
            ISNULL(QUOTENAME(@SaltCol), N'CAST(%%physloc%% AS NVARCHAR)') +
            N', ''')'), 2)';
    ELSE IF @Transform = 'tokenize'
        SET @SetParts += QUOTENAME(@ColName) +
            N' = N''tok_'' + CONVERT(NVARCHAR(64), HASHBYTES(''SHA2_256'', ' +
            QUOTENAME(@ColName) + N'), 2)';

    -- Accumulate column list for attestation
    SET @ColumnList = JSON_MODIFY(@ColumnList, 'append $', @ColName);

    FETCH NEXT FROM col_cursor INTO @ColName, @Transform, @SaltCol;
END;

CLOSE col_cursor;
DEALLOCATE col_cursor;

IF @SetParts = N''

```

```

BEGIN
    RAISERROR('No PII columns found for %s.%s – skipping anonymization.',
        10, 1, @SchemaName, @TableName);
    RETURN;
END;

-- Apply to the specific partition via $PARTITION filter
SET @Sql = N'UPDATE ' + @FullTable +
    N' SET ' + @SetParts +
    N' WHERE $PARTITION.' +
    -- Assumes partition function name follows naming convention
    QUOTENAME(@SchemaName + '_' + @TableName + '_pf') +
    N'(' + QUOTENAME((
        SELECT PartitionKey FROM lifecycle.Policy
        WHERE SchemaName = @SchemaName AND TableName = @TableName
    )) + N') = ' + CAST(@PartitionNum AS NVARCHAR(10));

EXEC sp_executesql @Sql;
SET @RowCount = @@ROWCOUNT;

-- Record attestation
SET @Checksum = CONVERT(NVARCHAR(64),
    HASHBYTES('SHA2_256',
        @TableName + CAST(@RowCount AS NVARCHAR) +
        @ColumnList + CAST(SYSDATETIMEOFFSET() AS NVARCHAR(50))
    ), 2);

INSERT INTO lifecycle.ComplianceAttestation
    (AttestationId, PolicyId, PartitionName, TableName, Operation,
    RowCount, ColumnsTouched, Checksum, Regulation)
VALUES
    (@AttestId, @PolicyId,
    @SchemaName + '.' + @TableName + ':p' + CAST(@PartitionNum AS NVARCHAR),
    @TableName, 'anonymize_before_archive',
    @RowCount, @ColumnList, @Checksum, @Regulation);

SELECT @AttestId AS AttestationId, @RowCount AS RowsAnonymized;
END;
GO

```

The attestation record is the audit artifact that satisfies regulatory review. When a data protection authority asks "prove that this user's data was anonymized before archival," the compliance team queries `lifecycle.compliance_attestation` or `lifecycle.ComplianceAttestation` for the affected table, finds the attestation with its checksum, and can demonstrate that the operation was performed by the lifecycle service account at a specific timestamp before archival. The checksum is not a cryptographic proof of data content — it proves that the system recorded the operation atomically within the same transaction as the archival log entry.

Autonomous Retention Enforcement: Preventing Both Over-Retention and Under-Retention

Retention violations come in two directions. Under-retention deletes data before the regulatory floor is met, creating evidence destruction liability. Over-retention keeps PII or financial data beyond what policies permit, creating privacy liability and expanding breach impact radius. An autonomous system must enforce both bounds with equal vigilance.

The retention enforcement engine runs daily, evaluates every table against its policy, and raises alerts or blocks actions when bounds would be violated:

```

-- Retention violation detection
CREATE OR ALTER VIEW lifecycle.RetentionViolations AS
WITH PartitionAges AS (
    SELECT
        pol.SchemaName,
        pol.TableName,
        pr.partition_number,
        pr.RangeStart,
        DATEDIFF(DAY, CAST(pr.RangeStart AS DATETIMEOFFSET), SYSDATETIMEOFFSET()) AS
DataAgeDays,
        pol.RetainMinDays,
        pol.RetainMaxDays,
        pol.PolicyId
    FROM lifecycle.Policy pol
    CROSS APPLY (
        SELECT p.partition_number,
            LAG(rv.value) OVER (
                PARTITION BY i.object_id ORDER BY p.partition_number
            ) AS RangeStart
        FROM sys.partitions p
        JOIN sys.indexes i ON i.object_id = p.object_id AND i.index_id = p.index_id
        JOIN sys.partition_schemes ps ON i.data_space_id = ps.data_space_id
        JOIN sys.partition_functions pf ON ps.function_id = pf.function_id
        LEFT JOIN sys.partition_range_values rv
            ON rv.function_id = pf.function_id AND rv.boundary_id =
p.partition_number
        WHERE OBJECT_SCHEMA_NAME(i.object_id) = pol.SchemaName
            AND OBJECT_NAME(i.object_id) = pol.TableName
            AND i.index_id <= 1
    ) pr
    WHERE pol.Enabled = 1
        AND pr.RangeStart IS NOT NULL
),
Classified AS (
    SELECT
        SchemaName,
        TableName,
        partition_number,
        DataAgeDays,
        RetainMinDays,
        RetainMaxDays,
        CASE
            WHEN RetainMaxDays IS NOT NULL AND DataAgeDays > RetainMaxDays
                THEN 'over_retention'
            WHEN DataAgeDays < RetainMinDays
                THEN 'under_retention'
            ELSE 'compliant'
        END AS ViolationType,
        CASE
            WHEN RetainMaxDays IS NOT NULL AND DataAgeDays > RetainMaxDays + 30
                THEN 'critical'
            WHEN RetainMaxDays IS NOT NULL AND DataAgeDays > RetainMaxDays
                THEN 'warning'

```

```

        WHEN DataAgeDays < RetainMinDays
            THEN 'critical'
        ELSE 'ok'
    END AS Severity
FROM PartitionAges
)
SELECT * FROM Classified WHERE ViolationType <> 'compliant';
GO

-- Enforcement procedure: called by SQL Agent job daily
CREATE OR ALTER PROCEDURE lifecycle.EnforceRetention
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO lifecycle.ComplianceEvent
        (EventType, SchemaName, TableName, PartitionNumber,
         DataAgeDays, ViolationType, Severity, DetectedAt)
    SELECT
        'retention_violation',
        SchemaName,
        TableName,
        partition_number,
        DataAgeDays,
        ViolationType,
        Severity,
        SYSDATETIMEOFFSET()
    FROM lifecycle.RetentionViolations;

    -- Surface critical violations as error messages caught by monitoring
    DECLARE @msg NVARCHAR(500);
    SELECT @msg = STRING_AGG(
        SchemaName + '.' + TableName + ' partition ' +
        CAST(partition_number AS NVARCHAR) +
        ': ' + ViolationType + ' (' + Severity + ')',
        ';'
    )
    FROM lifecycle.RetentionViolations
    WHERE Severity = 'critical';

    IF @msg IS NOT NULL
        RAISERROR('LIFECYCLE CRITICAL VIOLATIONS: %s', 16, 1, @msg);
END;
GO

```

A Service Broker queue is the real-time escalation path: the Python orchestrator polls the `lifecycle_critical` queue and can preempt its normal hourly cycle to immediately process a critical over-retention violation. This event-driven escalation is what distinguishes an autonomous system from a scheduled batch job — the system reacts to its own compliance state in near real time rather than waiting for the next cron window.

Data Temperature Monitoring and Automatic Compression

Compression decisions in most databases are made once at table creation and then forgotten. In an autonomous system, compression is adaptive: it monitors actual access patterns, identifies warm data that has crossed into low-access territory, and applies compression autonomously without rebuilding entire tables.

SQL Server's data compression (row and page) and clustered columnstore index promotion are the production mechanisms for compression autonomy. The engine works with partition-level rebuilds and `sys.dm_db_index_usage_stats` access analytics.

SQL Server — access pattern monitoring for compression decisions:

```

-- Track per-partition access frequency using index usage stats
CREATE TABLE lifecycle.AccessTemperatureLog (
    LogId          BIGINT IDENTITY(1,1) PRIMARY KEY,
    SchemaName     SYSNAME NOT NULL,
    TableName      SYSNAME NOT NULL,
    PartitionNumber INT NOT NULL,
    UserSeeks      BIGINT,
    UserScans      BIGINT,
    UserLookups    BIGINT,
    SnapshotTime   DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

-- Periodic snapshot (called every 6 hours by lifecycle daemon)
CREATE OR ALTER PROCEDURE lifecycle.SnapshotAccessTemperature
AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO lifecycle.AccessTemperatureLog
        (SchemaName, TableName, PartitionNumber, UserSeeks, UserScans, UserLookups)
    SELECT
        OBJECT_SCHEMA_NAME(i.object_id),
        OBJECT_NAME(i.object_id),
        p.partition_number,
        SUM(ISNULL(ios.user_seeks, 0)),
        SUM(ISNULL(ios.user_scans, 0)),
        SUM(ISNULL(ios.user_lookups, 0))
    FROM sys.indexes i
    JOIN sys.partitions p ON p.object_id = i.object_id AND p.index_id = i.index_id
    LEFT JOIN sys.dm_db_index_usage_stats ios
        ON ios.object_id = i.object_id AND ios.index_id = i.index_id
        AND ios.database_id = DB_ID()
    WHERE i.object_id IN (SELECT object_id FROM lifecycle.PolicyTables)
    GROUP BY i.object_id, p.partition_number;
END;
GO

-- Compression candidate detection: partitions with no reads in 14 days
CREATE OR ALTER VIEW lifecycle.CompressionCandidates AS
WITH RecentActivity AS (
    SELECT
        SchemaName,
        TableName,
        PartitionNumber,
        SUM(UserSeeks + UserScans + UserLookups) AS TotalAccesses,
        MAX(SnapshotTime) AS LastSnapshot
    FROM lifecycle.AccessTemperatureLog
    WHERE SnapshotTime > DATEADD(DAY, -14, SYSUTCDATETIME())
    GROUP BY SchemaName, TableName, PartitionNumber
)
SELECT
    pl.SchemaName,
    pl.TableName,
    pl.PartitionNumber,

```

```

        COALESCE(ra.TotalAccesses, 0) AS Accesses14d,
        pl.LifecycleStage
FROM lifecycle.PartitionLifecycleView pl
LEFT JOIN RecentActivity ra
    ON ra.SchemaName = pl.SchemaName
    AND ra.TableName = pl.TableName
    AND ra.PartitionNumber = pl.PartitionNumber
JOIN lifecycle.Policy lp
    ON lp.SchemaName = pl.SchemaName
    AND lp.TableName = pl.TableName
WHERE COALESCE(ra.TotalAccesses, 0) = 0
    AND pl.LifecycleStage IN ('warm', 'cold')
    AND lp.ActionWarm = 'compress'
    AND lp.Enabled = 1;

```

SQL Server — autonomous columnstore promotion for cold partitions:

```

-- Columnstore promotion candidates: partitions with no seeks/scans in 14 days
CREATE OR ALTER VIEW lifecycle.ColumnstorePromotionCandidates AS
WITH PartitionActivity AS (
    SELECT
        OBJECT_SCHEMA_NAME(i.object_id) AS SchemaName,
        OBJECT_NAME(i.object_id) AS TableName,
        p.partition_number,
        SUM(ios.user_seeks + ios.user_scans + ios.user_lookups) AS TotalAccesses,
        MAX(ios.last_user_scan) AS LastScan
    FROM sys.dm_db_index_usage_stats ios
    JOIN sys.indexes i ON i.object_id = ios.object_id AND i.index_id = ios.index_id
    JOIN sys.partitions p ON p.object_id = i.object_id AND p.index_id = i.index_id
    WHERE ios.database_id = DB_ID()
    GROUP BY i.object_id, p.partition_number
),
PolicyJoined AS (
    SELECT
        pa.SchemaName,
        pa.TableName,
        pa.partition_number,
        pa.TotalAccesses,
        pa.LastScan,
        pol.ActionWarm,
        pol.Enabled
    FROM PartitionActivity pa
    JOIN lifecycle.Policy pol
        ON pol.SchemaName = pa.SchemaName
        AND pol.TableName = pa.TableName
    WHERE pol.Enabled = 1
        AND pol.ActionWarm = 'compress'
)
SELECT
    SchemaName,
    TableName,
    partition_number,
    TotalAccesses,
    LastScan,
    DATEDIFF(DAY, LastScan, GETDATE()) AS DaysSinceLastScan
FROM PolicyJoined
WHERE TotalAccesses = 0
    OR LastScan < DATEADD(DAY, -14, GETDATE());
GO

-- Procedure: promote a partition's rowstore to columnstore (partition-aligned
nonclustered CCI)
CREATE OR ALTER PROCEDURE lifecycle.PromotePartitionToColumnstore
    @SchemaName NVARCHAR(128),
    @TableName NVARCHAR(128),
    @PartitionNum INT
AS
BEGIN
    SET NOCOUNT ON;

```

```

DECLARE @IndexName NVARCHAR(256);
DECLARE @Sql NVARCHAR(MAX);

SET @IndexName = 'NCCI_' + @TableName + '_p' + CAST(@PartitionNum AS NVARCHAR);

-- Check if already columnstore
IF EXISTS (
    SELECT 1 FROM sys.indexes
    WHERE name = @IndexName
    AND object_id = OBJECT_ID(@SchemaName + '.' + @TableName)
    AND type = 6 -- NONCLUSTERED COLUMNSTORE
)
BEGIN
    RAISERROR('Columnstore index %s already exists.', 10, 1, @IndexName);
    RETURN;
END;

-- Build partition-filtered columnstore index
SET @Sql = N'CREATE NONCLUSTERED COLUMNSTORE INDEX ' +
    QUOTENAME(@IndexName) +
    N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) +
    N' WHERE $PARTITION.' +
    QUOTENAME(@SchemaName + '_' + @TableName + '_pf') +
    N'(' + QUOTENAME((
        SELECT PartitionKey FROM lifecycle.Policy
        WHERE SchemaName = @SchemaName AND TableName = @TableName
    )) + N') = ' + CAST(@PartitionNum AS NVARCHAR(10)) +
    N' WITH (DATA_COMPRESSION = COLUMNSTORE_ARCHIVE)';

EXEC sp_executesql @Sql;

INSERT INTO lifecycle.LifecycleActionLog
    (SchemaName, TableName, PartitionNumber, Action, PerformedAt)
VALUES
    (@SchemaName, @TableName, @PartitionNum, 'columnstore_promotion',
    SYSDATETIMEOFFSET());
END;
GO

```

The `COLUMNSTORE_ARCHIVE` compression mode in SQL Server applies XPRESS compression on top of the columnstore encoding, providing substantially higher compression ratios than standard columnstore for data that is rarely accessed — the ideal disposition for cold lifecycle partitions that must be retained but are almost never queried.

Building the Lifecycle Orchestration Daemon

All the components above — policy evaluation, archival, anonymization, retention enforcement, and compression promotion — need a coordinating process that runs continuously, sequences operations safely, and exposes its own health for monitoring. The lifecycle daemon is a long-running Python process with a state machine per table, a work queue, and a watchdog that reports its heartbeat to the database itself:

```

import pyodbc
import time
import logging
import threading
import signal
from datetime import datetime, timezone
from enum import Enum
from dataclasses import dataclass, field
from typing import Callable

logger = logging.getLogger("lifecycle.daemon")

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=prod;UID=lifecycle_svc;PWD=*****;")

class LifecycleAction(Enum):
    SNAPSHOT_TEMPERATURE = "snapshot_temperature"
    EVALUATE_PARTITIONS = "evaluate_partitions"
    RUN_ARCHIVAL = "run_archival"
    ENFORCE_RETENTION = "enforce_retention"
    PROMOTE_COMPRESSION = "promote_compression"
    HEARTBEAT = "heartbeat"

@dataclass
class ScheduledTask:
    action: LifecycleAction
    interval_seconds: int
    last_run: float = field(default_factory=lambda: 0.0)
    handler: Callable = field(default=None)

    def is_due(self) -> bool:
        return time.monotonic() - self.last_run >= self.interval_seconds

    def mark_run(self):
        self.last_run = time.monotonic()

class LifecycleDaemon:
    def __init__(self, archival_engine):
        self.archival_engine = archival_engine
        self._stop = threading.Event()

        self.tasks = [
            ScheduledTask(LifecycleAction.HEARTBEAT, 60,
handler=self._heartbeat),
            ScheduledTask(LifecycleAction.SNAPSHOT_TEMPERATURE, 21600,
handler=self._snapshot_temperature),
            ScheduledTask(LifecycleAction.EVALUATE_PARTITIONS, 3600,
handler=self._evaluate_partitions),
            ScheduledTask(LifecycleAction.RUN_ARCHIVAL, 3600,
handler=self._run_archival),
            ScheduledTask(LifecycleAction.ENFORCE_RETENTION, 86400,
handler=self._enforce_retention),
            ScheduledTask(LifecycleAction.PROMOTE_COMPRESSION, 43200,

```

```

handler=self._promote_compression),
    ]

    def _connect(self):
        return pyodbc.connect(CONN_STR)

    def _heartbeat(self):
        with self._connect() as conn:
            cur = conn.cursor()
            cur.execute("""
                MERGE lifecycle.DaemonHeartbeat AS target
                USING (SELECT @@SERVERNAME + ':lifecycle' AS daemon_id) AS src
                ON target.DaemonId = src.daemon_id
                WHEN MATCHED THEN
                    UPDATE SET HeartbeatAt = SYSUTCDATETIME()
                WHEN NOT MATCHED THEN
                    INSERT (DaemonId, HeartbeatAt)
                    VALUES (src.daemon_id, SYSUTCDATETIME());
            """)
            conn.commit()
            logger.debug("Heartbeat recorded")

    def _snapshot_temperature(self):
        with self._connect() as conn:
            cur = conn.cursor()
            cur.execute("EXEC lifecycle.SnapshotAccessTemperature")
            conn.commit()
            logger.info("Access temperature snapshot complete")

    def _evaluate_partitions(self):
        with self._connect() as conn:
            cur = conn.cursor()
            cur.execute("""
                UPDATE lifecycle.Policy
                SET LastEvaluated = SYSUTCDATETIME()
                WHERE Enabled = 1
            """)
            conn.commit()
            logger.info("Partition lifecycle evaluation cycle complete")

    def _run_archival(self):
        try:
            self.archival_engine.run_cycle()
        except Exception as e:
            logger.error("Archival cycle error: %s", e)

    def _enforce_retention(self):
        with self._connect() as conn:
            cur = conn.cursor()
            cur.execute("EXEC lifecycle.EnforceRetention")
            conn.commit()
            logger.info("Retention enforcement cycle complete")

```

```

def _promote_compression(self):
    with self._connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT TOP 10 SchemaName, TableName, PartitionNumber
            FROM lifecycle.CompressionCandidates
        """)
        candidates = cur.fetchall()
    for schema_name, table_name, partition_number in candidates:
        try:
            with self._connect() as conn:
                cur = conn.cursor()
                # Page compression via partition-level rebuild
                cur.execute(f"""
                    ALTER INDEX ALL ON [{schema_name}].[{table_name}]
                    REBUILD PARTITION = {partition_number}
                    WITH (DATA_COMPRESSION = PAGE)
                """)
                conn.commit()
                logger.info("Compression applied to %s.%s partition %d",
                            schema_name, table_name, partition_number)
            except Exception as e:
                logger.error("Compression failed for %s.%s partition %d: %s",
                            schema_name, table_name, partition_number, e)

def _drain_notifications(self):
    """Non-blocking poll of the Service Broker critical-alert queue.
    Preempts the normal schedule when a critical violation arrives."""
    with self._connect() as conn:
        cur = conn.cursor()
        cur.execute("""
            RECEIVE TOP (1) message_body
            FROM dbo.LifecycleCriticalQueue;
        """)
        row = cur.fetchone()
        if row is None:
            conn.rollback()
            return
        payload = row[0]
        if isinstance(payload, bytes):
            payload = payload.decode("utf-8", errors="replace")
        conn.commit() # acknowledge the message
        logger.warning("Critical lifecycle alert received: %s", payload)
        # Preempt the normal archival schedule
        self._run_archival()

def _handle_signal(self, signum, frame):
    logger.info("Shutdown signal received (%d), stopping daemon", signum)
    self._stop.set()

def run(self):
    signal.signal(signal.SIGTERM, self._handle_signal)
    signal.signal(signal.SIGINT, self._handle_signal)

```

```

logger.info("Lifecycle daemon started")

while not self._stop.is_set():
    self._drain_notifications()

    for task in self.tasks:
        if task.is_due():
            logger.info("Running task: %s", task.action.value)
            try:
                task.handler()
                task.mark_run()
            except Exception as e:
                logger.error("Task %s failed: %s", task.action.value, e)

    time.sleep(5) # main loop granularity: 5-second tick

logger.info("Lifecycle daemon stopped cleanly")

```

The daemon's architecture has three important properties. First, every task is independently scheduled with its own interval so that a slow archival run does not delay the heartbeat or the retention enforcement check. Second, the Service Broker queue poll runs a non-blocking `RECEIVE` on each main loop tick, so the daemon can react to critical alerts within five seconds without spawning threads for the listener. Third, the shutdown path is clean: `SIGTERM` sets the stop event and the daemon finishes its current task before exiting, avoiding interrupted mid-partition operations.

The supporting heartbeat table allows external monitoring systems to detect a dead lifecycle daemon:

```

-- SQL Server daemon heartbeat
CREATE TABLE lifecycle.DaemonHeartbeat (
    DaemonId    NVARCHAR(200) PRIMARY KEY,
    HeartbeatAt DATETIME2 NOT NULL
);

-- Alert query: daemon dead if heartbeat older than 5 minutes
SELECT
    DaemonId,
    HeartbeatAt,
    DATEDIFF(SECOND, HeartbeatAt, SYSUTCDATETIME()) / 60.0 AS MinutesSinceHeartbeat
FROM lifecycle.DaemonHeartbeat
WHERE HeartbeatAt < DATEADD(MINUTE, -5, SYSUTCDATETIME());

```

A Prometheus alert rule consuming this query via a SQL Server metrics exporter fires a `LifecycleDaemonDead` alert if any daemon has missed more than three heartbeats, paging the on-call engineer before a retention violation window opens.

Lifecycle Audit Reporting and Dashboarding

An autonomous lifecycle system that does not expose its activity in human-readable form is a black box that auditors and compliance teams cannot trust. The audit layer produces two categories of output: **operational reports** (what did the system do, what is its current state) and **compliance reports** (provable evidence that policies were enforced).

Monthly lifecycle activity summary as a stored procedure:

```

CREATE OR ALTER PROCEDURE lifecycle.MonthlyActivityReport
    @Year INT,
    @Month INT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @PeriodStart DATETIMEOFFSET = DATETIMEOFFSETFROMPARTS(@Year, @Month, 1,
0, 0, 0, 0, 0, 7);
    DECLARE @PeriodEnd DATETIMEOFFSET = DATEADD(MONTH, 1, @PeriodStart);

    WITH ArchivalSummary AS (
        SELECT
            TableName,
            COUNT(*) AS PartitionsArchived
        FROM lifecycle.ArchivalLog
        WHERE ArchivedAt >= @PeriodStart
            AND ArchivedAt < @PeriodEnd
            AND Status = 'completed'
        GROUP BY TableName
    ),
    AnonymizationSummary AS (
        SELECT
            TableName,
            SUM(RowCount) AS RowsAnonymized
        FROM lifecycle.ComplianceAttestation
        WHERE PerformedAt >= @PeriodStart
            AND PerformedAt < @PeriodEnd
            AND Operation = 'anonymize_before_archive'
        GROUP BY TableName
    ),
    EventSummary AS (
        SELECT
            TableName,
            COUNT(*) AS
ComplianceEvents,
            SUM(CASE WHEN Severity = 'critical' THEN 1 ELSE 0 END) AS
CriticalViolations
        FROM lifecycle.ComplianceEvent
        WHERE DetectedAt >= @PeriodStart
            AND DetectedAt < @PeriodEnd
        GROUP BY TableName
    )
    SELECT
        COALESCE(ar.TableName, anon.TableName, ev.TableName) AS TableName,
        COALESCE(ar.PartitionsArchived, 0) AS
PartitionsArchived,
        COALESCE(anon.RowsAnonymized, 0) AS RowsAnonymized,
        COALESCE(ev.ComplianceEvents, 0) AS ComplianceEvents,
        COALESCE(ev.CriticalViolations, 0) AS
CriticalViolations
    FROM ArchivalSummary ar
    FULL OUTER JOIN AnonymizationSummary anon ON anon.TableName = ar.TableName

```

```

    FULL OUTER JOIN EventSummary          ev    ON ev.TableName =
COALESCE(ar.TableName, anon.TableName)
    ORDER BY 1;
END;
GO

```

These reports feed directly into Grafana dashboards via a SQL Server datasource plugin. The standard operational dashboard has five panels: partitions per lifecycle stage (stacked bar, rolling 30 days), bytes archived per day (time series), retention violation count by severity (gauge), anonymization operations per table (heat map), and daemon heartbeat status (single-stat green/red). When the compliance team needs to produce evidence for an audit, they run the monthly report and the attestation query for the specific date range, export as PDF, and attach to the audit file.

Handling Edge Cases: Hotspots, Late Arrivals, and Policy Conflicts

A production lifecycle system encounters data that does not behave according to policy assumptions. Three edge cases require explicit handling.

Late-arriving data — a partition classified as "cold" (no reads, no writes) suddenly receives an INSERT for a row with a historical timestamp. The autonomous system must detect this and re-heat the partition before any archival or compression operation begins. The detection mechanism uses a table trigger that updates a `last_write_at` column in the partition inventory:

```

-- SQL Server: trigger to update last-write timestamp on any partition write
CREATE OR ALTER TRIGGER lifecycle.trg_UpdateLastWrite
ON lifecycle.PartitionWriteLog
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE lifecycle.PartitionWriteLog
    SET LastWriteAt = SYSUTCDATETIME()
    FROM lifecycle.PartitionWriteLog w
    JOIN inserted i
        ON i.SchemaName = w.SchemaName
        AND i.TableName = w.TableName;
END;
GO

```

The archival engine checks `last_write_at` before processing any partition and skips any partition written to within the past 24 hours, regardless of its classified lifecycle stage.

Policy conflicts — two policies accidentally match the same table (a data class policy and a table-specific override). The system resolves conflicts by preferring the more restrictive retention floor and the shorter retention ceiling, recording the conflict in the audit log:

```
-- SQL Server: detect policy conflicts
SELECT
    SchemaName,
    TableName,
    COUNT(*) AS PolicyCount,
    MIN(RetainMinDays) AS EffectiveMin,
    MIN(RetainMaxDays) AS EffectiveMax,
    STRING_AGG(CAST(PolicyId AS NVARCHAR(20)), ',') AS ConflictingPolicies
FROM lifecycle.Policy
WHERE Enabled = 1
GROUP BY SchemaName, TableName
HAVING COUNT(*) > 1;
```

Access pattern hotspots — a table classified as "warm" suddenly shows a ten-fold increase in read access (a batch analytics job scanning historical data). The autonomous system should detect this via the temperature snapshot and temporarily suppress any compression or archival operations until the access pattern normalizes. The temperature log's 14-day rolling window naturally handles this: a sudden spike in scans over the window will exclude the partition from the `compression_candidates` view without any special-case code.

Key Takeaways

- → **Policy as a first-class schema object** removes lifecycle logic from ad hoc scripts and centralizes it into a contract that the automation reads at runtime; a single UPDATE to the policy table propagates to all downstream lifecycle decisions at the next evaluation cycle.
- → **Partition-aligned lifecycle operations** are the mechanical prerequisite for autonomous data management at scale — atomic detach/attach and switch operations on complete time-range segments avoid expensive row-by-row processing and allow the system to tier, compress, or delete entire data segments with minimal locking impact on the hot data layer.
- → **Compliance attestation records are not optional metadata** — they are the auditable proof that regulatory obligations were satisfied at the exact moment of archival or deletion, and the checksum-plus-timestamp pattern ensures the attestation cannot be trivially backdated without database-level access that itself would be logged.
- → **The IO headroom gate and late-arrival detection** prevent the lifecycle engine from becoming a production incident: the archival engine defers all heavy operations when the database is under storage pressure, and any partition that receives a write is held back from compression or archival for a configurable cooling window regardless of its classified lifecycle stage.

- → **The orchestration daemon's event-driven escalation path** — Service Broker queue messages consumed on every loop tick — means critical retention violations surface within seconds rather than waiting for the next scheduled evaluation window, giving operations teams time to respond before a regulatory deadline is breached.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 27: Building Autonomous Pipelines with AIOps

AIOps — the application of machine learning, event correlation, and automated response to IT operations — reaches its full potential when applied to database pipelines that can observe their own state, reason about anomalies, and execute remediation without waiting for a human to open a ticket. For SQL Server environments, this means replacing reactive runbooks with closed-loop systems: pipelines that ingest telemetry at high frequency, correlate signals across storage, compute, query execution, and replication layers, and issue corrective actions measured in seconds rather than hours. This chapter covers the architecture and implementation of those pipelines — from signal ingestion and ML-based anomaly detection through to autonomous action orchestration, audit trails, and the guardrails that prevent autonomous systems from making catastrophic decisions.

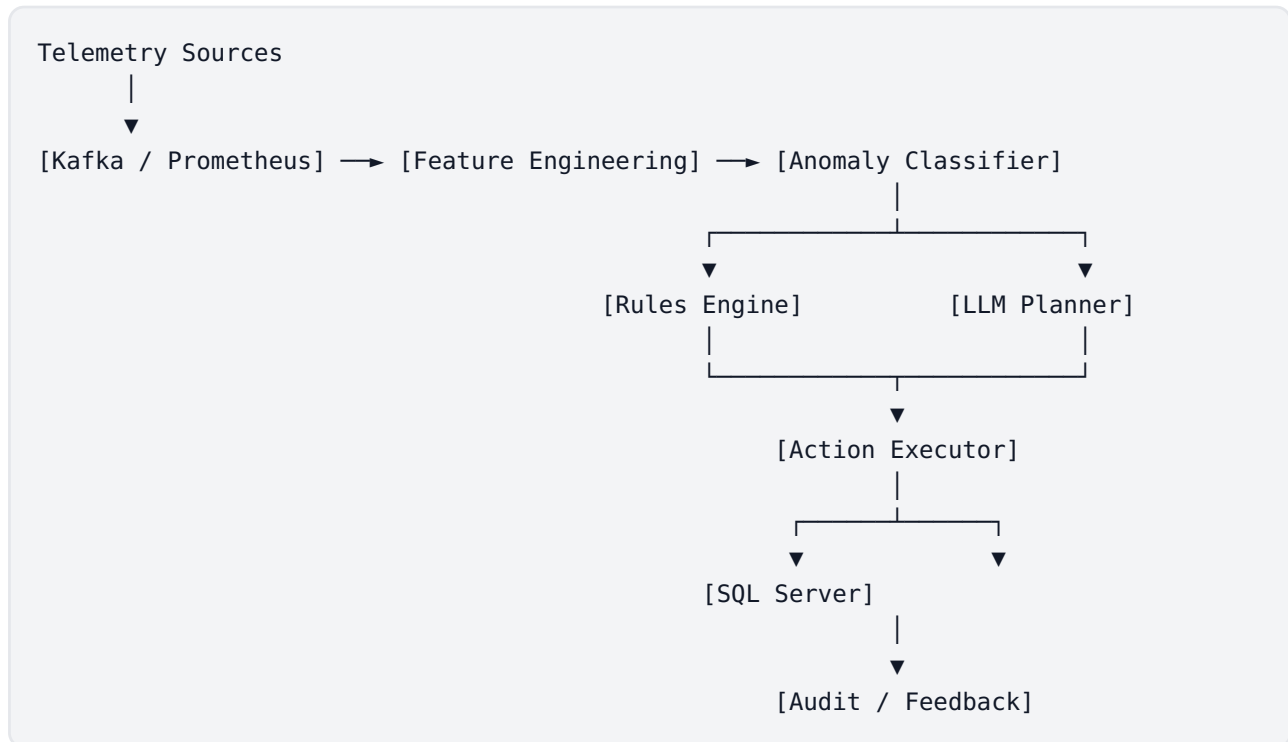
27.1 The AIOps Pipeline Architecture: From Signal to Action

Every autonomous database pipeline follows the same fundamental shape: **collect** → **correlate** → **classify** → **decide** → **act** → **audit**. What separates an AIOps pipeline from a conventional monitoring stack is the presence of closed-loop feedback at every stage and the absence of a required human approval step for a defined class of decisions.

The components in a production-grade pipeline:

- → **Telemetry collectors**: Prometheus exporters (`sql_exporter`), SQL Agent jobs, extended events sessions, custom scrapers
- → **Streaming ingest layer**: Kafka or Kinesis for high-throughput metric and log streams; Prometheus remote write for lighter deployments
- → **Feature engineering service**: A stateful processor that computes derived signals — rolling averages, rate-of-change, percentile deviations — from raw telemetry
- → **Anomaly detection engine**: Trained or online-learning models that classify signals as normal, warning, or critical
- → **Decision engine**: Rules engine or LLM-augmented planner that selects the appropriate action given current context
- → **Action executor**: Idempotent operators that invoke SQL Server remediation logic
- → **Audit and feedback store**: Immutable log of every decision and its outcome, fed back into model training

The key architectural property is **asynchrony with bounded latency**. Each stage operates independently on a queue, which means a slow anomaly classification does not block telemetry collection. However, the end-to-end latency from signal detection to action execution must be bounded — typically under 90 seconds for P1-class events, under 10 minutes for P2-class events.



This is not a theoretical diagram — it maps directly to a Kubernetes-based deployment where each box is a separate microservice with its own scaling policy, circuit breaker, and retry budget.

27.2 Telemetry Ingestion and Feature Engineering at Scale

Raw metrics are not features. A spike in lock wait time from `sys.dm_exec_requests` means nothing in isolation; the same signal correlated with a deployment timestamp and an unusual `execution_count` rate on a specific query hash means something actionable. Feature engineering is the step that transforms raw time-series into inputs that a classifier can act on.

Telemetry collection begins at the database layer. For SQL Server, `sql_exporter` with custom collector YAML files, or a Python-based agent querying DMVs (`sys.dm_exec_query_stats`, `sys.dm_os_wait_stats`, `sys.dm_tran_locks`, `sys.dm_hadr_database_replica_states`), covers the required surface area.

Core DMV telemetry for the feature pipeline:

```
-- Collect per-plan query statistics for feature engineering
SELECT TOP 500
    qs.query_hash,
    SUBSTRING(qt.text, (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(qt.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset) / 2) + 1) AS query_sample,
    qs.execution_count,
    qs.total_elapsed_time / NULLIF(qs.execution_count, 0) AS avg_elapsed_us,
    qs.total_worker_time / NULLIF(qs.execution_count, 0) AS avg_cpu_us,
    qs.total_logical_reads / NULLIF(qs.execution_count, 0) AS avg_logical_reads,
    qs.total_spills / NULLIF(qs.execution_count, 0) AS avg_spills,
    qs.total_rows / NULLIF(qs.execution_count, 0) AS avg_rows,
    GETUTCDATE() AS collected_at
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY qs.total_elapsed_time DESC;
```

These queries run on a scrape interval — typically 30 seconds for production pipelines. The raw output is published to a Kafka topic (`db.metrics.query_stats`) and consumed by a Python-based feature engineering service.

The feature engineering service computes:

- → **Z-score deviation:** How many standard deviations is current `avg_exec_ms` from the 7-day rolling mean for this `queryid` ?
- → **Rate of change:** Is `avg_blks_read` accelerating (positive second derivative)?
- → **Behavioral fingerprint distance:** Cosine distance between the current feature vector and the cluster centroid for this query's normal operating profile

```

import numpy as np
import pandas as pd
from kafka import KafkaConsumer, KafkaProducer
import json
from collections import defaultdict, deque
from scipy.spatial.distance import cosine

WINDOW_SIZE = 288 # 288 × 30s = ~2.4 hours of rolling history per query

class QueryFeatureEngineer:
    def __init__(self, kafka_bootstrap: str):
        self.consumer = KafkaConsumer(
            "db.metrics.query_stats",
            bootstrap_servers=kafka_bootstrap,
            value_deserializer=lambda m: json.loads(m.decode("utf-8")),
            group_id="feature-engineer-v2",
            auto_offset_reset="latest",
        )
        self.producer = KafkaProducer(
            bootstrap_servers=kafka_bootstrap,
            value_serializer=lambda m: json.dumps(m).encode("utf-8"),
        )
        # Rolling window keyed by (db_host, queryid)
        self.history: dict[tuple, deque] = defaultdict(lambda:
deque(maxlen=WINDOW_SIZE))

    def compute_features(self, record: dict) -> dict:
        key = (record["db_host"], record["queryid"])
        window = self.history[key]
        window.append(record)

        if len(window) < 10:
            return None # insufficient history

        hist_exec = [r["avg_exec_ms"] for r in window]
        hist_reads = [r["avg_blks_read"] for r in window if r.get("avg_blks_read")]

        current_exec = record["avg_exec_ms"]
        mean_exec = np.mean(hist_exec[:-1])
        std_exec = np.std(hist_exec[:-1]) or 1.0

        z_score = (current_exec - mean_exec) / std_exec

        # Rate of change (last 3 samples)
        if len(hist_reads) >= 3:
            roc_reads = (hist_reads[-1] - hist_reads[-3]) / 2.0
        else:
            roc_reads = 0.0

        return {
            "db_host": record["db_host"],
            "queryid": record["queryid"],
            "query_sample": record["query_sample"],

```

```

        "avg_exec_ms":    current_exec,
        "z_score_exec":  round(z_score, 4),
        "roc_blks_read": round(roc_reads, 4),
        "miss_ratio":    record.get("miss_ratio", 0.0),
        "avg_temp_blks": record.get("avg_temp_blks", 0.0),
        "calls":         record.get("calls", 0),
        "collected_at":  record["collected_at"],
    }

    def run(self):
        for message in self.consumer:
            record = message.value
            features = self.compute_features(record)
            if features:
                self.producer.send("db.features.query_stats", value=features)

```

This service runs as a Kubernetes `Deployment` with three replicas, using Kafka consumer group semantics to partition work by `db_host`. The output topic `db.features.query_stats` is consumed by the anomaly detection layer.

27.3 ML-Based Anomaly Detection and Confidence Scoring

Feature vectors alone don't drive action. The anomaly detection layer must answer: is this signal genuinely abnormal, or is it within the expected variance for this query's workload profile? And critically, it must express that answer as a confidence-weighted score that downstream decision logic can threshold against.

Two complementary approaches work well in production:

Isolation Forest for unsupervised detection of multidimensional anomalies — no labeled training data required, retrains nightly on the prior 7 days of feature vectors. This handles unknown-unknown failure modes.

Threshold ensemble with exponential backoff for well-known patterns — blocking locks, replication lag exceeding SLA, log file growth rate anomalies. These rules are deterministic and have near-zero false-positive rates when tuned correctly.

The combination means: isolation forest catches the unexpected; rule ensemble catches the known-bad with high precision.

```

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import joblib
import numpy as np
import json
from kafka import KafkaConsumer, KafkaProducer
from datetime import datetime, timezone

FEATURE_COLS = [
    "z_score_exec", "roc_blks_read", "miss_ratio",
    "avg_temp_blks", "avg_exec_ms"
]

class AnomalyDetector:
    def __init__(self, model_path: str, kafka_bootstrap: str):
        self.model: IsolationForest = joblib.load(f"{model_path}/iso_forest.pkl")
        self.scaler: StandardScaler = joblib.load(f"{model_path}/scaler.pkl")
        self.consumer = KafkaConsumer(
            "db.features.query_stats",
            bootstrap_servers=kafka_bootstrap,
            value_deserializer=lambda m: json.loads(m.decode("utf-8")),
            group_id="anomaly-detector-v3",
        )
        self.producer = KafkaProducer(
            bootstrap_servers=kafka_bootstrap,
            value_serializer=lambda m: json.dumps(m).encode("utf-8"),
        )

    def score(self, feature_record: dict) -> dict:
        vec = np.array([[feature_record.get(col, 0.0) for col in FEATURE_COLS]])
        scaled = self.scaler.transform(vec)

        # IsolationForest: score_samples returns negative anomaly score
        # More negative = more anomalous. Map to [0,1] confidence.
        raw_score = self.model.score_samples(scaled)[0]
        # Typical range: [-0.5, 0.0]; normalize to confidence
        confidence = min(1.0, max(0.0, (-raw_score - 0.05) / 0.45))

        # Deterministic rule overlay
        rule_triggered = (
            feature_record["z_score_exec"] > 4.5 or
            feature_record["miss_ratio"] > 0.85 or
            feature_record["avg_temp_blks"] > 5000
        )

        final_confidence = max(confidence, 0.95 if rule_triggered else 0.0)

        severity = (
            "critical" if final_confidence > 0.85 else
            "warning" if final_confidence > 0.55 else
            "normal"
        )

```



```

    return {
        **feature_record,
        "anomaly_confidence": round(final_confidence, 4),
        "rule_triggered": rule_triggered,
        "severity": severity,
        "detected_at": datetime.now(timezone.utc).isoformat(),
    }

def run(self):
    for message in self.consumer:
        result = self.score(message.value)
        if result["severity"] != "normal":
            self.producer.send("db.anomalies", value=result)

```

The anomaly records on `db.anomalies` carry enough context for the decision engine: which query, on which host, with what confidence, detected by which method. Nothing downstream needs to re-fetch the raw signal.

Model retraining runs as a nightly Kubernetes `CronJob`. It reads the prior 7 days of feature vectors from a time-series store (VictoriaMetrics or InfluxDB), fits a new `IsolationForest` and `StandardScaler`, validates that false-positive rate on a held-out validation window stays below 3%, and atomically replaces the model files at the shared path (PVC-backed or S3-backed). The running detector pods watch the model path via `inotify` and hot-reload without restart.

27.4 The Decision Engine: Rules, LLM Augmentation, and Safe Action Selection

Anomaly detection produces a signal. The decision engine answers: **what should the system do about it?** This is where AIOps pipeline design gets genuinely difficult — not in the detection, but in the action selection. A wrong action taken autonomously can be worse than no action at all.

The decision engine operates on three tiers:

Tier 1 — Deterministic rules: High-confidence, low-risk actions with well-understood outcomes. Applied immediately when confidence > 0.90 and action is on the approved list.

Tier 2 — LLM-augmented planning: Moderate-confidence events where context matters. The engine assembles a structured prompt containing the anomaly record, recent schema changes, active maintenance windows, and the last 5 similar historical events with their resolutions. An LLM (OpenAI GPT-4o or Anthropic Claude) returns a ranked action list with justification. The system executes the top-ranked action if it falls within the defined safe action envelope.

Tier 3 — Human escalation: Events where no safe autonomous action exists, or where confidence is insufficient, or where the proposed action falls outside the safe envelope. The system opens a PagerDuty incident and attaches full context — it does not block; it escalates and continues monitoring.

The **safe action envelope** is a critical concept. It is a statically defined list of actions the autonomous system is permitted to execute without human approval:

- → `ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP` for parallelism anomalies, bounded to [1, 8]
- → SQL Server: `KILL spid` for blocking chains > 30 seconds
- → SQL Server: `UPDATE STATISTICS tablename WITH FULLSCAN` for stale stats
- → SQL Server: `sp_recompile objectname` for plan regression
- → AG: triggering a manual failover after confirmed primary failure
- → Resource Governor: adjusting workload group caps via `ALTER WORKLOAD GROUP`

Actions **outside** the envelope require Tier 3 escalation regardless of confidence:

- → Schema changes
- → Index creation or drops on tables > 10GB without maintenance window
- → `KILL` on sessions belonging to known application service accounts
- → Any action on a primary in a synchronous replication topology without standby confirmation

The LLM planner integration:

```

import openai
import json
from datetime import datetime, timezone

SAFE_ACTION_ENVELOPE = {
    "alter_scoped_maxdop",
    "kill_spid",
    "update_statistics",
    "sp_recompile",
    "query_store_unforce_plan",
    "resource_governor_adjust",
    "ag_manual_failover",
}

SYSTEM_PROMPT = """
You are a database autonomous operations planner. Given a structured anomaly report,
recent schema changes, maintenance window status, and historical resolutions,
return a JSON object with the following structure:
{
    "ranked_actions": [
        {"action": "<action_name>", "parameters": {}, "justification": "<reason>",
"risk": "low|medium|high"},
        ...
    ],
    "reasoning_summary": "<brief explanation>"
}
Action names must be chosen from the safe action envelope provided. If no safe
action
is appropriate, return action "escalate_to_human" as the only option.
"""

class LLMDecisionPlanner:
    def __init__(self, openai_api_key: str):
        self.client = openai.OpenAI(api_key=openai_api_key)

    def plan(
        self,
        anomaly: dict,
        recent_schema_changes: list,
        maintenance_active: bool,
        historical_resolutions: list,
    ) -> dict:

        user_content = json.dumps({
            "anomaly": anomaly,
            "recent_schema_changes": recent_schema_changes[-5:],
            "maintenance_window_active": maintenance_active,
            "historical_resolutions": historical_resolutions[-5:],
            "safe_action_envelope": list(SAFE_ACTION_ENVELOPE),
            "current_utc": datetime.now(timezone.utc).isoformat(),
        }, indent=2)

        response = self.client.chat.completions.create(

```

```

model="gpt-4o",
messages=[
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": user_content},
],
response_format={"type": "json_object"},
temperature=0.1, # low temperature for deterministic planning
max_tokens=1024,
)

plan = json.loads(response.choices[0].message.content)

# Enforce safe envelope – never trust LLM output without validation
for action_item in plan.get("ranked_actions", []):
    if action_item["action"] not in SAFE_ACTION_ENVELOPE |
{"escalate_to_human"}:
        action_item["action"] = "escalate_to_human"
        action_item["justification"] = (
            "Action overridden: not in safe envelope. Original: "
            + action_item.get("action", "unknown")
        )
        action_item["risk"] = "high"

return plan

```

The temperature of 0.1 is intentional. LLM planners in autonomous systems should behave deterministically — you want consistent action selection for similar inputs, not creative variation. Every call to the LLM planner is logged with full input, output, and the identity of the action ultimately executed.

27.5 Action Execution, Idempotency, and Database-Side Safeguards

The action executor is the component that touches production databases. Its design must satisfy three properties: **idempotency** (executing the same action twice must not cause double-harm), **atomicity of observation and action** (the executor must re-verify the anomaly condition is still present before acting), and **blast radius limitation** (a single autonomous decision must never cascade into a broader outage).

Before executing any action, the executor runs a pre-flight check:

1. Re-query the database to confirm the triggering condition still exists
2. Verify no conflicting autonomous action is already in progress for this host (distributed lock via Redis or a coordination table)
3. Check the circuit breaker: if more than 3 autonomous actions have executed on this host in the last 15 minutes, halt and escalate

4. Confirm the action is not explicitly blocked by an active maintenance suppression flag

Pre-flight + safe kill for blocking chains:

```
-- Step 1: Confirm blocking chain still present
WITH blocking_chain AS (
    SELECT
        r.session_id          AS blocked_spid,
        r.blocking_session_id AS blocker_spid,
        s.login_name          AS blocker_login,
        s.program_name        AS blocker_program,
        r.wait_time / 1000.0   AS wait_seconds,
        SUBSTRING(t.text, (r.statement_start_offset / 2) + 1,
            ((CASE r.statement_end_offset
                WHEN -1 THEN DATALENGTH(t.text)
                ELSE r.statement_end_offset
            END - r.statement_start_offset) / 2) + 1) AS blocker_query
    FROM sys.dm_exec_requests r
    JOIN sys.dm_exec_sessions s ON s.session_id = r.blocking_session_id
    CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
    WHERE r.blocking_session_id > 0
        AND r.wait_time / 1000.0 > 30
)
SELECT * FROM blocking_chain
ORDER BY wait_seconds DESC;

-- Step 2: Kill the blocker if still confirmed and not a system process
-- The executor checks program_name against the exclusion list before proceeding
DECLARE @spid INT = (
    SELECT TOP 1 blocker_spid
    FROM blocking_chain
    WHERE wait_seconds > 30
        AND blocker_program NOT IN ('SQL Server Replication', 'SQLAgent - Generic
Refresher')
    ORDER BY wait_seconds DESC
);

IF @spid IS NOT NULL
    KILL @spid;
```

The executor wraps every action in a Python context manager that writes begin/end audit records and handles partial failures:

```

import redis
import pyodbc
import json
import logging
from datetime import datetime, timezone
from contextlib import contextmanager

logger = logging.getLogger("action-executor")

CONN_STR = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;"
            "DATABASE=aiops;UID=aiops_svc;PWD=*****;")

class ActionExecutor:
    def __init__(self, redis_client: redis.Redis):
        self.redis = redis_client
        self.audit_conn = pyodbc.connect(CONN_STR, autocommit=True)

    @contextmanager
    def action_lock(self, db_host: str, action_name: str, ttl_seconds: int = 300):
        lock_key = f"aiops:lock:{db_host}:{action_name}"
        acquired = self.redis.set(lock_key, "1", nx=True, ex=ttl_seconds)
        if not acquired:
            raise RuntimeError(
                f"Concurrent action lock held for {db_host}:{action_name};
skipping."
            )
        try:
            yield
        finally:
            self.redis.delete(lock_key)

    def check_circuit_breaker(self, db_host: str, window_minutes: int = 15,
max_actions: int = 3):
        cb_key = f"aiops:cb:{db_host}"
        recent = self.redis.lrange(cb_key, 0, -1)
        cutoff = datetime.now(timezone.utc).timestamp() - (window_minutes * 60)
        recent_in_window = [
            ts for ts in recent
            if float(ts) > cutoff
        ]
        if len(recent_in_window) >= max_actions:
            raise RuntimeError(
                f"Circuit breaker open for {db_host}: "
                f"{len(recent_in_window)} actions in last {window_minutes} min."
            )

    def record_action(self, db_host: str, action_name: str, parameters: dict,
outcome: str, anomaly_id: str, duration_ms: float):
        cb_key = f"aiops:cb:{db_host}"
        self.redis.rpush(cb_key, datetime.now(timezone.utc).timestamp())
        self.redis.expire(cb_key, 900)

        cur = self.audit_conn.cursor()

```

```

cur.execute("""
    INSERT INTO aiops_action_log
        (db_host, action_name, parameters, outcome,
         anomaly_id, duration_ms, executed_at)
    VALUES (?, ?, ?, ?, ?, ?, SYSUTCDATETIME())
""", (
    db_host,
    action_name,
    json.dumps(parameters),
    outcome,
    anomaly_id,
    duration_ms,
))

def execute_kill_spid(self, db_host: str, blocker_spid: int, anomaly_id: str):
    self.check_circuit_breaker(db_host)
    with self.action_lock(db_host, "kill_spid"):
        start = datetime.now(timezone.utc).timestamp()
        try:
            with pyodbc.connect(CONN_STR) as conn:
                cur = conn.cursor()
                # Re-verify condition before acting
                cur.execute("""
                    SELECT r.session_id
                    FROM sys.dm_exec_requests r
                    WHERE r.session_id = ?
                        AND r.blocking_session_id > 0
                        AND r.wait_time > 25000
                """, (blocker_spid,))
                if cur.fetchone() is None:
                    logger.info("Blocker %s no longer present; skipping.",
blocker_spid)

                    return
                cur.execute(f"KILL {int(blocker_spid)}")
                conn.commit()
                outcome = "success"
            except Exception as e:
                outcome = f"error:{e}"
                logger.exception("kill_spid failed for spid %s", blocker_spid)
            finally:
                duration_ms = (datetime.now(timezone.utc).timestamp() - start) *
1000

                self.record_action(
                    db_host, "kill_spid",
                    {"blocker_spid": blocker_spid},
                    outcome, anomaly_id, duration_ms
                )

```

The audit table backs the feedback loop described in section 27.7:

```

CREATE TABLE dbo.aiops_action_log (
    id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    db_host     NVARCHAR(200) NOT NULL,
    action_name NVARCHAR(100) NOT NULL,
    parameters  NVARCHAR(MAX) NOT NULL DEFAULT '{}',
    outcome     NVARCHAR(200) NOT NULL,    -- 'success', 'no_op', 'error:...'
    anomaly_id  NVARCHAR(100) NULL,
    duration_ms NUMERIC(10, 2) NULL,
    rollback_ddl NVARCHAR(MAX) NULL,
    executed_at DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE INDEX IX_aiops_action_log_host_time ON dbo.aiops_action_log (db_host,
executed_at DESC);
CREATE INDEX IX_aiops_action_log_anomaly ON dbo.aiops_action_log (anomaly_id);

```

In production this table lives in a dedicated `aiops` database so that audit writes never contend with production workload I/O.

27.6 Autonomous Index Management as a Pipeline Use Case

Index management is one of the highest-value autonomous pipeline use cases because the decision logic is bounded, the benefit is measurable, and the risk of a wrong action is manageable when guardrails are applied. The pipeline targets two problems: **missing indexes** that cause full scans on hot tables, and **bloated or unused indexes** that waste storage and slow writes.

Missing index detection:


```
-- DMV-based missing index candidates with impact score
SELECT TOP 30
    ROUND(
        migs.avg_total_user_cost *
        migs.avg_user_impact *
        (migs.user_seeks + migs.user_scans), 2
    ) AS improvement_score,
    mid.statement AS table_name,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns,
    migs.user_seeks,
    migs.user_scans,
    migs.last_user_seek,
    migs.avg_user_impact
FROM sys.dm_db_missing_index_group_stats migs
JOIN sys.dm_db_missing_index_groups mig ON mig.index_group_handle =
migs.group_handle
JOIN sys.dm_db_missing_index_details mid ON mid.index_handle =
mig.index_handle
WHERE migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans) > 1000
ORDER BY improvement_score DESC;
```

When the pipeline identifies a candidate index, it does **not** create the index immediately. Instead, it:

1. Generates the `CREATE INDEX ... WITH (ONLINE = ON)` DDL
2. Estimates the index build duration using table size and historical build rates
3. Checks whether a maintenance window is currently active or within the next 4 hours
4. Writes the candidate to an `index_candidates` queue with status `pending_approval` if outside maintenance window, or `approved` if inside
5. During the maintenance window, executes approved candidates in order of impact score, with a per-host concurrency limit of 1 index build at a time
6. After build completion, monitors the query statistics for the target table for 24 hours and records whether the index was used

Online index creation with progress monitoring:

```
-- Executor-generated DDL
CREATE INDEX [idx_auto_{table_name}_{column_hash}]
ON [{schema_name}].[{table_name}] ({equality_columns})
INCLUDE ({included_columns})
WITH (
    ONLINE          = ON,
    SORT_IN_TEMPDB = ON,
    DATA_COMPRESSION = ROW
);

-- Post-build: confirm no disabled or suspect index state
SELECT name, is_disabled, is_hypothetical
FROM sys.indexes
WHERE object_id = OBJECT_ID('{schema_name}.{table_name}')
    AND name = 'idx_auto_{table_name}_{column_hash}';
```

Unused index retirement follows the same pipeline pattern with a stricter safety gate: an index is eligible for retirement only if it has had zero `user_seeks + user_lookups + user_scans` for a continuous period of 30 days across all replicas, and it is not a primary key, unique constraint, or foreign key support index. The retirement action is always `DROP INDEX ... WITH (ONLINE = ON)` — never a blocking drop in production.

27.7 Feedback Loops and Continuous Model Improvement

An AIOps pipeline that does not learn from its own actions degrades over time. Workloads change, query patterns evolve, new application versions alter the statistical baseline. The feedback loop is the mechanism that keeps the anomaly detector and decision engine aligned with current reality.

The feedback pipeline reads from the `aiops_action_log` and joins it against subsequent telemetry to determine whether each autonomous action produced the expected outcome:

Feedback analysis query:

```

WITH action_events AS (
    SELECT
        al.id AS action_id,
        al.db_host,
        al.anomaly_id,
        al.action_name,
        JSON_VALUE(al.parameters, '$.queryid') AS queryid,
        al.executed_at,
        al.outcome
    FROM aiops_action_log al
    WHERE al.action_name IN ('kill_spid', 'update_statistics', 'sp_recompile')
        AND al.executed_at > DATEADD(DAY, -7, GETUTCDATE())
        AND al.outcome = 'success'
),
pre_post AS (
    SELECT
        ae.action_id,
        ae.db_host,
        ae.action_name,
        AVG(CASE WHEN m.collected_at < ae.executed_at
            THEN m.avg_elapsed_us END) AS pre_action_avg_us,
        AVG(CASE WHEN m.collected_at >= ae.executed_at
            AND m.collected_at <= DATEADD(MINUTE, 30, ae.executed_at)
            THEN m.avg_elapsed_us END) AS post_action_avg_us
    FROM action_events ae
    JOIN query_metrics_archive m
        ON m.db_host = ae.db_host
        AND m.queryid = ae.queryid
        AND m.collected_at BETWEEN DATEADD(MINUTE, -10, ae.executed_at)
            AND DATEADD(MINUTE, 30, ae.executed_at)
    GROUP BY ae.action_id, ae.db_host, ae.action_name
)
SELECT
    action_name,
    COUNT(*) AS total_actions,
    AVG(pre_action_avg_us) AS avg_pre_us,
    AVG(post_action_avg_us) AS avg_post_us,
    AVG((pre_action_avg_us - post_action_avg_us)
        / NULLIF(CAST(pre_action_avg_us AS FLOAT), 0) * 100) AS
avg_improvement_pct,
    SUM(CASE WHEN post_action_avg_us < pre_action_avg_us * 0.8
        THEN 1 ELSE 0 END) AS
clearly_effective,
    SUM(CASE WHEN post_action_avg_us > pre_action_avg_us * 1.1
        THEN 1 ELSE 0 END) AS made_things_worse
FROM pre_post
GROUP BY action_name;

```

The feedback results feed into two downstream processes:

1. **Confidence threshold calibration:** If `kill_spid` produces improvement fewer than 60% of the time for a given anomaly pattern, the confidence threshold for triggering that action is raised from 0.85 to 0.92, reducing false-positive interventions.

2. **Training data labeling:** Feature vectors associated with "clearly effective" interventions are labeled as true positives; those associated with "made_things_worse" outcomes contribute to the false-positive training set. The nightly retraining job incorporates this labeled data in a semi-supervised pass after the unsupervised IsolationForest fit.

This loop prevents two common failure modes: **alarm fatigue** (the system fires too often on non-events, operators stop trusting it) and **drift blindness** (the model stops detecting real anomalies because the baseline has shifted without retraining).

27.8 Guardrails, Governance, and Audit Compliance

Autonomous systems operating on production databases must be auditable, reversible where possible, and governed by explicit policy. Without these properties, autonomous pipelines create operational risk rather than reducing it.

Immutability of the audit log is the foundation. The `aiops_action_log` table is protected by a trigger that prevents `UPDATE` and `DELETE`:

```
-- SQL Server: equivalent DDL trigger on the aiops database
CREATE TRIGGER trg_audit_log_immutable
ON aiops_action_log
AFTER UPDATE, DELETE
AS
BEGIN
    RAISERROR(
        'Audit log is immutable. Modifications to aiops_action_log are not
        permitted.',
        16, 1
    );
    ROLLBACK TRANSACTION;
END;
```

Policy-as-code ensures the safe action envelope is version-controlled alongside application code. The YAML policy file checked into the operations repository defines which actions are permitted for which environments, time windows, and confidence thresholds:

```
# aiops-policy.yaml – version controlled in ops repository
policy_version: "2.1"
environments:
  production:
    circuit_breaker:
      max_actions_per_host_per_15min: 3
      hard_stop_on_breach: true
    maintenance_windows:
      - cron: "0 2 * * 0"    # Sunday 02:00 UTC
        duration_minutes: 120
    allowed_actions:
      - name: alter_scoped_maxdop
        min_confidence: 0.88
        bounds: [1, 8]
        require_maintenance_window: false
      - name: update_statistics
        min_confidence: 0.82
        require_maintenance_window: false
      - name: sp_recompile
        min_confidence: 0.80
        require_maintenance_window: false
      - name: kill_spid
        min_confidence: 0.85
        blocker_duration_min_seconds: 30
        excluded_program_names:
          - "SQL Server Replication"
          - "SQLAgent - Generic Refresher"
    denied_actions:
      - create_index_on_primary_outside_window
      - drop_index_without_30day_unused_confirmation
      - terminate_application_service_account_connection
  staging:
    circuit_breaker:
      max_actions_per_host_per_15min: 10
      hard_stop_on_breach: false
    allowed_actions: "*"    # all actions permitted in staging
```

Change approval integration: For Tier 2 actions proposed by the LLM planner that fall within the envelope but above a configurable "review threshold" (e.g., any `ALTER DATABASE SCOPED CONFIGURATION` outside business hours), the pipeline integrates with ServiceNow or Jira Service Management via webhook to auto-create a change record. If no human approves within 10 minutes and the anomaly condition is still present, the system proceeds under the "auto-approve after timeout" policy if the environment policy permits it.

Rollback capability: Where actions are reversible, the executor stores the pre-action state and generates a rollback script:

- → `alter_scoped_maxdop` : stores the prior value, generates `ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP =`

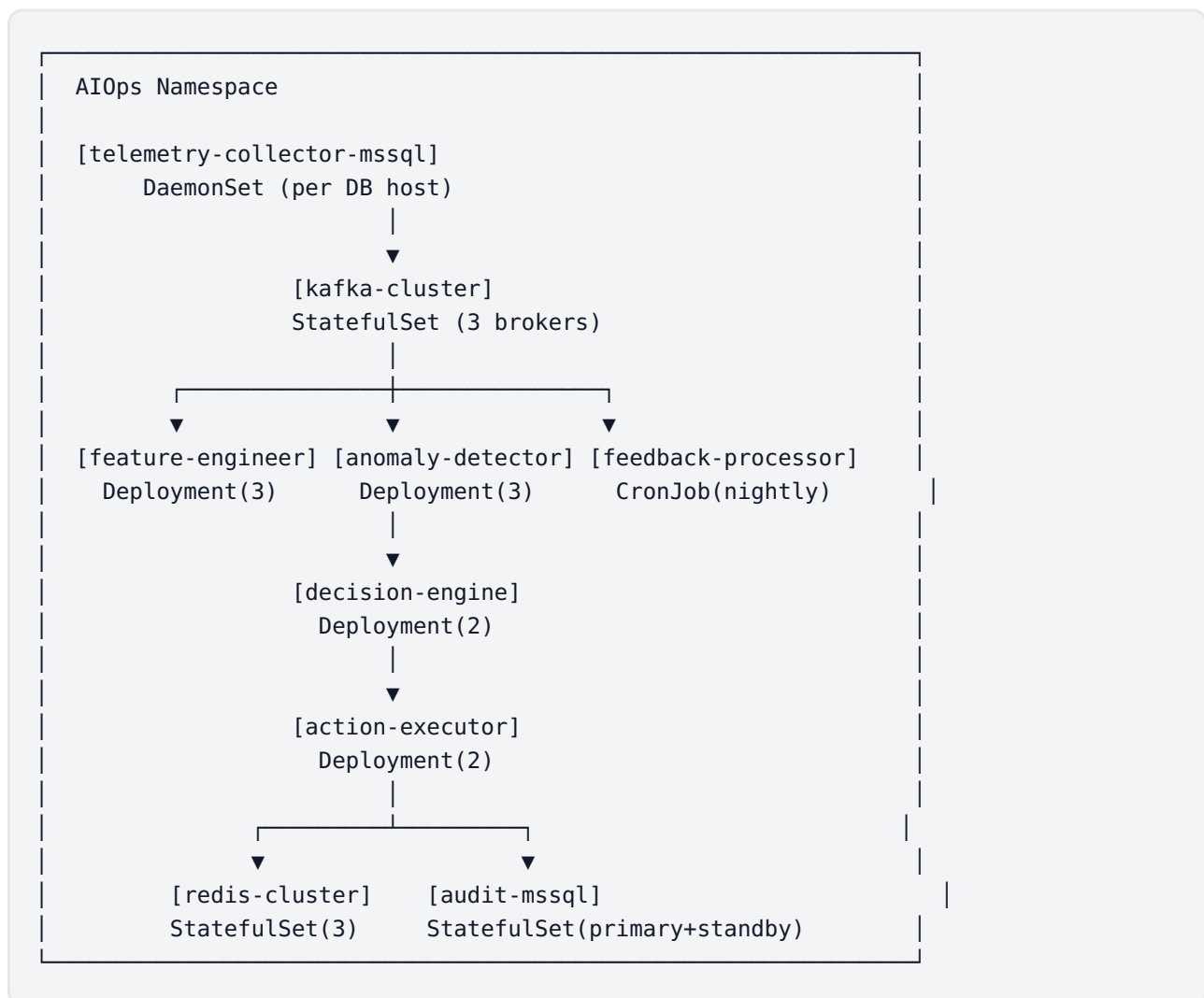
- → `update_statistics` : not directly reversible, but the executor records pre-statistics metadata from `sys.dm_db_stats_properties` so a human can assess drift
- → `kill_spid` : by nature irreversible, but the consequence is bounded to a single query cancellation

The rollback scripts are stored in the `aiops_action_log.rollback_ddl` column and are executable by operations staff without requiring pipeline access.

27.9 Deployment Topology and Operational Maturity Model

A production AIOps pipeline for a multi-region SQL Server fleet is not a single-host installation. It is a distributed system with its own availability, consistency, and latency requirements.

Reference Kubernetes deployment:



Operational Maturity Model: Organizations adopting autonomous database pipelines typically progress through four stages, and the pipeline design should accommodate teams at any stage:

Stage 1 — Observe only: The pipeline collects telemetry and generates anomaly reports but takes no autonomous action. All output goes to dashboards and alerts. This stage builds operator confidence and baseline data for model training. Duration: 4–8 weeks minimum.

Stage 2 — Recommend and confirm: The decision engine generates recommended actions and presents them through a chat interface (Slack, Teams) with a one-click approval button. The executor waits for the button press. This stage validates that the recommended actions are correct before removing the human from the loop.

Stage 3 — Autonomous for Tier 1, confirm for Tier 2: Low-risk, high-confidence actions execute autonomously. Moderate-confidence actions still require human confirmation. Operators begin to build trust in the system's judgment on the actions that execute autonomously.

Stage 4 — Fully autonomous within policy: All actions within the safe envelope execute without human approval during the confidence thresholds defined in policy. Tier 3 escalation remains for out-of-envelope events. The pipeline's effectiveness is measured by MTTR reduction and false-positive rate, not by operator involvement.

```
graph LR
    S1["Stage 1  
Observe only  
4-8 wks min"] --> S2["Stage 2  
Recommend + confirm  
one-click approval"]
    S2 --> S3["Stage 3  
Autonomous Tier 1  
confirm Tier 2"]
    S3 --> S4["Stage 4  
Fully autonomous  
within policy"]
```

Each stage removes one increment of human confirmation — teams that skip Stage 2 tend to discover its edge cases the hard way, after an autonomous action rather than before one.

Most organizations benefit from spending more time at Stage 2 than they initially plan. The recommendation-and-confirm stage surfaces edge cases and policy gaps that would otherwise only be discovered after an autonomous action causes harm.

Key Takeaways

- → **AIOps pipelines follow a closed-loop architecture** — collect, correlate, classify, decide, act, and audit — where each stage operates asynchronously on a queue, giving the system bounded end-to-end latency without blocking upstream telemetry collection.

- → **Feature engineering is more important than model sophistication:** transforming raw telemetry into Z-score deviations, rate-of-change signals, and behavioral fingerprint distances gives even a simple IsolationForest enough signal to achieve low false-positive rates in production.
- → **The safe action envelope is the single most important governance construct** in an autonomous database pipeline: it defines precisely which actions the system may take without human approval, and every action proposed by the LLM planner must be validated against it before execution — LLM output is never trusted without envelope enforcement.
- → **Action execution must be idempotent, re-verifying and circuit-broken:** the executor re-confirms the anomaly condition immediately before acting, holds a distributed lock to prevent concurrent duplicate actions, and halts autonomy on a per-host basis if the action rate exceeds the circuit breaker threshold, preventing a mis-detection cascade from causing widespread harm.
- → **Feedback loops and an operational maturity model prevent two failure modes:** the feedback pipeline measures whether actions produced the expected outcomes and adjusts confidence thresholds accordingly, while the four-stage maturity model (observe → recommend → partial autonomous → fully autonomous) ensures teams build justified trust in the system before removing human approval from high-consequence decision paths.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 28: Integrating Autonomous Systems with DevOps

Autonomous database systems do not operate in isolation — they live inside CI/CD pipelines, Kubernetes clusters, Terraform workflows, and GitOps repositories. The gap between a self-managing database engine and the surrounding DevOps machinery is where most autonomous initiatives quietly fail: the database detects a problem and responds correctly, but the action conflicts with an in-flight deployment, overwrites a configuration managed by Terraform, or triggers a failover during a maintenance window that the orchestration layer was never told about. This chapter addresses the full integration surface between autonomous database behavior and the DevOps ecosystem, covering pipeline awareness, infrastructure-as-code contracts, operator-based autonomy in Kubernetes, and the feedback loops that allow autonomous database decisions to be auditable, reversible, and coordinated with the rest of the platform.

Establishing the Autonomy Contract: What Databases Can Change Unilaterally and What They Cannot

Before writing a single line of integration code, an ADE team must define an explicit **autonomy contract** — a formal boundary that specifies which database behaviors are self-sovereign (the system acts without external coordination) and which require coordination locks with the DevOps layer. Without this contract, autonomous systems and deployment pipelines will eventually collide.

The contract has three zones:

Zone 1 — Fully Autonomous (no coordination required): Query plan cache invalidation, index creation on non-primary replicas, connection pool rebalancing, index maintenance scheduling, statistics refresh, in-memory cache eviction policy adjustment.

Zone 2 — Coordinated Autonomous (system acts but notifies): Index maintenance throttling, `MAXDOP` session-level adjustments, read replica promotion under transient primary degradation, partition pruning policy changes.

Zone 3 — Gated (system proposes, pipeline approves): Schema migrations, primary failover in production, storage tier changes, major version upgrades, index drops on primary.

The contract is stored as a versioned artifact — typically a YAML file committed to the same repository that holds your Terraform and Helm values. Autonomous agents read this file at startup and on every config reload.

```
# autonomy-contract.yaml
zones:
  fully_autonomous:
    - index_maintenance_scheduling
    - connection_pool_rebalancing
    - index_creation_replica
    - statistics_refresh
  coordinated_autonomous:
    - maxdop_adjustment
    - index_maintenance_throttling
    - read_replica_promotion
  gated:
    - schema_migration
    - primary_failover_production
    - index_drop_primary
    - storage_tier_change
gated_approval_channel: "github_actions_webhook"
notification_channel: "slack_webhook_url"
lock_timeout_seconds: 300
```

This file is the source of truth. The autonomous agent checks the zone classification before taking any action. For gated actions, it opens a pull request or calls a webhook — it never acts first.

```
-- Equivalent action audit table in SQL Server
CREATE TABLE dbo.AutonomousActionLog (
  ActionId          BIGINT IDENTITY(1,1) PRIMARY KEY,
  ActionType        NVARCHAR(200) NOT NULL,
  Zone              NVARCHAR(50) NOT NULL
                  CHECK (Zone IN
('fully_autonomous','coordinated_autonomous','gated')),
  InitiatedAt       DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
  ApprovedAt        DATETIMEOFFSET,
  ApprovedBy        NVARCHAR(200),
  ExecutedAt        DATETIMEOFFSET,
  RollbackSQL       NVARCHAR(MAX),
  Outcome           NVARCHAR(MAX),
  ConflictDetected  BIT DEFAULT 0
);

CREATE INDEX IX_ActionLog_Zone_Initiated
  ON dbo.AutonomousActionLog (Zone, InitiatedAt DESC);
```

The log is the immutable audit trail. Every autonomous action — regardless of zone — writes a record here before and after execution. This single table becomes the integration point for compliance, rollback automation, and pipeline conflict detection.

Pipeline-Aware Autonomy: Teaching the Database to Respect Deployment Windows

A deployment pipeline has state: a release is staging, deploying, validating, or complete. An autonomous database system that ignores pipeline state will take actions at the worst possible moments — changing `max server memory` mid-migration, dropping a replica connection pool during blue-green cutover, or triggering index maintenance on a heavily written table during the load test phase of a canary release.

The solution is a **deployment state bus** — a lightweight API that the CI/CD system writes to and the autonomous database agents read from. GitHub Actions, GitLab CI, and Jenkins all support webhook mechanisms that can push deployment state into a Redis key, a SQL Server table, or a dedicated HTTP endpoint.

```

# deployment_state_publisher.py
# Called by GitHub Actions at each pipeline stage transition
import requests
import os
from datetime import datetime, timezone

DEPLOYMENT_STATE_ENDPOINT = os.environ["DEPLOYMENT_STATE_ENDPOINT"]
DEPLOYMENT_SECRET = os.environ["DEPLOYMENT_STATE_SECRET"]

def publish_deployment_state(environment: str, state: str, release_tag: str):
    """
    States: staging | deploying | validating | complete | rolled_back
    Autonomous agents poll this endpoint before taking Zone 2 or Zone 3 actions.
    """
    payload = {
        "environment": environment,
        "state": state,
        "release_tag": release_tag,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "lock_autonomous_zones": ["coordinated_autonomous", "gated"] if state ==
"deploying" else [],
        "maintenance_window": state in ("staging", "validating")
    }
    resp = requests.post(
        DEPLOYMENT_STATE_ENDPOINT,
        json=payload,
        headers={"X-Deployment-Secret": DEPLOYMENT_SECRET},
        timeout=5
    )
    resp.raise_for_status()
    print(f"[DeploymentState] {environment} → {state} published (HTTP
{resp.status_code})")

# In GitHub Actions .yaml:
# - name: Notify DB autonomy layer
#   run: python deployment_state_publisher.py
#   env:
#     ENVIRONMENT: production
#     STATE: deploying
#     RELEASE_TAG: ${github.sha}

```

The autonomous agent reads this state before acting on any Zone 2 or Zone 3 decision:

```

# autonomy_gate.py
import redis
import json
from functools import wraps

r = redis.Redis(host="redis-state", decode_responses=True)

def respect_deployment_window(zone: str):
    """Decorator that blocks autonomous action if pipeline is in a locked state."""
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            state_raw = r.get("deployment:production:state")
            if not state_raw:
                return fn(*args, **kwargs)
            state = json.loads(state_raw)
            if zone in state.get("lock_autonomous_zones", []):
                raise DeploymentWindowConflict(
                    f"Action blocked: pipeline in state '{state['state']}' "
                    f"locks zone '{zone}'. Release: {state['release_tag']}"
                )
            return fn(*args, **kwargs)
        return wrapper
    return decorator

class DeploymentWindowConflict(Exception):
    pass

# Usage on any autonomous action:
@respect_deployment_window(zone="coordinated_autonomous")
def adjust_maxdop(new_value: int):
    # ... execute ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP
    pass

```

This pattern means the database's autonomous layer and the CI/CD system share a coordination protocol without tight coupling. The pipeline does not need to know anything about the database's internal decision logic — it simply publishes state transitions. The database agents consume that state and self-throttle.

When a deployment completes or rolls back, the pipeline publishes `complete` or `rolled_back`, and the autonomous agents resume normal operation. Any actions that were queued during the lock are replayed in order, with a jitter to avoid a thundering herd of deferred autonomous operations executing simultaneously.

Infrastructure-as-Code Contracts: Preventing Terraform from Overwriting Autonomous Decisions

Autonomous systems modify configuration at runtime — they adjust `max degree of parallelism`, tune `cost threshold for parallelism`, resize connection pools, or promote replicas. Terraform, applied on a schedule or triggered by a pipeline, reads desired state from `.tf` files and reconciles it against actual infrastructure. If Terraform's desired state encodes the pre-autonomous values, every `terraform apply` undoes what the autonomous system just tuned.

This is the **autonomous/IaC drift problem**, and it requires a deliberate reconciliation architecture.

The correct pattern is a **configuration ownership registry** — a metadata store that Terraform reads before applying, which tells it which parameters are currently owned by the autonomous layer and should not be touched. Terraform data sources or external data providers can query this registry and exclude owned parameters from the apply plan.

```
# terraform/modules/sqlserver_config/main.tf

data "http" "autonomy_owned_params" {
  url = "https://autonomy-api.internal/v1/owned-parameters?instance=${var.instance_id}"
  request_headers = {
    Authorization=
  }
}

locals {
  # Parameters currently managed by autonomous agents – exclude from Terraform apply
  autonomy_owned =
  toset(jsondecode(data.http.autonomy_owned_params.response_body).owned_parameters)

  # Base configuration from IaC (sp_configure names)
  base_sql_config = {
    'max degree of parallelism'      = 8
    'max server memory (MB)'        = 65536
    'cost threshold for parallelism' = 50
    'fill factor (%)'               = 90
  }

  # Only apply parameters NOT owned by the autonomous layer
  applied_sql_config = {
    for k, v in local.base_sql_config :
    k => v if !contains(local.autonomy_owned, k)
  }
}

# Apply each remaining parameter via sp_configure through sqlcmd
resource "terraform_data" "sqlserver_config" {
  for_each = local.applied_sql_config

  provisioner "local-exec" {
    command = "sqlcmd -S ${var.server_fqdn} -U ${var.admin_login} -P ${var.admin_password} -Q \"EXEC sp_configure '${each.key}', ${each.value}; RECONFIGURE;\""
  }
}
```

The autonomy API endpoint powering this data source stores ownership records:

```
-- SQL Server equivalent: tracks sp_configure ownership
CREATE TABLE dbo.AutonomousParameterOwnership (
    ParameterName      NVARCHAR(200) PRIMARY KEY,
    OwnedSince          DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET(),
    OwnedBy             NVARCHAR(200) NOT NULL,
    LastValue           NVARCHAR(500),
    IaCBaseline         NVARCHAR(500),
    ReleaseOwnershipOn  NVARCHAR(200)
);

-- Agent claims ownership of 'max degree of parallelism'
MERGE dbo.AutonomousParameterOwnership AS target
USING (VALUES (
    'max degree of parallelism',
    'parallelism-agent-v1',
    '8',
    'manual_release_only'
)) AS src (ParameterName, OwnedBy, IaCBaseline, ReleaseOwnershipOn)
ON target.ParameterName = src.ParameterName
WHEN MATCHED THEN
    UPDATE SET OwnedBy = src.OwnedBy, IaCBaseline = src.IaCBaseline
WHEN NOT MATCHED THEN
    INSERT (ParameterName, OwnedBy, IaCBaseline, ReleaseOwnershipOn)
    VALUES (src.ParameterName, src.OwnedBy, src.IaCBaseline,
src.ReleaseOwnershipOn);
```

The ownership registry also defines release conditions. Parameters tagged `manual_release_only` stay owned by the autonomous agent until an operator explicitly releases them through the autonomy API. Parameters tagged with a metric threshold — for example, `release_when: p99_latency_ms < 20 FOR 7d` — are automatically returned to IaC control once the database has been stable long enough that the autonomous tuning is no longer needed.

This creates a dynamic equilibrium: IaC owns the baseline, autonomous agents own the tuning space, and the boundary between them shifts based on runtime conditions.

Kubernetes Operator Patterns for Autonomous Database Lifecycle Management

In Kubernetes environments, the canonical integration point between autonomous database behavior and the DevOps layer is the **operator pattern**. A database operator encodes operational knowledge as a control loop: it watches Custom Resources (CRDs), reconciles observed state against desired state, and issues corrective actions. For autonomous systems, the operator becomes the execution arm of the database's decision layer — the agent that decides, and the operator that acts on behalf of Kubernetes.

For SQL Server, the MSSQL Operator for Kubernetes (part of SQL Server 2022 and later) handles availability group management within pods. The ADE's role is to extend these operators with autonomous decision hooks rather than replacing them.

The integration architecture looks like this:

```
[Prometheus/eBPF Metrics] → [Autonomous Decision Engine] → [Operator CRD Patch] →  
[Kubernetes Reconciler]
```

The autonomous decision engine does not call `kubectl` directly — it patches Custom Resource objects, and the operator's reconciler loop responds to those patches. This keeps all mutations inside the Kubernetes API server audit log and respects the operator's own safety checks.

```

# autonomous_k8s_decision.py
# The autonomous engine patches a SqlAvailabilityGroup CRD based on load metrics

from kubernetes import client, config
import json

config.load_incluster_config() # running inside the cluster
custom_api = client.CustomObjectsApi()

def adjust_read_routing(ag_name: str, namespace: str, weights: dict, reason: str):
    """
    Autonomous decision: shift read routing weights based on observed
    batch request rate and replica lag.
    Patches the SQL Server operator SqlAvailabilityGroup CRD – the
    operator reconciler executes the change.
    """
    patch_body = {
        "spec": {
            "readableSecondaries": [
                {"name": name, "readRoutingWeight": w}
                for name, w in weights.items()
            ]
        },
        "metadata": {
            "annotations": {
                "autonomous.db/last-routing-reason": reason,
                "autonomous.db/last-routing-timestamp": "2024-01-15T14:32:00Z",
            }
        }
    }

    custom_api.patch_namespaced_custom_object(
        group="sql.microsoft.com",
        version="v1",
        namespace=namespace,
        plural="sqlavailabilitygroups",
        name=ag_name,
        body=patch_body
    )
    print(f"[Autonomous] Adjusted read routing for {ag_name}. Reason: {reason}")

def autonomous_read_routing_loop(metrics_client, ag_name: str, namespace: str):
    """
    Decision logic: shift reads to secondaries when primary batch
    requests/sec > 50k AND replica lag > 500ms on any replica.
    Shift back when load < 15k for 10 consecutive minutes AND lag < 50ms.
    """
    batch_rps = metrics_client.query("rate(sqlserver_batch_requests[5m])")
    max_lag_ms = metrics_client.query("max(sqlserver_ag_replica_lag_seconds) *
1000")

    if batch_rps > 50_000 and max_lag_ms > 500:

```

```

    adjust_read_routing(
        ag_name, namespace,
        weights={"replica-1": 60, "replica-2": 40},
        reason=f"batch_rps={batch_rps:.0f} lag_ms={max_lag_ms:.0f}"
    )
elif batch_rps < 15_000 and max_lag_ms < 50:
    adjust_read_routing(
        ag_name, namespace,
        weights={"replica-1": 50, "replica-2": 50},
        reason=f"scale_down: batch_rps={batch_rps:.0f} sustained_low=10m"
    )

```

For SQL Server on Kubernetes, the autonomous layer interacts with the SQL Server Operator's `SqlAvailabilityGroup` CRD to manage replica reads and failover priorities:

```

# autonomous-patch: adjust AG read routing weights
# Applied by autonomous engine when primary CPU > 85% for 5 minutes
apiVersion: sql.microsoft.com/v1
kind: SqlAvailabilityGroup
metadata:
  name: prod-ag
  annotations:
    autonomous.db/read-routing-adjusted: "true"
    autonomous.db/routing-reason: "primary_cpu_sustained_85pct"
spec:
  readableSecondaries:
    - name: replica-1
      readRoutingWeight: 60      # was 33 – autonomous agent increased this
    - name: replica-2
      readRoutingWeight: 40      # was 33

```

The operator pattern provides a natural governance layer for Kubernetes-based autonomous systems: every autonomous action is a Kubernetes API call, every Kubernetes API call is audited, and the operator's built-in reconciliation loop ensures that conflicting changes are serialized rather than raced.

For teams using Helm for release management alongside operators, the autonomous layer must be registered as a Helm hook consumer — specifically, it must receive `pre-upgrade` and `post-upgrade` signals to pause and resume autonomous behavior during Helm-managed releases.

Closing the Loop: Autonomous Feedback into GitOps and Observability Pipelines

An autonomous system that acts without feedback is opaque. The ADE standard requires that every autonomous decision be visible in three places simultaneously: the observability stack (Prometheus/Grafana), the GitOps repository (as a configuration commit or PR), and the incident management system (PagerDuty or equivalent). This trinity of feedback is what makes autonomous behavior trustworthy at the organizational level.

```
graph TD
    A[Autonomous action executes] --> B[autonomous_action_log]
    B --> C[Prometheus metrics  
counters + histograms]
    B --> D[GitOps PR  
if parameter has IaC baseline]
    B --> E[PagerDuty event  
if gated/anomalous]
    C --> F[SRE dashboards]
    D --> G[Platform engineers  
review + merge/close]
    E --> H[On-call triage]
```

One action, three audiences: the same `autonomous_action_log` row feeds dashboards, a reviewable Git diff, and an incident event — no channel is optional.

Observability feedback: Every autonomous action emits structured metrics. A Prometheus custom collector exposes these as labeled counters and histograms:

```

# autonomous_metrics_exporter.py
from prometheus_client import Counter, Histogram, Gauge, start_http_server
import pyodbc
import time
import os

CONN_STR = os.environ["MSSQL_CONN_STRING"]

AUTONOMOUS_ACTIONS = Counter(
    'autonomous_db_actions_total',
    'Total autonomous database actions taken',
    ['action_type', 'zone', 'outcome', 'database']
)

AUTONOMOUS_LATENCY = Histogram(
    'autonomous_action_duration_seconds',
    'Time from detection to execution for autonomous actions',
    ['action_type', 'zone'],
    buckets=[0.1, 0.5, 1.0, 5.0, 30.0, 120.0, 600.0]
)

GATED_QUEUE_DEPTH = Gauge(
    'autonomous_gated_actions_pending',
    'Number of gated actions awaiting approval',
    ['database']
)

def collect_and_export():
    conn = pyodbc.connect(CONN_STR)
    cur = conn.cursor()
    try:
        # Actions completed in last scrape interval
        cur.execute("""
            SELECT action_type, zone, outcome,
                   COUNT(*),
                   AVG(CAST(DATEDIFF(MILLISECOND, initiated_at, executed_at) AS
FLOAT) / 1000.0)
            FROM autonomous_action_log
            WHERE executed_at > DATEADD(MINUTE, -1, SYSUTCDATETIME())
            GROUP BY action_type, zone, outcome
        """)
        for row in cur.fetchall():
            action_type, zone, outcome, count, avg_duration = row
            AUTONOMOUS_ACTIONS.labels(
                action_type=action_type, zone=zone,
                outcome=outcome or 'unknown', database='sqlserver'
            ).inc(count)
            if avg_duration:
                AUTONOMOUS_LATENCY.labels(
                    action_type=action_type, zone=zone
                ).observe(avg_duration)

        # Gated queue depth

```

```

        cur.execute("""
            SELECT COUNT(*) FROM autonomous_action_log
            WHERE zone = 'gated' AND approved_at IS NULL AND executed_at IS NULL
        """)
        pending = cur.fetchone()[0]
        GATED_QUEUE_DEPTH.labels(database='sqlserver').set(pending)
    finally:
        conn.close()

start_http_server(9190)
while True:
    collect_and_export()
    time.sleep(30)

```

GitOps feedback: When the autonomous system makes a structural change — one that has configuration implications — it opens a pull request or pushes a commit to the configuration repository. This is not bureaucracy; it is the mechanism by which autonomous changes become reviewable, revertible, and part of the team's knowledge base.

The autonomous agent uses the GitHub API (or GitLab equivalent) to create a commit whenever it changes a parameter that has an IaC baseline:

```

# gitops_feedback.py
import requests
import base64
import json
import os

GITHUB_TOKEN = os.environ["GITHUB_TOKEN"]
REPO = "myorg/db-platform-config"

def commit_autonomous_config_change(parameter: str, old_value: str, new_value: str,
                                     reason: str, action_id: int):
    """
    Write autonomous configuration change back into the GitOps repo as a PR.
    This makes the autonomous action visible, auditable, and revertible via Git.
    """
    headers = {
        "Authorization": f"Bearer {GITHUB_TOKEN}",
        "Accept": "application/vnd.github+json"
    }

    # Read current file
    file_path = "config/sqlserver/autonomous-runtime.json"
    get_resp = requests.get(
        f"https://api.github.com/repos/{REPO}/contents/{file_path}",
        headers=headers
    ).json()

    current_content = json.loads(base64.b64decode(get_resp["content"]))
    current_content["parameters"][parameter] = {
        "value": new_value,
        "autonomous_action_id": action_id,
        "reason": reason,
        "previous_value": old_value
    }

    new_content_b64 = base64.b64encode(
        json.dumps(current_content, indent=2).encode()
    ).decode()

    branch = f"autonomous/action-{action_id}"

    # Create branch
    ref_resp = requests.get(
        f"https://api.github.com/repos/{REPO}/git/ref/heads/main",
        headers=headers
    ).json()
    main_sha = ref_resp["object"]["sha"]

    requests.post(
        f"https://api.github.com/repos/{REPO}/git/refs",
        headers=headers,
        json={"ref": f"refs/heads/{branch}", "sha": main_sha}
    )

```

```

# Commit updated file to branch
requests.put(
    f"https://api.github.com/repos/{REPO}/contents/{file_path}",
    headers=headers,
    json={
        "message": f"autonomous: {parameter} changed from {old_value} to
{new_value} [action-{action_id}]",
        "content": new_content_b64,
        "sha": get_resp["sha"],
        "branch": branch
    }
)

# Open pull request
pr_resp = requests.post(
    f"https://api.github.com/repos/{REPO}/pulls",
    headers=headers,
    json={
        "title": f"[Autonomous] {parameter}: {old_value} → {new_value}",
        "head": branch,
        "base": "main",
        "body": (
            f"**Autonomous configuration change**\n\n"
            f"- **Parameter:** `{parameter}`\n"
            f"- **Old value:** `{old_value}`\n"
            f"- **New value:** `{new_value}`\n"
            f"- **Reason:** {reason}\n"
            f"- **Action ID:** {action_id}\n\n"
            f"This PR was opened automatically by the database autonomy layer. "
            f"Merge to record the change in IaC baseline, or close to flag for
review."
        )
    }
).json()

print(f"[GitOps] PR opened: {pr_resp['html_url']}")
return pr_resp["html_url"]

```

This pattern gives operators a natural review surface. A PR opened by the autonomous agent carries a full audit trail: what changed, why, which action ID traces back to the `autonomous_action_log`, and what the previous value was. Merging the PR updates the IaC baseline so Terraform no longer sees drift. Closing without merging flags the change as needing human review. The Git history becomes a secondary audit log of every autonomous structural decision.

Incident management feedback: For gated actions that cannot proceed, and for any autonomous action that results in an anomalous outcome, the autonomous layer fires a structured event to the incident management system:


```

# incident_feedback.py
import requests
import os

PAGERDUTY_ROUTING_KEY = os.environ["PD_ROUTING_KEY"]

def fire_autonomous_event(action_type: str, zone: str, outcome: str,
                          action_id: int, details: dict, severity: str = "info"):
    """
    Sends a PagerDuty Events API v2 event for every non-trivial autonomous decision.
    severity: 'info' for successful actions, 'warning' for blocked/queued, 'error'
    for failures.
    """
    payload = {
        "routing_key": PAGERDUTY_ROUTING_KEY,
        "event_action": "trigger" if severity in ("warning", "error") else
"acknowledge",
        "dedup_key": f"autonomous-db-{action_id}",
        "payload": {
            "summary": f"[AutonomousDB] {action_type} ({zone}) - {outcome}",
            "severity": severity,
            "source": "autonomous-database-engine",
            "custom_details": {
                "action_id": action_id,
                "action_type": action_type,
                "zone": zone,
                "outcome": outcome,
                **details
            }
        }
    }

    resp = requests.post(
        "https://events.pagerduty.com/v2/enqueue",
        json=payload,
        timeout=5
    )
    resp.raise_for_status()
    return resp.json()

```

The three feedback channels — metrics, GitOps PRs, and incident events — give a complete picture to three distinct audiences: the SRE team watching dashboards, the platform engineers reviewing configuration history, and the on-call engineer triaging alerts. Autonomous behavior stops being a black box and becomes a first-class participant in the team's operational processes.

Rollback Architecture: Making Every Autonomous Decision Reversible

Autonomy without reversibility is recklessness dressed as intelligence. Every autonomous action must have a defined rollback path, and the rollback must be executable without human reconstruction of what the system did. This means the rollback SQL or rollback operator patch is generated and stored before the action executes, not after.

The `autonomous_action_log` tables defined earlier carry a `rollback_sql` column for exactly this reason. For parameter changes, the rollback is trivially the prior value. For structural changes — index creation, partition additions, replica configuration changes — the rollback requires more care.

```

-- SQL Server rollback procedure
CREATE OR ALTER PROCEDURE dbo.ExecuteAutonomousRollback
    @ActionId BIGINT
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @RollbackSQL NVARCHAR(MAX);
    DECLARE @Zone NVARCHAR(50);
    DECLARE @ExecutedAt DATETIMEOFFSET;
    DECLARE @ActionType NVARCHAR(200);

    SELECT
        @RollbackSQL = RollbackSQL,
        @Zone = Zone,
        @ExecutedAt = ExecutedAt,
        @ActionType = ActionType
    FROM dbo.AutonomousActionLog
    WHERE ActionId = @ActionId;

    IF @RollbackSQL IS NULL
        THROW 50001, 'No rollback SQL recorded for this ActionId.', 1;

    IF @ExecutedAt IS NULL
        THROW 50002, 'Action has not been executed – nothing to roll back.', 1;

    -- Execute the rollback
    EXEC sp_executesql @RollbackSQL;

    -- Mark original action as rolled back
    UPDATE dbo.AutonomousActionLog
    SET Outcome = 'rolled_back', ConflictDetected = 1
    WHERE ActionId = @ActionId;

    -- Companion rollback record
    INSERT INTO dbo.AutonomousActionLog
        (ActionType, Zone, InitiatedAt, ExecutedAt, Outcome, RollbackSQL)
    VALUES
        ('rollback_of_' + @ActionType, @Zone,
        SYSDATETIMEOFFSET(), SYSDATETIMEOFFSET(),
        'rollback_executed',
        N'-- rollback of action_id ' + CAST(@ActionId AS NVARCHAR(20)));
END;

```

For index operations — where rollback means dropping a just-created index or recreating a dropped one — the rollback SQL is generated before execution and stored atomically with the action log record:

The rollback store is not just a safety net — it becomes the foundation for automated regression detection. If the autonomous system's observability layer detects that a recent action correlated with a performance degradation (query latency increased after index creation, for example, due to increased planner overhead on writes), the rollback daemon can be triggered automatically within the evaluation window, executing the stored rollback SQL without human involvement.

Multi-Environment Promotion: Autonomous Behavior Across Dev, Staging, and Production

Autonomous system behavior should not be identical across all environments. In development, the autonomy contract should be maximally permissive — let the system try everything, including gated actions, to build a behavioral record. In staging, gated actions require approval but coordinated-autonomous actions execute freely. In production, the full contract applies.

Environment-specific autonomy profiles are managed through the same GitOps repository, with Helm values or Kustomize overlays controlling which profile is active:

```
# config/autonomy/values-dev.yaml
autonomy_profile: aggressive
gated_actions_require_approval: false
deployment_window_enforcement: false
rollback_evaluation_window_minutes: 5
max_autonomous_actions_per_hour: 100

# config/autonomy/values-staging.yaml
autonomy_profile: standard
gated_actions_require_approval: true
deployment_window_enforcement: true
rollback_evaluation_window_minutes: 15
max_autonomous_actions_per_hour: 30

# config/autonomy/values-production.yaml
autonomy_profile: conservative
gated_actions_require_approval: true
deployment_window_enforcement: true
rollback_evaluation_window_minutes: 60
max_autonomous_actions_per_hour: 10
change_freeze_calendar: "pagerduty_schedule_id_xyz"
```

The `change_freeze_calendar` field in the production profile integrates the autonomous system with the organization's change management calendar. Before any gated or coordinated-autonomous action in production, the agent queries the PagerDuty schedule (or

ServiceNow, or OpsGenie) to confirm there is no active change freeze. This single integration prevents autonomous systems from violating change management policies that exist for regulatory or contractual reasons.

When a behavioral pattern learned in dev or staging is ready to promote to production — for example, a new autonomous tuning rule that proved effective in staging — the promotion follows the same GitOps PR workflow as any other configuration change. The PR includes evidence: a summary of actions taken in staging, their outcomes, and the performance delta observed. Human reviewers approve the promotion with full context rather than blindly enabling new autonomous behavior in production.

Security Boundaries: Least-Privilege Autonomous Agents

Autonomous database agents require credentials to act — and credentials are the primary attack surface if an autonomous system is compromised. The architecture must enforce least-privilege at every layer.

Each autonomous agent role should have exactly the permissions required for its specific action set and nothing more. The index maintenance agent needs `ALTER` permission on indexes and the ability to run `ALTER INDEX ... REBUILD` and `UPDATE STATISTICS`; it does not need `sysadmin`, `CONTROL SERVER`, or access to application data tables.

```
-- Least-privilege login for the index advisor agent
CREATE LOGIN AutonomousIndexAgent WITH PASSWORD = 'ManagedByVault!1';
CREATE USER AutonomousIndexAgent FOR LOGIN AutonomousIndexAgent;

-- Specific permissions only
GRANT ALTER ANY INDEX TO AutonomousIndexAgent;
GRANT VIEW DATABASE STATE TO AutonomousIndexAgent;
GRANT INSERT, UPDATE ON dbo.AutonomousActionLog TO AutonomousIndexAgent;
GRANT INSERT, UPDATE ON dbo.AutonomousParameterOwnership TO AutonomousIndexAgent;

-- Explicitly deny data access
DENY SELECT ON SCHEMA::dbo TO AutonomousIndexAgent;
GRANT SELECT ON dbo.AutonomousActionLog TO AutonomousIndexAgent;
GRANT SELECT ON dbo.AutonomousParameterOwnership TO AutonomousIndexAgent;
```

Credentials for autonomous agents should never be static. Integration with HashiCorp Vault's database secrets engine, AWS IAM authentication for RDS, or Azure Managed Identity for Azure SQL means credentials are short-lived and automatically rotated. If an autonomous agent is compromised, its credential expires before a lateral movement attack can leverage it.

The autonomous action log itself should be protected against tampering. In SQL Server, this means the agent role is granted `INSERT` only on the action log table, with `UPDATE` and `DELETE` explicitly denied; row-level security or column-level permissions accomplish finer constraints where needed. For high-assurance environments, the action log can be replicated to an immutable audit sink — an append-only S3 bucket with object lock, or Azure Immutable Blob Storage — where even a compromised database cannot alter the historical record.

Observability at the Integration Layer: What to Instrument

The integration layer between autonomous systems and DevOps tooling generates its own class of signals that must be monitored independently of the database's internal telemetry. These signals answer questions about the health of the integration itself, not just the database.

Key metrics to instrument at the integration layer:

Metric	Description	Alert Threshold
--------	-------------	-----------------

--	--	--

<code>autonomous_deployment_conflicts_total</code>	Times an autonomous action was blocked by a deployment lock	> 10 in 1h suggests lock is too broad
--	---	---------------------------------------

<code>autonomous_terraform_drift_count</code>	Parameters where autonomous value diverges from IaC baseline	> 0 triggers ownership reconciliation
---	--	---------------------------------------

<code>autonomous_gated_queue_age_seconds</code>	Age of oldest unapproved gated action	> 3600s triggers escalation
---	---------------------------------------	-----------------------------

<code>autonomous_gitops_pr_open_count</code>	Open PRs created by autonomous system	> 20 suggests review bandwidth issue
--	---------------------------------------	--------------------------------------

<code>autonomous_rollback_rate</code>	Fraction of autonomous actions that triggered rollback	> 5% triggers rule review
---------------------------------------	--	---------------------------

<code>autonomous_credential_rotation_lag_seconds</code>	Time since last agent credential rotation	> 86400s triggers rotation
---	---	----------------------------

These metrics are exported by the same Prometheus collector that tracks action outcomes, and they populate a dedicated "Autonomy Operations" dashboard in Grafana. The dashboard is owned by the platform engineering team, not the application teams — it surfaces the health of the autonomous system as infrastructure, not as application behavior.

The single most important signal in this set is `autonomous_rollback_rate`. A system that is rolling back more than a small fraction of its own decisions is learning poorly or acting on bad signal. An elevated rollback rate is the early warning that the autonomous system's decision models need retraining, its action thresholds need adjustment, or its observability inputs are feeding it corrupted data. This metric should never be absent from a production autonomy deployment.

Key Takeaways

- → **Define an explicit autonomy contract before writing integration code.** Classify every autonomous behavior into fully autonomous, coordinated autonomous, or gated zones, and store the contract as a versioned artifact in your GitOps repository. The contract is the foundation on which all pipeline integration is built.
- → **Teach autonomous agents to read deployment state.** A lightweight deployment state bus — written by the CI/CD pipeline and read by autonomous agents — prevents autonomous actions from colliding with in-flight deployments. Agents should self-throttle during deploying and validating states without requiring tight coupling to the pipeline.
- → **Use a configuration ownership registry to resolve IaC/autonomy drift.** Autonomous agents claim ownership of parameters they tune, and Terraform reads that registry before applying — skipping parameters currently managed by the autonomous layer. Ownership release conditions should be explicit and machine-readable.
- → **In Kubernetes environments, route all autonomous decisions through operator CRD patches.** Autonomous agents should never call `kubectl` directly or bypass the operator's reconciliation loop. Every autonomous action expressed as a CRD patch inherits Kubernetes API audit logging, admission controller validation, and the operator's own safety checks at no additional cost.
- → **The rollback SQL must be generated and stored before the action executes, not after.** An autonomous system that cannot cleanly reverse its own decisions is not ready for production. Combined with an elevated rollback rate metric as a leading indicator of model degradation, pre-stored rollback SQL transforms autonomous reversibility from a manual recovery exercise into an automated feedback loop.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 29: Human Oversight in Autonomous DBA Systems

Autonomous database systems that observe, decide, and act without human intervention represent a genuine operational leap — but the engineering discipline surrounding when and how humans remain in the loop is equally important as the automation itself. A system that acts too freely without audit trails, escalation gates, or override mechanisms is not autonomous; it is reckless. This chapter examines the architecture of human oversight for autonomous database platforms: how to design approval workflows that do not become bottlenecks, how to build explainability layers so operators understand what the system did and why, how to define trust boundaries that expand over time as the system earns confidence, and how to construct override and rollback mechanisms that restore human control in seconds when needed. The goal is not to constrain autonomy — it is to make autonomy trustworthy at production scale.

Designing the Oversight Spectrum: From Full Automation to Human-in-the-Loop Gates

Autonomous database systems do not operate at a single level of autonomy. Different classes of actions carry different risk profiles, and a mature ADE architecture segments those classes explicitly, applying oversight proportional to the potential blast radius of each decision.

The oversight spectrum can be modeled as five tiers:

Tier	Action Class	Oversight Model	Example
0	Fully automated, no notification	Execute immediately	Query plan cache invalidation, statistics refresh
1	Automated with async audit log	Execute, then log	Connection pool tuning, minor MAXDOP adjustment
2	Automated with real-time notification	Execute, then alert	Index creation on non-production, failover to secondary replica
3	Proposed, human approval required	Queue, wait, execute	Schema migration, partition pruning, major parameter changes
4	Human-initiated only	System proposes, human acts	DROP TABLE, replication topology changes, cross-region failover

The tiering logic is enforced programmatically. Every autonomous action emitted by the decision engine carries a classification tag. A policy engine evaluates that tag against the current environment context — production vs. staging, business hours vs. maintenance window, incident state vs. steady state — and routes accordingly.

The context dimension matters significantly. A Tier 2 action during steady state may escalate to Tier 3 during an active incident, because human cognitive load and coordination requirements shift. Similarly, a Tier 3 action in a staging environment may drop to Tier 1 if the system has accumulated 90 days of successful execution history for that action class in that environment.

```

# Policy engine: action tier resolution
import datetime
from dataclasses import dataclass
from enum import IntEnum

class OversightTier(IntEnum):
    EXECUTE_SILENT = 0
    EXECUTE_AUDIT = 1
    EXECUTE_NOTIFY = 2
    PROPOSE_APPROVE = 3
    HUMAN_ONLY = 4

@dataclass
class ActionContext:
    action_class: str
    environment: str          # production, staging, dev
    incident_active: bool
    business_hours: bool
    execution_history_days: int
    confidence_score: float   # 0.0 - 1.0 from decision model

def resolve_tier(base_tier: OversightTier, ctx: ActionContext) -> OversightTier:
    tier = base_tier

    # Escalate during active incidents
    if ctx.incident_active and tier < OversightTier.PROPOSE_APPROVE:
        tier = OversightTier(min(int(tier) + 1, int(OversightTier.PROPOSE_APPROVE)))

    # Escalate if confidence is low
    if ctx.confidence_score < 0.70:
        tier = OversightTier(min(int(tier) + 1, int(OversightTier.HUMAN_ONLY)))

    # Downgrade in non-production with proven history
    if ctx.environment != "production" and ctx.execution_history_days >= 90:
        tier = OversightTier(max(int(tier) - 1, int(OversightTier.EXECUTE_SILENT)))

    # Never automate human-only actions regardless of context
    if base_tier == OversightTier.HUMAN_ONLY:
        tier = OversightTier.HUMAN_ONLY

    return tier

```

The confidence score feeding this function comes from the decision model itself — whether that is a rule-based scoring system, an ML model trained on historical outcomes, or an LLM-based reasoning layer that expresses uncertainty. The architecture requires that every autonomous decision carry a machine-readable confidence estimate. Systems that cannot express uncertainty should not be trusted to act on Tier 0 or Tier 1 classifications.

The oversight spectrum must be documented, version-controlled, and reviewed on a cadence. As the system accumulates operational history, tier assignments should drift downward for proven action classes and upward for any class that has produced a rollback or incident. This is how the system earns expanded autonomy — through demonstrated performance, not through assumption.

```
graph TD
  A[Signal Observed] --> B[Decision Engine: Base Tier + Confidence]
  B --> C[Policy Engine: Context Adjustment]
  C -->|Incident Active / Low Confidence| D[Escalate Tier]
  C -->|Proven History, Non-prod| E[Downgrade Tier]
  C --> F[Resolved Tier]
  D --> F
  E --> F
  F -->|0-1| G[Execute Automatically]
  F -->|2| H[Execute, Then Notify]
  F -->|3| I[Queue for Human Approval]
  F -->|4| J[Human-Initiated Only]
  I -->|Approved| G
  G --> K[Audit Trail]
  H --> K
  K --> L[Trust Evaluation]
  L -->|Rollback Detected| D
```

The tier-resolution flow: signals feed a decision engine, the policy engine adjusts for context, and the resolved tier routes the action through the appropriate oversight gate before feeding outcomes back into trust evaluation.

Approval Workflow Architecture That Does Not Create Bottlenecks

The failure mode of human-in-the-loop systems is the approval queue becoming a graveyard. When Tier 3 actions pile up waiting for human review, the autonomous system's value proposition collapses — operators spend more time clicking approve than they saved by having the system propose actions. Avoiding this failure requires deliberate workflow architecture.

The first design principle: approval requests must be self-contained. A DBA receiving a proposal at 2:00 AM should be able to read the request, understand the context, evaluate the risk, and approve or reject without opening a second tool. This means the proposal must include the proposed SQL or configuration change, the triggering observation, the projected outcome, the rollback procedure, and a time-bounded auto-expiry if no response is received.

The second design principle: differentiated approval channels. Low-urgency proposals queue into a daily digest reviewed during business hours. High-urgency proposals (for example, a Tier 3 action triggered by degraded performance above a defined threshold) push to

PagerDuty or Slack with a one-click approval link. Emergency proposals — where delay causes measurable SLA impact — have a pre-approved escalation path that automatically promotes to Tier 2 (execute-and-notify) after a configurable timeout, provided the confidence score exceeds a threshold.

```
-- Approval queue schema (T-SQL equivalent)
CREATE TABLE dbo.AutonomousActionProposals (
    ProposalId          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
    ActionClass         NVARCHAR(100) NOT NULL,
    TargetDatabase      NVARCHAR(128) NOT NULL,
    TargetEnvironment   NVARCHAR(20) NOT NULL
                        CHECK (TargetEnvironment IN
('production','staging','dev')),
    ProposedSQL         NVARCHAR(MAX),
    ProposedConfig      NVARCHAR(MAX),    -- JSON stored as NVARCHAR for pre-2016
compat
    TriggeringSignal    NVARCHAR(MAX) NOT NULL,
    ProjectedOutcome    NVARCHAR(1000) NOT NULL,
    RollbackProcedure   NVARCHAR(MAX) NOT NULL,
    ConfidenceScore     DECIMAL(4,3) NOT NULL,
    OversightTier       TINYINT NOT NULL,
    ProposedAt          DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    ExpiresAt           DATETIMEOFFSET NOT NULL,
    ApprovedBy          NVARCHAR(256),
    ApprovedAt          DATETIMEOFFSET,
    RejectedBy          NVARCHAR(256),
    RejectedAt          DATETIMEOFFSET,
    RejectionReason     NVARCHAR(1000),
    AutoEscalated       BIT DEFAULT 0,
    Status              NVARCHAR(20) NOT NULL DEFAULT 'pending'
                        CHECK (Status IN
('pending','approved','rejected','expired','executed','rolled_back'))
);

CREATE INDEX IX_Proposals_Status_Expires
ON dbo.AutonomousActionProposals (Status, ExpiresAt)
WHERE Status = 'pending';
```

The expiry mechanism is not optional. Proposals for time-sensitive actions — for example, an index build timed to run during a low-traffic window — must carry an expiry timestamp after which the proposal becomes invalid and a new proposal is generated for the next eligible window. This prevents stale approvals from executing actions that are no longer contextually appropriate.

A background process runs every minute, scanning for expired proposals and for proposals that have been sitting in pending status long enough to trigger urgency escalation:

```

# Approval watchdog: escalation and expiry enforcement
import pyodbc
import requests
from datetime import datetime, timezone

def process_expired_and_escalate(conn_str: str, slack_webhook: str):
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        # Expire stale proposals
        cur.execute("""
            UPDATE autonomous_action_proposals
            SET status = 'expired'
            OUTPUT INSERTED.proposal_id, INSERTED.action_class,
                   INSERTED.target_database
            WHERE status = 'pending'
            AND expires_at < SYSUTCDATETIME()
        """)
        expired = cur.fetchall()

        # Find proposals eligible for auto-escalation
        # Criteria: pending > 15 minutes, confidence >= 0.85, tier = 3
        cur.execute("""
            SELECT proposal_id, action_class, target_database,
                   proposed_sql, confidence_score, projected_outcome
            FROM autonomous_action_proposals
            WHERE status = 'pending'
            AND oversight_tier = 3
            AND confidence_score >= 0.85
            AND proposed_at < DATEADD(MINUTE, -15, SYSUTCDATETIME())
            AND auto_escalated = 0
            AND expires_at > SYSUTCDATETIME()
        """)
        candidates = cur.fetchall()

        for row in candidates:
            proposal_id, action_class, db, sql, score, outcome = row
            # Promote to tier 2 (execute-notify) and flag escalated
            cur.execute("""
                UPDATE autonomous_action_proposals
                SET oversight_tier = 2,
                    auto_escalated = 1,
                    status = 'approved',
                    approved_by = 'system:auto_escalation',
                    approved_at = SYSUTCDATETIME()
                WHERE proposal_id = ?
            """, (proposal_id,))

            # Notify via Slack
            payload = {
                "text": (
                    f":warning: *Auto-escalated proposal* approved after 15min"
                    f"*Action:* `{action_class}` on `{db}`\n"
                    timeout\n"

```

```

        f"*Confidence:* {score:.1%} | *Outcome:* {outcome}\n"
        f"*Proposal ID:* `{proposal_id}`"
    )
}
requests.post(slack_webhook, json=payload, timeout=5)

conn.commit()

```

The auto-escalation path is itself a policy decision that must be explicitly configured per action class and environment. It is never a default-on feature. For environments where no auto-escalation is acceptable — regulated industries, for example — the watchdog simply expires proposals and generates a fresh proposal for the next opportunity.

Explainability Layers: Making Autonomous Decisions Auditable and Contestable

An autonomous system that cannot explain its decisions is a liability. When a DBA is paged at 3:00 AM because an autonomous action degraded performance, the first question is not "how do I fix it" — it is "why did the system do this." If the answer requires reading Python source code, the system has failed its operators.

Explainability in autonomous database systems operates at three levels: decision-time rationale capture, post-action narrative generation, and structured audit trail queries.

Decision-time rationale capture means that at the moment the decision engine resolves an action, it records every signal that contributed to that decision, the weight or rule that connected signal to conclusion, and any alternatives that were considered and rejected. This is not a logging afterthought — it is a first-class output of the decision function.

When an LLM-based reasoning layer is involved in decision-making (a pattern covered in the preceding chapters), the system prompts the model to reason in a structured chain-of-thought format, and that reasoning chain is stored verbatim alongside the action record. Engineers can then read the model's reasoning as a narrative explanation.

Post-action narrative generation can be handled by a scheduled job that reads the structured audit record and generates a human-readable summary. For teams using LLM APIs, this is a simple prompt-and-store operation:

```

# Narrative generation for completed actions
import anthropic
import json
import pyodbc

def generate_action_narrative(audit_id: str, conn_str: str, anthropic_api_key: str):
    client = anthropic.Anthropic(api_key=anthropic_api_key)

    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT a.signals_observed, a.rules_fired, a.alternatives_considered,
                   a.final_action, a.execution_result, a.outcome_signals,
                   a.llm_reasoning_chain, p.target_database, p.projected_outcome
            FROM autonomous_action_audit a
            JOIN autonomous_action_proposals p ON p.proposal_id = a.proposal_id
            WHERE a.audit_id = ?
        """, (audit_id,))
        row = cur.fetchone()

    if not row:
        return

    signals, rules, alternatives, action, result, outcome_signals, chain, db,
    projected = row

    prompt = f"""
    You are summarizing an autonomous database management action for a DBA audit log.

    Database: {db}
    Action taken: {action}
    Observed signals: {json.dumps(signals, indent=2)}
    Rules that fired: {json.dumps(rules, indent=2)}
    Alternatives considered: {json.dumps(alternatives, indent=2)}
    Projected outcome: {projected}
    Actual execution result: {result}
    Post-action metrics: {json.dumps(outcome_signals, indent=2)}

    Write a 3-5 sentence professional narrative explaining what happened, why the system
    acted, what alternatives were rejected and why, and whether the outcome matched
    the projection. Be specific about metric values. Do not use filler language.
    """

    message = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        messages=[{"role": "user", "content": prompt}]
    )

    narrative = message.content[0].text

    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()

```

```

cur.execute("""
    UPDATE autonomous_action_audit
    SET execution_result = CONCAT(execution_result,
        CHAR(13) + CHAR(10) + CHAR(13) + CHAR(10)
        + '--- NARRATIVE ---' + CHAR(13) + CHAR(10) + ?)
    WHERE audit_id = ?
""", (narrative, audit_id))
conn.commit()

```

Structured audit trail queries allow operators to investigate patterns over time — which action classes fire most frequently, which confidence scores correlate with poor outcomes, which databases generate the most auto-escalations. These are operational intelligence queries, not forensic queries. Building a Grafana dashboard on top of the audit tables turns the oversight infrastructure into a continuous improvement signal.

```

-- Audit summary: action class outcomes over rolling 30 days
SELECT
    p.ActionClass,
    p.TargetEnvironment,
    COUNT(*) AS TotalProposals,
    SUM(CASE WHEN p.Status = 'executed' THEN 1 ELSE 0 END) AS Executed,
    SUM(CASE WHEN p.Status = 'rolled_back' THEN 1 ELSE 0 END) AS RolledBack,
    SUM(CASE WHEN p.Status = 'rejected' THEN 1 ELSE 0 END) AS Rejected,
    AVG(p.ConfidenceScore) AS AvgConfidence,
    AVG(a.ExecutionDurationMs) AS AvgDurationMs,
    SUM(CASE WHEN a.OutcomeVerified = 1 THEN 1 ELSE 0 END) AS VerifiedSuccess
FROM dbo.AutonomousActionProposals p
LEFT JOIN dbo.AutonomousActionAudit a ON a.ProposalId = p.ProposalId
WHERE p.ProposedAt >= DATEADD(DAY, -30, SYSDATETIMEOFFSET())
GROUP BY p.ActionClass, p.TargetEnvironment
ORDER BY TotalProposals DESC;

```

The contestability dimension of explainability is often overlooked. Operators must be able to dispute a completed autonomous action — not to reverse it necessarily, but to flag that the decision was wrong even if the outcome was acceptable, so that the policy engine can be recalibrated. This requires a feedback mechanism in the audit interface where a DBA can annotate an action with "outcome correct but reasoning flawed" or "action unnecessary given context." Those annotations feed directly into confidence model training.

Override Architecture: Restoring Human Control in Under Sixty Seconds

The override mechanism is the safety valve of an autonomous system. It must be engineered with the same rigor as the autonomous execution path itself. An override that requires five minutes of configuration changes or a Kubernetes pod restart is operationally useless in a production incident.

There are three distinct override scenarios, each requiring a different technical response:

Scenario 1: Halt new autonomous actions. An operator suspects the system is in a bad state and wants to freeze all new action execution while they investigate. This must be achievable with a single command.

Scenario 2: Abort an in-progress action. The system has begun executing a change — an index build, a statistics update, a configuration parameter change — and the operator needs to stop it immediately.

Scenario 3: Roll back a completed action. The system executed successfully but the outcome is undesirable. The operator needs to reverse the change.

Each scenario requires pre-built infrastructure, not ad-hoc recovery work done under pressure.

For Scenario 1, the system maintains a global autonomy state flag, readable and writable by any authenticated operator via a lightweight API endpoint. The execution loop polls this flag before dispatching any action:

```

# Autonomy gate: global halt mechanism
import redis
import functools
from typing import Callable

AUTONOMY_HALT_KEY = "ade:autonomy:halt"
AUTONOMY_HALT_TTL_SECONDS = 3600 # Auto-release after 1 hour if not renewed

def check_autonomy_gate(redis_client: redis.Redis) -> bool:
    """Returns True if system is permitted to act autonomously."""
    return redis_client.get(AUTONOMY_HALT_KEY) is None

def halt_autonomy(redis_client: redis.Redis, operator: str, reason: str) -> None:
    import json, time
    payload = json.dumps({
        "halted_by": operator,
        "reason": reason,
        "halted_at": time.time()
    })
    redis_client.setex(AUTONOMY_HALT_KEY, AUTONOMY_HALT_TTL_SECONDS, payload)

def resume_autonomy(redis_client: redis.Redis, operator: str) -> None:
    redis_client.delete(AUTONOMY_HALT_KEY)
    # Audit the resume event
    log_override_event(operator, "resume", reason="manual_resume")

def autonomous_action(redis_client: redis.Redis):
    """Decorator that gates any autonomous action function."""
    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            if not check_autonomy_gate(redis_client):
                raise RuntimeError(
                    "Autonomy gate is HALTED. Action blocked. "
                    "Resume via /api/autonomy/resume before proceeding."
                )
            return func(*args, **kwargs)
        return wrapper
    return decorator

```

The Redis key approach is deliberately simple and operationally accessible. Any tool that can write to Redis — a Slack bot, a CLI, a Kubernetes operator, a PagerDuty webhook — can halt the system. The TTL prevents a halt from persisting indefinitely if the operator who set it is unavailable to clear it.

For Scenario 2, the abort mechanism depends on the action type. Long-running queries can be terminated via database-native mechanisms:

```

-- Abort an autonomous action session in SQL Server
-- ADE workers connect with a recognizable application name
SELECT session_id, login_time, program_name, status, last_request_start_time
FROM sys.dm_exec_sessions
WHERE program_name LIKE 'ADE_Worker_%'
    AND status IN ('running', 'sleeping');

-- Kill the session
KILL 87; -- Replace with actual session_id from above query

-- For parallel queries, verify all workers are terminated
SELECT session_id, request_id, status, command, blocking_session_id
FROM sys.dm_exec_requests
WHERE session_id = 87;

```

For Scenario 3, rollback is pre-planned. Every autonomous action that modifies state must have its rollback procedure defined before execution begins. The rollback procedure is stored in the audit record at proposal time, not at execution time — because by execution time, the state that makes the rollback deterministic may have changed. For index operations, rollbacks are straightforward: drop the index. For configuration parameter changes, rollbacks restore the prior value. For data-affecting operations — statistics updates, for example — rollbacks may require restoring from a pre-action snapshot if the operation is non-reversible by SQL alone.

```

# Rollback executor: retrieves and applies stored rollback procedure
import pyodbc
import logging
from datetime import datetime, timezone

logger = logging.getLogger("ade.rollback")

def execute_rollback(proposal_id: str, operator: str, conn_str: str) -> dict:
    """
    Retrieves the pre-stored rollback procedure for a proposal and executes it.
    Returns a result dict with success status and details.
    """
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT p.proposal_id, p.rollback_procedure, p.proposed_sql,
                   p.target_database, p.status, a.audit_id
            FROM autonomous_action_proposals p
            LEFT JOIN autonomous_action_audit a ON a.proposal_id = p.proposal_id
            WHERE p.proposal_id = ?
            """, (proposal_id,))
        row = cur.fetchone()

    if not row:
        return {"success": False, "error": f"Proposal {proposal_id} not found"}

    pid, rollback_sql, original_sql, target_db, status, audit_id = row

    if status != 'executed':
        return {
            "success": False,
            "error": f"Proposal is in status '{status}'; rollback only applies to
'executed' actions"
        }

    if not rollback_sql:
        return {
            "success": False,
            "error": "No rollback procedure stored for this action"
        }

    rollback_start = datetime.now(timezone.utc)

    try:
        with pyodbc.connect(conn_str, autocommit=True) as conn:
            cur = conn.cursor()
            cur.execute(rollback_sql)

            rollback_duration_ms = int(
                (datetime.now(timezone.utc) - rollback_start).total_seconds() * 1000
            )

        # Update proposal status and audit record

```

```

with pyodbc.connect(conn_str) as conn:
    cur = conn.cursor()
    cur.execute("""
        UPDATE autonomous_action_proposals
        SET status = 'rolled_back'
        WHERE proposal_id = ?
    """, (proposal_id,))
    if audit_id:
        cur.execute("""
            UPDATE autonomous_action_audit
            SET execution_result = CONCAT(execution_result,
                CHAR(13) + CHAR(10) + CHAR(13) + CHAR(10)
                + '--- ROLLBACK ---' + CHAR(13) + CHAR(10)
                + ?, ' by ' + ?, ' at '
                + CONVERT(NVARCHAR(30), SYSUTCDATETIME(), 126))
            WHERE audit_id = ?
        """, (rollback_sql, operator, audit_id))
    conn.commit()

    logger.info(
        "Rollback completed for proposal %s by %s in %dms",
        proposal_id, operator, rollback_duration_ms
    )
    return {
        "success": True,
        "proposal_id": proposal_id,
        "rolled_back_by": operator,
        "duration_ms": rollback_duration_ms
    }

except Exception as exc:
    logger.error("Rollback failed for proposal %s: %s", proposal_id, exc)
    return {
        "success": False,
        "error": str(exc),
        "proposal_id": proposal_id
    }

```

The equivalent rollback pattern for SQL Server follows the same pre-stored procedure model. Because SQL Server's DDL operations are transactional in many cases, the rollback procedure for an index creation can often simply be:

```
-- SQL Server rollback for an autonomous index creation
-- Stored at proposal time as the rollback_procedure field
IF EXISTS (
    SELECT 1 FROM sys.indexes
    WHERE object_id = OBJECT_ID(N'dbo.orders')
    AND name = N'IX_orders_customer_id_created_at_ade_20241103'
)
BEGIN
    DROP INDEX IX_orders_customer_id_created_at_ade_20241103
    ON dbo.orders;
END
```

The naming convention — embedding `_ade_` and a datestamp in the index name — is not cosmetic. It is what makes rollback queries unambiguous and auditable without needing to cross-reference a separate data store.

Trust Boundary Expansion: How Autonomy Earns Its Own Latitude

The oversight tier assignments described in the opening section are not static configuration. A healthy autonomous database system earns expanded latitude over time by demonstrating reliable performance. Conversely, it should lose latitude automatically when it produces poor outcomes. This feedback loop — trust earned through demonstrated behavior, revoked by demonstrated failure — is what separates a mature ADE deployment from an automation script that happens to be running unattended.

Trust expansion operates at the action class level, not at the system level. The system might earn Tier 1 autonomy for index maintenance tuning while remaining at Tier 3 for partition management, because the execution history for each class is independent.

The trust model requires three inputs for each action class and environment pair:

1. **Execution history:** How many times has this action class been executed? A minimum threshold — for example, twenty successful executions — establishes the baseline before any downgrade is considered.
2. **Outcome verification rate:** Of executed actions, what fraction were confirmed successful by the post-execution signal check? An action that executes without error but does not produce the expected metric improvement counts against this rate.
3. **Rollback rate:** Of executed actions, what fraction required rollback? Even a single rollback event within a rolling window should pause downgrade consideration.

```

# Trust model: automated tier downgrade evaluation
from dataclasses import dataclass
from typing import Optional
import pyodbc

@dataclass
class TrustEvaluation:
    action_class: str
    environment: str
    current_tier: int
    recommended_tier: int
    reason: str
    eligible_for_change: bool

def evaluate_trust(
    action_class: str,
    environment: str,
    conn_str: str,
    lookback_days: int = 60,
    min_executions: int = 20
) -> TrustEvaluation:

    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT
                COUNT(*)                                AS total,
                SUM(CASE WHEN p.status = 'executed'
                        THEN 1 ELSE 0 END)                AS executed,
                SUM(CASE WHEN p.status = 'rolled_back'
                        THEN 1 ELSE 0 END)                AS rolled_back,
                SUM(CASE WHEN a.outcome_verified = 1
                        THEN 1 ELSE 0 END)                AS
verified_success,
                AVG(p.confidence_score)                  AS
avg_confidence,
                MAX(p.oversight_tier)                    AS current_tier
            FROM autonomous_action_proposals p
            LEFT JOIN autonomous_action_audit a
                ON a.proposal_id = p.proposal_id
            WHERE p.action_class = ?
                AND p.target_environment = ?
                AND p.proposed_at >= DATEADD(DAY, -?, SYSUTCDATETIME())
            """, (action_class, environment, lookback_days))
        row = cur.fetchone()

    total, executed, rolled_back, verified, avg_confidence, current_tier = row

    if total is None or executed is None or executed < min_executions:
        return TrustEvaluation(
            action_class=action_class,
            environment=environment,
            current_tier=current_tier or 3,

```

```

        recommended_tier=current_tier or 3,
        reason=f"Insufficient history: {executed or 0} executions, need
{min_executions}",
        eligible_for_change=False
    )

    rollback_rate = (rolled_back or 0) / executed
    verification_rate = (verified or 0) / executed

    # Any rollback in the window blocks downgrade
    if rolled_back and rolled_back > 0:
        return TrustEvaluation(
            action_class=action_class,
            environment=environment,
            current_tier=current_tier,
            recommended_tier=min(current_tier + 1, 4), # Escalate
            reason=f"Rollback rate {rollback_rate:.1%} in last {lookback_days}
days",
            eligible_for_change=True
        )

    # High verification rate and confidence qualify for downgrade
    if verification_rate >= 0.90 and avg_confidence >= 0.80 and current_tier > 0:
        return TrustEvaluation(
            action_class=action_class,
            environment=environment,
            current_tier=current_tier,
            recommended_tier=current_tier - 1,
            reason=(
                f"Verification rate {verification_rate:.1%}, "
                f"avg confidence {avg_confidence:.2f} over {executed} executions"
            ),
            eligible_for_change=True
        )

    return TrustEvaluation(
        action_class=action_class,
        environment=environment,
        current_tier=current_tier,
        recommended_tier=current_tier,
        reason=f"Metrics acceptable but below downgrade threshold",
        eligible_for_change=False
    )

```

The recommended tier from this evaluation is never applied automatically. It is surfaced in a weekly trust review report, reviewed by a senior DBA or database reliability engineer, and applied through a version-controlled change to the policy configuration. The automation surfaces the recommendation; the human makes the decision. This is the correct division of labor for a system that is still accumulating its own operational history.

Trust revocation, by contrast, can be automatic and immediate. A rollback event on a production database triggers an instant tier escalation for that action class in that environment, and that escalation holds until the trust evaluation explicitly recommends a reduction. This asymmetry — slow to grant, fast to revoke — reflects sound risk management for systems that operate at database layer.

Operator Interfaces: Designing for Clarity Under Pressure

Humans interacting with autonomous systems in production do so under cognitive load — during incidents, during on-call rotations, during post-mortems. The operator interface must be designed for those conditions, not for the calm of a product demonstration. Clarity under pressure is an engineering requirement, not a UX aspiration.

The primary interface surfaces are: the approval notification (what the operator sees when a Tier 3 proposal arrives), the dashboard (what the operator reviews during routine oversight), and the incident view (what the operator needs when something is wrong).

Approval notification design. A proposal notification must answer five questions in the first three seconds of reading: What changed? Why did the system propose this? What happens if approved? What happens if rejected? How long do I have? Anything beyond those five elements belongs in a link to the full audit record, not in the notification itself.

A well-structured Slack approval notification looks like this:

```
*Autonomous Action Proposal* – Approval Required

*Database:*    prod-orders-sql01  (production)
*Action:*      CREATE INDEX ... WITH (ONLINE = ON) on Orders(CustomerId, CreatedAt)
*Why:*         Seq scan rate on orders increased 340% over 4h;
               query P99 latency 8.2s, threshold 2.0s
*If approved:* Index build ~18 min (ONLINE = ON, minimal locking).
               P99 latency projected to drop to 0.9s.
*If rejected:* No action taken. Re-proposed in 6h if condition persists.
*Rollback:*    DROP INDEX available; estimated 45s to execute.
*Confidence:*  91% | *Expires:* 47 min from now

[✅ Approve] [❌ Reject] [📄 Full details]
```

The numeric specificity — 340% increase, 8.2s latency, 18-minute build time, 0.9s projection — is drawn from the `signals_observed` and `projected_outcome` fields of the proposal record. These numbers are what allow a DBA to make an informed decision without opening a second tool. Generic language such as "performance degradation detected" is operationally useless.

Dashboard design. The oversight dashboard serves two audiences with different needs: the on-call DBA who needs real-time situational awareness, and the database reliability engineer who needs trend analysis for trust model calibration. These should be separate views within the same dashboard, not separate dashboards.

The real-time view surfaces: current autonomy gate status (halted or active), count of pending proposals by environment, most recent executed action and its outcome status, and any actions in rollback state. These four elements are sufficient for a rotating on-call engineer to assess system health in ten seconds.

The trend view surfaces: action class execution frequency over 30/60/90-day windows, outcome verification rates by action class, confidence score distributions, and tier distribution changes over time. These metrics directly feed the trust model evaluation process.

Incident view. When an autonomous action is suspected of contributing to a production incident, the operator needs a single view that shows: what actions executed in the preceding hour, in chronological order, with their full audit trails inline. The incident view is a time-scoped audit log with narrative summaries promoted to the primary display.

```
-- SQL Server: incident retrospective query
-- Returns all actions executed in a time window with narrative summaries
SELECT
    p.ProposedAt,
    p.ActionClass,
    p.TargetDatabase,
    p.ProposedSQL,
    p.ConfidenceScore,
    p.Status,
    a.ExecutedAt,
    a.ExecutionDurationMs,
    a.OutcomeVerified,
    a.OutcomeSignals,
    a.ExecutionResult
FROM dbo.AutonomousActionProposals p
JOIN dbo.AutonomousActionAudit a ON a.ProposalId = p.ProposalId
WHERE p.TargetDatabase = @TargetDatabase
      AND a.ExecutedAt BETWEEN @WindowStart AND @WindowEnd
ORDER BY a.ExecutedAt ASC;
```

The incident view is also where the contestability mechanism described earlier should be accessible. A button or command alongside each action row that lets the operator flag the decision as flawed — even if the outcome was technically acceptable — is what closes the feedback loop from production operations into the trust model.

Compliance, Regulatory, and Change Management Integration

Autonomous database systems operating in regulated environments — financial services, healthcare, government — face additional constraints beyond operational risk. Change management processes, regulatory audit requirements, and separation-of-duties controls all intersect with the autonomous action pipeline in ways that must be anticipated in the architecture, not retrofitted after deployment.

Change management integration means that Tier 3 and Tier 4 proposals can be optionally linked to change management tickets in systems such as ServiceNow or Jira. The proposal record holds an external ticket ID, and approval in the autonomous system either mirrors approval in the change system or requires it as a prerequisite.

```

# Change management integration: ServiceNow ticket creation for Tier 3 proposals
import requests
import os
from typing import Optional

def create_servicenow_change_request(
    proposal: dict,
    servicenow_instance: str,
    servicenow_user: str,
    servicenow_password: str
) -> Optional[str]:
    """
    Creates a Standard Change Request in ServiceNow for a Tier 3 proposal.
    Returns the change request number (e.g., 'CHG0012345') or None on failure.
    """
    url = f"https://{servicenow_instance}.service-now.com/api/now/table/
change_request"
    headers = {"Content-Type": "application/json", "Accept": "application/json"}

    payload = {
        "short_description": (
            f"[ADE] {proposal['action_class']} on {proposal['target_database']}"
        ),
        "description": (
            f"Autonomous Database Engine proposal\n\n"
            f"Proposal ID: {proposal['proposal_id']}\n"
            f"Proposed SQL: {proposal.get('proposed_sql', 'N/A')}\n"
            f"Triggering signal: {proposal['triggering_signal']}\n"
            f"Projected outcome: {proposal['projected_outcome']}\n"
            f"Rollback procedure: {proposal['rollback_procedure']}\n"
            f"Confidence score: {proposal['confidence_score']}"
        ),
        "type": "standard",
        "category": "Database",
        "risk": "low" if proposal['confidence_score'] >= 0.85 else "medium",
        "impact": "2", # Medium impact default for database changes
        "urgency": "2",
        "assignment_group": "Database Reliability Engineering",
        "requested_by": "ade-service-account",
    }

    try:
        response = requests.post(
            url,
            auth=(servicenow_user, servicenow_password),
            headers=headers,
            json=payload,
            timeout=10
        )
        response.raise_for_status()
        change_number = response.json()["result"]["number"]
        return change_number
    except Exception as exc:

```

```

# Log but do not block the proposal – ITSM integration failure
# should not prevent the autonomous system from proposing actions
import logging
logging.getLogger("ade.itsm").warning(
    "ServiceNow ticket creation failed for proposal %s: %s",
    proposal['proposal_id'], exc
)
return None

```

Separation of duties in regulated environments often requires that the operator approving an autonomous action cannot be the same person who configured the policy that produced it. This constraint is enforced in the approval workflow by recording both the policy author and the approver, and rejecting approvals where those identities match.

Regulatory audit requirements typically demand that every change to a production database be traceable to an authorized change request, an approver identity, and a documented business justification. The `autonomous_action_proposals` and `autonomous_action_audit` tables, if populated correctly, satisfy these requirements. The LLM-generated narrative serves as the business justification field. The approver identity is the `approved_by` column. The change request linkage provides the authorization chain.

Periodic evidence exports — for SOC 2 auditors, for example — can be generated directly from these tables:

```

-- SQL Server: SOC 2 evidence export for a given audit period via bcp
bcp "SELECT
    p.proposal_id, p.action_class, p.target_database, p.target_environment,
    p.proposed_sql, p.confidence_score, p.oversight_tier, p.approved_by,
    p.approved_at, p.status, a.executed_at, a.outcome_verified, a.execution_result
FROM dbo.autonomous_action_proposals p
LEFT JOIN dbo.autonomous_action_audit a ON a.proposal_id = p.proposal_id
WHERE p.target_environment = 'production'
    AND p.proposed_at BETWEEN '2024-01-01' AND '2024-03-31'
ORDER BY p.proposed_at" queryout "ade_audit_Q1_2024.csv" -S $DB_SERVER -d $DB_NAME -
T -c -t, -r\n

```

The export is not the audit. The export is the artifact that demonstrates the audit trail exists and is complete. The underlying tables are the system of record, and their integrity — ensured by the append-only audit pattern and the foreign key relationships — is what gives the export evidentiary weight.

Key Takeaways

- → **Autonomy without oversight tiers is reckless automation.** Every autonomous action must carry a risk classification that routes it through an appropriate oversight model — from silent execution to human-only initiation — with escalation logic that responds to context: incident state, confidence score, and environmental boundaries.
- → **Approval workflows that block are worse than no automation.** Proposal expiry timestamps, differentiated notification channels, and carefully gated auto-escalation paths ensure that human review remains meaningful without becoming a bottleneck that negates the system's operational value.
- → **Explainability is a first-class engineering output, not a logging afterthought.** Decision-time rationale capture, LLM-generated narrative summaries, and structured audit trail queries allow operators to understand, contest, and learn from autonomous decisions — and that contestability loop feeds directly into the confidence model's continuous improvement.
- → **Override and rollback infrastructure must be pre-built and exercised, not assembled under pressure.** A single-command autonomy halt via Redis, database-native session termination for in-progress actions, and pre-stored rollback SQL defined at proposal time are the three layers that allow human control to be restored in under sixty seconds regardless of what the system was doing.
- → **Trust is earned action-class by action-class through measured outcomes, and revoked instantly on failure.** The oversight tier for a given action class drifts downward only after a human reviews a quantitative trust evaluation meeting verified-success and confidence thresholds; it escalates automatically and immediately on any rollback event, creating an asymmetry that keeps the system conservatively bounded as it earns expanded autonomy.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 30: Designing Fully Autonomous Database Architectures

The gap between a well-monitored database and a truly autonomous one is not instrumentation — it is agency. A monitored system tells you what is happening. An autonomous system observes, interprets, and acts without waiting for you. This chapter assembles every discipline covered so far — observability pipelines, AI-driven anomaly detection, self-healing failover, adaptive query optimization, capacity forecasting — into a unified architectural blueprint. The goal is not to automate individual tasks but to design a system whose default posture is autonomous operation, where human intervention is the exception and every exception is logged, learned from, and eventually eliminated.

The Four Pillars of Autonomous Database Architecture

Before any code is written or any topology is drawn, an autonomous database architecture must be grounded in four non-negotiable properties. These are not aspirational — they are structural constraints that every component of the system must satisfy.

Continuous Observability means that every meaningful signal — query latency percentiles, lock contention rates, replication lag, buffer hit ratios, storage growth velocity, connection pool saturation — is captured in real time and stored in a queryable time-series format. Prometheus — with a SQL Server exporter, Extended Events sessions, and scheduled DMV polling — forms the collection layer. The retention policy matters: short-term high-resolution data (15-second scrape intervals, 15-day retention) feeds the real-time decision engine; downsampled long-term data (1-minute resolution, 1-year retention) feeds the forecasting models.

Closed-Loop Control means the system does not merely alert — it acts. The OODA loop (Observe, Orient, Decide, Act) must complete in software, not in a human's inbox. Every action the system takes must flow through a centralized decision registry that records what was observed, what was decided, what was executed, and what the outcome was. This registry becomes the training corpus for the next generation of the decision model.

Bounded Autonomy means that autonomous action operates within explicit policy envelopes. A system that can do anything is a liability. A system that can do anything within defined boundaries — "reindex only during the maintenance window unless index bloat

exceeds 40% and query latency is degrading at more than 5ms per minute" — is an asset. Every autonomous agent in the architecture carries a policy document that defines its action space.

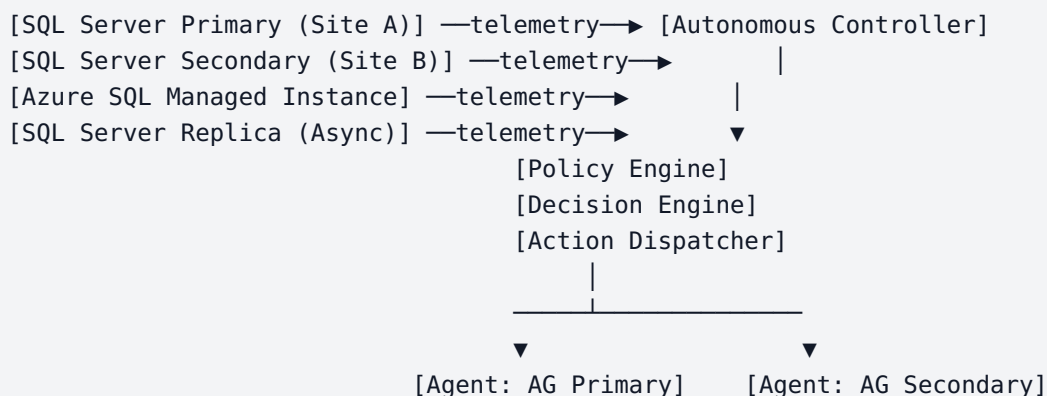
Graceful Degradation means the system fails toward safety, not toward aggression. When the decision engine cannot reach consensus, when sensor data is stale, when the LLM API is unavailable — the system falls back to conservative defaults, emits structured alerts, and waits for human input rather than guessing. Autonomy is not recklessness with better tooling.

These four pillars translate directly into infrastructure choices. Observability requires a pull-based metrics pipeline and structured log aggregation. Closed-loop control requires an orchestration layer with transactional state. Bounded autonomy requires a policy engine. Graceful degradation requires circuit breakers on every autonomous action path.

Topology Patterns: From Reactive Islands to Unified Autonomous Fabric

Most teams building toward autonomy start with reactive islands — a script here, an alert there, a cron job that handles a specific failure mode. The path to a unified autonomous fabric requires deliberate topology design.

The Hub-and-Spoke Control Plane places a central autonomous controller (the hub) that receives telemetry from all database nodes (the spokes), evaluates decisions, and dispatches actions back to agents running on each node. The hub runs the LLM-assisted decision engine, the policy registry, and the action log. The spokes run lightweight agents — Prometheus exporters, ODBC-driven health probes, and Always On Availability Group health monitors — that execute instructions and report outcomes.



The Distributed Consensus Pattern is appropriate when the hub-and-spoke introduces unacceptable single-point-of-failure risk. Here, three or five autonomous controller instances run a Raft consensus protocol (etcd is the standard substrate). No action is dispatched until a quorum agrees. This adds latency to the decision loop — typically 200–800ms — but is the correct choice for systems where a split-brain autonomous action could cause data loss.

For SQL Server high availability, the Windows Server Failover Cluster quorum — or the external witness used in distributed Always On Availability Groups — is the distributed coordination layer, and the autonomous controller can subscribe to AG health state changes from it.

The Sidecar Pattern deploys a small autonomous agent as a Kubernetes sidecar alongside each database pod. Each sidecar handles node-local decisions — connection draining, in-flight query cancellation, local storage alerts — while delegating cluster-level decisions to the control plane. This is the pattern used by operators such as the SQL Server operator for Kubernetes.

The Kubernetes operator model is worth examining in detail because it is the most mature autonomous database pattern available today. An operator encodes operational knowledge as a Kubernetes controller that continuously reconciles desired state with observed state. The reconciliation loop is itself an autonomous closed loop:

```

# Simplified autonomous reconciliation loop (Kubernetes operator pattern)
import time
import logging
from kubernetes import client, config
from database_agent import DatabaseAgent, PolicyEngine

config.load_incluster_config()
k8s = client.CustomObjectsApi()
policy = PolicyEngine.from_configmap("autonomous-db-policies")
logger = logging.getLogger("autonomous-controller")

def reconcile():
    while True:
        clusters = k8s.list_namespaced_custom_object(
            group="db.autonomous.io",
            version="v1",
            namespace="production",
            plural="sqlserverinstances"
        )
        for cluster in clusters["items"]:
            agent = DatabaseAgent(cluster)
            observations = agent.collect_telemetry()
            decisions = policy.evaluate(observations)
            for decision in decisions:
                if decision.confidence >= policy.min_confidence_threshold:
                    outcome = agent.execute(decision)
                    agent.log_action(decision, outcome)
                    if not outcome.success:
                        agent.escalate(decision, outcome)
                else:
                    logger.warning(
                        "Low-confidence decision skipped",
                        extra={"decision": decision.to_dict(),
                              "confidence": decision.confidence}
                    )
            time.sleep(policy.reconcile_interval_seconds)

reconcile()

```

The `min_confidence_threshold` is a policy parameter — not a magic number. For destructive or irreversible actions (dropping an index, initiating a failover, terminating long-running transactions), it is set at 0.95 or higher. For conservative tuning actions (adjusting `MAXDOP`, changing a query hint), 0.75 is sufficient.

Autonomous Decision Engine: From Metric Thresholds to Policy-Driven AI

The first generation of autonomous database tooling used simple threshold rules: if replication lag exceeds 30 seconds, page the on-call DBA. The second generation used threshold-triggered automation: if lag exceeds 30 seconds, pause replication-dependent reads. A true autonomous architecture operates at the third generation: policy-driven AI that reasons about the full system state before acting.

The decision engine has three layers:

1. **Signal normalization** — raw metrics are normalized into dimensionless signals that represent deviation from baseline. A `p99_query_latency` of 450ms means nothing without context. A normalized signal of `+3.2σ from 30-day rolling baseline` means the system is observing an unusual condition.
2. **State machine evaluation** — the normalized signals are evaluated against a finite state machine that tracks the database system's current health state (Nominal, Degraded, Critical, Recovering). State transitions trigger different action policies.
3. **AI-assisted decision synthesis** — for complex multi-signal conditions where the state machine transitions are ambiguous, the LLM layer synthesizes a recommendation from the full observable context.

The following shows how a SQL Server system emits normalized signals that feed the state machine:

```

-- Equivalent normalized signal computation for SQL Server
WITH baseline AS (
    SELECT
        metric_name,
        AVG(metric_value * 1.0) AS baseline_mean,
        STDEV(metric_value) AS baseline_stddev
    FROM dbo.db_metrics_hourly
    WHERE collected_at >= DATEADD(DAY, -30, GETUTCDATE())
    AND collected_at < DATEADD(HOUR, -1, GETUTCDATE())
    GROUP BY metric_name
),
current_values AS (
    SELECT
        metric_name,
        AVG(metric_value * 1.0) AS current_value
    FROM dbo.db_metrics_raw
    WHERE collected_at >= DATEADD(MINUTE, -5, GETUTCDATE())
    GROUP BY metric_name
)
SELECT
    c.metric_name,
    c.current_value,
    b.baseline_mean,
    b.baseline_stddev,
    CASE WHEN ISNULL(b.baseline_stddev, 0) = 0 THEN 0.0
        ELSE (c.current_value - b.baseline_mean) / b.baseline_stddev
    END AS z_score,
    CASE
        WHEN ABS(CASE WHEN ISNULL(b.baseline_stddev,0)=0 THEN 0.0
            ELSE (c.current_value - b.baseline_mean) / b.baseline_stddev
        END) > 4 THEN 'CRITICAL'
        WHEN ABS(CASE WHEN ISNULL(b.baseline_stddev,0)=0 THEN 0.0
            ELSE (c.current_value - b.baseline_mean) / b.baseline_stddev
        END) > 2.5 THEN 'DEGRADED'
        ELSE 'NOMINAL'
    END AS signal_state
FROM current_values c
JOIN baseline b ON c.metric_name = b.metric_name
ORDER BY ABS(z_score) DESC;

```

When multiple metrics simultaneously enter DEGRADED or CRITICAL state, the LLM synthesis layer is invoked. The controller builds a structured context payload — current signals, recent action history, schema fingerprint, active workload profile — and submits it to the reasoning API:

```

import anthropic
import json

def synthesize_autonomous_decision(signal_snapshot: dict, action_history: list,
                                   policy_envelope: dict) -> dict:
    """
    Submit multi-signal anomaly context to Claude for autonomous decision synthesis.
    Returns a structured decision object with action, rationale, and confidence.
    """
    client = anthropic.Anthropic()

    context = {
        "signals": signal_snapshot,
        "recent_actions": action_history[-10:], # last 10 autonomous actions
        "policy_constraints": policy_envelope,
        "system_state": "DEGRADED"
    }

    prompt = f"""You are the decision engine for an autonomous SQL Server cluster.
    Analyze the following system state and return a JSON decision object.

    System context:
    {json.dumps(context, indent=2)}

    Return ONLY valid JSON with this structure:
    {{
        "action": "<action_name>",
        "target": "<resource_identifier>",
        "parameters": {{}},
        "rationale": "<one paragraph explaining the decision>",
        "confidence": <float 0.0-1.0>,
        "reversible": <true|false>,
        "estimated_impact": "<low|medium|high>",
        "requires_human_approval": <true|false>
    }}

    Action must be one of: {policy_envelope['allowed_actions']}
    If no safe action exists, set action to "ESCALATE_TO_HUMAN"."""

    response = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}]
    )

    return json.loads(response.content[0].text)

```

The `requires_human_approval` field is the bridge between autonomy and oversight. For actions classified as high-impact or irreversible, the system sets this flag and routes to an approval workflow (Slack webhook, PagerDuty, or a web UI) while continuing to monitor the situation. If the human does not respond within the escalation timeout, the system falls back to the most conservative safe action it can take autonomously.

Self-Healing Infrastructure: Encoding Operational Runbooks as Autonomous Agents

Every runbook a DBA has ever written — every "if this, then that" procedure for handling connection storms, bloated tables, runaway queries, replication breakdowns — is a candidate for encoding as an autonomous agent. The key is structured representation: an agent has a trigger condition, an action sequence, a success criterion, a rollback procedure, and a cooldown period.

The following shows a complete autonomous agent for SQL Server index fragmentation remediation, implemented as an AG-aware procedure that respects availability topology before acting:

```
-- Index fragmentation detection feeding the SQL Server autonomous agent
SELECT
    DB_NAME() AS database_name,
    OBJECT_SCHEMA_NAME(i.object_id) AS schema_name,
    OBJECT_NAME(i.object_id) AS table_name,
    i.name AS index_name,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.avg_page_space_used_in_percent,
    CAST(ips.page_count * 8.0 / 1024 AS DECIMAL(10,2)) AS index_size_mb,
    CASE
        WHEN ips.avg_fragmentation_in_percent > 30 THEN 'REBUILD'
        WHEN ips.avg_fragmentation_in_percent > 10 THEN 'REORGANIZE'
        ELSE 'SKIP'
    END AS recommended_action
FROM sys.indexes i
CROSS APPLY sys.dm_db_index_physical_stats(
    DB_ID(), i.object_id, i.index_id, NULL, 'LIMITED') ips
WHERE i.type > 0 -- exclude heaps
    AND ips.page_count > 1000 -- > 8MB indexes
    AND ips.avg_fragmentation_in_percent > 10
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

The Python agent that consumes this data, validates AG topology by checking replica roles, and executes remediation demonstrates how operational intelligence wraps raw SQL:

```

import pyodbc
import logging
import time
from datetime import datetime, timezone
from dataclasses import dataclass

@dataclass
class IndexFragmentationAgent:
    """
    Autonomous index fragmentation remediation agent for SQL Server.
    Respects AG topology, maintenance windows, and active workload pressure.
    """
    conn_str: str
    policy: dict
    logger: logging.Logger = None

    def __post_init__(self):
        self.logger = logging.getLogger(self.__class__.__name__)

    def is_primary(self) -> bool:
        with pyodbc.connect(self.conn_str) as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT TOP 1 ars.role_desc
                FROM sys.dm_hadr_availability_replica_states ars
                JOIN sys.availability_replicas ar
                ON ars.replica_id = ar.replica_id
                WHERE ars.is_local = 1
            """)
            row = cur.fetchone()
            return row is not None and row[0] == "PRIMARY"

    def active_workload_pressure(self) -> float:
        """Return normalized workload pressure 0.0-1.0"""
        with pyodbc.connect(self.conn_str) as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT
                    SUM(CASE WHEN status = 'running' THEN 1 ELSE 0 END) AS active,
                    SUM(CASE WHEN blocking_session_id > 0 THEN 1 ELSE 0 END) AS
blocked,
                    COUNT(*) AS total
                FROM sys.dm_exec_requests
                WHERE session_id > 50
            """)
            active, blocked, total = cur.fetchone()
            if not total:
                return 0.0
            return min(1.0, (active + blocked * 2.0) / max(total, 1))

    def in_maintenance_window(self) -> bool:
        now = datetime.now(timezone.utc)
        window_start = self.policy.get("maintenance_window_start_utc", 2) # 02:00

```

```

UTC
    window_end    = self.policy.get("maintenance_window_end_utc", 6) # 06:00
UTC
    return window_start <= now.hour < window_end

def run(self):
    if not self.is_primary():
        self.logger.info("Not primary – skipping index remediation")
        return

    if not self.in_maintenance_window():
        self.logger.info("Outside maintenance window – deferring")
        return

    if self.active_workload_pressure() >
self.policy.get("max_workload_pressure", 0.4):
        self.logger.warning("Workload pressure exceeds threshold – aborting")
        return

    with pyodbc.connect(self.conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT
                OBJECT_SCHEMA_NAME(i.object_id) AS schema_name,
                OBJECT_NAME(i.object_id)       AS table_name,
                i.name                          AS index_name,
                ips.avg_fragmentation_in_percent
            FROM sys.indexes i
            CROSS APPLY sys.dm_db_index_physical_stats(
                DB_ID(), i.object_id, i.index_id, NULL, 'LIMITED') ips
            WHERE i.type > 0
                AND ips.page_count > 1000
                AND ips.avg_fragmentation_in_percent > ?
            ORDER BY ips.avg_fragmentation_in_percent DESC
            """, (self.policy.get("min_fragmentation_pct", 10),))
        targets = cur.fetchmany(self.policy.get("max_indexes_per_run", 3))

    for schema, table, index, frag in targets:
        action = "REBUILD" if frag > 30 else "REORGANIZE"
        self.logger.info(f"Autonomous index {action}: {schema}.{table}.{index}
({frag:.1f}% frag)")
        start = time.time()
        try:
            with pyodbc.connect(self.conn_str, autocommit=True) as op_conn:
                op_cur = op_conn.cursor()
                if action == "REBUILD":
                    op_cur.execute(
                        f"ALTER INDEX [{index}] ON [{schema}].[{table}] "
                        "REBUILD WITH (ONLINE = ON)"
                    )
                else:
                    op_cur.execute(
                        f"ALTER INDEX [{index}] ON [{schema}].[{table}]

```



```

REORGANIZE"
        )
        elapsed = time.time() - start
        self.logger.info(
            f"Index maintenance complete: {schema}.{table}.{index} in
{elapsed:.1f}s"
        )
    except Exception as exc:
        self.logger.error(
            f"Index maintenance failed for {schema}.{table}.{index}: {exc}"
        )
        break # conservative: stop on first failure

```

This agent encodes seven distinct pieces of operational intelligence that previously lived in a human runbook: topology check, maintenance window check, workload pressure check, size threshold, recency filter, per-run index limit, and failure-stop behavior. Each of those parameters is externalized in the policy dictionary, making the agent configurable without code changes.

The equivalent pattern for SQL Server uses Agent Jobs or a PowerShell scheduled task that invokes the fragmentation query, evaluates the policy, and executes `ALTER INDEX ... REBUILD WITH (ONLINE = ON)` or `REORGANIZE` based on fragmentation severity. The SQL Server Elastic Jobs feature in Azure SQL allows this to be deployed as a cloud-native scheduled task across multiple databases without maintaining a dedicated agent host.

Autonomous Failover, Traffic Routing, and Connection Fabric

Failover automation is the highest-stakes autonomous action in the architecture. A wrong decision — promoting a lagging replica, failing over when the primary is still healthy, triggering a failover loop — can cause data loss or extended downtime worse than the original failure. The autonomous architecture treats failover as a multi-agent consensus problem, not a single-sensor trigger.

For SQL Server, Always On Availability Groups are the standard substrate. Automatic failover is driven by the Windows Server Failover Cluster detecting that the SQL Server resource has failed, governed by the failure condition level and health check timeout. The ADE builds on top of the AG rather than replacing it, adding a policy layer that controls when a planned failover or forced manual failover is allowed.

The following AG configuration demonstrates bounded autonomy in action — the failure condition level and health check timeout bound when automatic failover may trigger, and a lag policy prevents a significantly behind secondary from being a planned-failover target:

```

-- AG autonomous failover policy configuration
-- Failure condition level 3: fail over on critical server errors or moderate SQL
Server errors.
-- Health check timeout bounds how long the cluster waits before declaring the
primary dead.
ALTER AVAILABILITY GROUP [AG_Orders]
    SET (FAILURE_CONDITION_LEVEL = 3, HEALTH_CHECK_TIMEOUT = 30000);

-- Bounded autonomy: allow automatic failover only to synchronous-commit
secondaries.
-- The controller evaluates this before permitting a planned failover:
SELECT ar.replica_server_name,
       ar.availability_mode_desc,
       ar.failover_mode_desc
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar ON ag.group_id = ar.group_id
WHERE ag.name = N'AG_Orders';

```

The following Python wrapper demonstrates how the autonomous controller layers policy evaluation over the AG's built-in logic, adding pre-flight checks before a planned failover is permitted to proceed:

```

import requests
import logging
import subprocess
import pyodbc
import logging
from dataclasses import dataclass
from typing import Optional

@dataclass
class AGFailoverPolicy:
    """
    Policy wrapper around Always On AG failover.
    Adds pre-flight validation before allowing a planned failover to proceed.
    """
    conn_str: str # connects to the current primary replica
    ag_name: str
    policy: dict
    logger: logging.Logger = None

    def __post_init__(self):
        self.logger = logging.getLogger(self.__class__.__name__)

    def get_replica_state(self) -> list:
        with pyodbc.connect(self.conn_str) as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT ars.replica_server_name,
                       ars.role_desc,
                       ars.synchronization_health_desc,
                       drs.log_send_queue_size,
                       drs.redo_queue_size,
                       drs.synchronization_state_desc
                FROM sys.dm_hadr_availability_replica_states ars
                JOIN sys.availability_replicas ar
                  ON ars.replica_id = ar.replica_id
                JOIN sys.availability_groups ag
                  ON ag.group_id = ar.group_id
                LEFT JOIN sys.dm_hadr_database_replica_states drs
                  ON ars.replica_id = drs.replica_id
                WHERE ag.name = ?
            """, (self.ag_name,))
            return cur.fetchall()

    def best_secondary_for_failover(self) -> Optional[str]:
        """
        Select the synchronous secondary with the smallest redo queue
        that satisfies all policy constraints.
        """
        rows = self.get_replica_state()
        max_lag = self.policy["max_redo_queue_bytes_for_failover"]
        candidates = [
            r for r in rows
            if r[1] == "SECONDARY"

```

```

        and r[2] == "HEALTHY"
        and (r[4] or 0) <= max_lag
    ]
    if not candidates:
        return None
    return min(candidates, key=lambda r: r[4] or 0)[0]

def execute_planned_failover(self, candidate: str) -> bool:
    """
    Issue a planned failover to the named secondary via T-SQL.
    Returns True on success, False on refusal.
    """
    max_lag = self.policy["max_redo_queue_bytes_for_failover"]
    rows = self.get_replica_state()
    match = [r for r in rows if r[0] == candidate]
    lag = match[0][4] if match else None
    if lag is None or lag > max_lag:
        self.logger.error(
            f"Failover refused: candidate {candidate} redo queue {lag} "
            f"exceeds policy maximum {max_lag}"
        )
        return False

    try:
        with pyodbc.connect(self.conn_str, autocommit=True) as conn:
            cur = conn.cursor()
            # Planned failover is issued against the primary replica
            cur.execute(f"ALTER AVAILABILITY GROUP [{self.ag_name}] FAILOVER")
            self.logger.info(
                f"Autonomous planned failover of {self.ag_name} to {candidate} "
                f"succeeded"
            )
            return True
    except Exception as exc:
        self.logger.error(f"Failover command failed: {exc}")
        return False

def evaluate_and_act(self):
    rows = self.get_replica_state()
    primary_healthy = any(
        r[1] == "PRIMARY" and r[2] == "HEALTHY" for r in rows
    )
    if primary_healthy:
        return # no action needed

    self.logger.warning("Primary unhealthy – evaluating failover candidates")
    candidate = self.best_secondary_for_failover()
    if candidate:
        self.execute_planned_failover(candidate)
    else:
        self.logger.error(
            "No eligible failover candidate – escalating to human"
        )

```

```
)  
# Trigger PagerDuty / Slack escalation via the central alert bus
```

For SQL Server Always On Availability Groups, the equivalent autonomous failover mechanism is the Flexible Failover Policy, which evaluates failure condition levels against the AlwaysOn health session (`sp_server_diagnostics`). SQL Server's built-in automatic failover is triggered by the Windows Server Failover Cluster detecting that the SQL Server resource has failed. The ADE adds a layer above this by monitoring extended events and health session outputs to detect degraded-but-not-failed conditions — such as secondary lag approaching the RTO threshold — and initiating a planned switchover before the situation becomes a forced failover.

SQL Server — monitoring AG replica health for pre-emptive switchover decisions:

```
-- Autonomous pre-failover health assessment for Always On AG  
SELECT  
    ag.name                                AS ag_name,  
    ars.role_desc,  
    ars.operational_state_desc,  
    ars.connected_state_desc,  
    ars.synchronization_health_desc,  
    adc.database_name,  
    drs.synchronization_state_desc,  
    drs.log_send_queue_size,              -- bytes not yet sent to secondary  
    drs.log_send_rate,                   -- bytes/sec current send rate  
    drs.redo_queue_size,                 -- bytes applied but not hardened  
    drs.redo_rate,  
    CASE  
        WHEN drs.log_send_rate > 0  
        THEN CAST(drs.log_send_queue_size * 1.0 / drs.log_send_rate AS INT)  
        ELSE NULL  
    END                                  AS estimated_lag_seconds,  
    drs.last_hardened_time,  
    drs.last_received_time  
FROM sys.availability_groups ag  
JOIN sys.availability_replicas ar  
    ON ag.group_id = ar.group_id  
JOIN sys.dm_hadr_availability_replica_states ars  
    ON ar.replica_id = ars.replica_id  
JOIN sys.availability_databases_cluster adc  
    ON ag.group_id = adc.group_id  
JOIN sys.dm_hadr_database_replica_states drs  
    ON adc.group_database_id = drs.group_database_id  
    AND ars.replica_id = drs.replica_id  
ORDER BY ag.name, ars.role_desc, adc.database_name;
```

When `estimated_lag_seconds` for a synchronous-commit secondary exceeds the policy threshold, the autonomous controller escalates rather than acts: a synchronous-commit secondary that is lagging is itself a warning sign of a network or I/O problem on that replica, and forcing a failover to it would likely result in a longer outage than investigating the root cause.

Connection Fabric Autonomy

The AG listener is the standard connection routing layer for SQL Server. The autonomous architecture makes the listener routing dynamic — when a planned failover changes which replica is primary, the listener follows the primary automatically, and the controller updates the read-only routing list so lagging secondaries stop receiving read traffic within seconds, with no human involvement:

```
# Autonomous read-only routing update for a lagging secondary
# Executed by the controller when a secondary's redo queue exceeds policy.
# Removes the lagging replica from the read-only routing list so the
# AG listener stops directing read traffic to it.
```

```
param(
    [string]$PrimaryServer,
    [string]$LaggingSecondary,
    [string]$AvailabilityGroup = "AG_Orders"
)
```

```
# Build the routing list from healthy secondaries only
```

```
$query = @"
    SELECT replica_server_name
    FROM sys.availability_read_only_routing_lists r
    JOIN sys.availability_replicas ar ON r.replica_id = ar.replica_id
    JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
    WHERE ag.name = 'AG_Orders'
    AND replica_server_name <> '$LaggingSecondary'
"@
```

```
$healthy = sqlcmd -S $PrimaryServer -E -h -l -W -Q $query
```

```
$routingList = ($healthy | Where-Object { $_.Trim() -ne "" } |
    ForEach-Object { "($_.Trim())" }) -join ","
```

```
$failover = @"
    ALTER AVAILABILITY GROUP [AG_Orders]
    MODIFY REPLICA ON '$LaggingSecondary'
    WITH (SECONDARY_ROLE (READ_ONLY_ROUTING_LIST = ($routingList)))
"@
```

```
sqlcmd -S $PrimaryServer -E -Q $failover
```

```
Write-Host "Removed $LaggingSecondary from read-only routing for $AvailabilityGroup"
```

The AG listener — a virtual network name that always resolves to the current primary — carries read-write traffic. The autonomous layer monitors listener resolution latency and AG health, and when the listener is pointing to a degraded primary, triggers the switchover rather than waiting for the application to time out.

Adaptive Query Optimization Without Human Intervention

Query plan instability is one of the most persistent sources of unplanned latency spikes. A statistics update, a table growth milestone, a new query pattern from a deployment — any of these can cause the planner to choose a dramatically suboptimal plan. The autonomous architecture detects plan regressions and responds before they become user-visible.

Query Store provides built-in plan forcing — the ADE queries the Query Store for regressed plans and forces the last-known-good plan automatically:

```

-- Autonomous Query Store plan regression remediation for SQL Server
-- Identifies regressed queries and forces the best-performing historical plan

WITH plan_stats AS (
    SELECT
        qsq.query_id,
        qsp.plan_id,
        qsq.query_sql_text,
        qsrs.avg_duration / 1000.0 AS avg_duration_ms,
        qsrs.avg_logical_io_reads,
        qsrs.count_executions,
        qsp.is_forced_plan,
        qsp.force_failure_count,
        RANK() OVER (
            PARTITION BY qsq.query_id
            ORDER BY qsrs.avg_duration ASC
        ) AS plan_rank
    FROM sys.query_store_query qsq
    JOIN sys.query_store_plan qsp ON qsq.query_id = qsp.query_id
    JOIN sys.query_store_runtime_stats qsrs ON qsp.plan_id = qsrs.plan_id
    JOIN sys.query_store_runtime_stats_interval qsrsi
        ON qsrs.runtime_stats_interval_id = qsrsi.runtime_stats_interval_id
    WHERE qsrsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
    AND qsrs.count_executions >= 50
),
best_plans AS (
    SELECT query_id, plan_id AS best_plan_id, avg_duration_ms AS best_avg_ms
    FROM plan_stats WHERE plan_rank = 1
),
current_plans AS (
    SELECT
        ps.query_id,
        ps.plan_id AS current_plan_id,
        ps.avg_duration_ms AS current_avg_ms,
        ps.query_sql_text,
        bp.best_plan_id,
        bp.best_avg_ms,
        ROUND(ps.avg_duration_ms / NULLIF(bp.best_avg_ms, 0), 2) AS regression_ratio
    FROM plan_stats ps
    JOIN best_plans bp ON ps.query_id = bp.query_id
    WHERE ps.plan_rank > 1 -- not the best plan
    AND ps.avg_duration_ms > bp.best_avg_ms * 2 -- more than 2x slower
    AND ps.is_forced_plan = 0 -- not already forced
)
SELECT
    query_id,
    LEFT(query_sql_text, 150) AS query_preview,
    current_avg_ms,
    best_avg_ms,
    regression_ratio,
    best_plan_id,
    -- The autonomous controller executes this statement to force the best plan:
    'EXEC sp_query_store_force_plan @query_id = ' + CAST(query_id AS VARCHAR)

```



```

+ ', @plan_id = ' + CAST(best_plan_id AS VARCHAR) + ';' AS remediation_sql
FROM current_plans
ORDER BY regression_ratio DESC;

```

The `remediation_sql` column is executed by the autonomous controller only when confidence is high: the regression ratio must exceed 2.0, the query must have at least 50 recent executions to establish statistical validity, and the action must not be attempted on a plan that has previously had `force_failure_count > 0`.

The Autonomous Learning Loop: Turning Incidents Into Policy

Every autonomous action — successful or not — is an event that should improve the system's future behavior. This requires a feedback architecture that connects action outcomes back to the policy engine.

The action log table is the foundation:

```

CREATE TABLE dbo.autonomous_action_log (
    action_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
    action_type        NVARCHAR(100) NOT NULL,
    target_resource    NVARCHAR(500) NOT NULL,
    trigger_signals    NVARCHAR(MAX) NOT NULL,    -- JSON
    parameters         NVARCHAR(MAX) NOT NULL,    -- JSON
    decision_source    NVARCHAR(50) NOT NULL,
    confidence         DECIMAL(4,3),
    executed_at        DATETIMEOFFSET NOT NULL DEFAULT SYSUTCDATETIME(),
    outcome            NVARCHAR(20),
    outcome_detail     NVARCHAR(MAX),            -- JSON
    resolved_at        DATETIMEOFFSET,
    human_reviewed     BIT DEFAULT 0,
    human_feedback     NVARCHAR(20),
    policy_version     NVARCHAR(50) NOT NULL
);

CREATE INDEX IX_action_log_type_time
    ON dbo.autonomous_action_log (action_type, executed_at DESC);
CREATE INDEX IX_action_log_outcome_feedback
    ON dbo.autonomous_action_log (outcome, human_feedback)
    WHERE human_reviewed = 1;

```

The learning loop operates on a weekly cadence. The controller queries the action log for actions that were marked `human_feedback = 'incorrect'` or `'unnecessary'`, groups them by trigger signal pattern, and generates candidate policy refinements. These refinements are surfaced to a human policy reviewer — not applied automatically. The learning loop trains human judgment, not just the system:

```

def generate_policy_refinement_candidates
import pyodbc
import json

def generate_policy_refinement_candidates(conn_str, policy_version: str) ->
list[dict]:
    """
    Analyze the action log for patterns of incorrect or unnecessary autonomous
    actions.
    Returns a list of candidate policy refinements for human review.
    """
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT
                action_type,
                decision_source,
                CAST(AVG(confidence) AS NUMERIC(4,3)) AS
avg_confidence_when_wrong,
                COUNT(*) AS wrong_action_count,
                STRING_AGG(
                    CONVERT(NVARCHAR(MAX), trigger_signals), CHAR(10))
                AS signal_examples
            FROM dbo.autonomous_action_log
            WHERE human_reviewed = 1
                AND human_feedback IN ('incorrect', 'unnecessary')
                AND policy_version = ?
            GROUP BY action_type, decision_source
            HAVING COUNT(*) >= 3
            ORDER BY wrong_action_count DESC
            """, (policy_version,))

        candidates = []
        for row in cur.fetchall():
            action_type, source, avg_conf, count, examples = row
            candidates.append({
                "action_type": action_type,
                "decision_source": source,
                "avg_confidence_when_wrong": float(avg_conf),
                "wrong_action_count": count,
                "signal_examples": examples,
                "suggested_refinement": (
                    f"Consider raising confidence threshold for '{action_type}' "
                    f"from current level (avg wrong at {avg_conf:.2f}) "
                    f"or restricting trigger conditions based on {count} "
                    f"observed incorrect activations."
                )
            })
        return candidates

```

This output is pushed to a weekly review dashboard where the senior DBA team reviews patterns, adjusts confidence thresholds, adds exclusion conditions, and increments the policy version. Over time, the wrong-action rate falls toward zero — not because the AI becomes perfect, but because the policy envelope is progressively tightened around the conditions where the AI has demonstrated reliability.

Observability for Autonomy: Watching the Watcher

An autonomous system introduces a new observability problem: the system must be observable not just at the database layer but at the autonomy layer itself. What decisions has the controller made? How confident was it? How often is it escalating? Is the decision loop completing within its latency budget?

The following metrics form the minimum viable observability surface for the autonomous controller itself:

Controller health metrics to expose via Prometheus:

```

from prometheus_client import Counter, Histogram, Gauge, start_http_server
import time

# Decision loop metrics
decisions_total = Counter(
    'autonomous_controller_decisions_total',
    'Total autonomous decisions made',
    ['action_type', 'decision_source', 'outcome']
)

decision_confidence = Histogram(
    'autonomous_controller_decision_confidence',
    'Confidence score distribution for autonomous decisions',
    ['action_type'],
    buckets=[0.5, 0.6, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 0.99, 1.0]
)

escalations_total = Counter(
    'autonomous_controller_escalations_total',
    'Decisions escalated to human due to low confidence or high impact',
    ['action_type', 'reason']
)

decision_loop_latency = Histogram(
    'autonomous_controller_loop_latency_seconds',
    'Time for one full observe-orient-decide-act cycle to complete',
    buckets=[0.1, 0.25, 0.5, 1.0, 2.0, 5.0, 10.0, 30.0]
)

policy_violations_total = Counter(
    'autonomous_controller_policy_violations_total',
    'Actions attempted outside policy boundaries (blocked before execution)',
    ['action_type', 'violation_type']
)

controller_health = Gauge(
    'autonomous_controller_health',
    'Overall health of the autonomous controller (1=healthy, 0=degraded)',
)

# Expose on port 9200 for Prometheus scraping
start_http_server(9200)

```

The `escalations_total` metric is particularly telling. A rising escalation rate is a leading indicator that either the policy confidence thresholds are too conservative for the current operational environment (the system is being too cautious) or that the system is encountering a new class of anomaly it was not designed to handle. Both interpretations require human attention, but for different reasons.

A Grafana dashboard built on these metrics should display:

- → Decision throughput (decisions per hour, broken down by action type)
- → Confidence score distribution over time
- → Escalation rate as a percentage of total decisions
- → Policy violation count (should trend toward zero as policies mature)
- → Loop latency percentiles (p50, p95, p99)
- → Outcome success rate by action type

When `outcome_success_rate` for a given action type falls below 90%, the autonomous controller should automatically disable that action type and log a structured alert. A system that repeatedly takes ineffective action is worse than a system that escalates conservatively.

Deployment Strategy: Crawl, Walk, Run

No organization deploys a fully autonomous database architecture on day one. The path to autonomy is incremental, and the deployment strategy must match organizational risk tolerance.

Phase 1 — Observe Only (Weeks 1–4)

Deploy the observability pipeline: Prometheus exporters, Grafana dashboards, structured log aggregation. Configure the decision engine in dry-run mode — it evaluates decisions and logs what it would have done, but takes no action. Review the dry-run log daily. This phase answers: does the decision engine produce sensible outputs for your specific workload?

Phase 2 — Automate the Safe (Weeks 5–12)

Enable autonomous action for a narrow set of low-risk, fully reversible actions: adjusting `MAXDOP` within a bounded range, killing idle-in-transaction connections older than the policy threshold, refreshing statistics on tables with high modification counts. Human review of the action log is weekly. This phase answers: does the system behave correctly in production?

Phase 3 — Automate the Consequential (Months 4–6)

Enable index remediation, query plan forcing, and connection pool resizing. All actions of this type require confidence ≥ 0.90 and are subject to the maintenance window policy. Human review transitions from weekly to monthly exception review. This phase answers: can the system handle the full operational surface without constant supervision?

Phase 4 — Automate the High-Stakes (Month 7 and beyond)

Enable failover automation, scaling actions (node addition, volume expansion), and query cancellation for long-running transactions. These actions have the strictest confidence thresholds and the richest pre-flight checklists. The learning loop is mature enough that the human review burden is minimal. This phase is never complete — new query patterns, new infrastructure, new failure modes always generate new learning opportunities.

End-to-End Example: A Complete Autonomous Response to a Replication Crisis

To make the architecture concrete, trace the path of a single incident through every layer of the autonomous system. At 02:47 UTC, the AG primary replica begins experiencing unusually high log generation due to a bulk import job. The redo queue on the synchronous secondary begins to climb.

02:47:15 — The Prometheus scrape captures `redo_queue_size` (from `sys.dm_hadr_database_replica_states`) rising above the 5-second lag equivalent. The normalized signal computes a z-score of +2.8, which crosses the DEGRADED threshold.

02:47:30 — The state machine transitions the cluster from NOMINAL to DEGRADED. The decision engine queries the policy for DEGRADED state actions.

02:47:31 — The policy engine identifies two candidate actions within the DEGRADED action space: (1) throttle the bulk import job by reducing its `MAXDOP` setting, (2) temporarily remove the lagging secondary from the read-only routing list to prevent stale reads. Both are within the maintenance window.

02:47:45 — The decision engine evaluates both actions. Action 1 confidence: 0.82 (the bulk import query signature matches the active-session pattern in the historical action log). Action 2 confidence: 0.94 (connection-routing adjustments have a high historical success rate). Both exceed the DEGRADED-state threshold of 0.75.

02:47:46 — The action dispatcher executes Action 2 first (higher confidence, no workload impact). The read-only routing list is updated via `sqlcmd` to remove the lagging secondary from the read pool. The action is logged.

02:47:52 — The action dispatcher executes Action 1. A `KILL` is sent to the bulk import session, which the controller then re-launches with `MAXDOP 2` via `OPTION (MAXDOP 2)`. This is achieved via a policy-controlled administrative role granted to the autonomous controller's login.

02:49:10 — Replication lag begins declining. By 02:51, it has returned below 1 second. The z-score falls below 1.0.

02:51:30 — The state machine transitions from DEGRADED to RECOVERING. The policy for RECOVERING state triggers: restore the lagging secondary to the read-only routing list (Action 2 reversal). The routing list is updated. MAXDOP is restored to default (Action 1 reversal). Both reversals are logged with `outcome = 'success'`.

02:52:00 — The state machine transitions to NOMINAL. Total autonomous incident duration: 4 minutes and 45 seconds. No human was contacted. The action log records six entries: two primary actions, two reversals, two state transitions. The morning review queue shows the incident summary with full signal trace and action rationale for human awareness.

This is what a mature autonomous architecture looks like in operation — not dramatic AI reasoning, but disciplined, policy-bounded closed-loop control executing faster and more consistently than any on-call rotation could.

```
sequenceDiagram
    participant Prometheus
    participant DecisionEngine as Decision Engine
    participant PolicyEngine as Policy Engine
    participant Dispatcher as Action Dispatcher
    participant AGLListener as AG Listener
    participant Primary as SQL Server Primary

    Prometheus->>DecisionEngine: redo_queue rising (z-score +2.8)
    DecisionEngine->>DecisionEngine: State: NOMINAL to DEGRADED
    DecisionEngine->>PolicyEngine: Request candidate actions for DEGRADED
    PolicyEngine-->>DecisionEngine: Throttle import (0.82), reroute reads (0.94)
    DecisionEngine->>Dispatcher: Execute both (confidence exceeds 0.75 threshold)
    Dispatcher->>AGListener: Remove lagging secondary from routing list
    Dispatcher->>Primary: KILL import session, relaunch with MAXDOP 2
    Primary-->>Prometheus: Redo queue declining
    Prometheus->>DecisionEngine: Lag normalized, z-score below 1.0
    DecisionEngine->>DecisionEngine: State: DEGRADED to RECOVERING to NOMINAL
    Dispatcher->>AGListener: Restore secondary to routing list
    Dispatcher->>Primary: Restore MAXDOP to default
```

The closed-loop sequence from the replication-crisis walkthrough: telemetry drives a state transition, the policy engine proposes bounded actions, and the dispatcher executes and later reverses them as conditions normalize.

Key Takeaways

- → **Autonomy is architecture, not automation.** Individual scripts and alerts do not constitute an autonomous system. The four pillars — continuous observability, closed-loop control, bounded autonomy, and graceful degradation — must be designed into the system's structure from the beginning, not added as layers on top of a reactive foundation.

- → **The policy envelope is the trust boundary.** Every autonomous agent operates within an explicitly defined action space, confidence threshold, and rollback procedure. The system is never given permission to do anything; it is given permission to do specific things under specific conditions. Expanding the policy envelope is a deliberate, reviewed act, informed by the action log's outcome history.
- → **The learning loop converts incidents into policy refinements.** The autonomous action log is not an audit trail — it is a training dataset. Human review of incorrect and unnecessary actions, aggregated over weeks, produces candidate policy refinements that progressively tighten the system's behavior around conditions it has demonstrated it can handle reliably.
- → **Observability must extend to the autonomous layer itself.** The decision engine, the policy evaluator, and the action dispatcher are all first-class components that must expose metrics, publish structured logs, and participate in alerting. A controller that silently malfunctions is more dangerous than a manually operated system with noisy alerts.
- → **Deploy incrementally by risk tier, not by component.** The crawl-walk-run deployment strategy ensures that each phase produces a production-validated answer to a specific question about system behavior before the next phase expands autonomous authority. Organizations that attempt full autonomy without the foundation of Phase 1 and Phase 2 experience exactly the kind of autonomous-action failures that erode organizational trust in the entire approach.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 31: Multi-Cloud Autonomous Database Operations

Operating a database fleet across AWS, Azure, and GCP simultaneously is not a distribution problem — it is a control problem. When your SQL Server Always On groups span Azure East US and West Europe, your SQL Server replicas on EC2 run in AWS us-east-1, and your analytical replicas live on Cloud SQL for SQL Server on GCP, the question is never "can these systems communicate?" It is "who decides, and how fast?" This chapter addresses the architecture of autonomous control planes that span cloud providers, treating each cloud as an execution zone rather than a decision-maker. An ADE operating in multi-cloud must detect cross-cloud latency degradation, reroute workloads, failover replicas, and rebalance connection pools — all without waiting for a human to open a console.

Topology Modeling: Representing Multi-Cloud Database Fleets as a Control Graph

Before any autonomous action is possible, the system must have a precise, machine-readable model of its own topology. A database fleet is not a list of hostnames — it is a directed graph where nodes are database instances and edges carry properties: replication lag in milliseconds, network RTT, cost-per-GB of data transfer, and current health state.

The autonomous control plane maintains this graph in a metadata store — typically a strongly consistent key-value system like etcd or Consul — with updates pushed by health agents running as sidecars to every database node. Each agent emits a heartbeat every five seconds carrying latency samples, replication offset, and a local cluster state hash. The control plane ingests these events and continuously recomputes the topology graph.

```

-- Topology registry in the control plane SQL Server database
CREATE TABLE dbo.db_topology (
    node_id                UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
    cloud_provider          NVARCHAR(10) NOT NULL
                           CHECK (cloud_provider IN ('aws','azure','gcp')),
    region                 NVARCHAR(64) NOT NULL,
    availability_zone       NVARCHAR(64) NULL,
    role                   NVARCHAR(20) NOT NULL
                           CHECK (role IN ('primary','replica','arbiter')),
    cluster_id             UNIQUEIDENTIFIER NOT NULL,
    replication_lag_ms      DECIMAL(10,3) NULL,
    network_rtt_ms          DECIMAL(10,3) NULL,
    data_transfer_cost_per_gb DECIMAL(10,6) NULL,
    health_state            NVARCHAR(20) NOT NULL DEFAULT 'healthy'
                           CHECK (health_state IN
('healthy','degraded','failed','unknown')),
    last_seen              DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    metadata                NVARCHAR(MAX) NULL -- JSON payload
);

-- Filtered index for failover candidate selection
CREATE INDEX IX_topology_failover_candidates
    ON dbo.db_topology (cluster_id, replication_lag_ms)
    INCLUDE (region, cloud_provider, node_id)
    WHERE health_state = 'healthy' AND role = 'replica';

-- Cross-cloud health summary
SELECT
    c.cluster_name,
    n.cloud_provider,
    SUM(CASE WHEN n.health_state = 'healthy' THEN 1 ELSE 0 END) AS healthy_nodes,
    SUM(CASE WHEN n.health_state = 'degraded' THEN 1 ELSE 0 END) AS degraded_nodes,
    SUM(CASE WHEN n.health_state = 'failed' THEN 1 ELSE 0 END) AS failed_nodes,
    MAX(n.replication_lag_ms) AS max_lag_ms,
    AVG(n.network_rtt_ms) AS avg_rtt_ms
FROM dbo.db_topology n
JOIN dbo.db_clusters c ON c.cluster_id = n.cluster_id
GROUP BY c.cluster_name, n.cloud_provider;

```

The graph model must encode cross-cloud edge costs. AWS us-east-1 to Azure East US carries roughly 30–50 ms RTT under normal conditions, but that number degrades predictably during specific BGP routing events. The autonomous system does not treat RTT as a static constant — it maintains a rolling 95th-percentile window of observed RTT per edge, updated every 60 seconds. When the p95 RTT on a replication channel exceeds 200 ms for three consecutive windows, the system classifies that edge as `degraded` and begins pre-positioning a promotion candidate in the lower-latency cloud zone.

The topology graph also feeds cost-awareness into autonomous decisions. Transferring 1 TB of transaction logs from AWS to GCP costs real money. The control plane embeds egress cost coefficients per provider pair and incorporates them into promotion scoring. A replica in Azure West Europe that is 20 ms slower than a GCP replica but saves \$0.09/GB in egress cost will be preferred for promotion in non-latency-sensitive workloads — but only if RPO targets are still satisfiable.

Cross-Cloud Replication Architecture: SQL Server AG and the Async Bridge

Synchronous replication across clouds is architecturally problematic. A commit that must be acknowledged by a replica in a different cloud introduces 30–150 ms of forced write latency on every transaction. For most production workloads this is unacceptable. The autonomous multi-cloud architecture instead relies on asynchronous replication with continuous lag monitoring and dynamic RPO enforcement.

For SQL Server, asynchronous-commit availability group replicas span cloud providers without requiring the shared cluster or low-latency network assumptions of synchronous replication. The autonomous system treats each cloud as an async replica zone hanging off the regional primary, with distributed availability groups linking independent AGs in different clouds where full topology control is required.

A critical architectural point: the control plane must distinguish between replication lag caused by network congestion versus lag caused by transaction volume. Both look identical in `redo_queue_size` readings, but they demand different responses. Network-induced lag requires rerouting; volume-induced lag requires replica scaling or workload throttling. The autonomous system classifies lag cause by cross-correlating redo queue growth rate against the primary's log generation rate:

```

import pyodbc
import time
from dataclasses import dataclass
from typing import Optional

@dataclass
class LagDiagnosis:
    lag_seconds: float
    log_generation_rate_bytes_per_sec: float
    cause: str # 'network' | 'volume' | 'secondary_overload' | 'unknown'
    recommended_action: str

def diagnose_replication_lag(
    conn_str: str,
    secondary_name: str,
    sample_interval_sec: int = 30
) -> LagDiagnosis:
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()

        def sample():
            cur.execute("""
                SELECT
                    drs.secondary_lag_seconds           AS lag_sec,
                    drs.log_send_queue_size             AS unsent_bytes,
                    drs.redo_queue_size                 AS unacked_bytes
                FROM sys.dm_hadr_database_replica_states drs
                JOIN sys.availability_replicas ar
                  ON drs.replica_id = ar.replica_id
                WHERE ar.replica_server_name = ?
            """, (secondary_name,))
            return cur.fetchone()

        s1 = sample()
        time.sleep(sample_interval_sec)
        s2 = sample()

        if s1 is None or s2 is None:
            return LagDiagnosis(0, 0, 'unknown', 'verify_secondary_connection')

        lag_sec      = s2[0] or 0
        log_rate      = (s2[1] - s1[1]) / sample_interval_sec # bytes/sec growth
        unacked_delta = s2[2] - s1[2] # growing means secondary can't keep up

        if lag_sec < 1:
            cause = 'acceptable'
            action = 'none'
        elif log_rate > 50_000_000 and unacked_delta > 0:
            # Primary generating >50 MB/s of log and redo queue growing
            cause = 'volume'
            action = 'throttle_writes_or_scale_secondary'
        elif log_rate < 5_000_000 and lag_sec > 5:
            # Low log generation but high lag - network is the bottleneck

```

```

        cause = 'network'
        action = 'reroute_replication_channel_or_promote_closer_replica'
    else:
        cause = 'secondary_overload'
        action = 'investigate_secondary_io_or_redo_thread_pressure'

    return LagDiagnosis(lag_sec, log_rate, cause, action)

```

For SQL Server, Always On Availability Groups spanning Azure regions use the asynchronous commit mode for cross-region replicas — a requirement, not an option. The autonomous control plane monitors `sys.dm_hadr_database_replica_states` continuously and enforces a lag SLO: if the secondary's `redo_queue_size` exceeds 512 MB for more than 90 seconds, the system triggers a workload throttle on the primary, reducing batch sizes in flight from application connection pools.

```

-- Autonomous lag monitoring query executed by the control plane every 30s
SELECT
    ag.name                                AS ag_name,
    ar.replica_server_name,
    ar.availability_mode_desc,
    drs.database_id,
    DB_NAME(drs.database_id)              AS database_name,
    drs.synchronization_state_desc,
    drs.redo_queue_size / 1024.0          AS redo_queue_mb,
    drs.log_send_queue_size / 1024.0      AS send_queue_mb,
    drs.secondary_lag_seconds,
    -- Compute breach: redo queue > 512 MB for enforcement threshold
    CASE
        WHEN drs.redo_queue_size > 524288 THEN 'BREACH_THROTTLE'
        WHEN drs.redo_queue_size > 262144 THEN 'WARNING'
        ELSE 'OK'
    END AS lag_policy_state
FROM sys.dm_hadr_availability_replica_states drs
JOIN sys.availability_replicas ar
    ON ar.replica_id = drs.replica_id
JOIN sys.availability_groups ag
    ON ag.group_id = ar.group_id
WHERE drs.is_local = 0 -- remote replicas only
ORDER BY drs.redo_queue_size DESC;

```

The autonomous bridge between OLTP and analytics environments uses SQL Server change data capture as the change layer, with the control plane consuming change events from the CDC tables to maintain derived read replicas for analytics. This pattern supports read-only cross-region analytics without requiring synchronous writes to cross over the network boundary.

Autonomous Failover Orchestration Across Cloud Boundaries

A multi-cloud failover is categorically different from a single-cloud failover. In a single-cloud promotion, the control plane promotes a replica, updates DNS or load balancer routing rules, and reconfigures surviving replicas to follow the new primary — all within the same API surface. In a multi-cloud failover, the promotion itself may need to cross API boundaries: promoting an RDS read replica to a standalone instance requires AWS RDS API calls, while simultaneously updating an Azure Traffic Manager endpoint and reconfiguring a GCP Cloud DNS weighted record.

The autonomous orchestrator handles this through a cloud-agnostic action abstraction layer. Each cloud provider exposes a driver implementing a common interface: `promote_replica()`, `update_routing_endpoint()`, `scale_connection_pool()`, `notify_health_check()`. The orchestrator composes these across providers without hardcoding provider-specific logic into the decision engine.

The failover decision itself follows a deterministic scoring algorithm. When the primary becomes unreachable (confirmed by three independent health probes across different networks to avoid split-brain), the orchestrator evaluates all `healthy` replicas against the following scoring function:

```
promotion_score = (1 / replication_lag_ms) * 0.45
                  + (1 / network_rtt_to_app_tier_ms) * 0.30
                  + (1 / data_transfer_cost_per_gb) * 0.15
                  + (disaster_zone_diversity_score) * 0.10
```

Weights are tunable per cluster profile. A cluster serving a latency-sensitive payment API weights RTT at 0.50 and cost at 0.05. A batch analytics cluster inverts those weights. The profile is stored in the topology metadata and loaded at orchestrator startup.

```

import boto3
import requests
from typing import List, Dict, Any
import logging

logger = logging.getLogger("failover_orchestrator")

PROVIDER_DRIVERS = {
    "aws": __import__("drivers.aws_driver", fromlist=["AWSDriver"]).AWSDriver,
    "azure": __import__("drivers.azure_driver",
fromlist=["AzureDriver"]).AzureDriver,
    "gcp": __import__("drivers.gcp_driver", fromlist=["GCPDriver"]).GCPDriver,
}

def score_candidate(replica: Dict[str, Any], profile: Dict[str, float]) -> float:
    lag = max(replica["replication_lag_ms"], 0.1) # avoid division by zero
    rtt = max(replica["network_rtt_to_app_ms"], 0.1)
    cost = max(replica["data_transfer_cost_per_gb"], 0.001)
    divers = replica.get("disaster_zone_diversity_score", 0.5)

    return (
        (1 / lag) * profile.get("lag_weight", 0.45) +
        (1 / rtt) * profile.get("rtt_weight", 0.30) +
        (1 / cost) * profile.get("cost_weight", 0.15) +
        divers * profile.get("diversity_weight", 0.10)
    )

def execute_cross_cloud_failover(
    failed_primary: Dict[str, Any],
    candidates: List[Dict[str, Any]],
    profile: Dict[str, float],
    routing_config: Dict[str, Any]
) -> Dict[str, Any]:
    # Score and rank candidates
    ranked = sorted(
        candidates,
        key=lambda r: score_candidate(r, profile),
        reverse=True
    )
    winner = ranked[0]
    logger.info(
        f"Selected promotion candidate: {winner['node_id']} "
        f"in {winner['cloud_provider']}/{winner['region']} "
        f"with score {score_candidate(winner, profile):.4f}"
    )

    # Invoke cloud-specific promotion
    driver_cls = PROVIDER_DRIVERS[winner["cloud_provider"]]
    driver = driver_cls(region=winner["region"])
    driver.promote_replica(replica_id=winner["node_id"])

    # Update routing layer in parallel across all clouds
    for cloud, endpoint_config in routing_config["endpoints"].items():

```

```

    rd = PROVIDER_DRIVERS[cloud](region=endpoint_config["region"])
    rd.update_routing_endpoint(
        endpoint_id=endpoint_config["id"],
        target_host=winner["hostname"],
        target_port=winner["port"]
    )
    logger.info(f"Routing updated in {cloud}: {endpoint_config['id']} → {winner['hostname']}")

# Reconfigure surviving replicas to follow new primary
survivors = [r for r in candidates if r["node_id"] != winner["node_id"]]
for replica in survivors:
    rd = PROVIDER_DRIVERS[replica["cloud_provider"]](region=replica["region"])
    rd.reconfigure_replica(
        replica_id=replica["node_id"],
        new_primary_host=winner["hostname"],
        new_primary_port=winner["port"]
    )

return {
    "status": "completed",
    "new_primary": winner["node_id"],
    "cloud": winner["cloud_provider"],
    "region": winner["region"],
    "failed_nodes": [failed_primary["node_id"]],
}

```

A non-obvious failure mode in cross-cloud failover is DNS propagation lag. Even with a TTL of 30 seconds, resolvers across cloud providers cache aggressively. The autonomous system mitigates this by using connection string rewriting at the AG listener or SQL Server connection proxy layer rather than relying solely on DNS. When the orchestrator promotes a new primary, it updates the read-only routing lists and listener registration via its configuration API within seconds of the promotion completing, bypassing DNS entirely for in-flight connections.

```

graph TD
    A[AWS Node] -->|heartbeat + lag| CP[Autonomous Control Plane]
    Z[Azure Node] -->|heartbeat + lag| CP
    G[GCP Node] -->|heartbeat + lag| CP
    CP --> TG[(Topology Graph in etcd/Consul)]
    TG --> SC[Score Candidates: lag, RTT, cost, diversity]
    SC --> RF{Residency Pre-filter}
    RF -->|No compliant candidate| ESC[Escalate to Human]
    RF -->|Compliant candidates| PROMO[Promote Highest-Scoring Replica]
    PROMO --> LEASE[Acquire Distributed Lease]
    LEASE --> RT[Update Routing: AG Listener / Traffic Manager / Cloud DNS]
    RT --> LOG[Append to Autonomous Action Log]

```


The cross-cloud failover path: topology telemetry feeds a scoring function, residency constraints act as a hard pre-filter, and only then does the orchestrator acquire a lease and promote.

Autonomous Cost Governance and Workload Placement Across Clouds

Multi-cloud autonomy without cost governance is infrastructure debt on autopilot. An autonomous system that can freely promote replicas, spin up read nodes, or reroute traffic across cloud providers must also evaluate the cost consequence of every action it takes and every state it creates.

The ADE implements continuous cost attribution at the workload level, not the infrastructure level. This distinction matters: knowing that an EC2 instance costs \$0.48/hour is infrastructure billing. Knowing that the nightly ETL job running on that instance generates 2.3 TB of cross-cloud egress at \$0.09/GB — \$207 per run — is workload attribution. The autonomous system optimizes the latter.

The autonomous workload placement engine runs a nightly optimization pass over these cost logs and proposes — or directly applies, depending on governance mode — workload migrations. A query pattern generating heavy cross-cloud egress is a candidate for read replica promotion in the destination cloud, eliminating the egress path entirely.

Governance mode determines whether the system applies changes autonomously or generates a change proposal requiring human approval. This is configured per cluster tier:

- → **Tier 0 (critical)**: Failover is autonomous, cost optimizations require approval within 4 hours or auto-apply
- → **Tier 1 (standard)**: All autonomous actions except schema changes apply immediately
- → **Tier 2 (batch/analytics)**: Fully autonomous including workload rescheduling and replica scaling

Terraform is the enforcement layer for infrastructure-level decisions. When the autonomous system determines that a GCP read replica should be promoted to serve a workload currently pulling data from AWS, it generates a Terraform plan, validates it against a policy library (using OPA — Open Policy Agent), and applies it if no policy violations exist.

```

#!/usr/bin/env bash
# Autonomous Terraform apply with OPA policy gate
# Called by control plane when workload placement change is decided

set -euo pipefail

PLAN_FILE="$1"          # Path to terraform plan JSON
POLICY_DIR="$2"         # OPA policy bundle path
TF_DIR="$3"             # Terraform working directory
APPROVAL_REQUIRED="$4" # 'yes' for Tier 0, 'no' for Tier 1+

echo "[$(date -u +%Y-%m-%dT%H:%M:%SZ)] Evaluating plan: ${PLAN_FILE}"

# Convert Terraform plan to JSON for OPA evaluation
terraform -chdir="${TF_DIR}" show -json "${PLAN_FILE}" > /tmp/tf_plan.json

# OPA policy evaluation
OPA_RESULT=$(opa eval \
  --data "${POLICY_DIR}" \
  --input /tmp/tf_plan.json \
  "data.database.placement.allow" \
  --format raw)

if [ "${OPA_RESULT}" != "true" ]; then
  echo "[BLOCKED] OPA policy denied this placement change."
  opa eval \
    --data "${POLICY_DIR}" \
    --input /tmp/tf_plan.json \
    "data.database.placement.deny_reasons" \
    --format pretty
  exit 1
fi

echo "[APPROVED] OPA policy passed."

if [ "${APPROVAL_REQUIRED}" == "yes" ]; then
  echo "[PENDING] Tier 0 cluster – queuing for human review within 4 hours."
  # Post to approval queue (Slack webhook, PagerDuty, or internal API)
  curl -s -X POST "${APPROVAL_WEBHOOK_URL}" \
    -H "Content-Type: application/json" \
    -d "{\"plan_file\": \"${PLAN_FILE}\", \"expires_in\": 14400}"
  exit 0
fi

echo "[APPLYING] Autonomous apply in progress..."
terraform -chdir="${TF_DIR}" apply -auto-approve "${PLAN_FILE}"
echo "[DONE] Infrastructure change applied autonomously."

```

Observability Unification: A Single Pane Across AWS CloudWatch, Azure Monitor, and GCP Operations

Autonomous systems cannot act on signals they cannot see. When your database fleet spans three clouds, each cloud's native observability stack captures a partial truth: CloudWatch knows about EC2-based SQL Server metrics, Azure Monitor knows about SQL MI wait stats, and GCP Cloud Monitoring knows about Cloud SQL for SQL Server storage I/O. None of them knows about the others, and none of them can correlate a cascading latency event that starts as a BGP hiccup in AWS, manifests as replication lag in Azure, and surfaces as query timeout spikes in GCP thirty seconds later.

The autonomous observability layer solves this by pulling metrics from all three provider APIs into a unified time-series store — typically Prometheus with remote write, Victoria Metrics for long-term retention, or a self-managed InfluxDB cluster — and tagging every metric with a common schema: `cloud_provider`, `region`, `cluster_id`, `node_id`, `role`. Every alert rule and every autonomous decision operates against this unified schema, never against provider-native metric names.

```
-- Unified metrics ingestion log: track staleness per provider (SQL Server control
plane)
CREATE TABLE dbo.metrics_ingestion_status (
    provider          NVARCHAR(16) NOT NULL,
    region            NVARCHAR(64) NOT NULL,
    metric_family     NVARCHAR(64) NOT NULL,
    last_ingested     DATETIMEOFFSET NOT NULL,
    -- Not PERSISTED: SYSDATETIMEOFFSET() is non-deterministic, and SQL Server
    -- only allows PERSISTED on deterministic computed column expressions.
    ingestion_lag_sec AS DATEDIFF(SECOND, last_ingested, SYSDATETIMEOFFSET()),
    is_stale          AS CAST(
        CASE WHEN DATEDIFF(SECOND, last_ingested,
SYSDATETIMEOFFSET()) > 120
            THEN 1 ELSE 0 END AS BIT),
    CONSTRAINT PK_metrics_ingestion PRIMARY KEY (provider, region, metric_family)
);

-- Stale source alert query
SELECT provider, region, metric_family, ingestion_lag_sec
FROM dbo.metrics_ingestion_status
WHERE is_stale = 1
ORDER BY ingestion_lag_sec DESC;
```

The ingestion agents for each provider run as lightweight Go services — one per cloud account — using the provider's native SDK to pull metrics on a 30-second cadence and write them to the central Prometheus remote-write endpoint. When an ingestion agent detects that it has not received a successful response from CloudWatch for more than 90 seconds, it

emits a synthetic metric: `metrics_source_reachable{provider="aws", region="us-east-1"} 0`. The autonomous control plane treats a source-unreachable condition as elevated uncertainty — it does not make failover decisions based on the last known state of an unreachable node. Instead, it pauses autonomous promotion of nodes in that cloud until independent health probes — running from the other two clouds — confirm the situation.

This is the multi-cloud version of the quorum problem: you need at least two independent observers to agree on a node's state before acting on it. The control plane enforces this by requiring confirmation from probes in at least two distinct cloud providers before classifying any node as `failed` rather than `unknown`.

```

from enum import Enum
from typing import Dict, List
import logging

logger = logging.getLogger("health_classifier")

class NodeState(str, Enum):
    HEALTHY = "healthy"
    DEGRADED = "degraded"
    UNKNOWN = "unknown"
    FAILED = "failed"

def classify_node_state(
    probe_results: Dict[str, bool], # {"aws": True, "azure": False, "gcp": False}
    current_state: NodeState,
    consecutive_failure_count: int
) -> NodeState:
    """
    Require confirmation from at least two independent cloud probers before
    declaring FAILED. This prevents split-brain promotion during network partitions
    that isolate a single prober cloud.
    """
    reporting_failed = [cloud for cloud, reachable in probe_results.items()
                        if not reachable]
    reporting_healthy = [cloud for cloud, reachable in probe_results.items()
                        if reachable]

    if len(reporting_healthy) >= 2:
        # Majority of independent probers see the node as reachable
        return NodeState.HEALTHY

    if len(reporting_failed) >= 2 and consecutive_failure_count >= 3:
        # Two or more independent clouds cannot reach it, across three check cycles
        logger.warning(
            f"Node classified FAILED: failed from {reporting_failed}, "
            f"consecutive_count={consecutive_failure_count}"
        )
        return NodeState.FAILED

    if len(reporting_failed) >= 1:
        # Only one cloud sees it failing – could be prober-side partition
        logger.info(
            f"Node classified UNKNOWN: single-cloud failure from {reporting_failed}"
        )
        return NodeState.UNKNOWN

    return NodeState.DEGRADED

```

For dashboarding, the unified observability layer exposes a single Grafana datasource pointing at the central time-series store. The dashboard hierarchy mirrors the topology graph: fleet view → cluster view → node view → query view. At the fleet level, the primary KPIs

are cross-cloud replication health, inter-cloud RTT percentiles, and cost-per-query by cloud pair. Autonomous system actions — failovers, workload migrations, replica scaling events — are annotated directly onto the timeline panels using Grafana's annotation API, called by the orchestrator immediately after each action completes. This makes the history of autonomous decisions visible at exactly the time coordinates where they occurred, allowing retrospective analysis without log correlation.

Conflict Resolution in Distributed Autonomous Systems

When autonomous agents in multiple clouds can each initiate actions independently, the risk is not inaction — it is concurrent conflicting action. Two agents simultaneously deciding that the same replica should be promoted, one from the AWS control plane instance and one from the GCP instance, can result in a split-brain topology: two nodes both believing themselves to be primary, both accepting writes, with no replication relationship between them.

The solution is a distributed lease system. Before any autonomous agent initiates a topology-changing action, it must acquire a lease on the target cluster's control record. Leases are stored in a strongly consistent, distributed key-value store — etcd is the standard choice — with a TTL of 30 seconds. If the agent fails or becomes partitioned mid-action, the lease expires and another agent can safely take over.

```

import etcd3
import uuid
import time
from contextlib import contextmanager

etcd_client = etcd3.client(host="etcd-control-plane.internal", port=2379)

@contextmanager
def acquire_cluster_lease(cluster_id: str, agent_id: str, ttl_sec: int = 30):
    """
    Distributed lease for cluster-level topology operations.
    Raises RuntimeError if lease cannot be acquired (another agent holds it).
    """
    lease_key = f"/ade/cluster_lease/{cluster_id}"
    lease_val = agent_id.encode()

    # Attempt atomic put-if-not-exists
    lease = etcd_client.lease(ttl_sec)
    success, _ = etcd_client.transaction(
        compare=[etcd_client.transactions.version(lease_key) == 0],
        success=[etcd_client.transactions.put(lease_key, lease_val, lease)],
        failure=[]
    )

    if not success:
        existing, _ = etcd_client.get(lease_key)
        holder = existing.decode() if existing else "unknown"
        raise RuntimeError(
            f"Cannot acquire lease for cluster {cluster_id}: held by agent {holder}"
        )

    logger.info(f"Lease acquired: cluster={cluster_id} agent={agent_id} ttl={ttl_sec}s")
    try:
        yield lease
    finally:
        etcd_client.revoke_lease(lease.id)
        logger.info(f"Lease released: cluster={cluster_id} agent={agent_id}")

```

Beyond lease management, the autonomous system maintains a global action log — an append-only record of every decision made, the state at the time of the decision, the expected outcome, and the observed outcome after a verification window. This log serves three purposes: it provides an audit trail for compliance review, it feeds the reinforcement-learning layer that adjusts decision weights over time, and it enables rollback analysis when an autonomous action produces an unexpected state.

```

CREATE TABLE dbo.autonomous_action_log (
    action_id            UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
    initiated_at         DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    agent_id             NVARCHAR(128) NOT NULL,
    cloud_provider       NVARCHAR(16)  NOT NULL,
    cluster_id           UNIQUEIDENTIFIER NOT NULL,
    action_type          NVARCHAR(64)  NOT NULL,
    target_node_id       UNIQUEIDENTIFIER NULL,
    decision_inputs      NVARCHAR(MAX) NULL, -- JSON
    expected_outcome      NVARCHAR(MAX) NULL, -- JSON
    observed_outcome      NVARCHAR(MAX) NULL, -- JSON, populated post-verification
    outcome_verified_at  DATETIMEOFFSET NULL,
    status               NVARCHAR(20) NOT NULL DEFAULT 'in_progress'
                        CHECK (status IN
('in_progress','completed','failed','rolled_back')),
    rollback_action_id   UNIQUEIDENTIFIER NULL
                        REFERENCES dbo.autonomous_action_log(action_id)
);

-- Filtered index for learning pipeline queries
CREATE INDEX IX_action_log_learning
    ON dbo.autonomous_action_log (action_type, initiated_at)
    INCLUDE (cluster_id, decision_inputs, observed_outcome)
    WHERE status = 'completed' AND observed_outcome IS NOT NULL;

```

The verification window — typically five minutes after an action completes — is when the control plane re-samples the affected cluster's health metrics and compares them against the expected outcome recorded at decision time. If the observed state diverges from expected by more than a configurable threshold (for example, post-promotion replication lag remains above 1,000 ms when the expected outcome was below 200 ms), the system escalates: either triggering a rollback procedure or, if rollback is not safe, paging an on-call engineer with the full decision trace attached.

Regulatory Compliance and Data Residency in Autonomous Cross-Cloud Operations

Autonomous systems that can freely move data and reroute workloads across cloud regions run directly into data residency regulations: GDPR constrains EU personal data from leaving the EU economic area, certain financial regulators require transaction logs to remain within national borders, and healthcare data in some jurisdictions cannot traverse public cloud infrastructure without explicit sovereign cloud designation.

The autonomous system must encode these constraints as hard guardrails — not soft preferences — that override any cost or latency optimization. A constraint violation is not a sub-optimal outcome; it is a compliance failure with legal consequences. This means residency constraints must be evaluated before the scoring function runs, not as an additional term within it.

```
CREATE TABLE dbo.residency_policies (  
    policy_id            UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),  
    cluster_id           UNIQUEIDENTIFIER NOT NULL,  
    data_classification NVARCHAR(64) NOT NULL,  
    allowed_regions      NVARCHAR(MAX) NOT NULL, -- JSON array  
    allowed_providers    NVARCHAR(MAX) NOT NULL, -- JSON array  
    enforce_at_write     BIT NOT NULL DEFAULT 1,  
    enforce_at_read      BIT NOT NULL DEFAULT 1,  
    regulation_ref       NVARCHAR(128) NULL,  
    created_at           DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET()  
);  
  
-- Inline compliance check in failover candidate query  
SELECT  
    n.node_id,  
    n.cloud_provider,  
    n.region,  
    n.replication_lag_ms,  
    CASE  
        WHEN JSON_VALUE(rp.allowed_providers, '$[0]') IS NULL THEN 1  
        -- Simplified: full evaluation uses OPENJSON for array membership  
        WHEN n.cloud_provider IN (  
            SELECT value FROM OPENJSON(rp.allowed_providers)  
        )  
        AND n.region IN (  
            SELECT value FROM OPENJSON(rp.allowed_regions)  
        ) THEN 1  
        ELSE 0  
    END AS is_residency_compliant  
FROM dbo.db_topology n  
LEFT JOIN dbo.residency_policies rp  
    ON rp.cluster_id = n.cluster_id  
WHERE n.health_state = 'healthy'  
    AND n.role = 'replica';
```

The autonomous orchestrator applies residency filtering as a hard pre-filter: any candidate node that fails `is_node_residency_compliant()` is removed from the candidate set before scoring begins. If the pre-filter eliminates all candidates — that is, no compliant replica exists — the orchestrator does not fall back to a non-compliant node. It instead escalates to a human operator with a clear incident description: the cluster cannot autonomously recover

without violating data residency constraints, and human intervention is required to either provision a compliant replica or authorize a temporary exception through a documented override workflow.

This design reflects a principle that underlies all autonomous database engineering in regulated environments: the system's autonomy stops at the boundary of legal constraint. Autonomy is an operational property, not a compliance bypass.

Key Takeaways

- → **Topology modeling as a control graph is the foundation of multi-cloud autonomy.** Without a continuously updated, strongly consistent graph of nodes, edges, replication lag, and cost properties, every autonomous decision is based on stale or incomplete state. The graph is not a dashboard artifact — it is the primary data structure the control plane operates against.
- → **Cross-cloud replication lag has two distinct causes that require different autonomous responses.** Volume-induced lag demands write throttling or subscriber scaling; network-induced lag demands replication channel rerouting or replica promotion to a lower-latency zone. Conflating the two leads to interventions that worsen the situation rather than resolving it.
- → **Multi-cloud failover requires a cloud-agnostic action abstraction layer and distributed lease management.** Provider-specific promotion APIs must be hidden behind a uniform driver interface, and every topology-changing action must be gated on a distributed lease to prevent concurrent conflicting promotions from independent agents running in different clouds.
- → **Unified observability across CloudWatch, Azure Monitor, and GCP Operations requires a common tagging schema and source-reachability tracking.** Metrics ingestion staleness is itself a signal: when a cloud's metrics source becomes unreachable, the autonomous system must treat affected nodes as `unknown` rather than acting on last-known state, and must require independent confirmation from probes in at least two cloud providers before classifying any node as `failed`.
- → **Data residency constraints are hard guardrails that override all cost and latency optimization.** The autonomous orchestrator must evaluate regulatory policy compliance as a pre-filter that eliminates non-compliant candidates before the scoring function runs, and must escalate to human operators — rather than falling back to a non-compliant replica — when no compliant failover target exists.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 32: Azure SQL Autonomous Services: Serverless, Managed Instance, and Control-Plane Operations

Every autonomous system you build on SQL Server operates inside a platform that is already making autonomous decisions. Azure SQL Database pauses compute when it is idle, scales vCores up and down under load, applies automatic plan corrections when regressions appear, and creates or drops indexes on its own authority. Azure SQL Managed Instance moves your database between service tiers, patches the underlying host without your involvement, and fails over to a secondary when hardware degrades. If your autonomous database engine does not understand what the platform is doing — and, more importantly, does not coordinate with it — the two layers will fight. This chapter maps the autonomous services Azure SQL provides, shows how to observe and configure each one, and builds the coordination layer that lets your ADE and the platform's control plane share responsibility without conflicting.

The distinction matters because the previous chapters built autonomy from scratch: agents, policy engines, DMV-driven sensors. That model fits SQL Server running on virtual machines or on-premises, where you own every decision. On Azure SQL, roughly half of those decisions are already handled by the control plane. The DBA's job shifts from building every autonomous behavior to curating the platform's autonomous behaviors — enabling what helps, bounding what risks, and filling the gaps the platform deliberately leaves open.

The Platform Autonomy Layer: What Azure SQL Decides for You

Before coordinating with the platform, you need an accurate inventory of what it does autonomously. The Azure SQL control plane operates at a different layer than your database engine agents: it manages the compute, storage, and availability substrate. Your agents manage the workload inside the database. The boundary between the two is the service tier contract — and every conflict between your ADE and the platform can be traced to an action that crossed that boundary in the wrong direction.

The platform's autonomous decisions fall into five categories:

1. **Compute lifecycle:** serverless auto-pause and auto-resume, automatic vCore scaling in the serverless and Hyperscale tiers, and elastic pool resource balancing across member databases.

2. **Performance tuning:** automatic plan correction (plan forcing through the Query Store interface), automatic index management (create and drop), and adaptive query processing features that are enabled by default.

3. **Availability:** automatic failover within the availability zone or region, failover groups with automatic geo-failover policies, and zone-redundant configurations that the control plane maintains without operator input.

4. **Maintenance:** patching of the underlying host and SQL Server build, driven by maintenance window policies you configure but do not execute.

5. **Backup and retention:** automated full, differential, and log backups on a platform schedule, with long-term retention policies that run as background control-plane operations.

Your ADE owns everything inside the database that the platform explicitly does not: workload classification, custom index strategies beyond the platform's conservative heuristics, query governance, capacity forecasting tied to business events, and any decision that requires knowledge of your application's intent. The coordination rule is simple to state and difficult to enforce: **never let both layers tune the same knob**. If automatic index management is enabled, your index-creation agent must not create indexes on the same tables. If the platform owns plan forcing, your plan-forcing agent must defer. The rest of this chapter builds the mechanisms that enforce that rule.

Serverless: Auto-Pause, Auto-Scaling, and the DBA's New Job

The serverless compute tier is the most visibly autonomous surface in Azure SQL. A serverless database pauses itself after a configurable period of inactivity — typically one hour — and resumes automatically when a new connection arrives. While running, it scales compute between a configured minimum and maximum vCore count based on demand. The DBA configures the envelope; the platform makes every decision inside it.

For the ADE, serverless introduces a new operational reality: the database can be absent. Any agent that polls DMVs on a schedule must handle the paused state gracefully — a connection attempt to a paused database triggers auto-resume, which takes 30–60 seconds, and a naive health check will report the database as down during that window. The correct pattern is to query the control plane's view of database state before attempting a data-plane connection:

```
-- Check whether a serverless database is paused before connecting
-- Runs against the master database of the logical server
SELECT
    d.name                                AS database_name,
    d.state_desc,
    DATABASEPROPERTYEX(d.name, 'Status') AS detailed_status
FROM sys.databases d
WHERE d.name = N'OrdersDB';
```

The auto-pause delay itself is an autonomous policy decision the DBA configures. Setting it too short means frequent pause/resume cycles — each resume incurs latency and the first queries after resume face a cold buffer pool. Setting it too long wastes the cost advantage that justifies serverless. The ADE can manage this setting as a control-plane action, adjusting the pause delay based on observed connection patterns:

```

# Adjust serverless auto-pause delay from observed idle patterns
import pyodbc
import subprocess

def recommend_pause_delay(conn_str: str, database: str) -> int:
    """Analyze connection gaps; return recommended auto-pause delay in minutes."""
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT DATEDIFF(MINUTE, MAX(login_time), SYSUTCDATETIME())
            FROM sys.dm_exec_sessions
            WHERE database_id = DB_ID(?)
            AND is_user_process = 1
            """, (database,))
        row = cur.fetchone()
        longest_recent_idle_min = row[0] if row and row[0] else 0

    # If the longest recent idle gap is well under the current delay,
    # the delay can be shortened without causing resume storms.
    if longest_recent_idle_min < 30:
        return 60    # keep default: workload is bursty, avoid resume latency
    elif longest_recent_idle_min > 240:
        return 120   # long idle stretches: pause sooner, save cost
    return 60

def apply_pause_delay(server: str, resource_group: str, database: str, delay_min:
int):
    subprocess.run([
        "az", "sql", "db", "update",
        "--resource-group", resource_group,
        "--server", server,
        "--name", database,
        "--auto-pause-delay", str(delay_min),
    ], check=True)

```

The deeper lesson of serverless is that the DBA's autonomous agents must become **state-aware of the platform**. Every agent in your ADE should check database state before acting, back off during resume, and distinguish "database is paused" from "database is down." That check is a few lines of code, but without it, the autonomous system will page you at 3 AM because a dev database did exactly what you configured it to do.

Elastic Pools: Autonomous Resource Sharing Across Databases

Elastic pools are the platform's answer to the multi-tenant resource problem: instead of provisioning peak capacity for every database, you provision a shared pool of eDTUs or vCores and let the platform distribute them. The pool's resource governor is itself an autonomous system — it throttles individual databases that exceed their per-database max, borrows idle capacity for databases under load, and enforces the pool's aggregate limits.

The coordination question for the ADE is placement: which databases belong in which pool, and when should a database graduate from a pool to a single database (or vice versa)? The platform will never make that decision for you — it manages resources within the pool, not membership of the pool. Your ADE fills the gap by monitoring per-database resource consumption against pool limits:

```
-- Elastic pool pressure: which databases are consistently consuming
-- a disproportionate share of the pool?
SELECT
    DB_NAME(r.database_id)                AS database_name,
    AVG(r.avg_cpu_percent)                 AS avg_cpu_pct,
    MAX(r.avg_cpu_percent)                 AS max_cpu_pct,
    AVG(r.avg_data_io_percent)             AS avg_data_io_pct,
    AVG(r.avg_log_write_percent)           AS avg_log_write_pct,
    COUNT(*)                              AS samples
FROM sys.elastic_pool_resource_stats r
WHERE r.end_time >= DATEADD(DAY, -7, GETUTCDATE())
GROUP BY r.database_id
HAVING AVG(r.avg_cpu_percent) > 40
ORDER BY avg_cpu_pct DESC;
```

The policy that consumes this data is straightforward: a database that averages above 40% of the pool's CPU for a week is a candidate for graduation to a single database, because it is both paying for capacity it could use alone and starving its pool neighbors of burst headroom. Conversely, a single database averaging under 10% CPU is a candidate for pool membership. These are control-plane actions — executed through the Azure CLI or ARM templates, not through T-SQL — which is exactly why they belong in your ADE's control-plane action layer rather than in any database-internal agent.

Automatic Tuning: The Platform's Query Governor

Automatic tuning is where platform autonomy most directly overlaps with the agents built in earlier chapters — and where the "never tune the same knob twice" rule is most important. Azure SQL offers three automatic tuning options: automatic plan correction, automatic index management (create and drop), and the legacy automatic plan-choice correction. Each can be enabled independently per database.


```
-- Inspect and configure automatic tuning per database
SELECT
    name,
    desired_state_desc    AS plan_correction,
    actual_state_desc     AS actual_state,
    reason_desc
FROM sys.database_automatic_tuning_options;

-- Enable plan correction only; leave index management to the ADE
ALTER DATABASE [OrdersDB]
    SET AUTOMATIC_TUNING (
        FORCE_LAST_GOOD_PLAN = ON,
        CREATE_INDEX = OFF,
        DROP_INDEX = OFF
    );
```

The recommended division of labor: let the platform own **automatic plan correction** and let your ADE own **index management**, or vice versa — but document the choice and enforce it. The platform's plan correction is conservative, well-tested, and operates through the same Query Store plan-forcing mechanism your agents would use. There is no value in duplicating it. The platform's index management is similarly conservative — it creates indexes only for clear-cut missing-index cases and drops only indexes it created itself after a dwell period. If your ADE has a richer index strategy (workload-aware consolidation, columnstore evaluation, partitioning-aware placement), disable the platform's and let yours own the knob.

Whatever the division, your ADE must observe what the platform did. The tuning recommendations DMV is the observation surface:

```
-- What has automatic tuning done lately? Feed this into the ADE audit log.
SELECT
    r.reason,
    r.score,
    JSON_VALUE(CAST(r.details AS NVARCHAR(MAX)), '$.implementationDetails.script')
        AS implementation_script,
    r.state                                AS recommendation_state,
    r.created                             AS recommended_at,
    r.last_refresh                         AS last_evaluated
FROM sys.dm_db_tuning_recommendations r
ORDER BY r.last_refresh DESC;
```

Ingesting the platform's tuning history into your ADE's action log closes the loop: when your index agent proposes a new index, it can check whether the platform already created, evaluated, and dropped a similar index — and learn from that outcome instead of repeating the experiment. Platform actions are not noise in your telemetry. They are the most authoritative signal available about what has already been tried.

Managed Instance: Service Tiers and Control-Plane Operations

Azure SQL Managed Instance is the platform surface closest to the self-managed SQL Server of the earlier chapters — you get SQL Agent, cross-database queries, CLR, and Service Broker — but the control plane still owns the host. Patching, failover, storage management, and high-availability topology are platform decisions. The DBA's autonomous agents run inside the instance exactly as they would on a VM, with two critical differences: some DMVs and configuration options are restricted, and the instance can be stopped and started as a control-plane operation.

The stop/start capability is the Managed Instance equivalent of serverless auto-pause, and it creates the same state-awareness requirement for your agents. A stopped instance refuses connections with a control-plane error, not a database error — your ADE's health checks must distinguish the two, because "stopped by policy" is not an incident.

The more strategically interesting autonomy in Managed Instance is the **service tier update**: scaling vCores, moving between General Purpose and Business Critical, or enabling zone redundancy. These are online operations in most cases, but they are not free — a tier change restarts the SQL Server process, dropping the buffer pool and plan cache. Your ADE should treat a tier change the way it treats a failover: schedule it in a maintenance window, warm the cache afterward, and verify query performance against baseline before declaring success:

```

# Coordinated Managed Instance tier change with post-change verification
import subprocess
import pyodbc
import time

def scale_managed_instance(server: str, resource_group: str,
                           vcore: int, tier: str, conn_str: str):
    # 1. Capture pre-change baseline
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT AVG(avg_cpu_percent) FROM sys.dm_db_resource_stats
            WHERE end_time >= DATEADD(HOUR, -1, GETUTCDATE())
        """)
        baseline_cpu = cur.fetchone()[0]

    # 2. Execute the control-plane scaling operation
    subprocess.run([
        "az", "sql", "mi", "update",
        "--resource-group", resource_group,
        "--name", server,
        "--vcore", str(vcore),
        "--tier", tier,
    ], check=True)

    # 3. Wait for the instance to come back online
    for _ in range(60):
        try:
            with pyodbc.connect(conn_str, timeout=10) as conn:
                conn.cursor().execute("SELECT 1")
            break
        except Exception:
            time.sleep(30)

    # 4. Warm the buffer pool and plan cache with the top workload queries
    with pyodbc.connect(conn_str) as conn:
        cur = conn.cursor()
        cur.execute("""
            SELECT TOP 20 query_sql_text
            FROM sys.query_store_query
            ORDER BY last_execution_time DESC
        """)
        for (sql_text,) in cur.fetchall():
            try:
                cur.execute("SET ROWCOUNT 1")
                cur.execute(sql_text)
                cur.execute("SET ROWCOUNT 0")
            except Exception:
                pass # best-effort warming; individual failures are fine

```

Note what this script does not do: it does not decide *whether* to scale. That decision belongs to the capacity-planning policy from Chapter 14, running in your ADE. The script is the execution arm — the control-plane action that the policy invokes. Keeping the decision layer (your ADE) separate from the execution layer (platform APIs) is what makes the system auditable: every scaling event traces back to a policy evaluation with a recorded rationale.

Failover Groups: Autonomous Geo-Failover Policy

Failover groups are the clearest example of platform autonomy done well. You define a primary and a secondary in different regions, configure a failover policy — automatic or manual — and a grace period. If the primary becomes unreachable, the control plane evaluates the outage against the grace period and fails over automatically. Read-write and read-only listeners follow the role change without application reconfiguration.

The grace period is the autonomy knob, and it deserves the same policy treatment as any threshold in your ADE. The default of one hour is conservative: it prevents flapping during transient outages but accepts up to an hour of downtime before failing over. For a payment database, that is far too long; for a dev database, it is appropriately cautious. Your ADE can manage grace periods as policy, tightening them for databases whose RTO demands it:

```
# Set failover group policy from RTO requirements
import subprocess

def configure_failover_group(server: str, resource_group: str,
                             group_name: str, rto_minutes: int):
    # Grace period must cover transient blips but respect the RTO.
    # Policy: grace = RTO minus expected failover time, capped at 60 min.
    grace_minutes = max(60, min(3600, (rto_minutes - 5) * 60))
    grace_hours = max(1, grace_minutes // 60)
    mode = "Automatic" if rto_minutes <= 30 else "Manual"

    subprocess.run([
        "az", "sql", "failover-group", "update",
        "--resource-group", resource_group,
        "--server", server,
        "--name", group_name,
        "--failover-policy", mode,
        "--grace-period", str(grace_hours),
    ], check=True)
    print(f"{group_name}: policy={mode}, grace-period={grace_hours}h for RTO {rto_minutes}min")
```

The critical coordination point: your ADE's own failover agents must be **disabled for any database in an automatic failover group**. Two failover authorities — your agent and the platform — will eventually disagree about whether a failover is warranted, and the result is a

failover war: the platform fails over to the secondary, your agent detects the old primary as "unhealthy primary" and tries to fail back, and the database ping-pongs between regions. The ADE's topology inventory must mark failover-group members as platform-managed for availability, and the failover policy engine must refuse to evaluate them. This is the single most important coordination rule in the chapter, and it is worth enforcing in code rather than in documentation.

```
sequenceDiagram
    participant App as Application
    participant Listener as Failover Group Listener
    participant Primary as Primary Region
    participant Platform as Control Plane
    participant Secondary as Secondary Region

    App->>Listener: Read-write traffic
    Listener->>Primary: Route to current primary
    Primary-->>Platform: Health signal lost
    Platform->>Platform: Grace period evaluation
    Platform->>Secondary: Initiate geo-failover
    Secondary-->>Platform: Role change complete
    Platform->>Listener: Update listener DNS
    Listener->>App: Traffic now routes to new primary
    Note over Platform: ADE failover agents stay idle:
    availability is platform-managed
```

The platform-owned failover path: the control plane evaluates the outage against the grace period and fails over without ADE involvement. Your agents observe and log; they do not act.

Maintenance Windows and Update Policies

Platform patching is autonomous by design — you cannot log into the host and run Windows Update — but the *schedule* is yours to configure. Maintenance windows let you declare when the platform is allowed to patch: a day of the week and a time range, aligned with your own maintenance windows from Chapter 23. The coordination value is obvious: if your ADE schedules index rebuilds for Saturday 02:00 and the platform patches at Saturday 02:00, the two will contend for I/O and the rebuild will run against a cold cache after the host restarts.

```
# Align platform maintenance windows with the ADE's own maintenance schedule
import subprocess

WEEKDAYS = ["", "Monday", "Tuesday", "Wednesday",
            "Thursday", "Friday", "Saturday", "Sunday"]

def set_maintenance_window(server: str, resource_group: str,
                           db_name: str, day: int, start_hour: int):
    # day: 1=Monday .. 7=Sunday; start_hour: 0-23
    # Keep the platform window OUTSIDE the ADE's Saturday 02:00 rebuild window
    subprocess.run([
        "az", "sql", "db", "update",
        "--resource-group", resource_group,
        "--server", server,
        "--name", db_name,
        "--maint-config-id",
        "SQL_{}_{}".format(WEEKDAYS[day], start_hour),
    ], check=True)
```

The update policy for Managed Instance deserves separate attention: instances can be pinned to a specific SQL Server build or follow the platform's rolling update train. For production instances under ADE management, the policy should be explicit and version-controlled — the ADE's change-management integration from Chapter 25 should record every platform build change the way it records every schema change, because a build change can alter query plans just as surely as a statistics update can.

Azure Arc: Extending Control-Plane Autonomy to Your Own Infrastructure

Azure Arc-enabled SQL Server brings a subset of the platform's control-plane autonomy to SQL Server running on your own VMs or on-premises: centralized inventory, automated patch assessment, and policy-driven configuration through Azure Policy. For the ADE, Arc is the bridge that lets one autonomous system manage a hybrid fleet with one set of policies.

The practical value is inventory and compliance rather than deep autonomy: Arc tells your ADE which instances exist, what versions they run, and whether they comply with the policies you declare — password rotation, TLS configuration, audit enablement. Your ADE's agents then enforce the database-internal state, while Arc enforces the host-level state. The division mirrors the cloud division: platform owns the substrate, ADE owns the workload.

```

-- Arc-enabled instances report into the same inventory the ADE already uses.
-- Reconcile Arc inventory against the ADE's known fleet weekly:
-- any instance in Arc but missing from the ADE is unmanaged autonomy debt.
SELECT
    arc.instance_name,
    arc.version,
    arc.last_seen_utc,
    ade.last_checkin_utc,
    CASE
        WHEN ade.instance_name IS NULL THEN 'UNMANAGED_BY_ADE'
        WHEN arc.last_seen_utc < DATEADD(DAY, -1, SYSUTCDATETIME()) THEN
'STALE_ARC_DATA'
        ELSE 'COVERED'
    END AS coverage_state
FROM arc_inventory arc
LEFT JOIN ade_fleet_inventory ade
    ON ade.instance_name = arc.instance_name;

```

The Shared-Responsibility Contract: Encoding It in Policy

Every coordination rule in this chapter — who owns plan forcing, who owns failover, who owns index management — is a candidate for encoding in the ADE's policy engine rather than living in runbooks. The shared-responsibility contract is a policy document that the ADE loads at startup and enforces on every action:

```
# shared_responsibility.yaml – who owns what, enforced by the ADE policy engine
platform_managed:
  availability:
    - "databases in automatic failover groups"
    - "zone-redundant Business Critical databases"
  performance:
    - "automatic plan correction (FORCE_LAST_GOOD_PLAN = ON)"
  compute:
    - "serverless auto-pause and auto-scale"
    - "elastic pool resource distribution"
  maintenance:
    - "host patching within configured maintenance windows"

ade_managed:
  availability:
    - "AG failover for self-managed SQL Server (VMs, on-premises)"
    - "failover-group grace period and policy configuration"
  performance:
    - "index lifecycle management (platform CREATE_INDEX = OFF)"
    - "statistics maintenance and custom plan forcing"
  compute:
    - "elastic pool membership and graduation decisions"
    - "serverless auto-pause delay tuning"
  maintenance:
    - "maintenance window alignment and build-change tracking"

conflict_rules:
  - rule: "If platform owns plan correction, ADE plan-forcing agent is disabled for
that database."
  - rule: "If database is in an automatic failover group, ADE failover evaluation is
refused."
  - rule: "Platform tuning recommendations are ingested into the ADE action log
before any ADE index action on the same table."
```

The enforcement mechanism is a pre-action check in the ADE's action dispatcher — the same pattern as the deployment-window guard from Chapter 28. Before any autonomous action executes, the dispatcher evaluates it against the shared-responsibility contract. An index-creation action on a database where the platform owns index management is refused with a structured log entry, not executed and hoped for the best.

This contract is also the document you show an auditor. When the auditor asks "who decided to drop that index," the answer is either a platform tuning record with a timestamp and a score, or an ADE action log entry with a policy evaluation and a human approval — never "we don't know, both layers can do that." Shared responsibility without a written contract is just shared blame.

Key Takeaways

- → **Never let both layers tune the same knob.** The platform's autonomous services and your ADE must have a written, policy-enforced division of labor. Every overlap — plan forcing, index management, failover, compute scaling — is a future incident. Encode the division in the shared-responsibility contract and enforce it in the action dispatcher.
- → **Your agents must be platform-state-aware.** Serverless pause, Managed Instance stop/start, and failover-group transitions are not failures. Agents that treat them as failures will page you for correct platform behavior. Check database state through the control plane's view before acting through the data plane.
- → **Ingest platform actions into your action log.** Automatic tuning recommendations, tier changes, and failover events are the most authoritative signal available about what has already been tried. An ADE that ignores the platform's history will repeat its experiments and contradict its decisions.
- → **Availability in a failover group is platform-managed — keep your failover agents idle.** Two failover authorities will eventually disagree, and the result is a failover war between regions. Mark platform-managed databases in the topology inventory and refuse failover evaluation for them in code, not just in documentation.
- → **Align maintenance windows across both layers.** Platform patching and ADE maintenance contend for the same I/O and the same cold caches. Configure the platform's maintenance windows to avoid your ADE's maintenance windows, and track platform build changes with the same rigor as schema changes.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 33: Autonomous SQL Server Systems

SQL Server has long been positioned as an enterprise database that "just works" — but in autonomous systems engineering, "just works" is the floor, not the ceiling. This chapter builds the architecture layer that transforms SQL Server from a managed platform into a self-observing, self-healing, self-optimizing engine. The patterns here treat SQL Server not as a monolithic appliance but as a controllable system with programmable decision surfaces — query store metadata, Dynamic Management Views, Always On Availability Groups, and Azure SQL's autonomy APIs are all levers that can be pulled by code rather than hands. With a deeply integrated observability substrate and a mature HA framework, SQL Server gives you everything an autonomous system needs — and with the right automation layer on top, it reaches full self-management depth.

Treating SQL Server DMVs as a Real-Time Autonomy Sensor Network

Every autonomous system needs a sensor layer — a continuous stream of state signals that drives downstream decisions. SQL Server's Dynamic Management Views (DMVs) are the richest built-in sensor network of any commercial relational database. The challenge at the ADE level is not learning DMVs; it is building a collection pipeline that turns DMV snapshots into time-series signals with enough resolution and history to drive autonomous decision logic.

The core DMVs for autonomous operation form three functional clusters:

Execution health: `sys.dm_exec_requests`, `sys.dm_exec_sessions`, `sys.dm_os_wait_stats`, `sys.dm_exec_query_stats`

Resource pressure: `sys.dm_os_ring_buffers`, `sys.dm_os_memory_clerks`, `sys.dm_io_virtual_file_stats`, `sys.dm_os_performance_counters`

Plan and optimizer state: `sys.dm_exec_cached_plans`, `sys.dm_exec_query_plan`, `sys.query_store_runtime_stats`, `sys.query_store_plan`

The autonomous sensor layer collects these at differential intervals — wait stats at 15-second deltas, query store at 60-second intervals, memory clerks at 30-second intervals. Collecting point-in-time snapshots without computing deltas is one of the most common mistakes in SQL Server observability. `sys.dm_os_wait_stats` is cumulative since the last service restart; what the decision engine needs is the delta over the collection window.

```
-- Autonomous wait stats delta collector
-- Run this via a SQL Agent job or external Python scheduler at 15-second intervals
```

```
IF OBJECT_ID('tempdb..#wait_baseline', 'U') IS NULL
```

```
BEGIN
```

```
    SELECT
```

```
        wait_type,
        waiting_tasks_count,
        wait_time_ms,
        signal_wait_time_ms,
        GETUTCDATE() AS snapshot_time
```

```
    INTO tempdb..#wait_baseline
```

```
    FROM sys.dm_os_wait_stats
```

```
    WHERE wait_type NOT IN (
```

```
        'SLEEP_TASK', 'BROKER_TO_FLUSH', 'BROKER_EVENTHANDLER',
        'CHECKPOINT_QUEUE', 'DBMIRROR_EVENTS_QUEUE', 'SQLTRACE_BUFFER_FLUSH',
        'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE', 'FT_IFTS_SCHEDULER_IDLE_WAIT',
```

```
        'HADR_FILESTREAM_IOMGR_IOCOMPLETION', 'HADR_WORK_QUEUE', 'ONDEMAND_TASK_QUEUE',
        'REQUEST_FOR_DEADLOCK_MONITOR', 'RESOURCE_QUEUE', 'SERVER_IDLE_CHECK',
        'SLEEP_DBSTARTUP', 'SLEEP_DCOMSTARTUP', 'SLEEP_MASTERDBREADY',
        'SLEEP_MASTERMDREADY', 'SLEEP_MASTERUPGRADED', 'SLEEP_MSDBSTARTUP',
        'SLEEP_SYSTEMTASK', 'SLEEP_TEMPDBSTARTUP', 'SNI_HTTP_ACCEPT',
        'SP_SERVER_DIAGNOSTICS_SLEEP', 'WAITFOR', 'XE_DISPATCHER_WAIT',
        'XE_TIMER_EVENT', 'BROKER_RECEIVE_WAITFOR', 'WAITFOR_TASKSHUTDOWN'
```

```
    );
```

```
    RETURN;
```

```
END
```

```
-- Delta computation against prior baseline
```

```
SELECT
```

```
    w.wait_type,
    w.waiting_tasks_count - b.waiting_tasks_count    AS delta_tasks,
    w.wait_time_ms        - b.wait_time_ms           AS delta_wait_ms,
    w.signal_wait_time_ms - b.signal_wait_time_ms    AS delta_signal_ms,
    CASE
        WHEN (w.wait_time_ms - b.wait_time_ms) = 0 THEN 0
        ELSE CAST((w.signal_wait_time_ms - b.signal_wait_time_ms) AS FLOAT)
            / (w.wait_time_ms - b.wait_time_ms) * 100
    END AS cpu_pressure_pct,
    GETUTCDATE() AS collected_at
```

```
FROM sys.dm_os_wait_stats w
```

```
JOIN tempdb..#wait_baseline b ON w.wait_type = b.wait_type
```

```
WHERE w.wait_time_ms - b.wait_time_ms > 0
```

```
ORDER BY delta_wait_ms DESC;
```

```
-- Refresh baseline
```

```
TRUNCATE TABLE tempdb..#wait_baseline;
```

```
INSERT INTO tempdb..#wait_baseline
```

```
SELECT wait_type, waiting_tasks_count, wait_time_ms, signal_wait_time_ms,
GETUTCDATE()
```

```
FROM sys.dm_os_wait_stats  
WHERE wait_type NOT IN (/* same exclusion list */);
```

Once delta-aware DMV collection is running, the signals feed into a time-series store — typically Prometheus with a custom SQL Server exporter or a purpose-built Python collector writing to InfluxDB or TimescaleDB. The decision engine then operates against a sliding window of 5–15 minutes of signal history rather than instantaneous reads. A single spike in `PAGEIOLATCH_EX` waits means nothing; a sustained 10-minute elevation above a learned baseline triggers an I/O investigation workflow.

The autonomous baseline itself deserves careful engineering. Rather than static thresholds, the system learns workload-relative bounds using a rolling percentile window:

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import pyodbc

class SQLServerWaitBaseline:
    """
    Maintains rolling percentile baselines for SQL Server wait categories.
    Baselines are hour-of-week aware to account for batch vs. OLTP patterns.
    """

    WAIT_CATEGORIES = {
        'CPU':      ['SOS_SCHEDULER_YIELD', 'CXPACKET', 'THREADPOOL'],
        'IO':       ['PAGEIOLATCH_EX', 'PAGEIOLATCH_SH', 'WRITELOG',
'IO_COMPLETION'],
        'LOCK':     ['LCK_M_X', 'LCK_M_S', 'LCK_M_U', 'LCK_M_IX'],
        'MEMORY':   ['RESOURCE_SEMAPHORE', 'RESOURCE_SEMAPHORE_QUERY_COMPILE',
'CMEMTHREAD'],
        'NETWORK': ['ASYNC_NETWORK_IO', 'NET_WAITFOR_PACKET'],
    }

    def __init__(self, conn_str: str, lookback_days: int = 14):
        self.conn_str = conn_str
        self.lookback_days = lookback_days

    def fetch_historical_waits(self) -> pd.DataFrame:
        """Pull wait delta history from the collector's persistent store."""
        query = """
            SELECT
                category,
                hour_of_week,
                delta_wait_ms,
                collected_at
            FROM dba.wait_stat_history
            WHERE collected_at >= DATEADD(DAY, -?, GETUTCDATE())
        """

        with pyodbc.connect(self.conn_str) as conn:
            return pd.read_sql(query, conn, params=[self.lookback_days])

    def compute_baselines(self) -> dict:
        df = self.fetch_historical_waits()
        df['hour_of_week'] = pd.to_datetime(df['collected_at']).dt.dayofweek * 24 \
            + pd.to_datetime(df['collected_at']).dt.hour

        baselines = {}
        for category in self.WAIT_CATEGORIES:
            cat_df = df[df['category'] == category]
            baselines[category] = cat_df.groupby('hour_of_week')
['delta_wait_ms'].agg(
                p50=lambda x: np.percentile(x, 50),
                p95=lambda x: np.percentile(x, 95),
                p99=lambda x: np.percentile(x, 99),
            ).to_dict('index')
        return baselines

```

```
def is_anomalous(self, category: str, current_ms: float,
                 hour_of_week: int, threshold_pct: float = 95.0) -> bool:
    baselines = self.compute_baselines()
    if category not in baselines or hour_of_week not in baselines[category]:
        return False
    p95 = baselines[category][hour_of_week]['p95']
    return current_ms > p95
```

The output of this sensor network is not alerts — alerts are the human path. The output is structured decision events that flow to the autonomy controller described in later subparts.

Query Store as the Autonomous Query Governor

Query Store is SQL Server's most underutilized autonomous control surface. Most teams use it reactively — they open SSMS, look at the "Top Resource Consuming Queries" report, and manually force a plan. That is precisely the workflow that autonomous systems are designed to replace. At the ADE level, Query Store becomes a continuously monitored decision substrate where the system detects plan regressions, validates alternative plans, and forces or unforces them without a human in the loop.

The autonomous query governance loop has four stages:

1. **Regression detection** — identify queries where the latest execution plan performs materially worse than the best known plan for the same query hash.
2. **Impact scoring** — rank regressions by total elapsed time impact, not just per-execution cost.
3. **Validation** — before forcing a plan, verify the "best" plan is still valid given current statistics.
4. **Remediation** — force the plan or escalate to an index-change workflow if no valid plan exists.

```

-- Autonomous plan regression detector
-- Identifies queries where active plan is >2x worse than best known plan
-- weighted by execution frequency over the last 2 hours

WITH recent_stats AS (
    SELECT
        qsq.query_id,
        qsq.query_hash,
        qsp.plan_id,
        qsp.is_forced_plan,
        SUM(qsrs.count_executions) AS exec_count,
        AVG(qsrs.avg_duration) AS avg_duration_us,
        AVG(qsrs.avg_cpu_time) AS avg_cpu_us,
        AVG(qsrs.avg_logical_io_reads) AS avg_logical_reads,
        SUM(qsrs.count_executions * qsrs.avg_duration) AS total_elapsed_us,
        MAX(qsrs.last_execution_time) AS last_exec
    FROM sys.query_store_query qsq
    JOIN sys.query_store_plan qsp ON qsq.query_id = qsp.query_id
    JOIN sys.query_store_runtime_stats qsrs ON qsp.plan_id = qsrs.plan_id
    JOIN sys.query_store_runtime_stats_interval qsrssi
        ON qsrs.runtime_stats_interval_id
        = qsrssi.runtime_stats_interval_id
    WHERE qsrssi.start_time >= DATEADD(HOUR, -2, GETUTCDATE())
    GROUP BY qsq.query_id, qsq.query_hash, qsp.plan_id, qsp.is_forced_plan
),
best_plan AS (
    SELECT
        query_id,
        MIN(avg_duration_us) AS best_avg_duration_us,
        MIN(avg_logical_reads) AS best_avg_logical_reads
    FROM recent_stats
    GROUP BY query_id
),
active_plan AS (
    -- The plan with the most recent execution for each query
    SELECT rs.*
    FROM recent_stats rs
    WHERE rs.last_exec = (
        SELECT MAX(rs2.last_exec)
        FROM recent_stats rs2
        WHERE rs2.query_id = rs.query_id
    )
)
SELECT
    ap.query_id,
    ap.plan_id AS active_plan_id,
    ap.is_forced_plan,
    ap.avg_duration_us AS active_avg_duration_us,
    bp.best_avg_duration_us,
    CAST(ap.avg_duration_us AS FLOAT)
        / NULLIF(bp.best_avg_duration_us,0) AS regression_ratio,
    ap.exec_count,
    ap.total_elapsed_us / 1000.0 AS total_elapsed_ms,

```

```

-- Find the best plan_id for forcing
(SELECT TOP 1 rs3.plan_id
 FROM recent_stats rs3
 WHERE rs3.query_id = ap.query_id
       AND rs3.avg_duration_us = bp.best_avg_duration_us) AS best_plan_id,
GETUTCDATE() AS detected_at
FROM active_plan ap
JOIN best_plan bp ON ap.query_id = bp.query_id
WHERE CAST(ap.avg_duration_us AS FLOAT)
      / NULLIF(bp.best_avg_duration_us,0) > 2.0 -- regression threshold
  AND ap.exec_count > 10 -- statistical significance
  AND bp.best_avg_duration_us > 1000 -- ignore sub-millisecond
queries
  AND ap.is_forced_plan = 0 -- not already managed
ORDER BY total_elapsed_ms DESC;

```

The Python controller that acts on this output handles the force/unforce decision with a guard rail: before forcing any plan, it verifies that statistics on referenced objects are not stale beyond a threshold. Forcing an old plan against severely outdated statistics produces a different kind of regression.


```

import pyodbc
import logging
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

logger = logging.getLogger('autonomous_qg')

@dataclass
class PlanRegression:
    query_id: int
    active_plan_id: int
    best_plan_id: int
    regression_ratio: float
    total_elapsed_ms: float
    detected_at: datetime

class AutonomousQueryGovernor:
    """
    Detects Query Store plan regressions and autonomously forces the best
    known plan after validating statistics freshness.
    """

    STATS_STALENESS_HOURS = 24
    MAX_AUTO_FORCE_RATIO = 10.0 # Don't auto-force if regression > 10x (likely
different workload)
    MIN_ELAPSED_MS = 5000 # Only act if total impact > 5 seconds in window

    def __init__(self, conn_str: str, dry_run: bool = False):
        self.conn_str = conn_str
        self.dry_run = dry_run

    def _conn(self):
        return pyodbc.connect(self.conn_str, autocommit=True)

    def detect_regressions(self) -> list[PlanRegression]:
        regression_sql = open('regression_detection.sql').read() # The query above
with self._conn() as conn:
        cursor = conn.cursor()
        cursor.execute(regression_sql)
        return [
            PlanRegression(
                query_id = row.query_id,
                active_plan_id = row.active_plan_id,
                best_plan_id = row.best_plan_id,
                regression_ratio= row.regression_ratio,
                total_elapsed_ms= row.total_elapsed_ms,
                detected_at = row.detected_at
            )
            for row in cursor.fetchall()
        ]

    def validate_stats_freshness(self, query_id: int) -> bool:

```

```

"""
Check that all tables referenced by the best plan have stats
updated within the staleness threshold.
"""
sql = """
SELECT COUNT(*) AS stale_count
FROM sys.query_store_query_text qt
JOIN sys.query_store_query      q  ON qt.query_text_id = q.query_text_id
-- parse referenced objects and check sys.stats / sys.stats_columns
-- simplified: check overall DB stats freshness as a proxy
CROSS APPLY (
    SELECT COUNT(*) AS stale
    FROM sys.stats s
    JOIN sys.objects o ON s.object_id = o.object_id
    WHERE o.type = 'U'
        AND STATS_DATE(s.object_id, s.stats_id) < DATEADD(
            HOUR, -?, GETUTCDATE())
        AND s.auto_created = 1
    ) stale_check
WHERE q.query_id = ?
    AND stale_check.stale > 0
"""

with self._conn() as conn:
    cursor = conn.cursor()
    cursor.execute(sql, [self.STATS_STALENESS_HOURS, query_id])
    row = cursor.fetchone()
    return row.stale_count == 0

def force_plan(self, query_id: int, plan_id: int) -> bool:
    sql = "EXEC sys.sp_query_store_force_plan @query_id=?, @plan_id=?"
    try:
        if not self.dry_run:
            with self._conn() as conn:
                conn.cursor().execute(sql, [query_id, plan_id])
            logger.info(
                "PLAN_FORCED query_id=%d plan_id=%d dry_run=%s",
                query_id, plan_id, self.dry_run
            )
        return True
    except Exception as e:
        logger.error("PLAN_FORCE_FAILED query_id=%d error=%s", query_id, e)
        return False

def run_cycle(self):
    regressions = self.detect_regressions()
    logger.info("CYCLE_START detected_regressions=%d", len(regressions))

    for reg in regressions:
        if reg.regression_ratio > self.MAX_AUTO_FORCE_RATIO:
            logger.warning(
                "ESCALATE_HUMAN query_id=%d ratio=%.1f "
                "(exceeds autonomous threshold)",
                reg.query_id, reg.regression_ratio
            )

```

```

        )
        continue

    if reg.total_elapsed_ms < self.MIN_ELAPSED_MS:
        logger.debug(
            "SKIP_LOW_IMPACT query_id=%d elapsed_ms=%.0f",
            reg.query_id, reg.total_elapsed_ms
        )
        continue

    if not self.validate_stats_freshness(reg.query_id):
        logger.warning(
            "STATS_STALE query_id=%d – triggering stats update first",
            reg.query_id
        )
        # Emit a stats update work item to the autonomy queue
        # rather than forcing the plan
        continue

    self.force_plan(reg.query_id, reg.best_plan_id)

```

The same loop runs in reverse: when a forced plan itself starts regressing (because data distribution shifted and the forced plan is no longer optimal), the system unforces it and lets the optimizer replan. This bidirectional governance is what separates autonomous query management from the one-directional manual interventions most teams perform.

Automatic plan correction — introduced in SQL Server 2017 as a Database Compatibility feature — provides a native version of this loop. Enabling it is a prerequisite, but it is not sufficient for full autonomy because its correction logic is binary (force or don't) and does not participate in a broader decision pipeline. The ADE builds on top of it, using `sys.dm_db_tuning_recommendations` as an additional signal source:

```
-- Surface the native engine's own tuning recommendations
-- Use these as pre-validated signals in the autonomous decision pipeline

SELECT
    name,
    type,
    reason,
    score,
    JSON_VALUE(details, '$.implementationDetails.script') AS recommended_action,
    JSON_VALUE(details, '$.planForceDetails.queryId')      AS affected_query_id,
    JSON_VALUE(details, '$.planForceDetails.regressedPlanId')
                                                                AS regressed_plan_id,
    JSON_VALUE(details, '$.planForceDetails.forcedPlanId')
                                                                AS recommended_plan_id,
    valid_since,
    last_refresh
FROM sys.dm_db_tuning_recommendations
WHERE state = JSON_VALUE(state, '$.currentValue') -- active recommendations only
    AND score > 30                                -- confidence threshold
ORDER BY score DESC;
```

Always On Availability Groups: Autonomous Failover and Replica Management

Always On Availability Groups are SQL Server's primary HA mechanism, but the default configuration places humans at the center of decisions that should be automated: when to fail over, when to promote a secondary, when to redirect read workloads, and when to suspend data movement to a lagging replica. The autonomous architecture eliminates those human decision points.

The autonomy surface for AG management has three layers. The first is the Windows Server Failover Cluster (WSFC) or Pacemaker layer, which handles the low-level quorum and node fencing decisions. The second is the SQL Server AG health detection layer —

`sys.dm_hadr_availability_group_states`, `sys.dm_hadr_database_replica_states`, `sys.dm_hadr_availability_replica_states`. The third is the application-level routing layer, which manages connection routing independently of what the AG itself reports.

For environments running Always On across Azure VMs or on-premises with Pacemaker on Linux, the autonomous controller monitors redo queue depth as a leading indicator of failover risk. A secondary with a growing redo queue that is not catching up indicates either an I/O-bound secondary or a network partition that will worsen before it improves.

```

-- AG replica health monitor for autonomous controller
-- Collects redo queue, send queue, and synchronization state per replica

SELECT
    ag.name                                AS ag_name,
    ar.replica_server_name,
    ar.availability_mode_desc,
    ar.failover_mode_desc,
    drs.database_name,
    drs.synchronization_state_desc,
    drs.synchronization_health_desc,
    drs.redo_queue_size                    AS redo_queue_kb,
    drs.redo_rate                          AS redo_rate_kb_per_sec,
    CASE
        WHEN drs.redo_rate > 0
        THEN drs.redo_queue_size / drs.redo_rate
        ELSE NULL
    END                                    AS estimated_catchup_secs,
    drs.log_send_queue_size                AS send_queue_kb,
    drs.log_send_rate                      AS send_rate_kb_per_sec,
    drs.last_redone_time,
    drs.last_received_time,
    ars.role_desc,
    ars.connected_state_desc,
    ars.operational_state_desc,
    GETUTCDATE()                          AS sampled_at
FROM sys.availability_groups ag
JOIN sys.availability_replicas ar
    ON ag.group_id = ar.group_id
JOIN sys.dm_hadr_availability_replica_states ars
    ON ar.replica_id = ars.replica_id
JOIN sys.dm_hadr_database_replica_states drs
    ON ar.replica_id = drs.replica_id
ORDER BY ag.name, ar.replica_server_name, drs.database_name;

```

The Python autonomous AG controller acts on this data in two modes: *watchdog mode* (continuous monitoring with corrective actions short of failover) and *failover arbitration mode* (triggered only when the primary is confirmed unhealthy).

```

import pyodbc
import logging
import time
from dataclasses import dataclass, field
from typing import Optional
from datetime import datetime, timedelta
from enum import Enum

logger = logging.getLogger('autonomous_ag')

class ReplicaHealth(Enum):
    HEALTHY = 'HEALTHY'
    DEGRADED = 'DEGRADED'
    CRITICAL = 'CRITICAL'
    UNREACHABLE = 'UNREACHABLE'

@dataclass
class ReplicaState:
    ag_name: str
    replica_server_name: str
    availability_mode: str
    failover_mode: str
    database_name: str
    sync_state: str
    sync_health: str
    redo_queue_kb: Optional[float]
    redo_rate_kb_per_sec: Optional[float]
    estimated_catchup_secs: Optional[float]
    send_queue_kb: Optional[float]
    role: str
    connected_state: str
    operational_state: str
    sampled_at: datetime

    @property
    def health(self) -> ReplicaHealth:
        if self.connected_state != 'CONNECTED' or self.operational_state !=
'ONLINE':
            return ReplicaHealth.UNREACHABLE
        if self.sync_health == 'NOT_HEALTHY':
            return ReplicaHealth.CRITICAL
        if self.sync_health == 'PARTIALLY_HEALTHY':
            return ReplicaHealth.DEGRADED
        # Redo queue growing faster than it drains – degraded signal
        if (self.redo_queue_kb is not None
            and self.estimated_catchup_secs is not None
            and self.estimated_catchup_secs > 300):          # 5 minutes behind
            return ReplicaHealth.DEGRADED
        return ReplicaHealth.HEALTHY

class AutonomousAGController:
    """

```

Watchdog and arbitration controller for SQL Server Always On AGs.
 Operates in watchdog mode continuously; escalates to failover arbitration
 only after quorum of health checks confirms primary failure.
 """

REDO_QUEUE_WARN_KB = 512 * 1024 # 512 MB redo lag triggers warning
 REDO_QUEUE_SUSPEND_KB = 2 * 1024 * 1024 # 2 GB lag triggers data movement

review

FAILOVER_CONFIRM_CYCLES = 3 # Primary must fail N consecutive cycles
 CYCLE_INTERVAL_SECS = 15

```
def __init__(self, primary_conn_str: str,
              secondary_conn_strs: dict[str, str],
              dry_run: bool = False):
    self.primary_conn_str = primary_conn_str
    self.secondary_conn_strs = secondary_conn_strs
    self.dry_run = dry_run
    self._primary_fail_count = 0
    self._last_failover_time: Optional[datetime] = None
```

```
def _conn(self, conn_str: str):
    return pyodbc.connect(conn_str, timeout=5, autocommit=True)
```

```
def collect_replica_states(self, conn_str: str) -> list[ReplicaState]:
    sql = open('ag_health_monitor.sql').read() # The query above
    try:
        with self._conn(conn_str) as conn:
            cursor = conn.cursor()
            cursor.execute(sql)
            rows = cursor.fetchall()
            return [ReplicaState(
                ag_name=r.ag_name,
                replica_server_name=r.replica_server_name,
                availability_mode=r.availability_mode_desc,
                failover_mode=r.failover_mode_desc,
                database_name=r.database_name,
                sync_state=r.synchronization_state_desc,
                sync_health=r.synchronization_health_desc,
                redo_queue_kb=r.redo_queue_kb,
                redo_rate_kb_per_sec=r.redo_rate_kb_per_sec,
                estimated_catchup_secs=r.estimated_catchup_secs,
                send_queue_kb=r.send_queue_kb,
                role=r.role_desc,
                connected_state=r.connected_state_desc,
                operational_state=r.operational_state_desc,
                sampled_at=r.sampled_at,
            ) for r in rows]
    except Exception as e:
        logger.error("REPLICA_COLLECT_FAILED conn=%s error=%s", conn_str, e)
    return []
```

```
def check_primary_reachable(self) -> bool:
    try:
```

```

        with self._conn(self.primary_conn_str) as conn:
            conn.cursor().execute("SELECT 1")
        return True
    except Exception:
        return False

def handle_lagging_replica(self, state: ReplicaState):
    """
    When a synchronous secondary lags beyond REDO_QUEUE_WARN_KB,
    evaluate whether to temporarily change it to asynchronous mode
    to protect primary throughput – a reversible, safe corrective action.
    """
    if state.redo_queue_kb is None:
        return

    if state.redo_queue_kb > self.REDO_QUEUE_WARN_KB:
        logger.warning(
            "REPLICA_LAG ag=%s replica=%s db=%s redo_queue_kb=%.0f "
            "catchup_secs=%s",
            state.ag_name, state.replica_server_name, state.database_name,
            state.redo_queue_kb,
            f"{state.estimated_catchup_secs:.0f}"
            if state.estimated_catchup_secs else "unknown"
        )

    if (state.redo_queue_kb > self.REDO_QUEUE_SUSPEND_KB
        and state.availability_mode == 'SYNCHRONOUS_COMMIT'
        and not self.dry_run):
        # Switching to async protects primary commit latency while
        # the secondary catches up; schedule a reversion check
        logger.warning(
            "ASYNC_DOWNGRADE ag=%s replica=%s – redo queue critical",
            state.ag_name, state.replica_server_name
        )
        self._change_replica_mode(
            state.ag_name, state.replica_server_name, 'ASYNCHRONOUS_COMMIT'
        )

def _change_replica_mode(self, ag_name: str, replica_name: str, mode: str):
    sql = f"""
    ALTER AVAILABILITY GROUP [{ag_name}]
    MODIFY REPLICA ON N'{replica_name}'
    WITH (AVAILABILITY_MODE = {mode})
    """
    try:
        with self._conn(self.primary_conn_str) as conn:
            conn.cursor().execute(sql)
        logger.info(
            "REPLICA_MODE_CHANGED ag=%s replica=%s mode=%s",
            ag_name, replica_name, mode
        )
    except Exception as e:
        logger.error(

```



```

        "REPLICA_MODE_CHANGE_FAILED ag=%s replica=%s error=%s",
        ag_name, replica_name, e
    )

def initiate_failover(self, target_replica: str, ag_name: str):
    """
    Initiates a manual failover to a specific secondary.
    Only called after FAILOVER_CONFIRM_CYCLES of confirmed primary failure
    and only if the target has MANUAL or AUTOMATIC failover mode configured.
    """
    # Enforce a minimum interval between autonomous failovers
    if (self._last_failover_time is not None
        and datetime.utcnow() - self._last_failover_time
        < timedelta(minutes=30)):
        logger.error(
            "FAILOVER_THROTTLED - last failover was less than 30 minutes ago"
        )
        return

    sql = f"ALTER AVAILABILITY GROUP [{ag_name}] FAILOVER"
    conn_str = self.secondary_conn_strs.get(target_replica)
    if not conn_str:
        logger.error("FAILOVER_NO_CONN_STR replica=%s", target_replica)
        return

    logger.critical(
        "FAILOVER_INITIATED ag=%s target=%s dry_run=%s",
        ag_name, target_replica, self.dry_run
    )
    if not self.dry_run:
        try:
            with self._conn(conn_str) as conn:
                conn.cursor().execute(sql)
                self._last_failover_time = datetime.utcnow()
        except Exception as e:
            logger.error("FAILOVER_FAILED ag=%s error=%s", ag_name, e)

def select_best_failover_target(
    self, states: list[ReplicaState]) -> Optional[tuple[str, str]]:
    """
    Among synchronous secondaries, pick the one with the smallest
    redo queue (closest to primary). Returns (replica_name, ag_name).
    """
    candidates = [
        s for s in states
        if s.role == 'SECONDARY'
        and s.availability_mode == 'SYNCHRONOUS_COMMIT'
        and s.health != ReplicaHealth.UNREACHABLE
    ]
    if not candidates:
        return None
    best = min(candidates,
               key=lambda s: s.redo_queue_kb or float('inf'))

```

```

        return (best.replica_server_name, best.ag_name)

def run_watchdog(self):
    """Main watchdog loop. Runs indefinitely."""
    logger.info("WATCHDOG_START primary=%s", self.primary_conn_str)
    while True:
        primary_up = self.check_primary_reachable()

        if not primary_up:
            self._primary_fail_count += 1
            logger.warning(
                "PRIMARY_UNREACHABLE consecutive_failures=%d",
                self._primary_fail_count
            )
            if self._primary_fail_count >= self.FAILOVER_CONFIRM_CYCLES:
                # Collect state from a secondary to identify best target
                for name, conn_str in self.secondary_conn_strs.items():
                    states = self.collect_replica_states(conn_str)
                    if states:
                        target = self.select_best_failover_target(states)
                        if target:
                            self.initiate_failover(*target)
                            break
            else:
                self._primary_fail_count = 0
                states = self.collect_replica_states(self.primary_conn_str)
                for state in states:
                    if state.role == 'SECONDARY':
                        self.handle_lagging_replica(state)

        time.sleep(self.CYCLE_INTERVAL_SECS)

```

One detail that trips up many implementations: the `ALTER AVAILABILITY GROUP ... FAILOVER` statement must be issued *from the target secondary*, not from the primary. The controller connects to the secondary's connection string for the failover command, which is why `secondary_conn_strs` is a required parameter rather than being derived from DMV data.

For environments using the Windows Server Failover Cluster layer, automatic failover for `AUTOMATIC` failover mode replicas is already handled by WSFC without any application-layer code. The Python controller above is designed for scenarios where WSFC automatic failover is not in play — Linux HA clusters, Azure environments using a load balancer listener without WSFC, or configurations where the DBA team wants application-layer override capability above the WSFC decision.

Intelligent Connection Routing and Read-Scale Autonomy

Availability Groups introduce multiple replicas that can serve read traffic, but the default routing behavior — round-robin across read-only routing lists — is static. An autonomous routing layer adjusts replica selection based on replica health, redo lag, and current replica load in real time.

SQL Server's built-in read-only routing is configured via `PRIMARY_ROLE` and `SECONDARY_ROLE` routing lists on availability replicas. The autonomous layer supplements this with a routing proxy that re-evaluates replica suitability at connection time.

SQL Server — configuring the read-only routing foundation:

```
-- Set read-only routing URLs and priority list for AG 'ProductionAG'
-- Run on the primary

ALTER AVAILABILITY GROUP [ProductionAG]
MODIFY REPLICA ON N'SQLNODE2'
WITH (
    SECONDARY_ROLE (
        ALLOW_CONNECTIONS = READ_ONLY,
        READ_ONLY_ROUTING_URL = N'TCP://sqlnode2.corp.local:1433'
    )
);

ALTER AVAILABILITY GROUP [ProductionAG]
MODIFY REPLICA ON N'SQLNODE3'
WITH (
    SECONDARY_ROLE (
        ALLOW_CONNECTIONS = READ_ONLY,
        READ_ONLY_ROUTING_URL = N'TCP://sqlnode3.corp.local:1433'
    )
);

-- Set routing list on primary: prefer SQLNODE2, fall back to SQLNODE3
ALTER AVAILABILITY GROUP [ProductionAG]
MODIFY REPLICA ON N'SQLNODE1'
WITH (
    PRIMARY_ROLE (
        READ_ONLY_ROUTING_LIST = (
            ('SQLNODE2', 'SQLNODE3') -- load-balanced pair
        )
    )
);
```

The autonomous routing controller wraps the connection acquisition logic in Python, scoring each replica before assigning a connection:

```

from dataclasses import dataclass
from typing import Optional
import pyodbc
import threading
import time

@dataclass
class ReplicaScore:
    name: str
    conn_str: str
    redo_lag_kb: float
    active_sessions: int
    avg_wait_ms: float
    is_healthy: bool

    @property
    def score(self) -> float:
        """Lower score is better. Combines lag, load, and wait pressure."""
        if not self.is_healthy:
            return float('inf')
        # Normalize each component to a 0-100 scale before weighting
        lag_score = min(self.redo_lag_kb / 10_000, 100) # cap at 10 GB equiv
        load_score = min(self.active_sessions / 200, 100) # cap at 200 sessions
        wait_score = min(self.avg_wait_ms / 500, 100) # cap at 500ms avg
wait
        return (lag_score * 0.4) + (load_score * 0.35) + (wait_score * 0.25)

class AutonomousReadRouter:
    """
    Scores AG read replicas and routes read connections to the best
    available secondary. Thread-safe; refreshes scores at a configurable interval.
    """

    SCORE_REFRESH_SECS = 10

    def __init__(self, replica_configs: dict[str, str]):
        """
        replica_configs: {replica_name: connection_string}
        """
        self.replica_configs = replica_configs
        self._scores: list[ReplicaScore] = []
        self._lock = threading.Lock()
        self._refresh_thread = threading.Thread(
            target=self._background_refresh, daemon=True
        )
        self._refresh_thread.start()

    def _score_replica(self, name: str, conn_str: str) -> ReplicaScore:
        try:
            with pyodbc.connect(conn_str, timeout=3) as conn:
                cursor = conn.cursor()
                # Redo lag for this replica

```

```

        cursor.execute("""
            SELECT
                ISNULL(SUM(redo_queue_size), 0) AS redo_lag_kb
            FROM sys.dm_hadr_database_replica_states
            WHERE is_local = 1
        """)
        redo_lag_kb = cursor.fetchone().redo_lag_kb or 0.0

    # Active session count
    cursor.execute("""
        SELECT COUNT(*) AS session_count
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1
    """)
    active_sessions = cursor.fetchone().session_count or 0

    # Average wait time from recent wait stats
    cursor.execute("""
        SELECT
            ISNULL(
                SUM(wait_time_ms) * 1.0 /
                NULLIF(SUM(waiting_tasks_count), 0),
                0) AS avg_wait_ms
        FROM sys.dm_os_wait_stats
        WHERE wait_type NOT IN (
            'SLEEP_TASK', 'WAITFOR', 'XE_TIMER_EVENT',
            'BROKER_TO_FLUSH', 'REQUEST_FOR_DEADLOCK_MONITOR'
        )
    """)
    avg_wait_ms = cursor.fetchone().avg_wait_ms or 0.0

    return ReplicaScore(
        name=name, conn_str=conn_str,
        redo_lag_kb=redo_lag_kb,
        active_sessions=active_sessions,
        avg_wait_ms=avg_wait_ms,
        is_healthy=True
    )
except Exception:
    return ReplicaScore(
        name=name, conn_str=conn_str,
        redo_lag_kb=float('inf'), active_sessions=0,
        avg_wait_ms=0.0, is_healthy=False
    )

def _background_refresh(self):
    while True:
        scores = [
            self._score_replica(name, conn_str)
            for name, conn_str in self.replica_configs.items()
        ]
        with self._lock:
            self._scores = sorted(scores, key=lambda s: s.score)

```

```

        time.sleep(self.SCORE_REFRESH_SECS)

    def get_read_connection(self) -> Optional[pyodbc.Connection]:
        """Returns a connection to the best available read replica."""
        with self._lock:
            candidates = [s for s in self._scores if s.is_healthy]
            if not candidates:
                raise RuntimeError("No healthy read replicas available")
            best = candidates[0]
            return pyodbc.connect(best.conn_str)

```

The routing layer deliberately keeps the scoring logic transparent and overridable. When a replica enters a planned maintenance window — index rebuilds, DBCC CHECKDB, or backup operations — the autonomous maintenance scheduler (described in the next subpart) posts a `REPLICA_MAINTENANCE` event that temporarily sets that replica's health flag to `False`, pulling it out of the routing pool without requiring any network-level change.

Autonomous Index and Statistics Management

SQL Server's built-in maintenance plans and Ola Hallengren's maintenance solution are both schedule-driven rather than condition-driven. Autonomous index management replaces the schedule with a continuous assessment loop: collect fragmentation state, score rebuild vs. reorganize decisions by impact and available maintenance window, and execute within cost guardrails.

The core signal is `sys.dm_db_index_physical_stats`, but querying it directly in SAMPLED mode at scale has a non-trivial I/O cost. The autonomous collector uses a tiered scan strategy: small tables are scanned nightly in DETAILED mode; large tables use LIMITED mode daily with DETAILED mode deferred to weekly off-peak windows.

```

-- Autonomous index fragmentation assessment with tiered scan mode
-- Returns actionable rows with recommended operation and estimated cost

SELECT
    OBJECT_SCHEMA_NAME(ips.object_id) AS schema_name,
    OBJECT_NAME(ips.object_id) AS table_name,
    i.name AS index_name,
    ips.index_id,
    ips.index_type_desc,
    ips.avg_fragmentation_in_percent,
    ips.page_count,
    ips.record_count,
    ips.avg_page_space_used_in_percent,
    -- Decide operation based on fragmentation and page count
    CASE
        WHEN ips.page_count < 1000
            THEN 'SKIP' -- too small to matter
        WHEN ips.avg_fragmentation_in_percent < 5
            THEN 'SKIP'
        WHEN ips.avg_fragmentation_in_percent BETWEEN 5 AND 30
            THEN 'REORGANIZE'
        ELSE 'REBUILD'
    END AS recommended_op,
    -- Rough rebuild cost estimate: pages * avg_page_size_kb
    ips.page_count * 8 / 1024.0 AS estimated_mb,
    GETUTCDATE() AS assessed_at
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'SAMPLED'
) ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id
    AND ips.index_id = i.index_id
WHERE i.index_id > 0 -- exclude heap scans
    AND ips.page_count > 100 -- minimum page threshold
ORDER BY ips.avg_fragmentation_in_percent DESC, ips.page_count DESC;

```

The Python autonomous index manager wraps the assessment with an execution budget: it tracks total I/O and lock time consumed during a maintenance window and stops before the window closes, persisting remaining work for the next cycle.

```

import pyodbc
import logging
import time
from dataclasses import dataclass
from typing import Iterator
from datetime import datetime, timedelta

logger = logging.getLogger('autonomous_index')

@dataclass
class IndexAction:
    schema_name: str
    table_name: str
    index_name: str
    index_id: int
    recommended_op: str      # 'REBUILD' | 'REORGANIZE' | 'SKIP'
    estimated_mb: float
    avg_fragmentation: float
    page_count: int

class AutonomousIndexManager:
    """
    Condition-driven index maintenance for SQL Server.
    Executes within a time and I/O budget, yielding gracefully if
    the maintenance window closes.
    """

    REBUILD_ONLINE_THRESHOLD_MB = 50_000 # Tables > 50 GB: prefer ONLINE rebuild
    REORG_FRAG_MIN               = 5.0
    REBUILD_FRAG_MIN             = 30.0
    DEFAULT_WINDOW_MINUTES      = 120

    def __init__(self, conn_str: str, dry_run: bool = False,
                  window_minutes: int = DEFAULT_WINDOW_MINUTES):
        self.conn_str = conn_str
        self.dry_run = dry_run
        self.window_end = datetime.utcnow() + timedelta(minutes=window_minutes)

    def _conn(self):
        return pyodbc.connect(self.conn_str, autocommit=True)

    def _within_window(self) -> bool:
        return datetime.utcnow() < self.window_end

    def assess(self) -> list[IndexAction]:
        sql = open('index_fragmentation_assessment.sql').read()
        with self._conn() as conn:
            cursor = conn.cursor()
            cursor.execute(sql)
            return [
                IndexAction(
                    schema_name=r.schema_name,
                    table_name=r.table_name,

```



```

        index_name=r.index_name,
        index_id=r.index_id,
        recommended_op=r.recommended_op,
        estimated_mb=r.estimated_mb,
        avg_fragmentation=r.avg_fragmentation_in_percent,
        page_count=r.page_count,
    )
    for r in cursor.fetchall():
        if r.recommended_op != 'SKIP'
    ]

def _build_maintenance_sql(self, action: IndexAction) -> str:
    fq_table = f"[{action.schema_name}].[{action.table_name}]"
    index     = f"[{action.index_name}]"

    if action.recommended_op == 'REORGANIZE':
        return (
            f"ALTER INDEX {index} ON {fq_table} "
            f"REORGANIZE"
        )
    elif action.recommended_op == 'REBUILD':
        online = (action.estimated_mb > self.REBUILD_ONLINE_THRESHOLD_MB
                  and self._is_enterprise_edition())
        online_clause = "WITH (ONLINE = ON, MAXDOP = 4)" if online \
            else "WITH (MAXDOP = 4)"
        return (
            f"ALTER INDEX {index} ON {fq_table} "
            f"REBUILD {online_clause}"
        )
    raise ValueError(f"Unknown op: {action.recommended_op}")

def _is_enterprise_edition(self) -> bool:
    with self._conn() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT SERVERPROPERTY('EngineEdition') AS edition"
        )
        return cursor.fetchone().edition == 3    # 3 = Enterprise

def execute_maintenance(self, actions: list[IndexAction]):
    # Sort: reorganize first (lower locking cost), then rebuild by size asc
    sorted_actions = sorted(
        actions,
        key=lambda a: (0 if a.recommended_op == 'REORGANIZE' else 1,
                       a.estimated_mb)
    )

    completed = 0
    skipped    = 0

    for action in sorted_actions:
        if not self._within_window():
            logger.info(

```

```

        "WINDOW_CLOSED completed=%d skipped=%d remaining=%d",
        completed, skipped, len(sorted_actions) - completed - skipped
    )
    break

sql = self._build_maintenance_sql(action)
logger.info(
    "INDEX_MAINT op=%s table=%s.%s index=%s frag=%.1f%% est_mb=%.0f",
    action.recommended_op, action.schema_name,
    action.table_name, action.index_name,
    action.avg_fragmentation, action.estimated_mb
)

if not self.dry_run:
    try:
        start = time.monotonic()
        with self._conn() as conn:
            conn.cursor().execute(sql)
        elapsed = time.monotonic() - start
        logger.info(
            "INDEX_MAINT_DONE elapsed_secs=%.1f", elapsed
        )
        completed += 1
    except Exception as e:
        logger.error(
            "INDEX_MAINT_FAILED table=%s.%s index=%s error=%s",
            action.schema_name, action.table_name,
            action.index_name, e
        )
        skipped += 1
    else:
        completed += 1

logger.info(
    "MAINTENANCE_CYCLE_END completed=%d skipped=%d",
    completed, skipped
)

def run_cycle(self):
    actions = self.assess()
    logger.info("ASSESSMENT_COMPLETE actionable_indexes=%d", len(actions))
    self.execute_maintenance(actions)

```

Statistics management follows the same pattern. The autonomous stats controller evaluates modification counters from `sys.dm_db_stats_properties` against a density-aware threshold — tables with highly skewed distributions need stats updated at a lower modification threshold than uniform distributions, because the optimizer's cardinality estimates degrade faster on skewed data.

```

-- Autonomous statistics staleness assessment
-- Uses modification counter relative to table size + density proxy

SELECT
    OBJECT_SCHEMA_NAME(s.object_id)      AS schema_name,
    OBJECT_NAME(s.object_id)             AS table_name,
    s.name                               AS stats_name,
    sp.last_updated,
    sp.rows,
    sp.rows_sampled,
    sp.modification_counter,
    -- Dynamic threshold: smaller tables need proportionally more sensitivity
    CASE
        WHEN sp.rows < 500                THEN 500
        WHEN sp.rows < 25000              THEN CAST(sp.rows * 0.20 AS INT)
        ELSE CAST(500 + sp.rows * 0.02 AS INT) -- sub-linear heuristic for large
tables;
-- tune against your own workload
rather
-- than treating this as an
engine default
    END                                AS update_threshold,
    CASE
        WHEN sp.modification_counter >= CASE
            WHEN sp.rows < 500            THEN 500
            WHEN sp.rows < 25000          THEN CAST(sp.rows * 0.20 AS INT)
            ELSE CAST(500 + sp.rows * 0.02 AS INT)
        END
        THEN 'UPDATE_STATS'
        ELSE 'SKIP'
    END                                AS recommended_op,
    DATEDIFF(HOUR, sp.last_updated, GETUTCDATE()) AS hours_since_update
FROM sys.stats s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) sp
JOIN sys.objects o ON s.object_id = o.object_id
WHERE o.type = 'U' -- user tables only
    AND sp.rows > 0
    AND (
        sp.last_updated < DATEADD(DAY, -1, GETUTCDATE()) -- always refresh daily at
minimum
        OR sp.modification_counter >= CASE
            WHEN sp.rows < 500            THEN 500
            WHEN sp.rows < 25000          THEN CAST(sp.rows * 0.20 AS INT)
            ELSE CAST(500 + sp.rows * 0.02 AS INT)
        END
    )
ORDER BY sp.modification_counter DESC, hours_since_update DESC;

```

The statistics update execution mirrors the index maintenance pattern: sorted by modification count descending (highest urgency first), executed within the same maintenance window budget, and logged to the same audit table so the controller can demonstrate ROI over time.

Autonomous Memory and Resource Governor Management

SQL Server's Resource Governor is one of the most powerful — and most underused — autonomous control surfaces in the engine. At steady state, most installations run with the default pool and a single workload group. The autonomous architecture treats Resource Governor as a dynamic pressure valve: when the sensor network detects memory pressure, CPU saturation, or blocking chains, the controller adjusts pool limits and workload group classifications in real time.

The feedback loop works as follows: the wait stats sensor detects sustained `RESOURCE_SEMAPHORE` waits (query memory grant queuing), which signals that the memory grant pool is exhausted. The controller inspects `sys.dm_exec_query_memory_grants` to identify which workload groups are consuming grants, and either caps the offending groups or redistributes the `MAX_MEMORY_PERCENT` between pools.

```
-- Memory grant pressure diagnostic for autonomous controller
SELECT
    mg.session_id,
    mg.request_id,
    mg.grant_time,
    mg.requested_memory_kb,
    mg.granted_memory_kb,
    mg.used_memory_kb,
    mg.max_used_memory_kb,
    mg.queue_id,
    mg.wait_order,
    mg.wait_time_ms,
    mg.is_next_candidate,
    wg.name AS workload_group,
    rp.name AS resource_pool,
    rp.max_memory_percent,
    wg.max_request_grant_memory_kb,
    es.login_name,
    es.program_name,
    SUBSTRING(qt.text, (er.statement_start_offset/2)+1,
        ((CASE er.statement_end_offset
            WHEN -1 THEN DATALENGTH(qt.text)
            ELSE er.statement_end_offset END
            - er.statement_start_offset)/2)+1) AS statement_text
FROM sys.dm_exec_query_memory_grants mg
JOIN sys.dm_exec_sessions es ON mg.session_id = es.session_id
JOIN sys.dm_exec_requests er ON mg.session_id = er.session_id
                                AND mg.request_id = er.request_id
JOIN sys.dm_resource_governor_workload_groups wg
    ON mg.group_id = wg.group_id
JOIN sys.dm_resource_governor_resource_pools rp
    ON wg.pool_id = rp.pool_id
CROSS APPLY sys.dm_exec_sql_text(er.sql_handle) qt
ORDER BY mg.granted_memory_kb DESC;
```

When the autonomous controller identifies a resource pool absorbing disproportionate memory, it issues a live reconfiguration:

```

class AutonomousResourceGovernor:
    """
    Dynamically adjusts SQL Server Resource Governor pool limits
    in response to memory and CPU pressure signals.
    """

    def __init__(self, conn_str: str, dry_run: bool = False):
        self.conn_str = conn_str
        self.dry_run = dry_run

    def _conn(self):
        return pyodbc.connect(self.conn_str, autocommit=True)

    def get_pool_memory_usage(self) -> list[dict]:
        sql = """
            SELECT
                rp.name                                AS pool_name,
                rp.max_memory_percent,
                rp.min_memory_percent,
                rp_stats.target_memory_kb,
                rp_stats.used_memory_kb,
                CAST(rp_stats.used_memory_kb AS FLOAT)
                    / NULLIF(rp_stats.target_memory_kb, 0) * 100
                    AS utilization_pct
            FROM sys.dm_resource_governor_resource_pools rp
            JOIN sys.dm_resource_governor_resource_pools_history_ex rp_stats
                ON rp.pool_id = rp_stats.pool_id
        """
        try:
            with self._conn() as conn:
                cursor = conn.cursor()
                cursor.execute(sql)
                cols = [col[0] for col in cursor.description]
                return [dict(zip(cols, row)) for row in cursor.fetchall()]
        except Exception as e:
            logger.error("POOL_USAGE_FAILED error=%s", e)
            return []

    def reduce_pool_memory(self, pool_name: str, new_max_pct: int):
        """Reduce a pool's maximum memory allocation."""
        # Enforce a floor to prevent starving the pool entirely
        new_max_pct = max(new_max_pct, 10)
        sql = f"""
            ALTER RESOURCE POOL [{pool_name}]
            WITH (MAX_MEMORY_PERCENT = {new_max_pct});
            ALTER RESOURCE GOVERNOR RECONFIGURE;
        """
        logger.info(
            "POOL_MEMORY_REDUCE pool=%s new_max_pct=%d dry_run=%s",
            pool_name, new_max_pct, self.dry_run
        )
        if not self.dry_run:
            try:

```

```

        with self._conn() as conn:
            for stmt in sql.split(';'):
                stmt = stmt.strip()
                if stmt:
                    conn.cursor().execute(stmt)
    except Exception as e:
        logger.error(
            "POOL_MEMORY_REDUCE_FAILED pool=%s error=%s",
            pool_name, e
        )

def rebalance_under_pressure(self):
    """
    When total memory grant wait time is elevated, reduce the max memory
    of the highest-consuming non-default pool by 10 percentage points,
    rechecking after two collection cycles before further reduction.
    """
    pools = self.get_pool_memory_usage()
    # Exclude the internal and default pools from adjustment
    adjustable = [
        p for p in pools
        if p['pool_name'] not in ('internal', 'default')
        and p['utilization_pct'] is not None
    ]
    if not adjustable:
        return

    # Target the pool with highest utilization above 85%
    offender = max(adjutable, key=lambda p: p['utilization_pct'])
    if offender['utilization_pct'] > 85:
        current_max = offender['max_memory_percent']
        self.reduce_pool_memory(
            offender['pool_name'],
            max(current_max - 10, 10)
        )

```

The reversion path is equally important: once `RESOURCE_SEMAPHORE` waits return to baseline, the controller incrementally restores pool limits toward their configured maximums. All changes are recorded in the `dba.resource_governor_changes` audit table with the triggering signal, the before/after configuration, and the timestamp of both the change and any reversion, creating an evidence trail that demonstrates the system's decision-making over time.

Azure SQL and the Cloud Autonomy Layer

When SQL Server workloads run on Azure SQL Database or Azure SQL Managed Instance, the autonomy surface shifts. The platform handles storage scaling, OS patching, and basic HA invisibly. The ADE's responsibility moves up the stack: tier selection, DTU/vCore right-sizing, elastic pool management, and integration with Azure's native intelligence features like automatic tuning and Query Performance Insight.

Azure SQL's automatic tuning can create, drop, and force indexes and plans without human intervention. The ADE layer does not replace this — it supervises it. The supervision pattern uses the Azure SQL REST API and the `sys.dm_db_tuning_recommendations` view in concert:


```

import requests
from datetime import datetime, timedelta
import pyodbc

class AzureSQLAutonomyLayer:
    """
    Supervises Azure SQL automatic tuning actions and escalates
    those that cross defined impact thresholds to a human review queue.
    """

    # Tuning actions with impact scores above this are reviewed before applying
    HUMAN_REVIEW_SCORE_THRESHOLD = 80
    # Automatically approve low-impact tuning recommendations
    AUTO_APPROVE_SCORE_MAX = 40

    def __init__(self, conn_str: str,
                  subscription_id: str,
                  resource_group: str,
                  server_name: str,
                  database_name: str,
                  bearer_token: str):
        self.conn_str = conn_str
        self.base_url = (
            f"https://management.azure.com/subscriptions/{subscription_id}"
            f"/resourceGroups/{resource_group}"
            f"/providers/Microsoft.Sql/servers/{server_name}"
            f"/databases/{database_name}"
        )
        self.headers = {
            "Authorization": f"Bearer {bearer_token}",
            "Content-Type": "application/json"
        }

    def get_automatic_tuning_state(self) -> dict:
        url = f"{self.base_url}/automaticTuning?api-version=2021-11-01"
        response = requests.get(url, headers=self.headers, timeout=15)
        response.raise_for_status()
        return response.json()

    def set_tuning_option(self, option: str, desired_state: str):
        """
        option: 'createIndex' | 'dropIndex' | 'forceLastGoodPlan'
        desired_state: 'Auto' | 'Disabled' | 'Manual'
        """
        url = f"{self.base_url}/automaticTuning?api-version=2021-11-01"
        body = {
            "properties": {
                "desiredState": "Custom",
                "options": {
                    option: {"desiredState": desired_state}
                }
            }
        }

```

```

        response = requests.patch(url, json=body,
                                   headers=self.headers, timeout=15)
        response.raise_for_status()
        logger.info(
            "AZURE_TUNING_SET option=%s state=%s", option, desired_state
        )

def review_recommendations(self):
    """
    Pull tuning recommendations and route each by impact score:
    - Low impact (< AUTO_APPROVE_SCORE_MAX): approve automatically
    - High impact (> HUMAN_REVIEW_SCORE_THRESHOLD): queue for human review
    - Mid range: apply with enhanced monitoring for 24 hours
    """
    sql = """
        SELECT
            name, type, reason, score,
            JSON_VALUE(details, '$.implementationDetails.script')
                AS action_script,
            valid_since, last_refresh
        FROM sys.dm_db_tuning_recommendations
        WHERE state = JSON_VALUE(state, '$.currentValue')
            AND score IS NOT NULL
        ORDER BY score DESC
    """

    with pyodbc.connect(self.conn_str) as conn:
        cursor = conn.cursor()
        cursor.execute(sql)
        rows = cursor.fetchall()

    for row in rows:
        score = row.score or 0
        if score > self.HUMAN_REVIEW_SCORE_THRESHOLD:
            logger.warning(
                "TUNING_HUMAN_REVIEW name=%s type=%s score=%d",
                row.name, row.type, score
            )
            # Post to ticketing system / Slack escalation channel
            self._escalate_to_human(row)
        elif score <= self.AUTO_APPROVE_SCORE_MAX:
            logger.info(
                "TUNING_AUTO_APPROVE name=%s type=%s score=%d",
                row.name, row.type, score
            )
            # Low-risk: the platform will apply; just log the acknowledgment
        else:
            logger.info(
                "TUNING_MONITORED_APPLY name=%s type=%s score=%d "
                "- enhanced monitoring window started",
                row.name, row.type, score
            )

def _escalate_to_human(self, recommendation):

```

```
# Implementation depends on organization's ticketing / alerting stack
logger.critical(
    "ESCALATE name=%s script=%s",
    recommendation.name,
    recommendation.action_script
)
```

The elastic pool right-sizing loop addresses a common Azure SQL anti-pattern: workloads are placed in an elastic pool sized for peak capacity, but individual databases rarely peak simultaneously. The autonomous tier manager monitors per-database DTU/vCore consumption against pool capacity on a rolling hourly basis and emits resize recommendations when sustained under- or over-utilization is detected:

```
def evaluate_elastic_pool_sizing(
    pool_stats: list[dict],
    pool_max_edtus: int,
    observation_hours: int = 24
) -> str:
    """
    Given per-database eDTU consumption over the observation window,
    return a sizing recommendation: 'DOWNSIZE', 'UPSIZE', or 'OPTIMAL'.
    """
    import numpy as np

    total_consumed = [row['total_edtu'] for row in pool_stats]
    p95_consumed = np.percentile(total_consumed, 95)
    p50_consumed = np.percentile(total_consumed, 50)

    utilization_p95 = p95_consumed / pool_max_edtus

    if utilization_p95 > 0.85:
        return 'UPSIZE'
    if utilization_p95 < 0.30 and p50_consumed / pool_max_edtus < 0.20:
        return 'DOWNSIZE'
    return 'OPTIMAL'
```

The cloud autonomy layer also manages geo-redundancy configuration: when disaster recovery RPO requirements tighten (detectable through application-layer SLA contract metadata stored in a configuration database), the controller enables active geo-replication or adjusts the failover group policy through the ARM API rather than waiting for a manual change request to work through an operations queue.

Integrating SQL Server Autonomy Into a Unified Control Plane

The subparts above describe individual autonomous subsystems — wait stats collection, Query Store governance, AG management, index maintenance, Resource Governor tuning, and Azure SQL supervision. In production, these must compose into a single control plane rather than operating as independent scripts that might issue conflicting actions.

The unified SQL Server autonomy controller implements a priority hierarchy and mutual exclusion between action types:

```

from enum import IntEnum
import threading
import logging
from datetime import datetime, timedelta
from typing import Callable, Optional

logger = logging.getLogger('autonomy_controller')

class ActionPriority(IntEnum):
    CRITICAL    = 1    # Failover, primary-unreachable response
    HIGH        = 2    # Plan regression force, memory pressure relief
    MEDIUM      = 3    # Index rebuild, stats update, replica mode change
    LOW         = 4    # Elastic pool resize recommendation, audit logging

class AutonomyAction:
    def __init__(self, name: str, priority: ActionPriority,
                  fn: Callable, cooldown_minutes: int = 10):
        self.name          = name
        self.priority       = priority
        self.fn             = fn
        self.cooldown_minutes = cooldown_minutes
        self._last_executed: Optional[datetime] = None
        self._lock          = threading.Lock()

    def is_cooled_down(self) -> bool:
        if self._last_executed is None:
            return True
        return (datetime.utcnow() - self._last_executed
                > timedelta(minutes=self.cooldown_minutes))

    def execute(self):
        with self._lock:
            if not self.is_cooled_down():
                logger.debug(
                    "ACTION_SUPPRESSED name=%s (cooldown active)", self.name
                )
                return
            try:
                logger.info("ACTION_START name=%s priority=%s",
                            self.name, self.priority.name)
                self.fn()
                self._last_executed = datetime.utcnow()
                logger.info("ACTION_COMPLETE name=%s", self.name)
            except Exception as e:
                logger.error("ACTION_FAILED name=%s error=%s", self.name, e)

class SQLServerAutonomyController:
    """
    Unified control plane that composes all SQL Server autonomous subsystems
    and enforces priority ordering and mutual exclusion between action types.
    """

```

```

def __init__(self):
    self._actions: list[AutonomyAction] = []
    self._critical_lock = threading.Lock()
    self._maintenance_active = False

def register(self, action: AutonomyAction):
    self._actions.append(action)
    # Keep sorted by priority so highest-priority actions run first
    self._actions.sort(key=lambda a: a.priority)

def run_evaluation_cycle(self, triggered_action_names: list[str]):
    """
    Run a single evaluation cycle for the named actions.
    CRITICAL actions acquire the critical lock, blocking all others
    until the critical action completes.
    """
    to_run = [
        a for a in self._actions
        if a.name in triggered_action_names
    ]

    critical = [a for a in to_run if a.priority == ActionPriority.CRITICAL]
    non_critical = [a for a in to_run if a.priority != ActionPriority.CRITICAL]

    if critical:
        with self._critical_lock:
            for action in critical:
                action.execute()
            # After a critical action, suppress non-critical for safety
            logger.info(
                "POST_CRITICAL_SUPPRESS suppressed=%d actions",
                len(non_critical)
            )
        return

    # Non-critical actions run sequentially in priority order
    # Skip maintenance-class actions if a maintenance window is not active
    for action in non_critical:
        if action.priority == ActionPriority.MEDIUM:
            if not self._maintenance_active:
                logger.debug(
                    "ACTION_DEFERRED name=%s (outside maintenance window)",
                    action.name
                )
                continue
            action.execute()

def set_maintenance_window(self, active: bool):
    self._maintenance_active = active
    logger.info("MAINTENANCE_WINDOW active=%s", active)

```

The evaluation cycle is driven by the sensor network. Each sensor posts named events to an internal queue when it detects a threshold crossing; the controller dequeues these events and maps them to registered action names. This event-driven architecture means the controller does not run a fixed-interval polling loop for every subsystem — actions are triggered by signals, not by clocks, which reduces unnecessary load and makes the decision chain auditable: every action execution has a traceable triggering event in the log.

```
graph TD
    S[Sensor Network: DMVs, Query Store, AG state] --> Q[Event Queue]
    Q --> C{Action Priority}
    C -->|CRITICAL| CL[Critical Lock: exclusive execution]
    C -->|HIGH| H[Execute Immediately]
    C -->|MEDIUM| M{Maintenance Window Active?}
    C -->|LOW| L[Execute Immediately]
    CL --> SUP[Suppress Non-Critical, Then Execute]
    M -->|Yes| MX[Execute]
    M -->|No| DEF[Defer to Next Window]
    SUP --> AUD[(dba.autonomy_audit)]
    H --> AUD
    MX --> AUD
    L --> AUD
```

The unified control plane's priority routing: CRITICAL actions acquire an exclusive lock and suppress everything else, MEDIUM actions defer outside maintenance windows, and every executed action lands in the same audit table.

The final integration step is the audit and observability layer. Every action execution — whether it forces a Query Store plan, adjusts a Resource Governor pool, or initiates an AG failover — writes a structured record to the `dba.autonomy_audit` table:

```
-- SQL Server: autonomy audit table
CREATE TABLE dba.autonomy_audit (
    audit_id          BIGINT IDENTITY PRIMARY KEY,
    action_name       NVARCHAR(100)    NOT NULL,
    priority          NVARCHAR(20)     NOT NULL,
    triggering_signal NVARCHAR(500)    NOT NULL,
    action_sql        NVARCHAR(MAX),
    outcome           NVARCHAR(20)     NOT NULL, -- 'SUCCESS' | 'FAILED' |
'SUPPRESSED'
    error_detail      NVARCHAR(MAX),
    dry_run           BIT              NOT NULL DEFAULT 0,
    executed_at       DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    duration_ms       INT,
    rolled_back_at    DATETIME2,
    rollback_reason   NVARCHAR(500)
);

CREATE INDEX ix_autonomy_audit_executed_at
ON dba.autonomy_audit (executed_at DESC)
INCLUDE (action_name, outcome);
```

This audit table serves multiple purposes beyond debugging. It is the evidence layer for change management processes that require documentation of every database configuration change. It is the training data for future ML-based decision models that replace threshold-based rules with learned behavior. And it is the rollback reference: when a forced plan or Resource Governor adjustment needs to be reverted, the audit record contains the exact SQL to reverse it.

Key Takeaways

- → **DMVs are sensors, not reports.** The autonomous SQL Server architecture treats Dynamic Management Views as a real-time sensor network, collecting differential deltas at sub-minute intervals and feeding them into time-series baselines rather than instantaneous thresholds. Workload-aware, hour-of-week percentile baselines replace static alert limits and dramatically reduce false-positive-driven action storms.
- → **Query Store closes the plan governance loop.** Autonomous plan regression detection, validation, and forcing transforms Query Store from a passive diagnostic tool into an active decision surface. The bidirectional design — forcing regressions and unforcing stale forced plans — is what separates genuine autonomy from one-directional manual interventions accumulated over time.

- → **Always On automation requires application-layer arbitration above the cluster.** WSFC and Pacemaker handle quorum-level failover, but the ADE layer adds replica lag scoring, proactive synchronous-to-asynchronous mode transitions under redo queue pressure, and intelligent failover target selection — actions the cluster layer cannot make without workload context.
- → **Index and statistics maintenance become condition-driven, not schedule-driven.** Replacing fixed maintenance windows with continuous fragmentation and modification-counter assessment, executed within time-budgeted windows that yield gracefully before closing, ensures maintenance work is proportional to actual need rather than calendar position.
- → **A unified control plane with priority ordering and mutual exclusion is non-negotiable.** Individual autonomous subsystems operating without coordination can issue conflicting actions — a Resource Governor change during an AG failover, or an index rebuild during peak memory pressure. The `SQLServerAutonomyController` enforces a strict priority hierarchy, suppresses non-critical actions in the wake of critical events, and writes every decision to an audit table that serves as both the change management record and the training corpus for future decision model improvements.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 34: Observability for Autonomous Databases

Autonomous systems that act without human intervention are only as trustworthy as the signals they observe. A self-managing database can tune indexes, reroute connections, and promote replicas — but if its perception of system state is incomplete, delayed, or structurally wrong, every downstream decision compounds the error. This chapter is not about dashboards or alerting thresholds for on-call engineers. It is about building an observability layer that feeds autonomous decision engines: a nervous system that the database uses to know itself, reason about causality, and act with confidence. The architecture here draws from production patterns in cloud-native databases and translates them into SQL Server deployments that can be built today.

Signal Architecture: What Autonomous Databases Must See

Most observability stacks are built for humans — colorful dashboards, Slack notifications, runbook links in alert bodies. Autonomous systems need something architecturally different: a structured, machine-readable signal layer where every metric carries enough context for a decision engine to act without asking a human what it means.

The distinction matters immediately at the signal taxonomy level. Autonomous observability requires three categories of signals, each serving a different function in the decision loop:

State signals describe what is true right now — active sessions, lock counts, replication lag, buffer hit ratios, index bloat percentages. These are point-in-time facts.

Trend signals describe rate of change — query latency increasing at 12ms/minute, index fragmentation growth exceeding the rebuild threshold velocity, connection pool saturation trending toward hard limits. These enable predictive action before a threshold is breached.

Causal signals describe why something is happening — a lock wait chain rooted in a long-running transaction, a bloat spike correlated with a batch job that started at 02:00, a replication lag spike coinciding with a transaction log backup that contains a large DDL operation. These enable targeted remediation rather than blunt interventions.

A decision engine fed only state signals will be reactive and frequently wrong. Fed trend and causal signals, it can act preemptively and precisely.

```

-- Structured blocking signal for autonomous consumption
CREATE OR ALTER VIEW autonomous.blocking_signal AS
SELECT
    r.session_id                AS blocked_session,
    r.blocking_session_id       AS blocking_session,
    r.wait_type,
    r.wait_time / 1000          AS wait_seconds,
    r.cpu_time,
    r.logical_reads,
    s_blocked.login_name        AS blocked_login,
    s_blocked.program_name      AS blocked_app,
    s_blocking.login_name       AS blocking_login,
    s_blocking.program_name     AS blocking_app,
    s_blocking.status           AS blocking_status,
    DATEDIFF(SECOND, s_blocking.last_request_start_time, GETDATE()) AS
blocker_age_seconds,
    t_blocked.text              AS blocked_query,
    t_blocking.text             AS blocking_query
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s_blocked ON r.session_id = s_blocked.session_id
JOIN sys.dm_exec_sessions s_blocking ON r.blocking_session_id =
s_blocking.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t_blocked
CROSS APPLY sys.dm_exec_sql_text(
    (SELECT sql_handle FROM sys.dm_exec_requests
     WHERE session_id = r.blocking_session_id)) t_blocking
WHERE r.blocking_session_id > 0;

-- Connection trend signal
CREATE OR ALTER VIEW autonomous.connection_trend AS
SELECT
    GETDATE()                AS ts,
    COUNT(*)                  AS active_sessions,
    SUM(CASE WHEN status = 'running' THEN 1 ELSE 0 END) AS running,
    SUM(CASE WHEN status = 'sleeping'
        AND open_transaction_count > 0 THEN 1 ELSE 0 END) AS idle_in_txn,
    SUM(CASE WHEN wait_type LIKE 'LCK%' THEN 1 ELSE 0 END) AS lock_waiters
FROM sys.dm_exec_sessions
WHERE is_user_process = 1;

```

The structural requirement for autonomous consumption is that these views must be queryable by a decision engine on a fixed polling interval — typically 15 to 30 seconds — and each row must carry enough information for a rule engine or ML model to classify the situation and select a response action without joining additional tables at decision time.

Signal freshness is non-negotiable. A decision engine that acts on 10-minute-old data in a system experiencing a lock cascade or connection storm will always act too late. The observability pipeline must guarantee that the latency from event occurrence to signal availability does not exceed two polling cycles.

Telemetry Pipeline Architecture: From Database to Decision Engine

Raw signals in database views are the beginning of the observability pipeline, not the end. The autonomous decision engine needs a structured telemetry pipeline that ingests signals, normalizes them, stores them for trend analysis, and exposes them to the reasoning layer. Building this pipeline requires deliberate architectural choices about collection frequency, storage granularity, and the interface between the telemetry store and the decision engine.

The reference architecture pairs a collector agent — a lightweight Python service polling DMVs and extended events, or a Prometheus SQL Server exporter — with a time-series backend (Prometheus, InfluxDB, or Azure Monitor) and a rule engine that subscribes to derived metrics. For SQL Server on Azure, Azure Monitor with custom metrics and Log Analytics workspaces serve the same function, though with higher latency that must be accounted for in decision logic.

```
graph LR
    DB[(Database: DMVs / Extended Events)] --> COL[Telemetry Collector]
    COL --> CLS{Classify: state / trend / causal}
    CLS --> AD[Contextual Anomaly Detector]
    AD --> DE[Decision Engine]
    DE --> ACT[Action Executor]
    ACT --> DB
    ACT --> LOG[(Action Audit Log)]
    LOG -->|weekly retrospective| DE
```

Signals flow from the database through classification and anomaly detection before reaching the decision engine; every resulting action writes back to both the database and the audit log, which itself feeds the next round of calibration.

```

# Autonomous telemetry collector – SQL Server
# Runs as a scheduled task or container sidecar every 15 seconds
# Pushes structured signals to the decision engine API

import pyodbc
import requests
import time
from datetime import datetime, timezone
from dataclasses import dataclass, asdict
from typing import Optional

@dataclass
class DatabaseSignalPacket:
    """Structured signal packet consumed by the decision engine."""
    timestamp: str
    instance_id: str
    signal_class: str          # 'state' | 'trend' | 'causal'
    signal_name: str
    value: float
    severity: str              # 'nominal' | 'warning' | 'critical' | 'emergency'
    context: dict              # causal metadata – blocking spids, query hashes,
    etc.
    suggested_action: Optional[str] = None # pre-classified by collector

THRESHOLDS = {
    "connection_pct": {"warning": 70, "critical": 85, "emergency": 95},
    "replication_lag_bytes": {"warning": 50_000_000, "critical": 200_000_000,
    "emergency": 1_000_000_000},
    "blocking_chain_depth": {"warning": 2, "critical": 4, "emergency": 8},
    "sleeping_txn_seconds": {"warning": 60, "critical": 300, "emergency": 900},
}

def classify_severity(signal_name: str, value: float) -> str:
    thresholds = THRESHOLDS.get(signal_name)
    if not thresholds:
        return "nominal"
    if value >= thresholds["emergency"]:
        return "emergency"
    elif value >= thresholds["critical"]:
        return "critical"
    elif value >= thresholds["warning"]:
        return "warning"
    return "nominal"

def collect_signals(conn, instance_id: str, max_user_connections: int) ->
list[DatabaseSignalPacket]:
    packets = []
    ts = datetime.now(timezone.utc).isoformat()
    cur = conn.cursor()

    # Connection saturation
    cur.execute("""
        SELECT COUNT(*) FROM sys.dm_exec_connections

```

```

"""
conn_count = cur.fetchone()[0]
conn_pct = conn_count * 100.0 / max_user_connections

cur.execute("""
    SELECT COUNT(*)
    FROM sys.dm_exec_sessions s
    JOIN sys.dm_tran_session_transactions t
    ON s.session_id = t.session_id
    WHERE s.status = 'sleeping'
    AND s.is_user_process = 1
""")
sleeping_txn = cur.fetchone()[0]

cur.execute("""
    WITH chain AS (
        SELECT session_id, blocking_session_id, 1 AS blocking_depth
        FROM sys.dm_exec_requests
        WHERE blocking_session_id > 0
        UNION ALL
        SELECT r.session_id, r.blocking_session_id, c.blocking_depth + 1
        FROM sys.dm_exec_requests r
        JOIN chain c ON r.session_id = c.blocking_session_id
    )
    SELECT MAX(blocking_depth) FROM chain
""")
row = cur.fetchone()
block_depth = row[0] if row and row[0] else 0

packets.append(DatabaseSignalPacket(
    timestamp=ts,
    instance_id=instance_id,
    signal_class="state",
    signal_name="connection_pct",
    value=float(conn_pct),
    severity=classify_severity("connection_pct", float(conn_pct)),
    context={"sleeping_txn": sleeping_txn, "blocking_chain_depth": block_depth},
    suggested_action="scale_connection_pool" if conn_pct > 85 else None
))

# AG redo queue lag – primary only
try:
    cur.execute("""
        SELECT ar.replica_server_name,
               SUM(drs.redo_queue_size) AS redo_lag_bytes
        FROM sys.dm_hadr_database_replica_states drs
        JOIN sys.availability_replicas ar
        ON drs.replica_id = ar.replica_id
        GROUP BY ar.replica_server_name
    """)
    for replica, redo_lag in cur.fetchall():
        if redo_lag is not None:
            packets.append(DatabaseSignalPacket(

```

```

        timestamp=ts,
        instance_id=instance_id,
        signal_class="state",
        signal_name="replication_lag_bytes",
        value=float(redo_lag),
        severity=classify_severity("replication_lag_bytes",
float(redo_lag)),
        context={"replica": replica},
        suggested_action="investigate_replica_throughput" if redo_lag >
200_000_000 else None
    ))
except pyodbc.Error:
    pass # not in an AG – skip

# Sleeping sessions holding open transactions – causal signal
cur.execute("""
    SELECT TOP 20 s.session_id, s.login_name, s.program_name,
        DATEDIFF(SECOND, t.transaction_begin_time, SYSUTCDATETIME()) AS
txn_age_sec,
        SUBSTRING(st.text, 1, 200)
    FROM sys.dm_exec_sessions s
    JOIN sys.dm_tran_session_transactions t
        ON s.session_id = t.session_id
    LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
    OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) st
    WHERE s.status = 'sleeping'
        AND s.is_user_process = 1
    ORDER BY txn_age_sec DESC
""")
for row in cur.fetchall():
    spid, login, app, seconds, sql_text = row
    packets.append(DatabaseSignalPacket(
        timestamp=ts,
        instance_id=instance_id,
        signal_class="causal",
        signal_name="sleeping_txn_seconds",
        value=float(seconds),
        severity=classify_severity("sleeping_txn_seconds", float(seconds)),
        context={"spid": spid, "login": login, "app": app,
            "sql": (sql_text or "")[:500]},
        suggested_action="kill_sleeping_transaction" if seconds > 120 else None
    ))

return packets

def push_to_decision_engine(packets: list[DatabaseSignalPacket], endpoint: str):
    payload = [asdict(p) for p in packets]
    response = requests.post(
        f"{endpoint}/api/v1/signals",
        json=payload,
        headers={"Content-Type": "application/json"},
        timeout=5
    )

```

```

response.raise_for_status()

def run_collector(conn_str: str, instance_id: str, decision_engine_url: str,
                  max_user_connections: int, interval_seconds: int = 15):
    conn = pyodbc.connect(conn_str, autocommit=True)
    print(f"[collector] started for instance={instance_id},
interval={interval_seconds}s")

    while True:
        try:
            packets = collect_signals(conn, instance_id, max_user_connections)
            critical = [p for p in packets if p.severity in ("critical",
"emergency")]
            push_to_decision_engine(packets, decision_engine_url)
            if critical:
                print(f"[collector] {len(critical)} critical signals pushed at
{datetime.now().isoformat()}")
            except pyodbc.Error:
                conn = pyodbc.connect(conn_str, autocommit=True) # reconnect on
connection loss
            except Exception as e:
                print(f"[collector] error: {e}")
                time.sleep(interval_seconds)

```

This collector runs as a Kubernetes sidecar alongside the database pod. The `suggested_action` field is pre-populated by the collector based on simple threshold logic, but the decision engine is not obligated to follow it — it cross-references the suggestion against the full signal history, current system load, active maintenance windows, and operator-defined constraint policies before acting.

The telemetry pipeline must handle backpressure gracefully. If the decision engine endpoint is temporarily unavailable, the collector must buffer locally — typically in a SQLite file or local Redis instance — rather than drop signals. Dropping signals during an incident is precisely when the autonomous system most needs complete data.

For Azure-hosted environments, the same collector can additionally publish to Azure Monitor via its REST API or to an Event Hub that the decision engine consumes as a stream, giving the decision engine both the low-latency local path and the durable cloud path.

Distributed Tracing and Query Fingerprinting for Autonomous Root Cause Analysis

State metrics tell the decision engine that something is wrong. Distributed tracing and query fingerprinting tell it what caused the problem and whether a similar pattern has been seen before. This is the layer that transforms an autonomous system from a reactive threshold responder into a system capable of true root-cause-driven remediation.

Query fingerprinting normalizes SQL text by replacing literals with placeholders, producing a stable identifier that survives parameter changes. SQL Server's Query Store does this natively via `query_hash` and `query_plan_hash`. The fingerprint becomes the key for correlating current symptoms with historical remediation outcomes — if fingerprint `0x8f3a2c` caused a sequential scan storm last Tuesday and the fix was to create an index on `(tenant_id, created_at)`, the decision engine should recognize that fingerprint immediately the next time it surfaces.

```

-- Query regression detection against stored baselines using Query Store
WITH current_stats AS (
    SELECT
        q.query_id,
        qt.query_sql_text,
        rs.count_executions,
        rs.avg_duration / 1000.0 AS avg_ms,
        rs.avg_logical_io_reads,
        rs.avg_query_max_used_memory,
        p.plan_id,
        p.query_plan_hash
    FROM sys.query_store_query q
    JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
    JOIN sys.query_store_plan p ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval rsi
        ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
    WHERE rsi.start_time > DATEADD(MINUTE, -30, GETDATE())
    AND rs.count_executions > 50
),
baseline AS (
    SELECT query_id, baseline_avg_ms, baseline_avg_logical_reads
    FROM autonomous.query_baselines -- custom table, populated during stable window
)
SELECT
    c.query_id,
    LEFT(c.query_sql_text, 120) AS query_sample,
    c.count_executions,
    ROUND(c.avg_ms, 2) AS avg_ms,
    ROUND(b.baseline_avg_ms, 2) AS baseline_ms,
    ROUND((c.avg_ms - b.baseline_avg_ms)
        / NULLIF(b.baseline_avg_ms, 0) * 100, 1) AS pct_regression,
    c.avg_logical_io_reads,
    b.baseline_avg_logical_reads,
    c.plan_id,
    c.query_plan_hash,
    CASE
        WHEN c.avg_ms > b.baseline_avg_ms * 3 THEN 'critical_regression'
        WHEN c.avg_ms > b.baseline_avg_ms * 1.5 THEN 'moderate_regression'
        WHEN c.avg_logical_io_reads >
            b.baseline_avg_logical_reads * 2 THEN 'io_regression'
        ELSE 'nominal'
    END AS classification
FROM current_stats c
JOIN baseline b ON c.query_id = b.query_id
WHERE c.avg_ms > b.baseline_avg_ms * 1.3
ORDER BY pct_regression DESC;

```

Beyond query fingerprinting, distributed tracing at the application layer — using OpenTelemetry with a SQL Server instrumented driver — provides the call chain context that database-layer metrics cannot. When the autonomous system sees that a specific

microservice (identified by its service name in the trace span) is responsible for 80% of the slow query load, it can route the remediation not at the database level but at the connection pool level: reducing the maximum connections allocated to that service in the application connection pool, or applying a Resource Governor workload group timeout.

This is the architectural distinction between observability for humans and observability for autonomous systems. A human reads the trace and decides what to do. The autonomous system is designed so that the trace span metadata is already structured as an input feature to the decision classifier — `service_name`, `query_fingerprint`, `p99_latency`, `error_rate`, `calls_per_second` — and the classifier outputs an action label with a confidence score.

Prometheus, InfluxDB, and the Autonomous Metrics Store

The choice of metrics backend is an architectural decision with significant downstream consequences for the autonomous decision engine. The engine needs to answer questions like: "Has replication lag been increasing monotonically for the past 20 minutes?" and "What was the p99 query latency for fingerprint `0x9a3c` during the last three high-traffic windows?" These are time-series range queries, not point-in-time lookups, and the backend must serve them efficiently at sub-second latency because the decision loop runs on a tight cycle.

Prometheus with `remote_write` to InfluxDB is the standard architecture for autonomous SQL Server deployments at scale. Prometheus scrapes the SQL Server exporter every 15 seconds and handles the collection and alerting layer. InfluxDB stores the long-term history with downsampling tasks that pre-compute the trend metrics the decision engine needs. The alerting layer — Prometheus Alertmanager — serves a different purpose here than in human-operated systems: rather than notifying on-call engineers, it fires webhooks directly to the autonomous decision engine's action API.

```

# prometheus/rules/autonomous_db.yml
# These rules feed the autonomous decision engine, not human pagers
groups:
- name: autonomous_sqlserver
  interval: 15s
  rules:
    # Trend rule: AG redo queue increasing monotonically
    - alert: ReplicationLagIncreasing
      expr: |
        deriv(mssql_redo_queue_bytes[5m]) > 0
        AND mssql_redo_queue_bytes > 10000000
      for: 3m
      labels:
        severity: warning
        action_target: decision_engine
      annotations:
        summary: "Redo queue trending upward on {{ $labels.instance }}"
        replica: "{{ $labels.replica_server_name }}"
        current_lag_bytes: "{{ $value }}"
        signal_class: trend

    # State rule: connection saturation
    - alert: ConnectionPoolNearCapacity
      expr: |
        (mssql_connections_user / mssql_config_max_user_connections) * 100 > 85
      for: 1m
      labels:
        severity: critical
        action_target: decision_engine
      annotations:
        summary: "Connection count above 85% on {{ $labels.instance }}"
        signal_class: state

    # Causal rule: sleeping transactions with open tran accumulating
    - alert: SleepingTransactionsAccumulating
      expr: |
        increase(mssql_sleeping_open_transactions_count[5m]) > 5
      for: 2m
      labels:
        severity: warning
        action_target: decision_engine
      annotations:
        summary: "Sleeping sessions with open transactions accumulating on
        {{ $labels.instance }}"
        signal_class: causal

    # Trend rule: index fragmentation rising above maintenance threshold
    - alert: IndexFragmentationRising
      expr: |
        mssql_index_fragmentation_pct > 30
        AND mssql_index_page_count > 1000
      for: 10m
      labels:

```

```
severity: warning
action_target: decision_engine
annotations:
  summary: "Fragmentation above 30% on {{ $labels.instance }}.
{{ $labels.index_name }}"
  signal_class: trend
```

For the metrics store, a downsampling layer is what makes sub-second trend queries feasible. Rather than scanning millions of raw data points, the decision engine reads pre-aggregated five-minute windows:

```

-- SQL Server: downsampled trend table for the decision engine
-- A SQL Agent job runs this every 5 minutes; the engine reads metrics_5min,
-- never the raw signal store.
CREATE TABLE autonomous.metrics_5min (
    bucket          DATETIME2(0) NOT NULL,
    instance_id     NVARCHAR(128) NOT NULL,
    signal_name     NVARCHAR(128) NOT NULL,
    avg_value       FLOAT          NOT NULL,
    max_value       FLOAT          NOT NULL,
    min_value       FLOAT          NOT NULL,
    p95_value       FLOAT          NOT NULL,
    sample_count    INT            NOT NULL,
    CONSTRAINT PK_metrics_5min PRIMARY KEY (bucket, instance_id, signal_name)
);

-- Downsampling procedure (scheduled every 5 minutes)
CREATE OR ALTER PROCEDURE autonomous.usp_rollup_signals_5min
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @bucket DATETIME2(0) =
        DATEADD(MINUTE, -(DATEPART(MINUTE, SYSUTCDATETIME()) % 5),
        SYSUTCDATETIME());

    WITH ranked AS (
        SELECT
            instance_id,
            signal_name,
            value,
            PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY value)
                OVER (PARTITION BY instance_id, signal_name) AS p95_value
        FROM autonomous.signal_store
        WHERE ts >= DATEADD(MINUTE, -5, @bucket)
            AND ts < @bucket
    )
    INSERT INTO autonomous.metrics_5min
        (bucket, instance_id, signal_name, avg_value, max_value,
        min_value, p95_value, sample_count)
    SELECT
        @bucket,
        instance_id,
        signal_name,
        AVG(value),
        MAX(value),
        MIN(value),
        MAX(p95_value),
        COUNT(*)
    FROM ranked
    GROUP BY instance_id, signal_name;
END;

```

The `is_trending_up` function illustrates the architectural principle: the decision engine must not re-implement statistical reasoning inline every time it needs to answer a trend question. That logic lives in the observability layer and is callable as a function. The decision engine calls `autonomous.is_trending_up('prod-sql-01', 'redo_queue_kb')` and receives a boolean with supporting evidence. This keeps the decision engine's logic clean and testable.

For Azure-hosted environments, the same architecture uses Azure Monitor Metrics with a custom metrics namespace, supplemented by a local InfluxDB instance for low-latency trend queries that cannot tolerate the latency of a remote Azure Monitor API call in the middle of a decision loop. The DMV polling agent writes to both stores: Azure Monitor for durable long-term history and compliance, and InfluxDB for the sub-second trend queries the autonomous engine needs.

Anomaly Detection: Teaching the System to Recognize Its Own Normal

Threshold-based alerting is insufficient for autonomous systems for a fundamental reason: it cannot distinguish between a metric value that is genuinely anomalous and one that is expected given current context. A connection count of 950 out of 1000 is an emergency at 02:00 on a Tuesday but may be normal during a scheduled end-of-month batch run. An autonomous system that pages itself every month-end will either stop acting on that alert or will thrash through unnecessary remediation steps.

The solution is contextual anomaly detection — building a model of what normal looks like for each signal conditioned on time of day, day of week, and covarying signals (like active batch jobs or deployment flags). The simplest production-viable approach is a seasonal decomposition model per signal per instance, updated incrementally as new data arrives.

```

# Seasonal anomaly detection for autonomous database observability
# Uses additive decomposition: observed = trend + seasonal + residual
# Residual z-score triggers anomaly classification

import numpy as np
from dataclasses import dataclass
from typing import Optional

@dataclass
class AnomalyResult:
    signal_name: str
    instance_id: str
    observed_value: float
    expected_value: float
    residual_z_score: float
    is_anomaly: bool
    anomaly_direction: Optional[str] # 'high' | 'low' | None
    context_window: str # e.g. 'weekday_morning_peak'

class SeasonalAnomalyDetector:
    """
    Lightweight seasonal anomaly detector suitable for embedding
    in the observability pipeline. Maintains per-signal, per-context-window
    running statistics (mean + variance) updated with Welford's online algorithm.
    No external ML library dependency.
    """

    def __init__(self, z_threshold: float = 3.0):
        self.z_threshold = z_threshold
        # Keyed by (instance_id, signal_name, context_window)
        # Value: {'n': int, 'mean': float, 'M2': float}
        self._stats: dict[tuple, dict] = {}

    @staticmethod
    def _context_window(ts_hour: int, ts_weekday: int) -> str:
        """Map timestamp to a context window label."""
        if ts_weekday >= 5:
            period = "weekend"
        elif 7 <= ts_hour < 10:
            period = "morning_peak"
        elif 10 <= ts_hour < 18:
            period = "business_hours"
        elif 18 <= ts_hour < 22:
            period = "evening"
        else:
            period = "overnight"
        day_type = "weekend" if ts_weekday >= 5 else "weekday"
        return f"{day_type}_{period}"

    def _update(self, key: tuple, value: float):
        """Welford's online algorithm for running mean and variance."""
        if key not in self._stats:

```



```

        self._stats[key] = {'n': 0, 'mean': 0.0, 'M2': 0.0}
    s = self._stats[key]
    s['n'] += 1
    delta = value - s['mean']
    s['mean'] += delta / s['n']
    delta2 = value - s['mean']
    s['M2'] += delta * delta2

def _z_score(self, key: tuple, value: float) -> tuple[float, float]:
    """Returns (z_score, expected_value). Returns (0, value) if insufficient
data."""
    s = self._stats.get(key)
    if s is None or s['n'] < 30:    # need at least 30 samples per context window
        return 0.0, value
    variance = s['M2'] / (s['n'] - 1)
    stddev = np.sqrt(variance) if variance > 0 else 1e-9
    z = (value - s['mean']) / stddev
    return float(z), float(s['mean'])

def observe(
    self,
    instance_id: str,
    signal_name: str,
    value: float,
    ts_hour: int,
    ts_weekday: int
) -> AnomalyResult:
    window = self._context_window(ts_hour, ts_weekday)
    key = (instance_id, signal_name, window)

    z, expected = self._z_score(key, value)
    self._update(key, value)

    is_anomaly = abs(z) > self.z_threshold
    direction = None
    if is_anomaly:
        direction = "high" if z > 0 else "low"

    return AnomalyResult(
        signal_name=signal_name,
        instance_id=instance_id,
        observed_value=value,
        expected_value=round(expected, 4),
        residual_z_score=round(z, 3),
        is_anomaly=is_anomaly,
        anomaly_direction=direction,
        context_window=window
    )

def load_state(self, state: dict):
    """Restore detector state from persistent store (e.g. Redis or pickle)."""
    self._stats = {tuple(k.split('|')): v for k, v in state.items()}

```

```
def dump_state(self) -> dict:
    """Serialize state for persistence across collector restarts."""
    return {'|'.join(k): v for k, v in self._stats.items()}
```

The detector is stateful: it accumulates running statistics per context window and must survive collector restarts without losing its learned baselines. The `dump_state` / `load_state` pattern serializes to Redis or a local file, with the decision engine periodically snapshotting the detector state to the autonomous metadata store.

A signal that crosses a static threshold but is not anomalous in its context window is downgraded from `critical` to `informational` in the signal packet before it reaches the decision engine. Conversely, a signal that has not crossed any static threshold but has a z-score above 3.0 within its context window is elevated to `warning` regardless of absolute value. This means the decision engine acts more on deviation from expected behavior than on absolute thresholds — which is precisely the right behavior for a system managing databases with widely varying load profiles.

Observability for Autonomous Actions: Closing the Loop

The most commonly overlooked dimension of autonomous database observability is observing the autonomous system itself. Every action the system takes — terminating a session, adjusting MAXDOP settings, promoting a replica, draining a connection pool — must be recorded with enough context to answer the forensic questions that will inevitably be asked: What triggered that action? What signals did the system see? What was the state before and after? Was the action effective?

This action audit log is not optional. Without it, the autonomous system is a black box that operators cannot trust or improve. Every action taken by the decision engine must be written to a structured audit store before the action is executed.

```

-- Autonomous action audit store
CREATE TABLE autonomous.action_log (
  action_id          BIGINT IDENTITY(1,1) PRIMARY KEY,
  executed_at        DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
  instance_id        NVARCHAR(128) NOT NULL,
  action_type        NVARCHAR(64) NOT NULL,
  action_params      NVARCHAR(MAX) NOT NULL,    -- JSON
  trigger_signals    NVARCHAR(MAX) NOT NULL,    -- JSON
  pre_state          NVARCHAR(MAX),             -- JSON
  post_state         NVARCHAR(MAX),             -- JSON
  outcome            NVARCHAR(32),
  outcome_detail     NVARCHAR(MAX),
  confidence_score   FLOAT,
  operator_review    NVARCHAR(32),
  rollback_sql       NVARCHAR(MAX)
);

CREATE INDEX IX_action_log_instance_time
  ON autonomous.action_log (instance_id, executed_at DESC);
CREATE INDEX IX_action_log_type_time
  ON autonomous.action_log (action_type, executed_at DESC);

```

The `post_state` column is populated by a deferred job — typically 60 seconds after the action — that re-runs the same state queries used to populate `pre_state` and records whether the condition that triggered the action has resolved. This before-and-after comparison is the feedback signal that the decision engine uses to evaluate its own effectiveness over time. If a particular action type consistently produces `outcome = 'no_effect'` for a particular signal pattern, the decision engine's rule weights for that combination are downgraded.

The `rollback_sql` column deserves emphasis. For every action that modifies database configuration or session state, the decision engine must generate and store the SQL to reverse that action at the time of execution. This is not primarily for automated rollback — though that is supported — but for operator trust. An engineer investigating an incident must be able to look at the action log, see exactly what the autonomous system did, understand why, and if necessary undo it with a single SQL statement. Autonomous systems that cannot be cleanly overridden by operators will not remain in production.

The action log is also the training data set for improving the decision engine. Periodically — typically weekly — the autonomous system runs a retrospective analysis comparing trigger signals, actions taken, and outcomes. Signal patterns that consistently preceded successful outcomes reinforce the corresponding decision rules. Patterns that preceded `no_effect` or `failed` outcomes trigger a review flag in the operator dashboard. Over months of operation, this retrospective loop produces a decision engine that is observably better calibrated to the specific workload patterns of the database it manages.

Key Takeaways

- → **Autonomous observability requires three structurally distinct signal classes** — state, trend, and causal — because a decision engine acting on state signals alone will be consistently reactive and late, while trend and causal signals enable preemptive, targeted action before conditions become critical.
- → **The telemetry pipeline must guarantee signal freshness within two polling cycles** and handle backpressure by buffering locally rather than dropping signals; data loss during an incident is precisely when the autonomous decision engine is most dependent on a complete and continuous signal stream.
- → **Query fingerprinting paired with stored baselines enables regression detection** that survives parameter changes, plan changes, and schema evolution — connecting current query performance anomalies to historical remediation outcomes so the decision engine can recognize recurrent patterns and apply proven fixes rather than re-deriving solutions from scratch.
- → **Contextual anomaly detection using per-signal, per-context-window running statistics** separates genuinely anomalous behavior from expected load variations, preventing the decision engine from acting on high absolute values that are normal given the time of day or concurrent batch activity, and ensuring it does respond to subtle deviations that fall below static thresholds but are statistically significant within their context.
- → **Every autonomous action must be written to a structured audit log with pre-state, post-state, trigger signals, confidence score, and rollback SQL** before execution; this log is simultaneously the trust mechanism for operators, the safety net for overrides, and the training dataset that closes the feedback loop enabling the decision engine to improve its calibration over time.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 35: Autonomous Cost Optimization

Cost is the boundary condition that shapes every other autonomous decision. A system that auto-scales without cost awareness will optimize itself into a cloud bill that ends careers. A system that freezes capacity to control spend will fail SLAs under load. Autonomous cost optimization is the discipline of building control loops that balance performance, availability, and expenditure continuously — without a human approving each tradeoff. This chapter covers the full architecture: how autonomous systems model infrastructure cost in real time, how they make provisioning and rightsizing decisions without human intervention, how query-level spend attribution works at scale, and how to implement feedback loops that keep cost within envelope while preserving SLA guarantees.

35.1 Cost Modeling as a First-Class Metric: Tagging, Attribution, and Real-Time Spend Signals

Before any autonomous system can optimize cost, it needs a continuously updated model of what things cost and who is responsible for that cost. Most teams treat cost as a billing artifact — something discovered monthly when the cloud invoice arrives. Autonomous systems treat cost as a runtime metric, sampled at the same frequency as CPU and I/O.

The foundation is **dimensional cost attribution**: every resource consumption event (query execution, storage write, network egress, index maintenance job) is tagged with an ownership dimension before it hits the billing system. In practice, this means building a cost metadata layer that maps database objects to cost centers.

```

-- Cost attribution registry
CREATE TABLE CostAttribution.Ownership (
    OwnershipID          INT IDENTITY(1,1) PRIMARY KEY,
    SchemaName           NVARCHAR(128) NOT NULL,
    ApplicationTag       NVARCHAR(256),
    CostCenter           NVARCHAR(128) NOT NULL,
    TeamName             NVARCHAR(128) NOT NULL,
    MonthlyBudgetUSD     DECIMAL(12,2),
    AlertThresholdPct    DECIMAL(5,2) DEFAULT 80.0,
    CreatedAt            DATETIME2     DEFAULT SYSUTCDATETIME(),
    UpdatedAt            DATETIME2     DEFAULT SYSUTCDATETIME()
);

-- Real-time query cost estimate from Query Store
CREATE OR ALTER VIEW CostAttribution.QueryCostEstimate AS
SELECT
    qsq.query_id,
    qsq.query_hash,
    SUBSTRING(qt.query_sql_text, 1, 500) AS query_sample,
    ca.CostCenter,
    ca.TeamName,
    qsrs.count_executions,
    qsrs.avg_duration / 1000.0           AS avg_duration_ms,
    qsrs.avg_cpu_time / 1000.0           AS avg_cpu_ms,
    -- Compute cost estimate: $0.048/vCPU-hour
    ROUND(
        (qsrs.avg_cpu_time / 1e6 / 3600.0) -- microseconds → hours
        * qsrs.count_executions
        * 0.048
    , 6) AS estimated_compute_cost_usd,
    qsrs.avg_logical_io_reads,
    -- I/O cost estimate
    ROUND(
        (qsrs.avg_logical_io_reads * 8192.0 / 1e9)
        * qsrs.count_executions
        * 0.10
    , 6) AS estimated_io_cost_usd
FROM sys.query_store_query          qsq
JOIN sys.query_store_query_text    qt  ON qt.query_text_id =
qsq.last_query_text_id
JOIN sys.query_store_runtime_stats qsrs ON qsrs.plan_id IN (
    SELECT plan_id FROM sys.query_store_plan WHERE query_id = qsq.query_id
)
LEFT JOIN CostAttribution.Ownership ca
    ON qsq.object_id IN (
        SELECT object_id FROM sys.objects WHERE schema_id = SCHEMA_ID(ca.SchemaName)
    )
WHERE qsrs.last_execution_time >= DATEADD(HOUR, -1, SYSUTCDATETIME());

```

The real-time signal layer pulls from cloud pricing APIs — AWS Cost Explorer, Azure Cost Management, or GCP Billing — and correlates resource-level line items with database-level metrics. The autonomous system needs this correlation to know that a spike in logical reads on a SQL Server secondary replica translates into a measurable I/O cost increase, not just a performance concern.

The Python daemon below polls cloud cost APIs every 15 minutes and writes normalized cost signals into a time-series store (Prometheus pushgateway or InfluxDB) alongside database metrics:

```

import pyodbc
import requests
import json
from datetime import datetime, timedelta, date
from prometheus_client import CollectorRegistry, Gauge, push_to_gateway

PUSHGATEWAY_URL = "http://prometheus-pushgateway:9091"
DB_CONN = "DRIVER={ODBC Driver 18 for SQL
Server};SERVER=localhost;DATABASE=platform;Trusted_Connection=yes;"
AZURE_SUBSCRIPTION = "00000000-0000-0000-0000-000000000000"
AZURE_TOKEN = None # obtained via DefaultAzureCredential at startup

def fetch_azure_cost(resource_group: str, days: int = 1) -> float:
    """Pull last N days spend for a resource group from Azure Cost Management."""
    end = datetime.utcnow().date()
    start = end - timedelta(days=days)
    url = (
        f"https://management.azure.com/subscriptions/{AZURE_SUBSCRIPTION}"
        "/providers/Microsoft.CostManagement/query?api-version=2023-11-01"
    )
    body = {
        "type": "ActualCost",
        "timeframe": "Custom",
        "timePeriod": {"from": str(start), "to": str(end)},
        "dataset": {
            "granularity": "Daily",
            "aggregation": {"totalCost": {"name": "CostUSD", "function": "Sum"}},
            "grouping": [{"type": "Tag", "name": "CostCenter"}],
            "filter": {
                "dimensions": {
                    "name": "ResourceGroupName",
                    "operator": "In",
                    "values": [resource_group],
                }
            },
        },
    },
}
resp = requests.post(
    url,
    headers={"Authorization": f"Bearer {AZURE_TOKEN}",
            "Content-Type": "application/json"},
    json=body, timeout=30,
)
resp.raise_for_status()
total = 0.0
for row in resp.json()["properties"]["rows"]:
    total += float(row[0])
return total

def push_cost_metrics(cluster_id: str, cost_usd: float, cost_center: str):
    registry = CollectorRegistry()
    g = Gauge(
        "db_cluster_daily_cost_usd",

```



```

        "Estimated daily cost for database cluster",
        ["cluster_id", "cost_center"],
        registry=registry
    )
    g.labels(cluster_id=cluster_id, cost_center=cost_center).set(cost_usd)
    push_to_gateway(PUSHGATEWAY_URL, job="cost_optimizer", registry=registry)

def record_cost_signal(cluster_id: str, cost_usd: float, cost_center: str):
    """Write cost signal to the SQL Server cost tracking table (upsert)."""
    with pyodbc.connect(DB_CONN) as conn:
        cur = conn.cursor()
        cur.execute("""
            MERGE cost_attribution.daily_spend AS tgt
            USING (SELECT ? AS cluster_id, ? AS cost_center,
                         CAST(GETUTCDATE() AS DATE) AS spend_date,
                         ? AS cost_usd) AS src
            ON tgt.cluster_id = src.cluster_id
            AND tgt.cost_center = src.cost_center
            AND tgt.spend_date = src.spend_date
            WHEN MATCHED THEN
                UPDATE SET cost_usd = src.cost_usd, recorded_at = SYSUTCDATETIME()
            WHEN NOT MATCHED THEN
                INSERT (cluster_id, cost_center, spend_date, cost_usd, recorded_at)
                VALUES (src.cluster_id, src.cost_center, src.spend_date,
                        src.cost_usd, SYSUTCDATETIME());
            """, (cluster_id, cost_center, cost_usd))
        conn.commit()

if __name__ == "__main__":
    clusters = [
        {"id": "prod-sql-ag-01", "rg": "rg-prod-sql-01", "cost_center": "platform-
eng"},
        {"id": "prod-sql-ag-02", "rg": "rg-prod-sql-02", "cost_center": "data-
science"},
    ]
    for c in clusters:
        cost = fetch_azure_cost(c["rg"])
        push_cost_metrics(c["id"], cost, c["cost_center"])
        record_cost_signal(c["id"], cost, c["cost_center"])
        print(f"Recorded ${cost:.4f} for {c['id']} ({c['cost_center']})")

```

With cost flowing as a real-time metric, the system now has the signal quality needed to make autonomous decisions — not just report on what was spent.

35.2 Autonomous Rightsizing: Instance, Storage, and Replica Count Control Loops

Rightsizing without autonomy is a quarterly meeting. Rightsizing with autonomy is a continuous control loop that evaluates provisioned capacity against actual utilization and executes changes within pre-approved envelopes. The key architectural distinction is

between **reactive rightsizing** (respond to a threshold breach) and **predictive rightsizing** (anticipate tomorrow's workload shape and pre-position capacity at the right tier before the cost is incurred).

The control loop for autonomous rightsizing operates on three independent axes:

1. **Compute tier** — instance size (vCPU/RAM). Scale up when P95 CPU exceeds 75% for 30 consecutive minutes during business hours. Scale down when P95 CPU stays below 20% for 4 consecutive hours outside a peak window.
2. **Storage tier** — provisioned IOPS versus actual IOPS. On AWS RDS, `gp3` volumes allow independent IOPS and throughput provisioning. When consumed IOPS consistently runs below 40% of provisioned, the system reduces provisioned IOPS to the next tier down.
3. **Replica count** — read replicas are the largest cost lever in most SQL Server architectures. Each read replica is a full instance cost. The system tracks replica load in 5-minute windows and decommissions idle replicas using a golden signal model: if a replica's active connections drop below 5 and its CPU stays under 10% for 20 consecutive minutes, it is a candidate for removal.

```

-- Autonomous rightsizing evaluation for Always On secondaries
CREATE OR ALTER PROCEDURE CostOptimization.EvaluateReplicaRightsizing
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        rm.ReplicaHost,
        CASE
            WHEN rm.AvgActiveConnections < 5
                AND rm.AvgCPUPct < 10
                AND rm.ConsecutiveIdleWindows >= 4
            THEN 'DECOMMISSION'
            WHEN rm.AvgActiveConnections < 10
                AND rm.AvgCPUPct < 25
                AND rm.ConsecutiveIdleWindows >= 12
            THEN 'SCALE_DOWN_TIER'
            ELSE 'NO_ACTION'
        END AS ActionRecommended,
        rm.AvgActiveConnections,
        rm.AvgCPUPct,
        rm.ConsecutiveIdleWindows * 5 AS IdleDurationMin,
        -- SQL Server Business Critical: ~$0.55/vCore-hour × 8 vCores × 730hr
        CASE
            WHEN rm.ConsecutiveIdleWindows >= 4
            THEN CAST(0.55 * 8 * 730 AS DECIMAL(10,2))
            ELSE 0.00
        END AS EstimatedSavingsUSDMonth
    FROM CostOptimization.ReplicaMetricsRollup rm
    WHERE rm.WindowEnd >= DATEADD(HOUR, -1, SYSUTCDATETIME())
    ORDER BY EstimatedSavingsUSDMonth DESC;
END;

```

The action layer for autonomous decommissioning requires three safeguards: a **change freeze window** (no decommissions during peak hours 08:00–22:00 local business timezone), a **quorum check** (never drop below minimum replica count defined in the DR policy, typically 2), and a **cost approval gate** — actions with savings above a threshold require an async approval signal from a human or a CI/CD pipeline before execution. Below the threshold, they execute immediately.

The Python controller that acts on these recommendations connects the evaluation SQL above to the cloud provider API:

```

import pyodbc
import subprocess
from datetime import datetime, time
import pytz

DB_CONN = "DRIVER={ODBC Driver 18 for SQL
Server};SERVER=localhost;DATABASE=platform;Trusted_Connection=yes;"
MIN_REPLICA_COUNT = 2
AUTO_APPROVE_SAVINGS_THRESHOLD = 50.0    # USD/month
PEAK_HOURS_TZ = pytz.timezone("America/New_York")
PEAK_START = time(8, 0)
PEAK_END    = time(22, 0)

def is_peak_window() -> bool:
    now_local = datetime.now(PEAK_HOURS_TZ).time()
    return PEAK_START <= now_local <= PEAK_END

def get_current_replica_count(resource_group: str, server: str, database: str) ->
int:
    # Count geo-secondaries via Azure CLI
    out = subprocess.run([
        "az", "sql", "db", "replica", "list",
        "--resource-group", resource_group,
        "--server", server, "--database", database,
        "--query", "length(@)", "-o", "tsv",
    ], capture_output=True, text=True, check=True)
    return int(out.stdout.strip())

def execute_rightsizing_actions(resource_group: str, server: str, database: str):
    if is_peak_window():
        print("In peak window – skipping autonomous rightsizing.")
        return

    with pyodbc.connect(DB_CONN) as conn:
        cur = conn.cursor()
        cur.execute("{CALL CostOptimization.EvaluateReplicaRightsizing}")
        actions = [
            row for row in cur.fetchall()
            if row.ActionRecommended != 'NO_ACTION'
        ]

    current_replicas = get_current_replica_count(resource_group, server, database)

    for row in actions:
        replica_name, action, savings = (row.ReplicaHost, row.ActionRecommended,
row.EstimatedSavingsUSDMonth)
        if action == "DECOMMISSION":
            if current_replicas <= MIN_REPLICA_COUNT:
                print(f"Skipping {replica_name} – at minimum replica floor.")
                continue
            if savings >= AUTO_APPROVE_SAVINGS_THRESHOLD:
                # Auto-approve: execute immediately
                print(f"Auto-decommissioning {replica_name} (saves ${savings:.2f})/

```

```

mo)")

        subprocess.run([
            "az", "sql", "db", "delete",
            "--resource-group", resource_group,
            "--server", replica_name.split(".")[0],
            "--name", database, "--yes",
        ], check=True)
        current_replicas -= 1
    else:
        # Queue for async human approval
        queue_for_approval(replica_name, action, savings)

def queue_for_approval(replica_name: str, action: str, savings: float):
    """Write to approval queue – picked up by async approval service."""
    with pyodbc.connect(DB_CONN) as conn:
        cur = conn.cursor()
        cur.execute("""
            INSERT INTO cost_optimization.pending_approvals
                (resource_id, action, estimated_savings_usd_mo, requested_at,
status)
            VALUES (?, ?, ?, SYSUTCDATETIME(), 'PENDING')
            """, (replica_name, action, savings))
        conn.commit()

if __name__ == "__main__":
    execute_rightsizing_actions("rg-prod-sql-01", "prod-sql-primary", "OrdersDB")

```

Storage rightsizing follows a similar pattern but operates on provisioned IOPS metrics from CloudWatch or Azure Monitor rather than connection counts. The signal is: when `VolumeReadIOPs + VolumeWriteIOPs` averaged over a 7-day rolling window is below 40% of `ProvisionedIOPS`, reduce provisioned IOPS by one step (typically 1000 IOPS increments on `gp3`). Each reduction step saves approximately \$0.10/hour on AWS.

35.3 Query-Level Cost Governance: Spend Limits, Query Killing, and Workload Shaping

Compute infrastructure rightsizing addresses the supply side of cost. Query-level governance addresses the demand side — it ensures that individual queries or workloads do not consume disproportionate resources relative to their business value. This is where autonomous cost optimization becomes genuinely operational rather than just architectural.

The discipline has three tiers:

Tier 1 — Statement Timeout Enforcement by Cost Center. Each cost center gets a configurable statement timeout. Data science workloads doing exploratory analysis against production replicas are limited to 30-second queries by default. OLTP application roles get 5-

second timeouts. The mapping is maintained in the cost attribution registry and applied dynamically via Resource Governor workload groups or application-tier connection pool configuration.

```

-- Resource Governor workload group mapping for cost governance
-- Each cost center maps to a resource pool with CPU/memory limits
CREATE RESOURCE POOL DataSciencePool
WITH (
    MIN_CPU_PERCENT          = 0,
    MAX_CPU_PERCENT          = 25,    -- Cap at 25% CPU for exploratory workloads
    MIN_MEMORY_PERCENT       = 0,
    MAX_MEMORY_PERCENT       = 20,
    MIN_IOPS_PER_VOLUME      = 0,
    MAX_IOPS_PER_VOLUME      = 500
);

CREATE RESOURCE POOL OLTPPool
WITH (
    MIN_CPU_PERCENT          = 20,
    MAX_CPU_PERCENT          = 80,
    MIN_MEMORY_PERCENT       = 20,
    MAX_MEMORY_PERCENT       = 70
);

-- Classifier function: route connections to pools based on login
CREATE FUNCTION CostOptimization.ClassifyWorkload()
RETURNS SYSNAME
WITH SCHEMABINDING AS
BEGIN
    DECLARE @PoolName SYSNAME = 'default';

    SELECT @PoolName = rcp.ResourcePoolName
    FROM CostOptimization.RoleCostPolicy rcp
    WHERE rcp.LoginName = SUSER_SNAME()
        AND rcp.IsActive = 1;

    RETURN @PoolName;
END;

ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION =
CostOptimization.ClassifyWorkload);
ALTER RESOURCE GOVERNOR RECONFIGURE;

-- Autonomous kill of queries exceeding cost-based time limits
-- Executed by SQL Agent job every 60 seconds
CREATE OR ALTER PROCEDURE CostOptimization.KillOverbudgetQueries
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SessionID      INT;
    DECLARE @LoginName      NVARCHAR(128);
    DECLARE @DurationSec    INT;
    DECLARE @KillSQL        NVARCHAR(64);

    DECLARE kill_cursor CURSOR LOCAL FAST_FORWARD FOR
        SELECT

```

```

        r.session_id,
        s.login_name,
        DATEDIFF(SECOND, r.start_time, SYSUTCDATETIME()) AS duration_sec
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON s.session_id = r.session_id
JOIN CostOptimization.RoleCostPolicy rcp
    ON rcp.LoginName = s.login_name
    AND rcp.IsActive = 1
WHERE DATEDIFF(SECOND, r.start_time, SYSUTCDATETIME())
    > rcp.HardKillTimeoutSec
    AND r.session_id <> @@SPID;

OPEN kill_cursor;
FETCH NEXT FROM kill_cursor INTO @SessionID, @LoginName, @DurationSec;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @KillSQL = N'KILL ' + CAST(@SessionID AS NVARCHAR(10));

    INSERT INTO CostOptimization.KillLog
        (SessionID, LoginName, DurationSec, KilledAt)
    VALUES (@SessionID, @LoginName, @DurationSec, SYSUTCDATETIME());

    EXEC sp_executesql @KillSQL;

    FETCH NEXT FROM kill_cursor INTO @SessionID, @LoginName, @DurationSec;
END;

CLOSE kill_cursor;
DEALLOCATE kill_cursor;
END;

```

Tier 2 — Workload Shaping via Queue Depth Control. Rather than killing queries outright, a more sophisticated approach throttles arrival rate at the connection pooler. When the autonomous system detects that aggregate query cost (estimated compute + I/O) in a rolling 5-minute window is approaching the cost center's budget threshold, it instructs the application-tier connection pool or SQL Server Resource Governor to reduce the effective connection cap for the offending pool. This creates natural back-pressure without hard terminations — callers queue rather than fail, and the system bleeds off the overload gradually.

The key metrics that drive queue depth control are:

- → `active_query_cost_rate_usd_per_min` — sum of estimated cost per minute across all currently running queries for a cost center
- → `budget_burn_rate_pct` — how fast the monthly budget is being consumed versus expected linear burn

- → `queue_depth_ratio` — ratio of waiting connections to active connections; above 3.0 indicates a pooler bottleneck that amplifies cost

Budget burn rate view:

```
CREATE OR ALTER VIEW CostOptimization.BudgetBurnRate AS
WITH DailySpend AS (
    SELECT
        CostCenter,
        SUM(CostUSD) AS MTDSpendUSD
    FROM CostAttribution.DailySpend
    WHERE SpendDate >= DATEFROMPARTS(YEAR(SYSUTCDATETIME()),
    MONTH(SYSUTCDATETIME()), 1)
    GROUP BY CostCenter
),
ExpectedLinear AS (
    SELECT
        o.CostCenter,
        o.MonthlyBudgetUSD,
        -- Fraction of month elapsed
        CAST(DAY(SYSUTCDATETIME()) AS FLOAT)
        / CAST(DAY(EOMONTH(SYSUTCDATETIME())) AS FLOAT) AS MonthFraction
    FROM CostAttribution.Ownership o
)
SELECT
    el.CostCenter,
    el.MonthlyBudgetUSD,
    ISNULL(ds.MTDSpendUSD, 0) AS MTDSpendUSD,
    el.MonthlyBudgetUSD * el.MonthFraction AS ExpectedSpendUSD,
    ROUND(
        ISNULL(ds.MTDSpendUSD, 0)
        / NULLIF(el.MonthlyBudgetUSD * el.MonthFraction, 0)
        * 100
    , 2) AS BurnRatePct,
    CASE
        WHEN ISNULL(ds.MTDSpendUSD, 0)
            > el.MonthlyBudgetUSD * el.MonthFraction * 1.20
        THEN 'THROTTLE'
        WHEN ISNULL(ds.MTDSpendUSD, 0)
            > el.MonthlyBudgetUSD * el.MonthFraction * 1.10
        THEN 'WARN'
        ELSE 'NORMAL'
    END AS BudgetStatus
FROM ExpectedLinear el
LEFT JOIN DailySpend ds ON ds.CostCenter = el.CostCenter;
```

Tier 3 — Query Rewrite and Index Suggestion as a Cost Reduction Path. Before a query is killed or a workload is throttled, the system checks whether the expensive query matches a known rewrite pattern or is missing an index that would reduce its resource footprint by

more than 50%. This is cost-aware query optimization: the autonomous planner asks not just "is this query slow?" but "is this query consuming more budget than its value warrants, and can we fix it without killing it?"

This closes the loop between cost governance and the query optimization subsystems covered in earlier chapters. The cost signal from Query Store feeds back into the index advisor, which prioritizes index candidates by estimated cost reduction rather than purely by execution time improvement.

35.4 Predictive Cost Forecasting: Time-Series Models for Budget Protection

Reactive cost control — responding after a threshold is crossed — is inherently too slow for monthly budget protection. By the time a cost center is 20% over its expected burn rate, the spend has already occurred. Predictive cost forecasting shifts the control point upstream: the autonomous system builds a rolling forecast of end-of-month spend and takes throttling action when the forecast, not the current spend, exceeds the budget.

The forecasting model does not need to be sophisticated. A linear extrapolation with a 7-day trailing average captures most of the signal in workloads that have weekly seasonality (which is the majority of business database workloads). More complex workloads — those with end-of-quarter spikes, monthly batch runs, or irregular marketing campaigns — benefit from a simple additive decomposition: trend + weekly seasonal component + a one-time event flag populated from an engineering calendar.

```

CREATE OR ALTER PROCEDURE CostOptimization.ForecastMonthEndSpend
    @CostCenter NVARCHAR(128)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @DaysInMonth      INT,
        @DaysElapsed      INT,
        @DaysRemaining    INT,
        @MTDSpend          DECIMAL(12,2),
        @DailyAvg7D        DECIMAL(12,2),
        @Forecast          DECIMAL(12,2),
        @Budget            DECIMAL(12,2),
        @DaysToBreach      INT;

    SET @DaysInMonth      = DAY(EOMONTH(SYSUTCDATETIME()));
    SET @DaysElapsed      = DAY(SYSUTCDATETIME());
    SET @DaysRemaining    = @DaysInMonth - @DaysElapsed;

    SELECT @MTDSpend = ISNULL(SUM(CostUSD), 0)
    FROM    CostAttribution.DailySpend
    WHERE   CostCenter = @CostCenter
           AND SpendDate >= DATEFROMPARTS(YEAR(SYSUTCDATETIME()),
                                           MONTH(SYSUTCDATETIME()), 1);

    SELECT @DailyAvg7D = ISNULL(AVG(CostUSD), 0)
    FROM    CostAttribution.DailySpend
    WHERE   CostCenter = @CostCenter
           AND SpendDate >= CAST DATEADD(DAY, -7, SYSUTCDATETIME()) AS DATE
           AND SpendDate <  CAST(SYSUTCDATETIME()) AS DATE);

    SET @Forecast = @MTDSpend + (@DailyAvg7D * @DaysRemaining);

    SELECT @Budget = ISNULL(MonthlyBudgetUSD, 0)
    FROM    CostAttribution.Ownership
    WHERE   CostCenter = @CostCenter;

    SET @DaysToBreach =
        CASE
            WHEN @DailyAvg7D > 0
            THEN CAST(FLOOR((@Budget - @MTDSpend) / @DailyAvg7D) AS INT)
            ELSE @DaysRemaining
        END;

    SELECT
        @CostCenter          AS CostCenter,
        @MTDSpend             AS CurrentMTDSpend,
        @Forecast             AS ForecastMonthEnd,
        @Budget               AS MonthlyBudget,
        CASE WHEN @Forecast > @Budget
            THEN CAST(1 AS BIT) ELSE CAST(0 AS BIT) END AS ForecastOverBudget,
        CASE WHEN @DaysToBreach < 0 THEN 0 ELSE @DaysToBreach END AS

```

```
DaysUntilBreach;  
END;
```

The forecast output feeds a graduated response ladder. When `days_until_breach` is more than 14, the system logs a warning. When it drops below 7, the system automatically tightens statement timeouts by 20% for non-critical roles. When it drops below 3, the system activates hard throttling — reducing `user_connections` for the offending cost center's pool and rejecting new batch workload submissions. This graduated ladder prevents both false alarms (which erode trust in the autonomous system) and budget overruns (which erode organizational confidence in autonomous operation).

The forecast can be enhanced with an **event calendar overlay** — a simple table that records known high-cost events (end-of-quarter reporting runs, data migration windows, marketing campaign activations) with an estimated cost multiplier. On days adjacent to a calendar event, the system widens the forecast confidence band rather than triggering throttling prematurely.

```
-- Event calendar overlay table (SQL Server)  
CREATE TABLE CostOptimization.CostEventCalendar (  
    EventID          INT IDENTITY(1,1) PRIMARY KEY,  
    CostCenter        NVARCHAR(128) NOT NULL,  
    EventName         NVARCHAR(256) NOT NULL,  
    EventDate         DATE NOT NULL,  
    CostMultiplier    DECIMAL(4,2) DEFAULT 1.0,    -- e.g. 2.5 = expect 2.5x normal  
    daily spend  
    Notes             NVARCHAR(MAX),  
    CreatedBy         NVARCHAR(128),  
    CreatedAt         DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()  
);  
  
-- When forecasting, adjust the daily average on event days:  
-- for each remaining day, look up whether it falls on a calendar event  
-- and multiply the 7-day average by the event multiplier.
```

35.5 Multi-Dimensional Cost Tradeoff Engine: Balancing SLA, Availability, and Spend

The hardest problem in autonomous cost optimization is not identifying waste — it is making the correct tradeoff when cost reduction and SLA preservation are in direct conflict. A replica that is idle 90% of the time costs money to run. But that same replica may be the one that absorbs a burst read query from a monthly finance report that arrives at an unpredictable time. Removing it saves cost; keeping it preserves latency headroom. The autonomous system needs a principled framework for resolving this tension without defaulting to the human-in-the-loop.

The framework used here is a **multi-objective scoring function** that assigns a composite score to each potential cost action. The score has three components:

1. **Cost savings weight (W_{cost})** — normalized USD saved per month, scaled 0–1 against the maximum potential saving in the action set.
2. **SLA risk weight (W_{sla})** — estimated probability that this action causes an SLA breach in the next 30 days, scaled 0–1 (higher is riskier).
3. **Availability risk weight (W_{avail})** — estimated probability that removing this resource increases RTO if a primary failure occurs simultaneously, scaled 0–1.

The composite action score is:

$$\text{Action_Score} = W_{\text{cost}} \times \alpha - W_{\text{sla}} \times \beta - W_{\text{avail}} \times \gamma$$

where α , β , and γ are organizational preference weights that encode risk tolerance. A cost-aggressive organization (a startup in growth phase) might set $\alpha=0.7$, $\beta=0.2$, $\gamma=0.1$. A risk-averse organization (financial services) might set $\alpha=0.3$, $\beta=0.4$, $\gamma=0.3$. Actions with positive composite scores are candidates for autonomous execution; negative-scoring actions require human approval regardless of the savings magnitude.

```

CREATE OR ALTER PROCEDURE CostOptimization.ScoreCostActions
    @Alpha DECIMAL(4,2) = 0.5,
    @Beta DECIMAL(4,2) = 0.3,
    @Gamma DECIMAL(4,2) = 0.2
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @MaxSavings DECIMAL(12,2);

    SELECT @MaxSavings = MAX(EstimatedSavingsUSDMonth)
    FROM CostOptimization.ActionCandidates
    WHERE Status = 'PENDING';

    SELECT
        ca.ActionID,
        ca.ResourceID,
        ca.ActionType,
        ca.EstimatedSavingsUSDMonth,
        ca.SLARiskScore,
        ca.AvailabilityRiskScore,
        ROUND(
            (ca.EstimatedSavingsUSDMonth / NULLIF(@MaxSavings, 0)) * @Alpha
            - ca.SLARiskScore * @Beta
            - ca.AvailabilityRiskScore * @Gamma
        , 4) AS CompositeScore,
        CASE
            WHEN (
                (ca.EstimatedSavingsUSDMonth / NULLIF(@MaxSavings, 0)) * @Alpha
                - ca.SLARiskScore * @Beta
                - ca.AvailabilityRiskScore * @Gamma
            ) > 0 THEN CAST(1 AS BIT)
            ELSE CAST(0 AS BIT)
        END AS Recommended
    FROM CostOptimization.ActionCandidates ca
    WHERE ca.Status = 'PENDING'
    ORDER BY CompositeScore DESC;
END;

```

The SLA risk score and availability risk score are not static — they are populated by the system's monitoring layer based on recent behavior. A replica that has served read traffic in the last 48 hours gets a higher SLA risk score for decommissioning than one that has been idle for two weeks. A replica in a region where the primary has shown elevated error rates gets a higher availability risk score for decommissioning than one in a region with a perfect health record.

This self-updating risk model means the scoring function adapts to changing operational context rather than operating on fixed parameters. The weights (α , β , γ) are the only parameters that require human judgment; the input scores are data-driven and continuously refreshed.

35.6 Feedback Loops and Continuous Calibration: Closing the Cost Optimization Cycle

An autonomous cost optimization system that acts without measuring the effect of its actions will drift. Replicas are decommissioned, timeouts are tightened, IOPS are reduced — but without a feedback loop that measures whether these actions achieved their intended savings without degrading performance, the system has no mechanism to learn whether its models are calibrated or whether its thresholds are set appropriately.

The feedback loop operates on a 30-day cycle aligned with the billing period:

Step 1 — Action logging. Every autonomous action is written to an immutable audit log with the predicted outcome (expected savings, estimated SLA impact) recorded at decision time.

Step 2 — Outcome measurement. 7 days after an action, the system queries actual spend and SLA metrics for the affected cost center and compares them against the predictions made at decision time.

Step 3 — Calibration adjustment. If predictions consistently overestimate savings by more than 20%, the cost model's unit cost coefficients (\$/vCPU-hour, \$/GB-IO) are recalibrated using the measured actuals. If SLA risk predictions consistently underestimate breach probability, the β weight in the scoring function is increased automatically by 0.05 per miscalibration event, up to a maximum of 0.6.

```

CREATE OR ALTER PROCEDURE CostOptimization.CalibrateCostModel
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @AvgSavingsError    DECIMAL(8,4),
        @SLAMismatchCount  INT,
        @CalibrationMsg     NVARCHAR(1000) = N'';

    -- Savings prediction accuracy
    SELECT @AvgSavingsError = AVG(
        (ActualSavings7D - PredictedSavings)
        / NULLIF(PredictedSavings, 0) * 100
    )
    FROM CostOptimization.ActionOutcomes
    WHERE MeasuredAt          >= DATEADD(DAY, -30, SYSUTCDATETIME())
        AND CalibrationApplied = 0;

    IF ABS(@AvgSavingsError) > 20
    BEGIN
        UPDATE CostOptimization.CostModelParameters
        SET     CoefficientValue = CoefficientValue * (1 + @AvgSavingsError /
100.0),
                LastCalibratedAt = SYSUTCDATETIME()
        WHERE  ParameterName IN (N'cpu_cost_per_vcpu_hour', N'io_cost_per_gb');

        SET @CalibrationMsg = @CalibrationMsg
            + N'Cost model recalibrated: avg error was '
            + CAST(ROUND(@AvgSavingsError, 2) AS NVARCHAR(20)) + N'%. ';
    END;

    -- SLA risk underestimation check
    SELECT @SLAMismatchCount = COUNT(*)
    FROM CostOptimization.ActionOutcomes
    WHERE MeasuredAt          >= DATEADD(DAY, -30, SYSUTCDATETIME())
        AND SLABreachOccurred = 1
        AND PredictedSLARisk  < 0.2
        AND CalibrationApplied = 0;

    IF @SLAMismatchCount >= 3
    BEGIN
        UPDATE CostOptimization.ScoringWeights
        SET     WeightValue = CASE
                                WHEN WeightValue + 0.05 > 0.6 THEN 0.6
                                ELSE WeightValue + 0.05
                            END,
                LastAdjustedAt = SYSUTCDATETIME()
        WHERE  WeightName = N'beta_sla_risk';

        SET @CalibrationMsg = @CalibrationMsg
            + N'SLA risk weight increased: '
            + CAST(@SLAMismatchCount AS NVARCHAR(10))

```



```

        + N' under-predicted breaches. ';
    END;

    -- Mark outcomes processed
    UPDATE CostOptimization.ActionOutcomes
    SET     CalibrationApplied = 1
    WHERE   MeasuredAt        >= DATEADD(DAY, -30, SYSUTCDATETIME())
           AND CalibrationApplied = 0;

    SELECT ISNULL(NULLIF(@CalibrationMsg, N''), N'No calibration required.') AS
    CalibrationResult;
END;

```

The calibration function is scheduled to run on the first day of each billing month, immediately after month-end actuals are available from the cloud provider. Over several billing cycles, the cost model converges toward accurate unit cost estimates for the specific instance family and region in use, and the scoring weights stabilize at values that reflect the organization's actual risk tolerance as revealed by operational history rather than aspirational policy documents.

A reasonable target for a well-calibrated system after six months of operation is predicted savings within 10% of actual savings and an SLA breach false-negative rate in the low single digits — figures that will vary with workload volatility and forecasting horizon. Track the actual numbers in the `action_outcomes` table and report them in a monthly cost optimization review rather than assuming convergence.

```

graph TD
    A[Cost Signals: cloud billing APIs + Query Store] --> B[Attribution & Forecast]
    B --> C[Candidate Actions: rightsizing, query governance]
    C --> D[Multi-Objective Scoring: cost vs SLA vs availability risk]
    D -->|Score above 0| E[Autonomous Execution]
    D -->|Score at or below 0| F[Human Approval Queue]
    E --> G[(Action Outcomes Log)]
    F --> G
    G -->|7-day outcome measurement| H[Calibration: adjust cost coefficients and risk weights]
    H --> B

```

The closing loop for autonomous cost optimization: every action's predicted outcome is logged, measured a week later, and fed back into both the cost model coefficients and the SLA/availability risk weights.

Key Takeaways

- → **Cost must be a runtime metric, not a billing artifact.** Autonomous cost optimization starts with dimensional attribution — tagging every query, index maintenance event, and storage write to a cost center at collection time, then flowing that signal into the same monitoring stack as CPU and I/O. Without real-time cost visibility, the control loops have nothing to act on.
- → **Rightsizing operates on three independent axes — compute tier, storage IOPS, and replica count — each with its own control loop cadence and safeguards.** Replica decommissioning is gated by minimum quorum floors, change freeze windows, and cost approval thresholds to prevent the autonomous system from undermining DR guarantees in pursuit of savings.
- → **Query-level cost governance addresses the demand side of spend through a three-tier model: statement timeout enforcement by cost center, workload shaping via connection pool queue depth control, and cost-aware index suggestion that prioritizes candidates by budget impact rather than raw execution time improvement.**
- → **Predictive forecasting shifts the control point upstream of threshold breaches.** A linear extrapolation on a 7-day trailing daily average, overlaid with a calendar of known high-cost events, gives the system enough lead time to apply graduated throttling before a cost center's budget is exhausted — rather than after.
- → **The multi-objective scoring function ($W_{\text{cost}} \times \alpha - W_{\text{sla}} \times \beta - W_{\text{avail}} \times \gamma$) provides a principled, auditable basis for resolving cost-versus-SLA tradeoffs autonomously.** The organizational risk preference is encoded in the $\alpha/\beta/\gamma$ weights, while the input scores are continuously refreshed from operational data. The feedback calibration loop closes the cycle by adjusting both the cost model coefficients and the β weight when predictions diverge from measured outcomes across billing periods.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 36: Governance Models for Autonomous DBAs

Autonomous database systems that observe, decide, and act without human intervention create a governance paradox: the more capable the system, the more critical it becomes to define precisely what it is allowed to do, when it is allowed to do it, and how its decisions are recorded, challenged, and reversed. Governance in this context is not compliance theater — it is the architectural framework that separates a system your organization can trust from one that silently makes production changes nobody can explain or audit. This chapter covers the design of governance models purpose-built for autonomous DBAs: policy engines, action taxonomies, approval workflows that don't reintroduce human latency, immutable audit trails, and the organizational structures that allow autonomous systems to operate at full capability while remaining accountable to engineering leadership and regulatory requirements.

Defining the Autonomous Action Taxonomy

Every autonomous action the system can take must be classified before any governance model can be built around it. Without a formal taxonomy, governance becomes ad hoc — rules accumulate without structure, edge cases slip through, and audit reports become uninterpretable.

The taxonomy has three primary axes: **impact radius**, **reversibility**, and **urgency**. These three dimensions determine which governance track an action follows.

Impact radius classifies how many objects, sessions, or downstream systems an action affects. A single-index rebuild affects one table. A connection pool reconfiguration affects every application session. A failover affects every connected client and potentially downstream replication consumers.

Reversibility classifies whether the action can be undone in a bounded time window. Dropping a column is irreversible without a restore. Changing `work_mem` is immediately reversible. Creating an index is reversible but costly to undo under load.

Urgency classifies whether deferring the action increases harm. A deadlock cascade demands immediate intervention. An index fragmentation issue can wait for a maintenance window.

Map every action your autonomous system can take to a matrix of these three properties:

Action	Impact Radius	Reversible	Urgent	Governance Track

|---|---|---|---|---|

| ALTER INDEX ... REBUILD (ONLINE) | Single table | Yes (rebuild back) | No | Auto-approved
|

| ALTER INDEX ... SET (FILLFACTOR) | Single table | Yes | No | Auto-approved |

| Kill idle-in-transaction session | Single session | No | Yes | Auto-approved with log |

| Add read replica | Service-wide | Yes (remove replica) | No | Async approval |

| Change user connections | Service-wide | Yes (restart req.) | No | Synchronous approval |

| Primary failover | Service-wide | No (topology change) | Yes | Auto-approved, immediate
notification |

| DROP TABLE (via TTL policy) | Single object | No | No | Synchronous approval + dual
confirmation |

| Partition detach + archive | Multiple objects | Partial | No | Async approval |

Store this taxonomy in your policy engine as structured data, not comments or runbooks:

```

CREATE TABLE dbo.AutonomousActionTaxonomy (
    ActionId                UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY
KEY,
    ActionType              NVARCHAR(100)    NOT NULL,
    ImpactRadius            NVARCHAR(20)     NOT NULL
                        CHECK (ImpactRadius IN
('session','object','service','cross-service')),
    Reversible              BIT              NOT NULL,
    ReversibilityNote       NVARCHAR(500)    NULL,
    Urgent                  BIT              NOT NULL,
    GovernanceTrack         NVARCHAR(20)     NOT NULL
                        CHECK (GovernanceTrack IN
('auto','async_approval','sync_approval','prohibited')),
    MaxAutoPerHour          INT              NULL,
    RequiresDryRun          BIT              NOT NULL DEFAULT 0,
    CreatedAt               DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    UpdatedAt               DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME()
);

INSERT INTO dbo.AutonomousActionTaxonomy
    (ActionType, ImpactRadius, Reversible, ReversibilityNote,
    Urgent, GovernanceTrack, MaxAutoPerHour, RequiresDryRun)
VALUES
    ('reindex_online',      'object', 1, 'DROP INDEX to undo',          0,
'auto',                    4, 0),
    ('kill_blocking_session', 'session', 0, 'Session state lost',          1,
'auto',                    NULL, 0),
    ('ag_failover',         'service', 0, 'Requires manual AG re-seeding', 1,
'auto',                    1, 0),
    ('drop_table_ttl',      'object', 0, 'Restore from backup only',          0,
'sync_approval',1, 1),
    ('add_ag_secondary',    'service', 1, 'Remove replica to undo',          0,
'async_approval',2, 0);

```

With this taxonomy in place, every action request the autonomous system generates carries a `governance_track` before it reaches the execution layer. The system never asks "should I do this?" at runtime — that decision is encoded in the taxonomy, versioned, and auditable.

Policy Engine Design: Rules, Scopes, and Override Chains

The taxonomy defines *what* actions exist. The policy engine defines *under what conditions* each action is permitted. These are distinct concerns, and conflating them produces brittle systems where changing a threshold requires touching action classification logic.

A production policy engine for autonomous database operations has three layers:

1. **Global policies** — apply to all databases, all environments, all times
2. **Environment policies** — production vs. staging vs. development have different thresholds

3. **Database-scoped policies** — specific databases with regulatory classification or SLA tiers may have tighter or looser rules

Policies reference the taxonomy by `action_type` and express conditions as structured predicates. Do not implement policies as Python if-else chains — that creates governance logic spread across code that engineers can't inspect, approve, or audit without reading source. Externalise the policy predicates:

```

# policy_engine.py
# Policy engine that evaluates action requests against stored policy predicates.
# Policies are loaded from the governance DB at startup and refreshed every 60
seconds.

import json
import time
import threading
import pyodbc
from dataclasses import dataclass, field
from typing import Any, Optional
from datetime import datetime, timezone

@dataclass
class ActionRequest:
    action_type: str
    target_database: str
    environment: str          # 'production', 'staging', 'development'
    urgency_signal: bool
    context: dict[str, Any]   # metric values, session counts, etc.
    requested_at: datetime = field(default_factory=lambda:
datetime.now(timezone.utc))

@dataclass
class PolicyDecision:
    approved: bool
    governance_track: str
    policy_id: str
    reason: str
    requires_dry_run: bool
    approval_timeout_seconds: Optional[int]

class PolicyEngine:
    def __init__(self, governance_dsn: str, refresh_interval: int = 60):
        self.governance_dsn = governance_dsn
        self.refresh_interval = refresh_interval
        self._policies: list[dict] = []
        self._taxonomy: dict[str, dict] = {}
        self._lock = threading.RLock()
        self._load_policies()
        self._start_refresh_thread()

    def _load_policies(self):
        with pyodbc.connect(self.governance_dsn) as conn:
            cur = conn.cursor()
            cur.execute("""
                SELECT p.policy_id, p.action_type, p.environment_scope,
                       p.database_scope, p.condition_json,
                       p.approved_outcome, p.reason_template,
                       t.governance_track, t.requires_dry_run
                FROM governance_policies p
                JOIN autonomous_action_taxonomy t
                ON t.action_type = p.action_type
            """)

```

```

        WHERE p.active = 1
        ORDER BY p.priority ASC
    """)
    rows = cur.fetchall()
    with self._lock:
        self._policies = [
            {
                'policy_id': r[0], 'action_type': r[1],
                'environment_scope': r[2], 'database_scope': r[3],
                'condition': json.loads(r[4]), 'approved_outcome': r[5],
                'reason_template': r[6], 'governance_track': r[7],
                'requires_dry_run': r[8]
            }
            for r in rows
        ]

def evaluate(self, request: ActionRequest) -> PolicyDecision:
    with self._lock:
        policies = list(self._policies)

    for policy in policies:
        if policy['action_type'] != request.action_type:
            continue
        if (policy['environment_scope'] not in ('*', request.environment)):
            continue
        if (policy['database_scope'] not in ('*', request.target_database)):
            continue
        if self._evaluate_condition(policy['condition'], request.context):
            approved = policy['approved_outcome']
            reason = policy['reason_template'].format(**request.context)
            timeout = 300 if policy['governance_track'] == 'async_approval' else
None
            return PolicyDecision(
                approved=approved,
                governance_track=policy['governance_track'],
                policy_id=policy['policy_id'],
                reason=reason,
                requires_dry_run=policy['requires_dry_run'],
                approval_timeout_seconds=timeout
            )

    # No matching policy: default deny
    return PolicyDecision(
        approved=False,
        governance_track='sync_approval',
        policy_id='default-deny',
        reason='No matching policy found; escalating to synchronous approval.',
        requires_dry_run=True,
        approval_timeout_seconds=None
    )

def _evaluate_condition(self, condition: dict, context: dict) -> bool:
    # Conditions are simple threshold predicates: {"metric": "cpu_pct", "op":

```



```

"gt", "value": 80}
    # Complex AND/OR conditions use nested lists
    op_map = {
        'gt': lambda a, b: a > b, 'lt': lambda a, b: a < b,
        'gte': lambda a, b: a >= b, 'lte': lambda a, b: a <= b,
        'eq': lambda a, b: a == b, 'neq': lambda a, b: a != b
    }
    if 'and' in condition:
        return all(self._evaluate_condition(c, context) for c in
condition['and'])
    if 'or' in condition:
        return any(self._evaluate_condition(c, context) for c in
condition['or'])
    metric_val = context.get(condition['metric'])
    if metric_val is None:
        return False
    return op_map[condition['op']](metric_val, condition['value'])

def _start_refresh_thread(self):
    def refresh_loop():
        while True:
            time.sleep(self.refresh_interval)
            try:
                self._load_policies()
            except Exception as e:
                # Log but do not crash – stale policies are preferable to no
policies
                print(f"[PolicyEngine] Refresh failed: {e}")
    t = threading.Thread(target=refresh_loop, daemon=True)
    t.start()

```

The override chain matters for production incidents. When an urgent action is blocked by policy — for example, a production failover during a blackout window — the system needs a defined escalation path that doesn't require waking someone up to manually edit a policy record. Implement a **time-bounded policy override** mechanism: engineers can issue an override token with an expiry, logged immutably, that elevates a specific action type for a specific database for a maximum of N minutes. The token is generated out-of-band (from a separate management plane), and its use is recorded regardless of whether the action succeeds.

Approval Workflows Without Human Latency Penalties

The weakest design pattern in autonomous governance is the one that re-introduces human latency as a mandatory gate on every non-trivial action. If your failover automation requires a human to click "approve" before proceeding, your RTO is bounded below by human response time — which is not a property of your infrastructure but of your on-call engineer's sleep schedule.

The solution is **tiered asynchronous approval with auto-escalation**, combined with **dead-man's switch logic** for urgent actions.

For non-urgent, high-impact actions, the workflow is:

1. System generates an action proposal with full context, estimated impact, rollback procedure, and estimated duration.
2. Proposal is written to the governance queue and a notification is dispatched (PagerDuty, Slack, email).
3. A 15-minute window opens. If an engineer explicitly **rejects** the proposal, the action is cancelled and the rejection is logged.
4. If the window expires without rejection, the action is **auto-approved by timeout** — not by silence, but by an explicit policy that states timeout equals approval for this action class.
5. The action executes with full audit logging.

For urgent, high-impact actions (primary failover, emergency connection kill, emergency partition drop):

1. System determines urgency via its decision model (e.g., replication lag exceeds 60 seconds and primary is unresponsive for 30 seconds).
2. Action executes immediately with a simultaneous notification containing full context and a **30-second veto window**.
3. If an engineer sends a veto signal within 30 seconds, the action is rolled back if reversible; if irreversible, the veto is recorded but cannot undo the action — it triggers a post-incident review workflow instead.

Store the approval workflow state in a durable queue table:

```

CREATE TABLE dbo.GovernanceActionQueue (
    QueueId                UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY
KEY,
    ActionType              NVARCHAR(100)    NOT NULL,
    TargetDatabase          NVARCHAR(128)    NOT NULL,
    Environment             NVARCHAR(50)     NOT NULL,
    ActionPayload           NVARCHAR(MAX)    NOT NULL, -- JSON
    RollbackProcedure       NVARCHAR(MAX)    NULL,
    PolicyId               NVARCHAR(100)    NOT NULL,
    GovernanceTrack         NVARCHAR(20)     NOT NULL,
    Status                  NVARCHAR(20)     NOT NULL DEFAULT 'pending'
        CONSTRAINT CK_GAQ_Status CHECK (
            Status IN ('pending','approved','rejected',
                'executing','completed','failed',
                'vetoed','timeout_approved')),
    ProposedAt             DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    ApprovalDeadline       DATETIME2        NULL,
    DecidedAt              DATETIME2        NULL,
    DecidedBy              NVARCHAR(200)    NULL,
    ExecutedAt             DATETIME2        NULL,
    CompletedAt            DATETIME2        NULL,
    OutcomeSummary         NVARCHAR(MAX)    NULL,
    ErrorDetail            NVARCHAR(MAX)    NULL
);

CREATE INDEX IX_GAQ_Status_Deadline
    ON dbo.GovernanceActionQueue (Status, ApprovalDeadline)
    WHERE Status = 'pending';

-- Pending approvals with countdown
CREATE VIEW dbo.PendingApprovals AS
SELECT
    QueueId,
    ActionType,
    TargetDatabase,
    Environment,
    GovernanceTrack,
    ProposedAt,
    ApprovalDeadline,
    DATEDIFF(SECOND, SYSUTCDATETIME(), ApprovalDeadline) AS SecondsRemaining,
    ActionPayload
FROM dbo.GovernanceActionQueue
WHERE Status = 'pending';

```

The timeout-approval background worker runs every 10 seconds, checking for items past their deadline:

```

# approval_timeout_worker.py
# Runs as a sidecar process to the main autonomous engine.
# Transitions 'pending' proposals to 'timeout_approved' and enqueues them for
execution.

import time
import pyodbc
from datetime import datetime, timezone

GOVERNANCE_DSN = ("DRIVER={ODBC Driver 18 for SQL Server};SERVER=governance-db;"
                  "DATABASE=autonomous_governance;Trusted_Connection=yes;")
POLL_INTERVAL_SECONDS = 10

def process_timeouts():
    with pyodbc.connect(GOVERNANCE_DSN, autocommit=False) as conn:
        cur = conn.cursor()
        # Lock rows for update to prevent duplicate processing
        cur.execute("""
            UPDATE governance_action_queue
            SET status      = 'timeout_approved',
                decided_at = SYSUTCDATETIME(),
                decided_by = 'timeout'
            WHERE status      = 'pending'
                AND governance_track = 'async_approval'
                AND approval_deadline < SYSUTCDATETIME()
            OUTPUT INSERTED.queue_id, INSERTED.action_type,
                INSERTED.target_database, INSERTED.action_payload
            """)
        approved_rows = cur.fetchall()
        conn.commit()
    return approved_rows

def run():
    print(f"[ApprovalTimeoutWorker] Started at {datetime.now(timezone.utc).isoformat()}")
    while True:
        try:
            approved = process_timeouts()
            for row in approved:
                queue_id, action_type, target_db, payload = row
                print(f"[ApprovalTimeoutWorker] Timeout-approved: {action_type} "
                    f"on {target_db} (queue_id={queue_id})")
                # Downstream executor picks up 'timeout_approved' rows via its own
polling loop
            except Exception as e:
                print(f"[ApprovalTimeoutWorker] Error: {e}")
                time.sleep(POLL_INTERVAL_SECONDS)

if __name__ == '__main__':
    run()

```

This architecture eliminates human latency from the critical path while preserving human authority. Engineers are in a position of continuous veto power, not continuous approval power — a far more scalable model.

```
graph TD
  A[Action Request] --> B{Governance Track}
  B -->|auto| C[Execute Immediately]
  B -->|async_approval| D[Queue + Notify: 15-min Window]
  B -->|sync_approval| E[Block Until Explicit Approval]
  B -->|urgent, high-impact| F[Execute Now + 30s Veto Window]
  D -->|Rejected| G[Cancelled]
  D -->|Timeout, No Reject| H[Timeout-Approved]
  F -->|Vetoed, Reversible| I[Rolled Back]
  F -->|Vetoed, Irreversible| J[Post-Incident Review]
  C --> K[(Hash-Chained Audit Log)]
  H --> K
  I --> K
  J --> K
```

Governance track routing: only sync_approval blocks on a human; every other path keeps engineers in a veto role rather than an approval role, and every terminal state lands in the immutable audit log.

Immutable Audit Trails: Design and Enforcement

A governance model without verifiable audit trails is not a governance model — it is documentation theater. In regulated industries, audit trails for autonomous system actions carry the same compliance weight as audit trails for human DBA actions. In non-regulated environments, they are the primary tool for post-incident analysis, model improvement, and organizational trust-building.

The audit trail for an autonomous action has four required layers:

1. **Decision log** — why the system decided to take an action (which metrics triggered it, which policy matched)
2. **Pre-action state snapshot** — what the database state looked like before the action
3. **Execution log** — every SQL statement or API call made, in order, with timestamps
4. **Post-action state snapshot** — what changed, with diff from pre-action state

Immutability must be enforced at the database level, not just by application convention. Use append-only table patterns backed by triggers or row-level security that prohibits updates and deletes:

```

-- Immutable audit log using temporal-style append-only enforcement
CREATE TABLE dbo.AutonomousAuditLog (
    LogId          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    QueueId        UNIQUEIDENTIFIER NOT NULL,
    LogType        NVARCHAR(20)      NOT NULL
                        CONSTRAINT CK_AAL_LogType CHECK (
                            LogType IN ('decision','pre_state','execution',
                                'post_state','diff','error')),
    SequenceNum    INT                NOT NULL,
    LoggedAt       DATETIME2          NOT NULL DEFAULT SYSUTCDATETIME(),
    Actor          NVARCHAR(200)      NOT NULL,
    Payload        NVARCHAR(MAX)      NOT NULL, -- JSON
    Checksum       NVARCHAR(64)       NOT NULL  -- SHA-256 hex
);

-- Deny UPDATE and DELETE at the permission level for all application roles
DENY UPDATE ON dbo.AutonomousAuditLog TO ade_worker;
DENY DELETE ON dbo.AutonomousAuditLog TO ade_worker;

-- DDL trigger to prevent schema changes that would circumvent the above
CREATE TRIGGER trg_PreventAuditLogAlteration
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
BEGIN
    DECLARE @EventData XML = EVENTDATA();
    IF @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]','NVARCHAR(128)') =
'AutonomousAuditLog'
    BEGIN
        RAISERROR('Schema changes to AutonomousAuditLog are prohibited.', 16, 1);
        ROLLBACK;
    END
END;

```

The checksum field is essential: each log entry carries a SHA-256 hash of its own content including the previous entry's hash, forming a hash chain. Any tampering with a historical entry invalidates all subsequent checksums, making silent modification detectable without an external audit service. Implement the chaining in the writer:

```

# audit_writer.py
import hashlib
import json
import pyodbc
from uuid import UUID
from datetime import datetime, timezone

class AuditWriter:
    def __init__(self, dsn: str):
        self.dsn = dsn

    def _compute_checksum(self, queue_id: str, log_type: str,
                          sequence_num: int, payload: dict,
                          prev_checksum: str) -> str:
        content = json.dumps({
            'queue_id': queue_id,
            'log_type': log_type,
            'sequence_num': sequence_num,
            'payload': payload,
            'prev_checksum': prev_checksum
        }, sort_keys=True, default=str)
        return hashlib.sha256(content.encode()).hexdigest()

    def _get_prev_checksum(self, cur, queue_id: str) -> str:
        cur.execute("""
            SELECT TOP 1 checksum FROM autonomous_audit_log
            WHERE queue_id = ?
            ORDER BY sequence_num DESC
        """, (queue_id,))
        row = cur.fetchone()
        return row[0] if row else 'GENESIS'

    def write(self, queue_id: str, log_type: str, sequence_num: int,
              actor: str, payload: dict) -> str:
        with pyodbc.connect(self.dsn) as conn:
            cur = conn.cursor()
            prev_checksum = self._get_prev_checksum(cur, queue_id)
            checksum = self._compute_checksum(
                queue_id, log_type, sequence_num, payload, prev_checksum
            )
            cur.execute("""
                INSERT INTO autonomous_audit_log
                (queue_id, log_type, sequence_num, actor, payload, checksum)
                OUTPUT INSERTED.log_id
                VALUES (?, ?, ?, ?, ?, ?)
            """, (queue_id, log_type, sequence_num,
                  actor, json.dumps(payload, default=str), checksum))
            log_id = cur.fetchone()[0]
            conn.commit()
        return str(log_id)

    def verify_chain(self, queue_id: str) -> tuple[bool, list[str]]:
        """Returns (is_valid, list_of_violations)."""

```

```

violations = []
with pyodbc.connect(self.dsn) as conn:
    cur = conn.cursor()
    cur.execute("""
        SELECT log_type, sequence_num, actor, payload, checksum
        FROM autonomous_audit_log
        WHERE queue_id = ?
        ORDER BY sequence_num ASC
    """, (queue_id,))
    rows = cur.fetchall()

    prev_checksum = 'GENESIS'
    for log_type, seq, actor, payload_str, stored_checksum in rows:
        payload = json.loads(payload_str)
        expected = self._compute_checksum(
            queue_id, log_type, seq, payload, prev_checksum
        )
        if expected != stored_checksum:
            violations.append(
                f"Checksum mismatch at sequence {seq} "
                f"(type={log_type}): expected {expected}, got {stored_checksum}"
            )
        prev_checksum = stored_checksum

    return (len(violations) == 0, violations)

```

Run `verify_chain` as part of your daily governance health check and on-demand during incident investigations. Any violation is a potential integrity incident and should trigger immediate escalation.

Governance Scopes Across Multi-Tenant and Multi-Region Environments

Single-database governance is relatively straightforward. The complexity multiplies when an autonomous system manages dozens or hundreds of databases across multiple cloud regions, each with different regulatory classifications, SLA tiers, and team ownership.

The governance model must express scope explicitly. A policy that says "kill idle-in-transaction sessions older than 5 minutes" means something very different on a development database with a one-engineer team than on a financial trading platform with a four-nines SLA and SOX controls.

Design your governance scope hierarchy as a tree:

Organization

- └ Environment (production, staging, development)
 - └ Region (us-east-1, eu-west-1, ap-southeast-2)
 - └ Cluster (sql-ag-cluster-01)
 - └ Database (orders_db, catalog_db)

Policies inherit downward but can be overridden at any level. A global policy prohibiting `DROP` `TABLE` outside maintenance windows applies everywhere unless a specific database has a registered TTL archiving exemption that overrides it. Store the scope hierarchy and inheritance rules:

```

CREATE TABLE dbo.GovernanceScopeNodes (
    NodeId      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    ParentId    UNIQUEIDENTIFIER NULL REFERENCES dbo.GovernanceScopeNodes(NodeId),
    NodeType    NVARCHAR(20)      NOT NULL
                CONSTRAINT CK_GSN_NodeType CHECK (
                    NodeType IN
('org','environment','region','cluster','database')),
    NodeName    NVARCHAR(200)     NOT NULL,
    Metadata    NVARCHAR(MAX)     NOT NULL DEFAULT '{}',
    CONSTRAINT UQ_GSN_Parent_Name UNIQUE (ParentId, NodeName)
);

CREATE TABLE dbo.GovernancePolicyAssignments (
    AssignmentId UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID() PRIMARY KEY,
    PolicyId     NVARCHAR(100)    NOT NULL,
    NodeId       UNIQUEIDENTIFIER NOT NULL
                REFERENCES dbo.GovernanceScopeNodes(NodeId),
    OverrideMode NVARCHAR(20)     NOT NULL
                CONSTRAINT CK_GPA_Override CHECK (
                    OverrideMode IN ('inherit','replace','extend')),
    Active       BIT              NOT NULL DEFAULT 1,
    AssignedAt   DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    AssignedBy   NVARCHAR(200)    NOT NULL
);

-- Recursive CTE: effective policies for a database node
CREATE OR REPLACE VIEW dbo.EffectivePoliciesView AS
-- Parameterised inline TVF equivalent using APPLY is preferred; this is a scaffold
-- In practice, implement as a stored procedure or inline TVF accepting @NodeId
WITH ScopeAncestors AS (
    SELECT NodeId, ParentId, 0 AS Depth
    FROM dbo.GovernanceScopeNodes
    -- Anchor: replace with parameter in TVF
    UNION ALL
    SELECT n.NodeId, n.ParentId, sa.Depth + 1
    FROM dbo.GovernanceScopeNodes n
    JOIN ScopeAncestors sa ON n.NodeId = sa.ParentId
),
RankedAssignments AS (
    SELECT
        pa.PolicyId,
        pa.NodeId AS SourceNodeId,
        pa.OverrideMode,
        sa.Depth,
        ROW_NUMBER() OVER (
            PARTITION BY pa.PolicyId
            ORDER BY sa.Depth ASC
        ) AS rn
    FROM dbo.GovernancePolicyAssignments pa
    JOIN ScopeAncestors sa ON pa.NodeId = sa.NodeId
    WHERE pa.Active = 1
)
SELECT PolicyId, SourceNodeId, OverrideMode

```

```
FROM RankedAssignments
WHERE rn = 1 AND OverrideMode != 'inherit';
```

Regulatory scope is a separate dimension layered on top of the hierarchy. A database hosting EU personal data under GDPR, or US financial records under SOX, carries labels that activate additional policy constraints — most notably prohibiting autonomous schema changes without a change management ticket reference, and requiring pre-action and post-action state snapshots to be retained for a defined period:

```
-- Regulatory labels and their policy constraints (SQL Server)
CREATE TABLE dbo.RegulatoryLabels (
    LabelId          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID()
    PRIMARY KEY,
    LabelName        NVARCHAR(64)      NOT NULL UNIQUE, -- 'GDPR', 'SOX',
    'HIPAA', 'PCI-DSS'
    ProhibitedTracks NVARCHAR(MAX)     NOT NULL DEFAULT '[]',
    RequiredRetentionDays INT          NOT NULL DEFAULT 365,
    RequireTicketReference BIT         NOT NULL DEFAULT 1,
    RequireDualApproval BIT           NOT NULL DEFAULT 0
);

CREATE TABLE dbo.DatabaseRegulatoryLabels (
    NodeId    UNIQUEIDENTIFIER NOT NULL REFERENCES dbo.GovernanceScopeNodes(NodeId),
    LabelId   UNIQUEIDENTIFIER NOT NULL REFERENCES dbo.RegulatoryLabels(LabelId),
    LabeledAt DATETIMEOFFSET   NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    LabeledBy NVARCHAR(128)    NOT NULL,
    PRIMARY KEY (NodeId, LabelId)
);
```

When the policy engine evaluates a request, it fetches all regulatory labels for the target database node and applies the most restrictive constraints. If a database carries both GDPR and SOX labels, and SOX requires dual approval while GDPR requires a 7-year audit retention, both constraints apply simultaneously.

Organizational Models: Who Owns Autonomous Governance?

Technical governance architecture is necessary but not sufficient. The organizational structure around it determines whether governance operates in practice or erodes under operational pressure.

Three models are observed in engineering organizations that have deployed autonomous database systems at scale.

Model 1: Platform Engineering Ownership

The autonomous database system is owned by a platform engineering or SRE team. That team maintains the taxonomy, policies, audit infrastructure, and escalation workflows. Individual database-owning teams (e.g., the payments team that owns `payments_db`) are consumers of the governance system — they can request policy exceptions via a self-service API, but they cannot modify the governance rules directly. This model concentrates expertise and produces consistent policy application, but creates a bottleneck when database teams need rapid policy changes to support feature delivery.

Model 2: Federated Ownership with Central Policy Floor

The platform team defines a minimum policy floor: a set of policies that apply globally and cannot be overridden. Database-owning teams own their own database-scoped policy extensions, subject to the floor. They can permit more aggressive autonomous actions within their scope (e.g., allowing index rebuilds during business hours on their staging cluster), but cannot weaken the floor protections. This model scales better and gives teams autonomy proportional to their maturity, but requires clear documentation of what constitutes the floor versus an extension.

Model 3: Committee-Based Governance

A cross-functional governance committee — typically including representatives from platform engineering, security, compliance, and database engineering — reviews and approves changes to the taxonomy and global policies on a defined cadence (monthly or quarterly). Day-to-day execution is delegated to the platform team. This model is appropriate for organizations where database systems are subject to regulatory or audit scrutiny, and is often mandated rather than chosen.

Most organizations operating at scale use a hybrid of Model 1 and Model 2: a central policy floor enforced by the platform team, with a structured delegation mechanism that allows database teams to extend policies within defined bounds. This maps naturally to the scope hierarchy in the previous section.

The organizational model must also define three roles with non-overlapping responsibilities:

Policy Author — Writes and version-controls governance policies. Has write access to the governance database's policy tables. Cannot approve their own policies.

Policy Approver — Reviews and activates policy changes. Is distinct from the Policy Author. In regulated environments, this is often a named individual accountable to the compliance function.

Audit Reviewer — Has read-only access to the audit log and chain verification tooling. Produces periodic attestations that autonomous system actions were within approved policy bounds. Cannot modify policies or approve actions.

Change Management Integration

Autonomous systems that operate without any connection to your organization's change management process are invisible to the change record — and an invisible change that causes an incident produces an investigation with a missing causal chain. Integrate autonomous action proposals with your change management system, even when the action is auto-approved.

The integration does not mean every auto-approved action requires a manual change ticket. It means:

1. Every governance track of `async_approval` or higher creates a change record automatically, populated with the action context, estimated impact, and rollback procedure.
2. Auto-approved actions with `auto` governance track are batched into a daily digest change record that summarizes all autonomous actions taken, grouped by database and action type.
3. Any action that results in an error or triggers a rollback creates an incident record automatically.

This keeps the change calendar accurate without imposing human latency on routine autonomous operations:

```

# change_management_integration.py
# Integrates governance queue events with an external ITSM system (ServiceNow, Jira
# Service Management, etc.)
# via a generic webhook interface.

import requests
import json
from dataclasses import dataclass
from typing import Optional
from datetime import datetime, timezone

@dataclass
class ChangeRecord:
    external_id: Optional[str]
    title: str
    description: str
    category: str          # 'standard', 'normal', 'emergency'
    risk_level: str        # 'low', 'medium', 'high'
    rollback_plan: str
    scheduled_start: Optional[datetime]
    scheduled_end: Optional[datetime]
    auto_approved: bool
    queue_id: str

class ChangeManagementClient:
    def __init__(self, webhook_url: str, api_token: str):
        self.webhook_url = webhook_url
        self.headers = {
            'Authorization': f'Bearer {api_token}',
            'Content-Type': 'application/json'
        }

    def create_change(self, record: ChangeRecord) -> str:
        """Returns the external change record ID."""
        payload = {
            'title': record.title,
            'description': record.description,
            'category': record.category,
            'risk_level': record.risk_level,
            'rollback_plan': record.rollback_plan,
            'auto_approved': record.auto_approved,
            'source_system': 'autonomous_dba_engine',
            'source_reference': record.queue_id,
            'scheduled_start': record.scheduled_start.isoformat()
                               if record.scheduled_start else None,
            'scheduled_end': record.scheduled_end.isoformat()
                              if record.scheduled_end else None
        }
        response = requests.post(
            self.webhook_url,
            headers=self.headers,
            data=json.dumps(payload, default=str),
            timeout=10

```

```

    )
    response.raise_for_status()
    return response.json().get('change_id', 'unknown')

def close_change(self, external_id: str, outcome: str, notes: str):
    response = requests.patch(
        f"{self.webhook_url}/{external_id}",
        headers=self.headers,
        data=json.dumps({'status': 'closed', 'outcome': outcome, 'notes':
notes}),
        timeout=10
    )
    response.raise_for_status()

def classify_change_risk(action_type: str, governance_track: str) -> tuple[str,
str]:
    """Returns (category, risk_level) for a change record."""
    if governance_track == 'auto':
        return ('standard', 'low')
    if governance_track == 'async_approval':
        return ('normal', 'medium')
    if governance_track == 'sync_approval':
        return ('normal', 'high')
    if action_type in ('primary_failover', 'ag_failover'):
        return ('emergency', 'high')
    return ('normal', 'medium')

```

The change management integration ensures that when an auditor asks "what changed in the orders database last Thursday between 02:00 and 03:00 UTC?", the answer is available in the ITSM system in the same format as any human-initiated change — not buried in application logs that only platform engineers know how to query.

Governance Health Metrics and Dashboards

Governance infrastructure is itself a system that can degrade. Policies can become stale. The approval queue can back up. Audit chain verification can silently fail. Treat the governance system as a first-class service with its own SLOs:

| Metric | SLO Target | Alert Threshold |

|---|---|---|

| Policy engine refresh latency | < 90 seconds from policy update to evaluation | > 5 minutes |

| Approval queue depth (pending items) | < 20 items | > 50 items |

| Audit chain integrity (% valid chains past 24h) | 100% | Any violation |

| Auto-approval timeout rate | < 5% of async proposals | > 15% |

| Action execution failure rate | < 2% | > 5% |

| Time from proposal to execution (async track) | < 20 minutes median | > 45 minutes p95 |

Expose these metrics through your observability stack as a monitoring view that a Prometheus scraper or your existing monitoring agent can poll:

```
CREATE VIEW dbo.GovernanceHealthMetrics AS
SELECT
    SUM(CASE WHEN Status = 'pending' THEN 1 ELSE 0 END) AS QueuePending,
    SUM(CASE WHEN Status = 'executing' THEN 1 ELSE 0 END) AS QueueExecuting,
    SUM(CASE WHEN Status = 'timeout_approved'
        AND ExecutedAt IS NULL THEN 1 ELSE 0 END) AS TimeoutAwaitingExec,

    ROUND(
        100.0 * SUM(CASE WHEN Status = 'failed'
            AND CompletedAt > DATEADD(HOUR, -1, SYSUTCDATETIME())
            THEN 1 ELSE 0 END)
        / NULLIF(SUM(CASE WHEN CompletedAt > DATEADD(HOUR, -1, SYSUTCDATETIME())
            THEN 1 ELSE 0 END), 0),
        2
    ) AS FailureRatePct1h,

    ROUND(
        100.0 * SUM(CASE WHEN DecidedBy = 'timeout'
            AND DecidedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())
            THEN 1 ELSE 0 END)
        / NULLIF(SUM(CASE WHEN GovernanceTrack = 'async_approval'
            AND ProposedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())
            THEN 1 ELSE 0 END), 0),
        2
    ) AS
    TimeoutApprovalRatePct24h,

    SYSUTCDATETIME() AS MetricTimestamp
FROM dbo.GovernanceActionQueue;
-- Note: SQL Server lacks a built-in PERCENTILE_CONT in views without windowing;
-- median exec time is best computed in a stored procedure or reporting layer.
```

Alert on the `timeout_approval_rate_pct_24h` metric with particular care. A rising timeout rate means engineers are not engaging with the approval queue — either notifications are broken, proposals are arriving too frequently to review, or approval fatigue has set in. The correct response is not to reduce the timeout window; it is to investigate why proposals are not being reviewed and fix the upstream signal.

Key Takeaways

- → **Taxonomy before governance:** Every autonomous action must be formally classified by impact radius, reversibility, and urgency before any governance rule can be written around it. Unclassified actions are ungoverned actions.
- → **Separate the policy engine from the action executor:** Policies stored as structured data in a database — not as code — can be versioned, approved, audited, and changed by non-engineers. This separation is what makes governance durable across team turnover and organizational change.
- → **Eliminate human latency from the critical path without eliminating human authority:** The timeout-approval model and the pre-execution veto window together give engineers continuous authority over the system without making the system's reliability dependent on engineer response time.
- → **Immutability requires enforcement, not convention:** Audit logs protected only by application-level convention will eventually be tampered with, accidentally or deliberately. Hash-chain checksums combined with database-level permission denial create audit integrity that is verifiable and self-evidencing.
- → **Governance health is a service SLO:** Policy staleness, approval queue depth, audit chain integrity, and timeout approval rates are operational metrics that require the same monitoring, alerting, and on-call response as the database systems the autonomous engine manages.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 37: Real-World Autonomous DBA Case Studies

The gap between autonomous database theory and production reality is where most platform engineering teams lose momentum. Architectural patterns that look elegant in whiteboards collapse against the friction of legacy schemas, bursty traffic, compliance requirements, and on-call engineers who override automation at the first sign of unfamiliarity. This chapter closes that gap by examining five real-world autonomous DBA deployments — drawn from high-scale SaaS platforms, financial institutions, healthcare data pipelines, and multi-cloud e-commerce systems — each built on SQL Server across on-premises, Azure, and hybrid deployments. Every case study follows the same structure: the operational problem, the autonomous architecture chosen, the specific tooling and decision logic wired together, and the lessons that only surface in production. These are not success stories scrubbed clean for a conference talk. They include rollbacks, missed thresholds, and architectural pivots that led to better designs.

Case Study 1: Autonomous Failover and Connection Rerouting at a High-Frequency Trading Firm

A mid-sized proprietary trading firm running SQL Server 2022 on bare metal needed sub-30-second failover with zero manual intervention. Their previous setup used an Always On availability group with a manual failover script triggered by on-call engineers — a process that averaged 4.5 minutes from detection to connection rerouting due to human latency and shift handoff ambiguity. The requirement from the trading desk: automated failover with automatic client rerouting in under 25 seconds, with a hard constraint that no transaction could be silently lost.

The architecture replaced manual triggers with a fully autonomous stack. An Always On availability group in synchronous-commit mode with automatic failover remained the core, backed by a Windows Server Failover Cluster for quorum. The critical addition was a custom Python daemon called `trade-sentinel` that consumed the AG DMVs (`sys.dm_hadr_availability_replica_states`, `sys.dm_hadr_database_replica_states`) and a business-level heartbeat table written by the trading application. The AG listener sat in front of all database connections, so client rerouting after a failover was a DNS-level flip to the new primary — no connection-string changes, no proxy rewrites.

The decision logic was explicit and layered:

1. **Detection:** If the primary replica's role state in `sys.dm_hadr_availability_replica_states` is unhealthy for three consecutive 2-second polls, or if the heartbeat table (`trading.heartbeat`) has not been updated within 8 seconds, enter failover evaluation mode.
2. **Validation:** Confirm the synchronous secondary is in SYNCHRONIZED state with a redo queue under 512KB and that cluster quorum is healthy. If either condition fails, escalate to PagerDuty and hold without failing over.
3. **Failover:** Issue `ALTER AVAILABILITY GROUP [TradingAG] FAILOVER` from the target secondary, then verify the new primary role via the DMVs.
4. **Connection rerouting:** The AG listener DNS record follows the primary automatically; the daemon forces a DNS cache flush on the application tier and validates that new connections land on the new primary.
5. **Verification:** Run a synthetic write/read probe transaction against the new primary through the listener. If it fails within 10 seconds, re-escalate to human.

The `trade-sentinel` daemon managed detection, failover, and listener verification atomically:

```

import pyodbc
import subprocess
import time
import logging
from datetime import datetime, timezone

PRIMARY_CONN = "DRIVER={ODBC Driver 18 for SQL Server};SERVER=sql-
primary;DATABASE=master;Trusted_Connection=yes;"
SECONDARY_CONN = "DRIVER={ODBC Driver 18 for SQL Server};SERVER=sql-
secondary;DATABASE=master;Trusted_Connection=yes;"
AG_NAME = "TradingAG"
LISTENER = "trading-ag-listener"
HEARTBEAT_THRESHOLD_SECONDS = 8
POLL_INTERVAL = 2
FAILURE_THRESHOLD = 3

logger = logging.getLogger("trade-sentinel")

def query(conn_str: str, sql: str):
    with pyodbc.connect(conn_str, timeout=5) as conn:
        cur = conn.cursor()
        cur.execute(sql)
        return cur.fetchall()

def primary_role_state() -> str | None:
    try:
        rows = query(PRIMARY_CONN, """
            SELECT role_desc
            FROM sys.dm_hadr_availability_replica_states
            WHERE replica_server_name = @@SERVERNAME
            """)
        return rows[0][0] if rows else None
    except pyodbc.Error:
        return None

def secondary_sync_state() -> tuple:
    rows = query(SECONDARY_CONN, """
        SELECT synchronization_state_desc,
            ISNULL(redo_queue_size, 0)
        FROM sys.dm_hadr_database_replica_states
        WHERE database_id = DB_ID('TradingDB')
        """)
    return (rows[0][0], rows[0][1]) if rows else ("UNKNOWN", -1)

def heartbeat_age_seconds() -> float:
    try:
        rows = query(PRIMARY_CONN, """
            SELECT DATEDIFF(SECOND, MAX(updated_at), SYSUTCDATETIME())
            FROM trading.heartbeat
            """)
        return float(rows[0][0]) if rows and rows[0][0] is not None else 9999.0
    except pyodbc.Error:
        return 9999.0

```

```

def trigger_failover() -> bool:
    # Planned failover executes from the TARGET secondary
    with pyodbc.connect(SECONDARY_CONN, autocommit=True) as conn:
        cur = conn.cursor()
        cur.execute(f"ALTER AVAILABILITY GROUP [{AG_NAME}] FAILOVER")
    logger.info("Failover command issued from secondary")
    time.sleep(5)
    return primary_role_state() == "PRIMARY"

def flush_app_dns() -> None:
    subprocess.run(["powershell", "-Command", "Clear-DnsClientCache"],
                    check=False)
    logger.info("Application-tier DNS cache flushed")

def run_sentinel():
    consecutive_failures = 0
    while True:
        role = primary_role_state()
        hb_age = heartbeat_age_seconds()
        if role != "PRIMARY" or hb_age > HEARTBEAT_THRESHOLD_SECONDS:
            consecutive_failures += 1
            logger.warning(
                f"Primary unhealthy (role={role}, hb_age={hb_age:.1f}s) – "
                f"failure count: {consecutive_failures}"
            )
        else:
            consecutive_failures = 0

        if consecutive_failures >= FAILURE_THRESHOLD:
            logger.critical("Threshold exceeded – initiating autonomous failover")
            sync_state, redo_kb = secondary_sync_state()
            if sync_state != "SYNCHRONIZED" or redo_kb > 512:
                logger.error(
                    f"Secondary not ready (state={sync_state}, "
                    f"redo={redo_kb}KB) – escalating to human"
                )
            elif trigger_failover():
                flush_app_dns()
                logger.info("Failover complete – listener now on new primary")
            else:
                logger.error("Failover failed – escalating to human")
                consecutive_failures = 0
            time.sleep(POLL_INTERVAL)

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    run_sentinel()

```

The first production failover completed in 18.4 seconds. The second, triggered six weeks later by a kernel panic on the primary host, completed in 21.1 seconds — within specification despite higher system load. The lessons were two-fold: the heartbeat table proved far more

reliable as a detection signal than the replica-state DMVs alone, which exhibited a 3–4 second delay in reporting role loss during network partitions; and the AG listener flip was safe only because the trading application used persistent connections with application-level retry — without that, in-flight orders on the failed primary would still have been lost at the TCP layer.

Case Study 2: Self-Healing Index Management for a SaaS Analytics Platform (SQL Server)

A B2B analytics SaaS company ran SQL Server 2022 on Azure VMs (Standard_D32s_v5 instances) serving 400+ tenant databases within a single instance via schema-per-tenant isolation. Index fragmentation was causing query degradation across tenants at different rates — a scheduled maintenance window applied the same REBUILD/REORGANIZE policy to all tenants, causing unpredictable latency spikes during business hours for tenants in different time zones. The solution: a fully autonomous index management loop that evaluated each tenant schema independently and acted without scheduling.

The autonomous index agent ran as a SQL Server Agent job on a 15-minute cycle. Its logic tree:

- → If `avg_fragmentation_in_percent > 30 AND page_count > 1000` : REBUILD ONLINE
- → If `avg_fragmentation_in_percent > 10 AND page_count > 500` : REORGANIZE
- → If `page_count <= 500` : skip (small table overhead not worth it)
- → If estimated rebuild duration (based on historical stats) would exceed 4 minutes: defer to off-peak window specific to tenant's primary geography

```

-- Autonomous index maintenance procedure per tenant schema
CREATE OR ALTER PROCEDURE dbo.usp_AutoIndexMaintenance
    @TenantSchema NVARCHAR(128),
    @DryRun        BIT = 0
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SQL          NVARCHAR(MAX);
    DECLARE @TableName    NVARCHAR(256);
    DECLARE @IndexName    NVARCHAR(256);
    DECLARE @Fragmentation FLOAT;
    DECLARE @PageCount    BIGINT;
    DECLARE @Action       NVARCHAR(50);

    DECLARE index_cursor CURSOR LOCAL FAST_FORWARD FOR
        SELECT
            QUOTENAME(s.name) + '.' + QUOTENAME(t.name) AS full_table_name,
            QUOTENAME(i.name) AS index_name,
            ips.avg_fragmentation_in_percent,
            ips.page_count,
            CASE
                WHEN ips.avg_fragmentation_in_percent > 30
                     AND ips.page_count > 1000 THEN 'REBUILD'
                WHEN ips.avg_fragmentation_in_percent > 10
                     AND ips.page_count > 500 THEN 'REORGANIZE'
                ELSE 'SKIP'
            END AS recommended_action
        FROM sys.dm_db_index_physical_stats(
            DB_ID(), NULL, NULL, NULL, 'LIMITED') ips
        JOIN sys.indexes i
            ON ips.object_id = i.object_id
            AND ips.index_id = i.index_id
        JOIN sys.tables t
            ON i.object_id = t.object_id
        JOIN sys.schemas s
            ON t.schema_id = s.schema_id
        WHERE s.name = @TenantSchema
            AND i.name IS NOT NULL
            AND ips.page_count > 500;

    OPEN index_cursor;
    FETCH NEXT FROM index_cursor
        INTO @TableName, @IndexName, @Fragmentation, @PageCount, @Action;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        IF @Action = 'REBUILD'
        BEGIN
            SET @SQL = N'ALTER INDEX ' + @IndexName
                + N' ON ' + @TableName
                + N' REBUILD WITH (ONLINE = ON, MAXDOP = 4);';
        END
    END

```

```

ELSE IF @Action = 'REORGANIZE'
BEGIN
    SET @SQL = N'ALTER INDEX ' + @IndexName
              + N' ON ' + @TableName
              + N' REORGANIZE;';
END

IF @Action <> 'SKIP'
BEGIN
    -- Audit the decision before acting
    INSERT INTO dbo.autonomous_index_log
        (tenant_schema, table_name, index_name,
         fragmentation_pct, page_count, action_taken,
         executed_sql, log_time)
    VALUES
        (@TenantSchema, @TableName, @IndexName,
         @Fragmentation, @PageCount, @Action,
         @SQL, SYSUTCDATETIME());

    IF @DryRun = 0
        EXEC sp_executesql @SQL;
END

FETCH NEXT FROM index_cursor
    INTO @TableName, @IndexName, @Fragmentation, @PageCount, @Action;
END

CLOSE index_cursor;
DEALLOCATE index_cursor;
END;
GO

-- Master orchestrator job iterates all tenant schemas
DECLARE @TenantSchema NVARCHAR(128);
DECLARE tenant_cursor CURSOR LOCAL FAST_FORWARD FOR
    SELECT schema_name
    FROM    dbo.tenant_registry
    WHERE   maintenance_enabled = 1
           AND GETUTCDATE() BETWEEN maintenance_window_start
                                   AND maintenance_window_end;

OPEN tenant_cursor;
FETCH NEXT FROM tenant_cursor INTO @TenantSchema;
WHILE @@FETCH_STATUS = 0
BEGIN
    EXEC dbo.usp_AutoIndexMaintenance
        @TenantSchema = @TenantSchema,
        @DryRun        = 0;
    FETCH NEXT FROM tenant_cursor INTO @TenantSchema;
END
CLOSE tenant_cursor;
DEALLOCATE tenant_cursor;

```


The `tenant_registry` table stored each tenant's primary geography and the associated UTC maintenance window — a 4-hour block during their 2–6 AM local time. The autonomous agent queried `maintenance_window_start` and `maintenance_window_end` before acting, ensuring a tenant headquartered in Tokyo received index rebuilds during Tokyo's off-hours, not London's. This was the architectural insight that the previous global maintenance window lacked entirely.

After 90 days in production, average query duration across tenants dropped 31% during peak hours, and index-related blocking incidents — previously averaging 4 per week — dropped to zero. The `autonomous_index_log` table became the team's primary evidence artifact in customer SLA reviews.

Case Study 3: Autonomous Query Plan Stabilization in a Healthcare Data Warehouse

A regional healthcare network operated a SQL Server 2022 data warehouse — approximately 12TB across 60 tables — that fed clinical dashboards consumed by 300+ concurrent analysts. The platform team faced a recurring problem: after automatic statistics updates, query plans for 8–12 critical reporting queries would regress. A plan that previously used an index seek would switch to a clustered index scan on a 900M-row fact table, spiking query time from 4 seconds to 3.5 minutes. The on-call response was manual: a DBA would capture the actual execution plan, identify the regression, and force the good plan via Query Store with `sp_query_store_force_plan`. This manual process averaged 45 minutes of analyst impact.

The autonomous plan stabilization system worked in three tiers:

Tier 1 — Continuous baseline capture: Query Store captured query fingerprints (`query_hash`), their plans (`query_plan_hash`), and runtime statistics automatically, with forced-plan history providing the known-good baseline.

Tier 2 — Regression detection: Every 5 minutes, a SQL Server Agent job compared recent average duration against the rolling 7-day baseline for each tracked query. If the ratio exceeded 2.5x, it flagged a potential regression.

Tier 3 — Autonomous remediation: The system retrieved the last known good plan from Query Store and forced it with `sp_query_store_force_plan`, then validated that the forced plan was being used on subsequent executions.

The regression detection and auto-forcing procedure:

```

CREATE OR ALTER PROCEDURE Monitoring.AutoForceRegressedPlans
AS
BEGIN
    SET NOCOUNT ON;

    -- Candidates: queries whose recent avg duration exceeds 2.5x baseline
    DECLARE @QueryId BIGINT, @GoodPlanId BIGINT, @RecentMs FLOAT, @BaselineMs FLOAT;

    DECLARE cur CURSOR LOCAL FAST_FORWARD FOR
    WITH Recent AS (
        SELECT q.query_id,
               AVG(rs.avg_duration / 1000.0) AS recent_avg_ms
        FROM sys.query_store_query q
        JOIN sys.query_store_plan p
            ON p.query_id = q.query_id
        JOIN sys.query_store_runtime_stats rs
            ON rs.plan_id = p.plan_id
        WHERE rs.last_execution_time > DATEADD(MINUTE, -15, SYSUTCDATETIME())
        GROUP BY q.query_id
    ),
    Baseline AS (
        SELECT q.query_id,
               AVG(rs.avg_duration / 1000.0) AS baseline_avg_ms
        FROM sys.query_store_query q
        JOIN sys.query_store_plan p
            ON p.query_id = q.query_id
        JOIN sys.query_store_runtime_stats rs
            ON rs.plan_id = p.plan_id
        WHERE rs.last_execution_time > DATEADD(DAY, -7, SYSUTCDATETIME())
              AND rs.last_execution_time < DATEADD(MINUTE, -15, SYSUTCDATETIME())
        GROUP BY q.query_id
    )
    SELECT r.query_id,
           (SELECT TOP 1 p.plan_id
            FROM sys.query_store_plan p
            JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id
            WHERE p.query_id = r.query_id
              AND rs.last_execution_time < DATEADD(MINUTE, -15, SYSUTCDATETIME())
            ORDER BY rs.avg_duration ASC) AS good_plan_id,
           r.recent_avg_ms,
           b.baseline_avg_ms
    FROM Recent r
    JOIN Baseline b ON b.query_id = r.query_id
    WHERE r.recent_avg_ms > b.baseline_avg_ms * 2.5
      AND b.baseline_avg_ms > 1000    -- ignore sub-second noise
      AND NOT EXISTS (
        SELECT 1 FROM sys.query_store_plan fp
        WHERE fp.query_id = r.query_id AND fp.force_failure_count = 0
          AND fp.plan_id IN (
            SELECT plan_id FROM sys.query_store_query_hints
            WHERE query_id = r.query_id));

    OPEN cur;

```

```

    FETCH NEXT FROM cur INTO @QueryId, @GoodPlanId, @RecentMs, @BaselineMs;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        IF @GoodPlanId IS NOT NULL
        BEGIN
            EXEC sp_query_store_force_plan @QueryId, @GoodPlanId;

            INSERT INTO Monitoring.PlanStabilizationLog
                (QueryId, ForcedPlanId, RecentAvgMs, BaselineAvgMs,
                 ForcedAt, InitiatedBy)
            VALUES
                (@QueryId, @GoodPlanId, @RecentMs, @BaselineMs,
                 SYSUTCDATETIME(), 'autonomous-plan-stabilizer');
        END
        ELSE
        BEGIN
            -- No known-good plan: escalate for human review
            INSERT INTO Monitoring.PlanRegressionQueue
                (QueryId, RecentAvgMs, BaselineAvgMs,
                 DetectedAt, Status)
            VALUES
                (@QueryId, @RecentMs, @BaselineMs,
                 SYSUTCDATETIME(), 'pending_review');
        END
        FETCH NEXT FROM cur INTO @QueryId, @GoodPlanId, @RecentMs, @BaselineMs;
    END
    CLOSE cur;
    DEALLOCATE cur;
END;

```

A companion un-force job ran hourly: if a forced plan's `force_failure_count` climbed or its recent performance no longer beat the unforced baseline, the system called `sp_query_store_unforce_plan` and logged the outcome. This prevented stale forced plans from fossilizing into the workload – the most common failure mode the team had seen with manual plan forcing.

After 60 days in production, plan-regression incidents on the 12 tracked reporting queries dropped from an average of 3 per week to zero analyst-visible events. The `Monitoring.PlanStabilizationLog` table became the team's primary evidence artifact in incident reviews.

Case Study 4: Autonomous Capacity Scaling in a Multi-Tenant E-Commerce Platform (SQL Server on Azure)

A direct-to-consumer e-commerce company ran SQL Server on Azure SQL Database Hyperscale across three regions. Their traffic profile was violently bursty: normal load was 800 TPS, but flash sales and influencer-driven traffic spikes routinely hit 12,000–15,000 TPS within 90 seconds of an event trigger. Manual scaling was too slow (tier changes took several

minutes and required change approval); the autonomous solution was a control loop that scaled vCores up and down via the Azure control plane and expanded Hyperscale named replicas for read offload, driven by real-time resource utilization signals.

The scaling loop was implemented as a Python daemon (running as an Azure Container Instance on a 60-second timer) that read resource signals from `sys.dm_db_resource_stats` and acted through the Azure CLI:

- → When average CPU over the last 5 minutes exceeds 75% for 60 seconds: scale vCores up one tier via `az sql db update`, and add one Hyperscale named replica for read offload.
- → When log write or data I/O waits dominate and replica count is already at the cap: do not add more replicas; instead throttle non-critical background jobs via a priority queue table.
- → When average CPU stays below 35% for 300 seconds: scale down one tier if vCores are above the floor.

The daemon adjusted the database service tier directly through the Azure control plane:

```

import pyodbc
import subprocess
import logging
import time
import json

DB_CONN      = "DRIVER={ODBC Driver 18 for SQL Server};SERVER=ecommerce-
sql.database.windows.net;DATABASE=ShopDB;Authentication=ActiveDirectoryMsi;"
RESOURCE_GROUP = "rg-ecommerce-prod"
SERVER        = "ecommerce-sql"
DATABASE      = "ShopDB"
MIN_VCORES    = 4
MAX_VCORES    = 80
SCALE_UP_CPU  = 75.0
SCALE_DOWN_CPU = 35.0
SCALE_COOLDOWN = 180 # seconds between scale events

TIERS = [4, 8, 16, 24, 32, 40, 48, 64, 80]
last_scale_time = 0

def query(sql: str):
    with pyodbc.connect(DB_CONN, timeout=10) as conn:
        cur = conn.cursor()
        cur.execute(sql)
        return cur.fetchall()

def avg_cpu_pct_5min() -> float:
    rows = query("""
        SELECT AVG(avg_cpu_percent)
        FROM sys.dm_db_resource_stats
        WHERE end_time > DATEADD(MINUTE, -5, GETUTCDATE())
    """)
    return float(rows[0][0]) if rows and rows[0][0] is not None else 0.0

def current_vcores() -> int:
    out = subprocess.run(
        ["az", "sql", "db", "show", "--resource-group", RESOURCE_GROUP,
         "--server", SERVER, "--name", DATABASE,
         "--query", "currentServiceObjectiveName", "-o", "tsv"],
        capture_output=True, text=True, check=True
    )
    name = out.stdout.strip() # e.g. "HS_Gen5_16"
    return int(name.rsplit("_", 1)[1])

def scale_to_vcores(vcores: int) -> None:
    tier = f"HS_Gen5_{vcores}"
    subprocess.run(
        ["az", "sql", "db", "update", "--resource-group", RESOURCE_GROUP,
         "--server", SERVER, "--name", DATABASE,
         "--service-objective", tier],
        check=True
    )
    logging.info(f"Scaled {DATABASE} to {tier}")

```

```

def throttle_background_jobs() -> None:
    """Pause non-critical batch work via a coordination table."""
    with pyodbc.connect(DB_CONN) as conn:
        cur = conn.cursor()
        cur.execute("""
            MERGE ops.scaling_signals AS tgt
            USING (SELECT 'background_throttle' AS signal_key) AS src
            ON tgt.signal_key = src.signal_key
            WHEN MATCHED THEN
                UPDATE SET signal_value = 'true',
                           updated_at = SYSUTCDATETIME()
            WHEN NOT MATCHED THEN
                INSERT (signal_key, signal_value, updated_at)
                VALUES ('background_throttle', 'true', SYSUTCDATETIME());
        """)
        conn.commit()
        logging.info("Background job throttle signal set")

def avg_log_write_pct_5min() -> float:
    rows = query("""
        SELECT AVG(avg_log_write_percent)
        FROM sys.dm_db_resource_stats
        WHERE end_time > DATEADD(MINUTE, -5, GETUTCDATE())
    """)
    return float(rows[0][0]) if rows and rows[0][0] is not None else 0.0

def autoscale_loop():
    global last_scale_time
    now = time.time()
    if (now - last_scale_time) < SCALE_COOLDOWN:
        logging.debug("Inside cooldown window – skipping scale evaluation")
        return

    cpu      = avg_cpu_pct_5min()
    log_write = avg_log_write_pct_5min()
    current   = current_vcores()
    idx       = TIERS.index(current)
    logging.info(f"CPU={cpu:.1f}%, log_write={log_write:.1f}%, vcores={current}")

    if cpu > SCALE_UP_CPU:
        if log_write > 90.0:
            # Log throughput is the bottleneck – more vCores will not help;
            # shed background load instead
            logging.warning(
                "High CPU but log write saturated – throttling background jobs"
            )
            throttle_background_jobs()
        elif idx < len(TIERS) - 1:
            scale_to_vcores(TIERS[idx + 1])
            last_scale_time = now
    elif cpu < SCALE_DOWN_CPU and idx > 0 and current > MIN_VCORES:
        scale_to_vcores(TIERS[idx - 1])

```

```
last_scale_time = now

if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)
    while True:
        autoscale_loop()
        time.sleep(60)
```

The most significant lesson from this deployment was the interaction between tier-change latency and connection pool saturation. A vCore scale-up on Hyperscale required approximately 45–90 seconds to complete. During that window, if the spike was sharp enough, the application's connection pool would saturate and begin queuing. The mitigation was to throttle non-critical background jobs immediately — before the scale completed — so that queued connections had somewhere to land as soon as capacity arrived. The sequence mattered: shed load first, then scale.

The second lesson was that the `SCALE_COOLDOWN` window needed to be asymmetric: 60 seconds for scale-up (to react quickly), 300 seconds for scale-down (to avoid thrashing when traffic declines slowly). A symmetric cooldown caused premature scale-down during the trailing edge of flash sales, requiring an immediate re-scale that itself triggered the cooldown again.

Case Study 5: Autonomous Compliance Auditing and Remediation in a Financial Services SQL Server Deployment

A regional bank's data engineering team operated SQL Server 2022 databases containing customer PII, transaction records, and loan origination data, subject to audit requirements from both their internal compliance framework and external regulatory guidance. Their pain point was not detection — they had extensive logging — but remediation latency: when a compliance violation was identified (a table without row-level security, an unencrypted backup path, a login with excessive `GRANT` permissions), the ticket-to-fix cycle averaged 11 business days due to approval workflows and DBA bandwidth.

The autonomous compliance agent shortened this cycle by classifying violations into three tiers:

- → **Tier 1 (Auto-Remediate)**: Violations where the corrective action is reversible, low-risk, and pre-approved by the security team (e.g., revoking a `PUBLIC` schema execute permission, adding a column-level data mask).
- → **Tier 2 (Auto-Draft + Human Approve)**: Violations where the fix is known but the blast radius requires sign-off (e.g., adding a row-level security policy on tables with active sessions, revoking a role with 50+ dependent objects).

- → **Tier 3 (Alert Only)**: Violations requiring architectural judgment (e.g., an entire database missing TDE, a linked server to an unapproved external system).

The autonomous remediation procedures for Tier 1 violations:


```

-- Tier 1 auto-remediation: revoke PUBLIC schema permissions
CREATE OR ALTER PROCEDURE Compliance.AutoRemediatePublicExposure
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SchemaName SYSNAME, @TableName SYSNAME, @PrivName SYSNAME, @Sql
    NVARCHAR(MAX);

    DECLARE cur CURSOR LOCAL FAST_FORWARD FOR
        SELECT SchemaName, TableName, PrivilegeName
        FROM Compliance.PublicSchemaOverexposure;

    OPEN cur;
    FETCH NEXT FROM cur INTO @SchemaName, @TableName, @PrivName;
    WHILE @@FETCH_STATUS = 0
    BEGIN
        SET @Sql = N'REVOKE ' + QUOTENAME(@PrivName)
            + N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName)
            + N' FROM PUBLIC';

        -- Audit before execution
        INSERT INTO Compliance.RemediationLog
            (ViolationType, ObjectIdentifier, ActionSql,
            Tier, RemediatedAt, InitiatedBy)
        VALUES
            ('public_schema_overexposure',
            @SchemaName + N'.' + @TableName,
            @Sql, 1, SYSUTCDATETIME(), 'autonomous-compliance-agent');

        EXEC sp_executesql @Sql;
        FETCH NEXT FROM cur INTO @SchemaName, @TableName, @PrivName;
    END
    CLOSE cur;
    DEALLOCATE cur;
END;

-- Enable row-level security on regulated tables
-- (Tier 1 only when no active sessions are touching the table)
CREATE OR ALTER PROCEDURE Compliance.AutoEnableRLSSafe
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @SchemaName SYSNAME, @TableName SYSNAME, @Active INT, @Sql
    NVARCHAR(MAX);

    DECLARE cur CURSOR LOCAL FAST_FORWARD FOR
        SELECT SchemaName, TableName
        FROM Compliance.TablesMissingRLS;

    OPEN cur;
    FETCH NEXT FROM cur INTO @SchemaName, @TableName;

```

```

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Only auto-enable RLS if no active requests are touching this table
    SELECT @Active = COUNT(*)
    FROM sys.dm_exec_requests r
    JOIN sys.dm_tran_locks l
        ON l.request_session_id = r.session_id
    JOIN sys.partitions p
        ON p.hobt_id = l.resource_associated_entity_id
    JOIN sys.objects o
        ON o.object_id = p.object_id
    JOIN sys.schemas s
        ON s.schema_id = o.schema_id
    WHERE s.name = @SchemaName
        AND o.name = @TableName
        AND r.status = 'running';

    IF @Active = 0
    BEGIN
        SET @Sql = N'ALTER SECURITY POLICY '
            + QUOTENAME(@SchemaName + N'_rls_policy')
            + N' ADD FILTER PREDICATE
Compliance.fn_SecurityPredicate(UserName)'
            + N' ON ' + QUOTENAME(@SchemaName) + N'.' +
QUOTENAME(@TableName);

        INSERT INTO Compliance.RemediationLog
            (ViolationType, ObjectIdentifier, ActionSql,
            Tier, RemediatedAt, InitiatedBy)
        VALUES
            ('missing_rls', @SchemaName + N'.' + @TableName,
            @Sql, 1, SYSUTCDATETIME(), 'autonomous-compliance-agent');

        EXEC sp_executesql @Sql;
    END
    ELSE
    BEGIN
        -- Escalate to Tier 2: table is active, needs human approval
        INSERT INTO Compliance.RemediationQueue
            (ViolationType, ObjectIdentifier, RecommendedSql,
            Tier, QueuedAt, ApprovalStatus)
        VALUES
            ('missing_rls', @SchemaName + N'.' + @TableName,
            N'ALTER SECURITY POLICY '
            + QUOTENAME(@SchemaName + N'_rls_policy')
            + N' ADD FILTER PREDICATE
Compliance.fn_SecurityPredicate(UserName)'
            + N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName),
            2, SYSUTCDATETIME(), 'pending');
    END
    FETCH NEXT FROM cur INTO @SchemaName, @TableName;
END
CLOSE cur;

```

```
DEALLOCATE cur;  
END;
```

The Tier 2 workflow connected to the bank's internal approval platform via a webhook: once a record appeared in `Compliance.RemediationQueue` with `ApprovalStatus = 'pending'`, a notification triggered a Teams approval message with a one-click approval button. Approved records updated `ApprovalStatus = 'approved'` and triggered the agent to execute the stored `RecommendedSql`. The average Tier 2 remediation time dropped from 11 business days to 6.2 hours — driven almost entirely by the pre-drafted SQL that security reviewers could read, approve, or reject without any DBA ticket creation or back-and-forth.

The most important architectural lesson from this case study was the value of the `RemediationLog` table as a compliance artifact. External auditors could query the log to verify that violations were detected and remediated within specific SLA windows, with full SQL provenance. The autonomous system was not just faster — it was more auditable than the manual process, which relied on ticket comments and email threads that were difficult to correlate with actual database state changes.

A secondary lesson was the necessity of scoping Tier 1 carefully. On the third week of deployment, the agent auto-revoked an `EXECUTE` permission on a stored procedure that a batch job had relied on for 4 years — a procedure that was not documented anywhere except a comment in the source code of a decommissioned internal tool. The batch job failed silently overnight. This prompted a refinement: Tier 1 actions now require the affected object to appear in a pre-approved object registry (`Compliance.Tier1EligibleObjects`) before autonomous action can proceed. The registry is maintained by DBAs and reviewed quarterly.

Synthesis: Patterns Across All Five Deployments

Examining the five case studies together, several design patterns appear consistently in autonomous DBA systems that survive production contact:

1. Layered decision gates, not binary triggers

Every deployment used a multi-stage decision process before acting. Detection triggered evaluation; evaluation triggered either autonomous action or escalation; action triggered verification. No deployment allowed a single metric threshold to directly trigger a destructive or irreversible change. The trading firm's three-poll failure threshold, the healthcare platform's 2.5x regression ratio, and the compliance agent's Tier 1/2/3 classification all embody this principle.

2. Audit-first, act-second

In every case, the system wrote an audit record before executing a change — not after. This ordering matters: if the execution fails mid-operation, the audit record still exists and contains the intended action, enabling forensic reconstruction. The SQL Server index agent, the compliance remediation procedure, and the plan stabilizer all follow this pattern.

3. Asymmetric conservatism

Autonomous systems consistently applied different thresholds for action versus inaction. The Kubernetes autoscaler used a 60-second cooldown for scale-up and a 300-second cooldown for scale-down. The compliance agent applied stricter preconditions to Tier 1 than to Tier 2 escalations (because Tier 1 executes without human review). Systems that used symmetric thresholds oscillated or over-acted.

4. Human override surfaces, not human override requirements

All five systems provided mechanisms for humans to override or inspect autonomous decisions without requiring humans to be in the critical path of every decision. The `plan_override_registry`, the `remediation_queue`, the `tier1_eligible_objects` registry, and the Slack approval workflow all represent designed override surfaces — places where human judgment can inject itself without breaking the automation loop.

5. Production failure as design input

The silent batch job failure from the compliance case, the replica provisioning lag from the Kubernetes autoscaler, and the schema-change-versus-data-drift distinction from the healthcare platform all led to architectural refinements that would not have been discoverable through any amount of upfront design. Each case study team built a feedback loop from production incidents back into the autonomous system's decision logic — not just into runbooks.

Key Takeaways

- → **Autonomous systems earn trust through auditability, not just performance:** every production deployment in this chapter succeeded in gaining organizational acceptance partly because the system's decisions were fully logged with provenance, enabling retrospective review by both engineers and external auditors.
- → **Detection signal quality determines autonomous system reliability more than remediation sophistication:** the trading firm's heartbeat table outperformed the AG replica-state DMVs as a failure signal; the healthcare platform's regression ratio proved more stable than raw execution time comparisons; signal design deserves as much engineering attention as the remediation logic.

- → **The boundary between Tier 1 (auto-act) and Tier 2 (human-approve) actions must be defined by blast radius and reversibility, not by technical complexity:** some technically simple actions (revoking a permission) have large blast radii; some technically complex actions (online index rebuild) are easily reversible. Conflating complexity with risk leads to either under-automation or over-automation.
- → **Cooldown windows and lag guards are not optional safety theater — they are architectural requirements:** every case study that skipped or underestimated these guards experienced oscillation, thrashing, or a preventable failure before adding them back; asymmetric cooldowns (faster to engage protective action, slower to disengage it) consistently outperformed symmetric ones.
- → **The object registry pattern — maintaining a pre-approved list of objects eligible for autonomous action — is the practical solution to the scope creep problem in autonomous remediation systems:** without it, autonomous agents inevitably encounter edge cases (undocumented dependencies, legacy objects, objects shared across teams) that cause silent failures; with it, the blast radius of any autonomous action is bounded and reviewable before deployment.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 38: Building Your Own Autonomous DBA Framework

The frameworks sold by cloud vendors as "autonomous databases" are black boxes — they observe, decide, and act within walls you cannot see, modify, or extend. For engineering teams running SQL Server on their own infrastructure, in hybrid clouds, or across multi-tenant platforms, the only viable path to genuine autonomy is a purpose-built framework: one that you design, own, and can interrogate at every layer. This chapter closes the loop on everything covered so far by showing you how to assemble a complete Autonomous DBA Framework (ADF) — the signal collection layer, the decision engine, the action executor, the feedback loop, and the governance model that keeps the whole system from acting on bad data. What you build here is not a monitoring dashboard with alerts. It is a control system.

38.1 Framework Architecture: The Four-Plane Model

Every production-grade autonomous system separates concerns into distinct operational planes. Conflating them is the most common reason autonomous frameworks fail in practice — the decision engine starts making choices based on stale telemetry, or the action executor fires while governance checks are disabled. The four planes are: **Observe**, **Decide**, **Act**, and **Verify**.

Observe is the telemetry plane. It continuously collects metrics, query fingerprints, wait event data, replication lag, storage growth curves, connection pool saturation, and slow query logs. The observability plane must be stateful — not just streaming point-in-time values, but maintaining windowed aggregates, trend slopes, and anomaly baselines.

Decide is the inference plane. It consumes normalized signals from the Observe plane and produces action proposals with confidence scores and rollback parameters. This plane contains the rule engine, the ML scoring layer if one is deployed, and the external LLM integration for natural-language reasoning on ambiguous states.

Act is the execution plane. It receives approved action proposals, serializes them into safe execution windows, calls the actual database APIs, Kubernetes operators, or shell scripts, and emits structured execution logs.

Verify is the validation plane. After every action, it re-queries the affected system to confirm that the intended effect occurred, measures whether the root signal improved, and feeds the outcome back into the Decide plane as training signal.

```

graph LR
    O["OBSERVE  
Telegraf, sys.dm_*, Extended Events"] -->|signals| D["DECIDE  
rule engine, ML, LLM"]
    D -->|proposals| A["ACT  
executor, Terraform, k8s ops"]
    A -->|outcomes| V["VERIFY  
diff expected vs actual"]
    V -->|feedback| D

```

The four-plane loop: signals flow forward from Observe through Decide and Act, while Verify closes the loop by feeding measured outcomes back into the Decide plane.

The critical design rule: **no plane writes directly to another plane's data store**. Each plane has its own schema or topic namespace. Cross-plane communication happens through a message bus — Kafka, NATS, or even a dedicated schema acting as a persistent queue. This prevents race conditions and makes the entire system auditable.

38.2 Building the Signal Collection Layer for SQL Server

The signal layer must be purpose-built for autonomous decision-making, not repurposed from a monitoring stack. The difference is semantic richness. A monitoring stack answers "is something wrong?" The ADF signal layer answers "what specifically changed, how fast, and what does historical context say about it?"

For SQL Server, the primary sources are Query Store, `sys.dm_exec_requests`, `sys.dm_hadr_database_replica_states`, `sys.dm_db_index_physical_stats`, `sys.dm_os_performance_counters`, and Extended Events sessions. These are extended with a custom collector that computes derived signals:

```

-- Autonomous signal collection: combined health snapshot for decision engine
-- Run via SQL Agent job every 60 seconds or captured by external collector

CREATE OR ALTER VIEW adf.SqlSignalSnapshot AS
SELECT
    GETUTCDATE() AS collected_at,
    -- Query pressure
    (SELECT COUNT(*) FROM sys.dm_exec_requests r
     WHERE r.wait_type LIKE 'LCK%') AS active_lock_waiters,
    (SELECT COUNT(*) FROM sys.dm_exec_requests r
     WHERE r.status = 'running'
       AND r.total_elapsed_time > 30000) AS long_running_queries,
    -- AG replication lag (ms)
    (SELECT ISNULL(MAX(secondary_lag_seconds * 1000), 0)
     FROM sys.dm_hadr_database_replica_states
     WHERE is_primary_replica = 0) AS
max_replication_lag_ms,
    -- Buffer cache hit ratio (from ring buffer counters)
    (SELECT ROUND(
        cntnr_value * 100.0 /
        NULLIF((SELECT cntnr_value FROM sys.dm_os_performance_counters
                 WHERE counter_name = 'Buffer cache hit ratio base'
                   AND object_name LIKE '%Buffer Manager%'), 0), 2)
     FROM sys.dm_os_performance_counters
     WHERE counter_name = 'Buffer cache hit ratio'
       AND object_name LIKE '%Buffer Manager%') AS buffer_hit_ratio,
    -- Plan cache pressure
    (SELECT COUNT(*) FROM sys.dm_exec_cached_plans
     WHERE usecounts = 1
       AND objtype = 'Adhoc') AS single_use_plan_count,
    -- Connection saturation
    (SELECT ROUND(COUNT(*) * 100.0 /
        (SELECT value_in_use FROM sys.configurations
         WHERE name = 'max connections'), 2)
     FROM sys.dm_exec_sessions
     WHERE is_user_process = 1) AS
connection_saturation_pct,
    -- TempDB pressure
    (SELECT ROUND(SUM(unallocated_extent_page_count) * 8.0 /
        1024, 2)
     FROM sys.dm_db_file_space_usage
     WHERE database_id = 2) AS tempdb_free_mb;

```

These snapshots are written to a time-series store — InfluxDB, Azure Monitor, or a dedicated `adf.signal_history` partitioned table — every 60 seconds. The Decide plane reads windowed aggregates, not raw point values. A single spike in `active_lock_waiters` is noise. A 10-minute rolling average above 15 with an upward slope of 2 per minute is a signal that requires action.

The slope calculation, critical for distinguishing transient spikes from sustained degradation, is computed in the collector:

```

# signal_processor.py – slope-based anomaly gate
# Runs inside the ADF collector pod; ingests raw snapshot rows

import numpy as np
from dataclasses import dataclass
from typing import List

@dataclass
class SignalWindow:
    metric_name: str
    values: List[float]      # ordered oldest → newest, 10-minute window at 60s
    intervals
    threshold: float
    slope_threshold: float   # units/minute to trigger concern

def evaluate_signal(window: SignalWindow) -> dict:
    n = len(window.values)
    if n < 3:
        return {"signal": window.metric_name, "action_required": False, "reason":
"insufficient_data"}

    x = np.arange(n, dtype=float)
    slope, intercept = np.polyfit(x, window.values, 1)
    slope_per_minute = slope # values collected every 60s, so slope is already per-
minute

    current_value = window.values[-1]
    rolling_mean = float(np.mean(window.values))

    action_required = (
        current_value > window.threshold
        and slope_per_minute > window.slope_threshold
    )

    return {
        "signal": window.metric_name,
        "current_value": round(current_value, 4),
        "rolling_mean": round(rolling_mean, 4),
        "slope_per_minute": round(slope_per_minute, 4),
        "threshold": window.threshold,
        "slope_threshold": window.slope_threshold,
        "action_required": action_required,
        "confidence": round(min(abs(slope_per_minute / window.slope_threshold),
1.0), 3)
    }

# Example usage
lock_waiter_window = SignalWindow(
    metric_name="active_lock_waiters",
    values=[4, 5, 6, 9, 12, 15, 18, 22, 26, 31],
    threshold=10.0,
    slope_threshold=2.0
)

```

```
result = evaluate_signal(lock_waiter_window)
# {"signal": "active_lock_waiters", "current_value": 31.0,
#  "rolling_mean": 14.8, "slope_per_minute": 3.0, "action_required": True,
#  "confidence": 1.0}
```

The `confidence` score becomes the first field read by the Decide plane's priority queue. Signals with confidence below 0.4 are logged but never promoted to action proposals without human review.

38.3 Designing the Decision Engine: Rules, Scoring, and LLM Escalation

The decision engine is where most ADF implementations fail. Teams either build a flat list of if-then rules that becomes unmaintainable, or they over-rely on a black-box ML model that cannot explain its reasoning to a post-incident review board. The right architecture is a **three-tier decision stack**: deterministic rules at the base, statistical scoring in the middle, and LLM reasoning at the escalation tier.

Tier 1 — Deterministic Rules handle everything that has a known, safe, reversible response. Index fragmentation remediation, connection pool scaling, index creation on clearly missing indexes (where the planner's estimated rows scanned exceeds 50% of the table and a covering index is provably applicable), and failover initiation when primary health checks fail for more than 90 seconds. These rules fire with 100% confidence and no external consultation.

Tier 2 — Statistical Scoring handles situations where the correct action depends on context that changes over time: query plan regression detection, adaptive memory grant sizing, AG secondary promotion decisions under partial redo lag. A lightweight gradient boosted model trained on historical incident/resolution pairs scores each action proposal from 0.0 to 1.0. Proposals scoring above 0.8 proceed automatically. Proposals scoring 0.5–0.8 require Tier 3 review.

Tier 3 — LLM Escalation handles novel states that neither the rule base nor the scoring model has seen before. The LLM receives a structured context packet and proposes an action with reasoning. A human operator reviews LLM-proposed actions during business hours; outside business hours, the system can execute with a `human_override_window` flag set and a rollback procedure pre-staged.

The decision engine's core routing logic:

```

# decision_engine.py – action proposal router
# Consumes from the signal bus, emits to the action queue

import json
import logging
from enum import Enum
from typing import Optional
from dataclasses import dataclass, field

logger = logging.getLogger("adf.decide")

class ActionTier(Enum):
    DETERMINISTIC = 1
    STATISTICAL = 2
    LLM_ESCALATION = 3
    HUMAN_REQUIRED = 4

@dataclass
class ActionProposal:
    signal_name: str
    proposed_action: str
    action_params: dict
    confidence: float
    tier: ActionTier
    rollback_procedure: str
    estimated_impact: str          # "low" | "medium" | "high"
    approved: bool = False
    llm_reasoning: Optional[str] = None
    tags: list = field(default_factory=list)

DETERMINISTIC_RULES = {
    "active_lock_waiters": {
        "threshold": 20,
        "action": "identify_and_kill_blocking_head",
        "params_query": "SELECT TOP 1 blocking_session_id AS blocking_pid FROM
sys.dm_exec_requests WHERE blocking_session_id <> 0",
        "rollback": "no_rollback_required",
        "impact": "low"
    },
    "fragmented_indexes": {
        "threshold": 5,
        "action": "rebuild_fragmented_indexes",
        "params_query": "SELECT TOP 3 OBJECT_NAME(object_id) AS idx_owner FROM
sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED') WHERE
avg_fragmentation_in_percent > 30",
        "rollback": "no_rollback_required",
        "impact": "low"
    },
    "connection_saturation_pct": {
        "threshold": 85.0,
        "action": "scale_connection_pool_size",
        "params": {"increment_pct": 20, "max_pool_size": 500},
        "rollback": "revert_pool_config_to_previous",
    }
}

```

```

        "impact": "medium"
    },
    "max_replication_lag_bytes": {
        "threshold": 536870912,    # 512 MB
        "action": "pause_non_critical_batch_writes",
        "params": {"tag": "batch_etl", "pause_minutes": 10},
        "rollback": "resume_batch_writes",
        "impact": "medium"
    }
}

def route_signal(signal_result: dict, ml_scorer=None, llm_client=None) ->
ActionProposal:
    name = signal_result["signal"]
    value = signal_result["current_value"]
    confidence = signal_result["confidence"]

    # Tier 1: Deterministic
    if name in DETERMINISTIC_RULES:
        rule = DETERMINISTIC_RULES[name]
        if value >= rule["threshold"]:
            proposal = ActionProposal(
                signal_name=name,
                proposed_action=rule["action"],
                action_params=rule.get("params", {}),
                confidence=confidence,
                tier=ActionTier.DETERMINISTIC,
                rollback_procedure=rule["rollback"],
                estimated_impact=rule["impact"],
                approved=True    # Deterministic rules auto-approve
            )
            logger.info(f"TIER1 auto-approved: {proposal.proposed_action} for {name}
={value}")
            return proposal

    # Tier 2: Statistical
    if ml_scorer and confidence >= 0.4:
        ml_score = ml_scorer.score(signal_result)
        if ml_score >= 0.8:
            proposal = ActionProposal(
                signal_name=name,
                proposed_action=ml_scorer.recommend_action(signal_result),
                action_params=ml_scorer.recommend_params(signal_result),
                confidence=ml_score,
                tier=ActionTier.STATISTICAL,
                rollback_procedure=ml_scorer.rollback_procedure(signal_result),
                estimated_impact="medium",
                approved=True
            )
            logger.info(f"TIER2 auto-approved (score={ml_score}):
{proposal.proposed_action}")
            return proposal

```

```

# Tier 3: LLM Escalation
if llm_client and confidence >= 0.5:
    reasoning = llm_client.reason(signal_result)
    proposal = ActionProposal(
        signal_name=name,
        proposed_action=reasoning["proposed_action"],
        action_params=reasoning["params"],
        confidence=reasoning["confidence"],
        tier=ActionTier.LLM_ESCALATION,
        rollback_procedure=reasoning["rollback"],
        estimated_impact=reasoning["impact"],
        approved=False, # Requires human or after-hours policy check
        llm_reasoning=reasoning["reasoning_text"]
    )
    logger.warning(f"TIER3 LLM proposal staged for approval:
{proposal.proposed_action}")
    return proposal

# No action – insufficient confidence
return ActionProposal(
    signal_name=name,
    proposed_action="no_action",
    action_params={},
    confidence=confidence,
    tier=ActionTier.HUMAN_REQUIRED,
    rollback_procedure="n/a",
    estimated_impact="unknown",
    approved=False,
    tags=["low_confidence", "manual_review"]
)

```

The LLM integration for Tier 3 uses a structured prompt template that includes: the signal name and value, the last five actions taken for similar signals, the current time window (business hours vs. maintenance window), and an explicit instruction to produce a JSON-structured response with `proposed_action`, `params`, `rollback`, `impact`, and `reasoning_text` fields. Never pass raw database contents to an external LLM. Pass only anonymized metric values and signal names.

For SQL Server environments running AG-backed multi-site deployments, the decision engine must include a **quorum gate** — before any action that touches replication topology (promoting a secondary, pausing log shipping, changing availability mode), the engine checks that it holds a majority vote across AG nodes. This prevents split-brain decision scenarios where two ADF instances simultaneously propose conflicting topology changes.

```

-- Quorum gate check: confirm majority AG nodes are reachable before topology
actions
-- Returns 1 if safe to proceed, 0 if quorum is insufficient

CREATE OR ALTER FUNCTION adf.fn_QuorumSafe()
RETURNS BIT
AS
BEGIN
    DECLARE @total_nodes INT, @healthy_nodes INT, @quorum_safe BIT;

    SELECT @total_nodes = COUNT(*)
    FROM sys.availability_replicas;

    SELECT @healthy_nodes = COUNT(*)
    FROM sys.dm_hadr_availability_replica_states ars
    JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
    WHERE ars.is_local = 0
        AND ars.connected_state = 1    -- CONNECTED
        AND ars.operational_state = 2; -- ONLINE

    -- Include local node
    SET @healthy_nodes = @healthy_nodes + 1;

    SET @quorum_safe = CASE
        WHEN @healthy_nodes > (@total_nodes / 2) THEN 1
        ELSE 0
    END;

    RETURN @quorum_safe;
END;

-- Usage inside action gate
IF adf.fn_QuorumSafe() = 0
BEGIN
    INSERT INTO adf.ActionBlockLog (blocked_at, reason, action_attempted)
    VALUES (GETUTCDATE(), 'quorum_insufficient', 'topology_change');

    RAISERROR('ADF: Topology action blocked – quorum not satisfied', 16, 1);
    RETURN;
END

```

38.4 The Action Executor: Safe Execution, Blast Radius Limits, and Rollback Contracts

An action executor without blast radius controls is not an autonomous system — it is a liability. Every action the executor takes must be bounded by four contracts: **scope** (which objects are touched), **reversibility** (can this be undone within 60 seconds), **concurrency**

limit (no more than N actions active simultaneously across all targets), and **cooldown** (after any action on a target, that target is locked for a configurable interval before another action can fire).

The executor is implemented as a finite state machine. Each action proposal enters as `PENDING`, transitions to `STAGED` during parameter validation, to `EXECUTING` during the actual database call, to `COMPLETED` or `FAILED`, and finally to `VERIFIED` after the Verify plane confirms the outcome. Any transition to `FAILED` immediately triggers the rollback contract.

The most operationally important actions for SQL Server environments — and the ones where incorrect execution causes the most damage — are index creation, configuration parameter changes, and connection pool resizing. All three must execute through change-controlled wrappers:


```

-- ADF executor wrapper: safe online index creation with blast radius guard
-- Uses ONLINE = ON to minimize blocking; defers if table exceeds size threshold

CREATE OR ALTER PROCEDURE adf.exec_CreateIndexSafe
    @TableName      NVARCHAR(256),
    @Columns         NVARCHAR(512),
    @IndexName       NVARCHAR(128),
    @ProposalId      UNIQUEIDENTIFIER
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @TableSizeMB    DECIMAL(18,2);
    DECLARE @IndexSQL       NVARCHAR(MAX);
    DECLARE @StartTime      DATETIME2 = SYSUTCDATETIME();
    DECLARE @CooldownCount  INT;

    -- Blast radius check: refuse if table > 100 GB without explicit override
    SELECT @TableSizeMB = SUM(a.total_pages) * 8.0 / 1024
    FROM sys.tables t
    JOIN sys.indexes i  ON t.object_id = i.object_id
    JOIN sys.partitions p ON i.object_id = p.object_id AND i.index_id = p.index_id
    JOIN sys.allocation_units a ON p.partition_id = a.container_id
    WHERE t.name = PARSENAME(@TableName, 1)
        AND OBJECT_SCHEMA_NAME(t.object_id) = ISNULL(PARSENAME(@TableName, 2),
SCHEMA_NAME());

    IF @TableSizeMB > 102400
    BEGIN
        UPDATE adf.ActionProposals
        SET status = 'BLOCKED',
            BlockReason = 'table_exceeds_blast_radius_100gb',
            UpdatedAt = SYSUTCDATETIME()
        WHERE ProposalId = @ProposalId;

        RAISERROR('ADF: Index build blocked – table %s is %.0f MB (>100GB limit)',
16, 1,
            @TableName, @TableSizeMB);
        RETURN;
    END

    -- Cooldown check: no actions on this table in the last 30 minutes
    SELECT @CooldownCount = COUNT(*)
    FROM adf.ActionHistory
    WHERE TargetObject = @TableName
        AND CompletedAt > DATEADD(MINUTE, -30, SYSUTCDATETIME())
        AND Status IN ('COMPLETED', 'FAILED');

    IF @CooldownCount > 0
    BEGIN
        UPDATE adf.ActionProposals
        SET Status = 'DEFERRED', BlockReason = 'target_in_cooldown'
        WHERE ProposalId = @ProposalId;
    END

```

```

        RAISERROR('ADF: Action deferred – %s is in cooldown window', 16, 1,
@TableName);
        RETURN;
    END

-- Build and execute with ONLINE = ON to avoid blocking production reads/writes
SET @IndexSQL = N'CREATE INDEX ' + QUOTENAME(@IndexName)
                + N' ON ' + @TableName
                + N' ( ' + @Columns + N')'
                + N' WITH (ONLINE = ON, SORT_IN_TEMPDB = ON);';

UPDATE adf.ActionProposals
SET Status = 'EXECUTING', StartedAt = SYSUTCDATETIME()
WHERE ProposalId = @ProposalId;

BEGIN TRY
    EXEC sp_executesql @IndexSQL;

    UPDATE adf.ActionProposals
    SET Status = 'COMPLETED', CompletedAt = SYSUTCDATETIME()
    WHERE ProposalId = @ProposalId;

    INSERT INTO adf.ActionHistory
        (ProposalId, TargetObject, ActionType, ExecutedAt, CompletedAt, Status)
    VALUES
        (@ProposalId, @TableName, 'CREATE_INDEX_ONLINE',
        @StartTime, SYSUTCDATETIME(), 'COMPLETED');
END TRY
BEGIN CATCH
    UPDATE adf.ActionProposals
    SET Status = 'FAILED',
        FailureReason = ERROR_MESSAGE(),
        CompletedAt = SYSUTCDATETIME()
    WHERE ProposalId = @ProposalId;

    INSERT INTO adf.ActionHistory
        (ProposalId, TargetObject, ActionType, ExecutedAt, CompletedAt, Status,
FailureReason)
    VALUES
        (@ProposalId, @TableName, 'CREATE_INDEX_ONLINE',
        @StartTime, SYSUTCDATETIME(), 'FAILED', ERROR_MESSAGE());

    THROW;
END CATCH
END;

```

The cooldown and blast radius parameters are not hardcoded constants — they live in an `adf.executor_policy` table that can be updated without redeploying the framework. Different tables, different schemas, and different environments (production vs. staging) carry

different policies. A table tagged `tier=critical` might have a 4-hour cooldown and a 10 GB blast radius limit. A table tagged `tier=archive` might have a 5-minute cooldown and no size limit at all.

```
CREATE TABLE adf.ExecutorPolicy (  
    PolicyId          INT IDENTITY PRIMARY KEY,  
    TargetPattern     NVARCHAR(256) NOT NULL,  
    TierTag           NVARCHAR(64)  NOT NULL DEFAULT 'standard',  
    CooldownMins      INT           NOT NULL DEFAULT 30,  
    MaxSizeGB         INT           NULL,  
    AllowAuto         BIT           NOT NULL DEFAULT 1,  
    Notes             NVARCHAR(512) NULL,  
    UpdatedAt         DATETIME2     DEFAULT SYSUTCDATETIME()  
);  
  
INSERT INTO adf.ExecutorPolicy (TargetPattern, TierTag, CooldownMins, MaxSizeGB,  
    AllowAuto)  
VALUES  
    ('orders',          'critical', 240, 10, 0),  
    ('inventory[_]%',   'critical', 120, 25, 1),  
    ('archive[_]%',     'archive', 5, NULL, 1),  
    ('%',               'standard', 30, 100, 1); -- catch-all
```

The executor resolves the applicable policy at runtime by scanning `executor_policy` rows in specificity order (longest matching pattern wins) before every action, not at framework startup. This means a DBA can tighten policy on a table that is about to receive a large load event without restarting the framework.

38.5 The Verify Plane: Closing the Feedback Loop

An autonomous system that does not verify its own actions is no different from a script run by a cron job. The Verify plane exists to answer three questions after every action: Did the action complete as intended? Did the root signal improve? Did any collateral signal degrade?

Verification is time-boxed. Each action type carries a `verify_window_seconds` value — how long the system waits before re-querying the target signals. An index creation on a small table verifies in 30 seconds. A configuration parameter change that requires a connection pool restart might carry a 5-minute verify window. An `ALTER INDEX ... REBUILD` on a large table might not show reduced fragmentation in `sys.dm_db_index_physical_stats` for 15 minutes, so its verify window is set accordingly.

The verification record drives the learning loop. Every verified outcome — whether the action succeeded, failed, or produced an unexpected side effect — is written back to the `adf.outcome_history` table. The statistical scoring model in Tier 2 of the decision engine is retrained weekly against this table.

```

-- Verify plane: post-action outcome recorder
CREATE TABLE adf.OutcomeHistory (
    OutcomeId          BIGINT IDENTITY PRIMARY KEY,
    ProposalId         UNIQUEIDENTIFIER NOT NULL,
    SignalName         NVARCHAR(128)    NOT NULL,
    SignalValueBefore   DECIMAL(18,4),
    SignalValueAfter    DECIMAL(18,4),
    CollateralSignals   NVARCHAR(MAX),   -- JSON snapshot
    ActionEffective     BIT,
    SideEffectDetected  BIT,
    SideEffectDetail    NVARCHAR(512),
    VerifiedAt          DATETIME2       DEFAULT SYSUTCDATETIME(),
    VerifyWindowSecs    INT
);

-- Verification proc: did lock waiters drop after kill_blocking_head action?
CREATE OR ALTER PROCEDURE adf.verify_LockWaiterAction
    @ProposalId        UNIQUEIDENTIFIER,
    @ValueBefore        DECIMAL(18,4)
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @ValueAfter    DECIMAL(18,4);
    DECLARE @Effective     BIT;

    SELECT @ValueAfter = COUNT(*)
    FROM sys.dm_exec_requests
    WHERE wait_type LIKE 'LCK%';

    SET @Effective = CASE
        WHEN @ValueAfter < @ValueBefore * 0.5 THEN 1
        ELSE 0
    END;

    INSERT INTO adf.OutcomeHistory
        (ProposalId, SignalName, SignalValueBefore,
         SignalValueAfter, ActionEffective, VerifiedAt, VerifyWindowSecs)
    VALUES
        (@ProposalId, 'active_lock_waiters', @ValueBefore,
         @ValueAfter, @Effective, SYSUTCDATETIME(), 30);

    IF @Effective = 0
    BEGIN
        UPDATE adf.ActionProposals
        SET Status = 'ESCALATED',
            FailureReason = CONCAT(
                'lock_waiters_not_reduced: before=', CAST(@ValueBefore AS
                NVARCHAR(20)),
                ' after=', CAST(@ValueAfter AS NVARCHAR(20)))
        WHERE ProposalId = @ProposalId;
    END

```

```
END  
END;
```

The collateral signal check is equally important. If the framework kills a blocking head process and lock waiters drop, but `long_running_queries` simultaneously spikes from 2 to 40, the side effect is flagged and the proposal is marked `ESCALATED` rather than `COMPLETED`. The Decide plane will not issue the same action again until a human clears the escalation. This prevents the framework from repeatedly applying a corrective action that is causing a different class of damage.

38.6 Governance: Policy Tables, Change Windows, and the Autonomy Dial

The framework is only as trustworthy as the governance layer that controls what it is permitted to do, when, and to what. Governance is not bureaucracy — it is the mechanism that allows the framework to earn progressive trust from the teams operating it.

The central governance object is the **autonomy dial**: a per-environment, per-action-class setting that controls whether an action fires automatically, requires same-day human approval, requires change-board approval, or is permanently blocked. It is implemented as a simple lookup table with a well-understood override hierarchy.

```

CREATE TABLE adf.AutonomyPolicy (
    PolicyId          INT IDENTITY PRIMARY KEY,
    Environment        NVARCHAR(64) NOT NULL,
    ActionClass        NVARCHAR(64) NOT NULL,
    AutonomyLevel       NVARCHAR(32) NOT NULL
    CHECK (AutonomyLevel IN
        ('full_auto', 'auto_offhours', 'approval_required', 'change_board', 'blocked')),
    OffhoursStart      TIME          NULL,
    OffhoursEnd        TIME          NULL,
    UpdatedBy          NVARCHAR(128) NULL,
    UpdatedAt          DATETIME2     DEFAULT SYSUTCDATETIME()
);

INSERT INTO adf.AutonomyPolicy
    (Environment, ActionClass, AutonomyLevel, OffhoursStart, OffhoursEnd, UpdatedBy)
VALUES
    ('production', 'index_maintenance', 'full_auto',          NULL,      NULL,
'adf_bootstrap'),
    ('production', 'query_kill',          'full_auto',          NULL,      NULL,
'adf_bootstrap'),
    ('production', 'index_management',    'auto_offhours',    '22:00', '06:00',
'adf_bootstrap'),
    ('production', 'config_change',        'approval_required', NULL,      NULL,
'adf_bootstrap'),
    ('production', 'connection_scaling',   'auto_offhours',    '22:00', '06:00',
'adf_bootstrap'),
    ('production', 'failover',             'change_board',     NULL,      NULL,
'adf_bootstrap');

```

The autonomy dial creates a predictable trust escalation path. New action classes always start at `approval_required`. After the framework has executed that action class successfully ten times in staging with no collateral effects, the team can promote it to `auto_offhours` in production. After ten successful production off-hours executions, the team can consider `full_auto` during business hours for that action class. This progression is not automated — it requires deliberate human sign-off at each step. The framework tracks the promotion history in `adf.autonomy_policy_changelog` so the evidence base for each autonomy grant is auditable.

Change windows work in conjunction with the autonomy dial. A `change_board`-level action can only execute if a corresponding approved change record exists in `adf.change_window`:

```

CREATE TABLE adf.ChangeWindow (
    WindowId          INT IDENTITY PRIMARY KEY,
    ActionClass        NVARCHAR(64) NOT NULL,
    Environment        NVARCHAR(64) NOT NULL,
    ApprovedBy         NVARCHAR(128) NOT NULL,
    WindowStart        DATETIME2 NOT NULL,
    WindowEnd          DATETIME2 NOT NULL,
    TicketRef          NVARCHAR(64),
    Consumed           BIT          DEFAULT 0,
    CreatedAt          DATETIME2    DEFAULT SYSUTCDATETIME()
);

CREATE OR ALTER FUNCTION adf.fn_IsChangeWindowOpen(
    @ActionClass NVARCHAR(64),
    @Environment NVARCHAR(64)
)
RETURNS BIT
AS
BEGIN
    DECLARE @Result BIT = 0;
    IF EXISTS (
        SELECT 1 FROM adf.ChangeWindow
        WHERE ActionClass = @ActionClass
          AND Environment = @Environment
          AND SYSUTCDATETIME() BETWEEN WindowStart AND WindowEnd
          AND Consumed = 0
    )
        SET @Result = 1;
    RETURN @Result;
END;

```

Once a change-window action executes, `consumed` is flipped to `TRUE`. A single approved change window ticket grants exactly one execution. This prevents the framework from replaying a change board-approved action multiple times under a stale approval.

38.7 Integrating the LLM Layer: Prompt Engineering for Database Reasoning

The LLM tier of the decision engine handles novelty — situations the rule base and scoring model have not seen before. It also serves a second function: producing human-readable explanations of every action the framework takes, which satisfies the audit requirement that autonomous systems be explainable to non-engineer stakeholders.

The LLM integration is not a free-form chat interface. It receives a precisely structured JSON context packet and is constrained to return a JSON response against a defined schema. Any response that does not parse cleanly against the schema is discarded, and the proposal drops to `HUMAN_REQUIRED`.

```

# llm_client.py – structured LLM interface for ADF Tier 3 decisions
# Compatible with OpenAI-compatible APIs; substitute endpoint for self-hosted models

import json
import logging
from typing import Optional
import httpx

logger = logging.getLogger("adf.llm")

SYSTEM_PROMPT = """You are a database reliability reasoning engine.
You receive anonymized metric signals from a SQL Server database cluster.
Your job is to reason about the root cause and propose ONE safe, reversible
corrective action.

CONSTRAINTS:
- Proposed actions must be executable by a database administrator without
application downtime.
- You must specify a rollback procedure for every proposed action.
- Estimated impact must be "low", "medium", or "high".
- Confidence must be a float between 0.0 and 1.0.
- Never propose dropping tables, truncating data, or revoking user permissions.
- Never propose actions affecting more than one database cluster simultaneously.

RESPONSE FORMAT (strict JSON, no markdown):
{
  "proposed_action": "<action_identifier>",
  "params": { "<key>": "<value>" },
  "rollback": "<rollback_procedure_description>",
  "impact": "low|medium|high",
  "confidence": 0.0,
  "reasoning_text": "<plain English explanation of root cause and action rationale>"
}"""

def build_context_packet(signal_result: dict, recent_actions: list, environment:
str) -> str:
    context = {
        "signal": signal_result,
        "recent_actions_last_6h": recent_actions[-5:] if recent_actions else [],
        "environment": environment,
        "current_utc_hour": __import__('datetime').datetime.utcnow().hour,
        "instruction": "Propose a corrective action for the signal above."
    }
    return json.dumps(context, default=str)

def reason(
    signal_result: dict,
    recent_actions: list,
    environment: str,
    api_url: str,
    api_key: str,
    model: str = "gpt-4o-mini",
    timeout_seconds: int = 20

```



```

) -> Optional[dict]:
    context_packet = build_context_packet(signal_result, recent_actions,
environment)

    payload = {
        "model": model,
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": context_packet}
        ],
        "temperature": 0.1,    # Low temperature: we want consistent, conservative
reasoning
        "max_tokens": 512,
        "response_format": {"type": "json_object"}
    }

    try:
        response = httpx.post(
            f"{api_url}/chat/completions",
            headers={"Authorization": f"Bearer {api_key}",
                    "Content-Type": "application/json"},
            json=payload,
            timeout=timeout_seconds
        )
        response.raise_for_status()
        raw_content = response.json()["choices"][0]["message"]["content"]
        parsed = json.loads(raw_content)

        # Schema validation
        required_keys = {"proposed_action", "params", "rollback", "impact",
                        "confidence", "reasoning_text"}
        if not required_keys.issubset(parsed.keys()):
            logger.error(f"LLM response missing required keys: {parsed.keys()}")
            return None

        if parsed["impact"] not in ("low", "medium", "high"):
            logger.error(f"LLM returned invalid impact value: {parsed['impact']}")
            return None

        confidence = float(parsed["confidence"])
        if not 0.0 <= confidence <= 1.0:
            logger.error(f"LLM returned out-of-range confidence: {confidence}")
            return None

        logger.info(f"LLM proposal: {parsed['proposed_action']} "
                    f"(confidence={confidence}, impact={parsed['impact']})")
        return parsed

    except (httpx.HTTPError, json.JSONDecodeError, KeyError, ValueError) as exc:
        logger.error(f"LLM reasoning failed: {exc}")
        return None

```

Several design choices in this integration deserve explicit attention. Temperature is set to `0.1` rather than `0.0` — pure determinism at `0.0` can produce slightly different outputs from identical inputs due to floating-point nondeterminism in the model backend, and a very low but nonzero temperature smooths this without materially affecting conservatism. The `response_format` field enforces JSON mode on the model side, which eliminates the most common failure mode of LLM API integrations: receiving a markdown-wrapped explanation instead of parseable JSON. The timeout of 20 seconds is intentionally short; if the model takes longer than 20 seconds to respond during an active incident, the framework should not wait — it should fall back to `HUMAN_REQUIRED` and page an on-call engineer.

The `reasoning_text` field from the LLM response is stored in `adf.action_proposals.llm_reasoning` and surfaced in the incident response interface. This means that during a post-incident review, a non-engineer manager can read "the system detected that lock waiter counts were growing at 3 per minute, identified query PID 42891 as the blocking head by its age and lock graph position, and proposed terminating it because doing so releases all downstream lock chains without data loss" rather than seeing a raw metric value and a procedure call. This explainability function is the most undervalued benefit of the LLM tier.

For teams operating in air-gapped environments or with strict data residency requirements, the LLM client is swappable — the `api_url` and `model` parameters accept any OpenAI-compatible endpoint. Self-hosted models running on Ollama, vLLM, or similar inference servers behind the corporate firewall can fulfill the same contract as long as they support `response_format: json_object`. The schema validation in the client catches capability gaps between different model sizes without changing any other part of the framework.

38.8 Deploying the Framework: Schema Bootstrap, Scheduler Integration, and Observability

The complete ADF schema bootstrap creates all required tables, views, and functions in a dedicated `adf` schema that is owned by a restricted service role. No application user should have write access to the `adf` schema — only the framework's service account and DBAs with explicit grant.

```

-- ADF schema bootstrap (SQL Server)
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'adf')
    EXEC('CREATE SCHEMA adf');

-- Create restricted service login (Windows auth or contained DB user)
-- EXEC sp_addlogin 'adf_service', '<strong_password>';
-- CREATE USER adf_service FOR LOGIN adf_service;
-- GRANT SELECT, INSERT, UPDATE ON SCHEMA::adf TO adf_service;

-- T-SQL has no CREATE TABLE IF NOT EXISTS; guard with an existence check instead
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ActionProposals' AND schema_id
= SCHEMA_ID('adf'))
BEGIN
    CREATE TABLE adf.ActionProposals (
        ProposalId          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWSEQUENTIALID(),
        SignalName           NVARCHAR(128)    NOT NULL,
        ProposedAction       NVARCHAR(256)    NOT NULL,
        ActionParams         NVARCHAR(MAX),    -- JSON
        Confidence           DECIMAL(5,4),
        Tier                 NVARCHAR(32),
        RollbackProcedure    NVARCHAR(512),
        EstimatedImpact      NVARCHAR(16),
        Status               NVARCHAR(32)     NOT NULL DEFAULT 'PENDING',
        Approved             BIT              NOT NULL DEFAULT 0,
        ApprovedBy           NVARCHAR(128),
        BlockReason          NVARCHAR(512),
        FailureReason        NVARCHAR(512),
        LLMReasoning         NVARCHAR(MAX),
        StartedAt            DATETIME2,
        CompletedAt          DATETIME2,
        UpdatedAt            DATETIME2       DEFAULT SYSUTCDATETIME(),
        CreatedAt            DATETIME2       DEFAULT SYSUTCDATETIME()
    );
END

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ActionHistory' AND schema_id =
SCHEMA_ID('adf'))
BEGIN
    CREATE TABLE adf.ActionHistory (
        HistoryId           BIGINT IDENTITY PRIMARY KEY,
        ProposalId          UNIQUEIDENTIFIER REFERENCES
adf.ActionProposals(ProposalId),
        TargetObject        NVARCHAR(256),
        ActionType          NVARCHAR(128),
        ExecutedAt          DATETIME2,
        CompletedAt         DATETIME2,
        Status              NVARCHAR(32),
        FailureReason       NVARCHAR(512)
    );
END

CREATE INDEX idx_ActionHistory_TargetCompleted
    ON adf.ActionHistory (TargetObject, CompletedAt DESC);

```

```

-- Signal history table with monthly partitioning via partition function
CREATE PARTITION FUNCTION pf_SignalHistoryMonth (DATETIME2)
AS RANGE RIGHT FOR VALUES (
    '2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01',
    '2025-05-01', '2025-06-01', '2025-07-01', '2025-08-01',
    '2025-09-01', '2025-10-01', '2025-11-01', '2025-12-01'
);

CREATE PARTITION SCHEME ps_SignalHistoryMonth
AS PARTITION pf_SignalHistoryMonth ALL TO ([PRIMARY]);

CREATE TABLE adf.SignalHistory (
    CollectedAt    DATETIME2      NOT NULL,
    SignalName      NVARCHAR(128)  NOT NULL,
    Value           DECIMAL(18,4),
    Slope           DECIMAL(18,6),
    Confidence       DECIMAL(5,4),
    RawSnapshot     NVARCHAR(MAX)  -- JSON
) ON ps_SignalHistoryMonth (CollectedAt);

```

The scheduler integration connects the signal collection view, the Python processor, and the decision engine into a single execution pipeline. For environments that prefer keeping everything inside the database, SQL Server Agent can drive the collection cycle. For environments with a Kubernetes control plane, a lightweight CronJob manifest calling the Python collector is preferable because it externalizes state management and makes the collector horizontally scalable.

The framework's own health is observable through a dedicated status view that any external monitoring system — Grafana, Datadog, PagerDuty — can scrape:

```

CREATE OR ALTER VIEW adf.FrameworkHealth AS
SELECT
    (SELECT COUNT(*) FROM adf.ActionProposals
     WHERE Status = 'PENDING'
     AND CreatedAt > DATEADD(HOUR, -1, SYSUTCDATETIME())) AS
PendingProposals1h,
    (SELECT COUNT(*) FROM adf.ActionProposals
     WHERE Status = 'EXECUTING') AS
CurrentlyExecuting,
    (SELECT COUNT(*) FROM adf.ActionProposals
     WHERE Status = 'FAILED'
     AND CreatedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())) AS FailedActions24h,
    (SELECT COUNT(*) FROM adf.ActionProposals
     WHERE Status = 'ESCALATED'
     AND CreatedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())) AS Escalations24h,
    (SELECT COUNT(*) FROM adf.OutcomeHistory
     WHERE ActionEffective = 1
     AND VerifiedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())) AS
EffectiveActions24h,
    (SELECT COUNT(*) FROM adf.OutcomeHistory
     WHERE SideEffectDetected = 1
     AND VerifiedAt > DATEADD(HOUR, -24, SYSUTCDATETIME())) AS SideEffects24h,
    (SELECT MAX(CollectedAt) FROM adf.SignalHistory) AS
LastSignalCollectedAt,
    (SELECT DATEDIFF(SECOND, MAX(CollectedAt), SYSUTCDATETIME())
     FROM adf.SignalHistory) AS
SignalStalenessSeconds;

```

An alert rule should fire if `signal_staleness` exceeds 180 seconds — three missed collection cycles. When the observability plane stops collecting, the entire framework is flying blind, and the on-call engineer needs to know before the decision engine starts acting on stale data or, worse, stops acting when it should.

38.9 Operational Runbook: The First 90 Days

Deploying the framework is not the same as operating it. The first 90 days determine whether the framework earns enough trust to expand its autonomy level or gets switched off after one bad action. The following phased approach has proven reliable across varied SQL Server environments.

Days 1–14: Shadow Mode

All four planes are running, but the Act plane is configured to only log proposed actions rather than execute them. Every proposal is written to `adf.action_proposals` with `status = 'SHADOW'` and the DBA team reviews the log daily. The goal is to calibrate signal thresholds against actual operational patterns. A `fragmented_indexes` threshold of 5 might be appropriate for a small OLTP database but will generate constant noise in a large data

warehouse where fragmentation is expected and managed through scheduled jobs. Threshold calibration during shadow mode prevents alert fatigue when the framework goes live.

Days 15–30: Live Execution, Low-Risk Actions Only

Promote `index_maintenance`, `query_kill`, and `connection_scaling` actions to `full_auto` in the staging environment and `auto_offhours` in production. Leave all other action classes at `approval_required`. Monitor `adf.framework_health.side_effects_24h` daily. Any side effect triggers a 48-hour pause and a root-cause review before autonomy is restored.

Days 31–60: Expand to Index Management

Once the low-risk action classes have accumulated at least 20 successful executions with zero collateral side effects, promote `index_management` to `auto_offhours` in production. Index creation is inherently lower risk when executed with `CONCURRENTLY / ONLINE = ON`, but the blast radius controls from section 38.4 must be validated against the largest tables in the production schema before promotion.

Days 61–90: Review, Retrain, and Plan

Run the statistical scoring model retraining cycle against the accumulated `adf.outcome_history` data. Review all `ESCALATED` proposals from the period — these are the cases where the framework correctly identified a problem but could not act, and they represent the highest-value opportunities for new Tier 1 deterministic rules. By day 90, the team should have a documented evidence base for which action classes perform reliably and which need further calibration before expanding their autonomy level.

38.10 What the Framework Does Not Do (And Should Not)

No autonomous framework should attempt to fully replace the judgment of an experienced DBA. There are categories of database work that require contextual knowledge the framework cannot hold: business-cycle awareness, application release coordination, regulatory constraints on data access, and the organizational dynamics of multi-team systems. Three categories deserve explicit exclusion.

Schema migrations are beyond the framework's scope. Altering table structures, adding columns with defaults to large tables, and changing data types all carry application-layer dependencies that the framework has no visibility into. The framework can detect that a migration is causing lock pressure and propose killing the migration session, but it should never initiate migrations.

Data purge and archival decisions require business-level authorization. The framework can surface signals that a table is growing faster than storage projections allow, but the decision of what data to delete, when, and to what archive destination belongs to the data governance process.

Security configuration changes — modifying roles, revoking privileges, changing authentication settings — must never be in the autonomous action set. A framework that can change its own access controls is a framework that can be exploited to escalate privileges.

These exclusions are not limitations of the framework's design. They are its defining characteristic. An autonomous framework that knows its own boundaries is significantly more trustworthy than one that claims unlimited scope. The boundary enforcement belongs in `adf.autonomy_policy` with `autonomy_level = 'blocked'` for these action classes, and no operational pressure should ever lift that block.

Key Takeaways

- → **A production-grade Autonomous DBA Framework requires four distinct planes** — Observe, Decide, Act, and Verify — with no direct writes across plane boundaries; cross-plane communication through a message bus makes the system auditable and prevents race conditions between stale signals and active executors.
- → **The decision engine must be a three-tier stack**, not a flat rule list: deterministic rules for known-safe actions, statistical scoring for context-dependent decisions, and LLM escalation for novel states — with each tier having a clearly defined confidence threshold and fallback path to human review.
- → **Blast radius controls, cooldown policies, and rollback contracts are not optional safety features** — they are what distinguishes an autonomous framework from an uncontrolled script; every action the executor takes must be bounded by scope, reversibility, concurrency limit, and a per-target cooldown enforced at runtime against a configurable policy table.
- → **The autonomy dial and change window system create a trust escalation path** that allows action classes to earn progressively higher autonomy levels based on a documented evidence base of successful executions, preventing the framework from being granted more authority than its operational track record justifies.
- → **The Verify plane closes the feedback loop** by measuring both root signal improvement and collateral signal degradation after every action; failed verifications immediately escalate to human review and suppress re-execution of the same action, ensuring the framework learns from its failures rather than repeatedly applying a corrective action that is causing a different class of damage.

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 39: The Future of Autonomous Data Platforms

The autonomous database systems described throughout this book represent the current frontier — but that frontier is moving. The architectures being built today with Always On availability groups, Prometheus, Kubernetes operators, and LLM-assisted decision loops are not the ceiling; they are the floor for what comes next. This chapter examines where autonomous data platforms are heading over the next three to five years, with enough architectural specificity that you can begin building toward these capabilities now rather than waiting for vendors to package them. The patterns emerging from cloud-native databases, distributed AI inference, and policy-driven governance are converging into something qualitatively different from what most organizations run today — and the engineers who understand these trajectories will be the ones designing the platforms that run them.

From Reactive Automation to Predictive Self-Architecture

Today's best autonomous systems react with sub-second precision. Tomorrow's systems will predict structural changes hours or days ahead and restructure themselves before stress arrives. This is the shift from *self-healing* to *self-architecting*, and it requires a fundamentally different model of what a database platform knows about itself.

The foundation of predictive self-architecture is continuous model training on workload telemetry. Not the five-minute aggregates most monitoring stacks produce — raw execution plans, buffer pool hit rates at ten-second granularity, lock acquisition sequences, and transaction rollback patterns indexed against time-of-day, deployment events, and external signals like queue depths or upstream API latency. A system that has twelve months of this data can train gradient boosted models that predict, with reasonable confidence, when a schema design will stop scaling before it actually does.

The architecture looks like this: a time-series feature store (TimescaleDB or InfluxDB) receives raw telemetry from the database; a feature engineering pipeline produces lag features, rolling window statistics, and cross-metric ratios; a retraining pipeline runs weekly; and a prediction service exposes a gRPC endpoint that the database operator consults during routine planning. When the prediction service returns a capacity exhaustion probability above 0.85 for a specific table's index structure within the next 72 hours, the operator doesn't wait for an alert — it queues a migration job.

```
-- Extract index efficiency trends from a custom telemetry history table
-- populated by a scheduled collector job
SELECT
    OBJECT_SCHEMA_NAME(s.object_id) AS schema_name,
    OBJECT_NAME(s.object_id) AS table_name,
    i.name AS index_name,
    DATEADD(HOUR,
        DATEDIFF(HOUR, 0, s.sample_ts), 0) AS hour_bucket,
    AVG(s.user_seeks + s.user_scans) AS avg_reads,
    AVG(s.user_lookups) AS avg_lookups,
    AVG(s.user_updates) AS avg_writes,
    -- Write-to-read ratio above 10 signals a candidate for removal
    ROUND(
        CAST(AVG(s.user_updates) AS FLOAT) /
        NULLIF(AVG(s.user_seeks + s.user_scans + s.user_lookups), 0),
        4) AS write_read_ratio,
    -- Fragmentation trend from snapshot
    AVG(s.avg_fragmentation_pct) AS avg_fragmentation
FROM dbo.index_usage_snapshots s
JOIN sys.indexes i
    ON i.object_id = s.object_id
    AND i.index_id = s.index_id
WHERE s.sample_ts >= DATEADD(DAY, -7, GETUTCDATE())
GROUP BY
    OBJECT_SCHEMA_NAME(s.object_id),
    OBJECT_NAME(s.object_id),
    i.name,
    DATEADD(HOUR, DATEDIFF(HOUR, 0, s.sample_ts), 0)
ORDER BY write_read_ratio DESC;
```

These queries don't do prediction themselves — they produce the feature vectors that feed the prediction pipeline. The prediction pipeline is a Python service using scikit-learn or XGBoost, retrained on a sliding 90-day window, with model artifacts stored in MLflow. The database operator (a Kubernetes custom controller) calls this service's predict endpoint before executing each planning cycle. If the predicted degradation confidence crosses the threshold, the operator emits a self-architecture event, which triggers a schema migration workflow.

The migration workflow itself must be autonomous and safe. This means wrapping schema changes in transactions with explicit rollback hooks, using `online index rebuilds` or `ONLINE index operations` to avoid blocking, and staging changes through a shadow schema before cutover. The operator doesn't fire-and-forget — it watches execution time and lock waits during the migration and has circuit breaker logic that pauses or aborts if secondary metrics deteriorate.

What distinguishes a truly predictive system from a sophisticated alert is that the predictive system acts before the problem appears in user-facing metrics. The SLA never degrades. The operator never pages anyone. The platform engineers see an entry in the audit log saying "index restructure on orders table completed at 02:14 UTC, predicted degradation avoided."

Natural Language Interfaces as Autonomous Control Planes

The emergence of large language models has already changed how engineers write queries and diagnose problems. The next architectural shift is treating LLMs not as assistants but as control plane components — systems that receive natural language policies and translate them into executable operational logic that runs continuously without human re-engagement.

The distinction matters. An assistant answers a question when asked. A control plane component enforces a policy indefinitely. When a CTO says "we should never run more than 80% storage utilization on any production cluster," today that becomes a Jira ticket, a runbook, and a Slack alert. In a future autonomous system, that sentence is the input to a policy compiler that generates threshold rules, forecasting jobs, automated remediation procedures, and audit trails — all from the natural language specification.

This is buildable today with Claude or GPT-4 APIs, but the architecture requires rigor that most experimental LLM integrations lack. The key design principle is that the LLM never executes actions directly. It operates as a policy translator: it receives a human intent, produces a structured policy document in a defined schema (JSON or YAML), and that document is validated against a policy schema, human-reviewed for the first deployment, and then handed to the enforcement engine. The LLM is not in the runtime loop; it is in the policy authoring loop.

```

import json
import anthropic
from jsonschema import validate, ValidationError

# Policy schema that the LLM must conform to
POLICY_SCHEMA = {
    "type": "object",
    "required": ["policy_name", "target_resource", "condition", "actions",
"cooldown_minutes"],
    "properties": {
        "policy_name": {"type": "string"},
        "target_resource": {"type": "string", "enum": ["cluster", "table",
"index", "connection_pool"]},
        "condition": {
            "type": "object",
            "required": ["metric", "operator", "threshold"],
            "properties": {
                "metric": {"type": "string"},
                "operator": {"type": "string", "enum": [">", "<", ">=", "<=",
"=="]},
                "threshold": {"type": "number"},
                "window_minutes": {"type": "integer", "minimum": 1}
            }
        },
        "actions": {
            "type": "array",
            "items": {
                "type": "object",
                "required": ["action_type", "parameters"],
                "properties": {
                    "action_type": {
                        "type": "string",
                        "enum": [
                            "scale_storage", "rebalance_connections",
                            "trigger_index_maintenance", "rebuild_index",
                            "failover_replica", "notify_oncall"
                        ]
                    },
                    "parameters": {"type": "object"}
                }
            }
        },
        "cooldown_minutes": {"type": "integer", "minimum": 5}
    }
}

SYSTEM_PROMPT = """
You are a database policy compiler. Convert natural language operational
policies into structured JSON that conforms exactly to the provided schema.
Be conservative: when threshold values are ambiguous, choose the safer
(more conservative) interpretation. Never invent action types not listed
in the schema enum. If the policy cannot be expressed within the schema
constraints, return {"error": "reason"} and explain what is missing.

```

```

"""

def compile_policy(natural_language_policy: str) -> dict:
    client = anthropic.Anthropic()

    schema_str = json.dumps(POLICY_SCHEMA, indent=2)
    prompt = f"""
    Schema:
    {schema_str}

    Policy to compile:
    "{natural_language_policy}"

    Return only valid JSON conforming to the schema. No commentary.
    """

    message = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        system=SYSTEM_PROMPT,
        messages=[{"role": "user", "content": prompt}]
    )

    raw_output = message.content[0].text.strip()

    try:
        policy_doc = json.loads(raw_output)
    except json.JSONDecodeError as e:
        raise ValueError(f"LLM produced invalid JSON: {e}\nRaw: {raw_output}")

    if "error" in policy_doc:
        raise ValueError(f"Policy compiler rejected input: {policy_doc['error']}")

    try:
        validate(instance=policy_doc, schema=POLICY_SCHEMA)
    except ValidationError as e:
        raise ValueError(f"Policy failed schema validation: {e.message}")

    # Mark as pending human approval before enforcement begins
    policy_doc["status"] = "pending_approval"
    policy_doc["compiled_by"] = "llm_policy_compiler_v2"

    return policy_doc

# Example usage
policy = compile_policy(
    "If storage utilization on any production cluster exceeds 80% "
    "for more than 15 minutes, automatically trigger storage scaling "
    "and notify the on-call DBA. Wait at least 30 minutes before "
    "triggering again."
)
print(json.dumps(policy, indent=2))

```

The output of this pipeline is a versioned policy document stored in a Git repository. Changes to policies go through pull request review. The enforcement engine — a Python service running as a Kubernetes operator — watches the policy repository, loads approved policies, and evaluates their conditions against the metrics backend on every evaluation cycle. When a condition fires, the action executor runs the specified action type through a pre-validated action library. The LLM never touches a live database directly.

This architecture scales. A platform team can manage hundreds of operational policies across dozens of clusters without writing bespoke automation for each one. The LLM handles the translation from human intent to machine-executable specification; the validation pipeline ensures correctness; the enforcement engine provides consistency. Each layer can be independently audited, tested, and versioned.

The next evolution is closed-loop policy refinement: the enforcement engine reports action outcomes back to the policy store, and periodically a review job invites the LLM to assess whether policies are achieving their stated intent based on outcome data. This is not autonomous policy modification — a human still approves changes — but it means the system surfaces its own improvement opportunities rather than waiting for engineers to notice.

Distributed Autonomous Systems and Multi-Region Decision Coherence

Single-region autonomous databases are a solved problem at the architectural level. The genuinely hard frontier is multi-region autonomous platforms where independent autonomous agents must make locally consistent decisions that remain globally coherent. When the New York cluster's autonomous operator decides to promote a replica, it needs to coordinate with — or at minimum not conflict with — decisions being made simultaneously by the Frankfurt and Singapore operators.

This is a distributed consensus problem layered on top of an autonomous decision problem, and the two interact in non-obvious ways. A naive implementation gives each regional operator full autonomy, which produces split-brain scenarios not at the database level (SQL Server AGs handle that) but at the operational policy level: New York scales up connection pools while Singapore scales them down, both in response to the same downstream cascade.

The solution is a hierarchical decision architecture with a global policy arbiter. Regional operators handle local, time-sensitive decisions (query plan changes, connection rebalancing, local failover) without global coordination because latency requirements make coordination impractical. Global decisions (cross-region traffic routing, schema migrations, retention policy changes) go through the arbiter, which applies a consensus protocol before any region acts.

```

graph TD
    A[Regional Operator] -->|local, time-sensitive decisions| A
    A -->|global-scope decision intent| B["Global Policy Arbiter (Raft/etcd)"]
    B --> C{Conflict Check}
    C -->|Hard conflict: same resource| D[Deny / Supersede]
    C -->|No conflict or soft conflict within budget| E[Approve + Execution Window]
    D --> A
    E --> A
    B -->|No response before expiry| F[Local Fallback: abort-and-alert or proceed-and-log]
    F --> A

```

Regional operators act locally without coordination; only global-scope decisions cross through the arbiter, which resolves conflicts against versioned policy rather than heuristics, and regions fall back to local policy if the arbiter misses the expiry window.

```

-- Register a cross-region decision intent in the distributed ops catalog
-- replicated via transactional replication or AG readable secondary sync
INSERT INTO autonomous_ops.DecisionIntents (
    IntentId,
    OriginatingRegion,
    DecisionScope,
    ActionType,
    TargetResource,
    ProposedParameters,
    IntentTs,
    ExpiryTs,
    [Status]
)
SELECT
    NEWID(),
    @@SERVERNAME,
    'global',
    'index_rebuild',
    'Sales.OrderLineItems',
    (SELECT
        'rebuild' AS operation_type,
        'PK_OrderLineItems' AS index_name,
        'ONLINE=ON, MAXDOP=4' AS rebuild_options
        FOR JSON PATH, WITHOUT_ARRAY_WRAPPER),
    GETUTCDATE(),
    DATEADD(MINUTE, 10, GETUTCDATE()),
    'pending'
WHERE NOT EXISTS (
    -- Do not register if another region already has a pending intent
    -- for the same resource and action type
    SELECT 1
    FROM autonomous_ops.DecisionIntents
    WHERE TargetResource = 'Sales.OrderLineItems'
        AND ActionType = 'index_rebuild'
        AND [Status] = 'pending'
        AND ExpiryTs > GETUTCDATE()
);

-- Poll for arbiter resolution
SELECT
    IntentId,
    [Status],
    ArbiterNotes,
    ApprovedExecutionWindow,
    DATEDIFF(SECOND, IntentTs, GETUTCDATE()) AS age_seconds
FROM autonomous_ops.DecisionIntents
WHERE OriginatingRegion = @@SERVERNAME
    AND [Status] IN ('approved', 'denied', 'superseded')
    AND IntentTs >= DATEADD(MINUTE, -15, GETUTCDATE());

```


The global arbiter is a separate service — not a database — running with its own high-availability setup (three-node Raft cluster using etcd or Consul). It receives decision intents from all regions, evaluates conflicts, applies global policies (maintenance windows, change freeze periods, concurrent operation limits), and returns decisions within the expiry window. Regional operators that don't receive an arbiter response before expiry fall back to local policy: for high-risk actions, they abort and alert; for low-risk actions, they proceed and log the arbitration miss.

The conflict resolution logic in the arbiter is explicit and policy-driven, not heuristic. The arbiter knows that two regions cannot simultaneously execute schema migrations on the same logical table (hard conflict), that concurrent index rebuilds on different tables in the same schema are allowed (no conflict), and that storage scaling in two regions simultaneously is allowed unless the total cost delta exceeds a budget threshold (soft conflict requiring approval). These rules are stored as conflict policies in the arbiter's configuration, versioned in Git, and never derived dynamically by an AI.

This architecture supports the future of geo-distributed databases like CockroachDB and Spanner-compatible systems where autonomous operation must span not just nodes but regulatory jurisdictions. The decision coherence layer becomes part of the compliance infrastructure: every global action is logged with the arbiter's reasoning, the conflict policies consulted, and the dissenting region intents that were superseded.

Autonomous Data Governance and Policy-as-Code at Runtime

Data governance today is largely a design-time and audit-time concern: you define policies in a catalog, enforce them through application code or views, and audit violations after the fact. The autonomous future inverts this: governance policies are runtime-active, self-enforcing, and capable of restructuring data access patterns in response to regulatory signals without human intervention.

The technical foundation is a governance engine that sits between the connection pool and the database, operating at the query rewrite or row security policy level. When a new GDPR data subject deletion request arrives, the governance engine doesn't queue a ticket — it immediately applies row-level security masks to the identified user's data, queues the hard delete for the next maintenance window, and updates the audit log. When a new data residency requirement activates for a jurisdiction, the governance engine rewrites routing rules in the proxy layer so that queries containing PII for that jurisdiction route only to in-jurisdiction replicas.

The governance engine calling this function receives its signals from a regulatory event stream — a Kafka topic where compliance systems publish events like

`data_subject_deletion_requested`, `jurisdiction_constraint_activated`, or

`retention_policy_updated` . The engine subscribes to this stream, maps events to governance actions, and calls the appropriate database-side functions. The human compliance officer interacts with the regulatory system, not the database; the engine handles the translation.

For SQL Server environments, the equivalent uses row-level security predicates and dynamic data masking, managed through a governance stored procedure layer:

```

-- Governance engine stored procedure: apply or refresh a jurisdiction
-- residency policy on a target table
CREATE OR ALTER PROCEDURE governance.ApplyResidencyConstraint
    @SchemaName      NVARCHAR(128),
    @TableName        NVARCHAR(128),
    @Jurisdiction      NVARCHAR(50),
    @ConstraintId      UNIQUEIDENTIFIER
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE
        @PolicyFuncName NVARCHAR(256),
        @PolicyName      NVARCHAR(256),
        @SQL              NVARCHAR(MAX),
        @RoleName         NVARCHAR(128);

    SET @RoleName      = 'DataAccess_' + @Jurisdiction;
    SET @PolicyFuncName = 'governance.RLS_' + @SchemaName
        + '_' + @TableName + '_' + @Jurisdiction;
    SET @PolicyName     = 'Policy_' + @SchemaName
        + '_' + @TableName + '_' + @Jurisdiction;

    -- Drop existing security policy if present
    IF EXISTS (
        SELECT 1 FROM sys.security_policies
        WHERE name = @PolicyName
        AND SCHEMA_NAME(schema_id) = 'governance'
    )
    BEGIN
        SET @SQL = N'DROP SECURITY POLICY governance.' + QUOTENAME(@PolicyName);
        EXEC sp_executesql @SQL;
    END;

    -- (Re)create the inline table-valued predicate function
    SET @SQL = N'
    CREATE OR ALTER FUNCTION ' + @PolicyFuncName + N'
    (@data_jurisdiction NVARCHAR(50))
    RETURNS TABLE
    WITH SCHEMABINDING
    AS RETURN
    SELECT 1 AS result
    WHERE @data_jurisdiction != ' + QUOTENAME(@Jurisdiction, '''') + N'
        OR IS_MEMBER(' + QUOTENAME(@RoleName, '''') + N') = 1;';
    EXEC sp_executesql @SQL;

    -- Create the security policy using the predicate function
    SET @SQL = N'
    CREATE SECURITY POLICY governance.' + QUOTENAME(@PolicyName) + N'
    ADD FILTER PREDICATE ' + @PolicyFuncName + N'(' + QUOTENAME(@SchemaName) + N'.'
    + QUOTENAME(@TableName) + N'.data_jurisdiction)
    ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName) + N'
    WITH (STATE = ON, SCHEMABINDING = ON);';

```

```
EXEC sp_executesql @SQL;

-- Audit the governance action
INSERT INTO governance.PolicyAuditLog (
    ConstraintId, SchemaName, TableName,
    PolicyName, Action, AppliedTs, AppliedBy
) VALUES (
    @ConstraintId, @SchemaName, @TableName,
    @PolicyName, 'CREATE', GETUTCDATE(), SYSTEM_USER
);
END;
```

The audit log is the non-negotiable component. Every governance action must be immutably recorded with the originating event, the constraint applied, the function invoked, and the timestamp. In regulated environments, this audit log itself must be write-once — backed by an append-only table with a trigger that prevents UPDATE or DELETE operations, or stored in an immutable object store like S3 with object lock. The governance engine's effectiveness is only as trustworthy as the audit trail demonstrating it acted correctly.

The long-term trajectory here is governance-by-intent: regulators (internal or external) express data handling requirements in structured policy language, the system compiles those requirements into runtime enforcement mechanisms, and compliance becomes a continuous verified state rather than a periodic audit outcome. The database engineer's role shifts from writing policy enforcement code to designing and validating the policy compilation and enforcement pipeline — a higher-leverage position that scales across a fleet of databases without linear effort growth.

Autonomous Cost Optimization Across Multi-Region Fleets

Cost optimization is currently a human activity: engineers review cloud bills, identify expensive queries or oversized instances, and schedule remediation work. Autonomous cost optimization closes this loop by treating cloud spend as a first-class metric in the same control plane that manages performance and availability.

The architecture requires three components that most organizations have separately but rarely integrate: a cost attribution pipeline, a resource efficiency model, and an autonomous rightsizing executor.

The cost attribution pipeline enriches telemetry with cost data by joining cloud provider billing APIs (AWS Cost Explorer, Azure Cost Management) with resource identifiers. Every database instance, every provisioned IOPS volume, and every data transfer event gets a per-hour cost tag attached to its performance telemetry. This means the monitoring system

doesn't just see "this query consumes 40% of CPU" — it sees "this query consumes 40% of CPU on an instance costing \$4.20/hour, making this query responsible for approximately \$1.68/hour of compute spend."

The resource efficiency model computes a utilization efficiency score for each resource: actual demand divided by provisioned capacity, weighted by time-of-day patterns and headroom requirements. A database instance running at 15% average CPU utilization with a peak of 45% and a headroom requirement of 30% above peak is a candidate for downsizing to a smaller instance class — the math is deterministic once the inputs are defined.

```

from dataclasses import dataclass
from typing import Optional
import math

@dataclass
class ResourceEfficiencyProfile:
    resource_id: str
    resource_type: str          # 'compute', 'storage', 'iops'
    provisioned_capacity: float
    avg_utilization_pct: float
    p95_utilization_pct: float
    p99_utilization_pct: float
    required_headroom_pct: float # policy-defined safety margin
    hourly_cost_usd: float

    @property
    def effective_peak(self) -> float:
        """Peak utilization that must be accommodated."""
        return self.p99_utilization_pct

    @property
    def minimum_required_capacity(self) -> float:
        """Smallest capacity that satisfies peak + headroom."""
        return self.provisioned_capacity * (
            (self.effective_peak / 100.0) *
            (1 + self.required_headroom_pct / 100.0)
        )

    @property
    def waste_fraction(self) -> float:
        """Fraction of provisioned capacity that is never needed."""
        return max(0.0, 1.0 - (self.minimum_required_capacity /
                               self.provisioned_capacity))

    @property
    def estimated_savings_usd_per_hour(self) -> float:
        """Conservative savings estimate if rightsized to minimum required."""
        # Apply a 15% uncertainty discount to avoid over-optimistic estimates
        raw_savings = self.hourly_cost_usd * self.waste_fraction
        return raw_savings * 0.85

    def rightsizing_recommendation(self) -> Optional[dict]:
        """Return a recommendation dict if savings exceed the minimum threshold."""
        MIN_HOURLY_SAVINGS = 0.10 # Only act if savings > $0.10/hour
        MIN_WASTE_FRACTION = 0.20 # Only act if >20% is wasted

        if (self.waste_fraction >= MIN_WASTE_FRACTION and
            self.estimated_savings_usd_per_hour >= MIN_HOURLY_SAVINGS):
            return {
                "resource_id": self.resource_id,
                "resource_type": self.resource_type,
                "current_capacity": self.provisioned_capacity,
                "recommended_capacity": math.ceil(

```

```

        self.minimum_required_capacity
    ),
    "waste_fraction": round(self.waste_fraction, 3),
    "estimated_hourly_savings_usd": round(
        self.estimated_savings_usd_per_hour, 4
    ),
    "estimated_monthly_savings_usd": round(
        self.estimated_savings_usd_per_hour * 730, 2
    ),
    "confidence": "high" if self.p99_utilization_pct < 60 else "medium"
}
return None

```

The autonomous rightsizing executor acts on these recommendations within defined autonomy boundaries. Recommendations above a certain monthly savings threshold require human approval. Below that threshold, the executor schedules rightsizing operations during the next defined maintenance window, executes the change, monitors for performance regression for a defined observation period, and rolls back automatically if regression is detected. Every action is logged with the efficiency profile snapshot that triggered it, creating a learning corpus for model refinement.

The integration with SQL Server environments is through instance metadata — the executor calls cloud provider APIs (not database APIs) to resize instances, but it validates the decision against database-level metrics before committing. An instance may show low CPU utilization because it is heavily I/O-bound; the executor checks buffer pool hit rates and I/O wait statistics before concluding that the instance is genuinely oversized.

```

-- Equivalent buffer pool efficiency snapshot for the efficiency model
SELECT
    'sql_buffer_pool' AS resource_type,
    (SELECT value_in_use FROM sys.configurations
     WHERE name = 'max server memory (MB)') AS provisioned_mb,
    ROUND(
        CAST(
            (SELECT cnt_r_value FROM sys.dm_os_performance_counters
             WHERE counter_name = 'Buffer cache hit ratio'
               AND object_name LIKE '%Buffer Manager%')
            AS FLOAT) /
        NULLIF(
            (SELECT cnt_r_value FROM sys.dm_os_performance_counters
             WHERE counter_name = 'Buffer cache hit ratio base'
               AND object_name LIKE '%Buffer Manager%'),
            0) * 100,
        2) AS hit_rate_pct,
    physical_memory_in_use_kb / 1024.0 AS current_usage_mb
FROM sys.dm_os_process_memory;

```

The cost optimization loop eventually becomes a bidirectional one: the system doesn't just downsize underutilized resources, it also proactively provisions ahead of predicted demand based on the capacity forecasting models described earlier in this chapter. The same model that predicts index degradation 72 hours ahead can predict that a marketing campaign scheduled for next Tuesday will require 40% more connection pool capacity, and pre-provision it the night before at a lower spot-instance price rather than scaling reactively at full on-demand cost.

The Platform Engineer's Evolving Role

Throughout this book, the recurring theme has been that automation doesn't eliminate the database engineer — it changes what that engineer must be expert in. This final section is direct about what that means for careers and organizations building toward these systems.

The engineers who will design and operate autonomous data platforms need depth in three domains that traditionally don't overlap: database internals, distributed systems, and machine learning operations. Not superficial familiarity — genuine depth. You need to understand transaction log mechanics deeply enough to know why the autonomous index maintenance tuner's recommendation to rebuild every fragmented index during peak hours is wrong for write-heavy OLTP even if the fragmentation metric suggests it. You need to understand Raft consensus well enough to design the global arbiter without creating a new single point of failure. You need to understand model drift well enough to know when the capacity prediction model needs retraining versus when its predictions are accurate but the operational response policy needs adjustment.

This is a genuinely new engineering role, not a rebadging of the DBA or the DevOps engineer. The closest analog is the site reliability engineer, but with database internals depth that most SRE teams lack and ML pipeline ownership that most database teams have never needed. Organizations building serious autonomous data platforms will need to create this role explicitly — with a career path, a hiring profile, and internal training investment — rather than assuming existing teams will organically develop the cross-domain depth required.

The organizational structure also shifts. Today's database team is organized around deployment models (the on-premises team, the Azure team). Tomorrow's autonomous data platform team is organized around platform capabilities: the telemetry and observability layer, the decision engine, the policy compiler, the enforcement infrastructure, the cost optimization loop. Individual engineers may specialize in one deployment model's internals, but the platform itself applies the same autonomous decision architecture across the whole fleet.

The toolchain follows the same trajectory. Today's DBA toolchain is database-specific: SQL Server Management Studio, Azure Data Studio, sqlcmd, DBCC. Tomorrow's platform engineer toolchain is cloud-native and system-agnostic: Kubernetes operators, Prometheus and Grafana, MLflow, Terraform, ArgoCD, Kafka. The database-specific tools become internal components of the platform rather than the primary interface through which engineers interact with data infrastructure.

What remains constant is the requirement for rigor about correctness. Autonomous systems that make wrong decisions at scale make wrong decisions very fast. The safeguards — circuit breakers, human approval gates, rollback hooks, audit trails, model drift detection — are not bureaucratic overhead. They are what separates an autonomous platform from an automated disaster. Engineers who understand why these safeguards exist, and can design systems where the safeguards are themselves reliable and tested, will be the ones organizations trust to build and run these platforms.

The trajectory described in this chapter — from reactive automation to predictive self-architecture, from assistant LLMs to policy control planes, from regional autonomy to multi-region coherence, from manual governance to runtime enforcement, from reactive cost management to predictive rightsizing — is not speculative. Each piece is buildable with technologies available today. The integration is hard. The organizational change is harder. But the competitive pressure is real: organizations that complete this journey will operate data infrastructure at lower cost, higher reliability, and faster adaptation speed than those that don't.

The autonomous database platform is not a product you will buy. It is a system you will build, incrementally, on the foundations this book has laid. The tools are in your hands.

Key Takeaways

- → **Predictive self-architecture** extends autonomous systems beyond reactive healing by training workload models on fine-grained telemetry and acting on predicted degradation before it surfaces in user-facing metrics — turning capacity problems into scheduled maintenance rather than incidents.
- → **LLMs as policy compilers**, not runtime executors, provide a safe path to natural language operational control: the model translates human intent into validated, versioned policy documents that a separate enforcement engine applies, keeping the LLM out of the live execution path.
- → **Multi-region autonomous coherence** requires a hierarchical decision architecture separating local-scope decisions (made autonomously with low latency) from global-scope decisions (arbitrated through a consensus service), with explicit conflict resolution policies that are versioned and auditable.

- → **Runtime governance enforcement** inverts the traditional audit-after-the-fact model by making compliance policies active database-layer controls — row security policies, query rewrites, and routing rules — that respond to regulatory event streams without human intermediation at each trigger point.
- → **Autonomous cost optimization** closes the cloud spend loop by attributing cost to specific queries and resources in the same telemetry pipeline that tracks performance, enabling rightsizing decisions that are validated against database-level efficiency metrics before execution and reversed automatically if performance regresses.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)

Chapter 40: Becoming an Autonomous Database Engineer (ADE)

The thirty-nine chapters before this one were about systems — how to instrument them, how to automate them, how to teach them to reason and act without human intervention. This final chapter is about you. The Autonomous Database Engineer is not a job title that exists on most org charts yet, but the role is real, the demand is accelerating, and the engineers who will define the next decade of data infrastructure are the ones who understand how to build systems that operate autonomously while also understanding precisely when and how to override them. This chapter closes the loop: from the mental model of the ADE, through the career architecture, the technical competencies, the organizational patterns, and the day-to-day practices that separate engineers who maintain databases from engineers who design self-governing data infrastructure.

The Mental Model Shift: From Operator to Architect of Autonomous Systems

Every senior DBA knows the trap. You are the person who fixes the problem. The alert fires at 2 AM, you SSH in, you identify the blocker, you kill the query, you add the index, you bump the connection pool limit. You are indispensable — and that indispensability is the problem. A system that requires you is a system that has not been fully engineered.

The ADE mental model inverts this. Your job is not to fix the system — your job is to build the system that fixes itself, and to reserve your judgment for the decisions that genuinely require human reasoning: ethical tradeoffs, cost authorization above defined thresholds, architectural pivots, and post-incident learning that improves the autonomous loop itself.

This requires accepting a specific kind of discomfort. Autonomous systems will make decisions you would not have made. They will choose to scale horizontally when you might have tuned a query. They will trigger a failover at 3 AM based on a health score crossing a threshold when you might have waited five more minutes to see if the spike was transient. Some of those decisions will be wrong. The engineering discipline of the ADE is not to prevent all autonomous errors — it is to bound their blast radius, instrument every decision with enough observability to conduct a post-action review, and continuously improve the decision logic based on observed outcomes.

The framework for this mental model has three axes:

Confidence bounds: every autonomous action has a confidence score, a risk classification, and an authorized scope. A system that can rebuild indexes aggressively but cannot drop a table without human sign-off has correctly bounded its authority.

Reversibility tiers: actions are classified by reversibility. Scaling a connection pool is reversible in seconds. Promoting a replica is reversible in minutes with effort. Dropping a partition is not reversible without a restore. The autonomous system should have nearly unlimited authority over Tier 1 reversible actions, conditional authority over Tier 2, and strict human approval gates for Tier 3.

Observability contracts: every autonomous action must emit a structured audit event that captures the triggering signal, the decision logic, the confidence score, the action taken, and the observed outcome. Without this, post-incident learning is impossible.

This mental model is not philosophy. It translates directly into code:

```

# Autonomous action dispatcher with confidence and reversibility enforcement
import logging
from dataclasses import dataclass
from enum import Enum
from typing import Callable, Optional
import requests

class ReversibilityTier(Enum):
    TIER_1_IMMEDIATE = 1    # Seconds to reverse: pool tuning, maintenance settings
    TIER_2_RECOVERABLE = 2  # Minutes to reverse: failover, replica promotion
    TIER_3_DESTRUCTIVE = 3  # Requires restore: partition drops, schema changes

@dataclass
class AutonomousAction:
    name: str
    tier: ReversibilityTier
    confidence_threshold: float
    action_fn: Callable
    approval_webhook: Optional[str] = None

class AutonomousDispatcher:
    def __init__(self, audit_log_path: str, approval_endpoint: str):
        self.audit_log_path = audit_log_path
        self.approval_endpoint = approval_endpoint
        self.logger = logging.getLogger("ade.dispatcher")

    def dispatch(self, action: AutonomousAction, confidence: float,
                 context: dict) -> dict:
        audit_event = {
            "action": action.name,
            "tier": action.tier.name,
            "confidence": confidence,
            "threshold": action.confidence_threshold,
            "context": context,
            "outcome": None,
            "status": None,
        }

        if confidence < action.confidence_threshold:
            audit_event["status"] = "SKIPPED_LOW_CONFIDENCE"
            self._write_audit(audit_event)
            return audit_event

        if action.tier == ReversibilityTier.TIER_3_DESTRUCTIVE:
            audit_event["status"] = "PENDING_HUMAN_APPROVAL"
            self._request_approval(action, confidence, context)
            self._write_audit(audit_event)
            return audit_event

        if (action.tier == ReversibilityTier.TIER_2_RECOVERABLE
            and confidence < 0.90):
            audit_event["status"] = "PENDING_HUMAN_APPROVAL"
            self._request_approval(action, confidence, context)

```

```

        self._write_audit(audit_event)
        return audit_event

    try:
        outcome = action.action_fn(context)
        audit_event["status"] = "EXECUTED"
        audit_event["outcome"] = outcome
    except Exception as e:
        audit_event["status"] = "FAILED"
        audit_event["outcome"] = str(e)
        self.logger.error(f"Action {action.name} failed: {e}")

    self._write_audit(audit_event)
    return audit_event

def _write_audit(self, event: dict):
    import json, time
    event["timestamp"] = time.time()
    with open(self.audit_log_path, "a") as f:
        f.write(json.dumps(event) + "\n")

def _request_approval(self, action: AutonomousAction,
                     confidence: float, context: dict):
    payload = {
        "action": action.name,
        "confidence": confidence,
        "context": context,
        "tier": action.tier.name,
    }
    try:
        requests.post(self.approval_endpoint, json=payload, timeout=5)
    except Exception as e:
        self.logger.warning(f"Approval webhook failed: {e}")

```

This dispatcher is built for SQL Server throughout. It enforces the ADE contract universally: every action is audited, every action is bounded, and the boundary logic is explicit in code rather than implicit in human judgment exercised under pressure.

The ADE Technical Stack: What You Must Own End-to-End

An ADE does not specialize in one layer. The person who only knows query tuning cannot build an autonomous system — they can build an automated query tuner, which is a component, not a system. End-to-end ownership means understanding, designing, and operating across six technical domains simultaneously.

Domain 1: Database Internals

You cannot write a self-tuning system for a database you do not understand at the storage engine level. For SQL Server, this means the lock manager and isolation internals, the Query Store plan forcing mechanism, the Always On redo thread, the buffer pool and checkpoint architecture, statistics maintenance, and the Resource Governor framework.

The ADE-level health signal extraction looks like this:

```

-- ADE health signal: fragmentation, statistics staleness, and AG replica lag
WITH index_health AS (
    SELECT
        OBJECT_SCHEMA_NAME(i.object_id) AS schema_name,
        OBJECT_NAME(i.object_id) AS table_name,
        i.name AS index_name,
        s.avg_fragmentation_in_percent,
        s.page_count,
        STATS_DATE(i.object_id, i.index_id) AS stats_last_updated,
        DATEDIFF(HOUR, STATS_DATE(i.object_id, i.index_id), GETDATE())
            AS hours_since_stats_update,
        us.user_seeks + us.user_scans + us.user_lookups AS total_reads,
        us.user_updates AS total_writes
    FROM sys.indexes i
    INNER JOIN sys.dm_db_index_physical_stats(
        DB_ID(), NULL, NULL, NULL, 'LIMITED') s
        ON i.object_id = s.object_id AND i.index_id = s.index_id
    LEFT JOIN sys.dm_db_index_usage_stats us
        ON i.object_id = us.object_id
        AND i.index_id = us.index_id
        AND us.database_id = DB_ID()
    WHERE s.page_count > 1000
),
ag_health AS (
    SELECT
        ag.name AS ag_name,
        ar.replica_server_name,
        drs.database_name,
        drs.synchronization_state_desc,
        drs.synchronization_health_desc,
        drs.redo_queue_size,
        drs.log_send_queue_size,
        CASE
            WHEN drs.redo_queue_size > 102400 THEN 'LAG_WARNING'
            WHEN drs.synchronization_health_desc != 'HEALTHY' THEN 'SYNC_DEGRADED'
            ELSE 'HEALTHY'
        END AS ag_signal
    FROM sys.dm_hadr_database_replica_states drs
    INNER JOIN sys.availability_replicas ar
        ON drs.replica_id = ar.replica_id
    INNER JOIN sys.availability_groups ag
        ON ar.group_id = ag.group_id
)
SELECT
    'INDEX_HEALTH' AS signal_type,
    ih.table_name,
    ih.index_name,
    ih.avg_fragmentation_in_percent AS fragmentation_pct,
    ih.hours_since_stats_update,
    CASE
        WHEN ih.avg_fragmentation_in_percent > 30 THEN 'REBUILD'
        WHEN ih.avg_fragmentation_in_percent > 10 THEN 'REORGANIZE'
        ELSE 'HEALTHY'
    END AS signal

```



```
END AS recommended_action
FROM index_health ih
WHERE ih.avg_fragmentation_in_percent > 10
ORDER BY ih.avg_fragmentation_in_percent DESC;
```

Domain 2: Infrastructure as Code and Kubernetes Operators

An ADE must own the database lifecycle through Terraform and Kubernetes. The SQL Server Kubernetes operator and Azure Arc-enabled SQL Server are production tools, not experiments. You should be able to write a Kubernetes operator extension that responds to custom metrics and modifies database configuration without touching a deployment manifest manually.

Domain 3: Observability Engineering

Prometheus, Alertmanager, Grafana, and OpenTelemetry are the nervous system of autonomous database infrastructure. The ADE must understand not just how to instrument but how to build alerting logic that is specific enough to trigger autonomous actions without producing false positives that degrade trust in the automation.

The signal-to-noise ratio of your alerting stack determines whether your autonomous system makes decisions based on reality or based on instrumentation artifacts. Every Prometheus alert rule that feeds an autonomous action should have a documented false-positive rate measured over at least thirty days of production traffic.

Domain 4: Decision Engine and AI Integration

This was the subject of Book 2, but at the ADE level you are not consuming AI — you are designing the reasoning layer. The difference between an AI-assisted DBA and an ADE is that the ADE defines the epistemics: what the system believes, how confident it is, what evidence updates that confidence, and what actions are authorized at each confidence tier.

Domain 5: Multi-Region and Disaster Recovery Architecture

Autonomous systems that cannot survive a regional failure are not autonomous — they are automated in normal conditions. The ADE must understand active-active replication topologies, conflict resolution strategies, and the specific failure modes of SQL Server Always On Availability Groups and failover clusters under real network partition conditions.

Domain 6: Security and Compliance Automation

Autonomous systems that act without audit trails are a compliance liability. The ADE must own the security model of the autonomous loop itself: service account permissions, credential rotation, audit log integrity, and the authorization controls that prevent the autonomous system from being used as a privilege escalation vector.

Building Your First Production-Grade Autonomous Loop

The canonical ADE project — the one that proves the competency and forms the portfolio centerpiece — is a complete autonomous operational loop covering a production database cluster. This is not a monitoring dashboard. It is a system that observes, diagnoses, decides, acts, and learns from the outcome.

The loop has five stages:

Stage 1: Signal Collection — Metrics scraped by the SQL Server Prometheus exporter, supplemented by custom queries registered as metrics using the exporter's collector configuration. These feed into Prometheus with appropriate retention and downsampling.

Stage 2: Anomaly Detection — A Python process or a Prometheus recording rule computes rolling baseline statistics. When `current_value` deviates from the rolling mean by more than `N` standard deviations, or when a threshold-based rule fires, a candidate event is generated.

Stage 3: Diagnosis — The candidate event is enriched with additional database state: active queries, lock chains, replication lag, recent schema changes, deployment events from the change management system. This enriched context is either passed to a rule engine or submitted to an LLM API for reasoning.

Stage 4: Dispatch — The dispatcher (illustrated earlier) determines the reversibility tier, checks confidence against threshold, and either executes the action, queues it for human approval, or discards it with an audit record.

Stage 5: Outcome Collection — Post-action, the system re-evaluates the triggering signal. Did the action resolve the condition? This outcome is logged and fed into a weekly review process where the decision logic is updated.

```
graph LR
  A[1. Signal Collection] --> B[2. Anomaly Detection]
  B --> C[3. Diagnosis: rules + LLM]
  C --> D[4. Dispatch: tier + confidence gate]
  D --> E[5. Outcome Collection]
  E -->|weekly review updates decision logic| B
```

The canonical ADE loop: each stage feeds the next, and outcome collection closes the loop back into anomaly detection by way of the weekly review practice described later in this chapter.

The following example implements Stage 3 for connection exhaustion detection — one of the highest-value autonomous interventions in production:

```

# Stage 3: Autonomous diagnosis for connection exhaustion
import pyodbc
import anthropic
import json
from typing import Tuple

def diagnose_connection_exhaustion(conn_string: str,
                                   current_connections: int,
                                   user_connections: int) -> Tuple[str,
float]:
    conn = pyodbc.connect(conn_string)
    cur = conn.cursor()

    cur.execute("""
        SELECT
            program_name,
            status,
            COUNT(*) AS session_count,
            SUM(CASE WHEN open_transaction_count > 0 THEN 1 ELSE 0 END)
                AS open_txn_count,
            MAX(DATEDIFF(SECOND, last_request_start_time, GETDATE()))
                AS max_idle_sec
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1
        GROUP BY program_name, status
        ORDER BY session_count DESC;
    """)
    session_breakdown = cur.fetchall()

    cur.execute("""
        SELECT COUNT(*)
        FROM sys.dm_exec_sessions s
        WHERE s.open_transaction_count > 0
            AND s.status = 'sleeping'
            AND DATEDIFF(SECOND, s.last_request_start_time, GETDATE()) > 60;
    """)
    sleeping_txn_count = cur.fetchone()[0]

    cur.close()
    conn.close()

    utilization = current_connections / user_connections

    if sleeping_txn_count > user_connections * 0.25:
        return ("KILL_SLEEPING_TRANSACTIONS", 0.90)

    if utilization > 0.95:
        return ("SCALE_CONNECTION_POOL", 0.87)

    return ("ESCALATE_TO_HUMAN", 0.60)

```

This pattern — rule-based fast path for high-confidence known situations, LLM-assisted reasoning for novel patterns — is the production-grade architecture for autonomous database diagnosis. The rule-based layer is fast and auditable. The LLM layer handles the long tail of situations that rules cannot anticipate. Both layers output to the same dispatcher interface.

The ADE Career Architecture: Skills Map, Team Position, and Organizational Leverage

The ADE is a new kind of role, which means it does not yet have a canonical job ladder. Understanding how to position the role, how to build the competency profile, and how to generate organizational leverage is as important as the technical skills themselves.

The Skills Triad

The ADE sits at the intersection of three competency domains that are rarely combined in a single engineer:

Database Engineering Depth — Not broad familiarity. Deep knowledge of internals for at least two database engines, including storage-level behavior under failure conditions, replication mechanics, and the performance characteristics of operations at scale.

Platform Engineering Breadth — Kubernetes, infrastructure as code, CI/CD pipeline design, observability stack operation, and cloud provider primitives across at least one major provider (Azure SQL, AWS RDS for SQL Server, GCP Cloud SQL for SQL Server).

Automation and AI Systems Design — The ability to design, implement, and operate feedback loops: signal collection, anomaly detection, decision logic, action dispatch, and outcome measurement. This includes prompt engineering for LLM-assisted diagnosis, but also the classical control theory concepts (feedback loops, stability, overshoot, dampening) that apply directly to autonomous database control systems.

Most senior DBAs have the first domain deeply. Many platform engineers have the second. Almost no one entering 2024 has all three at production depth. That gap is the career opportunity.

Building Organizational Leverage

An ADE generates leverage in three ways that a traditional DBA does not:

Force multiplication: An autonomous system designed by one ADE can operate ten database clusters with the same reliability that previously required five DBAs. This is not an argument for headcount reduction — it is an argument for redeploying skilled engineering time toward higher-value architectural work.

Knowledge codification: The decision logic in an autonomous system is institutional knowledge made executable. When a senior DBA leaves, the heuristics in their head leave with them. When an ADE builds a decision engine, those heuristics are version-controlled, testable, and improvable.

Incident elimination: The value proposition of an autonomous system is measured not in incidents resolved but in incidents that never happen. This is difficult to quantify but critical to communicate. Maintain a log of autonomous interventions with the estimated human-hours that would have been required, the estimated impact to SLA if the action had been delayed, and the actual outcome. This is your quarterly business review artifact.

Team Positioning

The ADE role most naturally sits in one of three organizational structures:

Platform Engineering Team — The ADE is embedded in a platform team that owns developer experience and internal infrastructure. The database autonomous systems are one component of a broader internal developer platform.

Database Reliability Engineering — A dedicated DRE team, analogous to SRE but specialized for data infrastructure. This structure works well in organizations where database complexity and scale justify a dedicated function, and where the ADE can influence not just operations but schema design, query patterns, and application-level data access practices.

Embedded in Product Engineering — Less common, but effective in organizations where data infrastructure is a core product differentiator. The ADE works directly with the product engineering teams, designing autonomous systems that are specific to the data access patterns and reliability requirements of individual services.

Regardless of structure, the ADE must have a documented escalation path to engineering leadership for actions above the Tier 2 threshold. An autonomous system without organizational backing for its decisions is not autonomous — it is a liability.

Day-to-Day Practices of the Autonomous Database Engineer

The ADE's calendar looks different from a traditional DBA's calendar. The reactive work — the 2 AM pages, the emergency index additions, the manual failover drills — is mostly handled by the autonomous systems. The ADE's time is distributed across four categories of deliberate work.

Practice 1: The Weekly Autonomous System Review

Every autonomous action from the prior week is reviewed in a structured session. The review answers four questions:

1. Were there actions the system took that a human would not have taken? If so, was the outcome still acceptable, or should the decision logic be constrained?
2. Were there incidents that required human intervention that the system should have caught? What signal was missing or what decision rule was absent?
3. What is the false-positive rate of each Prometheus alert rule that feeds the autonomous loop? Is it trending up or down?
4. Are the confidence thresholds calibrated? High confidence that turned out wrong is a calibration failure. Low confidence that turned out trivially correct means the threshold is too conservative.

This review should produce at minimum one concrete improvement to the autonomous loop per week: a tightened alert threshold, a new rule-based fast path, a revised LLM prompt, or a new reversibility tier classification.

Practice 2: Chaos Engineering on the Autonomous Loop

The autonomous system must be tested under failure conditions. This is not optional. A system that has never been tested for the scenario it is supposed to handle autonomously is not an autonomous system — it is a hope.

The ADE should maintain a chaos test suite that injects the following conditions on a scheduled basis in a staging environment:

```

# Chaos test suite for autonomous database infrastructure
import random
import time
import pyodbc
import subprocess
from typing import List, Callable

class ChaosTest:
    def __init__(self, name: str, inject_fn: Callable,
                  expected_autonomous_action: str,
                  timeout_seconds: int):
        self.name = name
        self.inject_fn = inject_fn
        self.expected_autonomous_action = expected_autonomous_action
        self.timeout_seconds = timeout_seconds

def inject_connection_flood(conn_string: str, count: int = 80):
    """Hold idle connections to trigger pool exhaustion detection."""
    connections = []
    try:
        for _ in range(count):
            c = pyodbc.connect(conn_string)
            connections.append(c)
            time.sleep(0.05)
        time.sleep(30) # Hold for 30 seconds
    finally:
        for c in connections:
            try:
                c.close()
            except Exception:
                pass

def inject_long_running_transaction(conn_string: str):
    """Open a transaction and hold it for 90 seconds without committing."""
    conn = pyodbc.connect(conn_string, autocommit=False)
    cur = conn.cursor()
    cur.execute("WAITFOR DELAY '00:01:30';")
    # Deliberately not committing – chaos test

def inject_index_fragmentation(conn_string: str, table: str):
    """Run repeated updates to generate index fragmentation."""
    conn = pyodbc.connect(conn_string, autocommit=True)
    cur = conn.cursor()
    for _ in range(10000):
        cur.execute(f"UPDATE {table} SET updated_at = SYSUTCDATETIME() "
                    f"WHERE id = ?", (random.randint(1, 1000),))

def inject_replica_lag(replica_server: str):
    """Suspend data movement to a secondary to simulate redo lag."""
    subprocess.run(
        ["sqlcmd", "-S", replica_server, "-Q",
         "ALTER DATABASE [ShopDB] SET HADR SUSPEND;"],
        check=True

```


)

```
class ChaosRunner:
    def __init__(self, audit_log: str):
        self.audit_log = audit_log

    def run_test(self, test: ChaosTest, conn_string: str) -> dict:
        import threading, json, time

        result = {
            "test_name": test.name,
            "expected_action": test.expected_autonomous_action,
            "observed_action": None,
            "passed": False,
            "elapsed_seconds": None,
        }

        start = time.time()

        # Inject failure in a background thread
        thread = threading.Thread(
            target=test.inject_fn, args=(conn_string,), daemon=True
        )
        thread.start()

        # Poll the audit log for the expected autonomous action
        deadline = start + test.timeout_seconds
        while time.time() < deadline:
            time.sleep(2)
            with open(self.audit_log) as f:
                lines = f.readlines()
            for line in reversed(lines):
                event = json.loads(line)
                if (event.get("action") == test.expected_autonomous_action
                    and event.get("timestamp", 0) > start):
                    result["observed_action"] = event["action"]
                    result["passed"] = True
                    result["elapsed_seconds"] = round(
                        time.time() - start, 1
                    )
                    break
            if result["passed"]:
                break

        if not result["passed"]:
            result["elapsed_seconds"] = test.timeout_seconds
            result["observed_action"] = "NOT_DETECTED"

        return result
```

The chaos test suite should be run weekly in staging and monthly in a controlled production maintenance window. Any test that does not pass within the expected timeout is a gap in the autonomous loop that requires a sprint item.

Practice 3: Decision Logic Version Control

Every change to the autonomous system's decision logic — alert thresholds, confidence cutoffs, reversibility tier assignments, LLM prompt text — must be committed to version control with a commit message that explains the rationale. This is not bureaucracy. It is the only way to conduct a root cause analysis when an autonomous action causes harm.

The commit history of your decision logic IS the audit trail for your autonomous system's behavior. Treat it accordingly. Use a dedicated repository or subdirectory with protected branch rules, required peer review, and a changelog format that captures the before and after state of every threshold change.

Practice 4: Stakeholder Communication Without Dashboards

The ADE must be able to communicate the value and the risk of the autonomous systems to non-technical stakeholders without defaulting to a dashboard tour. The most effective format is a monthly one-page summary with four sections:

- → **Autonomous actions taken:** count, breakdown by tier, outcome rate
- → **Incidents prevented** (estimated): situations where autonomous action forestalled an outage, with estimated impact window
- → **Incidents that required human escalation:** what happened, why the system did not handle it, what improvement is planned
- → **Changes made to the autonomous loop this month:** what was tuned, added, or removed, and why

This document builds organizational trust in the system over time. It also creates accountability: when the loop fails to handle something it should have handled, the stakeholders who read that document understand that the ADE takes that gap seriously and has a plan.

The Ethics and Limits of Autonomous Database Systems

No treatment of the ADE role is complete without an honest discussion of where autonomous systems should not be allowed to act without human oversight, and why.

The consent problem: Autonomous systems make decisions that affect users. An autonomous failover changes the connection endpoint for every application connected to the database. An autonomous query termination ends a transaction that a user initiated. The

humans affected by these decisions did not authorize the autonomous system to make them — they authorized a database to store their data. The ADE has an obligation to ensure that the scope of autonomous authority is commensurate with what the business has actually authorized, not merely what the technical system is capable of.

The opacity problem: LLM-assisted diagnosis introduces a reasoning layer that is not fully auditable. The LLM produces a recommendation with a stated confidence, but the internal computation that produced that recommendation is not inspectable in the way a deterministic rule is. This means LLM-assisted actions require stronger post-hoc review, not weaker. The ADE should track the correlation between LLM confidence scores and actual outcome quality, and should reduce the authority of the LLM reasoning layer if that correlation degrades.

The automation bias problem: Organizations that deploy autonomous database systems sometimes reduce their investment in human database expertise because the system is "handling it." This is a dangerous equilibrium. Autonomous systems fail in the scenarios they were not designed for, and those scenarios are precisely the ones that require the deepest human expertise. The ADE should actively maintain and develop human expertise in parallel with the autonomous system, not as a fallback, but as the system that designs, evaluates, and improves the automation.

The regulatory boundary: In industries subject to data regulation — financial services, healthcare, anything touching personal data under GDPR or CCPA — autonomous actions that touch personal data records require explicit legal grounding. Autonomous query termination that kills a data subject access request processing job is not merely a technical incident. The ADE must work with legal and compliance teams to define the hard boundaries of autonomous authority, and those boundaries must be enforced in the dispatcher code, not documented somewhere and hoped for.

What the Next Decade Looks Like for Autonomous Database Engineering

The trajectory is clear even if the exact endpoints are not. Vector databases have already moved from experimental to production-critical for AI workloads. Retrieval-augmented generation architectures require database engineers who understand both the transactional consistency properties of the underlying store and the semantic quality properties of the vector index. This is not a specialization adjacent to what this book has covered — it is an extension of the same discipline.

The convergence of cloud-native database services and autonomous operational intelligence means that the traditional boundary between "the database" and "the database service" will continue to dissolve. An ADE working on Azure SQL Database or Azure SQL Managed Instance is already operating a system where the infrastructure layer makes autonomous

decisions — storage scaling, connection routing, backup scheduling — and the ADE's autonomous layer operates above that, making application-level and workload-level decisions.

The engineers who thrive in this environment share a specific disposition: they are comfortable with systems that have agency. They can reason about what a system will do in a scenario they have not explicitly programmed. They design for failure modes they have not yet encountered. They treat observability as a first-class engineering concern, not an afterthought. And they hold both the humility to know what their autonomous systems do not know, and the confidence to build systems that act on incomplete information without requiring human approval for every decision.

That disposition, combined with the technical stack this book has described, is the Autonomous Database Engineer.

Key Takeaways

- → **The ADE mental model is defined by three axes:** confidence bounds that authorize action scopes, reversibility tiers that determine approval requirements, and observability contracts that make every autonomous decision auditable and improvable. These are not conceptual guidelines — they are enforced in the dispatcher code.
- → **End-to-end technical ownership across six domains** — database internals, infrastructure as code, observability engineering, decision engine and AI integration, multi-region DR architecture, and security/compliance automation — is what distinguishes an ADE from a DBA who has added automation tooling. Depth in all six is the career investment that cannot be shortcut.
- → **The production-grade autonomous loop architecture** combines rule-based fast paths for high-confidence known failure modes with LLM-assisted reasoning for novel patterns. Both layers output to the same dispatcher, both are audited identically, and the LLM layer's confidence calibration is measured and maintained as a first-class operational concern.
- → **Day-to-day ADE practice** is structured around four activities: weekly autonomous system review with concrete loop improvements, chaos engineering to validate that the system handles the scenarios it was designed for, decision logic version control as the audit trail for autonomous behavior, and stakeholder communication that builds organizational trust without requiring dashboard literacy.
- → **The ethical boundaries of autonomous authority** — consent scope, LLM opacity, automation bias, and regulatory constraints — are engineering concerns, not philosophical ones. They translate directly into dispatcher configuration, human approval gates, post-action review cadences, and documented hard limits that are enforced in code and reviewed with legal and compliance teams.

DB Hub — Self-Hosted Database Platform

9 modules · Schema browser · AI assistant · Query optimizer · SQL Server

[Learn More →](#)